TahakomAVRLibDoc

1.0

Generated by Doxygen 1.15.0

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Namespace Index 5
2.1 Namespace List . . . 5

3 Hierarchical Index 7
3.1 Class Hierarchy . . . 7

4 Class Index 9
4.1 Class List . . . 9

5 File Index 11
5.1 File List . . . 11

6 Directory Documentation 15
6.1 applications Directory Reference . . . 15
6.2 AutonomousRobotController Directory Reference . . . 15
6.2.1 Detailed Description . . . 15
6.2.2 Autonomous Robot Controller Project . . . 15
6.2.3 Hardware Pinout . . . 16
6.3 BlinkLed Directory Reference . . . 16
6.3.1 Detailed Description . . . 16
6.3.2 Hardware . . . 16
6.3.3 Circuit . . . 16
6.3.4 Code . . . 16
6.3.5 Author . . . 17
6.3.6 License . . . 17
6.4 component Directory Reference . . . 17
6.5 core Directory Reference . . . 18
6.6 ESP32_SmartEnvironmentalNode Directory Reference . . . 18
6.6.1 Detailed Description . . . 18
6.6.2 ESP32 Smart Environmental Sensing Node . . . 18
6.6.3 Overview . . . 18
6.6.4 Hardware Components . . . 19
6.6.5 Pinout Connections . . . 19
6.7 ESP32_WiFi_Telemetry Directory Reference . . . 19
6.7.1 Detailed Description . . . 19
6.7.2 ESP32 Wi-Fi & Serial Telemetry Application . . . 19
6.7.3 Description . . . 19
6.7.4 Hardware Connections . . . 20
6.7.5 Telemetry Framing . . . 20
6.8 I2C_OLED_Dashboard Directory Reference . . . 20
6.8.1 Detailed Description . . . 20

6.8.2 I2C OLED Dashboard Application . . . 20
6.8.3 Hardware Connections . . . 20
6.9 io Directory Reference . . . 21
6.10 LCD_EnvironmentMonitor Directory Reference . . . 21
6.10.1 Detailed Description . . . 21
6.10.2 LCD Environment Light Monitor Application . . . 21
6.10.3 Hardware Connections . . . 21
6.11 lib Directory Reference . . . 21
6.12 LightMeter Directory Reference . . . 22
6.12.1 Detailed Description . . . 22
6.12.2 Hardware . . . 22
6.12.3 Circuit . . . 22
6.12.4 Code . . . 23
6.12.5 Author . . . 24
6.12.6 License . . . 24
6.13 MusicBox Directory Reference . . . 24
6.13.1 Detailed Description . . . 24
6.13.2 Hardware . . . 24
6.13.3 Circuit . . . 24
6.13.4 Code . . . 25
6.13.5 Author . . . 26
6.13.6 License . . . 26
6.14 Organ Directory Reference . . . 26
6.14.1 Detailed Description . . . 26
6.14.2 Hardware . . . 26
6.14.3 Circuit . . . 26
6.14.4 Code . . . 27
6.14.5 Author . . . 28
6.14.6 License . . . 28
6.15 PID_MotorControl Directory Reference . . . 28
6.15.1 Detailed Description . . . 28
6.15.2 PID Closed-Loop DC Motor Control Application . . . 28
6.15.3 Hardware Connections . . . 29
6.16 projects Directory Reference . . . 29
6.17 SmartHomeGateway Directory Reference . . . 29
6.17.1 Detailed Description . . . 29
6.17.2 Smart Home Gateway Project . . . 29
6.17.3 Hardware Pinout . . . 29
6.18 TelemetryStreamer Directory Reference . . . 30
6.18.1 Detailed Description . . . 30
6.18.2 Hardware Telemetry Streamer Application . . . 30
6.18.3 Hardware Connections . . . 30

6.19 tests Directory Reference . . . 30
6.20 utils Directory Reference . . . 30
6.21 WeakIsr Directory Reference . . . 31

7 Namespace Documentation **33**
7.1 component Namespace Reference . . . 33
7.1.1 Detailed Description . . . 33
7.1.2 Enumeration Type Documentation . . . 34
7.1.2.1 mode . . . 34
7.2 core Namespace Reference . . . 34
7.2.1 Detailed Description . . . 35
7.2.2 Enumeration Type Documentation . . . 35
7.2.2.1 autoTriggerSource . . . 35
7.2.2.2 BODMode . . . 36
7.2.2.3 channel . . . 36
7.2.2.4 clockPrescaler . . . 36
7.2.2.5 clockSource . . . 37
7.2.2.6 compareOutputMode . . . 38
7.2.2.7 operationMode . . . 38
7.2.2.8 pinChangePort . . . 39
7.2.2.9 referenceVoltage . . . 39
7.2.2.10 resolution . . . 39
7.2.2.11 senseControl . . . 40
7.2.2.12 sleepMode . . . 40
7.2.2.13 timeOut . . . 41
7.2.2.14 watchdogOperationMode . . . 41
7.2.3 Function Documentation . . . 42
7.2.3.1 operationModeToWDTCSR() . . . 42
7.2.3.2 timeOutToWDTCSR() . . . 42
7.3 io Namespace Reference . . . 43
7.3.1 Detailed Description . . . 44
7.3.2 Enumeration Type Documentation . . . 44
7.3.2.1 clockPrescaler . . . 44
7.3.2.2 communicationMode . . . 44
7.3.2.3 dataMode . . . 45
7.3.2.4 dataOrder . . . 45
7.3.2.5 frameSize . . . 46
7.3.2.6 operationMode . . . 46
7.3.2.7 parityMode . . . 46
7.3.2.8 stopBit . . . 47
7.3.2.9 TelemetryType . . . 47
7.3.2.10 transmissionMode . . . 47

7.3.3 Variable Documentation . . . 48
7.3.3.1 PortB . . . 48
7.3.3.2 PortC . . . 48
7.3.3.3 PortD . . . 48
7.4 utils Namespace Reference . . . 48
7.4.1 Function Documentation . . . 49
7.4.1.1 map() . . . 49

8 Class Documentation **51**
8.1 core::ADConverter Class Reference . . . 51
8.1.1 Detailed Description . . . 52
8.1.2 Constructor & Destructor Documentation . . . 52
8.1.2.1 ADConverter() [1/2] . . . 52
8.1.2.2 ~ADConverter() . . . 53
8.1.2.3 ADConverter() [2/2] . . . 53
8.1.3 Member Function Documentation . . . 53
8.1.3.1 conversionCompleteServiceRoutine() . . . 53
8.1.3.2 enableAutoTrigger() . . . 55
8.1.3.3 enableConversionCompleteInterrupt() . . . 56
8.1.3.4 getConversionResult() . . . 56
8.1.3.5 getInstance() . . . 57
8.1.3.6 operator=() . . . 58
8.1.3.7 selectAnalogInput() . . . 58
8.1.3.8 selectAutoTriggerSource() . . . 59
8.1.3.9 selectClockPrescaler() . . . 59
8.1.3.10 selectReferenceVoltage() . . . 59
8.1.3.11 start() . . . 60
8.1.3.12 stop() . . . 60
8.1.4 Member Data Documentation . . . 60
8.1.4.1 __externally_visible__ . . . 60
8.1.4.2 __used__ . . . 61
8.1.4.3 m_conversionResult . . . 61
8.1.4.4 m_resolution . . . 61
8.2 component::Buzzer Class Reference . . . 61
8.2.1 Detailed Description . . . 62
8.2.2 Constructor & Destructor Documentation . . . 62
8.2.2.1 Buzzer() . . . 62
8.2.2.2 ~Buzzer() . . . 63
8.2.3 Member Function Documentation . . . 63
8.2.3.1 buzz() [1/2] . . . 63
8.2.3.2 buzz() [2/2] . . . 64
8.2.3.3 pause() . . . 64

8.2.4 Member Data Documentation . . . 65
8.2.4.1 m_pin . . . 65
8.3 component::DCMotor Class Reference . . . 65
8.3.1 Detailed Description . . . 66
8.3.2 Constructor & Destructor Documentation . . . 66
8.3.2.1 DCMotor() . . . 66
8.3.2.2 ~DCMotor() . . . 66
8.3.3 Member Function Documentation . . . 66
8.3.3.1 connect() . . . 66
8.3.3.2 disconnect() . . . 67
8.3.3.3 off() . . . 67
8.3.3.4 on() . . . 67
8.3.3.5 spin() . . . 67
8.3.3.6 stop() . . . 68
8.3.3.7 toggle() . . . 68
8.3.4 Member Data Documentation . . . 68
8.3.4.1 m_pin . . . 68
8.4 core::ExternInterrupt Class Reference . . . 68
8.4.1 Detailed Description . . . 69
8.4.2 Constructor & Destructor Documentation . . . 69
8.4.2.1 ExternInterrupt() [1/2] . . . 69
8.4.2.2 ~ExternInterrupt() . . . 69
8.4.2.3 ExternInterrupt() [2/2] . . . 70
8.4.3 Member Function Documentation . . . 70
8.4.3.1 enableInt0() . . . 70
8.4.3.2 enableInt1() . . . 70
8.4.3.3 enablePinChange() . . . 71
8.4.3.4 enablePinChangeMaskPortB() . . . 71
8.4.3.5 enablePinChangeMaskPortC() . . . 71
8.4.3.6 enablePinChangeMaskPortD() . . . 72
8.4.3.7 getInstance() . . . 72
8.4.3.8 Int0ServiceRoutine() . . . 72
8.4.3.9 Int1ServiceRoutine() . . . 72
8.4.3.10 operator=() . . . 72
8.4.3.11 pinChangePortBServiceRoutine() . . . 73
8.4.3.12 pinChangePortCServiceRoutine() . . . 73
8.4.3.13 pinChangePortDServiceRoutine() . . . 73
8.4.3.14 setInt0SenseControl() . . . 73
8.4.3.15 setInt1SenseControl() . . . 73
8.4.4 Member Data Documentation . . . 73
8.4.4.1 __externally_visible__ . . . 73
8.4.4.2 __used__ . . . 74

8.4.4.3 weak . . . 74
8.5 component::LDR Class Reference . . . 74
8.5.1 Detailed Description . . . 74
8.5.2 Constructor & Destructor Documentation . . . 75
8.5.2.1 LDR() . . . 75
8.5.2.2 ~LDR() . . . 75
8.5.3 Member Function Documentation . . . 75
8.5.3.1 getLux() . . . 75
8.5.3.2 getResistance() . . . 76
8.5.4 Member Data Documentation . . . 77
8.5.4.1 ADC_ZERO_SENTINEL . . . 77
8.5.4.2 m_pin . . . 77
8.5.4.3 mr_ADC . . . 77
8.6 component::Led Class Reference . . . 77
8.6.1 Detailed Description . . . 78
8.6.2 Constructor & Destructor Documentation . . . 78
8.6.2.1 Led() . . . 78
8.6.2.2 ~Led() . . . 79
8.6.3 Member Function Documentation . . . 79
8.6.3.1 isOff() . . . 79
8.6.3.2 isOn() . . . 79
8.6.3.3 off() . . . 80
8.6.3.4 on() . . . 80
8.6.3.5 toggle() . . . 80
8.6.4 Member Data Documentation . . . 80
8.6.4.1 m_pin . . . 80
8.7 core::MCU Class Reference . . . 81
8.7.1 Detailed Description . . . 81
8.7.2 Member Function Documentation . . . 81
8.7.2.1 enableADC() . . . 81
8.7.2.2 enableInterrupts() . . . 81
8.7.2.3 enableSPI() . . . 82
8.7.2.4 enableTimerCounter0() . . . 82
8.7.2.5 enableTimerCounter1() . . . 82
8.7.2.6 enableTimerCounter2() . . . 83
8.7.2.7 enableTWI() . . . 83
8.7.2.8 enableUSART0() . . . 83
8.7.2.9 goToSleep() . . . 84
8.7.2.10 init() . . . 84
8.7.2.11 selectSleepMode() . . . 85
8.7.2.12 sleepEnable() . . . 85
8.8 component::PCF8574_LCD Class Reference . . . 85

8.8.1 Detailed Description . . . 86
8.8.2 Constructor & Destructor Documentation . . . 86
8.8.2.1 PCF8574_LCD() . . . 86
8.8.3 Member Function Documentation . . . 86
8.8.3.1 clear() . . . 86
8.8.3.2 init() . . . 87
8.8.3.3 print() . . . 87
8.8.3.4 sendByte() . . . 87
8.8.3.5 sendNibble() . . . 88
8.8.3.6 setBacklight() . . . 88
8.8.3.7 setCursor() . . . 88
8.8.4 Member Data Documentation . . . 89
8.8.4.1 m_backlightMask . . . 89
8.8.4.2 m_cols . . . 89
8.8.4.3 m_i2cAddr . . . 89
8.8.4.4 m_rows . . . 89
8.8.4.5 mr_twi . . . 89
8.9 utils::PIDController Class Reference . . . 89
8.9.1 Detailed Description . . . 90
8.9.2 Constructor & Destructor Documentation . . . 90
8.9.2.1 PIDController() . . . 90
8.9.3 Member Function Documentation . . . 91
8.9.3.1 compute() . . . 91
8.9.3.2 reset() . . . 92
8.9.3.3 setGains() . . . 92
8.9.3.4 setOutputLimits() . . . 92
8.9.4 Member Data Documentation . . . 93
8.9.4.1 m_firstRun . . . 93
8.9.4.2 m_integral . . . 93
8.9.4.3 m_kd . . . 93
8.9.4.4 m_ki . . . 93
8.9.4.5 m_kp . . . 93
8.9.4.6 m_maxOutput . . . 93
8.9.4.7 m_minOutput . . . 94
8.9.4.8 m_prevInput . . . 94
8.10 io::Pin Class Reference . . . 94
8.10.1 Detailed Description . . . 94
8.10.2 Constructor & Destructor Documentation . . . 95
8.10.2.1 Pin() . . . 95
8.10.2.2 ~Pin() . . . 95
8.10.3 Member Function Documentation . . . 95
8.10.3.1 getPinNumber() . . . 95

8.10.3.2 isHigh() . . . 96
8.10.3.3 isLow() . . . 96
8.10.3.4 setHigh() . . . 96
8.10.3.5 setLow() . . . 97
8.10.3.6 toggle() . . . 97
8.10.3.7 toInput() . . . 97
8.10.3.8 toOutput() . . . 98
8.10.4 Member Data Documentation . . . 98
8.10.4.1 m_pinNumber . . . 98
8.10.4.2 mr_portName . . . 98
8.11 io::Port Struct Reference . . . 98
8.11.1 Detailed Description . . . 99
8.11.2 Member Data Documentation . . . 99
8.11.2.1 mp_ddrReg . . . 99
8.11.2.2 mp_pinReg . . . 99
8.11.2.3 mp_portReg . . . 99
8.12 component::PushButton Class Reference . . . 99
8.12.1 Detailed Description . . . 100
8.12.2 Constructor & Destructor Documentation . . . 100
8.12.2.1 PushButton() . . . 100
8.12.2.2 ~PushButton() . . . 101
8.12.3 Member Function Documentation . . . 101
8.12.3.1 getPressedCount() . . . 101
8.12.3.2 isPressed() . . . 102
8.12.3.3 resetPressedCount() . . . 102
8.12.4 Member Data Documentation . . . 103
8.12.4.1 m_buttonPressed . . . 103
8.12.4.2 m_isActiveLow . . . 103
8.12.4.3 m_lastState . . . 103
8.12.4.4 m_pin . . . 103
8.12.4.5 m_useInternalPullUp . . . 103
8.13 component::RotaryEncoder Class Reference . . . 104
8.13.1 Detailed Description . . . 104
8.13.2 Constructor & Destructor Documentation . . . 104
8.13.2.1 RotaryEncoder() . . . 104
8.13.3 Member Function Documentation . . . 105
8.13.3.1 getDirection() . . . 105
8.13.3.2 getPosition() . . . 105
8.13.3.3 resetPosition() . . . 105
8.13.3.4 update() . . . 105
8.13.4 Member Data Documentation . . . 106
8.13.4.1 m_direction . . . 106

8.13.4.2 m_lastStateA . . . 106
8.13.4.3 m_lastStateB . . . 106
8.13.4.4 m_pinA . . . 106
8.13.4.5 m_pinB . . . 106
8.13.4.6 m_position . . . 106
8.14 component::ServoMotor Class Reference . . . 107
8.14.1 Detailed Description . . . 107
8.14.2 Constructor & Destructor Documentation . . . 107
8.14.2.1 ServoMotor() . . . 107
8.14.2.2 ∼ServoMotor() . . . 108
8.14.3 Member Function Documentation . . . 108
8.14.3.1 clampTimerCount() . . . 108
8.14.3.2 computePulseCycleCount() . . . 108
8.14.3.3 computePulseWidthMaxCount() . . . 109
8.14.3.4 computePulseWidthMidCount() . . . 109
8.14.3.5 computePulseWidthMinCount() . . . 109
8.14.3.6 computeRotationAngleCount() . . . 109
8.14.3.7 connect() . . . 110
8.14.3.8 disconnect() . . . 110
8.14.3.9 off() . . . 110
8.14.3.10 on() . . . 110
8.14.3.11 rotate() . . . 111
8.14.3.12 toggle() . . . 111
8.14.4 Member Data Documentation . . . 111
8.14.4.1 m_pin . . . 111
8.14.4.2 m_pulseCycle . . . 111
8.14.4.3 m_pulseWidthMax . . . 112
8.14.4.4 m_pulseWidthMid . . . 112
8.14.4.5 m_pulseWidthMin . . . 112
8.15 io::SPI Class Reference . . . 112
8.15.1 Detailed Description . . . 113
8.15.2 Constructor & Destructor Documentation . . . 114
8.15.2.1 SPI() [1/2] . . . 114
8.15.2.2 ∼SPI() . . . 114
8.15.2.3 SPI() [2/2] . . . 114
8.15.3 Member Function Documentation . . . 115
8.15.3.1 enableTransferCompleteInterrupt() . . . 115
8.15.3.2 getInstance() . . . 115
8.15.3.3 masterReceiveByte() . . . 115
8.15.3.4 masterSendByte() . . . 116
8.15.3.5 operator=() . . . 116
8.15.3.6 selectClockPrescaler() . . . 116

8.15.3.7 selectDataMode() . . . 117
8.15.3.8 selectDataOrder() . . . 117
8.15.3.9 selectOperationMode() . . . 117
8.15.3.10 selectSlave() . . . 118
8.15.3.11 slaveReceiveByte() . . . 118
8.15.3.12 transferComplete() . . . 118
8.15.3.13 transferCompleteServiceRoutine() . . . 118
8.15.3.14 writeCollision() . . . 119
8.15.4 Member Data Documentation . . . 119
8.15.4.1 __externally_visible__ . . . 119
8.15.4.2 __used__ . . . 119
8.15.4.3 m_data . . . 119
8.15.4.4 m_pinMISO . . . 119
8.15.4.5 m_pinMOSI . . . 120
8.15.4.6 m_pinSCK . . . 120
8.15.4.7 m_pinSS . . . 120
8.15.4.8 m_transferComplete . . . 120
8.16 component::SSD1306_OLED Class Reference . . . 120
8.16.1 Detailed Description . . . 121
8.16.2 Constructor & Destructor Documentation . . . 121
8.16.2.1 SSD1306_OLED() . . . 121
8.16.3 Member Function Documentation . . . 122
8.16.3.1 clear() . . . 122
8.16.3.2 init() . . . 122
8.16.3.3 print() . . . 123
8.16.3.4 sendCommand() . . . 123
8.16.3.5 sendData() . . . 123
8.16.3.6 setCursor() . . . 124
8.16.4 Member Data Documentation . . . 124
8.16.4.1 m_height . . . 124
8.16.4.2 m_i2cAddr . . . 124
8.16.4.3 m_width . . . 124
8.16.4.4 mr_twi . . . 125
8.17 component::StepperMotor Class Reference . . . 125
8.17.1 Detailed Description . . . 126
8.17.2 Constructor & Destructor Documentation . . . 126
8.17.2.1 StepperMotor() . . . 126
8.17.2.2 ~StepperMotor() . . . 126
8.17.3 Member Function Documentation . . . 126
8.17.3.1 beginMove() . . . 126
8.17.3.2 computeStepDelay() . . . 127
8.17.3.3 currentPos() . . . 128

8.17.3.4 goalReached() . . . 129
8.17.3.5 setCurrentPos() . . . 129
8.17.3.6 step() [1/3] . . . 129
8.17.3.7 step() [2/3] . . . 130
8.17.3.8 step() [3/3] . . . 131
8.17.3.9 stepDelay() . . . 132
8.17.3.10 stepPulse() . . . 132
8.17.4 Member Data Documentation . . . 133
8.17.4.1 m_accelTime . . . 133
8.17.4.2 m_constSpeedTime . . . 133
8.17.4.3 m_currentPos . . . 134
8.17.4.4 m_decelTime . . . 134
8.17.4.5 m_goalReached . . . 134
8.17.4.6 m_pinCoil1 . . . 134
8.17.4.7 m_pinCoil2 . . . 134
8.17.4.8 m_pinCoil3 . . . 135
8.17.4.9 m_pinCoil4 . . . 135
8.17.4.10 m_profilePosition . . . 135
8.17.4.11 m_profileSpeed . . . 135
8.17.4.12 m_profileTime . . . 135
8.17.4.13 m_speedTimeProduct . . . 136
8.17.4.14 m_stepNumber . . . 136
8.17.4.15 m_targetStep . . . 136
8.17.4.16 stepMode . . . 136
8.18 io::Telemetry Class Reference . . . 136
8.18.1 Detailed Description . . . 137
8.18.2 Member Function Documentation . . . 137
8.18.2.1 init() . . . 137
8.18.2.2 sendADC() . . . 137
8.18.2.3 sendLog() . . . 138
8.18.2.4 sendMotor() . . . 138
8.18.2.5 sendPin() . . . 139
8.18.3 Member Data Documentation . . . 139
8.18.3.1 mp_usart . . . 139
8.19 core::TimerCounter Class Reference . . . 139
8.19.1 Detailed Description . . . 140
8.19.2 Constructor & Destructor Documentation . . . 140
8.19.2.1 ~TimerCounter() . . . 140
8.19.3 Member Function Documentation . . . 140
8.19.3.1 enableOutputCompareMatchInterrupt() . . . 140
8.19.3.2 enableOverflowInterrupt() . . . 141
8.19.3.3 getClockPrescaler() . . . 141

8.19.3.4 getCounter() . . . 141
8.19.3.5 getOutputCompareRegister() . . . 141
8.19.3.6 operator delete() . . . 141
8.19.3.7 selectClockSource() . . . 141
8.19.3.8 selectCompareOutputMode() . . . 142
8.19.3.9 selectOperationMode() . . . 142
8.19.3.10 setCounter() . . . 142
8.19.3.11 setOutputCompareRegister() . . . 142
8.19.3.12 start() . . . 142
8.19.3.13 stop() . . . 142
8.20 core::TimerCounter0 Class Reference . . . 143
8.20.1 Detailed Description . . . 144
8.20.2 Constructor & Destructor Documentation . . . 144
8.20.2.1 TimerCounter0() [1/2] . . . 144
8.20.2.2 ~TimerCounter0() . . . 144
8.20.2.3 TimerCounter0() [2/2] . . . 145
8.20.3 Member Function Documentation . . . 145
8.20.3.1 enableOutputCompareMatchInterrupt() . . . 145
8.20.3.2 enableOverflowInterrupt() . . . 146
8.20.3.3 getClockPrescaler() . . . 146
8.20.3.4 getCounter() . . . 146
8.20.3.5 getInstance() . . . 146
8.20.3.6 getOutputCompareRegister() . . . 147
8.20.3.7 operator=() . . . 147
8.20.3.8 outputCompareMatchAServiceRoutine() . . . 147
8.20.3.9 outputCompareMatchBServiceRoutine() . . . 147
8.20.3.10 overflowServiceRoutine() . . . 147
8.20.3.11 selectClockSource() . . . 148
8.20.3.12 selectCompareOutputMode() . . . 149
8.20.3.13 selectOperationMode() . . . 149
8.20.3.14 setCounter() . . . 150
8.20.3.15 setOutputCompareRegister() . . . 150
8.20.3.16 start() . . . 151
8.20.3.17 stop() . . . 151
8.20.4 Member Data Documentation . . . 151
8.20.4.1 __externally_visible__ . . . 151
8.20.4.2 __used__ . . . 151
8.20.4.3 m_clockPrescaler . . . 151
8.20.4.4 m_clockSource . . . 152
8.20.4.5 weak . . . 152
8.21 core::TimerCounter1 Class Reference . . . 152
8.21.1 Detailed Description . . . 153

8.21.2 Constructor & Destructor Documentation . . . 153
8.21.2.1 TimerCounter1() [1/2] . . . 153
8.21.2.2 ~TimerCounter1() . . . 154
8.21.2.3 TimerCounter1() [2/2] . . . 154
8.21.3 Member Function Documentation . . . 154
8.21.3.1 enableInputCaptureInterrupt() . . . 154
8.21.3.2 enableOutputCompareMatchInterrupt() . . . 154
8.21.3.3 enableOverflowInterrupt() . . . 155
8.21.3.4 getClockPrescaler() . . . 155
8.21.3.5 getCounter() . . . 156
8.21.3.6 getInputCaptureRegister() . . . 156
8.21.3.7 getInstance() . . . 156
8.21.3.8 getOutputCompareRegister() . . . 157
8.21.3.9 inputCaptureServiceRoutine() . . . 157
8.21.3.10 operator=() . . . 157
8.21.3.11 outputCompareMatchAServiceRoutine() . . . 157
8.21.3.12 outputCompareMatchBServiceRoutine() . . . 158
8.21.3.13 overflowServiceRoutine() . . . 158
8.21.3.14 selectClockSource() . . . 158
8.21.3.15 selectCompareOutputMode() . . . 159
8.21.3.16 selectOperationMode() . . . 159
8.21.3.17 setCounter() . . . 161
8.21.3.18 setInputCaptureRegister() . . . 161
8.21.3.19 setOutputCompareRegister() . . . 161
8.21.3.20 start() . . . 162
8.21.3.21 stop() . . . 162
8.21.4 Member Data Documentation . . . 162
8.21.4.1 __externally_visible__ . . . 162
8.21.4.2 __used__ . . . 163
8.21.4.3 m_clockPrescaler . . . 163
8.21.4.4 m_clockSource . . . 163
8.21.4.5 weak . . . 163
8.22 core::TimerCounter2 Class Reference . . . 163
8.22.1 Detailed Description . . . 164
8.22.2 Constructor & Destructor Documentation . . . 165
8.22.2.1 TimerCounter2() [1/2] . . . 165
8.22.2.2 ~TimerCounter2() . . . 165
8.22.2.3 TimerCounter2() [2/2] . . . 165
8.22.3 Member Function Documentation . . . 166
8.22.3.1 enableOutputCompareMatchInterrupt() . . . 166
8.22.3.2 enableOverflowInterrupt() . . . 166
8.22.3.3 getClockPrescaler() . . . 167

8.22.3.4 getCounter() . . . 167
8.22.3.5 getInstance() . . . 167
8.22.3.6 getOutputCompareRegister() . . . 168
8.22.3.7 operator=() . . . 168
8.22.3.8 outputCompareMatchAServiceRoutine() . . . 168
8.22.3.9 outputCompareMatchBServiceRoutine() . . . 168
8.22.3.10 overflowServiceRoutine() . . . 168
8.22.3.11 selectClockSource() . . . 169
8.22.3.12 selectCompareOutputMode() . . . 170
8.22.3.13 selectOperationMode() . . . 170
8.22.3.14 setCounter() . . . 171
8.22.3.15 setOutputCompareRegister() . . . 171
8.22.3.16 start() . . . 172
8.22.3.17 stop() . . . 172
8.22.4 Member Data Documentation . . . 172
8.22.4.1 __externally_visible__ . . . 172
8.22.4.2 __used__ . . . 172
8.22.4.3 m_clockPrescaler . . . 172
8.22.4.4 m_clockSource . . . 173
8.22.4.5 weak . . . 173
8.23 core::TWI Class Reference . . . 173
8.23.1 Detailed Description . . . 173
8.23.2 Constructor & Destructor Documentation . . . 174
8.23.2.1 TWI() [1/2] . . . 174
8.23.2.2 ~TWI() . . . 174
8.23.2.3 TWI() [2/2] . . . 174
8.23.3 Member Function Documentation . . . 174
8.23.3.1 getInstance() . . . 174
8.23.3.2 operator=() . . . 175
8.23.3.3 readAck() . . . 175
8.23.3.4 readNack() . . . 175
8.23.3.5 setClock() . . . 176
8.23.3.6 start() . . . 176
8.23.3.7 stop() . . . 176
8.23.3.8 write() . . . 177
8.24 io::USART0 Class Reference . . . 177
8.24.1 Detailed Description . . . 178
8.24.2 Constructor & Destructor Documentation . . . 179
8.24.2.1 USART0() [1/2] . . . 179
8.24.2.2 ~USART0() . . . 179
8.24.2.3 USART0() [2/2] . . . 179
8.24.3 Member Function Documentation . . . 180

8.24.3.1 dataOverrun() . . . 180
8.24.3.2 dataRegisterEmptyServiceRoutine() . . . 180
8.24.3.3 enableDataRegisterEmptyInterrupt() . . . 180
8.24.3.4 enableReceiveCompleteInterrupt() . . . 181
8.24.3.5 enableTransmitCompleteInterrupt() . . . 181
8.24.3.6 frameError() . . . 182
8.24.3.7 getInstance() . . . 182
8.24.3.8 getNumberBytesReceived() . . . 183
8.24.3.9 getNumberBytesSent() . . . 183
8.24.3.10 operator=() . . . 183
8.24.3.11 parityError() . . . 184
8.24.3.12 ready2Send() . . . 184
8.24.3.13 receiveChar() . . . 184
8.24.3.14 receiveCompleteServiceRoutine() . . . 185
8.24.3.15 receiveFrame() . . . 185
8.24.3.16 receiveString() . . . 186
8.24.3.17 resetNumberBytesReceived() . . . 186
8.24.3.18 sendByte() . . . 187
8.24.3.19 sendChar() . . . 187
8.24.3.20 sendFrame() . . . 188
8.24.3.21 sendLong() . . . 188
8.24.3.22 sendString() . . . 189
8.24.3.23 sendWord() . . . 189
8.24.3.24 setBaudRate() . . . 190
8.24.3.25 setCommunicationMode() . . . 190
8.24.3.26 setFrameSize() . . . 191
8.24.3.27 setParityMode() . . . 192
8.24.3.28 setStopBit() . . . 192
8.24.3.29 setTransmissionMode() . . . 193
8.24.3.30 transmitCompleteServiceRoutine() . . . 193
8.24.4 Member Data Documentation . . . 193
8.24.4.1 __externally_visible__ . . . 193
8.24.4.2 __used__ . . . 194
8.24.4.3 m_numberBytesReceived . . . 194
8.24.4.4 m_numberBytesSent . . . 194
8.24.4.5 m_ready2Send . . . 194
8.24.4.6 m_sizeData2Receive . . . 194
8.24.4.7 m_sizeData2Send . . . 195
8.24.4.8 m_status . . . 195
8.24.4.9 m_stringReceive . . . 195
8.24.4.10 mp_data2Receive . . . 195
8.24.4.11 mp_data2Send . . . 196

8.25 core::WatchdogTimer Class Reference . . . 196
8.25.1 Detailed Description . . . 197
8.25.2 Constructor & Destructor Documentation . . . 197
8.25.2.1 WatchdogTimer() [1/2] . . . 197
8.25.2.2 ~WatchdogTimer() . . . 197
8.25.2.3 WatchdogTimer() [2/2] . . . 197
8.25.3 Member Function Documentation . . . 197
8.25.3.1 getInstance() . . . 197
8.25.3.2 operator=() . . . 198
8.25.3.3 reset() . . . 198
8.25.3.4 selectTimeOut() . . . 198
8.25.3.5 start() [1/2] . . . 199
8.25.3.6 start() [2/2] . . . 199
8.25.3.7 stop() . . . 199
8.25.3.8 timeOutServiceRoutine() . . . 200
8.25.4 Member Data Documentation . . . 200
8.25.4.1 __externally_visible__ . . . 200
8.25.4.2 __used__ . . . 200
8.25.4.3 m_isRunning . . . 200
8.25.4.4 m_operationMode . . . 200
8.25.4.5 m_timeOut . . . 200
8.25.4.6 weak . . . 200

9 File Documentation **201**

9.1 applications/BlinkLed/main.cpp File Reference . . . 201
9.1.1 Detailed Description . . . 201
9.1.2 Macro Definition Documentation . . . 202
9.1.2.1 LED . . . 202
9.1.2.2 TIMEDELAY . . . 202
9.1.3 Function Documentation . . . 202
9.1.3.1 main() . . . 202
9.2 applications/BlinkLed/main.cpp . . . 203
9.3 applications/ESP32_WiFi_Telemetry/main.cpp File Reference . . . 203
9.3.1 Detailed Description . . . 203
9.3.2 Function Documentation . . . 204
9.3.2.1 main() . . . 204
9.4 applications/ESP32_WiFi_Telemetry/main.cpp . . . 204
9.5 applications/I2C_OLED_Dashboard/main.cpp File Reference . . . 205
9.5.1 Detailed Description . . . 205
9.5.2 Function Documentation . . . 206
9.5.2.1 main() . . . 206
9.6 applications/I2C_OLED_Dashboard/main.cpp . . . 206

9.7 applications/LCD_EnvironmentMonitor/main.cpp File Reference . . . 207
9.7.1 Detailed Description . . . 207
9.7.2 Function Documentation . . . 208
9.7.2.1 main() . . . 208
9.8 applications/LCD_EnvironmentMonitor/main.cpp . . . 208
9.9 applications/LightMeter/main.cpp File Reference . . . 209
9.9.1 Detailed Description . . . 210
9.9.2 Macro Definition Documentation . . . 210
9.9.2.1 LDRVD . . . 210
9.9.2.2 LED0 . . . 210
9.9.2.3 LED1 . . . 210
9.9.2.4 LED2 . . . 211
9.9.2.5 LED3 . . . 211
9.9.2.6 LED4 . . . 211
9.9.2.7 LED5 . . . 211
9.9.2.8 LED6 . . . 211
9.9.2.9 LED7 . . . 212
9.9.2.10 LUX_MAX . . . 212
9.9.2.11 LUX_MIN . . . 212
9.9.3 Function Documentation . . . 212
9.9.3.1 main() . . . 212
9.10 applications/LightMeter/main.cpp . . . 213
9.11 applications/MusicBox/main.cpp File Reference . . . 214
9.11.1 Detailed Description . . . 215
9.11.2 Macro Definition Documentation . . . 215
9.11.2.1 BUZZER . . . 215
9.11.2.2 NOTE_DURATION_MS . . . 215
9.11.2.3 PUSHBUTTON . . . 215
9.11.2.4 SONG_LENGTH . . . 216
9.11.3 Function Documentation . . . 216
9.11.3.1 main() . . . 216
9.12 applications/MusicBox/main.cpp . . . 217
9.13 applications/Organ/main.cpp File Reference . . . 218
9.13.1 Detailed Description . . . 218
9.13.2 Macro Definition Documentation . . . 219
9.13.2.1 BUZZER . . . 219
9.13.2.2 NOTE_DURATION . . . 219
9.13.3 Function Documentation . . . 219
9.13.3.1 main() . . . 219
9.14 applications/Organ/main.cpp . . . 221
9.15 applications/PID_MotorControl/main.cpp File Reference . . . 222
9.15.1 Detailed Description . . . 222

9.15.2 Function Documentation . . . 223
9.15.2.1 main() . . . 223
9.16 applications/PID_MotorControl/main.cpp . . . 223
9.17 applications/TelemetryStreamer/main.cpp File Reference . . . 224
9.17.1 Detailed Description . . . 224
9.17.2 Function Documentation . . . 225
9.17.2.1 main() . . . 225
9.18 applications/TelemetryStreamer/main.cpp . . . 225
9.19 applications/tests/WeakIsr/main.cpp File Reference . . . 226
9.19.1 Detailed Description . . . 226
9.19.2 Function Documentation . . . 226
9.19.2.1 main() . . . 226
9.20 applications/tests/WeakIsr/main.cpp . . . 227
9.21 projects/AutonomousRobotController/main.cpp File Reference . . . 228
9.21.1 Detailed Description . . . 228
9.21.2 Function Documentation . . . 229
9.21.2.1 main() . . . 229
9.22 projects/AutonomousRobotController/main.cpp . . . 229
9.23 projects/ESP32_SmartEnvironmentalNode/main.cpp File Reference . . . 230
9.23.1 Detailed Description . . . 231
9.23.2 Function Documentation . . . 231
9.23.2.1 main() . . . 231
9.24 projects/ESP32_SmartEnvironmentalNode/main.cpp . . . 232
9.25 projects/SmartHomeGateway/main.cpp File Reference . . . 234
9.25.1 Detailed Description . . . 234
9.25.2 Function Documentation . . . 234
9.25.2.1 main() . . . 234
9.26 projects/SmartHomeGateway/main.cpp . . . 235
9.27 README.md File Reference . . . 237
9.28 README.md File Reference . . . 237
9.29 README.md File Reference . . . 237
9.30 README.md File Reference . . . 237
9.31 README.md File Reference . . . 237
9.32 README.md File Reference . . . 237
9.33 README.md File Reference . . . 237
9.34 README.md File Reference . . . 237
9.35 README.md File Reference . . . 237
9.36 README.md File Reference . . . 237
9.37 README.md File Reference . . . 237
9.38 README.md File Reference . . . 237
9.39 README.md File Reference . . . 237
9.40 README.md File Reference . . . 237

9.41 Buzzer.cpp File Reference . . . 237
9.42 Buzzer.cpp . . . 238
9.43 Buzzer.h File Reference . . . 238
9.44 Buzzer.h . . . 239
9.45 DCMotor.cpp File Reference . . . 240
9.46 DCMotor.cpp . . . 240
9.47 DCMotor.h File Reference . . . 240
9.47.1 Detailed Description . . . 241
9.48 DCMotor.h . . . 242
9.49 LDR.cpp File Reference . . . 243
9.50 LDR.cpp . . . 243
9.51 LDR.h File Reference . . . 244
9.52 LDR.h . . . 244
9.53 Led.cpp File Reference . . . 244
9.54 Led.cpp . . . 245
9.55 Led.h File Reference . . . 245
9.56 Led.h . . . 246
9.57 PCF8574_LCD.cpp File Reference . . . 246
9.57.1 Detailed Description . . . 246
9.57.2 Macro Definition Documentation . . . 247
9.57.2.1 LCD_BACKLIGHT . . . 247
9.57.2.2 LCD_ENABLE_BIT . . . 247
9.57.2.3 LCD_RS_BIT . . . 247
9.58 PCF8574_LCD.cpp . . . 247
9.59 PCF8574_LCD.h File Reference . . . 248
9.59.1 Detailed Description . . . 249
9.60 PCF8574_LCD.h . . . 249
9.61 PushButton.cpp File Reference . . . 249
9.62 PushButton.cpp . . . 250
9.63 PushButton.h File Reference . . . 250
9.64 PushButton.h . . . 251
9.65 RotaryEncoder.cpp File Reference . . . 251
9.65.1 Detailed Description . . . 251
9.66 RotaryEncoder.cpp . . . 252
9.67 RotaryEncoder.h File Reference . . . 252
9.67.1 Detailed Description . . . 253
9.68 RotaryEncoder.h . . . 253
9.69 ServoMotor.cpp File Reference . . . 253
9.70 ServoMotor.cpp . . . 254
9.71 ServoMotor.h File Reference . . . 255
9.71.1 Detailed Description . . . 255
9.72 ServoMotor.h . . . 258

9.73 SSD1306_OLED.cpp File Reference . . . 259
9.73.1 Detailed Description . . . 259
9.73.2 Variable Documentation . . . 259
9.73.2.1 font5x7 . . . 259
9.74 SSD1306_OLED.cpp . . . 260
9.75 SSD1306_OLED.h File Reference . . . 262
9.75.1 Detailed Description . . . 262
9.76 SSD1306_OLED.h . . . 262
9.77 StepperMotor.cpp File Reference . . . 263
9.78 StepperMotor.cpp . . . 263
9.79 StepperMotor.h File Reference . . . 268
9.80 StepperMotor.h . . . 268
9.81 ADC.cpp File Reference . . . 270
9.82 ADC.cpp . . . 270
9.83 ADC.h File Reference . . . 274
9.84 ADC.h . . . 274
9.85 ExternInterrupt.cpp File Reference . . . 275
9.86 ExternInterrupt.cpp . . . 276
9.87 ExternInterrupt.h File Reference . . . 277
9.87.1 Detailed Description . . . 278
9.88 ExternInterrupt.h . . . 279
9.89 MCU.cpp File Reference . . . 280
9.90 MCU.cpp . . . 280
9.91 MCU.h File Reference . . . 282
9.91.1 Detailed Description . . . 283
9.92 MCU.h . . . 284
9.93 TimerCounter.h File Reference . . . 285
9.93.1 Detailed Description . . . 285
9.94 TimerCounter.h . . . 293
9.95 TimerCounter0.cpp File Reference . . . 300
9.96 TimerCounter0.cpp . . . 300
9.97 TimerCounter0.h File Reference . . . 303
9.98 TimerCounter0.h . . . 304
9.99 TimerCounter1.cpp File Reference . . . 307
9.100 TimerCounter1.cpp . . . 307
9.101 TimerCounter1.h File Reference . . . 312
9.102 TimerCounter1.h . . . 312
9.103 TimerCounter2.cpp File Reference . . . 316
9.104 TimerCounter2.cpp . . . 316
9.105 TimerCounter2.h File Reference . . . 320
9.106 TimerCounter2.h . . . 320
9.107 TWI.cpp File Reference . . . 324

9.107.1 Detailed Description . . . 324
9.107.2 Macro Definition Documentation . . . 325
9.107.2.1 TWI_TIMEOUT_CYCLES . . . 325
9.108 TWI.cpp . . . 325
9.109 TWI.h File Reference . . . 326
9.109.1 Detailed Description . . . 326
9.110 TWI.h . . . 327
9.111 WatchdogTimer.cpp File Reference . . . 327
9.112 WatchdogTimer.cpp . . . 327
9.113 WatchdogTimer.h File Reference . . . 328
9.113.1 Detailed Description . . . 329
9.114 WatchdogTimer.h . . . 330
9.115 Pin.cpp File Reference . . . 331
9.116 Pin.cpp . . . 331
9.117 Pin.h File Reference . . . 332
9.118 Pin.h . . . 333
9.119 SPI.cpp File Reference . . . 333
9.120 SPI.cpp . . . 333
9.121 SPI.h File Reference . . . 335
9.121.1 Detailed Description . . . 336
9.122 SPI.h . . . 337
9.123 Telemetry.cpp File Reference . . . 338
9.123.1 Detailed Description . . . 338
9.124 Telemetry.cpp . . . 338
9.125 Telemetry.h File Reference . . . 339
9.125.1 Detailed Description . . . 340
9.126 Telemetry.h . . . 340
9.127 USART0.cpp File Reference . . . 341
9.128 USART0.cpp . . . 341
9.129 USART0.h File Reference . . . 346
9.130 USART0.h . . . 346
9.131 buzzer_pitches_16bit.h File Reference . . . 348
9.131.1 Macro Definition Documentation . . . 350
9.131.1.1 A0 . . . 350
9.131.1.2 A1 . . . 350
9.131.1.3 A2 . . . 350
9.131.1.4 A3 . . . 350
9.131.1.5 A4 . . . 351
9.131.1.6 A5 . . . 351
9.131.1.7 A6 . . . 351
9.131.1.8 A7 . . . 351
9.131.1.9 Ax0 . . . 351

9.131.1.10 Ax1 . . . 351
9.131.1.11 Ax2 . . . 351
9.131.1.12 Ax3 . . . 352
9.131.1.13 Ax4 . . . 352
9.131.1.14 Ax5 . . . 352
9.131.1.15 Ax6 . . . 352
9.131.1.16 Ax7 . . . 352
9.131.1.17 B0 . . . 352
9.131.1.18 B1 . . . 352
9.131.1.19 B2 . . . 353
9.131.1.20 B3 . . . 353
9.131.1.21 B4 . . . 353
9.131.1.22 B5 . . . 353
9.131.1.23 B6 . . . 353
9.131.1.24 B7 . . . 353
9.131.1.25 C0 . . . 353
9.131.1.26 C1 . . . 354
9.131.1.27 C2 . . . 354
9.131.1.28 C3 . . . 354
9.131.1.29 C4 . . . 354
9.131.1.30 C5 . . . 354
9.131.1.31 C6 . . . 354
9.131.1.32 C7 . . . 354
9.131.1.33 Cx0 . . . 355
9.131.1.34 Cx1 . . . 355
9.131.1.35 Cx2 . . . 355
9.131.1.36 Cx3 . . . 355
9.131.1.37 Cx4 . . . 355
9.131.1.38 Cx5 . . . 355
9.131.1.39 Cx6 . . . 355
9.131.1.40 Cx7 . . . 356
9.131.1.41 D0 . . . 356
9.131.1.42 D1 . . . 356
9.131.1.43 D2 . . . 356
9.131.1.44 D3 . . . 356
9.131.1.45 D4 . . . 356
9.131.1.46 D5 . . . 356
9.131.1.47 D6 . . . 357
9.131.1.48 D7 . . . 357
9.131.1.49 Dx0 . . . 357
9.131.1.50 Dx1 . . . 357
9.131.1.51 Dx2 . . . 357

9.131.1.52 Dx3 . . . 357
9.131.1.53 Dx4 . . . 357
9.131.1.54 Dx5 . . . 358
9.131.1.55 Dx6 . . . 358
9.131.1.56 Dx7 . . . 358
9.131.1.57 E0 . . . 358
9.131.1.58 E1 . . . 358
9.131.1.59 E2 . . . 358
9.131.1.60 E3 . . . 358
9.131.1.61 E4 . . . 359
9.131.1.62 E5 . . . 359
9.131.1.63 E6 . . . 359
9.131.1.64 E7 . . . 359
9.131.1.65 F0 . . . 359
9.131.1.66 F1 . . . 359
9.131.1.67 F2 . . . 359
9.131.1.68 F3 . . . 360
9.131.1.69 F4 . . . 360
9.131.1.70 F5 . . . 360
9.131.1.71 F6 . . . 360
9.131.1.72 F7 . . . 360
9.131.1.73 Fx0 . . . 360
9.131.1.74 Fx1 . . . 360
9.131.1.75 Fx2 . . . 361
9.131.1.76 Fx3 . . . 361
9.131.1.77 Fx4 . . . 361
9.131.1.78 Fx5 . . . 361
9.131.1.79 Fx6 . . . 361
9.131.1.80 Fx7 . . . 361
9.131.1.81 G0 . . . 361
9.131.1.82 G1 . . . 362
9.131.1.83 G2 . . . 362
9.131.1.84 G3 . . . 362
9.131.1.85 G4 . . . 362
9.131.1.86 G5 . . . 362
9.131.1.87 G6 . . . 362
9.131.1.88 G7 . . . 362
9.131.1.89 Gx0 . . . 363
9.131.1.90 Gx1 . . . 363
9.131.1.91 Gx2 . . . 363
9.131.1.92 Gx3 . . . 363
9.131.1.93 Gx4 . . . 363

9.131.1.94 Gx5 . . . 363
9.131.1.95 Gx6 . . . 363
9.131.1.96 Gx7 . . . 364
9.132 buzzer_pitches_16bit.h . . . 364
9.133 buzzer_pitches_8bit.h File Reference . . . 365
9.133.1 Macro Definition Documentation . . . 366
9.133.1.1 A0 . . . 366
9.133.1.2 A1 . . . 366
9.133.1.3 A2 . . . 366
9.133.1.4 Ax0 . . . 366
9.133.1.5 Ax1 . . . 366
9.133.1.6 Ax2 . . . 367
9.133.1.7 B0 . . . 367
9.133.1.8 B1 . . . 367
9.133.1.9 B2 . . . 367
9.133.1.10 C1 . . . 367
9.133.1.11 C2 . . . 367
9.133.1.12 C3 . . . 367
9.133.1.13 Cx0 . . . 367
9.133.1.14 Cx2 . . . 368
9.133.1.15 Cx3 . . . 368
9.133.1.16 D1 . . . 368
9.133.1.17 D2 . . . 368
9.133.1.18 D3 . . . 368
9.133.1.19 Dx0 . . . 368
9.133.1.20 Dx2 . . . 368
9.133.1.21 Dx3 . . . 368
9.133.1.22 E1 . . . 369
9.133.1.23 E2 . . . 369
9.133.1.24 E3 . . . 369
9.133.1.25 F1 . . . 369
9.133.1.26 F2 . . . 369
9.133.1.27 F3 . . . 369
9.133.1.28 Fx1 . . . 369
9.133.1.29 Fx2 . . . 369
9.133.1.30 Fx3 . . . 370
9.133.1.31 G1 . . . 370
9.133.1.32 G2 . . . 370
9.133.1.33 G3 . . . 370
9.133.1.34 Gx0 . . . 370
9.133.1.35 Gx1 . . . 370
9.133.1.36 Gx2 . . . 370

9.134 buzzer_pitches_8bit.h . . . 371
9.135 ha_attiny85.h File Reference . . . 371
9.135.1 Detailed Description . . . 372
9.135.2 Macro Definition Documentation . . . 372
9.135.2.1 ADC_ADJUST_RESULT_LEFT . . . 372
9.135.2.2 ADC_ADJUST_RESULT_RIGHT . . . 372
9.135.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT . . . 372
9.135.2.4 ADC_DISABLE . . . 373
9.135.2.5 ADC_DISABLE_AUTOTRIGGER . . . 373
9.135.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT . . . 373
9.135.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER . . . 373
9.135.2.8 ADC_ENABLE . . . 373
9.135.2.9 ADC_ENABLE_AUTOTRIGGER . . . 374
9.135.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT . . . 374
9.135.2.11 ADC_SELECT_ANALOG_INPUT . . . 374
9.135.2.12 ADC_SELECT_CLOCK_PRESCALER . . . 374
9.135.2.13 ADC_SELECT_REF_VOLTAGE . . . 375
9.135.2.14 ADC_START_CONVERSION . . . 375
9.135.2.15 ADC_STOP_CONVERSION . . . 375
9.135.2.16 MCU_SELECT_SLEEP_MODE . . . 375
9.135.2.17 MCU_SLEEP_DISABLE . . . 376
9.135.2.18 MCU_SLEEP_ENABLE . . . 376
9.135.2.19 PUSHBUTTON_DEBOUNCE_TIME_US . . . 376
9.135.2.20 WATCHDOG_SELECT_TIMEOUT . . . 376
9.135.2.21 WATCHDOG_START . . . 376
9.135.2.22 WATCHDOG_STOP . . . 377
9.135.2.23 WATCHDOG_TIMEOUT_INTERRUPT . . . 377
9.136 ha_attiny85.h . . . 377
9.137 ha_base.h File Reference . . . 378
9.137.1 Detailed Description . . . 378
9.137.2 Macro Definition Documentation . . . 379
9.137.2.1 STR . . . 379
9.137.2.2 STRx . . . 379
9.138 ha_base.h . . . 379
9.139 ha_esp32.h File Reference . . . 380
9.139.1 Detailed Description . . . 381
9.139.2 Macro Definition Documentation . . . 382
9.139.2.1 ADC_ADJUST_RESULT_LEFT . . . 382
9.139.2.2 ADC_ADJUST_RESULT_RIGHT . . . 382
9.139.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT . . . 382
9.139.2.4 ADC_DISABLE . . . 382
9.139.2.5 ADC_DISABLE_AUTOTRIGGER . . . 382

9.139.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT 382
9.139.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER 382
9.139.2.8 ADC_ENABLE 383
9.139.2.9 ADC_ENABLE_AUTOTRIGGER 383
9.139.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT 383
9.139.2.11 ADC_SELECT_ANALOG_INPUT 383
9.139.2.12 ADC_SELECT_AUTO_TRIGGER_SOURCE 383
9.139.2.13 ADC_SELECT_CLOCK_PRESCALER 383
9.139.2.14 ADC_SELECT_REF_VOLTAGE 384
9.139.2.15 ADC_START_CONVERSION 384
9.139.2.16 ADC_STOP_CONVERSION 384
9.139.2.17 MCU_ADC_DISABLE 384
9.139.2.18 MCU_ADC_ENABLE 384
9.139.2.19 MCU_SELECT_SLEEP_MODE 384
9.139.2.20 MCU_SLEEP_DISABLE 385
9.139.2.21 MCU_SLEEP_ENABLE 385
9.139.2.22 MCU_SPI_DISABLE 385
9.139.2.23 MCU_SPI_ENABLE 385
9.139.2.24 MCU_TIMER0_DISABLE 385
9.139.2.25 MCU_TIMER0_ENABLE 385
9.139.2.26 MCU_TIMER1_DISABLE 386
9.139.2.27 MCU_TIMER1_ENABLE 386
9.139.2.28 MCU_TIMER2_DISABLE 386
9.139.2.29 MCU_TIMER2_ENABLE 386
9.139.2.30 MCU_TWI_DISABLE 386
9.139.2.31 MCU_TWI_ENABLE 386
9.139.2.32 MCU_USART0_DISABLE 387
9.139.2.33 MCU_USART0_ENABLE 387
9.139.2.34 PUSHBUTTON_DEBOUNCE_TIME_US 387
9.139.2.35 USART0_CONTROL_STATUS_REGISTER 387
9.139.2.36 USART0_DATA_OVERRUN 387
9.139.2.37 USART0_DATA_REGISTER 387
9.139.2.38 USART0_DATA_REGISTER_EMPTY_INTERRUPT 388
9.139.2.39 USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT 388
9.139.2.40 USART0_DISABLE_DOUBLE_SPEED_MODE 388
9.139.2.41 USART0_DISABLE_PARITY_MODE 388
9.139.2.42 USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT 388
9.139.2.43 USART0_DISABLE_RECEIVER 388
9.139.2.44 USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT 389
9.139.2.45 USART0_DISABLE_TRANSMITTER 389
9.139.2.46 USART0_ENABLE_ASYNC_TRANSMISSION_MODE 389
9.139.2.47 USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT 389

9.139.2.48 USART0_ENABLE_DOUBLE_SPEED_MODE . . . 389
9.139.2.49 USART0_ENABLE_EVEN_PARITY_MODE . . . 389
9.139.2.50 USART0_ENABLE_MASTER_SPI_MODE . . . 390
9.139.2.51 USART0_ENABLE_ODD_PARITY_MODE . . . 390
9.139.2.52 USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT . . . 390
9.139.2.53 USART0_ENABLE_RECEIVER . . . 390
9.139.2.54 USART0_ENABLE_SYNC_TRANSMISSION_MODE . . . 390
9.139.2.55 USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT . . . 390
9.139.2.56 USART0_ENABLE_TRANSMITTER . . . 391
9.139.2.57 USART0_FRAME_ERROR . . . 391
9.139.2.58 USART0_PARITY_ERROR . . . 391
9.139.2.59 USART0_RECEIVE_COMPLETE_INTERRUPT . . . 391
9.139.2.60 USART0_SET_5BIT_FRAME_SIZE . . . 391
9.139.2.61 USART0_SET_6BIT_FRAME_SIZE . . . 391
9.139.2.62 USART0_SET_7BIT_FRAME_SIZE . . . 392
9.139.2.63 USART0_SET_8BIT_FRAME_SIZE . . . 392
9.139.2.64 USART0_SET_9BIT_FRAME_SIZE . . . 392
9.139.2.65 USART0_SET_BAUDRATE_HIGH_REGISTER . . . 392
9.139.2.66 USART0_SET_BAUDRATE_LOW_REGISTER . . . 392
9.139.2.67 USART0_SET_ONE_STOP_BIT . . . 392
9.139.2.68 USART0_SET_TWO_STOP_BITS . . . 393
9.139.2.69 USART0_TRANSMIT_COMPLETE_INTERRUPT . . . 393
9.139.2.70 WATCHDOG_SELECT_TIMEOUT . . . 393
9.139.2.71 WATCHDOG_START . . . 393
9.139.2.72 WATCHDOG_STOP . . . 393
9.139.2.73 WATCHDOG_TIMEOUT_INTERRUPT . . . 393
9.140 ha_esp32.h . . . 394
9.141 ha_m168p.h File Reference . . . 395
9.141.1 Detailed Description . . . 395
9.142 ha_m168p.h . . . 396
9.143 ha_m2560.h File Reference . . . 396
9.143.1 Detailed Description . . . 399
9.143.2 Macro Definition Documentation . . . 399
9.143.2.1 ADC_ADJUST_RESULT_LEFT . . . 399
9.143.2.2 ADC_ADJUST_RESULT_RIGHT . . . 399
9.143.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT . . . 400
9.143.2.4 ADC_DISABLE . . . 400
9.143.2.5 ADC_DISABLE_AUTOTRIGGER . . . 400
9.143.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT . . . 400
9.143.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER . . . 400
9.143.2.8 ADC_ENABLE . . . 400
9.143.2.9 ADC_ENABLE_AUTOTRIGGER . . . 401

9.143.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT 401
9.143.2.11 ADC_SELECT_ANALOG_INPUT 401
9.143.2.12 ADC_SELECT_AUTO_TRIGGER_SOURCE 401
9.143.2.13 ADC_SELECT_CLOCK_PRESCALER 402
9.143.2.14 ADC_SELECT_REF_VOLTAGE 402
9.143.2.15 ADC_START_CONVERSION 402
9.143.2.16 ADC_STOP_CONVERSION 402
9.143.2.17 EXT_INT_DISABLE_INT0 402
9.143.2.18 EXT_INT_DISABLE_INT1 403
9.143.2.19 EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT 403
9.143.2.20 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB 403
9.143.2.21 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC 403
9.143.2.22 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD 403
9.143.2.23 EXT_INT_ENABLE_INT0 404
9.143.2.24 EXT_INT_ENABLE_INT1 404
9.143.2.25 EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT 404
9.143.2.26 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB 404
9.143.2.27 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC 404
9.143.2.28 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD 405
9.143.2.29 EXT_INT_INT0_INTERRUPT 405
9.143.2.30 EXT_INT_INT1_INTERRUPT 405
9.143.2.31 EXT_INT_PIN_CHANGE_PORTB_INTERRUPT 405
9.143.2.32 EXT_INT_PIN_CHANGE_PORTC_INTERRUPT 405
9.143.2.33 EXT_INT_PIN_CHANGE_PORTD_INTERRUPT 405
9.143.2.34 EXT_INT_SET_INT0_SENSE_CONTROL 406
9.143.2.35 EXT_INT_SET_INT1_SENSE_CONTROL 406
9.143.2.36 MCU_ADC_DISABLE 406
9.143.2.37 MCU_ADC_ENABLE 406
9.143.2.38 MCU_SELECT_SLEEP_MODE 407
9.143.2.39 MCU_SLEEP_DISABLE 407
9.143.2.40 MCU_SLEEP_ENABLE 407
9.143.2.41 MCU_SPI_DISABLE 407
9.143.2.42 MCU_SPI_ENABLE 407
9.143.2.43 MCU_TIMER0_DISABLE 407
9.143.2.44 MCU_TIMER0_ENABLE 408
9.143.2.45 MCU_TIMER1_DISABLE 408
9.143.2.46 MCU_TIMER1_ENABLE 408
9.143.2.47 MCU_TIMER2_DISABLE 408
9.143.2.48 MCU_TIMER2_ENABLE 408
9.143.2.49 MCU_TWI_DISABLE 408
9.143.2.50 MCU_TWI_ENABLE 408
9.143.2.51 MCU_USART0_DISABLE 408

9.143.2.52 MCU_USART0_ENABLE 409
9.143.2.53 PUSHBUTTON_DEBOUNCE_TIME_US 409
9.143.2.54 SERVOMOTOR_TIMER_ANGLE_COUNT 409
9.143.2.55 SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT 409
9.143.2.56 SPI_CONTROL_REGISTER 409
9.143.2.57 SPI_DATA_REGISTER 410
9.143.2.58 SPI_DISABLE 410
9.143.2.59 SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT 410
9.143.2.60 SPI_ENABLE 410
9.143.2.61 SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT 410
9.143.2.62 SPI_MASTER_MODE 410
9.143.2.63 SPI_SELECT_CLOCK_PRESCALER 411
9.143.2.64 SPI_SELECT_DATA_MODE 411
9.143.2.65 SPI_SELECT_DATA_ORDER 411
9.143.2.66 SPI_SELECT_MASTER_MODE 412
9.143.2.67 SPI_SELECT_SLAVE_MODE 412
9.143.2.68 SPI_STATUS_REGISTER 412
9.143.2.69 SPI_TRANSFER_COMPLETE 412
9.143.2.70 SPI_TRANSFER_COMPLETE_INTERRUPT 412
9.143.2.71 SPI_WRITE_COLLISION 412
9.143.2.72 TIMER0_COM_CHANNEL_A_INTERRUPT 413
9.143.2.73 TIMER0_COM_CHANNEL_B_INTERRUPT 413
9.143.2.74 TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT 413
9.143.2.75 TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT 413
9.143.2.76 TIMER0_DISABLE_OVERFLOW_INTERRUPT 413
9.143.2.77 TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT 413
9.143.2.78 TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT 414
9.143.2.79 TIMER0_ENABLE_OVERFLOW_INTERRUPT 414
9.143.2.80 TIMER0_OVERFLOW_INTERRUPT 414
9.143.2.81 TIMER0_SELECT_CLOCK_SOURCE 414
9.143.2.82 TIMER0_SELECT_COM_CHANNEL_A 414
9.143.2.83 TIMER0_SELECT_COM_CHANNEL_B 415
9.143.2.84 TIMER0_SELECT_OPERATION_MODE 415
9.143.2.85 TIMER0_STOP 415
9.143.2.86 TIMER1_COM_CHANNEL_A_INTERRUPT 416
9.143.2.87 TIMER1_COM_CHANNEL_B_INTERRUPT 416
9.143.2.88 TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT 416
9.143.2.89 TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT 416
9.143.2.90 TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT 416
9.143.2.91 TIMER1_DISABLE_OVERFLOW_INTERRUPT 416
9.143.2.92 TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT 417
9.143.2.93 TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT 417

9.143.2.94 TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT 417
9.143.2.95 TIMER1_ENABLE_OVERFLOW_INTERRUPT 417
9.143.2.96 TIMER1_INPUT_CAPTURE_INTERRUPT 417
9.143.2.97 TIMER1_OVERFLOW_INTERRUPT 417
9.143.2.98 TIMER1_SELECT_CLOCK_SOURCE 418
9.143.2.99 TIMER1_SELECT_COM_CHANNEL_A 418
9.143.2.100 TIMER1_SELECT_COM_CHANNEL_B 418
9.143.2.101 TIMER1_SELECT_OPERATION_MODE 419
9.143.2.102 TIMER1_STOP 419
9.143.2.103 TIMER2_COM_CHANNEL_A_INTERRUPT 419
9.143.2.104 TIMER2_COM_CHANNEL_B_INTERRUPT 419
9.143.2.105 TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT 419
9.143.2.106 TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT 420
9.143.2.107 TIMER2_DISABLE_OVERFLOW_INTERRUPT 420
9.143.2.108 TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT 420
9.143.2.109 TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT 420
9.143.2.110 TIMER2_ENABLE_OVERFLOW_INTERRUPT 420
9.143.2.111 TIMER2_OVERFLOW_INTERRUPT 420
9.143.2.112 TIMER2_SELECT_CLOCK_SOURCE 421
9.143.2.113 TIMER2_SELECT_COM_CHANNEL_A 421
9.143.2.114 TIMER2_SELECT_COM_CHANNEL_B 421
9.143.2.115 TIMER2_SELECT_OPERATION_MODE 422
9.143.2.116 TIMER2_STOP 422
9.143.2.117 USART0_CONTROL_STATUS_REGISTER 422
9.143.2.118 USART0_DATA_OVERRUN 422
9.143.2.119 USART0_DATA_REGISTER 422
9.143.2.120 USART0_DATA_REGISTER_EMPTY_INTERRUPT 423
9.143.2.121 USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT 423
9.143.2.122 USART0_DISABLE_DOUBLE_SPEED_MODE [1/2] 423
9.143.2.123 USART0_DISABLE_DOUBLE_SPEED_MODE [2/2] 423
9.143.2.124 USART0_DISABLE_PARITY_MODE 423
9.143.2.125 USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT 423
9.143.2.126 USART0_DISABLE_RECEIVER 423
9.143.2.127 USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT 423
9.143.2.128 USART0_DISABLE_TRANSMITTER 424
9.143.2.129 USART0_ENABLE_ASYNC_TRANSMISSION_MODE 424
9.143.2.130 USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT 424
9.143.2.131 USART0_ENABLE_DOUBLE_SPEED_MODE 424
9.143.2.132 USART0_ENABLE_EVEN_PARITY_MODE 424
9.143.2.133 USART0_ENABLE_MASTER_SPI_MODE 424
9.143.2.134 USART0_ENABLE_ODD_PARITY_MODE 424
9.143.2.135 USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT 424

9.143.2.136 USART0_ENABLE_RECEIVER . . . 425
9.143.2.137 USART0_ENABLE_SYNC_TRANSMISSION_MODE . . . 425
9.143.2.138 USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT . . . 425
9.143.2.139 USART0_ENABLE_TRANSMITTER . . . 425
9.143.2.140 USART0_FRAME_ERROR . . . 425
9.143.2.141 USART0_PARITY_ERROR . . . 425
9.143.2.142 USART0_RECEIVE_COMPLETE_INTERRUPT . . . 425
9.143.2.143 USART0_SET_5BIT_FRAME_SIZE . . . 426
9.143.2.144 USART0_SET_6BIT_FRAME_SIZE . . . 426
9.143.2.145 USART0_SET_7BIT_FRAME_SIZE . . . 426
9.143.2.146 USART0_SET_8BIT_FRAME_SIZE . . . 426
9.143.2.147 USART0_SET_9BIT_FRAME_SIZE . . . 427
9.143.2.148 USART0_SET_BAUDRATE_HIGH_REGISTER . . . 427
9.143.2.149 USART0_SET_BAUDRATE_LOW_REGISTER . . . 427
9.143.2.150 USART0_SET_ONE_STOP_BIT . . . 427
9.143.2.151 USART0_SET_TWO_STOP_BITS . . . 427
9.143.2.152 USART0_TRANSMIT_COMPLETE_INTERRUPT . . . 427
9.143.2.153 WATCHDOG_SELECT_TIMEOUT . . . 428
9.143.2.154 WATCHDOG_START . . . 428
9.143.2.155 WATCHDOG_STOP . . . 428
9.143.2.156 WATCHDOG_TIMEOUT_INTERRUPT . . . 428
9.144 ha_m2560.h . . . 429
9.145 ha_m328p.h File Reference . . . 433
9.145.1 Detailed Description . . . 437
9.145.2 Macro Definition Documentation . . . 437
9.145.2.1 ADC_ADJUST_RESULT_LEFT . . . 437
9.145.2.2 ADC_ADJUST_RESULT_RIGHT . . . 437
9.145.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT . . . 437
9.145.2.4 ADC_DISABLE . . . 437
9.145.2.5 ADC_DISABLE_AUTOTRIGGER . . . 437
9.145.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT . . . 438
9.145.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER . . . 438
9.145.2.8 ADC_ENABLE . . . 438
9.145.2.9 ADC_ENABLE_AUTOTRIGGER . . . 438
9.145.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT . . . 438
9.145.2.11 ADC_SELECT_ANALOG_INPUT . . . 438
9.145.2.12 ADC_SELECT_AUTO_TRIGGER_SOURCE . . . 439
9.145.2.13 ADC_SELECT_CLOCK_PRESCALER . . . 439
9.145.2.14 ADC_SELECT_REF_VOLTAGE . . . 439
9.145.2.15 ADC_START_CONVERSION . . . 439
9.145.2.16 ADC_STOP_CONVERSION . . . 440
9.145.2.17 EXT_INT_DISABLE_INT0 . . . 440

9.145.2.18 EXT_INT_DISABLE_INT1 . . . 440
9.145.2.19 EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT . . . 440
9.145.2.20 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB . . . 440
9.145.2.21 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC . . . 440
9.145.2.22 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD . . . 441
9.145.2.23 EXT_INT_ENABLE_INT0 . . . 441
9.145.2.24 EXT_INT_ENABLE_INT1 . . . 441
9.145.2.25 EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT . . . 441
9.145.2.26 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB . . . 441
9.145.2.27 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC . . . 441
9.145.2.28 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD . . . 442
9.145.2.29 EXT_INT_INT0_INTERRUPT . . . 442
9.145.2.30 EXT_INT_INT1_INTERRUPT . . . 442
9.145.2.31 EXT_INT_PIN_CHANGE_PORTB_INTERRUPT . . . 442
9.145.2.32 EXT_INT_PIN_CHANGE_PORTC_INTERRUPT . . . 442
9.145.2.33 EXT_INT_PIN_CHANGE_PORTD_INTERRUPT . . . 442
9.145.2.34 EXT_INT_SET_INT0_SENSE_CONTROL . . . 443
9.145.2.35 EXT_INT_SET_INT1_SENSE_CONTROL . . . 443
9.145.2.36 MCU_ADC_DISABLE . . . 443
9.145.2.37 MCU_ADC_ENABLE . . . 443
9.145.2.38 MCU_SELECT_SLEEP_MODE . . . 443
9.145.2.39 MCU_SLEEP_DISABLE . . . 444
9.145.2.40 MCU_SLEEP_ENABLE . . . 444
9.145.2.41 MCU_SPI_DISABLE . . . 444
9.145.2.42 MCU_SPI_ENABLE . . . 444
9.145.2.43 MCU_TIMER0_DISABLE . . . 444
9.145.2.44 MCU_TIMER0_ENABLE . . . 444
9.145.2.45 MCU_TIMER1_DISABLE . . . 444
9.145.2.46 MCU_TIMER1_ENABLE . . . 444
9.145.2.47 MCU_TIMER2_DISABLE . . . 445
9.145.2.48 MCU_TIMER2_ENABLE . . . 445
9.145.2.49 MCU_TWI_DISABLE . . . 445
9.145.2.50 MCU_TWI_ENABLE . . . 445
9.145.2.51 MCU_USART0_DISABLE . . . 445
9.145.2.52 MCU_USART0_ENABLE . . . 445
9.145.2.53 PUSHBUTTON_DEBOUNCE_TIME_US . . . 445
9.145.2.54 SERVOMOTOR_TIMER_ANGLE_COUNT . . . 446
9.145.2.55 SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT . . . 446
9.145.2.56 SPI_CONTROL_REGISTER . . . 446
9.145.2.57 SPI_DATA_REGISTER . . . 446
9.145.2.58 SPI_DISABLE . . . 446
9.145.2.59 SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT . . . 446

9.145.2.60 SPI_ENABLE . . . 447
9.145.2.61 SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT . . . 447
9.145.2.62 SPI_MASTER_MODE . . . 447
9.145.2.63 SPI_SELECT_CLOCK_PRESCALER . . . 447
9.145.2.64 SPI_SELECT_DATA_MODE . . . 447
9.145.2.65 SPI_SELECT_DATA_ORDER . . . 448
9.145.2.66 SPI_SELECT_MASTER_MODE . . . 448
9.145.2.67 SPI_SELECT_SLAVE_MODE . . . 448
9.145.2.68 SPI_STATUS_REGISTER . . . 448
9.145.2.69 SPI_TRANSFER_COMPLETE . . . 448
9.145.2.70 SPI_TRANSFER_COMPLETE_INTERRUPT . . . 448
9.145.2.71 SPI_WRITE_COLLISION . . . 449
9.145.2.72 TIMER0_COM_CHANNEL_A_INTERRUPT . . . 449
9.145.2.73 TIMER0_COM_CHANNEL_B_INTERRUPT . . . 449
9.145.2.74 TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT . . . 449
9.145.2.75 TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT . . . 449
9.145.2.76 TIMER0_DISABLE_OVERFLOW_INTERRUPT . . . 449
9.145.2.77 TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT . . . 449
9.145.2.78 TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT . . . 449
9.145.2.79 TIMER0_ENABLE_OVERFLOW_INTERRUPT . . . 450
9.145.2.80 TIMER0_OVERFLOW_INTERRUPT . . . 450
9.145.2.81 TIMER0_SELECT_CLOCK_SOURCE . . . 450
9.145.2.82 TIMER0_SELECT_COM_CHANNEL_A . . . 450
9.145.2.83 TIMER0_SELECT_COM_CHANNEL_B . . . 450
9.145.2.84 TIMER0_SELECT_OPERATION_MODE . . . 451
9.145.2.85 TIMER0_STOP . . . 451
9.145.2.86 TIMER1_COM_CHANNEL_A_INTERRUPT . . . 451
9.145.2.87 TIMER1_COM_CHANNEL_B_INTERRUPT . . . 451
9.145.2.88 TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT . . . 451
9.145.2.89 TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT . . . 452
9.145.2.90 TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT . . . 452
9.145.2.91 TIMER1_DISABLE_OVERFLOW_INTERRUPT . . . 452
9.145.2.92 TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT . . . 452
9.145.2.93 TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT . . . 452
9.145.2.94 TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT . . . 452
9.145.2.95 TIMER1_ENABLE_OVERFLOW_INTERRUPT . . . 452
9.145.2.96 TIMER1_INPUT_CAPTURE_INTERRUPT . . . 452
9.145.2.97 TIMER1_OVERFLOW_INTERRUPT . . . 453
9.145.2.98 TIMER1_SELECT_CLOCK_SOURCE . . . 453
9.145.2.99 TIMER1_SELECT_COM_CHANNEL_A . . . 453
9.145.2.100 TIMER1_SELECT_COM_CHANNEL_B . . . 453
9.145.2.101 TIMER1_SELECT_OPERATION_MODE . . . 454

9.145.2.102 TIMER1_STOP . . . 454
9.145.2.103 TIMER2_COM_CHANNEL_A_INTERRUPT . . . 454
9.145.2.104 TIMER2_COM_CHANNEL_B_INTERRUPT . . . 454
9.145.2.105 TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT . . . 454
9.145.2.106 TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT . . . 455
9.145.2.107 TIMER2_DISABLE_OVERFLOW_INTERRUPT . . . 455
9.145.2.108 TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT . . . 455
9.145.2.109 TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT . . . 455
9.145.2.110 TIMER2_ENABLE_OVERFLOW_INTERRUPT . . . 455
9.145.2.111 TIMER2_OVERFLOW_INTERRUPT . . . 455
9.145.2.112 TIMER2_SELECT_CLOCK_SOURCE . . . 455
9.145.2.113 TIMER2_SELECT_COM_CHANNEL_A . . . 456
9.145.2.114 TIMER2_SELECT_COM_CHANNEL_B . . . 456
9.145.2.115 TIMER2_SELECT_OPERATION_MODE . . . 456
9.145.2.116 TIMER2_STOP . . . 457
9.145.2.117 USART0_CONTROL_STATUS_REGISTER . . . 457
9.145.2.118 USART0_DATA_OVERRUN . . . 457
9.145.2.119 USART0_DATA_REGISTER . . . 457
9.145.2.120 USART0_DATA_REGISTER_EMPTY_INTERRUPT . . . 457
9.145.2.121 USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT . . . 457
9.145.2.122 USART0_DISABLE_DOUBLE_SPEED_MODE [1/2] . . . 457
9.145.2.123 USART0_DISABLE_DOUBLE_SPEED_MODE [2/2] . . . 458
9.145.2.124 USART0_DISABLE_PARITY_MODE . . . 458
9.145.2.125 USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT . . . 458
9.145.2.126 USART0_DISABLE_RECEIVER . . . 458
9.145.2.127 USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT . . . 458
9.145.2.128 USART0_DISABLE_TRANSMITTER . . . 458
9.145.2.129 USART0_ENABLE_ASYNC_TRANSMISSION_MODE . . . 458
9.145.2.130 USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT . . . 458
9.145.2.131 USART0_ENABLE_DOUBLE_SPEED_MODE . . . 459
9.145.2.132 USART0_ENABLE_EVEN_PARITY_MODE . . . 459
9.145.2.133 USART0_ENABLE_MASTER_SPI_MODE . . . 459
9.145.2.134 USART0_ENABLE_ODD_PARITY_MODE . . . 459
9.145.2.135 USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT . . . 459
9.145.2.136 USART0_ENABLE_RECEIVER . . . 459
9.145.2.137 USART0_ENABLE_SYNC_TRANSMISSION_MODE . . . 459
9.145.2.138 USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT . . . 460
9.145.2.139 USART0_ENABLE_TRANSMITTER . . . 460
9.145.2.140 USART0_FRAME_ERROR . . . 460
9.145.2.141 USART0_PARITY_ERROR . . . 460
9.145.2.142 USART0_RECEIVE_COMPLETE_INTERRUPT . . . 460
9.145.2.143 USART0_SET_5BIT_FRAME_SIZE . . . 460

9.145.2.144 USART0_SET_6BIT_FRAME_SIZE . . . 461
9.145.2.145 USART0_SET_7BIT_FRAME_SIZE . . . 461
9.145.2.146 USART0_SET_8BIT_FRAME_SIZE . . . 461
9.145.2.147 USART0_SET_9BIT_FRAME_SIZE . . . 461
9.145.2.148 USART0_SET_BAUDRATE_HIGH_REGISTER . . . 462
9.145.2.149 USART0_SET_BAUDRATE_LOW_REGISTER . . . 462
9.145.2.150 USART0_SET_ONE_STOP_BIT . . . 462
9.145.2.151 USART0_SET_TWO_STOP_BITS . . . 462
9.145.2.152 USART0_TRANSMIT_COMPLETE_INTERRUPT . . . 462
9.145.2.153 WATCHDOG_SELECT_TIMEOUT . . . 462
9.145.2.154 WATCHDOG_START . . . 463
9.145.2.155 WATCHDOG_STOP . . . 463
9.145.2.156 WATCHDOG_TIMEOUT_INTERRUPT . . . 463
9.146 ha_m328p.h . . . 463
9.147 PIDController.cpp File Reference . . . 469
9.147.1 Detailed Description . . . 469
9.148 PIDController.cpp . . . 469
9.149 PIDController.h File Reference . . . 470
9.149.1 Detailed Description . . . 471
9.150 PIDController.h . . . 471
9.151 utils_m328p.cpp File Reference . . . 471
9.152 utils_m328p.cpp . . . 472
9.153 utils_m328p.h File Reference . . . 472
9.154 utils_m328p.h . . . 472

Index **473**

Chapter 1

Directory Hierarchy

1.1 Directories

applications . . . 15
BlinkLed . . . 16
main.cpp . . . 201
ESP32_WiFi_Telemetry . . . 19
main.cpp . . . 203
I2C_OLED_Dashboard . . . 20
main.cpp . . . 205
LCD_EnvironmentMonitor . . . 21
main.cpp . . . 207
LightMeter . . . 22
main.cpp . . . 209
MusicBox . . . 24
main.cpp . . . 214
Organ . . . 26
main.cpp . . . 218
PID_MotorControl . . . 28
main.cpp . . . 222
TelemetryStreamer . . . 30
main.cpp . . . 224
tests . . . 30
WeakIsr . . . 31
main.cpp . . . 226
AutonomousRobotController . . . 15
main.cpp . . . 228
BlinkLed . . . 16
main.cpp . . . 201
component . . . 17
Buzzer.cpp . . . 237
Buzzer.h . . . 238
DCMotor.cpp . . . 240
DCMotor.h . . . 240
LDR.cpp . . . 243
LDR.h . . . 244
Led.cpp . . . 244
Led.h . . . 245
PCF8574_LCD.cpp . . . 246

PCF8574_LCD.h . . . 248
PushButton.cpp . . . 249
PushButton.h . . . 250
RotaryEncoder.cpp . . . 251
RotaryEncoder.h . . . 252
ServoMotor.cpp . . . 253
ServoMotor.h . . . 255
SSD1306_OLED.cpp . . . 259
SSD1306_OLED.h . . . 262
StepperMotor.cpp . . . 263
StepperMotor.h . . . 268
core . . . 18
ADC.cpp . . . 270
ADC.h . . . 274
ExternInterrupt.cpp . . . 275
ExternInterrupt.h . . . 277
MCU.cpp . . . 280
MCU.h . . . 282
TimerCounter.h . . . 285
TimerCounter0.cpp . . . 300
TimerCounter0.h . . . 303
TimerCounter1.cpp . . . 307
TimerCounter1.h . . . 312
TimerCounter2.cpp . . . 316
TimerCounter2.h . . . 320
TWI.cpp . . . 324
TWI.h . . . 326
WatchdogTimer.cpp . . . 327
WatchdogTimer.h . . . 328
ESP32_SmartEnvironmentalNode . . . 18
main.cpp . . . 230
ESP32_WiFi_Telemetry . . . 19
main.cpp . . . 203
I2C_OLED_Dashboard . . . 20
main.cpp . . . 205
io . . . 21
Pin.cpp . . . 331
Pin.h . . . 332
SPI.cpp . . . 333
SPI.h . . . 335
Telemetry.cpp . . . 338
Telemetry.h . . . 339
USART0.cpp . . . 341
USART0.h . . . 346
LCD_EnvironmentMonitor . . . 21
main.cpp . . . 207
lib . . . 21
component . . . 17
Buzzer.cpp . . . 237
Buzzer.h . . . 238
DCMotor.cpp . . . 240
DCMotor.h . . . 240
LDR.cpp . . . 243
LDR.h . . . 244
Led.cpp . . . 244
Led.h . . . 245

PCF8574_LCD.cpp . . . 246
PCF8574_LCD.h . . . 248
PushButton.cpp . . . 249
PushButton.h . . . 250
RotaryEncoder.cpp . . . 251
RotaryEncoder.h . . . 252
ServoMotor.cpp . . . 253
ServoMotor.h . . . 255
SSD1306_OLED.cpp . . . 259
SSD1306_OLED.h . . . 262
StepperMotor.cpp . . . 263
StepperMotor.h . . . 268
core . . . 18
ADC.cpp . . . 270
ADC.h . . . 274
ExternInterrupt.cpp . . . 275
ExternInterrupt.h . . . 277
MCU.cpp . . . 280
MCU.h . . . 282
TimerCounter.h . . . 285
TimerCounter0.cpp . . . 300
TimerCounter0.h . . . 303
TimerCounter1.cpp . . . 307
TimerCounter1.h . . . 312
TimerCounter2.cpp . . . 316
TimerCounter2.h . . . 320
TWI.cpp . . . 324
TWI.h . . . 326
WatchdogTimer.cpp . . . 327
WatchdogTimer.h . . . 328
io . . . 21
Pin.cpp . . . 331
Pin.h . . . 332
SPI.cpp . . . 333
SPI.h . . . 335
Telemetry.cpp . . . 338
Telemetry.h . . . 339
USART0.cpp . . . 341
USART0.h . . . 346
utils . . . 30
buzzer_pitches_16bit.h . . . 348
buzzer_pitches_8bit.h . . . 365
ha_attiny85.h . . . 371
ha_base.h . . . 378
ha_esp32.h . . . 380
ha_m168p.h . . . 395
ha_m2560.h . . . 396
ha_m328p.h . . . 433
PIDController.cpp . . . 469
PIDController.h . . . 470
utils_m328p.cpp . . . 471
utils_m328p.h . . . 472
LightMeter . . . 22
main.cpp . . . 209
MusicBox . . . 24
main.cpp . . . 214
Organ . . . 26

main.cpp . . . 218
PID_MotorControl . . . 28
main.cpp . . . 222
projects . . . 29
AutonomousRobotController . . . 15
main.cpp . . . 228
ESP32_SmartEnvironmentalNode . . . 18
main.cpp . . . 230
SmartHomeGateway . . . 29
main.cpp . . . 234
SmartHomeGateway . . . 29
main.cpp . . . 234
TelemetryStreamer . . . 30
main.cpp . . . 224
tests . . . 30
WeakIsr . . . 31
main.cpp . . . 226
utils . . . 30
buzzer_pitches_16bit.h . . . 348
buzzer_pitches_8bit.h . . . 365
ha_attiny85.h . . . 371
ha_base.h . . . 378
ha_esp32.h . . . 380
ha_m168p.h . . . 395
ha_m2560.h . . . 396
ha_m328p.h . . . 433
PIDController.cpp . . . 469
PIDController.h . . . 470
utils_m328p.cpp . . . 471
utils_m328p.h . . . 472
WeakIsr . . . 31
main.cpp . . . 226

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

component
AVR chip external components 33
core
AVR chip internal core components 34
io
AVR chip internal i/o components 43
utils 48

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

core::ADConverter . . . 51
component::Buzzer . . . 61
component::DCMotor . . . 65
core::ExternInterrupt . . . 68
component::LDR . . . 74
component::Led . . . 77
core::MCU . . . 81
component::PCF8574_LCD . . . 85
utils::PIDController . . . 89
io::Pin . . . 94
io::Port . . . 98
component::PushButton . . . 99
component::RotaryEncoder . . . 104
component::ServoMotor . . . 107
io::SPI . . . 112
component::SSD1306_OLED . . . 120
component::StepperMotor . . . 125
io::Telemetry . . . 136
core::TimerCounter . . . 139
    core::TimerCounter0 . . . 143
    core::TimerCounter1 . . . 152
    core::TimerCounter2 . . . 163
core::TWI . . . 173
io::USART0 . . . 177
core::WatchdogTimer . . . 196

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

core::ADConverter
: Class for handling ADC . . . 51

component::Buzzer
: Class for handling a Buzzer component . . . 61

component::DCMotor . . . 65

core::ExternInterrupt . . . 68

component::LDR
: Class for handling an LDR component . . . 74

component::Led
: Class for handling a Led component . . . 77

core::MCU . . . 81

component::PCF8574_LCD . . . 85

utils::PIDController . . . 89

io::Pin
: Class for handling a Pin . . . 94

io::Port
: Contains defintions of pointers to the AVR port's registers . . . 98

component::PushButton
: Class for handling a PushButton component . . . 99

component::RotaryEncoder . . . 104

component::ServoMotor . . . 107

io::SPI . . . 112

component::SSD1306_OLED . . . 120

component::StepperMotor . . . 125

io::Telemetry . . . 136

core::TimerCounter . . . 139

core::TimerCounter0 . . . 143

core::TimerCounter1 . . . 152

core::TimerCounter2 . . . 163

core::TWI . . . 173

io::USART0
: Class for handling USART0 . . . 177

core::WatchdogTimer . . . 196

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

applications/BlinkLed/main.cpp . . . 201
applications/ESP32_WiFi_Telemetry/main.cpp
ESP32 Wi-Fi & Serial Telemetry Streaming Application Example . . . 203
applications/I2C_OLED_Dashboard/main.cpp
I2C OLED display dashboard application . . . 205
applications/LCD_EnvironmentMonitor/main.cpp
Environment monitor application using PCF8574 I2C LCD and LDR light sensor . . . 207
applications/LightMeter/main.cpp . . . 209
applications/MusicBox/main.cpp . . . 214
applications/Organ/main.cpp . . . 218
applications/PID_MotorControl/main.cpp
Closed-loop DC motor velocity control using PID and Rotary Encoder . . . 222
applications/TelemetryStreamer/main.cpp
Continuous hardware telemetry streaming over USART0 . . . 224
applications/tests/WeakIsr/main.cpp
Link check: every library interrupt enabled with no app handlers . . . 226
projects/AutonomousRobotController/main.cpp
Autonomous Robot Controller Project . . . 228
projects/ESP32_SmartEnvironmentalNode/main.cpp
ESP32 Smart Environmental Sensing Node Project . . . 230
projects/SmartHomeGateway/main.cpp
Smart Home Peripheral Gateway Project . . . 234
Buzzer.cpp . . . 237
Buzzer.h . . . 238
DCMotor.cpp . . . 240
DCMotor.h
Header file of the DCMotor class . . . 240
LDR.cpp . . . 243
LDR.h . . . 244
Led.cpp . . . 244
Led.h . . . 245
PCF8574_LCD.cpp
Implementation of PCF8574 I2C LCD1602/LCD2004 character display driver . . . 246
PCF8574_LCD.h
Header file for PCF8574 I2C LCD1602/LCD2004 character display driver . . . 248

PushButton.cpp . . . 249
PushButton.h . . . 250
RotaryEncoder.cpp
Implementation of the Quadrature Rotary Encoder driver . . . 251
RotaryEncoder.h
Header file for the Quadrature Rotary Encoder driver . . . 252
ServoMotor.cpp . . . 253
ServoMotor.h
Header file of the ServoMotor class . . . 255
SSD1306_OLED.cpp
Implementation of SSD1306 128x64 / 128x32 I2C OLED display driver . . . 259
SSD1306_OLED.h
Header file for SSD1306 128x64 / 128x32 I2C OLED display driver . . . 262
StepperMotor.cpp . . . 263
StepperMotor.h . . . 268
ADC.cpp . . . 270
ADC.h . . . 274
ExternInterrupt.cpp . . . 275
ExternInterrupt.h
Header file of the ExternInterrupt class . . . 277
MCU.cpp . . . 280
MCU.h
Header file of the MCU class . . . 282
TimerCounter.h
Header file of the TimerCounter class . . . 285
TimerCounter0.cpp . . . 300
TimerCounter0.h . . . 303
TimerCounter1.cpp . . . 307
TimerCounter1.h . . . 312
TimerCounter2.cpp . . . 316
TimerCounter2.h . . . 320
TWI.cpp
Implementation of the Two-Wire Interface (I2C) class . . . 324
TWI.h
Header file for the Two-Wire Interface (I2C) class . . . 326
WatchdogTimer.cpp . . . 327
WatchdogTimer.h
Header file of the WatchdogTimer class . . . 328
Pin.cpp . . . 331
Pin.h . . . 332
SPI.cpp . . . 333
SPI.h
Header file of the SPI class . . . 335
Telemetry.cpp
Implementation of the Telemetry class . . . 338
Telemetry.h
Header file for the Telemetry class . . . 339
USART0.cpp . . . 341
USART0.h . . . 346
buzzer_pitches_16bit.h . . . 348
buzzer_pitches_8bit.h . . . 365
ha_attiny85.h
Header file for the hardware abstraction macros of ATtiny85 . . . 371
ha_base.h
Base header file for the basic hardware abstraction macros . . . 378
ha_esp32.h
Header file for the hardware abstraction macros of ESP32 / ESP32-S3 . . . 380

ha_m168p.h
Header file for the hardware abstraction macros of ATmega168P / ATmega88P / ATmega48P . 395
ha_m2560.h
Header file for the hardware abstraction macros of the ATmega2560 396
ha_m328p.h
Header file for the hardware abstraction macros of the Atmega328p 433
PIDController.cpp
Implementation of the PID Controller class 469
PIDController.h
Header file for the PID Controller class 470
utils_m328p.cpp 471
utils_m328p.h 472

Chapter 6

Directory Documentation

6.1 applications Directory Reference

Directories

- directory BlinkLed
- directory ESP32_WiFi_Telemetry
- directory I2C_OLED_Dashboard
- directory LCD_EnvironmentMonitor
- directory LightMeter
- directory MusicBox
- directory Organ
- directory PID_MotorControl
- directory TelemetryStreamer
- directory tests

6.2 AutonomousRobotController Directory Reference

Files

- file main.cpp

  Autonomous Robot Controller Project.

6.2.1 Detailed Description

6.2.2 Autonomous Robot Controller Project

A differential-drive robotic chassis controller using dual DC motors, quadrature encoder feedback, and dual PID velocity control loops.

6.2.3 Hardware Pinout

- Left Motor: PB0, PB1
- Left Encoder: PD2, PD3
- Right Motor: PB2, PB3
- Right Encoder: PD4, PD5

6.3 BlinkLed Directory Reference

Files

- file main.cpp

6.3.1 Detailed Description

This example demonstrates the library usage in a simple blink a Led example. It also shows how to use the Pin and Led abstraction objects.

The pinout of the **ATmega328P** AVR microcontroller is illustrated bellow:

6.3.2 Hardware

- Arduino UNO
- generic Led
- 220 ohm current limiting resistor
- A breadboard

6.3.3 Circuit

The Led with the current limiting resistor are connected to pin PB0 (digital pin 8 in Arduino UNO).

The circuit connection is as follows:

6.3.4 Code

The following code blinks the Led with a delay of 500 ms:

```
#include "Led.h"

#define LED 0  /**< Led pin number */
#define TIMEDELAY 500 /**< Time delay */

int main(void) {

  // Init

  // Instantiate a Led object
  component::Led Led(io::Pin(LED,io::PortB));

  // Mainloop
  while (1) {

      Led.on();
      _delay_ms(TIMEDELAY);

      Led.off();
      _delay_ms(TIMEDELAY);
  }
  return 0;
}
```

6.3.5 Author

- Farid Oubbati
- Date: 12-May-2018
- Copyright (c) 2018

6.3.6 License

This project is licensed under the MIT License - see the `LICENSE.txt` file for more details

6.4 component Directory Reference

Files

- file Buzzer.cpp
- file Buzzer.h
- file DCMotor.cpp
- file DCMotor.h

  Header file of the DCMotor class.
- file LDR.cpp
- file LDR.h
- file Led.cpp
- file Led.h
- file PCF8574_LCD.cpp

  Implementation of PCF8574 I2C LCD1602/LCD2004 character display driver.
- file PCF8574_LCD.h

  Header file for PCF8574 I2C LCD1602/LCD2004 character display driver.
- file PushButton.cpp
- file PushButton.h
- file RotaryEncoder.cpp

  Implementation of the Quadrature Rotary Encoder driver.
- file RotaryEncoder.h

  Header file for the Quadrature Rotary Encoder driver.
- file ServoMotor.cpp
- file ServoMotor.h

  Header file of the ServoMotor class.
- file SSD1306_OLED.cpp

  Implementation of SSD1306 128x64 / 128x32 I2C OLED display driver.
- file SSD1306_OLED.h

  Header file for SSD1306 128x64 / 128x32 I2C OLED display driver.
- file StepperMotor.cpp
- file StepperMotor.h

6.5 core Directory Reference

Files

- file ADC.cpp
- file ADC.h
- file ExternInterrupt.cpp
- file ExternInterrupt.h

  Header file of the ExternInterrupt class.
- file MCU.cpp
- file MCU.h

  Header file of the MCU class.
- file TimerCounter.h

  Header file of the TimerCounter class.
- file TimerCounter0.cpp
- file TimerCounter0.h
- file TimerCounter1.cpp
- file TimerCounter1.h
- file TimerCounter2.cpp
- file TimerCounter2.h
- file TWI.cpp

  Implementation of the Two-Wire Interface (I2C) class.
- file TWI.h

  Header file for the Two-Wire Interface (I2C) class.
- file WatchdogTimer.cpp
- file WatchdogTimer.h

  Header file of the WatchdogTimer class.

6.6 ESP32_SmartEnvironmentalNode Directory Reference

Files

- file main.cpp

  ESP32 Smart Environmental Sensing Node Project.

6.6.1 Detailed Description

6.6.2 ESP32 Smart Environmental Sensing Node

Complete hardware project for the **ESP32** microcontroller built with **TahakomAVRLib**.

6.6.3 Overview

The project integrates an SSD1306 I2C OLED display, LDR light sensor, Quadrature Rotary Encoder UI, PID velocity control loop, and telemetry streamer on an ESP32 board.

6.6.4 Hardware Components

1. **ESP32 Board** (NodeMCU / ESP32 DevKit)
2. **SSD1306 I2C OLED Display** (128x64 pixels)
3. **LDR Light Sensor** with 5k Ohm resistor
4. **Rotary Encoder** (A/B quadrature signals)
5. **Status LED**
6. **UART Telemetry Interface** to Web IDE

6.6.5 Pinout Connections

ESP32 Pin	Component	Function
GPIO 2	Status LED	Blinker Indicator
GPIO 21 (SDA)	SSD1306 OLED SDA	I2C Data Line
GPIO 22 (SCL)	SSD1306 OLED SCL	I2C Clock Line
GPIO 34 (ADC1_CH6)	LDR Light Sensor	Analog Light Reading
GPIO 4	Rotary Encoder A	Quadrature Phase A
GPIO 5	Rotary Encoder B	Quadrature Phase B
GPIO 1 (TX0)	Serial USB	Telemetry Stream Output

6.7 ESP32_WiFi_Telemetry Directory Reference

Files

- file main.cpp

  ESP32 Wi-Fi & Serial Telemetry Streaming Application Example.

6.7.1 Detailed Description

6.7.2 ESP32 Wi-Fi & Serial Telemetry Application

This application demonstrates C++ application development for the **ESP32** microcontroller using **TahakomAVRLib**.

6.7.3 Description

The application performs continuous analog light sensing via ADC channel 0, toggles an onboard status LED on GPIO 2, and streams live `$ADC` and `$LOG` telemetry frames to the **TahakomAVR Web IDE** dashboard.

6.7.4 Hardware Connections

ESP32 Pin	Component	Description
GPIO 2	Onboard LED	Status Blinker
GPIO 34 (ADC1_CH6)	LDR Light Sensor	Analog Light Input
GPIO 1 (TX0)	Serial / USB	Telemetry TX to Web IDE
GPIO 3 (RX0)	Serial / USB	Telemetry RX from Web IDE

6.7.5 Telemetry Framing

The application emits NMEA-style telemetry strings parsed by `server/services/telemetry_service.py`:

- `$ADC,0,<raw_value>`: Continuous ADC channel 0 reading (0 - 1023)
- `$LOG,<message>`: Status events

6.8 I2C_OLED_Dashboard Directory Reference

Files

- file main.cpp

  I2C OLED display dashboard application.

6.8.1 Detailed Description

6.8.2 I2C OLED Dashboard Application

This application demonstrates rendering real-time sensor measurements on an SSD1306 128x64 monochrome OLED screen over I2C.

6.8.3 Hardware Connections

- I2C Bus: SDA = PC4, SCL = PC5 (ATmega328P)
- Analog Input: PC0
- OLED I2C Address: `0x3C`

6.9 io Directory Reference

Files

- file Pin.cpp
- file Pin.h
- file SPI.cpp
- file SPI.h

  Header file of the SPI class.
- file Telemetry.cpp

  Implementation of the Telemetry class.
- file Telemetry.h

  Header file for the Telemetry class.
- file USART0.cpp
- file USART0.h

6.10 LCD_EnvironmentMonitor Directory Reference

Files

- file main.cpp

  Environment monitor application using PCF8574 I2C LCD and LDR light sensor.

6.10.1 Detailed Description

6.10.2 LCD Environment Light Monitor Application

This application monitors ambient light lux using an LDR light sensor and renders values on a character LCD1602 via a PCF8574 I2C backpack.

6.10.3 Hardware Connections

- LCD I2C Address: `0x27` (SDA = PC4, SCL = PC5)
- LDR Sensor: PC0

6.11 lib Directory Reference

Directories

- directory component
- directory core
- directory io
- directory utils

6.12 LightMeter Directory Reference

Files

- file main.cpp

6.12.1 Detailed Description

This example presents an AVR based Light Meter that displays its Lux measurements in a Led bank It also demonstrates the use of the LED, LDR and ADC abstraction objects.

The pinout of the **ATmega328P** AVR microcontroller is illustrated bellow:

6.12.2 Hardware

- Arduino UNO
- Light Dependant Resistor (LDR)
- 5 kOhm resistor or a Potentiometer
- A breadboard

6.12.3 Circuit

The LDR is a passive component. To measure its resistance change we have to send a current in the component between two potentials. We have to create a voltage divider bridge using the LDR and the 5 kOhm resistor. The voltage divider output is read by the AVR Analog-to-Digial-Converter via pin PC0 (A0 pin in Arduino UNO). The light Lux measurements are displayed in a Led bank.

The circuit connection is as follows:

> **Note:** the LED bank occupies all of port D, including PD0 (RxD) and PD1 (TxD), the hardware USART pins of the ATmega328P. With the LEDs wired on PD0/PD1 the USART cannot be used at the same time; to combine the light meter with serial output, move LED0 and LED1 to other pins and keep PD0/PD1 free.

6.12.4 Code

The following code permits to run the AVR based Light Meter that displays its Lux measurements in a Led bank:

```
#include "Led.h"
#include "ADC.h"
#include "LDR.h"
#include "utils_m328p.h"

#define LED0 0 /**< Led 0 */
#define LED1 1 /**< Led 1 */
#define LED2 2 /**< Led 2 */
#define LED3 3 /**< Led 3 */
#define LED4 4 /**< Led 4 */
#define LED5 5 /**< Led 5 */
#define LED6 6 /**< Led 6 */
#define LED7 7 /**< Led 7 */

#define LDRVD 0 /**< LDR */

// Display range of the light meter in lux, from the GL5528-type LDR
// characteristic that LDR::getLux() implements: LUX_MIN is a dimly lit
// room, LUX_MAX is bright indoor lighting close to a window.  Light
// above the range (direct sunlight) saturates the display, all 8 LEDs
// on; light below it switches all LEDs off.
#define LUX_MIN 1
#define LUX_MAX 1000

int main(void) {

  // Init

  // Instantiate Led objects
  component::Led Led[8]={
        {io::Pin(LED0,io::PortD)},
        {io::Pin(LED1,io::PortD)},
        {io::Pin(LED2,io::PortD)},
        {io::Pin(LED3,io::PortD)},
        {io::Pin(LED4,io::PortD)},
        {io::Pin(LED5,io::PortD)},
        {io::Pin(LED6,io::PortD)},
        {io::Pin(LED7,io::PortD)}};


  // Instantiate ADC object
  core::ADConverter &myADC = core::ADConverter::getInstance();

  // Start conversion
  myADC.start();

  // Instantiate LDR object
  component::LDR LDR(io::Pin(LDRVD,io::PortC),myADC);


  // Mainloop
  while (1) {

    // one measurement per loop iteration
    uint16_t l_lux = LDR.getLux(5);

    // clamp into the display range
    if (l_lux < LUX_MIN)
    {
        l_lux = LUX_MIN;
    }
    if (l_lux > LUX_MAX)
    {
        l_lux = LUX_MAX;
    }

    const uint8_t l_numLeds = static_cast<uint8_t>(utils::map(l_lux,
                                                              LUX_MIN,
                                                              LUX_MAX,
                                                              0,
                                                              8));

    for (uint8_t i=0;i<l_numLeds;i++)
    {
        Led[i].on();
    }

    for (uint8_t i=l_numLeds;i<8;i++)
    {
        Led[i].off();
    }

  }
```

```
  return 0;
}
```

6.12.5 Author

- Farid Oubbati
- Date: 12-May-2018
- Copyright (c) 2018

6.12.6 License

This project is licensed under the MIT License - see the `LICENSE.txt` file for more details

6.13 MusicBox Directory Reference

Files

- file main.cpp

6.13.1 Detailed Description

This example allows the user to step through a predefined song in a Buzzer, one note per PushButton press; a note keeps playing as long as the button is held. Every note is audibly 300 ms long. It also demonstrates the use of the Buzzer, TimerCounter1 and PushButton abstraction objects.

The pinout of the **ATmega328P** AVR microcontroller is illustrated bellow:

6.13.2 Hardware

- Arduino UNO
- Piezo speaker/Buzzer
- 100 ohm current limiting resistor
- Switch/PushButton
- A breadboard

6.13.3 Circuit

The Buzzer with the current limiting resistor are connected to pin PB2 (digital pin 10 in Arduino UNO), the output-compare pin OC1B of Timer1 that the timer-driven Buzzer signal is generated on, and the PushButton is connected to PB1 (digital pin 9 in Arduino UNO).

The circuit connection is as follows:

6.13.4 Code

The following code permits to turn a Buzzer into a Music Box that steps through a predefined song, one note per PushButton press:

```
#include "Buzzer.h"
#include "PushButton.h"
#include "TimerCounter1.h"
#include <avr/pgmspace.h>
// Note pitch data
#include "buzzer_pitches_16bit.h"


#define BUZZER 2 /**< Buzzer pin number: PB2 is the Timer1 output-compare
                    pin OC1B, the pin the timer-driven buzz() toggles */

#define PUSHBUTTON 1 /**< PushButton pin number */

// Audible length of every played note in ms: the notes of the song are
// audible at this length, one note per button press
#define NOTE_DURATION_MS 300

#define SONG_LENGTH (sizeof(l_song) / sizeof(uint16_t)) /**< Song length */

int main(void) {

  // Init

  // Instantiate a Buzzer object on the Timer1 output-compare pin OC1B:
  // the timer-driven buzz() drives the hardware compare output, so the
  // buzzer must be wired to the compare pin of the timer in use
  component::Buzzer Buzzer(io::Pin(BUZZER,io::PortB));

  // Instantiate a TimerCounter1 object: the pitch table holds the note
  // periods in us, so the timer is clocked at prescaler 64 where one
  // tick is exactly 4 us at F_CPU = 16 MHz and the tick value of a note
  // is a quarter of its period in us
  core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance(
      core::channel::B, core::operationMode::normal, core::clockSource::PS_64);

  // Instantiate a PushButton object
  component::PushButton PushButton(io::Pin(PUSHBUTTON,io::PortB));

  // Song notes: the table holds the note periods in us and lives in
  // flash so the song does not consume RAM
  const uint16_t l_song[] PROGMEM = {
  E6, E6, E6, C6, E6, G6, G5,
  C6, G5, E5, A5, B5, Ax5, A5,
  G5, E6, G6, A6, F6, G6, E6, C6, D6, B5,
  C6, G5, E5, A5, B5, Ax5, A5,
  G5, E6, G6, A6, F6, G6, E6, C6, D6, B5,
  };

  // Start song at start
  uint8_t l_note = SONG_LENGTH - 1;

  // Initialize PushButton state
  uint8_t l_statePushButton = 0;


  // Mainloop
  while (1) {
      // Check if PushButton is pressed
      if (PushButton.isPressed())
      {
          // Check if a new button press
          if (!l_statePushButton) {
              // play next note
              l_note++;
              // start from start
              if (l_note == SONG_LENGTH) {

                  l_note = 0;

              }

              l_statePushButton = 1;

          }
          // Play note: tick value = period in us / 4 (4 us per tick at
          // prescaler 64), audibly NOTE_DURATION_MS long
          Buzzer.buzz(myTimerCounter1,
                      pgm_read_word(&l_song[l_note]) / 4,
                      NOTE_DURATION_MS,
```

```
                    core::channel::B);
      } else {
          l_statePushButton=0;
      }
  }
  return 0;
}
```

6.13.5 Author

- Farid Oubbati
- Date: 12-May-2018
- Copyright (c) 2018

6.13.6 License

This project is licensed under the MIT License - see the `LICENSE.txt` file for more details

6.14 Organ Directory Reference

Files

- file main.cpp

6.14.1 Detailed Description

This example enables the user to enter a key note via a serial terminal and play the corresponding note in a Buzzer. It also shows how to use the Buzzer and USART0 abstraction objects.

The pinout of the **ATmega328P** AVR microcontroller is illustrated bellow:

6.14.2 Hardware

- Arduino UNO
- Piezo speaker/Buzzer
- 100 ohm current limiting resistor
- A breadboard

6.14.3 Circuit

The Buzzer with the current limiting resistor are connected to pin PB0 (digital pin 8 in Arduino UNO).

The circuit connection is as follows:

6.14.4 Code

The following code permits to turn a Buzzer into an Organ that plays notes entered by a user via a serial terminal (GTKTerm serial port terminal is used):

```
#include "Buzzer.h"
#include "USART0.h"
#include "MCU.h"
#include <avr/pgmspace.h>
// Note pitch data
#include "buzzer_pitches_16bit.h"


#define BUZZER 0 /**< Buzzer pin number */

#define NOTE_DURATION  0xF000 /**< Note duration */

int main(void) {

  // Init

  // Instantiate a Buzzer object
  component::Buzzer Buzzer(io::Pin(BUZZER,io::PortB));

  // Instantiate a USART0 object
  io::USART0 &myUSART0 = io::USART0::getInstance();

  // The serial receive path uses the receive-complete interrupt, so
  // interrupts must be enabled globally
  core::MCU::enableInterrupts();

  // List of possible keypresses: kept in flash so the lookup tables do
  // not consume RAM
  const uint8_t l_keys[] PROGMEM = { 'a', 'w', 's', 'e', 'd', 'f', 't',
      'g', 'y', 'h', 'j', 'i', 'k', 'o',
      'l', 'p', ';', '\"
  };

  // List of the keypresses corresponding pitches
  const uint16_t l_notes[] PROGMEM = { G4, Gx4, A4, Ax4, B4, C5, Cx5,
      D5, Dx5, E5, F5, Fx5, G5, Gx5,
      A5, Ax5, B5, C6
  };

  // Character key from computer serial terminal
  unsigned char l_key;

  // Current note length
  uint16_t l_currentNoteLength = NOTE_DURATION;

  // Check if note key
  uint8_t l_isNote;

  myUSART0.sendString("----- Serial Organ ------\r\n");

  // Mainloop
  while (1) {

      // Send N to signal that AVR chip is ready for next key note input
      myUSART0.sendString("Enter key note\r\n");

      // Wait for key note input: arm the receiver for a single byte
      // once; the receive-complete ISR stores the byte and counts it,
      // so the busy-wait polls the counter without re-arming the
      // buffer on every iteration
      myUSART0.receiveChar(l_key);
      while (myUSART0.getNumberBytesReceived()==0)
      {
      }
      myUSART0.resetNumberBytesReceived();

      /********** Play key notes ***********/

      l_isNote = 0;
      // Loop through keys table
      for (uint8_t i = 0; i < sizeof(l_keys); i++) {
          // Found match in lookup table
          if (l_key == pgm_read_byte(&l_keys[i])) {
              Buzzer.buzz(pgm_read_word(&l_notes[i]), l_currentNoteLength);
              // Record that we've found a note
              l_isNote = 1;
              break;
          }
      }

      // Handle non-note keys: tempo changes and rests
```

```
      if (!l_isNote) {
          if (l_key == '-') {
              // Code for short note: halve the length, but reject the
              // halving when the result would be zero, a zero length
              // would silence every following note
              if (l_currentNoteLength > 1)
              {
                  l_currentNoteLength = l_currentNoteLength / 2;
              }
          }
          else if (l_key == '+') {
              // Code for long note: double the length in 32 bits and
              // saturate at the 16-bit maximum instead of wrapping
              // around to a small or zero length
              uint32_t l_newNoteLength =
                  static_cast<uint32_t>(l_currentNoteLength) * 2UL;
              if (l_newNoteLength > 0xFFFFUL)
              {
                  l_newNoteLength = 0xFFFFUL;
              }
              l_currentNoteLength = static_cast<uint16_t>(l_newNoteLength);
          }
          else {
              // Unrecognized, just rest
              Buzzer.pause(l_currentNoteLength);
          }
      }

  }
  return 0;
}
```

6.14.5 Author

- Farid Oubbati
- Date: 12-May-2018
- Copyright (c) 2018

6.14.6 License

This project is licensed under the MIT License - see the LICENSE.txt file for more details

6.15 PID_MotorControl Directory Reference

Files

- file main.cpp

  Closed-loop DC motor velocity control using PID and Rotary Encoder.

6.15.1 Detailed Description

6.15.2 PID Closed-Loop DC Motor Control Application

This application demonstrates closed-loop speed control of a DC motor using:

- utils::PIDController: Proportional-Integral-Derivative control loop with anti-windup.
- component::DCMotor: H-bridge DC motor driver.
- component::RotaryEncoder: Quadrature encoder feedback.

6.15.3 Hardware Connections

- DC Motor Pins: Port B, Pins 0 and 1
- Rotary Encoder Pins: Port D, Pins 2 and 3
- Microcontroller: ATmega328P @ 16 MHz

6.16 projects Directory Reference

Directories

- directory AutonomousRobotController
- directory ESP32_SmartEnvironmentalNode
- directory SmartHomeGateway

6.17 SmartHomeGateway Directory Reference

Files

- file main.cpp

  Smart Home Peripheral Gateway Project.

6.17.1 Detailed Description

6.17.2 Smart Home Gateway Project

An integrated smart home peripheral node that provides:

- Servo door lock actuation
- Pushbutton door toggle control
- Buzzer audio confirmation tones
- LDR ambient light sensing
- OLED 128x64 display status rendering
- Real-time hardware telemetry over USART0

6.17.3 Hardware Pinout

- OLED I2C: PC4 (SDA), PC5 (SCL)
- Servo Lock: PB1
- Door Pushbutton: PD2
- Buzzer: PD3
- LDR Sensor: PC0

6.18 TelemetryStreamer Directory Reference

Files

- file main.cpp

  Continuous hardware telemetry streaming over USART0.

6.18.1 Detailed Description

6.18.2 Hardware Telemetry Streamer Application

This application streams framed hardware sensor packets (`$ADC,...`, `$PIN,...`, `$LOG,...`) over USART0 to the TahakomAVR Web IDE Oscilloscope and Logic Analyzer dashboard.

6.18.3 Hardware Connections

- Serial UART: PD0 (RX), PD1 (TX)
- Analog Sensor: PC0
- Digital Button: PD2

6.19 tests Directory Reference

Directories

- directory WeakIsr

6.20 utils Directory Reference

Files

- file buzzer_pitches_16bit.h
- file buzzer_pitches_8bit.h
- file ha_attiny85.h

  Header file for the hardware abstraction macros of ATtiny85.
- file ha_base.h

  Base header file for the basic hardware abstraction macros.
- file ha_esp32.h

  Header file for the hardware abstraction macros of ESP32 / ESP32-S3.
- file ha_m168p.h

  Header file for the hardware abstraction macros of ATmega168P / ATmega88P / ATmega48P.
- file ha_m2560.h

  Header file for the hardware abstraction macros of the ATmega2560.
- file ha_m328p.h

  Header file for the hardware abstraction macros of the Atmega328p.
- file PIDController.cpp

  Implementation of the PID Controller class.
- file PIDController.h

  Header file for the PID Controller class.
- file utils_m328p.cpp
- file utils_m328p.h

6.21 WeakIsr Directory Reference

Files

- file main.cpp

  Link check: every library interrupt enabled with no app handlers.

Chapter 7

Namespace Documentation

7.1 component Namespace Reference

AVR chip external components.

Classes

- class Buzzer

  Class for handling a Buzzer component.
- class DCMotor
- class LDR

  Class for handling an LDR component.
- class Led

  Class for handling a Led component.
- class PCF8574_LCD
- class PushButton

  Class for handling a PushButton component.
- class RotaryEncoder
- class ServoMotor
- class SSD1306_OLED
- class StepperMotor

Enumerations

- enum class mode : uint8_t { fullStep =0 , halfStep }

7.1.1 Detailed Description

AVR chip external components.

The namespace englobes all external components that, when hooked up, allows the AVR chip to interact with the external World

The namespace englobes all external components that, when hooked up, allows the AVR MCU to interact with the external World

7.1.2 Enumeration Type Documentation

7.1.2.1 mode

```
enum class component::mode :  uint8_t  [strong]
```

Enumerator

fullStep	
halfStep	

Definition at line 70 of file StepperMotor.h.

```
00070                 : uint8_t {
00071     fullStep=0,
00072     halfStep,
00073 };
```

7.2 core Namespace Reference

AVR chip internal core components.

Classes

- class ADConverter
  Class for handling ADC.
- class ExternInterrupt
- class MCU
- class TimerCounter
- class TimerCounter0
- class TimerCounter1
- class TimerCounter2
- class TWI
- class WatchdogTimer

Enumerations

- enum class resolution : uint8_t { res_8bit =0 , res_9bit , res_10bit , res_11bit , res_12bit , res_13bit , res_14bit , res_15bit , res_16bit }
- enum class referenceVoltage : uint8_t { AREF =0 , AVCC , internal =3 }
- enum class clockPrescaler : uint8_t { PS_2 = 1 , PS_4 , PS_8 , PS_16 , PS_32 , PS_64 , PS_128 }
- enum class autoTriggerSource : uint8_t { freeRunning , analogComparator , extInterrupt , timer0CompareA , timer0Overflow , timer1CompareB , timer1Overflow , timer1Capture }
- enum class senseControl : uint8_t { lowLevel =0 , logicalChange , fallingEdge , risingEdge }
- enum class pinChangePort : uint8_t { PCINTB =0 , PCINTC , PCINTD }
- enum class BODMode : uint8_t { enabled =0 , disabled }

- enum class sleepMode : uint8_t {
  Idle =0 , ADC_NoiseReduction , powerDown , powerSave ,
  standby =6 , extendedStandby }
- enum class channel : uint8_t { A =0 , B }
- enum class compareOutputMode : uint8_t { normal =0 , toggle , clear , set }
- enum class operationMode : uint8_t {
  normal =0 , PWM_PC , PWM_PC_8bit , PWM_PC_9bit ,
  PWM_PC_10bit , PWM_PFC_ICR , PWM_PFC_OCR , PWM_PC_ICR ,
  PWM_PC_OCR , fast_PWM , fast_PWM_8bit , fast_PWM_9bit ,
  fast_PWM_10bit , fast_PWM_ICR , fast_PWM_OCR , CTC_OCR ,
  CTC_ICR }
- enum class clockSource : uint16_t {
  noClock =0 , PS_1 , PS_8 , PS_32 ,
  PS_64 , PS_128 , PS_256 , PS_1024 ,
  extern_Clock_T0_Falling_Edge , extern_Clock_T0_Rising_Edge }
- enum class timeOut : uint8_t {
  to_16ms =0 , to_32ms , to_64ms , to_125ms ,
  to_250ms , to_500ms , to_1s , to_2s ,
  to_4s , to_8s }
- enum class watchdogOperationMode : uint8_t { interrupt =1 , reset , interrupt_reset }

Functions

- constexpr uint8_t timeOutToWDTCSR (const timeOut ar_timeOut)
- constexpr uint8_t operationModeToWDTCSR (const watchdogOperationMode ar_operationMode)

7.2.1 Detailed Description

AVR chip internal core components.

The namespace englobes all internal core components that, when configured, allows the AVR MCU to internally process data

7.2.2 Enumeration Type Documentation

7.2.2.1 autoTriggerSource

```
enum class core::autoTriggerSource : uint8_t [strong]
```

Enumerator

freeRunning	free running trigger mode
analogComparator	analog comparator atrigger mode
extInterrupt	external interrupt trigger mode
timer0CompareA	timer0 compare A trigger mode
timer0Overflow	timer0 overflow trigger mode
timer1CompareB	timer0 compare B trigger mode
timer1Overflow	timer1 overflow trigger mode

timer1Capture	timer1 capture trigger mode

Definition at line 86 of file ADC.h.

```
00086                           : uint8_t {
00087     freeRunning,
00088     analogComparator,
00089     extInterrupt,
00090     timer0CompareA,
00091     timer0Overflow,
00092     timer1CompareB,
00093     timer1Overflow,
00094     timer1Capture
00095 };
```

7.2.2.2 BODMode

```
enum class core::BODMode :  uint8_t  [strong]
```

Enumerator

enabled	
disabled	

Definition at line 77 of file MCU.h.

```
00077                   : uint8_t {
00078     enabled=0,
00079     disabled,
00080 };
```

7.2.2.3 channel

```
enum class core::channel :  uint8_t  [strong]
```

Enumerator

A	
B	

Definition at line 470 of file TimerCounter.h.

```
00470                   : uint8_t {
00471     A=0,
00472     B,
00473 };
```

7.2.2.4 clockPrescaler

```
enum class core::clockPrescaler :  uint8_t  [strong]
```

Enumerator

PS_2	prescaler /2
PS_4	prescaler /4
PS_8	prescaler /8
PS_16	prescaler /16
PS_32	prescaler /32
PS_64	prescaler /64
PS_128	prescaler /128

Definition at line 69 of file ADC.h.

```
00069                           : uint8_t {
00070       PS_2 = 1,
00071       PS_4,
00072       PS_8,
00073       PS_16,
00074       PS_32,
00075       PS_64,
00076       PS_128
00077 };
```

7.2.2.5 clockSource

```
enum class core::clockSource :  uint16_t  [strong]
```

Enumerator

noClock	
PS_1	
PS_8	
PS_32	
PS_64	
PS_128	
PS_256	
PS_1024	
extern_Clock_T0_Falling_Edge	
extern_Clock_T0_Rising_Edge	

Definition at line 504 of file TimerCounter.h.

```
00504                         : uint16_t {
00505       noClock=0,
00506       PS_1,
00507       PS_8,
00508       PS_32,
00509       PS_64,
00510       PS_128,
00511       PS_256,
00512       PS_1024,
00513       extern_Clock_T0_Falling_Edge,
00514       extern_Clock_T0_Rising_Edge,
00515 };
```

7.2.2.6 compareOutputMode

```
enum class core::compareOutputMode : uint8_t [strong]
```

Enumerator

normal	
toggle	
clear	
set	

Definition at line 476 of file TimerCounter.h.

```
00476                                : uint8_t {
00477     normal=0,
00478     toggle,
00479     clear,
00480     set,
00481 };
```

7.2.2.7 operationMode

```
enum class core::operationMode : uint8_t [strong]
```

Enumerator

normal	
PWM_PC	
PWM_PC_8bit	
PWM_PC_9bit	
PWM_PC_10bit	
PWM_PFC_ICR	
PWM_PFC_OCR	
PWM_PC_ICR	
PWM_PC_OCR	
fast_PWM	
fast_PWM_8bit	
fast_PWM_9bit	
fast_PWM_10bit	
fast_PWM_ICR	
fast_PWM_OCR	
CTC_OCR	
CTC_ICR	

Definition at line 485 of file TimerCounter.h.

```
00485                             : uint8_t {
00486     normal=0,
00487     PWM_PC,
00488     PWM_PC_8bit,
00489     PWM_PC_9bit,
```

```
00490     PWM_PC_10bit,
00491     PWM_PFC_ICR,
00492     PWM_PFC_OCR,
00493     PWM_PC_ICR,
00494     PWM_PC_OCR,
00495     fast_PWM,
00496     fast_PWM_8bit,
00497     fast_PWM_9bit,
00498     fast_PWM_10bit,
00499     fast_PWM_ICR,
00500     fast_PWM_OCR,
00501     CTC_OCR,
00502     CTC_ICR,
00503 };
```

7.2.2.8 pinChangePort

```
enum class core::pinChangePort :  uint8_t  [strong]
```

Enumerator

PCINTB	
PCINTC	
PCINTD	

Definition at line 156 of file ExternInterrupt.h.

```
00156                         : uint8_t {
00157     PCINTB=0,
00158     PCINTC,
00159     PCINTD,
00160 };
```

7.2.2.9 referenceVoltage

```
enum class core::referenceVoltage :  uint8_t  [strong]
```

Enumerator

AREF	external AREF pin voltage reference, internal 1.1V voltage reference turned off
AVCC	AVCC voltage reference with external capacitor at AREF pin
internal	internal 1.1V voltage reference with external capacitor at AREF pin

Definition at line 44 of file ADC.h.

```
00044                             : uint8_t {
00045     AREF=0,
00046     AVCC,
00047     internal=3
00048 };
```

7.2.2.10 resolution

```
enum class core::resolution :  uint8_t  [strong]
```

Enumerator

res_8bit	8 bits resolution
res_9bit	9 bits resolution
res_10bit	10 bits resolution
res_11bit	11 bits resolution
res_12bit	12 bits resolution
res_13bit	13 bits resolution
res_14bit	14 bits resolution
res_15bit	15 bits resolution
res_16bit	16 bits resolution

Definition at line 24 of file ADC.h.

```
00024                       : uint8_t {
00025     res_8bit=0,
00026     res_9bit,
00027     res_10bit,
00028     res_11bit,
00029     res_12bit,
00030     res_13bit,
00031     res_14bit,
00032     res_15bit,
00033     res_16bit
00034 };
```

7.2.2.11 senseControl

```
enum class core::senseControl :  uint8_t  [strong]
```

Enumerator

lowLevel	
logicalChange	
fallingEdge	
risingEdge	

Definition at line 149 of file ExternInterrupt.h.

```
00149                         : uint8_t {
00150     lowLevel=0,
00151     logicalChange,
00152     fallingEdge,
00153     risingEdge
00154 };
```

7.2.2.12 sleepMode

```
enum class core::sleepMode :  uint8_t  [strong]
```

Enumerator

Idle	

ADC_NoiseReduction	
powerDown	
powerSave	
standby	
extendedStandby	

Definition at line 82 of file MCU.h.

```
00082                       : uint8_t {
00083     Idle=0,
00084     ADC_NoiseReduction,
00085     powerDown,
00086     powerSave,
00087     standby=6,
00088     extendedStandby,
00089 };
```

7.2.2.13 timeOut

```
enum class core::timeOut :  uint8_t  [strong]
```

Enumerator

to_16ms	
to_32ms	
to_64ms	
to_125ms	
to_250ms	
to_500ms	
to_1s	
to_2s	
to_4s	
to_8s	

Definition at line 65 of file WatchdogTimer.h.

```
00065                       : uint8_t {
00066     to_16ms=0,
00067     to_32ms,
00068     to_64ms,
00069     to_125ms,
00070     to_250ms,
00071     to_500ms,
00072     to_1s,
00073     to_2s,
00074     to_4s,
00075     to_8s,
00076 };
```

7.2.2.14 watchdogOperationMode

```
enum class core::watchdogOperationMode :  uint8_t  [strong]
```

Enumerator

interrupt	
reset	
interrupt_reset	

Definition at line 78 of file WatchdogTimer.h.

```
00078                                 : uint8_t {
00079     interrupt=1,
00080     reset,
00081     interrupt_reset,
00082 };
```

7.2.3 Function Documentation

7.2.3.1 operationModeToWDTCSR()

```
uint8_t core::operationModeToWDTCSR (
            const watchdogOperationMode ar_operationMode)  [constexpr]
```

Map a watchdog operation mode to the WDTCSR mode bits (WDIE, WDE).

Parameters

ar_operationMode	Operation mode

Returns

WDTCSR mode bits

Definition at line 103 of file WatchdogTimer.h.

```
00104 {
00105     // mode bit 0 (value 1) maps to WDIE (bit 6); mode bit 1 (value 2) maps to
00106     // WDE (bit 3), so it must be shifted left by (WDE - 1): shifting by WDE
00107     // would land on WDCE (bit 4) instead
00108     return static_cast<uint8_t>(((static_cast<uint8_t>(ar_operationMode) & 1u) << WDIE) |
00109                                 ((static_cast<uint8_t>(ar_operationMode) & 2u) << (WDE - 1)));
00110 }
```

Referenced by core::WatchdogTimer::selectTimeOut(), core::WatchdogTimer::start(), and core::WatchdogTimer::start().

7.2.3.2 timeOutToWDTCSR()

```
uint8_t core::timeOutToWDTCSR (
            const timeOut ar_timeOut)  [constexpr]
```

Map a watchdog timeout setting to the WDTCSR timeout bits (WDP3:WDP0).

The enum value uses a plain binary encoding (0..9), the WDTCSR lays the prescaler bits out as WDP3:WDP2:↩ WDP1:WDP0 = bit5:bit2:bit1:bit0.

Parameters

ar_timeOut	Timeout setting

Returns

WDTCSR timeout bits

Definition at line 92 of file WatchdogTimer.h.

```
00093 {
00094     return static_cast<uint8_t>((static_cast<uint8_t>(ar_timeOut) & 7u) |
00095                                 ((static_cast<uint8_t>(ar_timeOut) & 8u) << 2u));
00096 }
```

Referenced by core::WatchdogTimer::selectTimeOut(), core::WatchdogTimer::start(), and core::WatchdogTimer::start().

7.3 io Namespace Reference

AVR chip internal i/o components.

Classes

- struct Port

  Contains defintions of pointers to the AVR port's registers.
- class Pin

  Class for handling a Pin.
- class SPI
- class Telemetry
- class USART0

  Class for handling USART0.

Enumerations

- enum class operationMode : uint8_t { master =0 , slave , submaster , disable }
- enum class clockPrescaler : uint8_t {
  PS_4 = 0 , PS_16 , PS_64 , PS_128 ,
  PS_2 , PS_8 , PS_32 }
- enum class dataMode : uint8_t { mode_0 = 0 , mode_1 , mode_2 , mode_3 }
- enum class dataOrder : uint8_t { first_MSB = 0 , first_LSB }
- enum class TelemetryType : uint8_t { ADCSample = 0 , PinState , MotorState , CustomLog }
- enum class transmissionMode : uint8_t { async =0 , sync , masterSPI }
- enum class communicationMode : uint8_t { duplex =0 , transmit , receive }
- enum class parityMode : uint8_t { noParity =0 , evenParity , oddParity }
- enum class frameSize : uint8_t {
  eightBits =0 , fiveBits , sixBits , sevenBits ,
  neineBits }
- enum class stopBit : uint8_t { oneStopBit =0 , twoStopBits }

Variables

- const io::Port PortB = { &DDRB, &PORTB, &PINB }
- const io::Port PortC = { &DDRC, &PORTC, &PINC }
- const io::Port PortD = { &DDRD, &PORTD, &PIND }

7.3.1 Detailed Description

AVR chip internal i/o components.

The namespace englobes all internal input/output components that, when configured, allows the AVR MCU to communicate with the external World

7.3.2 Enumeration Type Documentation

7.3.2.1 clockPrescaler

```
enum class io::clockPrescaler :  uint8_t  [strong]
```

Enumerator

PS_4	
PS_16	
PS_64	
PS_128	
PS_2	
PS_8	
PS_32	

Definition at line 63 of file SPI.h.

```
00063                           : uint8_t  {
00064       PS_4 = 0,
00065       PS_16,
00066       PS_64,
00067       PS_128,
00068       PS_2,
00069       PS_8,
00070       PS_32
00071 };
```

7.3.2.2 communicationMode

```
enum class io::communicationMode :  uint8_t  [strong]
```

Enumerator

duplex	full duplex mode
transmit	transmit mode
receive	receive mode

Definition at line 39 of file USART0.h.

```
00039                               : uint8_t {
00040     duplex=0,
00041     transmit,
00042     receive,
00043
00044 };
```

7.3.2.3 dataMode

```
enum class io::dataMode :  uint8_t  [strong]
```

Enumerator

mode↩ _0	
mode↩ _1	
mode↩ _2	
mode↩ _3	

Definition at line 73 of file SPI.h.

```
00073                     : uint8_t  {
00074       mode_0 = 0,
00075       mode_1,
00076       mode_2,
00077       mode_3,
00078 };
```

7.3.2.4 dataOrder

```
enum class io::dataOrder :  uint8_t  [strong]
```

Enumerator

first_MSB	
first_LSB	

Definition at line 80 of file SPI.h.

```
00080                     : uint8_t  {
00081       first_MSB = 0,
00082       first_LSB
00083 };
```

7.3.2.5 frameSize

```
enum class io::frameSize :  uint8_t  [strong]
```

Enumerator

eightBits	8 bits frame size
fiveBits	5 bits frame size
sixBits	6 bits frame size
sevenBits	7 bits frame size
neineBits	9 bits frame size

Definition at line 68 of file USART0.h.

```
00068                       : uint8_t {
00069     eightBits=0,
00070     fiveBits,
00071     sixBits,
00072     sevenBits,
00073     neineBits
00074 };
```

7.3.2.6 operationMode

```
enum class io::operationMode :  uint8_t  [strong]
```

Enumerator

master	
slave	
submaster	
disable	

Definition at line 56 of file SPI.h.

```
00056                           : uint8_t {
00057     master=0,
00058     slave,
00059     submaster,
00060     disable,
00061 };
```

7.3.2.7 parityMode

```
enum class io::parityMode :  uint8_t  [strong]
```

Enumerator

noParity	no parity check mode
evenParity	even parity check mode
oddParity	odd parity check mode

Definition at line 54 of file USART0.h.

```
00054                       : uint8_t {
00055     noParity=0,
00056     evenParity,
00057     oddParity
00058 };
```

7.3.2.8 stopBit

```
enum class io::stopBit :  uint8_t  [strong]
```

Enumerator

oneStopBit	1 stop bit
twoStopBits	2 stop bits

Definition at line 84 of file USART0.h.

```
00084                     : uint8_t{
00085     oneStopBit=0,
00086     twoStopBits
00087 };
```

7.3.2.9 TelemetryType

```
enum class io::TelemetryType :  uint8_t  [strong]
```

Enumerator

ADCSample	
PinState	
MotorState	
CustomLog	

Definition at line 20 of file Telemetry.h.

```
00020                           : uint8_t {
00021     ADCSample = 0,
00022     PinState,
00023     MotorState,
00024     CustomLog
00025 };
```

7.3.2.10 transmissionMode

```
enum class io::transmissionMode :  uint8_t  [strong]
```

Enumerator

async	asynchronous mode
sync	synchronous mode
masterSPI	masterSPI mode

Definition at line 25 of file USART0.h.

```
00025                               : uint8_t {
00026     async=0,
00027     sync,
00028     masterSPI
00029 };
```

7.3.3 Variable Documentation

7.3.3.1 PortB

```
const io::Port io::PortB = { &DDRB, &PORTB, &PINB }  [extern]
```

global Port B object

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

- See Pin.h for a description of this code global Port B object

Definition at line 11 of file Pin.cpp.

```
00011 { &DDRB, &PORTB, &PINB };
```

Referenced by main().

7.3.3.2 PortC

```
const io::Port io::PortC = { &DDRC, &PORTC, &PINC }  [extern]
```

global Port C object

Definition at line 12 of file Pin.cpp.

```
00012 { &DDRC, &PORTC, &PINC };
```

Referenced by main().

7.3.3.3 PortD

```
const io::Port io::PortD = { &DDRD, &PORTD, &PIND }  [extern]
```

global Port D object

Definition at line 13 of file Pin.cpp.

```
00013 { &DDRD, &PORTD, &PIND };
```

Referenced by main().

7.4 utils Namespace Reference

Classes

- class PIDController

Functions

- long map (long x, long in_min, long in_max, long out_min, long out_max)

7.4.1 Function Documentation

7.4.1.1 map()

```
long utils::map (
              long x,
              long in_min,
              long in_max,
              long out_min,
              long out_max)
```

Definition at line 3 of file utils_m328p.cpp.

```
00004 {
00005   return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
00006 }
```

Referenced by main().

Chapter 8

Class Documentation

8.1 core::ADConverter Class Reference

Class for handling ADC.

`#include <ADC.h>`

Public Member Functions

- void start ()
- void stop ()
- void selectReferenceVoltage (const referenceVoltage &ar_refVoltage)
- void selectAnalogInput (io::Pin a_pin)
- void selectClockPrescaler (const clockPrescaler &ar_clockPrescaler)
- void enableConversionCompleteInterrupt (const uint8_t a_enable)
- void enableAutoTrigger (const uint8_t a_enable)
- void selectAutoTriggerSource (const autoTriggerSource &ar_autoTriggerSource)
- uint16_t getConversionResult (const resolution &ar_resolution=resolution::res_10bit)

Static Public Member Functions

- static ADConverter & getInstance (const referenceVoltage &ar_refVoltage=referenceVoltage::AVCC, const clockPrescaler &ar_clockPrescaler=clockPrescaler::PS_128, const autoTriggerSource &ar_autoTrigger↩ Source=autoTriggerSource::freeRunning, const io::Pin &ar_pin=io::Pin(0, io::PortC))
- static void conversionCompleteServiceRoutine () __asm__(STR(ADC_CONVERSION_COMPLETE_INTERRUPT)) __attribute__((__signal__

Public Attributes

- static void __used__
- static void __externally_visible__

Private Member Functions

- ADConverter (const referenceVoltage &ar_refVoltage, const clockPrescaler &ar_clockPrescaler, const autoTriggerSource &ar_autoTriggerSource, const io::Pin &ar_pin)
- ~ADConverter ()
- ADConverter (const ADConverter &)
- const ADConverter & operator= (const ADConverter &)

Static Private Attributes

- static volatile uint16_t m_conversionResult = 0
- static uint8_t m_resolution = 10

8.1.1 Detailed Description

Class for handling ADC.

The class implements ADC operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 104 of file ADC.h.

8.1.2 Constructor & Destructor Documentation

8.1.2.1 ADConverter() [1/2]

```
core::ADConverter::ADConverter (
            const referenceVoltage & ar_refVoltage,
            const clockPrescaler & ar_clockPrescaler,
            const autoTriggerSource & ar_autoTriggerSource,
            const io::Pin & ar_pin)  [private]
```

Constructor.

Initalizes the ADC object

Parameters

ar_refVoltage	Defines reference voltage
ar_clockPrescaler	Defines clock prescaler

ar_autoTriggerSource	Defines auto trigger source
ar_pin	Defines a Pin object

Definition at line 23 of file ADC.cpp.

```
00027 {
00028     core::MCU::enableADC(1);
00029     selectAnalogInput(ar_pin);
00030     selectReferenceVoltage(ar_refVoltage);
00031     selectClockPrescaler(ar_clockPrescaler);
00032     enableAutoTrigger(1);
00033     selectAutoTriggerSource(ar_autoTriggerSource);
00034     enableConversionCompleteInterrupt(1);
00035
00036
00037 }
```

References core::MCU::enableADC(), enableAutoTrigger(), enableConversionCompleteInterrupt(), selectAnalogInput(), selectAutoTriggerSource(), selectClockPrescaler(), and selectReferenceVoltage().

Referenced by ADConverter(), getInstance(), and operator=().

8.1.2.2 ~ADConverter()

```
core::ADConverter::~ADConverter ()  [private]
```

Destructor

Definition at line 40 of file ADC.cpp.

```
00041 {
00042
00043 }
```

8.1.2.3 ADConverter() [2/2]

```
core::ADConverter::ADConverter (
            const ADConverter & )  [private]
```

Overried Copy constructor

References ADConverter().

8.1.3 Member Function Documentation

8.1.3.1 conversionCompleteServiceRoutine()

```
void core::ADConverter::conversionCompleteServiceRoutine ()  [static]
```

Conversion complete ISR

Definition at line 110 of file ADC.cpp.

```
00111 {
00112
00113     static uint32_t l_resultData = 0;
00114     static uint16_t l_resultDataIndex = 0;
00115
00116
00117
```

```
00118     switch (m_resolution)
00119     {
00120         case 8:
00121         {
00122             m_conversionResult = ADC » 8;
00123             break;
00124         }
00125         case 9:
00126         {
00127             m_conversionResult = ADC » 7;
00128              break;
00129         }
00130         case 10:
00131         {
00132             m_conversionResult = ADC;
00133             break;
00134         }
00135         case 11:
00136         {
00137 
00138             if (l_resultDataIndex < 4)
00139             {
00140                 l_resultData += ADC;
00141                 l_resultDataIndex++;
00142 
00143             }
00144             else
00145             {
00146                 m_conversionResult = l_resultData » 1;
00147                 l_resultData = 0;
00148                 l_resultDataIndex = 0;
00149             }
00150 
00151             break;
00152         }
00153         case 12:
00154         {
00155             if (l_resultDataIndex < 16)
00156             {
00157                 l_resultData += ADC;
00158                 l_resultDataIndex++;
00159 
00160             }
00161             else
00162             {
00163                 m_conversionResult = l_resultData » 2;
00164                 l_resultData = 0;
00165                 l_resultDataIndex = 0;
00166             }
00167             break;
00168         }
00169         case 13:
00170         {
00171             if (l_resultDataIndex < 64)
00172             {
00173                 l_resultData += ADC;
00174                 l_resultDataIndex++;
00175 
00176             }
00177             else
00178             {
00179                 m_conversionResult = l_resultData » 3;
00180                 l_resultData = 0;
00181                 l_resultDataIndex = 0;
00182             }
00183             break;
00184         }
00185         case 14:
00186         {
00187             if (l_resultDataIndex < 256)
00188             {
00189                 l_resultData += ADC;
00190                 l_resultDataIndex++;
00191 
00192             }
00193             else
00194             {
00195                 m_conversionResult = l_resultData » 4;
00196                 l_resultData = 0;
00197                 l_resultDataIndex = 0;
00198             }
00199             break;
00200         }
00201         case 15:
00202         {
00203             if (l_resultDataIndex < 1024)
00204             {
```

```
00205                 l_resultData += ADC;
00206                 l_resultDataIndex++;
00207
00208             }
00209             else
00210             {
00211                 m_conversionResult = l_resultData » 5;
00212                 l_resultData = 0;
00213                 l_resultDataIndex = 0;
00214             }
00215             break;
00216         }
00217         case 16:
00218         {
00219             if (l_resultDataIndex < 4096)
00220             {
00221
00222                 l_resultData += ADC;
00223                 l_resultDataIndex++;
00224
00225             }
00226             else
00227             {
00228                 m_conversionResult = l_resultData » 6;
00229                 l_resultData = 0;
00230                 l_resultDataIndex = 0;
00231             }
00232             break;
00233         }
00234
00235     }
00236
00237
00238
00239
00240 }
```

References m_conversionResult, and m_resolution.

8.1.3.2 enableAutoTrigger()

```
void core::ADConverter::enableAutoTrigger (
             const uint8_t a_enable)
```

Enable autotrigger mode

Parameters

a_enable	Defines enable flag

Definition at line 74 of file ADC.cpp.

```
00075 {
00076     if (a_enable) {
00077         ADC_ENABLE_AUTOTRIGGER;
00078
00079     } else {
00080         ADC_DISABLE_AUTOTRIGGER;
00081     }
00082
00083 }
```

References ADC_DISABLE_AUTOTRIGGER, and ADC_ENABLE_AUTOTRIGGER.

Referenced by ADConverter().

8.1.3.3 enableConversionCompleteInterrupt()

```
void core::ADConverter::enableConversionCompleteInterrupt (
            const uint8_t a_enable)
```

Enable conversion complete interrupt

Parameters

a_enable	Defines enable flag

Definition at line 85 of file ADC.cpp.

```
00086 {
00087     if (a_enable) {
00088         ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT;
00089 
00090     } else {
00091         ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT;
00092     }
00093 
00094 }
```

References ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT, and ADC_ENABLE_CONVERSION_COMPLETE_INTERRU

Referenced by ADConverter().

8.1.3.4 getConversionResult()

```
uint16_t core::ADConverter::getConversionResult (
            const resolution & ar_resolution = resolution::res_10bit)
```

Get ADC conversion result

Returns

Conversion result

Definition at line 245 of file ADC.cpp.

```
00246 {
00247 
00248 
00249     switch (ar_resolution)
00250     {
00251         case core::resolution::res_8bit:
00252         {
00253             ADC_ADJUST_RESULT_LEFT;
00254             m_resolution = 8;
00255             break;
00256         }
00257         case core::resolution::res_9bit:
00258         {
00259             ADC_ADJUST_RESULT_LEFT;
00260             m_resolution = 9;
00261             break;
00262         }
00263         case core::resolution::res_10bit:
00264         {
00265             ADC_ADJUST_RESULT_RIGHT;
00266             m_resolution = 10;
00267             break;
00268         }
00269         case core::resolution::res_11bit:
00270         {
00271 
00272             ADC_ADJUST_RESULT_RIGHT;
00273             m_resolution = 11;
00274             break;
```

```
00275         }
00276         case core::resolution::res_12bit:
00277         {
00278 
00279             ADC_ADJUST_RESULT_RIGHT;
00280             m_resolution = 12;
00281             break;
00282         }
00283         case core::resolution::res_13bit:
00284         {
00285 
00286             ADC_ADJUST_RESULT_RIGHT;
00287             m_resolution = 13;
00288             break;
00289         }
00290         case core::resolution::res_14bit:
00291         {
00292 
00293             ADC_ADJUST_RESULT_RIGHT;
00294             m_resolution = 14;
00295             break;
00296         }
00297         case core::resolution::res_15bit:
00298         {
00299 
00300             ADC_ADJUST_RESULT_RIGHT;
00301             m_resolution = 15;
00302             break;
00303         }
00304         case core::resolution::res_16bit:
00305         {
00306 
00307             ADC_ADJUST_RESULT_RIGHT;
00308             m_resolution = 16;
00309             break;
00310         }
00311     }
00312 
00313     uint16_t result;
00314     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00315     {
00316         result = m_conversionResult;
00317     }
00318     return result;
00319 
00320 
00321 }
```

References ADC_ADJUST_RESULT_LEFT, ADC_ADJUST_RESULT_RIGHT, m_conversionResult, m_resolution, core::res_10bit, core::res_11bit, core::res_12bit, core::res_13bit, core::res_14bit, core::res_15bit, core::res_16bit, core::res_8bit, and core::res_9bit.

Referenced by main().

8.1.3.5 getInstance()

```
core::ADConverter & core::ADConverter::getInstance (
            const referenceVoltage & ar_refVoltage = referenceVoltage::AVCC,
            const clockPrescaler & ar_clockPrescaler = clockPrescaler::PS_128,
            const autoTriggerSource & ar_autoTriggerSource = autoTriggerSource::freeRunning,
            const io::Pin & ar_pin = io::Pin(0,io::PortC))  [static]
```

Create a single instance of the ADC object

Parameters

ar_refVoltage	Defines reference voltage
ar_clockPrescaler	Defines clock prescaler
ar_autoTriggerSource	Defines auto trigger source

ar_pin	Defines a Pin object

Definition at line 9 of file ADC.cpp.

```
00013 {
00014
00015     static ADConverter l_instance(ar_refVoltage,
00016                                   ar_clockPrescaler,
00017                                   ar_autoTriggerSource,
00018                                   ar_pin);
00019
00020     return l_instance;
00021 }
```

References ADConverter().

Referenced by main().

8.1.3.6 operator=()

```
const ADConverter & core::ADConverter::operator= (
            const ADConverter & )  [private]
```

Override assign operator

References ADConverter().

8.1.3.7 selectAnalogInput()

```
void core::ADConverter::selectAnalogInput (
            io::Pin a_pin)
```

Select analog input

Parameters

a_pin	Defines Pin object

Definition at line 52 of file ADC.cpp.

```
00053 {
00054     a_pin.toInput(0);
00055     ADC_SELECT_ANALOG_INPUT(a_pin.getPinNumber());
00056     // disables the digital input buffer of the selected pin only
00057     ADC_DISABLE_DIGITAL_INPUT_REGISTER(a_pin.getPinNumber());
00058
00059 }
```

References ADC_DISABLE_DIGITAL_INPUT_REGISTER, ADC_SELECT_ANALOG_INPUT, io::Pin::getPinNumber(), and io::Pin::toInput().

Referenced by ADConverter().

8.1.3.8 selectAutoTriggerSource()

```
void core::ADConverter::selectAutoTriggerSource (
            const autoTriggerSource & ar_autoTriggerSource)
```

Select autotrigger source

Parameters

ar_autoTriggerSource	Defines autotrigger source

Definition at line 103 of file ADC.cpp.

```
00104 {
00105     ADC_SELECT_AUTO_TRIGGER_SOURCE(static_cast<uint8_t>(ar_autoTriggerSource));
00106
00107
00108 }
```

References ADC_SELECT_AUTO_TRIGGER_SOURCE.

Referenced by ADConverter().

8.1.3.9 selectClockPrescaler()

```
void core::ADConverter::selectClockPrescaler (
            const clockPrescaler & ar_clockPrescaler)
```

Select clock prescaler

Parameters

ar_clockPrescaler	Defines clock prescaler

Definition at line 96 of file ADC.cpp.

```
00097 {
00098     ADC_SELECT_CLOCK_PRESCALER(static_cast<uint8_t>(ar_clockPrescaler));
00099
00100 }
```

References ADC_SELECT_CLOCK_PRESCALER.

Referenced by ADConverter().

8.1.3.10 selectReferenceVoltage()

```
void core::ADConverter::selectReferenceVoltage (
            const referenceVoltage & ar_refVoltage)
```

Select reference voltage

Parameters

ar_refVoltage	Defines reference voltage

Definition at line 46 of file ADC.cpp.

```
00047 {
00048     ADC_SELECT_REF_VOLTAGE(static_cast<uint8_t>(ar_refVoltage));
00049
00050 }
```

References ADC_SELECT_REF_VOLTAGE.

Referenced by ADConverter().

8.1.3.11 start()

```
void core::ADConverter::start ()
```

Start ADC conversion

Definition at line 61 of file ADC.cpp.

```
00062 {
00063     ADC_ENABLE;
00064     ADC_START_CONVERSION;
00065 }
```

References ADC_ENABLE, and ADC_START_CONVERSION.

Referenced by main().

8.1.3.12 stop()

```
void core::ADConverter::stop ()
```

Stop ADC conversion

Definition at line 67 of file ADC.cpp.

```
00068 {
00069     // Clearing ADSC does not abort an ongoing conversion, it only stops the
00070     // auto-trigger sequence, so stopping relies on clearing ADEN instead.
00071     ADC_DISABLE;
00072 }
```

References ADC_DISABLE.

8.1.4 Member Data Documentation

8.1.4.1 __externally_visible__

```
void core::ADConverter::__externally_visible__
```

Definition at line 163 of file ADC.h.

8.1.4.2 __used__

```
void core::ADConverter::__used__
```

Definition at line 163 of file ADC.h.

8.1.4.3 m_conversionResult

```
volatile uint16_t core::ADConverter::m_conversionResult = 0  [static], [private]
```

Conversion result

Definition at line 197 of file ADC.h.

Referenced by conversionCompleteServiceRoutine(), and getConversionResult().

8.1.4.4 m_resolution

```
uint8_t core::ADConverter::m_resolution = 10  [static], [private]
```

Conversion resolution

Definition at line 199 of file ADC.h.

Referenced by conversionCompleteServiceRoutine(), and getConversionResult().

8.2 component::Buzzer Class Reference

Class for handling a Buzzer component.

```
#include <Buzzer.h>
```

Public Member Functions

- Buzzer (const io::Pin &ar_pin)
- ~Buzzer ()
- void buzz (const uint16_t &ar_period_us, const uint16_t &ar_duration_us)
- void pause (uint16_t a_duration_us)
- template<typename TC>
  void buzz (TC &ar_timerCounter, const uint16_t &ar_period_ticks, uint16_t a_duration_ms, const core::channel &ar_channel=core::channel::A, const core::clockSource &ar_clockSource=core::clockSource::PS_64)

Private Attributes

- io::Pin m_pin

8.2.1 Detailed Description

Class for handling a Buzzer component.

The class implements Buzzer operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 74 of file Buzzer.h.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 Buzzer()

```
component::Buzzer::Buzzer (
            const io::Pin & ar_pin)
```

Constructor

Initalizes the Buzzer object

Parameters

ar_pin	Defines a Pin object

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

- See Buzzer.h for a description of this code

Definition at line 13 of file Buzzer.cpp.

```
00014                   : m_pin(ar_pin)
00015 {
00016     m_pin.toOutput();
00017
00018 }
```

References m_pin.

8.2.2.2 ~Buzzer()

```
component::Buzzer::~Buzzer ()
```

Destructor

Definition at line 20 of file Buzzer.cpp.

```
00021 {
00022
00023 }
```

8.2.3 Member Function Documentation

8.2.3.1 buzz() [1/2]

```
void component::Buzzer::buzz (
            const uint16_t & ar_period_us,
            const uint16_t & ar_duration_us)
```

Generate a Buzzer signal

The pin toggles once per period for the given duration, so the fundamental frequency is half of one over the period: a period of 2273 us gives 220 Hz, an octave below A4. A period of 0 returns immediately (the pitch loop steps by the period and would spin forever). A duration of 0 toggles nothing.

Parameters

ar_period_us	Defines period of Buzzer signal
ar_duration_us	Defines duration of Buzzer signal

Definition at line 25 of file Buzzer.cpp.

```
00026 {
00027     // a zero period would step the pitch loop by 0 and spin forever
00028     if (ar_period_us == 0)
00029     {
00030         return;
00031     }
00032
00033     for (uint16_t i = 0; i < ar_duration_us; i += ar_period_us)
00034     {
00035         // for loop with variable delay selects the pitch
00036         for (uint16_t j = 0; j < ar_period_us; j++)
00037         {
00038             _delay_us(1);
00039         }
00040         m_pin.toggle();
00041     }
00042     m_pin.setLow();
00043
00044 }
```

References m_pin.

Referenced by main().

8.2.3.2 buzz() [2/2]

```
template<typename TC>
void component::Buzzer::buzz (
            TC & ar_timerCounter,
            const uint16_t & ar_period_ticks,
            uint16_t a_duration_ms,
            const core::channel & ar_channel = core::channel::A,
            const core::clockSource & ar_clockSource = core::clockSource::PS_64)  [inline]
```

Generate a Buzzer signal

Template Parameters

TC	Type Defines a TimerCounter object

Parameters

ar_timerCounter	Defines the TimerCounter generating the pitch
ar_period_ticks	Defines the period of the Buzzer signal in timer ticks: the value loaded into the output compare register, not microseconds. The toggle frequency is F_CPU / (2 ∗ prescaler ∗ ticks); with the default prescaler PS_64 at F_CPU = 16 MHz, 568 ticks toggle at 220 Hz. Timer0 and Timer2 are 8-bit, so their tick value must stay below 256. The duration is taken by value, so the caller's variable keeps its value across the call.
a_duration_ms	Defines duration of the signal in ms
ar_channel	Defines TimerCounter channel
ar_clockSource	Defines TimerCounter clock source, documented for the API; the call keeps the timer's current clock source

Definition at line 133 of file Buzzer.h.

```
00139     {
00140         (void)ar_clockSource;   // start() keeps the timer's current source
00141         ar_timerCounter.selectOperationMode(core::operationMode::CTC_OCR);
00142         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::toggle);
00143         ar_timerCounter.setCounter(0);
00144         ar_timerCounter.setOutputCompareRegister(ar_channel, ar_period_ticks);
00145         // start timer
00146         ar_timerCounter.start();
00147         // wait for the pitch duration
00148         while (a_duration_ms) {
00149           _delay_ms(1);
00150           a_duration_ms--;
00151         }
00152         // no buzz
00153         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::normal);
00154         ar_timerCounter.stop();
00155     }
```

References core::A, core::CTC_OCR, core::normal, core::PS_64, and core::toggle.

8.2.3.3 pause()

```
void component::Buzzer::pause (
            uint16_t a_duration_us)
```

Generate a time break

Waits for the given duration in microseconds. pause(0) returns immediately (the decrement loop would otherwise wrap around and delay for about 65536 us).

Parameters

a_duration_us	Defines duration of time break

Definition at line 47 of file Buzzer.cpp.

```
00048 {
00049     // the decrement loop would wrap a zero around to 65535
00050     if (a_duration_us == 0)
00051     {
00052         return;
00053     }
00054     do {
00055       _delay_us(1);
00056     } while (--a_duration_us);
00057 }
```

Referenced by main().

8.2.4 Member Data Documentation

8.2.4.1 m_pin

```
io::Pin component::Buzzer::m_pin  [private]
```

Pin object

Definition at line 162 of file Buzzer.h.

Referenced by buzz(), and Buzzer().

8.3 component::DCMotor Class Reference

```
#include <DCMotor.h>
```

Public Member Functions

- DCMotor (const io::Pin &ar_pin)
- ~DCMotor ()
- void on ()
- void off ()
- void toggle ()
- template<typename TC>
  void spin (TC &ar_timerCounter, const uint16_t &ar_speed, const core::channel &ar_channel=core::channel::A)
- template<typename TC>
  void stop (TC &ar_timerCounter)
- template<typename TC>
  void connect (TC &ar_timerCounter, const core::channel &ar_channel=core::channel::A)
- template<typename TC>
  void disconnect (TC &ar_timerCounter, const core::channel &ar_channel=core::channel::A)

Private Attributes

- io::Pin m_pin

8.3.1 Detailed Description

Definition at line 126 of file DCMotor.h.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 DCMotor()

```
component::DCMotor::DCMotor (
            const io::Pin & ar_pin)
```

Definition at line 3 of file DCMotor.cpp.

```
00004         : m_pin(ar_pin)
00005 {
00006     m_pin.toOutput();
00007
00008 }
```

References m_pin.

8.3.2.2 ~DCMotor()

```
component::DCMotor::~DCMotor ()
```

Definition at line 11 of file DCMotor.cpp.

```
00012 {
00013
00014 }
```

8.3.3 Member Function Documentation

8.3.3.1 connect()

```
template<typename TC>
void component::DCMotor::connect (
            TC & ar_timerCounter,
            const core::channel & ar_channel = core::channel::A)  [inline]
```

Definition at line 164 of file DCMotor.h.

```
00166     {
00167         ar_timerCounter.selectOperationMode(core::operationMode::fast_PWM);
00168         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::clear);
00169         ar_timerCounter.setCounter(0);
00170
00171     }
```

References core::A, core::clear, and core::fast_PWM.

Referenced by main().

8.3.3.2 disconnect()

```
template<typename TC>
void component::DCMotor::disconnect (
            TC & ar_timerCounter,
            const core::channel & ar_channel = core::channel::A)  [inline]
```

Definition at line 174 of file DCMotor.h.

```
00176     {
00177         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::normal);
00178         ar_timerCounter.stop();
00179     }
```

References core::A, and core::normal.

8.3.3.3 off()

```
void component::DCMotor::off ()
```

Turn servo motor Off.

Definition at line 21 of file DCMotor.cpp.

```
00022 {
00023     m_pin.setLow();
00024 }
```

References m_pin.

8.3.3.4 on()

```
void component::DCMotor::on ()
```

Turn servo motor On.

Definition at line 16 of file DCMotor.cpp.

```
00017 {
00018     m_pin.setHigh();
00019 }
```

References m_pin.

8.3.3.5 spin()

```
template<typename TC>
void component::DCMotor::spin (
            TC & ar_timerCounter,
            const uint16_t & ar_speed,
            const core::channel & ar_channel = core::channel::A)  [inline]
```

Definition at line 144 of file DCMotor.h.

```
00148     {
00149
00150         ar_timerCounter.setOutputCompareRegister(ar_channel, ar_speed);
00151         ar_timerCounter.start();
00152
00153
00154     }
```

References core::A.

Referenced by main().

8.3.3.6 stop()

```
template<typename TC>
void component::DCMotor::stop (
             TC & ar_timerCounter)  [inline]
```

Definition at line 156 of file DCMotor.h.

```
00157     {
00158         ar_timerCounter.stop();
00159
00160
00161     }
```

8.3.3.7 toggle()

```
void component::DCMotor::toggle ()
```

Toggle servo motor state.

Definition at line 26 of file DCMotor.cpp.

```
00027 {
00028     m_pin.toggle();
00029
00030 }
```

References m_pin.

8.3.4 Member Data Documentation

8.3.4.1 m_pin

```
io::Pin component::DCMotor::m_pin  [private]
```

pin object

Definition at line 185 of file DCMotor.h.

Referenced by DCMotor(), off(), on(), and toggle().

8.4 core::ExternInterrupt Class Reference

```
#include <ExternInterrupt.h>
```

Public Member Functions

- void setInt0SenseControl (const senseControl &ar_senseControl)
- void setInt1SenseControl (const senseControl &ar_senseControl)
- void enableInt0 (const uint8_t a_enable)
- void enableInt1 (const uint8_t a_enable)
- void enablePinChange (const pinChangePort &ar_pinChangePort, const uint8_t a_enable)
- void enablePinChangeMaskPortB (const uint8_t a_pinNumber, const uint8_t a_enable)
- void enablePinChangeMaskPortC (const uint8_t a_pinNumber, const uint8_t a_enable)
- void enablePinChangeMaskPortD (const uint8_t a_pinNumber, const uint8_t a_enable)

Static Public Member Functions

- static ExternInterrupt & getInstance ()
- static void Int0ServiceRoutine () __asm__(STR(EXT_INT_INT0_INTERRUPT)) __attribute__((__signal__
- static void Int1ServiceRoutine () __asm__(STR(EXT_INT_INT1_INTERRUPT)) __attribute__((__signal__
- static void pinChangePortBServiceRoutine () __asm__(STR(EXT_INT_PIN_CHANGE_PORTB_INTERRUPT)) __attribute__((__signal__
- static void pinChangePortCServiceRoutine () __asm__(STR(EXT_INT_PIN_CHANGE_PORTC_INTERRUPT)) __attribute__((__signal__
- static void pinChangePortDServiceRoutine () __asm__(STR(EXT_INT_PIN_CHANGE_PORTD_INTERRUPT)) __attribute__((__signal__

Public Attributes

- static void __used__
- static void __externally_visible__
- static void weak

Private Member Functions

- ExternInterrupt ()
- ~ExternInterrupt ()
- ExternInterrupt (const ExternInterrupt &)
- const ExternInterrupt & operator= (const ExternInterrupt &)

8.4.1 Detailed Description

Definition at line 162 of file ExternInterrupt.h.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 ExternInterrupt() [1/2]

```
core::ExternInterrupt::ExternInterrupt ()  [private]
```

Definition at line 11 of file ExternInterrupt.cpp.

```
00012 {
00013
00014 }
```

Referenced by ExternInterrupt(), getInstance(), and operator=().

8.4.2.2 ~ExternInterrupt()

```
core::ExternInterrupt::~ExternInterrupt ()  [private]
```

Destructor.

Definition at line 16 of file ExternInterrupt.cpp.

```
00017 {
00018
00019 }
```

8.4.2.3 ExternInterrupt() [2/2]

```
core::ExternInterrupt::ExternInterrupt (
            const ExternInterrupt & )  [private]
```

Overried Copy constructor.

References ExternInterrupt().

8.4.3 Member Function Documentation

8.4.3.1 enableInt0()

```
void core::ExternInterrupt::enableInt0 (
            const uint8_t a_enable)
```

Definition at line 33 of file ExternInterrupt.cpp.

```
00034 {
00035     if (a_enable) {
00036         EXT_INT_ENABLE_INT0;
00037 
00038     } else {
00039         EXT_INT_DISABLE_INT0;
00040     }
00041 
00042 }
```

References EXT_INT_DISABLE_INT0, and EXT_INT_ENABLE_INT0.

Referenced by main().

8.4.3.2 enableInt1()

```
void core::ExternInterrupt::enableInt1 (
            const uint8_t a_enable)
```

Definition at line 44 of file ExternInterrupt.cpp.

```
00045 {
00046     if (a_enable) {
00047         EXT_INT_ENABLE_INT1;
00048 
00049     } else {
00050         EXT_INT_DISABLE_INT1;
00051     }
00052 }
```

References EXT_INT_DISABLE_INT1, and EXT_INT_ENABLE_INT1.

Referenced by main().

8.4.3.3 enablePinChange()

```
void core::ExternInterrupt::enablePinChange (
            const pinChangePort & ar_pinChangePort,
            const uint8_t a_enable)
```

Definition at line 55 of file ExternInterrupt.cpp.

```
00056 {
00057     if (a_enable) {
00058         EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(static_cast<uint8_t>(ar_pinChangePort));
00059 
00060     } else {
00061         EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(static_cast<uint8_t>(ar_pinChangePort));
00062     }
00063 
00064 }
```

References EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT, and EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT.

Referenced by main().

8.4.3.4 enablePinChangeMaskPortB()

```
void core::ExternInterrupt::enablePinChangeMaskPortB (
            const uint8_t a_pinNumber,
            const uint8_t a_enable)
```

Definition at line 66 of file ExternInterrupt.cpp.

```
00067 {
00068     if (a_enable) {
00069         EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(a_pinNumber);
00070 
00071     } else {
00072         EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(a_pinNumber);
00073     }
00074 
00075 }
```

References EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB, and EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB.

8.4.3.5 enablePinChangeMaskPortC()

```
void core::ExternInterrupt::enablePinChangeMaskPortC (
            const uint8_t a_pinNumber,
            const uint8_t a_enable)
```

Definition at line 78 of file ExternInterrupt.cpp.

```
00079 {
00080     if (a_enable) {
00081         EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(a_pinNumber);
00082 
00083     } else {
00084         EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(a_pinNumber);
00085     }
00086 
00087 }
```

References EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC, and EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC.

8.4.3.6 enablePinChangeMaskPortD()

```
void core::ExternInterrupt::enablePinChangeMaskPortD (
            const uint8_t a_pinNumber,
            const uint8_t a_enable)
```

Definition at line 89 of file ExternInterrupt.cpp.

```
00090 {
00091     if (a_enable) {
00092         EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(a_pinNumber);
00093
00094     } else {
00095         EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(a_pinNumber);
00096     }
00097 }
```

References EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD, and EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD.

8.4.3.7 getInstance()

```
core::ExternInterrupt & core::ExternInterrupt::getInstance ()  [static]
```

Definition at line 3 of file ExternInterrupt.cpp.

```
00004 {
00005     static ExternInterrupt l_instance;
00006     return l_instance;
00007
00008 }
```

References ExternInterrupt().

Referenced by main().

8.4.3.8 Int0ServiceRoutine()

```
void core::ExternInterrupt::Int0ServiceRoutine ()  [static]
```

Definition at line 102 of file ExternInterrupt.cpp.

```
00102 {}
```

8.4.3.9 Int1ServiceRoutine()

```
void core::ExternInterrupt::Int1ServiceRoutine ()  [static]
```

Definition at line 104 of file ExternInterrupt.cpp.

```
00104 {}
```

8.4.3.10 operator=()

```
const ExternInterrupt & core::ExternInterrupt::operator= (
            const ExternInterrupt & )  [private]
```

Override assign operator.

References ExternInterrupt().

8.4.3.11 pinChangePortBServiceRoutine()

```
void core::ExternInterrupt::pinChangePortBServiceRoutine ()  [static]
```

Definition at line 106 of file ExternInterrupt.cpp.

```
00106 {}
```

8.4.3.12 pinChangePortCServiceRoutine()

```
void core::ExternInterrupt::pinChangePortCServiceRoutine ()  [static]
```

Definition at line 108 of file ExternInterrupt.cpp.

```
00108 {}
```

8.4.3.13 pinChangePortDServiceRoutine()

```
void core::ExternInterrupt::pinChangePortDServiceRoutine ()  [static]
```

Definition at line 110 of file ExternInterrupt.cpp.

```
00110 {}
```

8.4.3.14 setInt0SenseControl()

```
void core::ExternInterrupt::setInt0SenseControl (
            const senseControl & ar_senseControl)
```

Definition at line 21 of file ExternInterrupt.cpp.

```
00022 {
00023     EXT_INT_SET_INT0_SENSE_CONTROL(static_cast<uint8_t>(ar_senseControl));
00024
00025 }
```

References EXT_INT_SET_INT0_SENSE_CONTROL.

8.4.3.15 setInt1SenseControl()

```
void core::ExternInterrupt::setInt1SenseControl (
            const senseControl & ar_senseControl)
```

Definition at line 27 of file ExternInterrupt.cpp.

```
00028 {
00029     EXT_INT_SET_INT1_SENSE_CONTROL(static_cast<uint8_t>(ar_senseControl));
00030
00031 }
```

References EXT_INT_SET_INT1_SENSE_CONTROL.

8.4.4 Member Data Documentation

8.4.4.1 __externally_visible__

```
void core::ExternInterrupt::__externally_visible__
```

Definition at line 185 of file ExternInterrupt.h.

8.4.4.2 __used__

```
void core::ExternInterrupt::__used__
```

Definition at line 185 of file ExternInterrupt.h.

8.4.4.3 weak

```
void core::ExternInterrupt::weak
```

Definition at line 185 of file ExternInterrupt.h.

8.5 component::LDR Class Reference

Class for handling an LDR component.

```
#include <LDR.h>
```

Public Member Functions

- LDR (const io::Pin &ar_pin, core::ADConverter &ar_ADC)
- ~LDR ()
- uint32_t getResistance (const uint16_t &ar_VDResistor)
- uint16_t getLux (const uint16_t &ar_VDResistor)

Static Public Attributes

- static const uint32_t ADC_ZERO_SENTINEL = 0xFFFFFFFFUL

Private Attributes

- io::Pin m_pin
- core::ADConverter & mr_ADC

8.5.1 Detailed Description

Class for handling an LDR component.

The class implements LDR operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 27 of file LDR.h.

8.5.2 Constructor & Destructor Documentation

8.5.2.1 LDR()

```
component::LDR::LDR (
            const io::Pin & ar_pin,
            core::ADConverter & ar_ADC)
```

Constructor

Initalizes the LDR object and selects the LDR pin as the analog input of the ADC. The pin must be one of the ADC pins (PC0..PC5 on the ATmega328P).

Parameters

ar_pin	Defines a Pin object
ar_ADC	Defines the ADC object

Definition at line 4 of file LDR.cpp.

```
00005                 : m_pin(ar_pin), mr_ADC(ar_ADC)
00006
00007 {
00008     // selects the LDR pin as the ADC input and disables its digital
00009     // input buffer
00010     mr_ADC.selectAnalogInput(m_pin);
00011 }
```

References m_pin, and mr_ADC.

8.5.2.2 ~LDR()

```
component::LDR::~LDR ()
```

Destructor

Definition at line 13 of file LDR.cpp.

```
00014 {
00015
00016 }
```

8.5.3 Member Function Documentation

8.5.3.1 getLux()

```
uint16_t component::LDR::getLux (
            const uint16_t & ar_VDResistor)
```

Compute amount of illuminance

The formula is evaluated as float once and clamped to the 0..65535 range before the cast, so a brightly lit LDR saturates the result instead of wrapping around the 16-bit return value. An ADC result of 0 is the dark end of the scale and returns 0.

Parameters

ar_VDResistor	Defines the voltage devider resistor in Kohm

Returns

Amount of illuminance, clamped to 0..65535

Definition at line 30 of file LDR.cpp.

```
00031 {
00032     uint32_t l_resistance = getResistance(ar_VDResistor);
00033     // ADC result 0 is the dark end of the scale: no illuminance
00034     if (l_resistance == ADC_ZERO_SENTINEL)
00035     {
00036         return 0;
00037     }
00038     // see:
      https://www.digikey.com/en/maker/projects/design-a-luxmeter-with-an-ldr-and-an-arduino/623aeee0f93e427bb57e02c4592567d1
00039     float l_lux = 12518931.0f * powf((float)l_resistance, -1.405f);
00040     // clamp to the uint16_t range: a brightly lit LDR gives a value far
00041     // above 65535 and the cast used to wrap around
00042     if (l_lux < 0.0f)
00043     {
00044         l_lux = 0.0f;
00045     }
00046     if (l_lux > 65535.0f)
00047     {
00048         l_lux = 65535.0f;
00049     }
00050     return (uint16_t)l_lux;
00051 }
```

References ADC_ZERO_SENTINEL, and getResistance().

Referenced by main().

8.5.3.2 getResistance()

```
uint32_t component::LDR::getResistance (
            const uint16_t & ar_VDResistor)
```

Compute LDR resistance

An ADC result of 0 (the LDR is an open circuit) is guarded: the function returns ADC_ZERO_SENTINEL instead of dividing by zero.

Parameters

ar_VDResistor	Defines the voltage devider resistor in Kohm

Returns

LDR resistance in Ohm, or ADC_ZERO_SENTINEL

Definition at line 18 of file LDR.cpp.

```
00019 {
00020     uint16_t l_result = mr_ADC.getConversionResult(core::resolution::res_10bit);
00021     // an ADC result of 0 would divide by zero: report it as a sentinel
00022     if (l_result == 0)
00023     {
00024         return ADC_ZERO_SENTINEL;
00025     }
00026     // 10bit resolution -> 1023
00027     return (uint32_t)ar_VDResistor * ((1000UL * 1023UL / l_result) - 1000UL);
00028 }
```

References ADC_ZERO_SENTINEL, mr_ADC, and core::res_10bit.

Referenced by getLux().

8.5.4 Member Data Documentation

8.5.4.1 ADC_ZERO_SENTINEL

```
const uint32_t component::LDR::ADC_ZERO_SENTINEL = 0xFFFFFFFFUL  [static]
```

Value returned by getResistance() when the ADC reads 0: the voltage divider formula would divide by zero, so callers can test for this sentinel instead of crashing.

Definition at line 36 of file LDR.h.

Referenced by getLux(), and getResistance().

8.5.4.2 m_pin

```
io::Pin component::LDR::m_pin  [private]
```

Pin object

Definition at line 81 of file LDR.h.

Referenced by LDR().

8.5.4.3 mr_ADC

```
core::ADConverter& component::LDR::mr_ADC  [private]
```

ADC object reference

Definition at line 82 of file LDR.h.

Referenced by getResistance(), and LDR().

8.6 component::Led Class Reference

Class for handling a Led component.

```
#include <Led.h>
```

Public Member Functions

- Led (const io::Pin &ar_pin)
- ~Led ()
- void on ()
- void off ()
- void toggle ()
- uint8_t isOn ()
- uint8_t isOff ()

Private Attributes

- io::Pin m_pin

8.6.1 Detailed Description

Class for handling a Led component.

The class implements Led operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 26 of file Led.h.

8.6.2 Constructor & Destructor Documentation

8.6.2.1 Led()

```
component::Led::Led (
            const io::Pin & ar_pin)
```

Constructor

Initalizes the Led object

Parameters

ar_pin	Defines a Pin object

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

- See Led.h for a description of this code

Definition at line 12 of file Led.cpp.

```
00013          : m_pin(ar_pin)
00014 {
00015     m_pin.toOutput();
00016 }
```

References m_pin.

8.6.2.2 ~Led()

```
component::Led::~Led ()
```

Destructor

Definition at line 19 of file Led.cpp.

```
00020 {
00021
00022 }
```

8.6.3 Member Function Documentation

8.6.3.1 isOff()

```
uint8_t component::Led::isOff ()
```

Is Led Off

Returns

Led status

Definition at line 51 of file Led.cpp.

```
00052 {
00053     return m_pin.isLow();
00054
00055 }
```

References m_pin.

8.6.3.2 isOn()

```
uint8_t component::Led::isOn ()
```

Is Led On

Returns

Led status

Definition at line 44 of file Led.cpp.

```
00045 {
00046     return m_pin.isHigh();
00047
00048 }
```

References m_pin.

8.6.3.3 off()

```
void component::Led::off ()
```

Turn Led Off

Definition at line 31 of file Led.cpp.

```
00032 {
00033     m_pin.setLow();
00034
00035 }
```

References m_pin.

Referenced by main().

8.6.3.4 on()

```
void component::Led::on ()
```

Turn Led On

Definition at line 25 of file Led.cpp.

```
00026 {
00027     m_pin.setHigh();
00028
00029 }
```

References m_pin.

Referenced by main().

8.6.3.5 toggle()

```
void component::Led::toggle ()
```

Toggle Led state

Definition at line 38 of file Led.cpp.

```
00039 {
00040     m_pin.toggle();
00041
00042 }
```

References m_pin.

Referenced by main().

8.6.4 Member Data Documentation

8.6.4.1 m_pin

```
io::Pin component::Led::m_pin  [private]
```

Pin object

Definition at line 68 of file Led.h.

Referenced by isOff(), isOn(), Led(), off(), on(), and toggle().

8.7 core::MCU Class Reference

`#include <MCU.h>`

Static Public Member Functions

- static void init ()
- static void enableInterrupts ()
- static void selectSleepMode (const sleepMode &a_sleepMode)
- static void goToSleep (const BODMode &a_BODMode)
- static void sleepEnable (const uint8_t ar_enable)
- static void enableUSART0 (const uint8_t a_enable)
- static void enableTimerCounter0 (const uint8_t a_enable)
- static void enableTimerCounter1 (const uint8_t a_enable)
- static void enableTimerCounter2 (const uint8_t a_enable)
- static void enableTWI (const uint8_t a_enable)
- static void enableSPI (const uint8_t a_enable)
- static void enableADC (const uint8_t a_enable)

8.7.1 Detailed Description

Definition at line 91 of file MCU.h.

8.7.2 Member Function Documentation

8.7.2.1 enableADC()

```
void core::MCU::enableADC (
            const uint8_t a_enable)  [static]
```

Definition at line 133 of file MCU.cpp.

```
00134 {
00135     if (a_enable) {
00136         MCU_ADC_ENABLE;
00137     } else {
00138         MCU_ADC_DISABLE;
00139     }
00140 }
```

References MCU_ADC_DISABLE, and MCU_ADC_ENABLE.

Referenced by core::ADConverter::ADConverter(), and init().

8.7.2.2 enableInterrupts()

```
void core::MCU::enableInterrupts ()  [static]
```

Enable interrupts globally.

Applications call this once after the peripherals that need interrupts (USART0, SPI, ADC, ExternInterrupt, WatchdogTimer) have been created and configured. The peripheral constructors used to call sei() themselves, which silently enabled interrupts early.

Definition at line 5 of file MCU.cpp.

```
00006 {
00007     sei();
00008
00009 }
```

Referenced by main().

8.7.2.3 enableSPI()

```
void core::MCU::enableSPI (
            const uint8_t a_enable)  [static]
```

Definition at line 123 of file MCU.cpp.

```
00124 {
00125     if (a_enable) {
00126         MCU_SPI_ENABLE;
00127     } else {
00128         MCU_SPI_DISABLE;
00129     }
00130
00131 }
```

References MCU_SPI_DISABLE, and MCU_SPI_ENABLE.

Referenced by init(), and io::SPI::SPI().

8.7.2.4 enableTimerCounter0()

```
void core::MCU::enableTimerCounter0 (
            const uint8_t a_enable)  [static]
```

Definition at line 83 of file MCU.cpp.

```
00084 {
00085     if (a_enable) {
00086         MCU_TIMER0_ENABLE;
00087     } else {
00088         MCU_TIMER0_DISABLE;
00089     }
00090
00091 }
```

References MCU_TIMER0_DISABLE, and MCU_TIMER0_ENABLE.

Referenced by init(), and core::TimerCounter0::TimerCounter0().

8.7.2.5 enableTimerCounter1()

```
void core::MCU::enableTimerCounter1 (
            const uint8_t a_enable)  [static]
```

Definition at line 93 of file MCU.cpp.

```
00094 {
00095     if (a_enable) {
00096         MCU_TIMER1_ENABLE;
00097     } else {
00098         MCU_TIMER1_DISABLE;
00099     }
00100
00101 }
```

References MCU_TIMER1_DISABLE, and MCU_TIMER1_ENABLE.

Referenced by init(), and core::TimerCounter1::TimerCounter1().

8.7.2.6 enableTimerCounter2()

```
void core::MCU::enableTimerCounter2 (
            const uint8_t a_enable)  [static]
```

Definition at line 103 of file MCU.cpp.

```
00104 {
00105     if (a_enable) {
00106         MCU_TIMER2_ENABLE;
00107     } else {
00108         MCU_TIMER2_DISABLE;
00109     }
00110
00111 }
```

References MCU_TIMER2_DISABLE, and MCU_TIMER2_ENABLE.

Referenced by init(), and core::TimerCounter2::TimerCounter2().

8.7.2.7 enableTWI()

```
void core::MCU::enableTWI (
            const uint8_t a_enable)  [static]
```

Definition at line 113 of file MCU.cpp.

```
00114 {
00115     if (a_enable) {
00116         MCU_TWI_ENABLE;
00117     } else {
00118         MCU_TWI_DISABLE;
00119     }
00120
00121 }
```

References MCU_TWI_DISABLE, and MCU_TWI_ENABLE.

Referenced by init(), and core::TWI::TWI().

8.7.2.8 enableUSART0()

```
void core::MCU::enableUSART0 (
            const uint8_t a_enable)  [static]
```

Definition at line 73 of file MCU.cpp.

```
00074 {
00075     if (a_enable) {
00076         MCU_USART0_ENABLE;
00077     } else {
00078         MCU_USART0_DISABLE;
00079     }
00080
00081 }
```

References MCU_USART0_DISABLE, and MCU_USART0_ENABLE.

Referenced by init(), and io::USART0::USART0().

8.7.2.9 goToSleep()

```
void core::MCU::goToSleep (
            const BODMode & a_BODMode)  [static]
```

Enter sleep until an interrupt wakes the chip.

The sleep instruction must run with interrupts enabled, otherwise the chip never wakes up. In the BOD-disabled mode the timed avr-libc sleep_bod_disable() sequence runs immediately before sleep_cpu(), as the datasheet requires.

Parameters

a_BODMode	Defines whether the brown-out detector stays on

Definition at line 29 of file MCU.cpp.

```
00030 {
00031     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00032     {
00033         switch (a_BODMode)
00034         {
00035             case core::BODMode::enabled:
00036             {
00037                 sleepEnable(1);
00038                 // the sleep instruction must run with interrupts enabled so
00039                 // that the chip can wake up again
00040                 sei();
00041                 sleep_cpu();
00042                 sleepEnable(0);
00043                 break;
00044             }
00045             case core::BODMode::disabled:
00046             {
00047                 sleepEnable(1);
00048                 // the sleep instruction must run with interrupts enabled so
00049                 // that the chip can wake up again
00050                 sei();
00051                 // the timed BOD disable sequence must immediately precede
00052                 // sleep_cpu(), an interrupt in between would break it
00053                 sleep_bod_disable();
00054                 sleep_cpu();
00055                 sleepEnable(0);
00056                 break;
00057             }
00058         }
00059     }
00060
00061 }
```

References core::disabled, core::enabled, and sleepEnable().

8.7.2.10 init()

```
void core::MCU::init ()  [static]
```

Definition at line 11 of file MCU.cpp.

```
00012 {
00013     enableUSART0(0);
00014     enableTimerCounter0(0);
00015     enableTimerCounter1(0);
00016     enableTimerCounter2(0);
00017     enableTWI(0);
00018     enableSPI(0);
00019     enableADC(0);
00020
00021 }
```

References enableADC(), enableSPI(), enableTimerCounter0(), enableTimerCounter1(), enableTimerCounter2(), enableTWI(), and enableUSART0().

Referenced by main().

8.7.2.11 selectSleepMode()

```
void core::MCU::selectSleepMode (
            const sleepMode & a_sleepMode)  [static]
```

Definition at line 23 of file MCU.cpp.

```
00024 {
00025     MCU_SELECT_SLEEP_MODE(static_cast<uint8_t>(a_sleepMode));
00026
00027 }
```

References MCU_SELECT_SLEEP_MODE.

8.7.2.12 sleepEnable()

```
void core::MCU::sleepEnable (
            const uint8_t ar_enable)  [static]
```

Definition at line 63 of file MCU.cpp.

```
00064 {
00065     if (a_enable) {
00066         MCU_SLEEP_ENABLE;
00067     } else {
00068         MCU_SLEEP_DISABLE;
00069     }
00070
00071 }
```

References MCU_SLEEP_DISABLE, and MCU_SLEEP_ENABLE.

Referenced by goToSleep().

8.8 component::PCF8574_LCD Class Reference

```
#include <PCF8574_LCD.h>
```

Public Member Functions

- PCF8574_LCD (uint8_t a_i2cAddress=0x27, uint8_t a_cols=16, uint8_t a_rows=2)
- void init ()
- void clear ()
- void setCursor (uint8_t a_col, uint8_t a_row)
- void print (const char *ap_str)
- void setBacklight (uint8_t a_enable)

Private Member Functions

- void sendNibble (uint8_t a_nibble, uint8_t a_mode)
- void sendByte (uint8_t a_byte, uint8_t a_mode)

Private Attributes

- uint8_t m_i2cAddr
- uint8_t m_cols
- uint8_t m_rows
- uint8_t m_backlightMask
- core::TWI & mr_twi

8.8.1 Detailed Description

Definition at line 18 of file PCF8574_LCD.h.

8.8.2 Constructor & Destructor Documentation

8.8.2.1 PCF8574_LCD()

```
component::PCF8574_LCD::PCF8574_LCD (
            uint8_t a_i2cAddress = 0x27,
            uint8_t a_cols = 16,
            uint8_t a_rows = 2)
```

Constructor

Parameters

a_i2cAddress	I2C 7-bit bus address (default 0x27 or 0x3F)
a_cols	Number of display columns (e.g. 16 or 20)
a_rows	Number of display rows (e.g. 2 or 4)

Definition at line 15 of file PCF8574_LCD.cpp.

```
00016     : m_i2cAddr(a_i2cAddress),
00017       m_cols(a_cols),
00018       m_rows(a_rows),
00019       m_backlightMask(LCD_BACKLIGHT),
00020       mr_twi(core::TWI::getInstance(100000UL))
00021 {
00022 }
```

References LCD_BACKLIGHT, m_backlightMask, m_cols, m_i2cAddr, m_rows, and mr_twi.

8.8.3 Member Function Documentation

8.8.3.1 clear()

```
void component::PCF8574_LCD::clear ()
```

Clear LCD screen and reset cursor

Definition at line 41 of file PCF8574_LCD.cpp.

```
00042 {
00043     sendByte(0x01, 0);
00044     _delay_ms(2);
00045 }
```

References sendByte().

Referenced by init(), and main().

8.8.3.2 init()

```
void component::PCF8574_LCD::init ()
```

Initialize LCD in 4-bit mode over I2C

Definition at line 24 of file PCF8574_LCD.cpp.

```
00025 {
00026     _delay_ms(50);
00027     // Sequence to initialize HD44780 in 4-bit mode over PCF8574
00028     sendNibble(0x03, 0);
00029     _delay_ms(5);
00030     sendNibble(0x03, 0);
00031     _delay_us(150);
00032     sendNibble(0x03, 0);
00033     sendNibble(0x02, 0); // 4-bit mode
00034
00035     sendByte(0x28, 0); // 2 lines, 5x8 font
00036     sendByte(0x0C, 0); // Display ON, Cursor OFF
00037     clear();
00038     sendByte(0x06, 0); // Entry mode auto increment
00039 }
```

References clear(), sendByte(), and sendNibble().

Referenced by main().

8.8.3.3 print()

```
void component::PCF8574_LCD::print (
            const char * ap_str)
```

Print string to current cursor position

Parameters

ap_str	Null-terminated text string

Definition at line 54 of file PCF8574_LCD.cpp.

```
00055 {
00056     if (!ap_str) return;
00057     while (*ap_str) {
00058         sendByte(static_cast<uint8_t>(*ap_str++), LCD_RS_BIT);
00059     }
00060 }
```

References LCD_RS_BIT, and sendByte().

Referenced by main().

8.8.3.4 sendByte()

```
void component::PCF8574_LCD::sendByte (
            uint8_t a_byte,
            uint8_t a_mode)  [private]
```

Definition at line 80 of file PCF8574_LCD.cpp.

```
00081 {
00082     sendNibble(a_byte » 4, a_mode);
00083     sendNibble(a_byte & 0x0F, a_mode);
00084 }
```

References sendNibble().

Referenced by clear(), init(), print(), and setCursor().

8.8.3.5 sendNibble()

```
void component::PCF8574_LCD::sendNibble (
            uint8_t a_nibble,
            uint8_t a_mode)  [private]
```

Definition at line 68 of file PCF8574_LCD.cpp.

```
00069 {
00070     uint8_t payload = (a_nibble << 4) | a_mode | m_backlightMask;
00071     mr_twi.start();
00072     mr_twi.write(m_i2cAddr << 1);
00073     mr_twi.write(payload | LCD_ENABLE_BIT);
00074     _delay_us(1);
00075     mr_twi.write(payload & ~LCD_ENABLE_BIT);
00076     _delay_us(50);
00077     mr_twi.stop();
00078 }
```

References LCD_ENABLE_BIT, m_backlightMask, m_i2cAddr, and mr_twi.

Referenced by init(), sendByte(), and setBacklight().

8.8.3.6 setBacklight()

```
void component::PCF8574_LCD::setBacklight (
            uint8_t a_enable)
```

Toggle backlight on/off

Definition at line 62 of file PCF8574_LCD.cpp.

```
00063 {
00064     m_backlightMask = a_enable ? LCD_BACKLIGHT : 0x00;
00065     sendNibble(0x00, 0);
00066 }
```

References LCD_BACKLIGHT, m_backlightMask, and sendNibble().

8.8.3.7 setCursor()

```
void component::PCF8574_LCD::setCursor (
            uint8_t a_col,
            uint8_t a_row)
```

Set cursor position

Parameters

a_col	Column index (0..cols-1)
a_row	Row index (0..rows-1)

Definition at line 47 of file PCF8574_LCD.cpp.

```
00048 {
00049     static const uint8_t row_offsets[] = { 0x00, 0x40, 0x14, 0x54 };
00050     if (a_row >= m_rows) a_row = m_rows - 1;
00051     sendByte(0x80 | (a_col + row_offsets[a_row]), 0);
00052 }
```

References m_rows, and sendByte().

Referenced by main().

8.8.4 Member Data Documentation

8.8.4.1 m_backlightMask

```
uint8_t component::PCF8574_LCD::m_backlightMask  [private]
```

Definition at line 61 of file PCF8574_LCD.h.

Referenced by PCF8574_LCD(), sendNibble(), and setBacklight().

8.8.4.2 m_cols

```
uint8_t component::PCF8574_LCD::m_cols  [private]
```

Definition at line 59 of file PCF8574_LCD.h.

Referenced by PCF8574_LCD().

8.8.4.3 m_i2cAddr

```
uint8_t component::PCF8574_LCD::m_i2cAddr  [private]
```

Definition at line 58 of file PCF8574_LCD.h.

Referenced by PCF8574_LCD(), and sendNibble().

8.8.4.4 m_rows

```
uint8_t component::PCF8574_LCD::m_rows  [private]
```

Definition at line 60 of file PCF8574_LCD.h.

Referenced by PCF8574_LCD(), and setCursor().

8.8.4.5 mr_twi

```
core::TWI& component::PCF8574_LCD::mr_twi  [private]
```

Definition at line 62 of file PCF8574_LCD.h.

Referenced by PCF8574_LCD(), and sendNibble().

8.9 utils::PIDController Class Reference

```
#include <PIDController.h>
```

Public Member Functions

- PIDController (float a_kp, float a_ki, float a_kd, float a_minOutput=-255.0f, float a_maxOutput=255.0f)
- void setGains (float a_kp, float a_ki, float a_kd)
- void setOutputLimits (float a_minOutput, float a_maxOutput)
- void reset ()
- float compute (float a_setpoint, float a_input, float a_dt_s)

Private Attributes

- float m_kp
- float m_ki
- float m_kd
- float m_minOutput
- float m_maxOutput
- float m_integral
- float m_prevInput
- uint8_t m_firstRun

8.9.1 Detailed Description

Definition at line 17 of file PIDController.h.

8.9.2 Constructor & Destructor Documentation

8.9.2.1 PIDController()

```
utils::PIDController::PIDController (
            float a_kp,
            float a_ki,
            float a_kd,
            float a_minOutput = -255.0f,
            float a_maxOutput = 255.0f)
```

Constructor

Parameters

a_kp	Proportional gain
a_ki	Integral gain
a_kd	Derivative gain
a_minOutput	Minimum output limit
a_maxOutput	Maximum output limit

Definition at line 12 of file PIDController.cpp.

```
00013     : m_integral(0.0f),
00014       m_prevInput(0.0f),
00015       m_firstRun(1)
00016 {
00017     setGains(a_kp, a_ki, a_kd);
00018     setOutputLimits(a_minOutput, a_maxOutput);
00019 }
```

References m_firstRun, m_integral, m_prevInput, setGains(), and setOutputLimits().

8.9.3 Member Function Documentation

8.9.3.1 compute()

```
float utils::PIDController::compute (
            float a_setpoint,
            float a_input,
            float a_dt_s)
```

Compute PID output step

Parameters

a_setpoint	Desired target value
a_input	Measured feedback input
a_dt_s	Sample time in seconds

Returns

Calculated PID output clamped to limits

Definition at line 50 of file PIDController.cpp.

```
00051 {
00052     if (isnan(a_setpoint) || isinf(a_setpoint) || isnan(a_input) || isinf(a_input) || isnan(a_dt_s) ||
      isinf(a_dt_s) || a_dt_s <= 0.0f) {
00053         return 0.0f;
00054     }
00055
00056     if (m_firstRun) {
00057         m_prevInput = a_input;
00058         m_firstRun = 0;
00059     }
00060
00061     float error = a_setpoint - a_input;
00062
00063     // Proportional term
00064     float p_out = m_kp * error;
00065
00066     // Integral term
00067     m_integral += error * a_dt_s;
00068     float i_out = m_ki * m_integral;
00069
00070     // Derivative on measurement (prevents derivative kick on setpoint change)
00071     float d_input = (a_input - m_prevInput) / a_dt_s;
00072     float d_out = -m_kd * d_input;
00073
00074     float total = p_out + i_out + d_out;
00075
00076     // Anti-windup clamping
00077     if (total > m_maxOutput) {
00078         total = m_maxOutput;
00079         m_integral -= error * a_dt_s; // freeze integration
00080     } else if (total < m_minOutput) {
00081         total = m_minOutput;
00082         m_integral -= error * a_dt_s;
00083     }
00084
00085     m_prevInput = a_input;
00086     return total;
00087 }
```

References m_firstRun, m_integral, m_kd, m_ki, m_kp, m_maxOutput, m_minOutput, and m_prevInput.

Referenced by main().

8.9.3.2 reset()

```
void utils::PIDController::reset ()
```

Reset integral accumulator and previous input state

Definition at line 43 of file PIDController.cpp.

```
00044 {
00045     m_integral = 0.0f;
00046     m_prevInput = 0.0f;
00047     m_firstRun = 1;
00048 }
```

References m_firstRun, m_integral, and m_prevInput.

8.9.3.3 setGains()

```
void utils::PIDController::setGains (
            float a_kp,
            float a_ki,
            float a_kd)
```

Set PID gains

Definition at line 21 of file PIDController.cpp.

```
00022 {
00023     m_kp = (isnan(a_kp) || isinf(a_kp)) ? 0.0f : a_kp;
00024     m_ki = (isnan(a_ki) || isinf(a_ki)) ? 0.0f : a_ki;
00025     m_kd = (isnan(a_kd) || isinf(a_kd)) ? 0.0f : a_kd;
00026 }
```

References m_kd, m_ki, and m_kp.

Referenced by PIDController().

8.9.3.4 setOutputLimits()

```
void utils::PIDController::setOutputLimits (
            float a_minOutput,
            float a_maxOutput)
```

Set output saturation limits

Definition at line 28 of file PIDController.cpp.

```
00029 {
00030     if (isnan(a_minOutput) || isinf(a_minOutput)) a_minOutput = -255.0f;
00031     if (isnan(a_maxOutput) || isinf(a_maxOutput)) a_maxOutput = 255.0f;
00032
00033     if (a_minOutput > a_maxOutput) {
00034         float tmp = a_minOutput;
00035         a_minOutput = a_maxOutput;
00036         a_maxOutput = tmp;
00037     }
00038
00039     m_minOutput = a_minOutput;
00040     m_maxOutput = a_maxOutput;
00041 }
```

References m_maxOutput, and m_minOutput.

Referenced by PIDController().

8.9.4 Member Data Documentation

8.9.4.1 m_firstRun

```
uint8_t utils::PIDController::m_firstRun  [private]
```

Definition at line 59 of file PIDController.h.

Referenced by compute(), PIDController(), and reset().

8.9.4.2 m_integral

```
float utils::PIDController::m_integral  [private]
```

Definition at line 57 of file PIDController.h.

Referenced by compute(), PIDController(), and reset().

8.9.4.3 m_kd

```
float utils::PIDController::m_kd  [private]
```

Definition at line 54 of file PIDController.h.

Referenced by compute(), and setGains().

8.9.4.4 m_ki

```
float utils::PIDController::m_ki  [private]
```

Definition at line 53 of file PIDController.h.

Referenced by compute(), and setGains().

8.9.4.5 m_kp

```
float utils::PIDController::m_kp  [private]
```

Definition at line 52 of file PIDController.h.

Referenced by compute(), and setGains().

8.9.4.6 m_maxOutput

```
float utils::PIDController::m_maxOutput  [private]
```

Definition at line 56 of file PIDController.h.

Referenced by compute(), and setOutputLimits().

8.9.4.7 m_minOutput

```
float utils::PIDController::m_minOutput  [private]
```

Definition at line 55 of file PIDController.h.

Referenced by compute(), and setOutputLimits().

8.9.4.8 m_prevInput

```
float utils::PIDController::m_prevInput  [private]
```

Definition at line 58 of file PIDController.h.

Referenced by compute(), PIDController(), and reset().

8.10 io::Pin Class Reference

Class for handling a Pin.

```
#include <Pin.h>
```

Public Member Functions

- Pin (const uint8_t a_pinNumber, const Port &ar_portName)
- ~Pin ()
- void toOutput ()
- void toInput (const uint8_t &ar_useInternalPullUp)
- void setLow ()
- void setHigh ()
- void toggle ()
- uint8_t isHigh ()
- uint8_t isLow ()
- uint8_t getPinNumber ()

Private Attributes

- const Port & mr_portName
- const uint8_t m_pinNumber

8.10.1 Detailed Description

Class for handling a Pin.

The class implements Pin operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 35 of file Pin.h.

8.10.2 Constructor & Destructor Documentation

8.10.2.1 Pin()

```
io::Pin::Pin (
            const uint8_t a_pinNumber,
            const Port & ar_portName)
```

Constructor

Initalizes the pin object

Parameters

ar_portName	Defines the Port name in the AVR chip
a_pinNumber	Defines the Pin number in the AVR chip

Definition at line 15 of file Pin.cpp.

```
00016           : mr_portName(ar_portName), m_pinNumber(a_pinNumber)
00017 {
00018
00019 }
```

References m_pinNumber, and mr_portName.

8.10.2.2 ~Pin()

```
io::Pin::~Pin ()
```

Destructor

Definition at line 21 of file Pin.cpp.

```
00023 {
00024
00025 }
```

8.10.3 Member Function Documentation

8.10.3.1 getPinNumber()

```
uint8_t io::Pin::getPinNumber ()
```

Get Pin number

Returns

Pin number

Definition at line 83 of file Pin.cpp.

```
00084 {
00085     return m_pinNumber;
00086 }
```

References m_pinNumber.

Referenced by core::ADConverter::selectAnalogInput().

8.10.3.2 isHigh()

```
uint8_t io::Pin::isHigh ()
```

Check if Pin is logic high

Returns

1 if the pin reads high, 0 otherwise

Definition at line 73 of file Pin.cpp.

```
00074 {
00075     return (*mr_portName.mp_pinReg & (1 << m_pinNumber)) ? 1 : 0;
00076 }
```

References m_pinNumber, and mr_portName.

8.10.3.3 isLow()

```
uint8_t io::Pin::isLow ()
```

Check if Pin is logic low

Returns

1 if the pin reads low, 0 otherwise

Definition at line 78 of file Pin.cpp.

```
00079 {
00080     return (*mr_portName.mp_pinReg & (1 << m_pinNumber)) ? 0 : 1;
00081 }
```

References m_pinNumber, and mr_portName.

8.10.3.4 setHigh()

```
void io::Pin::setHigh ()
```

Set Pin to logic high

Note: this is a no-op while the pin is configured as an input, so the pull-up state of an input pin is not changed here; use toInput() to select the pull-up.

Definition at line 57 of file Pin.cpp.

```
00058 {
00059     if (*mr_portName.mp_ddrReg & (1 << m_pinNumber))
00060     {
00061         *mr_portName.mp_portReg |=  (1 << m_pinNumber);
00062     }
00063 }
```

References m_pinNumber, and mr_portName.

8.10.3.5 setLow()

```
void io::Pin::setLow ()
```

Set Pin to logic low

Note: this is a no-op while the pin is configured as an input, so the pull-up state of an input pin is not changed here; use toInput() to select the pull-up.

Definition at line 49 of file Pin.cpp.

```
00050 {
00051     if (*mr_portName.mp_ddrReg & (1 « m_pinNumber))
00052     {
00053         *mr_portName.mp_portReg &= ~(1 « m_pinNumber);
00054     }
00055 }
```

References m_pinNumber, and mr_portName.

8.10.3.6 toggle()

```
void io::Pin::toggle ()
```

Toggle Pin state

Definition at line 65 of file Pin.cpp.

```
00066 {
00067     if (*mr_portName.mp_ddrReg & (1 « m_pinNumber))
00068     {
00069         *mr_portName.mp_portReg ^=   1 « m_pinNumber;
00070     }
00071 }
```

References m_pinNumber, and mr_portName.

8.10.3.7 toInput()

```
void io::Pin::toInput (
            const uint8_t & ar_useInternalPullUp)
```

Configures Pin to input

Parameters

ar_useInternalPullUp	Indicates if internal pull up resistor is used

Definition at line 32 of file Pin.cpp.

```
00033 {
00034     if (ar_useInternalPullUp)
00035     {
00036         *mr_portName.mp_portReg |= (1 « m_pinNumber);
00037         *mr_portName.mp_ddrReg  &= ~(1 « m_pinNumber);
00038
00039
00040     }
00041     else
00042     {
00043         *mr_portName.mp_portReg &= ~(1 « m_pinNumber);
00044         *mr_portName.mp_ddrReg  &= ~(1 « m_pinNumber);
00045     }
00046
00047 }
```

References m_pinNumber, and mr_portName.

Referenced by core::ADConverter::selectAnalogInput().

8.10.3.8 toOutput()

```
void io::Pin::toOutput ()
```

Configures Pin to output

Definition at line 27 of file Pin.cpp.

```
00028 {
00029     *mr_portName.mp_ddrReg  |=  (1 << m_pinNumber);
00030 }
```

References m_pinNumber, and mr_portName.

8.10.4 Member Data Documentation

8.10.4.1 m_pinNumber

```
const uint8_t io::Pin::m_pinNumber  [private]
```

Pin number

Definition at line 101 of file Pin.h.

Referenced by getPinNumber(), isHigh(), isLow(), Pin(), setHigh(), setLow(), toggle(), toInput(), and toOutput().

8.10.4.2 mr_portName

```
const Port& io::Pin::mr_portName  [private]
```

Port object reference

Definition at line 100 of file Pin.h.

Referenced by isHigh(), isLow(), Pin(), setHigh(), setLow(), toggle(), toInput(), and toOutput().

8.11 io::Port Struct Reference

Contains defintions of pointers to the AVR port's registers.

```
#include <Pin.h>
```

Public Attributes

- volatile uint8_t * mp_ddrReg
- volatile uint8_t * mp_portReg
- volatile uint8_t * mp_pinReg

8.11.1 Detailed Description

Contains defintions of pointers to the AVR port's registers.

Definition at line 19 of file Pin.h.

8.11.2 Member Data Documentation

8.11.2.1 mp_ddrReg

```
volatile uint8_t* io::Port::mp_ddrReg
```

Pointer to the data direction register

Definition at line 21 of file Pin.h.

8.11.2.2 mp_pinReg

```
volatile uint8_t* io::Port::mp_pinReg
```

Pointer to the pin register

Definition at line 25 of file Pin.h.

8.11.2.3 mp_portReg

```
volatile uint8_t* io::Port::mp_portReg
```

Pointer to the port register

Definition at line 23 of file Pin.h.

8.12 component::PushButton Class Reference

Class for handling a PushButton component.

```
#include <PushButton.h>
```

Public Member Functions

- PushButton (const io::Pin &ar_pin, uint8_t ar_useInternalPullUp=1, uint8_t ar_isActiveLow=1)
- ~PushButton ()
- uint8_t isPressed ()
- uint8_t getPressedCount () const
- void resetPressedCount ()

Private Attributes

- io::Pin m_pin
- uint8_t m_isActiveLow
- uint8_t m_useInternalPullUp
- uint8_t m_lastState
- uint8_t m_buttonPressed

8.12.1 Detailed Description

Class for handling a PushButton component.

The class implements PushButton operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 26 of file PushButton.h.

8.12.2 Constructor & Destructor Documentation

8.12.2.1 PushButton()

```
component::PushButton::PushButton (
            const io::Pin & ar_pin,
            uint8_t ar_useInternalPullUp = 1,
            uint8_t ar_isActiveLow = 1)
```

Constructor.

Initializes The PushButton object

The configuration is copied into the object: storing references to the constructor arguments would leave them dangling after the call returns.

Parameters

ar_pin	Defines a Pin object
ar_useInternalPullUp	Defines if internal pull up resistor is used
ar_isActiveLow	Defines if input is active low or active high

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

- See PushButton.h for a description of this code

Definition at line 12 of file PushButton.cpp.

```
00013                                   : m_pin(ar_pin),
00014                                     m_isActiveLow(ar_isActiveLow),
00015                                     m_useInternalPullUp(ar_useInternalPullUp),
00016                                     m_lastState(0),
00017                                     m_buttonPressed(0)
00018
00019 {
00020     m_pin.toInput(m_useInternalPullUp);
00021 }
```

References m_buttonPressed, m_isActiveLow, m_lastState, m_pin, and m_useInternalPullUp.

8.12.2.2 ~PushButton()

```
component::PushButton::~PushButton ()
```

Destructor

Definition at line 23 of file PushButton.cpp.

```
00024 {
00025
00026 }
```

8.12.3 Member Function Documentation

8.12.3.1 getPressedCount()

```
uint8_t component::PushButton::getPressedCount () const
```

Get pushbutton pressed count

Returns

PushButton press count

Definition at line 59 of file PushButton.cpp.

```
00060 {
00061
00062     return m_buttonPressed;
00063
00064 }
```

References m_buttonPressed.

8.12.3.2 isPressed()

```
uint8_t component::PushButton::isPressed ()
```

Is pushbutton pressed

The level is selected by the active-low flag alone: active low reads the pin low as pressed, active high reads it high as pressed. The pin is sampled, and after the debounce time (PUSHBUTTON_DEBOUNCE_TIME_US, 1 ms) sampled again; the button counts as pressed only if both samples agree. The pressed count is incremented once per press, on the transition from released to pressed, so holding the button does not count as new presses.

Returns

PushButton status

Definition at line 29 of file PushButton.cpp.

```
00030 {
00031     uint8_t l_pressed = 0;
00032
00033     if (m_isActiveLow) {
00034         if (m_pin.isLow()) {
00035             _delay_us(PUSHBUTTON_DEBOUNCE_TIME_US);
00036             if (m_pin.isLow()) {
00037                 l_pressed = 1;
00038             }
00039         }
00040     } else {
00041         if (m_pin.isHigh()) {
00042             _delay_us(PUSHBUTTON_DEBOUNCE_TIME_US);
00043             if (m_pin.isHigh()) {
00044                 l_pressed = 1;
00045             }
00046         }
00047     }
00048
00049     // count one press per release->press transition
00050     if (l_pressed && !m_lastState) {
00051         ++m_buttonPressed;
00052     }
00053     m_lastState = l_pressed;
00054
00055     return l_pressed;
00056
00057 }
```

References m_buttonPressed, m_isActiveLow, m_lastState, m_pin, and PUSHBUTTON_DEBOUNCE_TIME_US.

Referenced by main().

8.12.3.3 resetPressedCount()

```
void component::PushButton::resetPressedCount ()
```

Reset pushbutton pressed count

Definition at line 66 of file PushButton.cpp.

```
00067 {
00068     m_buttonPressed = 0;
00069
00070 }
```

References m_buttonPressed.

8.12.4 Member Data Documentation

8.12.4.1 m_buttonPressed

```
uint8_t component::PushButton::m_buttonPressed  [private]
```

PushButton pressed count

Definition at line 80 of file PushButton.h.

Referenced by getPressedCount(), isPressed(), PushButton(), and resetPressedCount().

8.12.4.2 m_isActiveLow

```
uint8_t component::PushButton::m_isActiveLow  [private]
```

Input active state

Definition at line 77 of file PushButton.h.

Referenced by isPressed(), and PushButton().

8.12.4.3 m_lastState

```
uint8_t component::PushButton::m_lastState  [private]
```

Pressed state of the previous isPressed() call

Definition at line 79 of file PushButton.h.

Referenced by isPressed(), and PushButton().

8.12.4.4 m_pin

```
io::Pin component::PushButton::m_pin  [private]
```

Pin object

Definition at line 76 of file PushButton.h.

Referenced by isPressed(), and PushButton().

8.12.4.5 m_useInternalPullUp

```
uint8_t component::PushButton::m_useInternalPullUp  [private]
```

Internal pullup resistor use

Definition at line 78 of file PushButton.h.

Referenced by PushButton().

8.13 component::RotaryEncoder Class Reference

`#include <RotaryEncoder.h>`

Public Member Functions

- RotaryEncoder (const io::Pin &ar_pinA, const io::Pin &ar_pinB, uint8_t a_useInternalPullUp=1)
- void update ()
- int32_t getPosition () const
- void resetPosition ()
- int8_t getDirection () const

Private Attributes

- io::Pin m_pinA
- io::Pin m_pinB
- uint8_t m_lastStateA
- uint8_t m_lastStateB
- int32_t m_position
- int8_t m_direction

8.13.1 Detailed Description

Definition at line 18 of file RotaryEncoder.h.

8.13.2 Constructor & Destructor Documentation

8.13.2.1 RotaryEncoder()

```
component::RotaryEncoder::RotaryEncoder (
            const io::Pin & ar_pinA,
            const io::Pin & ar_pinB,
            uint8_t a_useInternalPullUp = 1)
```

Constructor

Parameters

ar_pinA	Pin for Channel A
ar_pinB	Pin for Channel B
a_useInternalPullUp	Enable internal pullup resistors

Definition at line 11 of file RotaryEncoder.cpp.

```
00012     : m_pinA(ar_pinA),
00013       m_pinB(ar_pinB),
00014       m_lastStateA(0),
00015       m_lastStateB(0),
00016       m_position(0),
00017       m_direction(0)
00018 {
00019     m_pinA.toInput(a_useInternalPullUp);
00020     m_pinB.toInput(a_useInternalPullUp);
00021     m_lastStateA = m_pinA.isHigh();
00022     m_lastStateB = m_pinB.isHigh();
00023 }
```

References m_direction, m_lastStateA, m_lastStateB, m_pinA, m_pinB, and m_position.

8.13.3 Member Function Documentation

8.13.3.1 getDirection()

```
int8_t component::RotaryEncoder::getDirection () const
```

Get last rotation direction (+1 = CW, -1 = CCW, 0 = stationary)

Definition at line 54 of file RotaryEncoder.cpp.

```
00055 {
00056     return m_direction;
00057 }
```

References m_direction.

8.13.3.2 getPosition()

```
int32_t component::RotaryEncoder::getPosition () const
```

Get current step position counter

Definition at line 43 of file RotaryEncoder.cpp.

```
00044 {
00045     return m_position;
00046 }
```

References m_position.

Referenced by main().

8.13.3.3 resetPosition()

```
void component::RotaryEncoder::resetPosition ()
```

Reset position counter to 0

Definition at line 48 of file RotaryEncoder.cpp.

```
00049 {
00050     m_position = 0;
00051     m_direction = 0;
00052 }
```

References m_direction, and m_position.

8.13.3.4 update()

```
void component::RotaryEncoder::update ()
```

Poll encoder pins and update position

Should be called periodically or inside a pin change ISR.

Definition at line 25 of file RotaryEncoder.cpp.

```
00026 {
00027     uint8_t stateA = m_pinA.isHigh();
00028     uint8_t stateB = m_pinB.isHigh();
00029
00030     if (stateA != m_lastStateA) {
00031         if (stateA != stateB) {
00032             m_position++;
00033             m_direction = 1;
00034         } else {
00035             m_position--;
00036             m_direction = -1;
00037         }
00038         m_lastStateA = stateA;
00039         m_lastStateB = stateB;
00040     }
00041 }
```

References m_direction, m_lastStateA, m_lastStateB, m_pinA, m_pinB, and m_position.

Referenced by main().

8.13.4 Member Data Documentation

8.13.4.1 m_direction

```
int8_t component::RotaryEncoder::m_direction  [private]
```

Definition at line 53 of file RotaryEncoder.h.

Referenced by getDirection(), resetPosition(), RotaryEncoder(), and update().

8.13.4.2 m_lastStateA

```
uint8_t component::RotaryEncoder::m_lastStateA  [private]
```

Definition at line 50 of file RotaryEncoder.h.

Referenced by RotaryEncoder(), and update().

8.13.4.3 m_lastStateB

```
uint8_t component::RotaryEncoder::m_lastStateB  [private]
```

Definition at line 51 of file RotaryEncoder.h.

Referenced by RotaryEncoder(), and update().

8.13.4.4 m_pinA

```
io::Pin component::RotaryEncoder::m_pinA  [private]
```

Definition at line 48 of file RotaryEncoder.h.

Referenced by RotaryEncoder(), and update().

8.13.4.5 m_pinB

```
io::Pin component::RotaryEncoder::m_pinB  [private]
```

Definition at line 49 of file RotaryEncoder.h.

Referenced by RotaryEncoder(), and update().

8.13.4.6 m_position

```
int32_t component::RotaryEncoder::m_position  [private]
```

Definition at line 52 of file RotaryEncoder.h.

Referenced by getPosition(), resetPosition(), RotaryEncoder(), and update().

8.14 component::ServoMotor Class Reference

`#include <ServoMotor.h>`

Public Member Functions

- ServoMotor (const io::Pin &ar_pin, const uint16_t &ar_pulseCycle=0, const uint16_t &ar_pulseWidthMin=0, const uint16_t &ar_pulseWidthMid=0, const uint16_t &ar_pulseWidthMax=0)
- ~ServoMotor ()
- void on ()
- void off ()
- void toggle ()
- uint16_t computePulseCycleCount (const uint16_t &ar_clockPrescaler)
- uint16_t computePulseWidthMinCount (const uint16_t &ar_clockPrescaler)
- uint16_t computePulseWidthMidCount (const uint16_t &ar_clockPrescaler)
- uint16_t computePulseWidthMaxCount (const uint16_t &ar_clockPrescaler)
- uint16_t computeRotationAngleCount (const uint8_t &ar_angle_deg, const uint16_t &ar_clockPrescaler)
- void rotate (core::TimerCounter1 &ar_timerCounter1, const uint8_t &ar_angle_deg, const core::channel &ar_channel=core::channel::A)
- void connect (core::TimerCounter1 &ar_timerCounter1, const core::channel &ar_channel=core::channel::A)
- void disconnect (core::TimerCounter1 &ar_timerCounter1, const core::channel &ar_channel=core::channel::A)

Static Private Member Functions

- static uint16_t clampTimerCount (uint32_t a_count)

Private Attributes

- io::Pin m_pin
- uint16_t m_pulseCycle
- uint16_t m_pulseWidthMin
- uint16_t m_pulseWidthMid
- uint16_t m_pulseWidthMax

8.14.1 Detailed Description

Definition at line 138 of file ServoMotor.h.

8.14.2 Constructor & Destructor Documentation

8.14.2.1 ServoMotor()

```
component::ServoMotor::ServoMotor (
            const io::Pin & ar_pin,
            const uint16_t & ar_pulseCycle = 0,
            const uint16_t & ar_pulseWidthMin = 0,
            const uint16_t & ar_pulseWidthMid = 0,
            const uint16_t & ar_pulseWidthMax = 0)
```

Definition at line 5 of file ServoMotor.cpp.

```
00010                       : m_pin(ar_pin),
00011                         m_pulseCycle(ar_pulseCycle),
00012                         m_pulseWidthMin(ar_pulseWidthMin),
00013                         m_pulseWidthMid(ar_pulseWidthMid),
00014                         m_pulseWidthMax(ar_pulseWidthMax)
00015 {
00016     m_pin.toOutput();
00017
00018 }
```

References m_pin, m_pulseCycle, m_pulseWidthMax, m_pulseWidthMid, and m_pulseWidthMin.

8.14.2.2 ~ServoMotor()

```
component::ServoMotor::~ServoMotor ()
```

Definition at line 20 of file ServoMotor.cpp.

```
00021 {
00022
00023 }
```

8.14.3 Member Function Documentation

8.14.3.1 clampTimerCount()

```
uint16_t component::ServoMotor::clampTimerCount (
            uint32_t a_count)  [static], [private]
```

Clamp a computed timer count to the 16-bit timer range.

Parameters

a_count	Timer count

Returns

Count clamped to 0..65535

Definition at line 29 of file ServoMotor.cpp.

```
00030 {
00031     if (a_count > UINT16_MAX)
00032     {
00033         a_count = UINT16_MAX;
00034     }
00035     return static_cast<uint16_t>(a_count);
00036 }
```

Referenced by computePulseCycleCount(), computePulseWidthMaxCount(), computePulseWidthMidCount(), computePulseWidthMinCount(), and computeRotationAngleCount().

8.14.3.2 computePulseCycleCount()

```
uint16_t component::ServoMotor::computePulseCycleCount (
            const uint16_t & ar_clockPrescaler)
```

Definition at line 54 of file ServoMotor.cpp.

```
00055 {
00056     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseCycle,ar_clockPrescaler));
00057 }
```

References clampTimerCount(), m_pulseCycle, and SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT.

Referenced by connect().

8.14.3.3 computePulseWidthMaxCount()

```
uint16_t component::ServoMotor::computePulseWidthMaxCount (
            const uint16_t & ar_clockPrescaler)
```

Definition at line 64 of file ServoMotor.cpp.

```
00065 {
00066     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseWidthMax,ar_clockPrescaler));
00067 }
```

References clampTimerCount(), m_pulseWidthMax, and SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT.

Referenced by computeRotationAngleCount().

8.14.3.4 computePulseWidthMidCount()

```
uint16_t component::ServoMotor::computePulseWidthMidCount (
            const uint16_t & ar_clockPrescaler)
```

Definition at line 69 of file ServoMotor.cpp.

```
00070 {
00071     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseWidthMid,ar_clockPrescaler));
00072 }
```

References clampTimerCount(), m_pulseWidthMid, and SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT.

Referenced by computeRotationAngleCount().

8.14.3.5 computePulseWidthMinCount()

```
uint16_t component::ServoMotor::computePulseWidthMinCount (
            const uint16_t & ar_clockPrescaler)
```

Definition at line 59 of file ServoMotor.cpp.

```
00060 {
00061     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseWidthMin,ar_clockPrescaler));
00062 }
```

References clampTimerCount(), m_pulseWidthMin, and SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT.

Referenced by computeRotationAngleCount().

8.14.3.6 computeRotationAngleCount()

```
uint16_t component::ServoMotor::computeRotationAngleCount (
            const uint8_t & ar_angle_deg,
            const uint16_t & ar_clockPrescaler)
```

Definition at line 74 of file ServoMotor.cpp.

```
00075 {
00076     return
      clampTimerCount(SERVOMOTOR_TIMER_ANGLE_COUNT(ar_angle_deg,static_cast<long>(computePulseWidthMinCount(ar_clockPrescaler
00077 }
```

References clampTimerCount(), computePulseWidthMaxCount(), computePulseWidthMidCount(), computePulseWidthMinCount(), and SERVOMOTOR_TIMER_ANGLE_COUNT.

Referenced by rotate().

8.14.3.7 connect()

```
void component::ServoMotor::connect (
            core::TimerCounter1 & ar_timerCounter1,
            const core::channel & ar_channel = core::channel::A)
```

Definition at line 95 of file ServoMotor.cpp.

```
00097 {
00098
00099
00100
      ar_timerCounter1.setInputCaptureRegister(computePulseCycleCount(ar_timerCounter1.getClockPrescaler()));
00101     ar_timerCounter1.selectOperationMode(core::operationMode::fast_PWM_ICR);
00102     ar_timerCounter1.selectCompareOutputMode(ar_channel, core::compareOutputMode::clear);
00103     ar_timerCounter1.setCounter(0);
00104
00105 }
```

References core::clear, computePulseCycleCount(), core::fast_PWM_ICR, core::TimerCounter1::getClockPrescaler(), core::TimerCounter1::selectCompareOutputMode(), core::TimerCounter1::selectOperationMode(), core::TimerCounter1::setCounter(and core::TimerCounter1::setInputCaptureRegister().

Referenced by main().

8.14.3.8 disconnect()

```
void component::ServoMotor::disconnect (
            core::TimerCounter1 & ar_timerCounter1,
            const core::channel & ar_channel = core::channel::A)
```

Definition at line 107 of file ServoMotor.cpp.

```
00109 {
00110     ar_timerCounter1.selectCompareOutputMode(ar_channel, core::compareOutputMode::normal);
00111     // stop timer
00112     ar_timerCounter1.stop();
00113 }
```

References core::normal, core::TimerCounter1::selectCompareOutputMode(), and core::TimerCounter1::stop().

8.14.3.9 off()

```
void component::ServoMotor::off ()
```

Turn servo motor Off.

Definition at line 43 of file ServoMotor.cpp.

```
00044 {
00045     m_pin.setLow();
00046 }
```

References m_pin.

8.14.3.10 on()

```
void component::ServoMotor::on ()
```

Turn servo motor On.

Definition at line 38 of file ServoMotor.cpp.

```
00039 {
00040     m_pin.setHigh();
00041 }
```

References m_pin.

8.14.3.11 rotate()

```
void component::ServoMotor::rotate (
            core::TimerCounter1 & ar_timerCounter1,
            const uint8_t & ar_angle_deg,
            const core::channel & ar_channel = core::channel::A)
```

Definition at line 81 of file ServoMotor.cpp.

```
00084 {
00085
00086     ar_timerCounter1.setOutputCompareRegister(ar_channel,
      computeRotationAngleCount(ar_angle_deg,ar_timerCounter1.getClockPrescaler()));
00087
00088     // start timer
00089     ar_timerCounter1.start();
00090
00091
00092 }
```

References computeRotationAngleCount(), core::TimerCounter1::getClockPrescaler(), core::TimerCounter1::setOutputCompareRegi and core::TimerCounter1::start().

Referenced by main().

8.14.3.12 toggle()

```
void component::ServoMotor::toggle ()
```

Toggle servo motor state.

Definition at line 48 of file ServoMotor.cpp.

```
00049 {
00050     m_pin.toggle();
00051
00052 }
```

References m_pin.

8.14.4 Member Data Documentation

8.14.4.1 m_pin

```
io::Pin component::ServoMotor::m_pin  [private]
```

pin object

Definition at line 195 of file ServoMotor.h.

Referenced by off(), on(), ServoMotor(), and toggle().

8.14.4.2 m_pulseCycle

```
uint16_t component::ServoMotor::m_pulseCycle  [private]
```

pulse cycle [us]

Definition at line 197 of file ServoMotor.h.

Referenced by computePulseCycleCount(), and ServoMotor().

8.14.4.3 m_pulseWidthMax

```
uint16_t component::ServoMotor::m_pulseWidthMax  [private]
```

pulse width max [us]

Definition at line 203 of file ServoMotor.h.

Referenced by computePulseWidthMaxCount(), and ServoMotor().

8.14.4.4 m_pulseWidthMid

```
uint16_t component::ServoMotor::m_pulseWidthMid  [private]
```

pulse width mid [us]

Definition at line 201 of file ServoMotor.h.

Referenced by computePulseWidthMidCount(), and ServoMotor().

8.14.4.5 m_pulseWidthMin

```
uint16_t component::ServoMotor::m_pulseWidthMin  [private]
```

pulse width min [us]

Definition at line 199 of file ServoMotor.h.

Referenced by computePulseWidthMinCount(), and ServoMotor().

8.15 io::SPI Class Reference

```
#include <SPI.h>
```

Public Member Functions

- void selectDataMode (const dataMode &ar_dataMode)
- void selectDataOrder (const dataOrder &ar_dataOrder)
- void selectOperationMode (const operationMode &ar_operationMode)
- void selectClockPrescaler (const clockPrescaler &ar_clockPrescaler)
- void selectSlave (const uint8_t a_select)
- uint8_t writeCollision ()
- uint8_t transferComplete ()
- void masterSendByte (const uint8_t &ar_byte)
- void masterReceiveByte (uint8_t &ar_byte)
- void slaveReceiveByte (uint8_t &ar_byte)

Static Public Member Functions

- static SPI & getInstance (const io::Pin &ar_pinSCK, const io::Pin &ar_pinMISO, const io::Pin &ar_pinMOSI, const io::Pin &ar_pinSS)
- static void enableTransferCompleteInterrupt (const uint8_t a_enable)
- static void transferCompleteServiceRoutine () __asm__(STR(SPI_TRANSFER_COMPLETE_INTERRUPT)) __attribute__((__signal__

Public Attributes

- static void __used__
- static void __externally_visible__

Private Member Functions

- SPI (const io::Pin &ar_pinSCK, const io::Pin &ar_pinMISO, const io::Pin &ar_pinMOSI, const io::Pin &ar_↩ pinSS)
- ~SPI ()
- SPI (const SPI &)
- const SPI & operator= (const SPI &)

Private Attributes

- io::Pin m_pinSCK
- io::Pin m_pinMISO
- io::Pin m_pinMOSI
- io::Pin m_pinSS

Static Private Attributes

- static volatile uint8_t m_data = 0
- static volatile uint8_t m_transferComplete = 0

8.15.1 Detailed Description

Definition at line 86 of file SPI.h.

8.15.2 Constructor & Destructor Documentation

8.15.2.1 SPI() [1/2]

```
io::SPI::SPI (
            const io::Pin & ar_pinSCK,
            const io::Pin & ar_pinMISO,
            const io::Pin & ar_pinMOSI,
            const io::Pin & ar_pinSS)  [private]
```

Constructor. Initalizes the USART0 object

```
@param ar_transMode defines transmission mode
@param ar_comMode defines communication mode
@param ar_frameSize defines data frame size
@param ar_stopBit defines number of stop bits
@param ar_parityMode defines parity mode
```

Definition at line 23 of file SPI.cpp.

```
00027         : m_pinSCK(ar_pinSCK),
00028           m_pinMISO(ar_pinMISO),
00029           m_pinMOSI(ar_pinMOSI),
00030           m_pinSS(ar_pinSS)
00031 {
00032     core::MCU::enableSPI(1);
00033     enableTransferCompleteInterrupt(1);
00034
00035 }
```

References core::MCU::enableSPI(), enableTransferCompleteInterrupt(), m_pinMISO, m_pinMOSI, m_pinSCK, and m_pinSS.

Referenced by getInstance(), operator=(), and SPI().

8.15.2.2 ~SPI()

```
io::SPI::~SPI ()  [private]
```

Destructor.

Definition at line 37 of file SPI.cpp.

```
00038 {
00039
00040 }
```

8.15.2.3 SPI() [2/2]

```
io::SPI::SPI (
            const SPI & )  [private]
```

Overried Copy constructor.

References SPI().

8.15.3 Member Function Documentation

8.15.3.1 enableTransferCompleteInterrupt()

```
void io::SPI::enableTransferCompleteInterrupt (
            const uint8_t a_enable)  [static]
```

Definition at line 110 of file SPI.cpp.

```
00111 {
00112     if (a_enable) {
00113         SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT;
00114     } else {
00115         SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT;
00116     }
00117 }
```

References SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT, and SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT.

Referenced by SPI().

8.15.3.2 getInstance()

```
io::SPI & io::SPI::getInstance (
            const io::Pin & ar_pinSCK,
            const io::Pin & ar_pinMISO,
            const io::Pin & ar_pinMOSI,
            const io::Pin & ar_pinSS)  [static]
```

Definition at line 9 of file SPI.cpp.

```
00013 {
00014     static SPI l_instance(ar_pinSCK,
00015                           ar_pinMISO,
00016                           ar_pinMOSI,
00017                           ar_pinSS);
00018
00019     return l_instance;
00020
00021 }
```

References SPI().

8.15.3.3 masterReceiveByte()

```
void io::SPI::masterReceiveByte (
            uint8_t & ar_byte)
```

Receive a byte from the slave as SPI master.

Sends a dummy byte (clocking 8 SCK pulses); the byte the slave shifted in during the same transfer is read by the transfer-complete ISR and returned through ar_byte once the transfer has completed.

Parameters

ar_byte	Defines byte to be received

Definition at line 154 of file SPI.cpp.

```
00155 {
00156     // a dummy write clocks 8 SCK pulses; the ISR reads the byte the slave
00157     // shifted into SPDR during the same transfer
00158     masterSendByte(0);
00159     ar_byte = m_data;
00160
00161 }
```

References m_data, and masterSendByte().

8.15.3.4 masterSendByte()

```
void io::SPI::masterSendByte (
            const uint8_t & ar_byte)
```

Send a byte to the slave as SPI master.

SS is asserted (low) before SPDR is written so the slave is selected when the transfer starts, then the call busy-waits until the transfer-complete ISR sets m_transferComplete and deasserts SS again. The wait runs with interrupts enabled and relies on the SPI transfer-complete interrupt, armed by the constructor; global interrupts must be enabled (core::MCU::enableInterrupts()).

Parameters

ar_byte	Defines byte to be sent

Definition at line 137 of file SPI.cpp.

```
00138 {
00139     // assert SS before writing SPDR, so the slave is selected when the
00140     // transfer starts
00141     selectSlave(1);
00142     m_transferComplete = 0;
00143     SPI_DATA_REGISTER = ar_byte;
00144     // busy-wait on the flag set by the transfer-complete ISR: the wait
00145     // runs with interrupts enabled, so the SPI interrupt (armed in the
00146     // constructor) can fire and release it
00147     while (m_transferComplete == 0) {}
00148     selectSlave(0);
00149
00150
00151
00152 }
```

References m_transferComplete, selectSlave(), and SPI_DATA_REGISTER.

Referenced by masterReceiveByte().

8.15.3.5 operator=()

```
const SPI & io::SPI::operator= (
            const SPI & )  [private]
```

Override assign operator.

References SPI().

8.15.3.6 selectClockPrescaler()

```
void io::SPI::selectClockPrescaler (
            const clockPrescaler & ar_clockPrescaler)
```

Definition at line 53 of file SPI.cpp.

```
00054 {
00055     SPI_SELECT_CLOCK_PRESCALER(static_cast<uint8_t>(ar_clockPrescaler));
00056
00057 }
```

References SPI_SELECT_CLOCK_PRESCALER.

8.15.3.7 selectDataMode()

```
void io::SPI::selectDataMode (
            const dataMode & ar_dataMode)
```

Definition at line 99 of file SPI.cpp.

```
00100 {
00101     SPI_SELECT_DATA_MODE(static_cast<uint8_t>(ar_dataMode));
00102 }
```

References SPI_SELECT_DATA_MODE.

8.15.3.8 selectDataOrder()

```
void io::SPI::selectDataOrder (
            const dataOrder & ar_dataOrder)
```

Definition at line 104 of file SPI.cpp.

```
00105 {
00106     SPI_SELECT_DATA_ORDER(static_cast<uint8_t>(ar_dataOrder));
00107
00108 }
```

References SPI_SELECT_DATA_ORDER.

8.15.3.9 selectOperationMode()

```
void io::SPI::selectOperationMode (
            const operationMode & ar_operationMode)
```

Definition at line 59 of file SPI.cpp.

```
00060 {
00061     switch (ar_operationMode)
00062     {
00063         case operationMode::master:
00064         {
00065             m_pinMOSI.toOutput();
00066             m_pinSCK.toOutput();
00067             m_pinMISO.toInput(1);
00068             m_pinSS.toOutput();
00069             m_pinSS.setHigh();
00070             SPI_SELECT_MASTER_MODE;
00071             SPI_ENABLE;
00072             break;
00073         }
00074         case operationMode::slave:
00075         {
00076             m_pinMISO.toOutput();
00077             SPI_SELECT_SLAVE_MODE;
00078             SPI_ENABLE;
00079             break;
00080         }
00081         case operationMode::submaster:
00082         {
00083             m_pinMOSI.toOutput();
00084             m_pinSCK.toOutput();
00085             m_pinMISO.toInput(1);
00086             m_pinSS.toInput(1);
00087             SPI_SELECT_MASTER_MODE;
00088             SPI_ENABLE;
00089             break;
00090         }
00091         case operationMode::disable:
00092         {
00093             SPI_DISABLE;
00094             break;
00095         }
00096     }
00097 }
```

References io::disable, m_pinMISO, m_pinMOSI, m_pinSCK, m_pinSS, io::master, io::slave, SPI_DISABLE, SPI_ENABLE, SPI_SELECT_MASTER_MODE, SPI_SELECT_SLAVE_MODE, and io::submaster.

8.15.3.10 selectSlave()

```
void io::SPI::selectSlave (
            const uint8_t a_select)
```

Definition at line 119 of file SPI.cpp.

```
00120 {
00121     if (a_select) {
00122         m_pinSS.setLow();
00123 
00124     } else {
00125         m_pinSS.setHigh();
00126     }
00127 
00128 }
```

References m_pinSS.

Referenced by masterSendByte().

8.15.3.11 slaveReceiveByte()

```
void io::SPI::slaveReceiveByte (
            uint8_t & ar_byte)
```

Definition at line 163 of file SPI.cpp.

```
00164 {
00165     ar_byte = m_data;
00166 
00167 }
```

References m_data.

8.15.3.12 transferComplete()

```
uint8_t io::SPI::transferComplete ()
```

Is serial transfer complete.

Definition at line 48 of file SPI.cpp.

```
00049 {
00050     return (SPI_STATUS_REGISTER & (1 << SPI_TRANSFER_COMPLETE));
00051 }
```

References SPI_STATUS_REGISTER, and SPI_TRANSFER_COMPLETE.

8.15.3.13 transferCompleteServiceRoutine()

```
void io::SPI::transferCompleteServiceRoutine ()  [static]
```

Serial transfer complete ISR.

Definition at line 129 of file SPI.cpp.

```
00130 {
00131 
00132         // reading SPDR also clears SPIF in the status register
00133         m_data = SPI_DATA_REGISTER;
00134         m_transferComplete = 1;
00135 }
```

References m_data, m_transferComplete, and SPI_DATA_REGISTER.

8.15.3.14 writeCollision()

```
uint8_t io::SPI::writeCollision ()
```

Is there write collision.

Definition at line 42 of file SPI.cpp.

```
00043 {
00044     return (SPI_STATUS_REGISTER & (1 << SPI_WRITE_COLLISION));
00045 }
```

References SPI_STATUS_REGISTER, and SPI_WRITE_COLLISION.

8.15.4 Member Data Documentation

8.15.4.1 __externally_visible__

```
void io::SPI::__externally_visible__
```

Definition at line 147 of file SPI.h.

8.15.4.2 __used__

```
void io::SPI::__used__
```

Definition at line 147 of file SPI.h.

8.15.4.3 m_data

```
volatile uint8_t io::SPI::m_data = 0  [static], [private]
```

Definition at line 177 of file SPI.h.

Referenced by masterReceiveByte(), slaveReceiveByte(), and transferCompleteServiceRoutine().

8.15.4.4 m_pinMISO

```
io::Pin io::SPI::m_pinMISO  [private]
```

pin object

Definition at line 182 of file SPI.h.

Referenced by selectOperationMode(), and SPI().

8.15.4.5 m_pinMOSI

```
io::Pin io::SPI::m_pinMOSI  [private]
```

pin object

Definition at line 183 of file SPI.h.

Referenced by selectOperationMode(), and SPI().

8.15.4.6 m_pinSCK

```
io::Pin io::SPI::m_pinSCK  [private]
```

pin object

Definition at line 181 of file SPI.h.

Referenced by selectOperationMode(), and SPI().

8.15.4.7 m_pinSS

```
io::Pin io::SPI::m_pinSS  [private]
```

pin object

Definition at line 184 of file SPI.h.

Referenced by selectOperationMode(), selectSlave(), and SPI().

8.15.4.8 m_transferComplete

```
volatile uint8_t io::SPI::m_transferComplete = 0  [static], [private]
```

set by the transfer-complete ISR, awaited by masterSendByte

Definition at line 179 of file SPI.h.

Referenced by masterSendByte(), and transferCompleteServiceRoutine().

8.16 component::SSD1306_OLED Class Reference

```
#include <SSD1306_OLED.h>
```

Public Member Functions

- SSD1306_OLED (uint8_t a_i2cAddress=0x3C, uint8_t a_width=128, uint8_t a_height=64)
- void init ()
- void clear ()
- void setCursor (uint8_t a_col, uint8_t a_page)
- void print (const char *ap_str)

Private Member Functions

- void sendCommand (uint8_t a_cmd)
- void sendData (uint8_t a_data)

Private Attributes

- uint8_t m_i2cAddr
- uint8_t m_width
- uint8_t m_height
- core::TWI & mr_twi

8.16.1 Detailed Description

Definition at line 18 of file SSD1306_OLED.h.

8.16.2 Constructor & Destructor Documentation

8.16.2.1 SSD1306_OLED()

```
component::SSD1306_OLED::SSD1306_OLED (
            uint8_t a_i2cAddress = 0x3C,
            uint8_t a_width = 128,
            uint8_t a_height = 64)
```

Constructor

Parameters

a_i2cAddress	I2C address (default 0x3C or 0x3D)
a_width	Display width in pixels (128)
a_height	Display height in pixels (64 or 32)

Definition at line 41 of file SSD1306_OLED.cpp.

```
00042     : m_i2cAddr(a_i2cAddress),
00043       m_width(a_width),
00044       m_height(a_height),
00045       mr_twi(core::TWI::getInstance(400000UL))
00046 {
00047 }
```

References m_height, m_i2cAddr, m_width, and mr_twi.

8.16.3 Member Function Documentation

8.16.3.1 clear()

```
void component::SSD1306_OLED::clear ()
```

Clear display RAM

Definition at line 71 of file SSD1306_OLED.cpp.

```
00072 {
00073     for (uint8_t page = 0; page < (m_height / 8); page++) {
00074         setCursor(0, page);
00075         mr_twi.start();
00076         mr_twi.write(m_i2cAddr « 1);
00077         mr_twi.write(0x40); // Data mode
00078         for (uint8_t col = 0; col < m_width; col++) {
00079             mr_twi.write(0x00);
00080         }
00081         mr_twi.stop();
00082     }
00083     setCursor(0, 0);
00084 }
```

References m_height, m_i2cAddr, m_width, mr_twi, and setCursor().

Referenced by init(), and main().

8.16.3.2 init()

```
void component::SSD1306_OLED::init ()
```

Initialize SSD1306 controller over I2C

Definition at line 49 of file SSD1306_OLED.cpp.

```
00050 {
00051     _delay_ms(10);
00052     sendCommand(0xAE); // Display OFF
00053     sendCommand(0xD5); sendCommand(0x80); // Set Display Clock Divide Ratio
00054     sendCommand(0xA8); sendCommand(m_height - 1); // Set Multiplex Ratio
00055     sendCommand(0xD3); sendCommand(0x00); // Display Offset
00056     sendCommand(0x40); // Start Line 0
00057     sendCommand(0x8D); sendCommand(0x14); // Enable Charge Pump
00058     sendCommand(0x20); sendCommand(0x00); // Memory Addressing Mode: Horizontal
00059     sendCommand(0xA1); // Segment Re-map
00060     sendCommand(0xC8); // COM Output Scan Direction
00061     sendCommand(0xDA); sendCommand((m_height == 64) ? 0x12 : 0x02); // COM Pins Configuration
00062     sendCommand(0x81); sendCommand(0xCF); // Set Contrast Control
00063     sendCommand(0xD9); sendCommand(0xF1); // Pre-charge Period
00064     sendCommand(0xDB); sendCommand(0x40); // VCOMH Deselect Level
00065     sendCommand(0xA4); // Entire Display ON
00066     sendCommand(0xA6); // Normal Display
00067     sendCommand(0xAF); // Display ON
00068     clear();
00069 }
```

References clear(), m_height, and sendCommand().

Referenced by main().

8.16.3.3 print()

```
void component::SSD1306_OLED::print (
            const char * ap_str)
```

Print string to display

Parameters

ap_str	Text string

Definition at line 93 of file SSD1306_OLED.cpp.

```
00094 {
00095     if (!ap_str) return;
00096     while (*ap_str) {
00097         char c = *ap_str++;
00098         uint8_t idx = (c >= ' ' && c <= '9') ? (c - ' ') : 0;
00099         mr_twi.start();
00100         mr_twi.write(m_i2cAddr << 1);
00101         mr_twi.write(0x40);
00102         for (uint8_t i = 0; i < 5; i++) {
00103             mr_twi.write(font5x7[idx][i]);
00104         }
00105         mr_twi.write(0x00); // char spacing
00106         mr_twi.stop();
00107     }
00108 }
```

References font5x7, m_i2cAddr, and mr_twi.

Referenced by main().

8.16.3.4 sendCommand()

```
void component::SSD1306_OLED::sendCommand (
            uint8_t a_cmd)  [private]
```

Definition at line 110 of file SSD1306_OLED.cpp.

```
00111 {
00112     mr_twi.start();
00113     mr_twi.write(m_i2cAddr << 1);
00114     mr_twi.write(0x00); // Command mode
00115     mr_twi.write(a_cmd);
00116     mr_twi.stop();
00117 }
```

References m_i2cAddr, and mr_twi.

Referenced by init(), and setCursor().

8.16.3.5 sendData()

```
void component::SSD1306_OLED::sendData (
            uint8_t a_data)  [private]
```

Definition at line 119 of file SSD1306_OLED.cpp.

```
00120 {
00121     mr_twi.start();
00122     mr_twi.write(m_i2cAddr << 1);
00123     mr_twi.write(0x40); // Data mode
00124     mr_twi.write(a_data);
00125     mr_twi.stop();
00126 }
```

References m_i2cAddr, and mr_twi.

8.16.3.6 setCursor()

```
void component::SSD1306_OLED::setCursor (
            uint8_t a_col,
            uint8_t a_page)
```

Set cursor page and column

Parameters

a_col	Column pixel index (0..127)
a_page	Page index (0..7 for 64px, 0..3 for 32px)

Definition at line 86 of file SSD1306_OLED.cpp.

```
00087 {
00088     sendCommand(0xB0 | (a_page & 0x07));
00089     sendCommand(0x00 | (a_col & 0x0F));
00090     sendCommand(0x10 | ((a_col » 4) & 0x0F));
00091 }
```

References sendCommand().

Referenced by clear(), and main().

8.16.4 Member Data Documentation

8.16.4.1 m_height

```
uint8_t component::SSD1306_OLED::m_height  [private]
```

Definition at line 56 of file SSD1306_OLED.h.

Referenced by clear(), init(), and SSD1306_OLED().

8.16.4.2 m_i2cAddr

```
uint8_t component::SSD1306_OLED::m_i2cAddr  [private]
```

Definition at line 54 of file SSD1306_OLED.h.

Referenced by clear(), print(), sendCommand(), sendData(), and SSD1306_OLED().

8.16.4.3 m_width

```
uint8_t component::SSD1306_OLED::m_width  [private]
```

Definition at line 55 of file SSD1306_OLED.h.

Referenced by clear(), and SSD1306_OLED().

8.16.4.4 mr_twi

```
core::TWI& component::SSD1306_OLED::mr_twi  [private]
```

Definition at line 57 of file SSD1306_OLED.h.

Referenced by clear(), print(), sendCommand(), sendData(), and SSD1306_OLED().

8.17 component::StepperMotor Class Reference

```
#include <StepperMotor.h>
```

Public Member Functions

- StepperMotor (const mode &ar_mode, const io::Pin &ar_pinCoil1, const io::Pin &ar_pinCoil2, const io::Pin &ar_pinCoil3, const io::Pin &ar_pinCoil4)
- ~StepperMotor ()
- void step (const int16_t a_step, const uint16_t a_speed)
- void step (const int16_t a_step, const uint16_t a_speed, const float a_stepAngle)
- void step (const int16_t a_step, const uint16_t a_speed, const uint16_t a_accel, const uint16_t a_decel)
- void stepPulse (const uint8_t a_stepPulse)
- void stepDelay (uint16_t a_stepDelay)
- uint8_t goalReached ()
- void setCurrentPos (uint16_t a_currentPos)
- uint16_t currentPos ()
- uint16_t computeStepDelay (uint16_t a_step, const uint16_t a_speed, const uint16_t a_accel, const uint16_t a_decel)

Public Attributes

- uint16_t m_accelTime
- uint16_t m_decelTime
- uint16_t m_constSpeedTime

Private Member Functions

- void beginMove (const int16_t a_step)

Private Attributes

- io::Pin m_pinCoil1
- io::Pin m_pinCoil2
- io::Pin m_pinCoil3
- io::Pin m_pinCoil4
- mode stepMode
- uint8_t m_goalReached
- int16_t m_stepNumber
- int16_t m_targetStep
- uint16_t m_currentPos
- uint32_t m_profileTime
- uint32_t m_profileSpeed
- uint64_t m_speedTimeProduct
- uint64_t m_profilePosition

8.17.1 Detailed Description

Definition at line 75 of file StepperMotor.h.

8.17.2 Constructor & Destructor Documentation

8.17.2.1 StepperMotor()

```
component::StepperMotor::StepperMotor (
            const mode & ar_mode,
            const io::Pin & ar_pinCoil1,
            const io::Pin & ar_pinCoil2,
            const io::Pin & ar_pinCoil3,
            const io::Pin & ar_pinCoil4)
```

Definition at line 7 of file StepperMotor.cpp.

```
00012     : m_pinCoil1(ar_pinCoil1),
00013       m_pinCoil2(ar_pinCoil2),
00014       m_pinCoil3(ar_pinCoil3),
00015       m_pinCoil4(ar_pinCoil4),
00016       stepMode(ar_mode),
00017       m_goalReached(0),
00018       m_stepNumber(0),
00019       m_targetStep(0),
00020       m_currentPos(0),
00021       m_profileTime(0),
00022       m_profileSpeed(0),
00023       m_speedTimeProduct(0),
00024       m_profilePosition(0)
00025 {
00026
00027     m_pinCoil1.toOutput();
00028     m_pinCoil2.toOutput();
00029     m_pinCoil3.toOutput();
00030     m_pinCoil4.toOutput();
00031
00032 }
```

References m_currentPos, m_goalReached, m_pinCoil1, m_pinCoil2, m_pinCoil3, m_pinCoil4, m_profilePosition, m_profileSpeed, m_profileTime, m_speedTimeProduct, m_stepNumber, m_targetStep, and stepMode.

8.17.2.2 ~StepperMotor()

```
component::StepperMotor::~StepperMotor ()
```

Definition at line 34 of file StepperMotor.cpp.

```
00035 {
00036
00037 }
```

8.17.3 Member Function Documentation

8.17.3.1 beginMove()

```
void component::StepperMotor::beginMove (
            const int16_t a_step)  [private]
```

Reset the state at the start of a move: the step counter, the speed profile, and the goal flag. Called when the goal of the previous move was reached or when the target changes, so the state of one move never leaks into the next.

Definition at line 39 of file StepperMotor.cpp.

```
00040 {
00041     m_stepNumber = 0;
00042     m_targetStep = a_step;
00043     m_profileTime = 0;
00044     m_profileSpeed = 0;
00045     m_speedTimeProduct = 0;
00046     m_profilePosition = 0;
00047     m_goalReached = (a_step == 0) ? 1 : 0;
00048 }
```

References m_goalReached, m_profilePosition, m_profileSpeed, m_profileTime, m_speedTimeProduct, m_stepNumber, and m_targetStep.

Referenced by step(), step(), and step().

8.17.3.2 computeStepDelay()

```
uint16_t component::StepperMotor::computeStepDelay (
            uint16_t a_step,
            const uint16_t a_speed,
            const uint16_t a_accel,
            const uint16_t a_decel)
```

Compute the step delay of one profile tick.

Units: speed is given in steps/s, acceleration and deceleration in steps/s^2, a_step in steps; the profile time unit is one call of step()/computeStepDelay() (1 ms when the caller polls at 1 kHz). The returned delay is in ms: 0 when this tick fires no step, 1 when it fires one, so a move of N steps takes as many ticks as the trapezoid accelTime + constSpeedTime + decelTime (in ms) that the caller sees when polling at 1 kHz. The function has no side effects beyond advancing the profile state. A speed, an acceleration or a deceleration of 0 cannot drive a profile and are guarded: the function returns 0 forever (no step fires, no division by zero).

Parameters

a_step	Defines the number of steps of the current move
a_speed	Defines the maximum speed in steps/s
a_accel	Defines the acceleration in steps/s^2
a_decel	Defines the deceleration in steps/s^2

Returns

Step delay in ms (0 = no step this tick)

Definition at line 142 of file StepperMotor.cpp.

```
00146 {
00147
00148     // a profile cannot be driven by a zero speed, acceleration or
00149     // deceleration: fire no step and avoid the division by zero
00150     if ((a_speed == 0) || (a_accel == 0) || (a_decel == 0))
00151     {
00152         return 0;
00153     }
00154
00155     uint32_t l_accelTime= (1000UL*a_speed/a_accel);
00156     uint32_t l_decelTime = (1000UL*a_speed/a_decel);
00157     uint16_t l_speed = a_speed;   // rescaled when the ramp is shortened
00158     // signed and 32-bit wide: the constant-speed phase can be negative
00159     // before the ramp correction below, and 1000*steps/step/s overflows
00160     // int16 for any move longer than a few steps
```

```
00161    int32_t l_constSpeedTime = static_cast<int32_t>(1000UL*a_step/a_speed)
00162                               - static_cast<int32_t>(l_accelTime/2UL)
00163                               - static_cast<int32_t>(l_decelTime/2UL);
00164
00165
00166
00167
00168    if (l_constSpeedTime<0)
00169    {
00170        l_constSpeedTime=0;
00171        // sqrtf of the float ratio: the old integer division
00172        // a_step/a_accel truncated to 0 for any realistic ramp
00173        l_accelTime = static_cast<uint32_t>(1000UL*sqrtf(static_cast<float>(a_step)
00174                                                         / static_cast<float>(a_accel)));
00175        l_speed=static_cast<uint16_t>(l_accelTime*a_accel/1000UL);
00176        l_decelTime = 1000UL*l_speed/a_decel;
00177
00178    }
00179
00180
00181
00182
00183    if (m_profileTime<=l_accelTime)
00184    {
00185        m_profileSpeed=a_accel*m_profileTime;
00186
00187    }
00188    else if ((m_profileTime>l_accelTime) && (m_profileTime<=(l_accelTime+l_constSpeedTime)))
00189    {
00190         m_profileSpeed=1000UL*l_speed;
00191
00192    }
00193    else if
      ((m_profileTime>(l_accelTime+l_constSpeedTime))&&(m_profileTime<=l_accelTime+l_constSpeedTime+l_decelTime))
00194    {
00195        m_profileSpeed=
      (1000UL*l_speed)-a_decel*(m_profileTime-l_accelTime-static_cast<uint32_t>(l_constSpeedTime));
00196
00197
00198    }
00199
00200
00201    m_profileTime=m_profileTime+1;
00202
00203
00204    m_speedTimeProduct = m_speedTimeProduct+m_profileSpeed;
00205
00206    if (m_speedTimeProduct - m_profilePosition >= 1000000UL)
00207    {
00208        m_profilePosition=m_profilePosition+1000000UL;
00209        return 1;
00210    }
00211    else
00212    {
00213        return 0;
00214    }
00215
00216
00217
00218
00219
00220 }
```

References m_profilePosition, m_profileSpeed, m_profileTime, and m_speedTimeProduct.

Referenced by step().

8.17.3.3 currentPos()

```
uint16_t component::StepperMotor::currentPos ()
```

Definition at line 55 of file StepperMotor.cpp.

```
00056 {
00057     return m_currentPos;
00058 }
```

References m_currentPos.

8.17.3.4 goalReached()

```
uint8_t component::StepperMotor::goalReached ()
```

Definition at line 411 of file StepperMotor.cpp.

```
00412 {
00413     return m_goalReached;
00414
00415 }
```

References m_goalReached.

8.17.3.5 setCurrentPos()

```
void component::StepperMotor::setCurrentPos (
            uint16_t a_currentPos)
```

Definition at line 50 of file StepperMotor.cpp.

```
00051 {
00052     m_currentPos = a_currentPos;
00053 }
```

References m_currentPos.

8.17.3.6 step() [1/3]

```
void component::StepperMotor::step (
            const int16_t a_step,
            const uint16_t a_speed)
```

Turn servo motor On.

Definition at line 223 of file StepperMotor.cpp.

```
00224 {
00225     // a different target, or a call after the goal was reached, starts a
00226     // new move; the step delay is recomputed on each call instead of
00227     // being cached once from the first call
00228     if (m_goalReached || (a_step != m_targetStep))
00229     {
00230         beginMove(a_step);
00231     }
00232     uint16_t l_stepDelay_ms = static_cast<uint16_t>(1000UL/a_speed);
00233
00234     if (a_step<=0)
00235     {
00236         m_stepNumber++;
00237         m_currentPos++;
00238         if (m_stepNumber == -a_step)
00239         {
00240             m_goalReached = 1;
00241         }
00242
00243     } else {
00244
00245         m_stepNumber--;
00246         m_currentPos--;
00247         if (m_stepNumber == -a_step)
00248         {
00249             m_goalReached = 1;
00250         }
00251     }
00252
00253     switch (stepMode)
00254     {
00255         case mode::fullStep:
00256         {
00257             // equivalent to m_stepNumber % 4
```

```
00258             // 1 3 5 7 <->
00259             stepPulse(2*(m_stepNumber & 3)+1);
00260             break;
00261         }
00262
00263         case mode::halfStep:
00264         {
00265             // equivalent to m_stepNumber % 8
00266             // 0 1 2 3 4 5 6 7 <->
00267             stepPulse(m_stepNumber & 7);
00268             break;
00269         }
00270     }
00271
00272     stepDelay(l_stepDelay_ms);
00273 }
```

References beginMove(), component::fullStep, component::halfStep, m_currentPos, m_goalReached, m_stepNumber, m_targetStep, stepDelay(), stepMode, and stepPulse().

8.17.3.7 step() [2/3]

```
void component::StepperMotor::step (
            const int16_t a_step,
            const uint16_t a_speed,
            const float a_stepAngle)
```

Definition at line 275 of file StepperMotor.cpp.

```
00276 {
00277     // a different target, or a call after the goal was reached, starts a
00278     // new move; the step delay is recomputed on each call instead of
00279     // being cached once from the first call
00280     if (m_goalReached || (a_step != m_targetStep))
00281     {
00282         beginMove(a_step);
00283     }
00284     uint16_t l_stepDelay_ms = static_cast<uint16_t>(1000UL*a_stepAngle/a_speed);
00285
00286     if (a_step<=0)
00287     {
00288         m_stepNumber++;
00289         m_currentPos++;
00290         if (m_stepNumber == -a_step)
00291         {
00292             m_goalReached = 1;
00293         }
00294
00295     } else {
00296
00297         m_stepNumber--;
00298         m_currentPos--;
00299         if (m_stepNumber == -a_step)
00300         {
00301             m_goalReached = 1;
00302         }
00303     }
00304
00305     switch (stepMode)
00306     {
00307         case mode::fullStep:
00308         {
00309             // equivalent to m_stepNumber % 4
00310             // 1 3 5 7 <->
00311             stepPulse(2*(m_stepNumber & 3)+1);
00312             break;
00313         }
00314
00315         case mode::halfStep:
00316         {
00317             // equivalent to m_stepNumber % 8
00318             // 0 1 2 3 4 5 6 7 <->
00319             stepPulse(m_stepNumber & 7);
00320             break;
00321         }
00322     }
00323
00324     stepDelay(l_stepDelay_ms);
00325 }
```

References beginMove(), component::fullStep, component::halfStep, m_currentPos, m_goalReached, m_stepNumber, m_targetStep, stepDelay(), stepMode, and stepPulse().

8.17.3.8 step() [3/3]

```
void component::StepperMotor::step (
            const int16_t a_step,
            const uint16_t a_speed,
            const uint16_t a_accel,
            const uint16_t a_decel)
```

Definition at line 61 of file StepperMotor.cpp.

```
00065 {
00066     uint16_t l_stepDelay_ms = 0;
00067     uint16_t l_step=0;
00068 
00069     // a different target, or a call after the goal was reached, starts
00070     // a new move and resets the profile state of the previous move
00071     if (m_goalReached || (a_step != m_targetStep))
00072     {
00073         beginMove(a_step);
00074     }
00075 
00076     if (a_step<=0)
00077     {
00078        l_step = -a_step;
00079     } else
00080     {
00081         l_step = a_step;
00082     }
00083 
00084     l_stepDelay_ms = computeStepDelay(l_step,
00085                                       a_speed,
00086                                       a_accel,
00087                                       a_decel);
00088 
00089     if (l_stepDelay_ms>0)
00090     {
00091 
00092     if (a_step<=0)
00093     {
00094         m_stepNumber++;
00095         m_currentPos++;
00096         if (m_stepNumber == -a_step)
00097         {
00098             m_goalReached = 1;
00099         }
00100 
00101     } else {
00102 
00103         m_stepNumber--;
00104         m_currentPos--;
00105         if (m_stepNumber == -a_step)
00106         {
00107             m_goalReached = 1;
00108         }
00109     }
00110 
00111 
00112     switch (stepMode)
00113     {
00114         case mode::fullStep:
00115         {
00116             // equivalent to m_stepNumber % 4
00117             // 1 3 5 7 <->
00118             stepPulse(2*(m_stepNumber & 3)+1);
00119             break;
00120         }
00121 
00122         case mode::halfStep:
00123         {
00124             // equivalent to m_stepNumber % 8
00125             // 0 1 2 3 4 5 6 7 <->
00126             stepPulse(m_stepNumber & 7);
00127             break;
00128         }
00129     }
00130 
00131     stepDelay(l_stepDelay_ms);
00132 
00133     }
00134     else
00135     {
00136         // no step fired this tick: keep the 1 ms polling cadence
00137         stepDelay(1);
```

```
00138     }
00139
00140 }
```

References beginMove(), computeStepDelay(), component::fullStep, component::halfStep, m_currentPos, m_goalReached, m_stepNumber, m_targetStep, stepDelay(), stepMode, and stepPulse().

8.17.3.9 stepDelay()

```
void component::StepperMotor::stepDelay (
            uint16_t a_stepDelay)
```

Wait for a number of milliseconds.

Parameters

a_stepDelay	Defines duration in ms

Definition at line 403 of file StepperMotor.cpp.

```
00404 {
00405     while(a_stepDelay--)
00406     {
00407         _delay_ms(1);
00408     }
00409 }
```

Referenced by step(), step(), and step().

8.17.3.10 stepPulse()

```
void component::StepperMotor::stepPulse (
            const uint8_t a_stepPulse)
```

Definition at line 328 of file StepperMotor.cpp.

```
00329 {
00330
00331     switch (a_stepPulse) {
00332       case 0:    // 1000
00333       {
00334         m_pinCoil1.setHigh();
00335         m_pinCoil2.setLow();
00336         m_pinCoil3.setLow();
00337         m_pinCoil4.setLow();
00338         break;
00339       }
00340       case 1:    // 1100
00341       {
00342         m_pinCoil1.setHigh();
00343         m_pinCoil2.setHigh();
00344         m_pinCoil3.setLow();
00345         m_pinCoil4.setLow();
00346         break;
00347       }
00348       case 2:    //0100
00349       {
00350         m_pinCoil1.setLow();
00351         m_pinCoil2.setHigh();
00352         m_pinCoil3.setLow();
00353         m_pinCoil4.setLow();
00354         break;
00355       }
00356       case 3:    //0110
00357       {
00358         m_pinCoil1.setLow();
00359         m_pinCoil2.setHigh();
00360         m_pinCoil3.setHigh();
```

```
00361         m_pinCoil4.setLow();
00362         break;
00363       }
00364       case 4:    //0010
00365       {
00366         m_pinCoil1.setLow();
00367         m_pinCoil2.setLow();
00368         m_pinCoil3.setHigh();
00369         m_pinCoil4.setLow();
00370         break;
00371       }
00372       case 5:    //0011
00373       {
00374         m_pinCoil1.setLow();
00375         m_pinCoil2.setLow();
00376         m_pinCoil3.setHigh();
00377         m_pinCoil4.setHigh();
00378         break;
00379       }
00380       case 6:    //0001
00381       {
00382         m_pinCoil1.setLow();
00383         m_pinCoil2.setLow();
00384         m_pinCoil3.setLow();
00385         m_pinCoil4.setHigh();
00386         break;
00387       }
00388       case 7:    //1001
00389       {
00390         m_pinCoil1.setHigh();
00391         m_pinCoil2.setLow();
00392         m_pinCoil3.setLow();
00393         m_pinCoil4.setHigh();
00394         break;
00395       }
00396     }
00397
00398
00399 }
```

References m_pinCoil1, m_pinCoil2, m_pinCoil3, and m_pinCoil4.

Referenced by step(), step(), and step().

8.17.4 Member Data Documentation

8.17.4.1 m_accelTime

```
uint16_t component::StepperMotor::m_accelTime
```

pulse delay in ms

Definition at line 142 of file StepperMotor.h.

8.17.4.2 m_constSpeedTime

```
uint16_t component::StepperMotor::m_constSpeedTime
```

pulse delay in ms

Definition at line 144 of file StepperMotor.h.

8.17.4.3 m_currentPos

```
uint16_t component::StepperMotor::m_currentPos  [private]
```

pulse delay in ms

Definition at line 171 of file StepperMotor.h.

Referenced by currentPos(), setCurrentPos(), step(), step(), step(), and StepperMotor().

8.17.4.4 m_decelTime

```
uint16_t component::StepperMotor::m_decelTime
```

pulse delay in ms

Definition at line 143 of file StepperMotor.h.

8.17.4.5 m_goalReached

```
uint8_t component::StepperMotor::m_goalReached  [private]
```

Definition at line 168 of file StepperMotor.h.

Referenced by beginMove(), goalReached(), step(), step(), step(), and StepperMotor().

8.17.4.6 m_pinCoil1

```
io::Pin component::StepperMotor::m_pinCoil1  [private]
```

pin object

Definition at line 162 of file StepperMotor.h.

Referenced by StepperMotor(), and stepPulse().

8.17.4.7 m_pinCoil2

```
io::Pin component::StepperMotor::m_pinCoil2  [private]
```

pin object

Definition at line 163 of file StepperMotor.h.

Referenced by StepperMotor(), and stepPulse().

8.17.4.8 m_pinCoil3

```
io::Pin component::StepperMotor::m_pinCoil3  [private]
```

pin object

Definition at line 164 of file StepperMotor.h.

Referenced by StepperMotor(), and stepPulse().

8.17.4.9 m_pinCoil4

```
io::Pin component::StepperMotor::m_pinCoil4  [private]
```

pin object

Definition at line 165 of file StepperMotor.h.

Referenced by StepperMotor(), and stepPulse().

8.17.4.10 m_profilePosition

```
uint64_t component::StepperMotor::m_profilePosition  [private]
```

product already turned into steps

Definition at line 176 of file StepperMotor.h.

Referenced by beginMove(), computeStepDelay(), and StepperMotor().

8.17.4.11 m_profileSpeed

```
uint32_t component::StepperMotor::m_profileSpeed  [private]
```

current speed in 1/1000 steps/s

Definition at line 174 of file StepperMotor.h.

Referenced by beginMove(), computeStepDelay(), and StepperMotor().

8.17.4.12 m_profileTime

```
uint32_t component::StepperMotor::m_profileTime  [private]
```

computeStepDelay() call counter

Definition at line 173 of file StepperMotor.h.

Referenced by beginMove(), computeStepDelay(), and StepperMotor().

8.17.4.13 m_speedTimeProduct

```
uint64_t component::StepperMotor::m_speedTimeProduct  [private]
```

accumulated speed-time product

Definition at line 175 of file StepperMotor.h.

Referenced by beginMove(), computeStepDelay(), and StepperMotor().

8.17.4.14 m_stepNumber

```
int16_t component::StepperMotor::m_stepNumber  [private]
```

step counter of the current move

Definition at line 169 of file StepperMotor.h.

Referenced by beginMove(), step(), step(), step(), and StepperMotor().

8.17.4.15 m_targetStep

```
int16_t component::StepperMotor::m_targetStep  [private]
```

target step count of the current move

Definition at line 170 of file StepperMotor.h.

Referenced by beginMove(), step(), step(), step(), and StepperMotor().

8.17.4.16 stepMode

```
mode component::StepperMotor::stepMode  [private]
```

steps per revolution

Definition at line 167 of file StepperMotor.h.

Referenced by step(), step(), step(), and StepperMotor().

8.18 io::Telemetry Class Reference

```
#include <Telemetry.h>
```

Static Public Member Functions

- static void init (io::USART0 &ar_usart=io::USART0::getInstance())
- static void sendADC (uint8_t a_channel, uint16_t a_value)
- static void sendPin (uint8_t a_port, uint8_t a_pin, uint8_t a_state)
- static void sendMotor (uint16_t a_pos, uint16_t a_speed)
- static void sendLog (const char *ap_msg)

Static Private Attributes

- static io::USART0 * mp_usart = nullptr

8.18.1 Detailed Description

Definition at line 27 of file Telemetry.h.

8.18.2 Member Function Documentation

8.18.2.1 init()

```
void io::Telemetry::init (
            io::USART0 & ar_usart = io::USART0::getInstance())  [static]
```

Initialize telemetry system

Parameters

ar_usart	Reference to active USART0 instance

Definition at line 13 of file Telemetry.cpp.

```
00014 {
00015     mp_usart = &ar_usart;
00016 }
```

References mp_usart.

Referenced by main().

8.18.2.2 sendADC()

```
void io::Telemetry::sendADC (
            uint8_t a_channel,
            uint16_t a_value)  [static]
```

Send ADC channel sample

Parameters

a_channel	ADC pin channel (0..15)
a_value	Raw 16-bit ADC value

Definition at line 18 of file Telemetry.cpp.

```
00019 {
00020     if (!mp_usart) {
00021         mp_usart = &io::USART0::getInstance();
00022     }
00023     a_channel &= 0x0F; // Clamp channel 0..15
00024
00025     while (!mp_usart->ready2Send()) {}
00026     mp_usart->sendString("$ADC,");
00027     mp_usart->sendByte(a_channel);
00028     while (!mp_usart->ready2Send()) {}
00029     mp_usart->sendString(",");
00030     mp_usart->sendWord(a_value);
00031     while (!mp_usart->ready2Send()) {}
00032     mp_usart->sendString("\r\n");
00033 }
```

References io::USART0::getInstance(), and mp_usart.

Referenced by main().

8.18.2.3 sendLog()

```
void io::Telemetry::sendLog (
            const char * ap_msg)  [static]
```

Send custom log string

Parameters

ap_msg	Text message

Definition at line 72 of file Telemetry.cpp.

```
00073 {
00074     if (!mp_usart) {
00075         mp_usart = &io::USART0::getInstance();
00076     }
00077     if (!ap_msg) {
00078         ap_msg = "NULL";
00079     }
00080     while (!mp_usart->ready2Send()) {}
00081     mp_usart->sendString("$LOG,");
00082     while (!mp_usart->ready2Send()) {}
00083     mp_usart->sendString(ap_msg);
00084     while (!mp_usart->ready2Send()) {}
00085     mp_usart->sendString("\r\n");
00086 }
```

References io::USART0::getInstance(), and mp_usart.

Referenced by main().

8.18.2.4 sendMotor()

```
void io::Telemetry::sendMotor (
            uint16_t a_pos,
            uint16_t a_speed)  [static]
```

Send Motor position & speed

Parameters

a_pos	Current step or pulse position
a_speed	Current speed

Definition at line 57 of file Telemetry.cpp.

```
00058 {
00059     if (!mp_usart) {
00060         mp_usart = &io::USART0::getInstance();
00061     }
00062     while (!mp_usart->ready2Send()) {}
00063     mp_usart->sendString("$MOTOR,");
00064     mp_usart->sendWord(a_pos);
00065     while (!mp_usart->ready2Send()) {}
00066     mp_usart->sendString(",");
00067     mp_usart->sendWord(a_speed);
00068     while (!mp_usart->ready2Send()) {}
00069     mp_usart->sendString("\r\n");
00070 }
```

References io::USART0::getInstance(), and mp_usart.

Referenced by main().

8.18.2.5 sendPin()

```
void io::Telemetry::sendPin (
            uint8_t a_port,
            uint8_t a_pin,
            uint8_t a_state)  [static]
```

Send Pin state

Parameters

a_port	Port index (0=PortB, 1=PortC, 2=PortD)
a_pin	Pin index (0..7)
a_state	Logic level (0 or 1)

Definition at line 35 of file Telemetry.cpp.

```
00036 {
00037     if (!mp_usart) {
00038         mp_usart = &io::USART0::getInstance();
00039     }
00040     a_port %= 3;      // Clamp port 0..2
00041     a_pin &= 0x07;    // Clamp pin 0..7
00042     a_state = a_state ? 1 : 0;
00043
00044     while (!mp_usart->ready2Send()) {}
00045     mp_usart->sendString("$PIN,");
00046     mp_usart->sendByte(a_port);
00047     while (!mp_usart->ready2Send()) {}
00048     mp_usart->sendString(",");
00049     mp_usart->sendByte(a_pin);
00050     while (!mp_usart->ready2Send()) {}
00051     mp_usart->sendString(",");
00052     mp_usart->sendByte(a_state);
00053     while (!mp_usart->ready2Send()) {}
00054     mp_usart->sendString("\r\n");
00055 }
```

References io::USART0::getInstance(), and mp_usart.

Referenced by main().

8.18.3 Member Data Documentation

8.18.3.1 mp_usart

```
io::USART0 * io::Telemetry::mp_usart = nullptr  [static], [private]
```

Definition at line 65 of file Telemetry.h.

Referenced by init(), sendADC(), sendLog(), sendMotor(), and sendPin().

8.19 core::TimerCounter Class Reference

```
#include <TimerCounter.h>
```

Public Member Functions

- virtual ~TimerCounter ()
- virtual void selectOperationMode (const operationMode &ar_operationMode)=0
- virtual void selectClockSource (const clockSource &ar_clockSource)=0
- virtual void start ()=0
- virtual void stop ()=0
- virtual void selectCompareOutputMode (const channel &ar_channel, const compareOutputMode &ar_↩ compareOutputMode)=0
- virtual void setCounter (const uint16_t &ar_dataBuffer)=0
- virtual uint16_t getCounter () const =0
- virtual void setOutputCompareRegister (const channel &ar_channel, const uint16_t &ar_dataBuffer)=0
- virtual uint16_t getOutputCompareRegister (const channel &ar_channel) const =0
- virtual void enableOutputCompareMatchInterrupt (const channel &ar_channel, const uint8_t a_enable)=0
- virtual void enableOverflowInterrupt (const uint8_t a_enable)=0
- virtual uint16_t getClockPrescaler ()=0

Static Public Member Functions

- static void operator delete (void *)

8.19.1 Detailed Description

Definition at line 517 of file TimerCounter.h.

8.19.2 Constructor & Destructor Documentation

8.19.2.1 ~TimerCounter()

```
virtual core::TimerCounter::~TimerCounter ()  [inline], [virtual]
```

Virtual destructor so subclass objects can be deleted through a TimerCounter pointer.

Definition at line 526 of file TimerCounter.h.

```
00526 {}
```

8.19.3 Member Function Documentation

8.19.3.1 enableOutputCompareMatchInterrupt()

```
virtual void core::TimerCounter::enableOutputCompareMatchInterrupt (
            const channel & ar_channel,
            const uint8_t a_enable)  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.2 enableOverflowInterrupt()

```
virtual void core::TimerCounter::enableOverflowInterrupt (
            const uint8_t a_enable)  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.3 getClockPrescaler()

```
virtual uint16_t core::TimerCounter::getClockPrescaler ()  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.4 getCounter()

```
virtual uint16_t core::TimerCounter::getCounter () const  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.5 getOutputCompareRegister()

```
virtual uint16_t core::TimerCounter::getOutputCompareRegister (
            const channel & ar_channel) const  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.6 operator delete()

```
void core::TimerCounter::operator delete (
            void * )  [inline], [static]
```

Freestanding C++ on AVR has no libstdc++ and therefore no global operator delete, which the deleting destructor above would otherwise reference and fail to link. Nothing in this library is heap-allocated (there is no operator new either), so the body is empty.

Definition at line 535 of file TimerCounter.h.

```
00535 {}
```

8.19.3.7 selectClockSource()

```
virtual void core::TimerCounter::selectClockSource (
            const clockSource & ar_clockSource)  [pure virtual]
```

Select the clock source (prescaler).

Each timer supports a subset of core::clockSource. A value the timer does not support is a programming error: the timer falls back to the stopped state (noClock) instead of keeping the previously selected source.

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.8 selectCompareOutputMode()

```
virtual void core::TimerCounter::selectCompareOutputMode (
            const channel & ar_channel,
            const compareOutputMode & ar_compareOutputMode)  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.9 selectOperationMode()

```
virtual void core::TimerCounter::selectOperationMode (
            const operationMode & ar_operationMode)  [pure virtual]
```

Select the operation mode.

Each timer supports a subset of core::operationMode. A value the timer does not support is a programming error: the timer falls back to normal mode instead of keeping the previously programmed mode.

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.10 setCounter()

```
virtual void core::TimerCounter::setCounter (
            const uint16_t & ar_dataBuffer)  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.11 setOutputCompareRegister()

```
virtual void core::TimerCounter::setOutputCompareRegister (
            const channel & ar_channel,
            const uint16_t & ar_dataBuffer)  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.12 start()

```
virtual void core::TimerCounter::start ()  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.19.3.13 stop()

```
virtual void core::TimerCounter::stop ()  [pure virtual]
```

Implemented in core::TimerCounter0, core::TimerCounter1, and core::TimerCounter2.

8.20 core::TimerCounter0 Class Reference

`#include <TimerCounter0.h>`

Public Member Functions

- void selectOperationMode (const operationMode &ar_operationMode) override
- void start () override
- void stop () override
- void selectClockSource (const clockSource &ar_clockSource) override
- void selectCompareOutputMode (const channel &ar_channel, const compareOutputMode &ar_compare↩ OutputMode) override
- void setCounter (const uint16_t &ar_dataBuffer) override
- uint16_t getCounter () const override
- void setOutputCompareRegister (const channel &ar_channel, const uint16_t &ar_dataBuffer) override
- uint16_t getOutputCompareRegister (const channel &ar_channel) const override
- void enableOutputCompareMatchInterrupt (const channel &ar_channel, const uint8_t a_enable) override
- void enableOverflowInterrupt (const uint8_t a_enable) override
- uint16_t getClockPrescaler () override

Public Member Functions inherited from core::TimerCounter

- virtual ~TimerCounter ()

Static Public Member Functions

- static TimerCounter0 & getInstance (const channel &ar_channel=channel::A, const operationMode &ar↩ _operationMode=operationMode::normal, const clockSource &ar_clockSource=clockSource::noClock, const compareOutputMode &ar_compareOutputMode=compareOutputMode::normal)
- static void outputCompareMatchAServiceRoutine () __asm__(STR(TIMER0_COM_CHANNEL_A_INTERRUPT)) __attribute__((__signal__
- static void outputCompareMatchBServiceRoutine () __asm__(STR(TIMER0_COM_CHANNEL_B_INTERRUPT)) __attribute__((__signal__
- static void overflowServiceRoutine () __asm__(STR(TIMER0_OVERFLOW_INTERRUPT)) __attribute__((↩ __signal__

Static Public Member Functions inherited from core::TimerCounter

- static void operator delete (void *)

Public Attributes

- static void __used__
- static void __externally_visible__
- static void weak

Private Member Functions

- TimerCounter0 (const channel &ar_channel, const operationMode &ar_operationMode, const clockSource &ar_clockSource, const compareOutputMode &ar_compareOutputMode)
- ~TimerCounter0 ()
- TimerCounter0 (const TimerCounter0 &)
- const TimerCounter0 & operator= (const TimerCounter0 &)

Private Attributes

- uint16_t m_clockPrescaler
- uint8_t m_clockSource

8.20.1 Detailed Description

Definition at line 279 of file TimerCounter0.h.

8.20.2 Constructor & Destructor Documentation

8.20.2.1 TimerCounter0() [1/2]

```
core::TimerCounter0::TimerCounter0 (
            const channel & ar_channel,
            const operationMode & ar_operationMode,
            const clockSource & ar_clockSource,
            const compareOutputMode & ar_compareOutputMode)  [private]
```

Definition at line 20 of file TimerCounter0.cpp.

```
00024 {
00025     core::MCU::enableTimerCounter0(1);
00026     stop();
00027     selectOperationMode(ar_operationMode);
00028     selectClockSource(ar_clockSource);
00029     selectCompareOutputMode(ar_channel,ar_compareOutputMode);
00030
00031
00032
00033 }
```

References core::MCU::enableTimerCounter0(), selectClockSource(), selectCompareOutputMode(), selectOperationMode(), and stop().

Referenced by getInstance(), operator=(), and TimerCounter0().

8.20.2.2 ~TimerCounter0()

```
core::TimerCounter0::~TimerCounter0 ()  [private]
```

Definition at line 34 of file TimerCounter0.cpp.

```
00035 {
00036
00037 }
```

8.20.2.3 TimerCounter0() [2/2]

```
core::TimerCounter0::TimerCounter0 (
            const TimerCounter0 & )  [private]
```

Overried Copy constructor.

References TimerCounter0().

8.20.3 Member Function Documentation

8.20.3.1 enableOutputCompareMatchInterrupt()

```
void core::TimerCounter0::enableOutputCompareMatchInterrupt (
            const channel & ar_channel,
            const uint8_t a_enable)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 235 of file TimerCounter0.cpp.

```
00236 {
00237     switch (ar_channel)
00238     {
00239         case core::channel::A:
00240         {
00241             if (a_enable) {
00242 
00243                 TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT;
00244 
00245             } else {
00246 
00247                 TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT;
00248             }
00249             break;
00250         }
00251         case core::channel::B:
00252         {
00253             if (a_enable) {
00254 
00255                 TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT;
00256 
00257             } else {
00258 
00259                 TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT;
00260 
00261             }
00262             break;
00263         }
00264     }
00265 
00266 }
```

References core::A, core::B, TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT, TIMER0_DISABLE_COM_CHANNEL_B_INTEF
TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT, and TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT.

Referenced by main().

8.20.3.2 enableOverflowInterrupt()

```
void core::TimerCounter0::enableOverflowInterrupt (
            const uint8_t a_enable)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 270 of file TimerCounter0.cpp.

```
00271 {
00272
00273     if (a_enable) {
00274
00275         TIMER0_ENABLE_OVERFLOW_INTERRUPT;
00276
00277     } else {
00278
00279         TIMER0_DISABLE_OVERFLOW_INTERRUPT;
00280     }
00281
00282 }
```

References TIMER0_DISABLE_OVERFLOW_INTERRUPT, and TIMER0_ENABLE_OVERFLOW_INTERRUPT.

Referenced by main().

8.20.3.3 getClockPrescaler()

```
uint16_t core::TimerCounter0::getClockPrescaler ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 156 of file TimerCounter0.cpp.

```
00157 {
00158     return m_clockPrescaler;
00159 }
```

References m_clockPrescaler.

8.20.3.4 getCounter()

```
uint16_t core::TimerCounter0::getCounter () const  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 191 of file TimerCounter0.cpp.

```
00192 {
00193     return TCNT0;
00194 }
```

8.20.3.5 getInstance()

```
core::TimerCounter0 & core::TimerCounter0::getInstance (
            const channel & ar_channel = channel::A,
            const operationMode & ar_operationMode = operationMode::normal,
            const clockSource & ar_clockSource = clockSource::noClock,
            const compareOutputMode & ar_compareOutputMode = compareOutputMode::normal)  [static]
```

Definition at line 5 of file TimerCounter0.cpp.

```
00009 {
00010     static TimerCounter0 l_instance(ar_channel,
00011                                     ar_operationMode,
00012                                     ar_clockSource,
00013                                     ar_compareOutputMode);
00014
00015     return l_instance;
00016
00017
00018 }
```

References TimerCounter0().

Referenced by main().

8.20.3.6 getOutputCompareRegister()

```
uint16_t core::TimerCounter0::getOutputCompareRegister (
            const channel & ar_channel) const  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 216 of file TimerCounter0.cpp.

```
00217 {
00218     uint16_t compareRegister = 0;
00219     switch (ar_channel)
00220     {
00221         case core::channel::A:
00222         {
00223             compareRegister = OCR0A;
00224             break;
00225         }
00226         case core::channel::B:
00227         {
00228             compareRegister = OCR0B;
00229             break;
00230         }
00231     }
00232     return compareRegister;
00233 }
```

References core::A, and core::B.

8.20.3.7 operator=()

```
const TimerCounter0 & core::TimerCounter0::operator= (
            const TimerCounter0 & )  [private]
```

Override assign operator.

References TimerCounter0().

8.20.3.8 outputCompareMatchAServiceRoutine()

```
void core::TimerCounter0::outputCompareMatchAServiceRoutine ()  [static]
```

Definition at line 287 of file TimerCounter0.cpp.

```
00287 {}
```

8.20.3.9 outputCompareMatchBServiceRoutine()

```
void core::TimerCounter0::outputCompareMatchBServiceRoutine ()  [static]
```

Definition at line 289 of file TimerCounter0.cpp.

```
00289 {}
```

8.20.3.10 overflowServiceRoutine()

```
void core::TimerCounter0::overflowServiceRoutine ()  [static]
```

Definition at line 291 of file TimerCounter0.cpp.

```
00291 {}
```

8.20.3.11 selectClockSource()

```
void core::TimerCounter0::selectClockSource (
            const clockSource & ar_clockSource)  [override], [virtual]
```

Select the clock source (prescaler).

Each timer supports a subset of core::clockSource. A value the timer does not support is a programming error: the timer falls back to the stopped state (noClock) instead of keeping the previously selected source.

Implements core::TimerCounter.

Definition at line 39 of file TimerCounter0.cpp.

```
00040 {
00041     switch (ar_clockSource)
00042     {
00043         case core::clockSource::noClock:
00044         {
00045             m_clockPrescaler=0;
00046             m_clockSource=0;
00047             break;
00048         }
00049         case core::clockSource::PS_1:
00050         {
00051             m_clockPrescaler=1;
00052             m_clockSource=1;
00053             break;
00054         }
00055         case core::clockSource::PS_8:
00056         {
00057             m_clockPrescaler=8;
00058             m_clockSource=2;
00059             break;
00060         }
00061         case core::clockSource::PS_64:
00062         {
00063             m_clockPrescaler=64;
00064             m_clockSource=3;
00065             break;
00066         }
00067         case core::clockSource::PS_256:
00068         {
00069             m_clockPrescaler=256;
00070             m_clockSource=4;
00071             break;
00072         }
00073         case core::clockSource::PS_1024:
00074         {
00075             m_clockPrescaler=1024;
00076             m_clockSource=5;
00077             break;
00078         }
00079         case core::clockSource::extern_Clock_T0_Falling_Edge:
00080         {
00081             m_clockPrescaler=0;
00082             m_clockSource=6;
00083             break;
00084         }
00085         case core::clockSource::extern_Clock_T0_Rising_Edge:
00086         {
00087             m_clockPrescaler=0;
00088             m_clockSource=7;
00089             break;
00090         }
00091         default:
00092         {
00093             // unsupported on this timer (e.g. PS_32, PS_128): stop the
00094             // clock instead of keeping the previously selected source
00095             m_clockPrescaler=0;
00096             m_clockSource=0;
00097             break;
00098         }
00099     }
00100
00101
00102 }
```

References core::extern_Clock_T0_Falling_Edge, core::extern_Clock_T0_Rising_Edge, m_clockPrescaler, m_clockSource, core::noClock, core::PS_1, core::PS_1024, core::PS_256, core::PS_64, and core::PS_8.

Referenced by main(), and TimerCounter0().

8.20.3.12 selectCompareOutputMode()

```
void core::TimerCounter0::selectCompareOutputMode (
            const channel & ar_channel,
            const compareOutputMode & ar_compareOutputMode)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 167 of file TimerCounter0.cpp.

```
00168 {
00169     switch (ar_channel)
00170     {
00171         case core::channel::A:
00172         {
00173             TIMER0_SELECT_COM_CHANNEL_A(static_cast<uint8_t>(ar_compareOutputMode));
00174             break;
00175         }
00176         case core::channel::B:
00177         {
00178             TIMER0_SELECT_COM_CHANNEL_B(static_cast<uint8_t>(ar_compareOutputMode));
00179             break;
00180         }
00181     }
00182 }
```

References core::A, core::B, TIMER0_SELECT_COM_CHANNEL_A, and TIMER0_SELECT_COM_CHANNEL_B.

Referenced by TimerCounter0().

8.20.3.13 selectOperationMode()

```
void core::TimerCounter0::selectOperationMode (
            const operationMode & ar_operationMode)  [override], [virtual]
```

Select the operation mode.

Each timer supports a subset of core::operationMode. A value the timer does not support is a programming error: the timer falls back to normal mode instead of keeping the previously programmed mode.

Implements core::TimerCounter.

Definition at line 104 of file TimerCounter0.cpp.

```
00105 {
00106     switch (ar_operationMode)
00107     {
00108         case core::operationMode::normal:
00109         {
00110             TIMER0_SELECT_OPERATION_MODE(0);
00111             break;
00112         }
00113         case core::operationMode::PWM_PC:
00114         {
00115             TIMER0_SELECT_OPERATION_MODE(1);
00116             break;
00117         }
00118         case core::operationMode::CTC_OCR:
00119         {
00120             TIMER0_SELECT_OPERATION_MODE(2);
00121             break;
00122         }
00123         case core::operationMode::fast_PWM:
00124         {
00125             TIMER0_SELECT_OPERATION_MODE(3);
00126             break;
00127         }
00128         case core::operationMode::PWM_PC_OCR:
00129         {
00130             TIMER0_SELECT_OPERATION_MODE(5);
00131             break;
00132         }
```

```
00133         case core::operationMode::fast_PWM_OCR:
00134         {
00135             TIMER0_SELECT_OPERATION_MODE(7);
00136             break;
00137         }
00138         default:
00139         {
00140             // unsupported on this timer: fall back to normal mode instead
00141             // of keeping the previously programmed mode
00142             TIMER0_SELECT_OPERATION_MODE(0);
00143             break;
00144         }
00145
00146     }
00147
00148 }
```

References core::CTC_OCR, core::fast_PWM, core::fast_PWM_OCR, core::normal, core::PWM_PC, core::PWM_PC_OCR, and TIMER0_SELECT_OPERATION_MODE.

Referenced by TimerCounter0().

8.20.3.14 setCounter()

```
void core::TimerCounter0::setCounter (
            const uint16_t & ar_dataBuffer)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 186 of file TimerCounter0.cpp.

```
00187 {
00188     TCNT0 = static_cast<uint8_t>(ar_dataBuffer);
00189 }
```

8.20.3.15 setOutputCompareRegister()

```
void core::TimerCounter0::setOutputCompareRegister (
            const channel & ar_channel,
            const uint16_t & ar_dataBuffer)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 196 of file TimerCounter0.cpp.

```
00197 {
00198     switch (ar_channel)
00199     {
00200         case core::channel::A:
00201         {
00202             OCR0A = static_cast<uint8_t>(ar_dataBuffer);
00203             break;
00204         }
00205         case core::channel::B:
00206         {
00207             OCR0B = static_cast<uint8_t>(ar_dataBuffer);
00208             break;
00209         }
00210
00211     }
00212 }
```

References core::A, and core::B.

8.20.3.16 start()

```
void core::TimerCounter0::start ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 151 of file TimerCounter0.cpp.

```
00152 {
00153     TIMER0_SELECT_CLOCK_SOURCE(m_clockSource);
00154 }
```

References m_clockSource, and TIMER0_SELECT_CLOCK_SOURCE.

8.20.3.17 stop()

```
void core::TimerCounter0::stop ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 161 of file TimerCounter0.cpp.

```
00162 {
00163     TIMER0_STOP;
00164 }
```

References TIMER0_STOP.

Referenced by TimerCounter0().

8.20.4 Member Data Documentation

8.20.4.1 __externally_visible__

```
void core::TimerCounter0::__externally_visible__
```

Definition at line 313 of file TimerCounter0.h.

8.20.4.2 __used__

```
void core::TimerCounter0::__used__
```

Definition at line 313 of file TimerCounter0.h.

8.20.4.3 m_clockPrescaler

```
uint16_t core::TimerCounter0::m_clockPrescaler  [private]
```

Definition at line 340 of file TimerCounter0.h.

Referenced by getClockPrescaler(), and selectClockSource().

8.20.4.4 m_clockSource

```
uint8_t core::TimerCounter0::m_clockSource  [private]
```

Definition at line 342 of file TimerCounter0.h.

Referenced by selectClockSource(), and start().

8.20.4.5 weak

```
void core::TimerCounter0::weak
```

Definition at line 313 of file TimerCounter0.h.

8.21 core::TimerCounter1 Class Reference

```
#include <TimerCounter1.h>
```

Public Member Functions

- void selectOperationMode (const operationMode &ar_operationMode) override
- void start () override
- void stop () override
- void selectClockSource (const clockSource &ar_clockSource) override
- void selectCompareOutputMode (const channel &ar_channel, const compareOutputMode &ar_compare↩ OutputMode) override
- void setCounter (const uint16_t &ar_dataBuffer) override
- uint16_t getCounter () const override
- void setOutputCompareRegister (const channel &ar_channel, const uint16_t &ar_dataBuffer) override
- uint16_t getOutputCompareRegister (const channel &ar_channel) const override
- void setInputCaptureRegister (const uint16_t &ar_dataBuffer)
- uint16_t getInputCaptureRegister () const
- void enableOutputCompareMatchInterrupt (const channel &ar_channel, const uint8_t a_enable) override
- void enableOverflowInterrupt (const uint8_t a_enable) override
- void enableInputCaptureInterrupt (const uint8_t a_enable)
- uint16_t getClockPrescaler () override

Public Member Functions inherited from core::TimerCounter

- virtual ~TimerCounter ()

Static Public Member Functions

- static TimerCounter1 & getInstance (const channel &ar_channel=channel::A, const operationMode &ar↩ _operationMode=operationMode::normal, const clockSource &ar_clockSource=clockSource::noClock, const compareOutputMode &ar_compareOutputMode=compareOutputMode::normal)
- static void outputCompareMatchAServiceRoutine () __asm__(STR(TIMER1_COM_CHANNEL_A_INTERRUPT)) __attribute__((__signal__
- static void outputCompareMatchBServiceRoutine () __asm__(STR(TIMER1_COM_CHANNEL_B_INTERRUPT)) __attribute__((__signal__
- static void overflowServiceRoutine () __asm__(STR(TIMER1_OVERFLOW_INTERRUPT)) __attribute__((↩ __signal__
- static void inputCaptureServiceRoutine () __asm__(STR(TIMER1_INPUT_CAPTURE_INTERRUPT)) __↩ attribute__((__signal__

Static Public Member Functions inherited from core::TimerCounter

- static void operator delete (void *)

Public Attributes

- static void __used__
- static void __externally_visible__
- static void weak

Private Member Functions

- TimerCounter1 (const channel &ar_channel, const operationMode &ar_operationMode, const clockSource &ar_clockSource, const compareOutputMode &ar_compareOutputMode)
- ~TimerCounter1 ()
- TimerCounter1 (const TimerCounter1 &)
- const TimerCounter1 & operator= (const TimerCounter1 &)

Private Attributes

- uint16_t m_clockPrescaler
- uint8_t m_clockSource

8.21.1 Detailed Description

Definition at line 279 of file TimerCounter1.h.

8.21.2 Constructor & Destructor Documentation

8.21.2.1 TimerCounter1() [1/2]

```
core::TimerCounter1::TimerCounter1 (
            const channel & ar_channel,
            const operationMode & ar_operationMode,
            const clockSource & ar_clockSource,
            const compareOutputMode & ar_compareOutputMode)  [private]
```

Definition at line 21 of file TimerCounter1.cpp.

```
00025 {
00026     core::MCU::enableTimerCounter1(1);
00027     stop();
00028     selectOperationMode(ar_operationMode);
00029     selectClockSource(ar_clockSource);
00030     selectCompareOutputMode(ar_channel,ar_compareOutputMode);
00031
00032
00033 }
```

References core::MCU::enableTimerCounter1(), selectClockSource(), selectCompareOutputMode(), selectOperationMode(), and stop().

Referenced by getInstance(), operator=(), and TimerCounter1().

8.21.2.2 ~TimerCounter1()

```
core::TimerCounter1::~TimerCounter1 ()  [private]
```

Definition at line 34 of file TimerCounter1.cpp.

```
00035 {
00036
00037 }
```

8.21.2.3 TimerCounter1() [2/2]

```
core::TimerCounter1::TimerCounter1 (
            const TimerCounter1 & )  [private]
```

Overried Copy constructor.

References TimerCounter1().

8.21.3 Member Function Documentation

8.21.3.1 enableInputCaptureInterrupt()

```
void core::TimerCounter1::enableInputCaptureInterrupt (
            const uint8_t a_enable)
```

Definition at line 349 of file TimerCounter1.cpp.

```
00350 {
00351     if (a_enable) {
00352
00353         TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT;
00354
00355     } else {
00356
00357         TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT;
00358     }
00359 }
```

References TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT, and TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT.

Referenced by main().

8.21.3.2 enableOutputCompareMatchInterrupt()

```
void core::TimerCounter1::enableOutputCompareMatchInterrupt (
            const channel & ar_channel,
            const uint8_t a_enable)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 300 of file TimerCounter1.cpp.

```
00301 {
00302     switch (ar_channel)
00303     {
00304         case core::channel::A:
00305         {
00306             if (a_enable) {
00307
00308                 TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT;
```

```
00309
00310             } else {
00311
00312                 TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT;
00313             }
00314             break;
00315         }
00316         case core::channel::B:
00317         {
00318             if (a_enable) {
00319
00320                 TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT;
00321
00322             } else {
00323
00324                 TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT;
00325
00326             }
00327             break;
00328         }
00329     }
00330
00331 }
```

References core::A, core::B, TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT, TIMER1_DISABLE_COM_CHANNEL_B_INTEF TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT, and TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT.

Referenced by main().

8.21.3.3 enableOverflowInterrupt()

```
void core::TimerCounter1::enableOverflowInterrupt (
             const uint8_t a_enable)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 335 of file TimerCounter1.cpp.

```
00336 {
00337
00338     if (a_enable) {
00339
00340         TIMER1_ENABLE_OVERFLOW_INTERRUPT;
00341
00342     } else {
00343
00344         TIMER1_DISABLE_OVERFLOW_INTERRUPT;
00345     }
00346
00347 }
```

References TIMER1_DISABLE_OVERFLOW_INTERRUPT, and TIMER1_ENABLE_OVERFLOW_INTERRUPT.

Referenced by main().

8.21.3.4 getClockPrescaler()

```
uint16_t core::TimerCounter1::getClockPrescaler ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 202 of file TimerCounter1.cpp.

```
00203 {
00204     return m_clockPrescaler;
00205 }
```

References m_clockPrescaler.

Referenced by component::ServoMotor::connect(), and component::ServoMotor::rotate().

8.21.3.5 getCounter()

```
uint16_t core::TimerCounter1::getCounter () const  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 240 of file TimerCounter1.cpp.

```
00241 {
00242     uint16_t counter;
00243     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00244     {
00245         counter = TCNT1;
00246     }
00247     return counter;
00248 }
```

8.21.3.6 getInputCaptureRegister()

```
uint16_t core::TimerCounter1::getInputCaptureRegister () const
```

Definition at line 370 of file TimerCounter1.cpp.

```
00371 {
00372     uint16_t captureRegister;
00373     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00374     {
00375         captureRegister = ICR1;
00376     }
00377     return captureRegister;
00378 }
```

8.21.3.7 getInstance()

```
core::TimerCounter1 & core::TimerCounter1::getInstance (
             const channel & ar_channel = channel::A,
             const operationMode & ar_operationMode = operationMode::normal,
             const clockSource & ar_clockSource = clockSource::noClock,
             const compareOutputMode & ar_compareOutputMode = compareOutputMode::normal)  [static]
```

Definition at line 6 of file TimerCounter1.cpp.

```
00010 {
00011     static TimerCounter1 l_instance(ar_channel,
00012                                     ar_operationMode,
00013                                     ar_clockSource,
00014                                     ar_compareOutputMode);
00015
00016     return l_instance;
00017
00018
00019 }
```

References TimerCounter1().

Referenced by main().

8.21.3.8 getOutputCompareRegister()

```
uint16_t core::TimerCounter1::getOutputCompareRegister (
            const channel & ar_channel) const  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 275 of file TimerCounter1.cpp.

```
00276 {
00277     uint16_t compareRegister = 0;
00278     switch (ar_channel)
00279     {
00280         case core::channel::A:
00281         {
00282             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00283             {
00284                 compareRegister = OCR1A;
00285             }
00286             break;
00287         }
00288         case core::channel::B:
00289         {
00290             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00291             {
00292                 compareRegister = OCR1B;
00293             }
00294             break;
00295         }
00296     }
00297     return compareRegister;
00298 }
```

References core::A, and core::B.

8.21.3.9 inputCaptureServiceRoutine()

```
void core::TimerCounter1::inputCaptureServiceRoutine ()  [static]
```

Definition at line 389 of file TimerCounter1.cpp.

```
00389 {}
```

8.21.3.10 operator=()

```
const TimerCounter1 & core::TimerCounter1::operator= (
            const TimerCounter1 & )  [private]
```

Override assign operator.

References TimerCounter1().

8.21.3.11 outputCompareMatchAServiceRoutine()

```
void core::TimerCounter1::outputCompareMatchAServiceRoutine ()  [static]
```

Definition at line 383 of file TimerCounter1.cpp.

```
00383 {}
```

8.21.3.12 outputCompareMatchBServiceRoutine()

```
void core::TimerCounter1::outputCompareMatchBServiceRoutine ()  [static]
```

Definition at line 385 of file TimerCounter1.cpp.

```
00385 {}
```

8.21.3.13 overflowServiceRoutine()

```
void core::TimerCounter1::overflowServiceRoutine ()  [static]
```

Definition at line 387 of file TimerCounter1.cpp.

```
00387 {}
```

8.21.3.14 selectClockSource()

```
void core::TimerCounter1::selectClockSource (
            const clockSource & ar_clockSource)  [override], [virtual]
```

Select the clock source (prescaler).

Each timer supports a subset of core::clockSource. A value the timer does not support is a programming error: the timer falls back to the stopped state (noClock) instead of keeping the previously selected source.

Implements core::TimerCounter.

Definition at line 39 of file TimerCounter1.cpp.

```
00040 {
00041     switch (ar_clockSource)
00042     {
00043         case core::clockSource::noClock:
00044         {
00045             m_clockPrescaler=0;
00046             m_clockSource=0;
00047             break;
00048         }
00049         case core::clockSource::PS_1:
00050         {
00051             m_clockPrescaler=1;
00052             m_clockSource=1;
00053             break;
00054         }
00055         case core::clockSource::PS_8:
00056         {
00057             m_clockPrescaler=8;
00058             m_clockSource=2;
00059             break;
00060         }
00061         case core::clockSource::PS_64:
00062         {
00063             m_clockPrescaler=64;
00064             m_clockSource=3;
00065             break;
00066         }
00067         case core::clockSource::PS_256:
00068         {
00069             m_clockPrescaler=256;
00070             m_clockSource=4;
00071             break;
00072         }
00073         case core::clockSource::PS_1024:
00074         {
00075             m_clockPrescaler=1024;
00076             m_clockSource=5;
00077             break;
00078         }
00079         case core::clockSource::extern_Clock_T0_Falling_Edge:
00080         {
```

```
00081             m_clockPrescaler=0;
00082             m_clockSource=6;
00083             break;
00084         }
00085         case core::clockSource::extern_Clock_T0_Rising_Edge:
00086         {
00087             m_clockPrescaler=0;
00088             m_clockSource=7;
00089             break;
00090         }
00091         default:
00092         {
00093             // unsupported on this timer (e.g. PS_32, PS_128): stop the
00094             // clock instead of keeping the previously selected source
00095             m_clockPrescaler=0;
00096             m_clockSource=0;
00097             break;
00098         }
00099     }
00100
00101
00102 }
```

References core::extern_Clock_T0_Falling_Edge, core::extern_Clock_T0_Rising_Edge, m_clockPrescaler, m_clockSource, core::noClock, core::PS_1, core::PS_1024, core::PS_256, core::PS_64, and core::PS_8.

Referenced by main(), and TimerCounter1().

8.21.3.15 selectCompareOutputMode()

```
void core::TimerCounter1::selectCompareOutputMode (
            const channel & ar_channel,
            const compareOutputMode & ar_compareOutputMode)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 213 of file TimerCounter1.cpp.

```
00214 {
00215     switch (ar_channel)
00216     {
00217         case core::channel::A:
00218         {
00219             TIMER1_SELECT_COM_CHANNEL_A(static_cast<uint8_t>(ar_compareOutputMode));
00220             break;
00221         }
00222         case core::channel::B:
00223         {
00224             TIMER1_SELECT_COM_CHANNEL_B(static_cast<uint8_t>(ar_compareOutputMode));
00225             break;
00226         }
00227     }
00228
00229 }
```

References core::A, core::B, TIMER1_SELECT_COM_CHANNEL_A, and TIMER1_SELECT_COM_CHANNEL_B.

Referenced by component::ServoMotor::connect(), component::ServoMotor::disconnect(), and TimerCounter1().

8.21.3.16 selectOperationMode()

```
void core::TimerCounter1::selectOperationMode (
            const operationMode & ar_operationMode)  [override], [virtual]
```

Select the operation mode.

Each timer supports a subset of core::operationMode. A value the timer does not support is a programming error: the timer falls back to normal mode instead of keeping the previously programmed mode.

Implements core::TimerCounter.

Definition at line 104 of file TimerCounter1.cpp.

```
00105 {
00106     switch (ar_operationMode)
00107     {
00108         case core::operationMode::normal:
00109         {
00110             TIMER1_SELECT_OPERATION_MODE(0);
00111             break;
00112         }
00113         case core::operationMode::PWM_PC_8bit:
00114         {
00115             TIMER1_SELECT_OPERATION_MODE(1);
00116             break;
00117         }
00118         case core::operationMode::PWM_PC_9bit:
00119         {
00120             TIMER1_SELECT_OPERATION_MODE(2);
00121             break;
00122         }
00123         case core::operationMode::PWM_PC_10bit:
00124         {
00125             TIMER1_SELECT_OPERATION_MODE(3);
00126             break;
00127         }
00128         case core::operationMode::CTC_OCR:
00129         {
00130             TIMER1_SELECT_OPERATION_MODE(4);
00131             break;
00132         }
00133         case core::operationMode::fast_PWM_8bit:
00134         {
00135             TIMER1_SELECT_OPERATION_MODE(5);
00136             break;
00137         }
00138         case core::operationMode::fast_PWM_9bit:
00139         {
00140             TIMER1_SELECT_OPERATION_MODE(6);
00141             break;
00142         }
00143         case core::operationMode::fast_PWM_10bit:
00144         {
00145             TIMER1_SELECT_OPERATION_MODE(7);
00146             break;
00147         }
00148         case core::operationMode::PWM_PFC_ICR:
00149         {
00150             TIMER1_SELECT_OPERATION_MODE(8);
00151             break;
00152         }
00153         case core::operationMode::PWM_PFC_OCR:
00154         {
00155             TIMER1_SELECT_OPERATION_MODE(9);
00156             break;
00157         }
00158         case core::operationMode::PWM_PC_ICR:
00159         {
00160             TIMER1_SELECT_OPERATION_MODE(10);
00161             break;
00162         }
00163         case core::operationMode::PWM_PC_OCR:
00164         {
00165             TIMER1_SELECT_OPERATION_MODE(11);
00166             break;
00167         }
00168         case core::operationMode::CTC_ICR:
00169         {
00170             TIMER1_SELECT_OPERATION_MODE(12);
00171             break;
00172         }
00173         case core::operationMode::fast_PWM_ICR:
00174         {
00175             TIMER1_SELECT_OPERATION_MODE(14);
00176             break;
00177         }
00178         case core::operationMode::fast_PWM_OCR:
00179         {
00180             TIMER1_SELECT_OPERATION_MODE(15);
00181             break;
00182         }
00183         default:
00184         {
00185             // unsupported on this timer: fall back to normal mode instead
00186             // of keeping the previously programmed mode
00187             TIMER1_SELECT_OPERATION_MODE(0);
```

```
00188             break;
00189         }
00190
00191     }
00192
00193
00194 }
```

References core::CTC_ICR, core::CTC_OCR, core::fast_PWM_10bit, core::fast_PWM_8bit, core::fast_PWM_9bit, core::fast_PWM_ICR, core::fast_PWM_OCR, core::normal, core::PWM_PC_10bit, core::PWM_PC_8bit, core::PWM_PC_9bit, core::PWM_PC_ICR, core::PWM_PC_OCR, core::PWM_PFC_ICR, core::PWM_PFC_OCR, and TIMER1_SELECT_OPERATION_MODE.

Referenced by component::ServoMotor::connect(), and TimerCounter1().

8.21.3.17 setCounter()

```
void core::TimerCounter1::setCounter (
             const uint16_t & ar_dataBuffer)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 232 of file TimerCounter1.cpp.

```
00233 {
00234     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00235     {
00236         TCNT1 = ar_dataBuffer;
00237     }
00238 }
```

Referenced by component::ServoMotor::connect().

8.21.3.18 setInputCaptureRegister()

```
void core::TimerCounter1::setInputCaptureRegister (
             const uint16_t & ar_dataBuffer)
```

Definition at line 361 of file TimerCounter1.cpp.

```
00362 {
00363     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00364     {
00365         ICR1 = ar_dataBuffer;
00366     }
00367 }
```

Referenced by component::ServoMotor::connect().

8.21.3.19 setOutputCompareRegister()

```
void core::TimerCounter1::setOutputCompareRegister (
             const channel & ar_channel,
             const uint16_t & ar_dataBuffer)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 250 of file TimerCounter1.cpp.

```
00251 {
00252     switch (ar_channel)
00253     {
00254         case core::channel::A:
```

```
00255         {
00256             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00257             {
00258                 OCR1A = ar_dataBuffer;
00259             }
00260             break;
00261         }
00262         case core::channel::B:
00263         {
00264             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00265             {
00266                 OCR1B = ar_dataBuffer;
00267             }
00268             break;
00269         }
00270     }
00271 }
```

References core::A, and core::B.

Referenced by component::ServoMotor::rotate().

8.21.3.20 start()

```
void core::TimerCounter1::start ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 197 of file TimerCounter1.cpp.

```
00198 {
00199     TIMER1_SELECT_CLOCK_SOURCE(m_clockSource);
00200 }
```

References m_clockSource, and TIMER1_SELECT_CLOCK_SOURCE.

Referenced by component::ServoMotor::rotate().

8.21.3.21 stop()

```
void core::TimerCounter1::stop ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 207 of file TimerCounter1.cpp.

```
00208 {
00209     TIMER1_STOP;
00210 }
```

References TIMER1_STOP.

Referenced by component::ServoMotor::disconnect(), and TimerCounter1().

8.21.4 Member Data Documentation

8.21.4.1 __externally_visible__

```
void core::TimerCounter1::__externally_visible__
```

Definition at line 320 of file TimerCounter1.h.

8.21.4.2 __used__

```
void core::TimerCounter1::__used__
```

Definition at line 320 of file TimerCounter1.h.

8.21.4.3 m_clockPrescaler

```
uint16_t core::TimerCounter1::m_clockPrescaler  [private]
```

Definition at line 349 of file TimerCounter1.h.

Referenced by getClockPrescaler(), and selectClockSource().

8.21.4.4 m_clockSource

```
uint8_t core::TimerCounter1::m_clockSource  [private]
```

Definition at line 351 of file TimerCounter1.h.

Referenced by selectClockSource(), and start().

8.21.4.5 weak

```
void core::TimerCounter1::weak
```

Definition at line 320 of file TimerCounter1.h.

8.22 core::TimerCounter2 Class Reference

```
#include <TimerCounter2.h>
```

Public Member Functions

- void selectOperationMode (const operationMode &ar_operationMode) override
- void start () override
- void stop () override
- void selectClockSource (const clockSource &ar_clockSource) override
- void selectCompareOutputMode (const channel &ar_channel, const compareOutputMode &ar_compare↩ OutputMode) override
- void setCounter (const uint16_t &ar_dataBuffer) override
- uint16_t getCounter () const override
- void setOutputCompareRegister (const channel &ar_channel, const uint16_t &ar_dataBuffer) override
- uint16_t getOutputCompareRegister (const channel &ar_channel) const override
- void enableOutputCompareMatchInterrupt (const channel &ar_channel, const uint8_t a_enable) override
- void enableOverflowInterrupt (const uint8_t a_enable) override
- uint16_t getClockPrescaler () override

Public Member Functions inherited from core::TimerCounter

- virtual ~TimerCounter ()

Static Public Member Functions

- static TimerCounter2 & getInstance (const channel &ar_channel=channel::A, const operationMode &ar↩ _operationMode=operationMode::normal, const clockSource &ar_clockSource=clockSource::noClock, const compareOutputMode &ar_compareOutputMode=compareOutputMode::normal)
- static void outputCompareMatchAServiceRoutine () __asm__(STR(TIMER2_COM_CHANNEL_A_INTERRUPT)) __attribute__((__signal__
- static void outputCompareMatchBServiceRoutine () __asm__(STR(TIMER2_COM_CHANNEL_B_INTERRUPT)) __attribute__((__signal__
- static void overflowServiceRoutine () __asm__(STR(TIMER2_OVERFLOW_INTERRUPT)) __attribute__((↩ __signal__

Static Public Member Functions inherited from core::TimerCounter

- static void operator delete (void *)

Public Attributes

- static void __used__
- static void __externally_visible__
- static void weak

Private Member Functions

- TimerCounter2 (const channel &ar_channel, const operationMode &ar_operationMode, const clockSource &ar_clockSource, const compareOutputMode &ar_compareOutputMode)
- ~TimerCounter2 ()
- TimerCounter2 (const TimerCounter2 &)
- const TimerCounter2 & operator= (const TimerCounter2 &)

Private Attributes

- uint16_t m_clockPrescaler
- uint8_t m_clockSource

8.22.1 Detailed Description

Definition at line 279 of file TimerCounter2.h.

8.22.2 Constructor & Destructor Documentation

8.22.2.1 TimerCounter2() [1/2]

```
core::TimerCounter2::TimerCounter2 (
            const channel & ar_channel,
            const operationMode & ar_operationMode,
            const clockSource & ar_clockSource,
            const compareOutputMode & ar_compareOutputMode)  [private]
```

Definition at line 20 of file TimerCounter2.cpp.

```
00024 {
00025     core::MCU::enableTimerCounter2(1);
00026     stop();
00027     selectOperationMode(ar_operationMode);
00028     selectClockSource(ar_clockSource);
00029     selectCompareOutputMode(ar_channel, ar_compareOutputMode);
00030
00031 }
```

References core::MCU::enableTimerCounter2(), selectClockSource(), selectCompareOutputMode(), selectOperationMode(), and stop().

Referenced by getInstance(), operator=(), and TimerCounter2().

8.22.2.2 ~TimerCounter2()

```
core::TimerCounter2::~TimerCounter2 ()  [private]
```

Definition at line 32 of file TimerCounter2.cpp.

```
00033 {
00034
00035 }
```

8.22.2.3 TimerCounter2() [2/2]

```
core::TimerCounter2::TimerCounter2 (
            const TimerCounter2 & )  [private]
```

Overried Copy constructor.

References TimerCounter2().

8.22.3 Member Function Documentation

8.22.3.1 enableOutputCompareMatchInterrupt()

```
void core::TimerCounter2::enableOutputCompareMatchInterrupt (
            const channel & ar_channel,
            const uint8_t a_enable)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 234 of file TimerCounter2.cpp.

```
00235 {
00236     switch (ar_channel)
00237     {
00238         case core::channel::A:
00239         {
00240             if (a_enable) {
00241
00242                 TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT;
00243
00244             } else {
00245
00246                 TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT;
00247             }
00248             break;
00249         }
00250         case core::channel::B:
00251         {
00252             if (a_enable) {
00253
00254                 TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT;
00255
00256             } else {
00257
00258                 TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT;
00259
00260             }
00261             break;
00262         }
00263     }
00264
00265 }
```

References core::A, core::B, TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT, TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT, TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT, and TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT.

Referenced by main().

8.22.3.2 enableOverflowInterrupt()

```
void core::TimerCounter2::enableOverflowInterrupt (
            const uint8_t a_enable)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 269 of file TimerCounter2.cpp.

```
00270 {
00271
00272     if (a_enable) {
00273
00274         TIMER2_ENABLE_OVERFLOW_INTERRUPT;
00275
00276     } else {
00277
00278         TIMER2_DISABLE_OVERFLOW_INTERRUPT;
00279     }
00280
00281 }
```

References TIMER2_DISABLE_OVERFLOW_INTERRUPT, and TIMER2_ENABLE_OVERFLOW_INTERRUPT.

Referenced by main().

8.22.3.3 getClockPrescaler()

```
uint16_t core::TimerCounter2::getClockPrescaler ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 153 of file TimerCounter2.cpp.

```
00154 {
00155     return m_clockPrescaler;
00156 }
```

References m_clockPrescaler.

8.22.3.4 getCounter()

```
uint16_t core::TimerCounter2::getCounter () const  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 191 of file TimerCounter2.cpp.

```
00192 {
00193     return TCNT2;
00194 }
```

8.22.3.5 getInstance()

```
core::TimerCounter2 & core::TimerCounter2::getInstance (
            const channel & ar_channel = channel::A,
            const operationMode & ar_operationMode = operationMode::normal,
            const clockSource & ar_clockSource = clockSource::noClock,
            const compareOutputMode & ar_compareOutputMode = compareOutputMode::normal)  [static]
```

Definition at line 5 of file TimerCounter2.cpp.

```
00009 {
00010     static TimerCounter2 l_instance(ar_channel,
00011                                     ar_operationMode,
00012                                     ar_clockSource,
00013                                     ar_compareOutputMode);
00014
00015     return l_instance;
00016
00017
00018 }
```

References TimerCounter2().

Referenced by main().

8.22.3.6 getOutputCompareRegister()

```
uint16_t core::TimerCounter2::getOutputCompareRegister (
            const channel & ar_channel) const  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 214 of file TimerCounter2.cpp.

```
00215 {
00216     uint16_t compareRegister = 0;
00217     switch (ar_channel)
00218     {
00219         case core::channel::A:
00220         {
00221             compareRegister = OCR2A;
00222             break;
00223         }
00224         case core::channel::B:
00225         {
00226             compareRegister = OCR2B;
00227             break;
00228         }
00229     }
00230     return compareRegister;
00231
00232 }
```

References core::A, and core::B.

8.22.3.7 operator=()

```
const TimerCounter2 & core::TimerCounter2::operator= (
            const TimerCounter2 & )  [private]
```

Override assign operator.

References TimerCounter2().

8.22.3.8 outputCompareMatchAServiceRoutine()

```
void core::TimerCounter2::outputCompareMatchAServiceRoutine ()  [static]
```

Definition at line 286 of file TimerCounter2.cpp.

```
00286 {}
```

8.22.3.9 outputCompareMatchBServiceRoutine()

```
void core::TimerCounter2::outputCompareMatchBServiceRoutine ()  [static]
```

Definition at line 288 of file TimerCounter2.cpp.

```
00288 {}
```

8.22.3.10 overflowServiceRoutine()

```
void core::TimerCounter2::overflowServiceRoutine ()  [static]
```

Definition at line 290 of file TimerCounter2.cpp.

```
00290 {}
```

8.22.3.11 selectClockSource()

```
void core::TimerCounter2::selectClockSource (
            const clockSource & ar_clockSource)  [override], [virtual]
```

Select the clock source (prescaler).

Each timer supports a subset of core::clockSource. A value the timer does not support is a programming error: the timer falls back to the stopped state (noClock) instead of keeping the previously selected source.

Implements core::TimerCounter.

Definition at line 37 of file TimerCounter2.cpp.

```
00038 {
00039     switch (ar_clockSource)
00040     {
00041         case core::clockSource::noClock:
00042         {
00043             m_clockPrescaler=0;
00044             m_clockSource=0;
00045             break;
00046         }
00047         case core::clockSource::PS_1:
00048         {
00049             m_clockPrescaler=1;
00050             m_clockSource=1;
00051             break;
00052         }
00053         case core::clockSource::PS_8:
00054         {
00055             m_clockPrescaler=8;
00056             m_clockSource=2;
00057             break;
00058         }
00059         case core::clockSource::PS_32:
00060         {
00061             m_clockPrescaler=32;
00062             m_clockSource=3;
00063             break;
00064         }
00065         case core::clockSource::PS_64:
00066         {
00067             m_clockPrescaler=64;
00068             m_clockSource=4;
00069             break;
00070         }
00071         case core::clockSource::PS_128:
00072         {
00073             m_clockPrescaler=128;
00074             m_clockSource=5;
00075             break;
00076         }
00077         case core::clockSource::PS_256:
00078         {
00079             m_clockPrescaler=256;
00080             m_clockSource=6;
00081             break;
00082         }
00083         case core::clockSource::PS_1024:
00084         {
00085             m_clockPrescaler=1024;
00086             m_clockSource=7;
00087             break;
00088         }
00089         default:
00090         {
00091             // unsupported on this timer (external clock options): stop the
00092             // clock instead of keeping the previously selected source
00093             m_clockPrescaler=0;
00094             m_clockSource=0;
00095             break;
00096         }
00097     }
00098
00099 }
```

References m_clockPrescaler, m_clockSource, core::noClock, core::PS_1, core::PS_1024, core::PS_128, core::PS_256, core::PS_32, core::PS_64, and core::PS_8.

Referenced by main(), and TimerCounter2().

8.22.3.12 selectCompareOutputMode()

```
void core::TimerCounter2::selectCompareOutputMode (
            const channel & ar_channel,
            const compareOutputMode & ar_compareOutputMode)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 165 of file TimerCounter2.cpp.

```
00166 {
00167     switch (ar_channel)
00168     {
00169         case core::channel::A:
00170         {
00171             TIMER2_SELECT_COM_CHANNEL_A(static_cast<uint8_t>(ar_compareOutputMode));
00172             break;
00173         }
00174         case core::channel::B:
00175         {
00176             TIMER2_SELECT_COM_CHANNEL_B(static_cast<uint8_t>(ar_compareOutputMode));
00177             break;
00178         }
00179     }
00180
00181
00182 }
```

References core::A, core::B, TIMER2_SELECT_COM_CHANNEL_A, and TIMER2_SELECT_COM_CHANNEL_B.

Referenced by TimerCounter2().

8.22.3.13 selectOperationMode()

```
void core::TimerCounter2::selectOperationMode (
            const operationMode & ar_operationMode)  [override], [virtual]
```

Select the operation mode.

Each timer supports a subset of core::operationMode. A value the timer does not support is a programming error: the timer falls back to normal mode instead of keeping the previously programmed mode.

Implements core::TimerCounter.

Definition at line 101 of file TimerCounter2.cpp.

```
00102 {
00103     switch (ar_operationMode)
00104     {
00105         case core::operationMode::normal:
00106         {
00107             TIMER2_SELECT_OPERATION_MODE(0);
00108             break;
00109         }
00110         case core::operationMode::PWM_PC:
00111         {
00112             TIMER2_SELECT_OPERATION_MODE(1);
00113             break;
00114         }
00115         case core::operationMode::CTC_OCR:
00116         {
00117             TIMER2_SELECT_OPERATION_MODE(2);
00118             break;
00119         }
00120         case core::operationMode::fast_PWM:
00121         {
00122             TIMER2_SELECT_OPERATION_MODE(3);
00123             break;
00124         }
00125         case core::operationMode::PWM_PC_OCR:
00126         {
00127             TIMER2_SELECT_OPERATION_MODE(5);
```

```
00128             break;
00129         }
00130         case core::operationMode::fast_PWM_OCR:
00131         {
00132             TIMER2_SELECT_OPERATION_MODE(7);
00133             break;
00134         }
00135         default:
00136         {
00137             // unsupported on this timer: fall back to normal mode instead
00138             // of keeping the previously programmed mode
00139             TIMER2_SELECT_OPERATION_MODE(0);
00140             break;
00141         }
00142
00143     }
00144
00145 }
```

References core::CTC_OCR, core::fast_PWM, core::fast_PWM_OCR, core::normal, core::PWM_PC, core::PWM_PC_OCR, and TIMER2_SELECT_OPERATION_MODE.

Referenced by TimerCounter2().

8.22.3.14 setCounter()

```
void core::TimerCounter2::setCounter (
             const uint16_t & ar_dataBuffer)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 186 of file TimerCounter2.cpp.

```
00187 {
00188     TCNT2 = static_cast<uint8_t>(ar_dataBuffer);
00189 }
```

8.22.3.15 setOutputCompareRegister()

```
void core::TimerCounter2::setOutputCompareRegister (
             const channel & ar_channel,
             const uint16_t & ar_dataBuffer)  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 196 of file TimerCounter2.cpp.

```
00197 {
00198     switch (ar_channel)
00199     {
00200         case core::channel::A:
00201         {
00202             OCR2A = static_cast<uint8_t>(ar_dataBuffer);
00203             break;
00204         }
00205         case core::channel::B:
00206         {
00207             OCR2B = static_cast<uint8_t>(ar_dataBuffer);
00208             break;
00209         }
00210     }
00211 }
```

References core::A, and core::B.

8.22.3.16 start()

```
void core::TimerCounter2::start ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 148 of file TimerCounter2.cpp.

```
00149 {
00150     TIMER2_SELECT_CLOCK_SOURCE(m_clockSource);
00151 }
```

References m_clockSource, and TIMER2_SELECT_CLOCK_SOURCE.

8.22.3.17 stop()

```
void core::TimerCounter2::stop ()  [override], [virtual]
```

Implements core::TimerCounter.

Definition at line 159 of file TimerCounter2.cpp.

```
00160 {
00161     TIMER2_STOP;
00162 }
```

References TIMER2_STOP.

Referenced by TimerCounter2().

8.22.4 Member Data Documentation

8.22.4.1 __externally_visible__

```
void core::TimerCounter2::__externally_visible__
```

Definition at line 313 of file TimerCounter2.h.

8.22.4.2 __used__

```
void core::TimerCounter2::__used__
```

Definition at line 313 of file TimerCounter2.h.

8.22.4.3 m_clockPrescaler

```
uint16_t core::TimerCounter2::m_clockPrescaler  [private]
```

Definition at line 338 of file TimerCounter2.h.

Referenced by getClockPrescaler(), and selectClockSource().

8.22.4.4 m_clockSource

```
uint8_t core::TimerCounter2::m_clockSource  [private]
```

Definition at line 340 of file TimerCounter2.h.

Referenced by selectClockSource(), and start().

8.22.4.5 weak

```
void core::TimerCounter2::weak
```

Definition at line 313 of file TimerCounter2.h.

8.23 core::TWI Class Reference

```
#include <TWI.h>
```

Public Member Functions

- void setClock (uint32_t a_clockFreq_hz)
- uint8_t start ()
- void stop ()
- uint8_t write (uint8_t a_data)
- uint8_t readAck ()
- uint8_t readNack ()

Static Public Member Functions

- static TWI & getInstance (uint32_t a_clockFreq_hz=100000UL)

Private Member Functions

- TWI (uint32_t a_clockFreq_hz)
- ~TWI ()
- TWI (const TWI &)
- const TWI & operator= (const TWI &)

8.23.1 Detailed Description

Definition at line 17 of file TWI.h.

8.23.2 Constructor & Destructor Documentation

8.23.2.1 TWI() [1/2]

```
core::TWI::TWI (
            uint32_t a_clockFreq_hz)  [private]
```

Definition at line 20 of file TWI.cpp.

```
00021 {
00022     core::MCU::enableTWI(1);
00023     setClock(a_clockFreq_hz);
00024 }
```

References core::MCU::enableTWI(), and setClock().

Referenced by getInstance(), operator=(), and TWI().

8.23.2.2 ~TWI()

```
core::TWI::~TWI ()  [private]
```

Definition at line 26 of file TWI.cpp.

```
00027 {
00028 }
```

8.23.2.3 TWI() [2/2]

```
core::TWI::TWI (
            const TWI & )  [private]
```

References TWI().

8.23.3 Member Function Documentation

8.23.3.1 getInstance()

```
core::TWI & core::TWI::getInstance (
            uint32_t a_clockFreq_hz = 100000UL)  [static]
```

Get singleton instance of the TWI hardware manager

Parameters

a_clockFreq_hz	Requested I2C clock rate (default 100 kHz)

Definition at line 14 of file TWI.cpp.

```
00015 {
00016     static TWI l_instance(a_clockFreq_hz);
00017     return l_instance;
00018 }
```

References TWI().

8.23.3.2 operator=()

```
const TWI & core::TWI::operator= (
            const TWI & )  [private]
```

References TWI().

8.23.3.3 readAck()

```
uint8_t core::TWI::readAck ()
```

Read byte from I2C bus and send ACK

Returns

Received byte

Definition at line 77 of file TWI.cpp.

```
00078 {
00079     TWCR = (1 « TWINT) | (1 « TWEN) | (1 « TWEA);
00080     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00081     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00082     if (timeout == 0) return 0xFF;
00083     return TWDR;
00084 }
```

References TWI_TIMEOUT_CYCLES.

8.23.3.4 readNack()

```
uint8_t core::TWI::readNack ()
```

Read byte from I2C bus and send NACK

Returns

Received byte

Definition at line 86 of file TWI.cpp.

```
00087 {
00088     TWCR = (1 « TWINT) | (1 « TWEN);
00089     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00090     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00091     if (timeout == 0) return 0xFF;
00092     return TWDR;
00093 }
```

References TWI_TIMEOUT_CYCLES.

8.23.3.5 setClock()

```
void core::TWI::setClock (
            uint32_t a_clockFreq_hz)
```

Initialize or change I2C clock frequency

Parameters

a_clockFreq_hz	Requested I2C clock rate in Hz

Definition at line 30 of file TWI.cpp.

```
00031 {
00032     // Clamp clock frequency to valid range [10 kHz, 400 kHz]
00033     if (a_clockFreq_hz < 10000UL) a_clockFreq_hz = 10000UL;
00034     if (a_clockFreq_hz > 400000UL) a_clockFreq_hz = 400000UL;
00035
00036     if (F_CPU < 16UL * a_clockFreq_hz) {
00037         TWBR = 0;
00038         TWSR = 0;
00039         return;
00040     }
00041
00042     uint32_t twbr_val = ((F_CPU / a_clockFreq_hz) - 16UL) / 2UL;
00043     if (twbr_val > 255UL) twbr_val = 255UL;
00044
00045     TWSR = 0; // Prescaler = 1
00046     TWBR = static_cast<uint8_t>(twbr_val);
00047 }
```

Referenced by TWI().

8.23.3.6 start()

```
uint8_t core::TWI::start ()
```

Transmit START condition on I2C bus

Returns

Status flag (1 = success, 0 = error)

Definition at line 49 of file TWI.cpp.

```
00050 {
00051     TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
00052     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00053     while (!(TWCR & (1 << TWINT)) && --timeout) {}
00054     if (timeout == 0) return 0; // Bus timeout
00055
00056     uint8_t status = TWSR & 0xF8;
00057     return (status == 0x08 || status == 0x10) ? 1 : 0;
00058 }
```

References TWI_TIMEOUT_CYCLES.

8.23.3.7 stop()

```
void core::TWI::stop ()
```

Transmit STOP condition on I2C bus

Definition at line 60 of file TWI.cpp.

```
00061 {
00062     TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN);
00063 }
```

8.23.3.8 write()

```
uint8_t core::TWI::write (
            uint8_t a_data)
```

Write byte to I2C bus

Parameters

a_data	Byte to send

Returns

Status flag (1 = ACK received, 0 = NACK/error)

Definition at line 65 of file TWI.cpp.

```
00066 {
00067     TWDR = a_data;
00068     TWCR = (1 « TWINT) | (1 « TWEN);
00069     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00070     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00071     if (timeout == 0) return 0; // Bus timeout
00072
00073     uint8_t status = TWSR & 0xF8;
00074     return (status == 0x18 || status == 0x28 || status == 0x40) ? 1 : 0;
00075 }
```

References TWI_TIMEOUT_CYCLES.

8.24 io::USART0 Class Reference

Class for handling USART0.

`#include <USART0.h>`

Public Member Functions

- void setBaudRate ()
- void setTransmissionMode (const transmissionMode &ar_transMode)
- void setCommunicationMode (const communicationMode &ar_comMode)
- void setParityMode (const parityMode &ar_parityMode)
- void setFrameSize (const frameSize &ar_frameSize)
- void setStopBit (const stopBit &ar_stopBit)
- void sendFrame (const uint8_t *ap_dataBuffer, const uint8_t a_size)
- void sendString (const char *ap_string)
- void receiveString (const char *ap_string)
- void sendChar (const uint8_t &ar_char)
- void sendByte (const uint8_t &ar_byte)
- void sendWord (const uint16_t &ar_word)
- void sendLong (const uint32_t &ar_long)
- void receiveChar (uint8_t &ar_char)
- void receiveFrame (uint8_t *ap_dataBuffer, const uint8_t a_size)
- void enableTransmitCompleteInterrupt (const uint8_t a_enable)
- void enableReceiveCompleteInterrupt (const uint8_t a_enable)
- uint8_t frameError ()
- uint8_t dataOverrun ()
- uint8_t parityError ()
- uint16_t getNumberBytesReceived ()
- uint16_t getNumberBytesSent ()
- uint8_t ready2Send ()
- void resetNumberBytesReceived ()

Static Public Member Functions

- static USART0 & getInstance (const transmissionMode &ar_transMode=transmissionMode::async, const communicationMode &ar_comMode=communicationMode::duplex, const frameSize &ar_frameSize=frameSize::eightBits, const stopBit &ar_stopBit=stopBit::oneStopBit, const parityMode &ar_parityMode=parityMode::noParity)
- static void enableDataRegisterEmptyInterrupt (const uint8_t a_enable)
- static void receiveCompleteServiceRoutine () __asm__(STR(USART0_RECEIVE_COMPLETE_INTERRUPT)) __attribute__((__signal__
- static void dataRegisterEmptyServiceRoutine () __asm__(STR(USART0_DATA_REGISTER_EMPTY_INTERRUPT)) __attribute__((__signal__
- static void transmitCompleteServiceRoutine () __asm__(STR(USART0_TRANSMIT_COMPLETE_INTERRUPT)) __attribute__((__signal__

Public Attributes

- static void __used__
- static void __externally_visible__

Private Member Functions

- USART0 (const transmissionMode &ar_transMode, const communicationMode &ar_comMode, const frameSize &ar_frameSize, const stopBit &ar_stopBit, const parityMode &ar_parityMode)
- ~USART0 ()
- USART0 (const USART0 &)
- const USART0 & operator= (const USART0 &)

Static Private Attributes

- static volatile uint8_t m_status = 0
- static const uint8_t * mp_data2Send = nullptr
- static uint8_t * mp_data2Receive = nullptr
- static uint16_t m_sizeData2Send = 0
- static volatile uint16_t m_sizeData2Receive = 0
- static volatile uint8_t m_stringReceive = 0
- static volatile uint16_t m_numberBytesReceived = 0
- static volatile uint16_t m_numberBytesSent = 0
- static volatile uint8_t m_ready2Send = 1

8.24.1 Detailed Description

Class for handling USART0.

The class implements USART0 operations

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition at line 98 of file USART0.h.

8.24.2 Constructor & Destructor Documentation

8.24.2.1 USART0() [1/2]

```
io::USART0::USART0 (
            const transmissionMode & ar_transMode,
            const communicationMode & ar_comMode,
            const frameSize & ar_frameSize,
            const stopBit & ar_stopBit,
            const parityMode & ar_parityMode)  [private]
```

Constructor.

Initalizes the USART0 object

Parameters

ar_transMode	Defines transmission mode
ar_comMode	Defines communication mode
ar_frameSize	Defines data frame size
ar_stopBit	Defines number of stop bits
ar_parityMode	Defines parity mode

Definition at line 45 of file USART0.cpp.

```
00051 {
00052     core::MCU::enableUSART0(1);
00053     setBaudRate();
00054     setTransmissionMode(ar_transMode);
00055     setCommunicationMode(ar_comMode);
00056     setParityMode(ar_parityMode);
00057     setFrameSize(ar_frameSize);
00058     setStopBit(ar_stopBit);
00059     enableReceiveCompleteInterrupt(1);
00060 }
```

References enableReceiveCompleteInterrupt(), core::MCU::enableUSART0(), setBaudRate(), setCommunicationMode(), setFrameSize(), setParityMode(), setStopBit(), and setTransmissionMode().

Referenced by getInstance(), operator=(), and USART0().

8.24.2.2 ~USART0()

```
io::USART0::~USART0 ()  [private]
```

Destructor.

Definition at line 62 of file USART0.cpp.

```
00063 {
00064
00065 }
```

8.24.2.3 USART0() [2/2]

```
io::USART0::USART0 (
            const USART0 & )  [private]
```

Overried Copy constructor.

References USART0().

8.24.3 Member Function Documentation

8.24.3.1 dataOverrun()

```
uint8_t io::USART0::dataOverrun ()
```

Is there data overrun in received data

Returns

Status of received data

Definition at line 205 of file USART0.cpp.

```
00206 {
00207     return (m_status & (1 << USART0_DATA_OVERRUN));
00208
00209 }
```

References m_status, and USART0_DATA_OVERRUN.

8.24.3.2 dataRegisterEmptyServiceRoutine()

```
void io::USART0::dataRegisterEmptyServiceRoutine ()  [static]
```

Data register empty ISR

Definition at line 344 of file USART0.cpp.

```
00345 {
00346
00347     if (m_sizeData2Send)
00348     {
00349         USART0_DATA_REGISTER = *mp_data2Send++;
00350         m_sizeData2Send--;
00351         m_numberBytesSent++;
00352
00353     }
00354     else
00355     {
00356         enableDataRegisterEmptyInterrupt(0);
00357         m_ready2Send = 1;
00358
00359     }
00360
00361 }
```

References enableDataRegisterEmptyInterrupt(), m_numberBytesSent, m_ready2Send, m_sizeData2Send, mp_data2Send, and USART0_DATA_REGISTER.

8.24.3.3 enableDataRegisterEmptyInterrupt()

```
void io::USART0::enableDataRegisterEmptyInterrupt (
            const uint8_t a_enable)  [static]
```

Enable data register empty interrupt

Parameters

ar_enable	Indicates if interrupt is enabled

Definition at line 385 of file USART0.cpp.

```
00386 {
00387     if (a_enable) {
00388         USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT;
00389 
00390     } else {
00391         USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT;
00392     }
00393 
00394 }
```

References USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT, and USART0_ENABLE_DATA_REGISTER_EMPTY_INT

Referenced by dataRegisterEmptyServiceRoutine(), sendFrame(), and sendString().

8.24.3.4 enableReceiveCompleteInterrupt()

```
void io::USART0::enableReceiveCompleteInterrupt (
             const uint8_t a_enable)
```

Enable receive complete interrupt

Parameters

ar_enable	Indicates if interrupt is enabled

Definition at line 374 of file USART0.cpp.

```
00375 {
00376     if (a_enable) {
00377         USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT;
00378 
00379     } else {
00380         USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT;
00381     }
00382 
00383 }
```

References USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT, and USART0_ENABLE_RECEIVE_COMPLETE_INTERRUP

Referenced by USART0().

8.24.3.5 enableTransmitCompleteInterrupt()

```
void io::USART0::enableTransmitCompleteInterrupt (
             const uint8_t a_enable)
```

Enable transmit complete interrupt

Parameters

ar_enable	Indicates if interrupt is enabled

Definition at line 364 of file USART0.cpp.

```
00365 {
00366     if (a_enable) {
00367         USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT;
00368     } else {
00369         USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT;
00370     }
00371 
00372 }
```

References USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT, and USART0_ENABLE_TRANSMIT_COMPLETE_INTERR

8.24.3.6 frameError()

```
uint8_t io::USART0::frameError ()
```

Is there frame error in received data

Returns

Status of received data

Definition at line 199 of file USART0.cpp.

```
00200 {
00201     return (m_status & (1 << USART0_FRAME_ERROR));
00202
00203 }
```

References m_status, and USART0_FRAME_ERROR.

8.24.3.7 getInstance()

```
io::USART0 & io::USART0::getInstance (
            const transmissionMode & ar_transMode = transmissionMode::async,
            const communicationMode & ar_comMode = communicationMode::duplex,
            const frameSize & ar_frameSize = frameSize::eightBits,
            const stopBit & ar_stopBit = stopBit::oneStopBit,
            const parityMode & ar_parityMode = parityMode::noParity)  [static]
```

Create a single instance of the USART0 object

Parameters

ar_transMode	Defines transmission mode
ar_comMode	Defines communication mode
ar_frameSize	Defines data frame size
ar_stopBit	Defines number of stop bits
ar_parityMode	Defines parity mode

Note

The parameters are only honored by the first call. Every later call returns the existing instance and ignores its arguments.

Definition at line 28 of file USART0.cpp.

```
00033 {
00034     static USART0 l_instance(ar_transMode,
00035                              ar_comMode,
00036                              ar_frameSize,
00037                              ar_stopBit,
00038                              ar_parityMode);
00039
00040     return l_instance;
00041 }
```

References USART0().

Referenced by main(), io::Telemetry::sendADC(), io::Telemetry::sendLog(), io::Telemetry::sendMotor(), and io::Telemetry::sendPin().

8.24.3.8 getNumberBytesReceived()

```
uint16_t io::USART0::getNumberBytesReceived ()
```

Get number of bytes received

Counts the bytes stored by the receive-complete ISR, not the bytes that arrived on the wire: bytes received while no buffer is armed, or after the armed buffer is full, are dropped and not counted.

Returns

Number of received bytes

Definition at line 406 of file USART0.cpp.

```
00407 {
00408     return m_numberBytesReceived;
00409 }
```

References m_numberBytesReceived.

Referenced by main().

8.24.3.9 getNumberBytesSent()

```
uint16_t io::USART0::getNumberBytesSent ()
```

Get number of bytes sent

Bytes sent by the current transmission; restarted at 0 at the start of each send.

Returns

Number of transmitted bytes

Definition at line 401 of file USART0.cpp.

```
00402 {
00403     return m_numberBytesSent;
00404 }
```

References m_numberBytesSent.

8.24.3.10 operator=()

```
const USART0 & io::USART0::operator= (
            const USART0 & )  [private]
```

Override assign operator.

References USART0().

8.24.3.11 parityError()

```
uint8_t io::USART0::parityError ()
```

Is there partity error in received data

Returns

Status of received data

Definition at line 211 of file USART0.cpp.

```
00212 {
00213     return (m_status & (1 << USART0_PARITY_ERROR));
00214
00215 }
```

References m_status, and USART0_PARITY_ERROR.

8.24.3.12 ready2Send()

```
uint8_t io::USART0::ready2Send ()
```

Is ready to send

Returns

Sender status

Definition at line 419 of file USART0.cpp.

```
00420 {
00421     return m_ready2Send;
00422
00423 }
```

References m_ready2Send.

Referenced by sendFrame(), and sendString().

8.24.3.13 receiveChar()

```
void io::USART0::receiveChar (
            uint8_t & ar_char)
```

Receive character

Arms the receiver for a single byte. The byte is delivered by the receive-complete ISR; poll getNumberBytesReceived() to know when it has arrived.

Parameters

ar_char	Defines character to be received

Note

ar_char must outlive the reception: the ISR stores into it until a byte has been received.

Definition at line 288 of file USART0.cpp.

```
00289 {
00290     receiveFrame(&ar_char, 1);
00291 }
```

References receiveFrame().

Referenced by main().

8.24.3.14 receiveCompleteServiceRoutine()

```
void io::USART0::receiveCompleteServiceRoutine ()  [static]
```

Receive complete ISR

Definition at line 320 of file USART0.cpp.

```
00321 {
00322     m_status = USART0_CONTROL_STATUS_REGISTER;
00323
00324     // always read UDR0 first, even with no buffer armed, so the hardware
00325     // receive register is drained and RXC is cleared
00326     const uint8_t l_data = USART0_DATA_REGISTER;
00327
00328     if ((mp_data2Receive != nullptr) && (m_sizeData2Receive != 0))
00329     {
00330         *mp_data2Receive++ = l_data;
00331         m_sizeData2Receive--;
00332         m_numberBytesReceived++;
00333
00334         if ((m_stringReceive != 0) && (l_data == 0))
00335         {
00336             // NUL terminator stored: string reception complete
00337             m_stringReceive = 0;
00338             m_sizeData2Receive = 0;
00339         }
00340     }
00341
00342 }
```

References m_numberBytesReceived, m_sizeData2Receive, m_status, m_stringReceive, mp_data2Receive, USART0_CONTROL_STATUS_REGISTER, and USART0_DATA_REGISTER.

8.24.3.15 receiveFrame()

```
void io::USART0::receiveFrame (
            uint8_t * ap_dataBuffer,
            const uint8_t a_size)
```

Receive data frame

Arms the receiver for a_size bytes. Bytes are delivered by the receive-complete ISR; poll getNumberBytesReceived() for the number of bytes stored so far.

Parameters

ap_dataBuffer	Defines pointer to receiver buffer
a_size	Defines size of receiver buffer

Note

ap_dataBuffer must outlive the reception: the ISR keeps writing into it until all a_size bytes have been received.

Definition at line 294 of file USART0.cpp.

```
00295 {
00296     // arming is shared with the receive ISR, which runs at any priority
00297     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00298     {
00299         mp_data2Receive = ap_dataBuffer;
00300         m_sizeData2Receive = a_size;
00301         m_stringReceive = 0;
00302     }
00303 }
```

References m_sizeData2Receive, m_stringReceive, and mp_data2Receive.

Referenced by receiveChar().

8.24.3.16 receiveString()

```
void io::USART0::receiveString (
            const char * ap_string)
```

Receive string

Arms the receiver for a NUL-terminated string. Reception stops when a NUL byte is stored, or after 255 bytes if no NUL arrives.

Parameters

ap_string	Defines pointer to string

Note

The signature keeps the const qualifier of the original API, but the ISR writes received bytes into the buffer, so ap_string must point to modifiable storage and must never be a string literal. The buffer must outlive the reception.

Definition at line 305 of file USART0.cpp.

```
00306 {
00307     // the const signature is kept for compatibility, but the receive ISR
00308     // stores into the buffer, so ap_string must be modifiable storage
00309     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00310     {
00311         mp_data2Receive = const_cast<uint8_t*>(reinterpret_cast<const uint8_t*>(ap_string));
00312         // no length parameter: bound the write by the receive counter and
00313         // stop after a NUL byte
00314         m_sizeData2Receive = 255;
00315         m_stringReceive = 1;
00316     }
00317 }
```

References m_sizeData2Receive, m_stringReceive, and mp_data2Receive.

8.24.3.17 resetNumberBytesReceived()

```
void io::USART0::resetNumberBytesReceived ()
```

Reset number of bytes received

Definition at line 411 of file USART0.cpp.

```
00412 {
00413     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00414     {
00415         m_numberBytesReceived = 0;
00416     }
00417 }
```

References m_numberBytesReceived.

Referenced by main().

8.24.3.18 sendByte()

```
void io::USART0::sendByte (
            const uint8_t & ar_byte)
```

Transmit byte

Always transmits 3 characters, zero-padded: sendByte(5) sends "005", sendByte(255) sends "255".

Parameters

ar_byte	Defines byte to be sent

Definition at line 242 of file USART0.cpp.

```
00243 {
00244     static uint8_t l_byte2Send[3];
00245     l_byte2Send[0] = '0' + (ar_byte / 100);
00246     l_byte2Send[1] = '0' + ((ar_byte / 10) % 10);
00247     l_byte2Send[2] = '0' + (ar_byte % 10);
00248     sendFrame(l_byte2Send, 3);
00249
00250 }
```

References sendFrame().

8.24.3.19 sendChar()

```
void io::USART0::sendChar (
            const uint8_t & ar_char)
```

Transmit character

The byte is copied into internal storage, so ar_char does not have to outlive the call.

Parameters

ar_char	Defines character to be sent

Definition at line 279 of file USART0.cpp.

```
00280 {
00281     // copied into internal storage: ar_char does not have to outlive the call
00282     static uint8_t l_char2Send = 0;
00283     l_char2Send = ar_char;
00284     sendFrame(&l_char2Send, 1);
00285
00286 }
```

References sendFrame().

8.24.3.20 sendFrame()

```
void io::USART0::sendFrame (
            const uint8_t * ap_dataBuffer,
            const uint8_t a_size)
```

Transmit data frame

Claims the transmitter and enables the data-register-empty interrupt; ready2Send() is cleared synchronously before the interrupt is enabled, and getNumberBytesSent() is restarted at 0.

Parameters

ap_dataBuffer	Defines pointer to transmitter buffer
a_size	Defines size of transmitter buffer

Note

ap_dataBuffer must outlive the transmission: the transmit ISR reads from it until all a_size bytes have been sent.

Definition at line 217 of file USART0.cpp.

```
00218 {
00219     while (!ready2Send()){};
00220     // claimed here, before the interrupt is enabled, so a second send cannot
00221     // observe a ready transmitter while this one is still running
00222     m_numberBytesSent = 0;
00223     m_sizeData2Send = a_size;
00224     mp_data2Send = ap_dataBuffer;
00225     m_ready2Send = 0;
00226     enableDataRegisterEmptyInterrupt(1);
00227 }
```

References enableDataRegisterEmptyInterrupt(), m_numberBytesSent, m_ready2Send, m_sizeData2Send, mp_data2Send, and ready2Send().

Referenced by sendByte(), sendChar(), sendLong(), and sendWord().

8.24.3.21 sendLong()

```
void io::USART0::sendLong (
            const uint32_t & ar_long)
```

Transmit long word

Always transmits 10 characters, zero-padded: sendLong(5) sends "0000000005".

Parameters

ar_long	Defines long word to be sent

Definition at line 251 of file USART0.cpp.

```
00252 {
00253     static uint8_t l_word2Send[10];
00254     l_word2Send[0] = '0' + (ar_long / 1000000000);
00255     l_word2Send[1] = '0' + ((ar_long / 100000000) % 10);
```

```
00256     l_word2Send[2] = '0' + ((ar_long /  10000000) % 10);
00257     l_word2Send[3] = '0' + ((ar_long /   1000000) % 10);
00258     l_word2Send[4] = '0' + ((ar_long /    100000) % 10);
00259     l_word2Send[5] = '0' + ((ar_long /     10000) % 10);
00260     l_word2Send[6] = '0' + ((ar_long /      1000) % 10);
00261     l_word2Send[7] = '0' + ((ar_long /       100) % 10);
00262     l_word2Send[8] = '0' + ((ar_long /        10) % 10);
00263     l_word2Send[9] = '0' + (ar_long % 10);
00264     sendFrame(l_word2Send, 10);
00265
00266 }
```

References sendFrame().

8.24.3.22 sendString()

```
void io::USART0::sendString (
            const char * ap_string)
```

Transmit string

Parameters

ap_string	Defines pointer to string

Note

ap_string must outlive the transmission (string literals are fine, they live in program storage for the whole program).

Definition at line 230 of file USART0.cpp.

```
00231 {
00232     while (!ready2Send()){};
00233     m_numberBytesSent = 0;
00234     m_sizeData2Send = strlen(ap_string);
00235     mp_data2Send = reinterpret_cast<const uint8_t*>(ap_string);
00236     m_ready2Send = 0;
00237     enableDataRegisterEmptyInterrupt(1);
00238
00239 }
```

References enableDataRegisterEmptyInterrupt(), m_numberBytesSent, m_ready2Send, m_sizeData2Send, mp_data2Send, and ready2Send().

Referenced by main().

8.24.3.23 sendWord()

```
void io::USART0::sendWord (
            const uint16_t & ar_word)
```

Transmit word

Always transmits 5 characters, zero-padded: sendWord(5) sends "00005".

Parameters

ar_word	Defines word to be sent

Definition at line 267 of file USART0.cpp.

```
00268 {
00269     static uint8_t l_word2Send[5];
00270     l_word2Send[0] = '0' + (ar_word / 10000);
00271     l_word2Send[1] = '0' + ((ar_word / 1000) % 10);
00272     l_word2Send[2] = '0' + ((ar_word /  100) % 10);
00273     l_word2Send[3] = '0' + ((ar_word /   10) % 10);
00274     l_word2Send[4] = '0' + (ar_word %    10);
00275     sendFrame(l_word2Send, 5);
00276
00277 }
```

References sendFrame().

8.24.3.24 setBaudRate()

```
void io::USART0::setBaudRate ()
```

Set baud rate.

Definition at line 67 of file USART0.cpp.

```
00068 {
00069     USART0_SET_BAUDRATE_HIGH_REGISTER;
00070     USART0_SET_BAUDRATE_LOW_REGISTER;
00071
00072 }
```

References USART0_SET_BAUDRATE_HIGH_REGISTER, and USART0_SET_BAUDRATE_LOW_REGISTER.

Referenced by USART0().

8.24.3.25 setCommunicationMode()

```
void io::USART0::setCommunicationMode (
            const communicationMode & ar_comMode)
```

Set communication mode

Parameters

ar_comMode	Defines communication mode

Definition at line 99 of file USART0.cpp.

```
00100 {
00101     switch (ar_comMode)
00102     {
00103         case communicationMode::duplex:
00104         {
00105             USART0_ENABLE_TRANSMITTER;
00106             USART0_ENABLE_RECEIVER;
00107             break;
00108         }
00109         case communicationMode::receive:
00110         {
00111             USART0_ENABLE_RECEIVER;
00112             USART0_DISABLE_TRANSMITTER;
00113             break;
```

```
00114         }
00115         case communicationMode::transmit:
00116         {
00117             USART0_ENABLE_TRANSMITTER;
00118             USART0_DISABLE_RECEIVER;
00119             break;
00120         }
00121     }
00122
00123 }
```

References io::duplex, io::receive, io::transmit, USART0_DISABLE_RECEIVER, USART0_DISABLE_TRANSMITTER, USART0_ENABLE_RECEIVER, and USART0_ENABLE_TRANSMITTER.

Referenced by USART0().

8.24.3.26 setFrameSize()

```
void io::USART0::setFrameSize (
             const frameSize & ar_frameSize)
```

Set data frame size

Parameters

ar_frameSize	Defines data frame size

Definition at line 148 of file USART0.cpp.

```
00149 {
00150     switch (ar_frameSize)
00151     {
00152         case frameSize::eightBits:
00153         {
00154             USART0_SET_8BIT_FRAME_SIZE;
00155             break;
00156         }
00157         case frameSize::sevenBits:
00158         {
00159             USART0_SET_7BIT_FRAME_SIZE;
00160             break;
00161         }
00162         case frameSize::sixBits:
00163         {
00164             USART0_SET_6BIT_FRAME_SIZE;
00165             break;
00166         }
00167         case frameSize::fiveBits:
00168         {
00169             USART0_SET_5BIT_FRAME_SIZE;
00170             break;
00171         }
00172         case frameSize::neineBits:
00173         {
00174             USART0_SET_9BIT_FRAME_SIZE;
00175             break;
00176         }
00177     }
00178
00179 }
```

References io::eightBits, io::fiveBits, io::neineBits, io::sevenBits, io::sixBits, USART0_SET_5BIT_FRAME_SIZE, USART0_SET_6BIT_FRAME_SIZE, USART0_SET_7BIT_FRAME_SIZE, USART0_SET_8BIT_FRAME_SIZE, and USART0_SET_9BIT_FRAME_SIZE.

Referenced by USART0().

8.24.3.27 setParityMode()

```
void io::USART0::setParityMode (
            const parityMode & ar_parityMode)
```

Set parity mode in data frame

Parameters

ar_parityMode	Defines parity mode

Definition at line 125 of file USART0.cpp.

```
00126 {
00127     switch (ar_parityMode)
00128     {
00129         case parityMode::noParity:
00130         {
00131             USART0_DISABLE_PARITY_MODE;
00132             break;
00133         }
00134         case parityMode::evenParity:
00135         {
00136             USART0_ENABLE_EVEN_PARITY_MODE;
00137             break;
00138         }
00139         case parityMode::oddParity:
00140         {
00141             USART0_ENABLE_ODD_PARITY_MODE;
00142             break;
00143         }
00144     }
00145
00146 }
```

References io::evenParity, io::noParity, io::oddParity, USART0_DISABLE_PARITY_MODE, USART0_ENABLE_EVEN_PARITY_MOD and USART0_ENABLE_ODD_PARITY_MODE.

Referenced by USART0().

8.24.3.28 setStopBit()

```
void io::USART0::setStopBit (
            const stopBit & ar_stopBit)
```

Set number of stop bits in data frame

Parameters

ar_stopBit	Defines number of stop bits

Definition at line 180 of file USART0.cpp.

```
00181 {
00182     switch (ar_stopBit)
00183     {
00184         case stopBit::oneStopBit:
00185         {
00186             USART0_SET_ONE_STOP_BIT;
00187             break;
00188         }
00189         case stopBit::twoStopBits:
00190         {
00191             USART0_SET_TWO_STOP_BITS;
00192             break;
00193         }
00194     }
00195
00196 }
```

References io::oneStopBit, io::twoStopBits, USART0_SET_ONE_STOP_BIT, and USART0_SET_TWO_STOP_BITS.

Referenced by USART0().

8.24.3.29 setTransmissionMode()

```
void io::USART0::setTransmissionMode (
            const transmissionMode & ar_transMode)
```

Set transnmission mode

Parameters

ar_transMode	Defines transmission mode

Definition at line 74 of file USART0.cpp.

```
00075 {
00076     switch (ar_transMode)
00077     {
00078         case transmissionMode::async:
00079         {
00080             USART0_ENABLE_ASYNC_TRANSMISSION_MODE;
00081             break;
00082         }
00083         case transmissionMode::sync:
00084         {
00085             USART0_DISABLE_DOUBLE_SPEED_MODE;
00086             USART0_ENABLE_SYNC_TRANSMISSION_MODE;
00087             break;
00088         }
00089     case transmissionMode::masterSPI:
00090         {
00091             USART0_ENABLE_MASTER_SPI_MODE;
00092             break;
00093     }
00094     }
00095
00096
00097 }
```

References io::async, io::masterSPI, io::sync, USART0_DISABLE_DOUBLE_SPEED_MODE, USART0_ENABLE_ASYNC_TRANSM
USART0_ENABLE_MASTER_SPI_MODE, and USART0_ENABLE_SYNC_TRANSMISSION_MODE.

Referenced by USART0().

8.24.3.30 transmitCompleteServiceRoutine()

```
void io::USART0::transmitCompleteServiceRoutine ()  [static]
```

Transmit complete ISR

Definition at line 396 of file USART0.cpp.

```
00397 {
00398
00399 }
```

8.24.4 Member Data Documentation

8.24.4.1 __externally_visible__

```
void io::USART0::__externally_visible__
```

Definition at line 337 of file USART0.h.

8.24.4.2 __used__

```
void io::USART0::__used__
```

Definition at line 337 of file USART0.h.

8.24.4.3 m_numberBytesReceived

```
volatile uint16_t io::USART0::m_numberBytesReceived = 0  [static], [private]
```

Nnumber of received bytes

Definition at line 394 of file USART0.h.

Referenced by getNumberBytesReceived(), receiveCompleteServiceRoutine(), and resetNumberBytesReceived().

8.24.4.4 m_numberBytesSent

```
volatile uint16_t io::USART0::m_numberBytesSent = 0  [static], [private]
```

Number of trasnmitted bytes

Definition at line 396 of file USART0.h.

Referenced by dataRegisterEmptyServiceRoutine(), getNumberBytesSent(), sendFrame(), and sendString().

8.24.4.5 m_ready2Send

```
volatile uint8_t io::USART0::m_ready2Send = 1  [static], [private]
```

Ready to send flag

Definition at line 398 of file USART0.h.

Referenced by dataRegisterEmptyServiceRoutine(), ready2Send(), sendFrame(), and sendString().

8.24.4.6 m_sizeData2Receive

```
volatile uint16_t io::USART0::m_sizeData2Receive = 0  [static], [private]
```

Number of bytes the ISR still stores into mp_data2Receive

Definition at line 390 of file USART0.h.

Referenced by receiveCompleteServiceRoutine(), receiveFrame(), and receiveString().

8.24.4.7 m_sizeData2Send

```
uint16_t io::USART0::m_sizeData2Send = 0  [static], [private]
```

Size of data to be transmitted

Definition at line 388 of file USART0.h.

Referenced by dataRegisterEmptyServiceRoutine(), sendFrame(), and sendString().

8.24.4.8 m_status

```
volatile uint8_t io::USART0::m_status = 0  [static], [private]
```

Status of received data

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

See USART0.h for a description of this code

Definition at line 382 of file USART0.h.

Referenced by dataOverrun(), frameError(), parityError(), and receiveCompleteServiceRoutine().

8.24.4.9 m_stringReceive

```
volatile uint8_t io::USART0::m_stringReceive = 0  [static], [private]
```

Receiving a NUL-terminated string: ISR disarms on NUL

Definition at line 392 of file USART0.h.

Referenced by receiveCompleteServiceRoutine(), receiveFrame(), and receiveString().

8.24.4.10 mp_data2Receive

```
uint8_t * io::USART0::mp_data2Receive = nullptr  [static], [private]
```

Pointer to the next free byte of the receiver buffer

Definition at line 386 of file USART0.h.

Referenced by receiveCompleteServiceRoutine(), receiveFrame(), and receiveString().

8.24.4.11 mp_data2Send

```
const uint8_t * io::USART0::mp_data2Send = nullptr  [static], [private]
```

Pointer to transmitter buffer

Definition at line 384 of file USART0.h.

Referenced by dataRegisterEmptyServiceRoutine(), sendFrame(), and sendString().

8.25 core::WatchdogTimer Class Reference

```
#include <WatchdogTimer.h>
```

Public Member Functions

- void selectTimeOut (const timeOut &ar_timeOut)
- void reset ()
- void start (const watchdogOperationMode &ar_operationMode)
- void start (const watchdogOperationMode &ar_operationMode, const timeOut &ar_timeOut)
- void stop ()

Static Public Member Functions

- static WatchdogTimer & getInstance ()
- static void timeOutServiceRoutine () __asm__(STR(WATCHDOG_TIMEOUT_INTERRUPT)) __attribute__↩ ((__signal__

Public Attributes

- static void __used__
- static void __externally_visible__
- static void weak

Private Member Functions

- WatchdogTimer ()
- ~WatchdogTimer ()
- WatchdogTimer (const WatchdogTimer &)
- const WatchdogTimer & operator= (const WatchdogTimer &)

Private Attributes

- uint8_t m_timeOut
- uint8_t m_operationMode
- uint8_t m_isRunning

8.25.1 Detailed Description

Definition at line 119 of file WatchdogTimer.h.

8.25.2 Constructor & Destructor Documentation

8.25.2.1 WatchdogTimer() [1/2]

```
core::WatchdogTimer::WatchdogTimer ()  [private]
```

Definition at line 12 of file WatchdogTimer.cpp.

```
00013 {
00014     stop();
00015 }
```

References stop().

Referenced by getInstance(), operator=(), and WatchdogTimer().

8.25.2.2 ~WatchdogTimer()

```
core::WatchdogTimer::~WatchdogTimer ()  [private]
```

Destructor.

Definition at line 17 of file WatchdogTimer.cpp.

```
00018 {
00019
00020 }
```

8.25.2.3 WatchdogTimer() [2/2]

```
core::WatchdogTimer::WatchdogTimer (
            const WatchdogTimer & )  [private]
```

Overried Copy constructor.

References WatchdogTimer().

8.25.3 Member Function Documentation

8.25.3.1 getInstance()

```
core::WatchdogTimer & core::WatchdogTimer::getInstance ()  [static]
```

Definition at line 5 of file WatchdogTimer.cpp.

```
00006 {
00007     static WatchdogTimer l_instance;
00008     return l_instance;
00009
00010 }
```

References WatchdogTimer().

Referenced by main().

8.25.3.2 operator=()

```
const WatchdogTimer & core::WatchdogTimer::operator= (
            const WatchdogTimer & )  [private]
```

Override assign operator.

References WatchdogTimer().

8.25.3.3 reset()

```
void core::WatchdogTimer::reset ()
```

Definition at line 46 of file WatchdogTimer.cpp.

```
00047 {
00048     wdt_reset();
00049
00050 }
```

8.25.3.4 selectTimeOut()

```
void core::WatchdogTimer::selectTimeOut (
            const timeOut & ar_timeOut)
```

Select the watchdog timeout.

Rule: if the watchdog is already running (start() was called), the timeout change keeps the watchdog running↩ : it preserves the programmed mode bits (WDIE/WDE) and only applies the new timeout. If the watchdog is not running, the timeout is only staged in WDTCSR without enabling it; start() turns it on.

Parameters

ar_timeOut	Defines the timeout

Definition at line 22 of file WatchdogTimer.cpp.

```
00023 {
00024     m_timeOut = static_cast<uint8_t>(ar_timeOut);
00025     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00026     {
00027         wdt_reset();
00028         if (m_isRunning)
00029         {
00030             // watchdog is running: follow the timed change sequence and keep the
00031             // programmed mode bits (WDIE/WDE) set, so the watchdog keeps running
00032             // with the new timeout
00033             WDTCSR |= (1 << WDCE) | (1 << WDE);
00034             WDTCSR = timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)) |
00035                      operationModeToWDTCSR(static_cast<watchdogOperationMode>(m_operationMode));
00036         }
00037         else
00038         {
00039             // watchdog not running: stage the timeout bits only
00040             WATCHDOG_SELECT_TIMEOUT(timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)));
00041         }
00042     }
00043
00044 }
```

References m_isRunning, m_operationMode, m_timeOut, core::operationModeToWDTCSR(), core::timeOutToWDTCSR(), and WATCHDOG_SELECT_TIMEOUT.

8.25.3.5 start() [1/2]

```
void core::WatchdogTimer::start (
            const watchdogOperationMode & ar_operationMode)
```

Definition at line 52 of file WatchdogTimer.cpp.

```
00053 {
00054     m_operationMode = static_cast<uint8_t>(ar_operationMode);
00055     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00056     {
00057         wdt_reset();
00058         WATCHDOG_START(operationModeToWDTCSR(static_cast<watchdogOperationMode>(m_operationMode)),
00059                        timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)));
00060         m_isRunning = 1;
00061     }
00062
00063 }
```

References m_isRunning, m_operationMode, m_timeOut, core::operationModeToWDTCSR(), core::timeOutToWDTCSR(), and WATCHDOG_START.

Referenced by main().

8.25.3.6 start() [2/2]

```
void core::WatchdogTimer::start (
            const watchdogOperationMode & ar_operationMode,
            const timeOut & ar_timeOut)
```

Definition at line 65 of file WatchdogTimer.cpp.

```
00066 {
00067     m_timeOut = static_cast<uint8_t>(ar_timeOut);
00068     m_operationMode = static_cast<uint8_t>(ar_operationMode);
00069     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00070     {
00071         wdt_reset();
00072         WATCHDOG_START(operationModeToWDTCSR(static_cast<watchdogOperationMode>(m_operationMode)),
00073                        timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)));
00074         m_isRunning = 1;
00075     }
00076
00077 }
```

References m_isRunning, m_operationMode, m_timeOut, core::operationModeToWDTCSR(), core::timeOutToWDTCSR(), and WATCHDOG_START.

8.25.3.7 stop()

```
void core::WatchdogTimer::stop ()
```

Definition at line 79 of file WatchdogTimer.cpp.

```
00080 {
00081     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00082     {
00083         wdt_reset();
00084         WATCHDOG_STOP;
00085         m_isRunning = 0;
00086     }
00087
00088 }
```

References m_isRunning, and WATCHDOG_STOP.

Referenced by WatchdogTimer().

8.25.3.8 timeOutServiceRoutine()

```
void core::WatchdogTimer::timeOutServiceRoutine ()  [static]
```

Definition at line 93 of file WatchdogTimer.cpp.

```
00093 {}
```

8.25.4 Member Data Documentation

8.25.4.1 __externally_visible__

```
void core::WatchdogTimer::__externally_visible__
```

Definition at line 147 of file WatchdogTimer.h.

8.25.4.2 __used__

```
void core::WatchdogTimer::__used__
```

Definition at line 147 of file WatchdogTimer.h.

8.25.4.3 m_isRunning

```
uint8_t core::WatchdogTimer::m_isRunning  [private]
```

Watchdog running state, tracked in software

Definition at line 172 of file WatchdogTimer.h.

Referenced by selectTimeOut(), start(), start(), and stop().

8.25.4.4 m_operationMode

```
uint8_t core::WatchdogTimer::m_operationMode  [private]
```

Definition at line 170 of file WatchdogTimer.h.

Referenced by selectTimeOut(), start(), and start().

8.25.4.5 m_timeOut

```
uint8_t core::WatchdogTimer::m_timeOut  [private]
```

Definition at line 168 of file WatchdogTimer.h.

Referenced by selectTimeOut(), start(), and start().

8.25.4.6 weak

```
void core::WatchdogTimer::weak
```

Definition at line 147 of file WatchdogTimer.h.

Chapter 9

File Documentation

9.1 applications/BlinkLed/main.cpp File Reference

```
#include "Led.h"
```

Macros

- #define LED 0
- #define TIMEDELAY 500

Functions

- int main (void)

9.1.1 Detailed Description

Demo example: Blink a Led

Usage demonstration of the TahakomAVRLib in a simple blink a Led example. This example demonstrates the use of the component::Led() and io::Pin() abstraction objects.

- Compiler: avr-gcc (GCC) 5.4.0
- Supported devices: ATmega328P only

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition in file applications/BlinkLed/main.cpp.

9.1.2 Macro Definition Documentation

9.1.2.1 LED

```
#define LED 0
```

Led pin number

Definition at line 19 of file applications/BlinkLed/main.cpp.

Referenced by main().

9.1.2.2 TIMEDELAY

```
#define TIMEDELAY 500
```

Time delay

Definition at line 21 of file applications/BlinkLed/main.cpp.

Referenced by main().

9.1.3 Function Documentation

9.1.3.1 main()

```
int main (
            void )
```

Definition at line 23 of file applications/BlinkLed/main.cpp.

```
00023               {
00024
00025   // Init
00026
00027   // Instantiate a Led object
00028   component::Led Led(io::Pin(LED,io::PortB));
00029
00030   // Mainloop
00031   while (1) {
00032
00033       Led.on();
00034       _delay_ms(TIMEDELAY);
00035
00036       Led.off();
00037       _delay_ms(TIMEDELAY);
00038   }
00039   return 0;
00040 }
```

References LED, component::Led::off(), component::Led::on(), io::PortB, and TIMEDELAY.

9.2 applications/BlinkLed/main.cpp

Go to the documentation of this file.

```
00001
00016
00017 #include "Led.h"
00018
00019 #define LED 0
00020
00021 #define TIMEDELAY 500
00022
00023 int main(void) {
00024
00025   // Init
00026
00027   // Instantiate a Led object
00028   component::Led Led(io::Pin(LED,io::PortB));
00029
00030   // Mainloop
00031   while (1) {
00032
00033       Led.on();
00034       _delay_ms(TIMEDELAY);
00035
00036       Led.off();
00037       _delay_ms(TIMEDELAY);
00038   }
00039   return 0;
00040 }
```

9.3 applications/ESP32_WiFi_Telemetry/main.cpp File Reference

ESP32 Wi-Fi & Serial Telemetry Streaming Application Example.

```
#include "Led.h"
#include "ADC.h"
#include "Telemetry.h"
#include "MCU.h"
```

Functions

- int main (void)

9.3.1 Detailed Description

ESP32 Wi-Fi & Serial Telemetry Streaming Application Example.

Demonstrates basic GPIO output, ADC light sensor reading, and telemetry frame transmission ($ADC, $LOG) on ESP32 using TahakomAVRLib.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file applications/ESP32_WiFi_Telemetry/main.cpp.

9.3.2 Function Documentation

9.3.2.1 main()

```
int main (
            void )
```

Definition at line 17 of file applications/ESP32_WiFi_Telemetry/main.cpp.

```
00018 {
00019     // Initialize MCU hardware abstraction
00020     core::MCU::init();
00021
00022     // Configure status LED on GPIO 2 (NodeMCU onboard LED)
00023     component::Led statusLed(io::Pin(2, io::PortB));
00024     statusLed.on();
00025
00026     // Get singleton instance of USART0 for telemetry framing
00027     io::USART0 &usart = io::USART0::getInstance();
00028     io::Telemetry::init(usart);
00029
00030     io::Telemetry::sendLog("ESP32 TahakomAVRLib Telemetry Node Booted");
00031
00032     // Configure ADC Converter
00033     core::ADConverter &adc = core::ADConverter::getInstance(
00034         core::referenceVoltage::AVCC,
00035         core::clockPrescaler::PS_128,
00036         core::autoTriggerSource::freeRunning,
00037         io::Pin(0, io::PortC)
00038     );
00039     adc.start();
00040
00041     uint16_t sampleCounter = 0;
00042
00043     while (1) {
00044         // Read 10-bit analog light sensor value
00045         uint16_t sensorVal = adc.getConversionResult(core::resolution::res_10bit);
00046
00047         // Send telemetry frame to Web IDE oscilloscope dashboard
00048         io::Telemetry::sendADC(0, sensorVal);
00049
00050         // Toggle LED indicator
00051         statusLed.toggle();
00052
00053         sampleCounter++;
00054         if (sampleCounter % 50 == 0) {
00055             io::Telemetry::sendLog("ESP32 Telemetry Status: OK");
00056         }
00057
00058         _delay_ms(100);
00059     }
00060
00061     return 0;
00062 }
```

References core::AVCC, core::freeRunning, core::ADConverter::getConversionResult(), core::ADConverter::getInstance(), io::USART0::getInstance(), core::MCU::init(), io::Telemetry::init(), component::Led::on(), io::PortB, io::PortC, core::PS_128, core::res_10bit, io::Telemetry::sendADC(), io::Telemetry::sendLog(), core::ADConverter::start(), and component::Led::toggle().

9.4 applications/ESP32_WiFi_Telemetry/main.cpp

Go to the documentation of this file.

```
00001
00011
00012 #include "Led.h"
00013 #include "ADC.h"
00014 #include "Telemetry.h"
00015 #include "MCU.h"
00016
00017 int main(void)
00018 {
00019     // Initialize MCU hardware abstraction
00020     core::MCU::init();
```

```
00021
00022     // Configure status LED on GPIO 2 (NodeMCU onboard LED)
00023     component::Led statusLed(io::Pin(2, io::PortB));
00024     statusLed.on();
00025
00026     // Get singleton instance of USART0 for telemetry framing
00027     io::USART0 &usart = io::USART0::getInstance();
00028     io::Telemetry::init(usart);
00029
00030     io::Telemetry::sendLog("ESP32 TahakomAVRLib Telemetry Node Booted");
00031
00032     // Configure ADC Converter
00033     core::ADConverter &adc = core::ADConverter::getInstance(
00034         core::referenceVoltage::AVCC,
00035         core::clockPrescaler::PS_128,
00036         core::autoTriggerSource::freeRunning,
00037         io::Pin(0, io::PortC)
00038     );
00039     adc.start();
00040
00041     uint16_t sampleCounter = 0;
00042
00043     while (1) {
00044         // Read 10-bit analog light sensor value
00045         uint16_t sensorVal = adc.getConversionResult(core::resolution::res_10bit);
00046
00047         // Send telemetry frame to Web IDE oscilloscope dashboard
00048         io::Telemetry::sendADC(0, sensorVal);
00049
00050         // Toggle LED indicator
00051         statusLed.toggle();
00052
00053         sampleCounter++;
00054         if (sampleCounter % 50 == 0) {
00055             io::Telemetry::sendLog("ESP32 Telemetry Status: OK");
00056         }
00057
00058         _delay_ms(100);
00059     }
00060
00061     return 0;
00062 }
```

9.5 applications/I2C_OLED_Dashboard/main.cpp File Reference

I2C OLED display dashboard application.

```
#include "MCU.h"
#include "TWI.h"
#include "ADC.h"
#include "SSD1306_OLED.h"
```

Functions

- int main (void)

9.5.1 Detailed Description

I2C OLED display dashboard application.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file applications/I2C_OLED_Dashboard/main.cpp.

9.5.2 Function Documentation

9.5.2.1 main()

```
int main (
            void )
```

Definition at line 14 of file applications/I2C_OLED_Dashboard/main.cpp.

```
00014                {
00015     core::MCU::init();
00016
00017     // Initialize ADC on PC0
00018     core::ADConverter &adc = core::ADConverter::getInstance(
00019         core::referenceVoltage::AVCC,
00020         core::clockPrescaler::PS_128,
00021         core::autoTriggerSource::freeRunning,
00022         io::Pin(0, io::PortC)
00023     );
00024     adc.start();
00025
00026     // Initialize SSD1306 OLED over I2C at address 0x3C
00027     component::SSD1306_OLED oled(0x3C, 128, 64);
00028     oled.init();
00029     oled.clear();
00030
00031     oled.setCursor(0, 0);
00032     oled.print("TahakomAVR OLED");
00033
00034     while (1) {
00035         uint16_t val = adc.getConversionResult(core::resolution::res_10bit);
00036
00037         oled.setCursor(0, 2);
00038         oled.print("ADC Ch0: ");
00039
00040         char buf[6];
00041         buf[0] = '0' + (val / 1000);
00042         buf[1] = '0' + ((val / 100) % 10);
00043         buf[2] = '0' + ((val / 10) % 10);
00044         buf[3] = '0' + (val % 10);
00045         buf[4] = '\0';
00046         oled.print(buf);
00047
00048         _delay_ms(200);
00049     }
00050
00051     return 0;
00052 }
```

References core::AVCC, component::SSD1306_OLED::clear(), core::freeRunning, core::ADConverter::getConversionResult(), core::ADConverter::getInstance(), component::SSD1306_OLED::init(), core::MCU::init(), io::PortC, component::SSD1306_OLED::prin core::PS_128, core::res_10bit, component::SSD1306_OLED::setCursor(), and core::ADConverter::start().

9.6 applications/I2C_OLED_Dashboard/main.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "MCU.h"
00010 #include "TWI.h"
00011 #include "ADC.h"
00012 #include "SSD1306_OLED.h"
00013
00014 int main(void) {
00015     core::MCU::init();
00016
00017     // Initialize ADC on PC0
00018     core::ADConverter &adc = core::ADConverter::getInstance(
00019         core::referenceVoltage::AVCC,
00020         core::clockPrescaler::PS_128,
00021         core::autoTriggerSource::freeRunning,
00022         io::Pin(0, io::PortC)
00023     );
00024     adc.start();
```

```
00025
00026     // Initialize SSD1306 OLED over I2C at address 0x3C
00027     component::SSD1306_OLED oled(0x3C, 128, 64);
00028     oled.init();
00029     oled.clear();
00030
00031     oled.setCursor(0, 0);
00032     oled.print("TahakomAVR OLED");
00033
00034     while (1) {
00035         uint16_t val = adc.getConversionResult(core::resolution::res_10bit);
00036
00037         oled.setCursor(0, 2);
00038         oled.print("ADC Ch0: ");
00039
00040         char buf[6];
00041         buf[0] = '0' + (val / 1000);
00042         buf[1] = '0' + ((val / 100) % 10);
00043         buf[2] = '0' + ((val / 10) % 10);
00044         buf[3] = '0' + (val % 10);
00045         buf[4] = '\0';
00046         oled.print(buf);
00047
00048         _delay_ms(200);
00049     }
00050
00051     return 0;
00052 }
```

9.7 applications/LCD_EnvironmentMonitor/main.cpp File Reference

Environment monitor application using PCF8574 I2C LCD and LDR light sensor.

```
#include "MCU.h"
#include "PCF8574_LCD.h"
#include "LDR.h"
#include "ADC.h"
```

Functions

- int main (void)

9.7.1 Detailed Description

Environment monitor application using PCF8574 I2C LCD and LDR light sensor.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file applications/LCD_EnvironmentMonitor/main.cpp.

9.7.2 Function Documentation

9.7.2.1 main()

```
int main (
            void )
```

Definition at line 14 of file applications/LCD_EnvironmentMonitor/main.cpp.

```
00014                {
00015     core::MCU::init();
00016
00017     core::ADConverter &adc = core::ADConverter::getInstance(
00018         core::referenceVoltage::AVCC,
00019         core::clockPrescaler::PS_128,
00020         core::autoTriggerSource::freeRunning,
00021         io::Pin(0, io::PortC)
00022     );
00023     adc.start();
00024
00025     // Instantiate LDR light sensor on PC0
00026     component::LDR ldr(io::Pin(0, io::PortC), adc);
00027
00028     // Instantiate I2C LCD1602 at address 0x27
00029     component::PCF8574_LCD lcd(0x27, 16, 2);
00030     lcd.init();
00031     lcd.clear();
00032
00033     lcd.setCursor(0, 0);
00034     lcd.print("Light Monitor");
00035
00036     while (1) {
00037         uint16_t lux = ldr.getLux(5); // 5 kOhm fixed resistor
00038
00039         lcd.setCursor(0, 1);
00040         lcd.print("Lux: ");
00041
00042         char buf[6];
00043         buf[0] = '0' + (lux / 10000);
00044         buf[1] = '0' + ((lux / 1000) % 10);
00045         buf[2] = '0' + ((lux / 100) % 10);
00046         buf[3] = '0' + ((lux / 10) % 10);
00047         buf[4] = '0' + (lux % 10);
00048         buf[5] = '\0';
00049         lcd.print(buf);
00050
00051         _delay_ms(300);
00052     }
00053
00054     return 0;
00055 }
```

References core::AVCC, component::PCF8574_LCD::clear(), core::freeRunning, core::ADConverter::getInstance(), component::LDR::getLux(), component::PCF8574_LCD::init(), core::MCU::init(), io::PortC, component::PCF8574_LCD::print(), core::PS_128, component::PCF8574_LCD::setCursor(), and core::ADConverter::start().

9.8 applications/LCD_EnvironmentMonitor/main.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "MCU.h"
00010 #include "PCF8574_LCD.h"
00011 #include "LDR.h"
00012 #include "ADC.h"
00013
00014 int main(void) {
00015     core::MCU::init();
00016
00017     core::ADConverter &adc = core::ADConverter::getInstance(
00018         core::referenceVoltage::AVCC,
00019         core::clockPrescaler::PS_128,
00020         core::autoTriggerSource::freeRunning,
00021         io::Pin(0, io::PortC)
```

```
00022     );
00023     adc.start();
00024
00025     // Instantiate LDR light sensor on PC0
00026     component::LDR ldr(io::Pin(0, io::PortC), adc);
00027
00028     // Instantiate I2C LCD1602 at address 0x27
00029     component::PCF8574_LCD lcd(0x27, 16, 2);
00030     lcd.init();
00031     lcd.clear();
00032
00033     lcd.setCursor(0, 0);
00034     lcd.print("Light Monitor");
00035
00036     while (1) {
00037         uint16_t lux = ldr.getLux(5); // 5 kOhm fixed resistor
00038
00039         lcd.setCursor(0, 1);
00040         lcd.print("Lux: ");
00041
00042         char buf[6];
00043         buf[0] = '0' + (lux / 10000);
00044         buf[1] = '0' + ((lux / 1000) % 10);
00045         buf[2] = '0' + ((lux / 100) % 10);
00046         buf[3] = '0' + ((lux / 10) % 10);
00047         buf[4] = '0' + (lux % 10);
00048         buf[5] = '\0';
00049         lcd.print(buf);
00050
00051         _delay_ms(300);
00052     }
00053
00054     return 0;
00055 }
```

9.9 applications/LightMeter/main.cpp File Reference

```
#include "Led.h"
#include "ADC.h"
#include "LDR.h"
#include "utils_m328p.h"
```

Macros

- #define LED0 0
- #define LED1 1
- #define LED2 2
- #define LED3 3
- #define LED4 4
- #define LED5 5
- #define LED6 6
- #define LED7 7
- #define LDRVD 0
- #define LUX_MIN 1
- #define LUX_MAX 1000

Functions

- int main (void)

9.9.1 Detailed Description

Demo example: Light Meter

Usage demonstration of the TahakomAVRLib in an AVR based Light Meter. The example presents an AVR based Light Meter that displays its Lux measurements in a Led bank. This example demonstrates the use of the component::LED(), component::LDR() and core::ADC() abstraction objects.

- Compiler: avr-gcc (GCC) 5.4.0
- Supported devices: ATmega328P only

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition in file applications/LightMeter/main.cpp.

9.9.2 Macro Definition Documentation

9.9.2.1 LDRVD

```
#define LDRVD 0
```

LDR

Definition at line 32 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.2 LED0

```
#define LED0 0
```

Led 0

Definition at line 23 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.3 LED1

```
#define LED1 1
```

Led 1

Definition at line 24 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.4 LED2

```
#define LED2 2
```

Led 2

Definition at line 25 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.5 LED3

```
#define LED3 3
```

Led 3

Definition at line 26 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.6 LED4

```
#define LED4 4
```

Led 4

Definition at line 27 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.7 LED5

```
#define LED5 5
```

Led 5

Definition at line 28 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.8 LED6

```
#define LED6 6
```

Led 6

Definition at line 29 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.9 LED7

```
#define LED7 7
```

Led 7

Definition at line 30 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.10 LUX_MAX

```
#define LUX_MAX 1000
```

Definition at line 40 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.2.11 LUX_MIN

```
#define LUX_MIN 1
```

Definition at line 39 of file applications/LightMeter/main.cpp.

Referenced by main().

9.9.3 Function Documentation

9.9.3.1 main()

```
int main (
             void )
```

Definition at line 42 of file applications/LightMeter/main.cpp.

```
00042                {
00043
00044   // Init
00045
00046   // Instantiate Led objects
00047   component::Led Led[8]={
00048         {io::Pin(LED0,io::PortD)},
00049         {io::Pin(LED1,io::PortD)},
00050         {io::Pin(LED2,io::PortD)},
00051         {io::Pin(LED3,io::PortD)},
00052         {io::Pin(LED4,io::PortD)},
00053         {io::Pin(LED5,io::PortD)},
00054         {io::Pin(LED6,io::PortD)},
00055         {io::Pin(LED7,io::PortD)}};
00056
00057
00058   // Instantiate ADC object
00059   core::ADConverter &myADC = core::ADConverter::getInstance();
00060
00061   // Start conversion
00062   myADC.start();
00063
00064   // Instantiate LDR object
00065   component::LDR LDR(io::Pin(LDRVD,io::PortC),myADC);
00066
00067
00068   // Mainloop
```

```
00069   while (1) {
00070
00071     // one measurement per loop iteration
00072     uint16_t l_lux = LDR.getLux(5);
00073
00074     // clamp into the display range
00075     if (l_lux < LUX_MIN)
00076     {
00077         l_lux = LUX_MIN;
00078     }
00079     if (l_lux > LUX_MAX)
00080     {
00081         l_lux = LUX_MAX;
00082     }
00083
00084     const uint8_t l_numLeds = static_cast<uint8_t>(utils::map(l_lux,
00085                                                               LUX_MIN,
00086                                                               LUX_MAX,
00087                                                               0,
00088                                                               8));
00089
00090     for (uint8_t i=0;i<l_numLeds;i++)
00091     {
00092         Led[i].on();
00093     }
00094
00095     for (uint8_t i=l_numLeds;i<8;i++)
00096     {
00097         Led[i].off();
00098     }
00099
00100   }
00101   return 0;
00102 }
```

References core::ADConverter::getInstance(), component::LDR::getLux(), LDRVD, LED0, LED1, LED2, LED3, LED4, LED5, LED6, LED7, LUX_MAX, LUX_MIN, utils::map(), component::Led::off(), component::Led::on(), io::PortC, io::PortD, and core::ADConverter::start().

9.10 applications/LightMeter/main.cpp

Go to the documentation of this file.

```
00001
00017
00018 #include "Led.h"
00019 #include "ADC.h"
00020 #include "LDR.h"
00021 #include "utils_m328p.h"
00022
00023 #define LED0 0
00024 #define LED1 1
00025 #define LED2 2
00026 #define LED3 3
00027 #define LED4 4
00028 #define LED5 5
00029 #define LED6 6
00030 #define LED7 7
00031
00032 #define LDRVD 0
00033
00034 // Display range of the light meter in lux, from the GL5528-type LDR
00035 // characteristic that LDR::getLux() implements: LUX_MIN is a dimly lit
00036 // room, LUX_MAX is bright indoor lighting close to a window.  Light
00037 // above the range (direct sunlight) saturates the display, all 8 LEDs
00038 // on; light below it switches all LEDs off.
00039 #define LUX_MIN 1
00040 #define LUX_MAX 1000
00041
00042 int main(void) {
00043
00044   // Init
00045
00046   // Instantiate Led objects
00047   component::Led Led[8]={
00048         {io::Pin(LED0,io::PortD)},
00049         {io::Pin(LED1,io::PortD)},
00050         {io::Pin(LED2,io::PortD)},
00051         {io::Pin(LED3,io::PortD)},
00052         {io::Pin(LED4,io::PortD)},
```

```
00053         {io::Pin(LED5,io::PortD)},
00054         {io::Pin(LED6,io::PortD)},
00055         {io::Pin(LED7,io::PortD)}};
00056
00057
00058   // Instantiate ADC object
00059   core::ADConverter &myADC = core::ADConverter::getInstance();
00060
00061   // Start conversion
00062   myADC.start();
00063
00064   // Instantiate LDR object
00065   component::LDR LDR(io::Pin(LDRVD,io::PortC),myADC);
00066
00067
00068   // Mainloop
00069   while (1) {
00070
00071     // one measurement per loop iteration
00072     uint16_t l_lux = LDR.getLux(5);
00073
00074     // clamp into the display range
00075     if (l_lux < LUX_MIN)
00076     {
00077         l_lux = LUX_MIN;
00078     }
00079     if (l_lux > LUX_MAX)
00080     {
00081         l_lux = LUX_MAX;
00082     }
00083
00084     const uint8_t l_numLeds = static_cast<uint8_t>(utils::map(l_lux,
00085                                                               LUX_MIN,
00086                                                               LUX_MAX,
00087                                                               0,
00088                                                               8));
00089
00090     for (uint8_t i=0;i<l_numLeds;i++)
00091     {
00092         Led[i].on();
00093     }
00094
00095     for (uint8_t i=l_numLeds;i<8;i++)
00096     {
00097         Led[i].off();
00098     }
00099
00100   }
00101   return 0;
00102 }
```

9.11 applications/MusicBox/main.cpp File Reference

```
#include "Buzzer.h"
#include "PushButton.h"
#include "TimerCounter1.h"
#include <avr/pgmspace.h>
#include "buzzer_pitches_16bit.h"
```

Macros

- #define BUZZER 2
- #define PUSHBUTTON 1
- #define NOTE_DURATION_MS 300
- #define SONG_LENGTH (sizeof(l_song) / sizeof(uint16_t))

Functions

- int main (void)

9.11.1 Detailed Description

Demo example: AVR Music Box

Usage demonstration of the TahakomAVRLib in an AVR based Music Box. The example allows the user to step through a predefined song in a Buzzer one note at a time via a PushButton; a note plays as long as the button is held. This example demonstrates the use of the component::Buzzer() and component::PushButton() abstraction objects.

- Compiler: avr-gcc (GCC) 5.4.0
- Supported devices: ATmega328P only

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition in file applications/MusicBox/main.cpp.

9.11.2 Macro Definition Documentation

9.11.2.1 BUZZER

```
#define BUZZER 2
```

Buzzer pin number: PB2 is the Timer1 output-compare pin OC1B, the pin the timer-driven buzz() toggles

Definition at line 27 of file applications/MusicBox/main.cpp.

Referenced by main().

9.11.2.2 NOTE_DURATION_MS

```
#define NOTE_DURATION_MS 300
```

Definition at line 34 of file applications/MusicBox/main.cpp.

Referenced by main().

9.11.2.3 PUSHBUTTON

```
#define PUSHBUTTON 1
```

PushButton pin number

Definition at line 30 of file applications/MusicBox/main.cpp.

Referenced by main().

9.11.2.4 SONG_LENGTH

```
#define SONG_LENGTH (sizeof(l_song) / sizeof(uint16_t))
```

Song length

Definition at line 36 of file applications/MusicBox/main.cpp.

Referenced by main().

9.11.3 Function Documentation

9.11.3.1 main()

```
int main (
            void )
```

Definition at line 38 of file applications/MusicBox/main.cpp.

```
00038               {
00039
00040   // Init
00041
00042   // Instantiate a Buzzer object on the Timer1 output-compare pin OC1B:
00043   // the timer-driven buzz() drives the hardware compare output, so the
00044   // buzzer must be wired to the compare pin of the timer in use
00045   component::Buzzer Buzzer(io::Pin(BUZZER,io::PortB));
00046
00047   // Instantiate a TimerCounter1 object: the pitch table holds the note
00048   // periods in us, so the timer is clocked at prescaler 64 where one
00049   // tick is exactly 4 us at F_CPU = 16 MHz and the tick value of a note
00050   // is a quarter of its period in us
00051   core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance(
00052       core::channel::B, core::operationMode::normal, core::clockSource::PS_64);
00053
00054   // Instantiate a PushButton object
00055   component::PushButton PushButton(io::Pin(PUSHBUTTON,io::PortB));
00056
00057   // Song notes: the table holds the note periods in us and lives in
00058   // flash so the song does not consume RAM
00059   const uint16_t l_song[] PROGMEM = {
00060   E6, E6, E6, C6, E6, G6, G5,
00061   C6, G5, E5, A5, B5, Ax5, A5,
00062   G5, E6, G6, A6, F6, G6, E6, C6, D6, B5,
00063   C6, G5, E5, A5, B5, Ax5, A5,
00064   G5, E6, G6, A6, F6, G6, E6, C6, D6, B5,
00065   };
00066
00067   // Start song at start
00068   uint8_t l_note = SONG_LENGTH - 1;
00069
00070   // Initialize PushButton state
00071   uint8_t l_statePushButton = 0;
00072
00073
00074   // Mainloop
00075   while (1) {
00076       // Check if PushButton is pressed
00077       if (PushButton.isPressed())
00078       {
00079           // Check if a new button press
00080           if (!l_statePushButton) {
00081               // play next note
00082               l_note++;
00083               // start from start
00084               if (l_note == SONG_LENGTH) {
00085
00086                   l_note = 0;
00087
00088               }
00089
00090               l_statePushButton = 1;
00091
00092           }
00093           // Play note: tick value = period in us / 4 (4 us per tick at
```

```
00094           // prescaler 64), audibly NOTE_DURATION_MS long
00095           Buzzer.buzz(myTimerCounter1,
00096                       pgm_read_word(&l_song[l_note]) / 4,
00097                       NOTE_DURATION_MS,
00098                       core::channel::B);
00099
00100       } else {
00101
00102           l_statePushButton=0;
00103
00104       }
00105
00106   }
00107   return 0;
00108 }
```

References A5, A6, Ax5, core::B, B5, component::Buzzer::buzz(), BUZZER, C6, D6, E5, E6, F6, G5, G6, core::TimerCounter1::getInstance(), component::PushButton::isPressed(), core::normal, NOTE_DURATION_MS, io::PortB, core::PS_64, PUSHBUTTON, and SONG_LENGTH.

9.12 applications/MusicBox/main.cpp

Go to the documentation of this file.

```
00001
00018
00019 #include "Buzzer.h"
00020 #include "PushButton.h"
00021 #include "TimerCounter1.h"
00022 #include <avr/pgmspace.h>
00023 // Note pitch data
00024 #include "buzzer_pitches_16bit.h"
00025
00026
00027 #define BUZZER 2
00029
00030 #define PUSHBUTTON 1
00031
00032 // Audible length of every played note in ms: the notes of the song are
00033 // audible at this length, one note per button press
00034 #define NOTE_DURATION_MS 300
00035
00036 #define SONG_LENGTH (sizeof(l_song) / sizeof(uint16_t))
00037
00038 int main(void) {
00039
00040   // Init
00041
00042   // Instantiate a Buzzer object on the Timer1 output-compare pin OC1B:
00043   // the timer-driven buzz() drives the hardware compare output, so the
00044   // buzzer must be wired to the compare pin of the timer in use
00045   component::Buzzer Buzzer(io::Pin(BUZZER,io::PortB));
00046
00047   // Instantiate a TimerCounter1 object: the pitch table holds the note
00048   // periods in us, so the timer is clocked at prescaler 64 where one
00049   // tick is exactly 4 us at F_CPU = 16 MHz and the tick value of a note
00050   // is a quarter of its period in us
00051   core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance(
00052       core::channel::B, core::operationMode::normal, core::clockSource::PS_64);
00053
00054   // Instantiate a PushButton object
00055   component::PushButton PushButton(io::Pin(PUSHBUTTON,io::PortB));
00056
00057   // Song notes: the table holds the note periods in us and lives in
00058   // flash so the song does not consume RAM
00059   const uint16_t l_song[] PROGMEM = {
00060   E6, E6, E6, C6, E6, G6, G5,
00061   C6, G5, E5, A5, B5, Ax5, A5,
00062   G5, E6, G6, A6, F6, G6, E6, C6, D6, B5,
00063   C6, G5, E5, A5, B5, Ax5, A5,
00064   G5, E6, G6, A6, F6, G6, E6, C6, D6, B5,
00065   };
00066
00067   // Start song at start
00068   uint8_t l_note = SONG_LENGTH - 1;
00069
00070   // Initialize PushButton state
00071   uint8_t l_statePushButton = 0;
00072
00073
```

```
00074   // Mainloop
00075   while (1) {
00076       // Check if PushButton is pressed
00077       if (PushButton.isPressed())
00078       {
00079           // Check if a new button press
00080           if (!l_statePushButton) {
00081               // play next note
00082               l_note++;
00083               // start from start
00084               if (l_note == SONG_LENGTH) {
00085
00086                   l_note = 0;
00087
00088               }
00089
00090               l_statePushButton = 1;
00091
00092           }
00093           // Play note: tick value = period in us / 4 (4 us per tick at
00094           // prescaler 64), audibly NOTE_DURATION_MS long
00095           Buzzer.buzz(myTimerCounter1,
00096                       pgm_read_word(&l_song[l_note]) / 4,
00097                       NOTE_DURATION_MS,
00098                       core::channel::B);
00099
00100       } else {
00101
00102           l_statePushButton=0;
00103
00104       }
00105
00106   }
00107   return 0;
00108 }
```

9.13 applications/Organ/main.cpp File Reference

```
#include "Buzzer.h"
#include "USART0.h"
#include "MCU.h"
#include <avr/pgmspace.h>
#include "buzzer_pitches_16bit.h"
```

Macros

- #define BUZZER 0
- #define NOTE_DURATION 0xF000

Functions

- int main (void)

9.13.1 Detailed Description

Demo example: AVR Square-Wave Organ

Usage demonstration of the TahakomAVRLib in a Buzzer based and serial driven Square-Wave Organ. The example enables the user to enter a key note via serial terminal from the keyboard and play the corresponding note in a Buzzer This example demonstrates the use of the component::Buzzer() and io::USART0() abstraction objects.

- Compiler: avr-gcc (GCC) 5.4.0
- Supported devices: ATmega328P only

Author

Farid Oubbati (farid.oubbati@outlook.com)

Date

March 2018

Definition in file applications/Organ/main.cpp.

9.13.2 Macro Definition Documentation

9.13.2.1 BUZZER

```
#define BUZZER 0
```

Buzzer pin number

Definition at line 26 of file applications/Organ/main.cpp.

9.13.2.2 NOTE_DURATION

```
#define NOTE_DURATION 0xF000
```

Note duration

Definition at line 28 of file applications/Organ/main.cpp.

Referenced by main().

9.13.3 Function Documentation

9.13.3.1 main()

```
int main (
            void )
```

Definition at line 30 of file applications/Organ/main.cpp.

```
00030               {
00031
00032   // Init
00033
00034   // Instantiate a Buzzer object
00035   component::Buzzer Buzzer(io::Pin(BUZZER,io::PortB));
00036
00037   // Instantiate a USART0 object
00038   io::USART0 &myUSART0 = io::USART0::getInstance();
00039
00040   // The serial receive path uses the receive-complete interrupt, so
00041   // interrupts must be enabled globally
00042   core::MCU::enableInterrupts();
00043
00044   // List of possible keypresses: kept in flash so the lookup tables do
00045   // not consume RAM
00046   const uint8_t l_keys[] PROGMEM = { 'a', 'w', 's', 'e', 'd', 'f', 't',
00047       'g', 'y', 'h', 'j', 'i', 'k', 'o',
00048       'l', 'p', ';', '\"
```

```
00049   };
00050
00051   // List of the keypresses corresponding pitches
00052   const uint16_t l_notes[] PROGMEM = { G4, Gx4, A4, Ax4, B4, C5, Cx5,
00053       D5, Dx5, E5, F5, Fx5, G5, Gx5,
00054       A5, Ax5, B5, C6
00055   };
00056
00057   // Character key from computer serial terminal
00058   unsigned char l_key;
00059
00060   // Current note length
00061   uint16_t l_currentNoteLength = NOTE_DURATION;
00062
00063   // Check if note key
00064   uint8_t l_isNote;
00065
00066   myUSART0.sendString("----- Serial Organ ------\r\n");
00067
00068   // Mainloop
00069   while (1) {
00070
00071       // Send N to signal that AVR chip is ready for next key note input
00072       myUSART0.sendString("Enter key note\r\n");
00073
00074       // Wait for key note input: arm the receiver for a single byte
00075       // once; the receive-complete ISR stores the byte and counts it,
00076       // so the busy-wait polls the counter without re-arming the
00077       // buffer on every iteration
00078       myUSART0.receiveChar(l_key);
00079       while (myUSART0.getNumberBytesReceived()==0)
00080       {
00081       }
00082       myUSART0.resetNumberBytesReceived();
00083
00084       /********** Play key notes **********/
00085
00086       l_isNote = 0;
00087       // Loop through keys table
00088       for (uint8_t i = 0; i < sizeof(l_keys); i++) {
00089           // Found match in lookup table
00090           if (l_key == pgm_read_byte(&l_keys[i])) {
00091               Buzzer.buzz(pgm_read_word(&l_notes[i]), l_currentNoteLength);
00092               // Record that we've found a note
00093               l_isNote = 1;
00094               break;
00095           }
00096       }
00097
00098       // Handle non-note keys: tempo changes and rests
00099       if (!l_isNote) {
00100           if (l_key == '-') {
00101               // Code for short note: halve the length, but reject the
00102               // halving when the result would be zero, a zero length
00103               // would silence every following note
00104               if (l_currentNoteLength > 1)
00105               {
00106                   l_currentNoteLength = l_currentNoteLength / 2;
00107               }
00108           }
00109           else if (l_key == '+') {
00110               // Code for long note: double the length in 32 bits and
00111               // saturate at the 16-bit maximum instead of wrapping
00112               // around to a small or zero length
00113               uint32_t l_newNoteLength =
00114                   static_cast<uint32_t>(l_currentNoteLength) * 2UL;
00115               if (l_newNoteLength > 0xFFFFUL)
00116               {
00117                   l_newNoteLength = 0xFFFFUL;
00118               }
00119               l_currentNoteLength = static_cast<uint16_t>(l_newNoteLength);
00120           }
00121           else {
00122               // Unrecognized, just rest
00123               Buzzer.pause(l_currentNoteLength);
00124           }
00125       }
00126
00127   }
00128   return 0;
00129 }
```

References A4, A5, Ax4, Ax5, B4, B5, component::Buzzer::buzz(), BUZZER, C5, C6, Cx5, D5, Dx5, E5, core::MCU::enableInterrupts(), F5, Fx5, G4, G5, io::USART0::getInstance(), io::USART0::getNumberBytesReceived(), Gx4, Gx5, NOTE_DURATION, component::Buzzer::pause(), io::PortB, io::USART0::receiveChar(), io::USART0::resetNumberBytesRe and io::USART0::sendString().

9.14 applications/Organ/main.cpp

Go to the documentation of this file.

```
00001
00017
00018 #include "Buzzer.h"
00019 #include "USART0.h"
00020 #include "MCU.h"
00021 #include <avr/pgmspace.h>
00022 // Note pitch data
00023 #include "buzzer_pitches_16bit.h"
00024
00025
00026 #define BUZZER 0
00027
00028 #define NOTE_DURATION  0xF000
00029
00030 int main(void) {
00031
00032   // Init
00033
00034   // Instantiate a Buzzer object
00035   component::Buzzer Buzzer(io::Pin(BUZZER,io::PortB));
00036
00037   // Instantiate a USART0 object
00038   io::USART0 &myUSART0 = io::USART0::getInstance();
00039
00040   // The serial receive path uses the receive-complete interrupt, so
00041   // interrupts must be enabled globally
00042   core::MCU::enableInterrupts();
00043
00044   // List of possible keypresses: kept in flash so the lookup tables do
00045   // not consume RAM
00046   const uint8_t l_keys[] PROGMEM = { 'a', 'w', 's', 'e', 'd', 'f', 't',
00047       'g', 'y', 'h', 'j', 'i', 'k', 'o',
00048       'l', 'p', ';', '\"
00049   };
00050
00051   // List of the keypresses corresponding pitches
00052   const uint16_t l_notes[] PROGMEM = { G4, Gx4, A4, Ax4, B4, C5, Cx5,
00053       D5, Dx5, E5, F5, Fx5, G5, Gx5,
00054       A5, Ax5, B5, C6
00055   };
00056
00057   // Character key from computer serial terminal
00058   unsigned char l_key;
00059
00060   // Current note length
00061   uint16_t l_currentNoteLength = NOTE_DURATION;
00062
00063   // Check if note key
00064   uint8_t l_isNote;
00065
00066   myUSART0.sendString("----- Serial Organ ------\r\n");
00067
00068   // Mainloop
00069   while (1) {
00070
00071       // Send N to signal that AVR chip is ready for next key note input
00072       myUSART0.sendString("Enter key note\r\n");
00073
00074       // Wait for key note input: arm the receiver for a single byte
00075       // once; the receive-complete ISR stores the byte and counts it,
00076       // so the busy-wait polls the counter without re-arming the
00077       // buffer on every iteration
00078       myUSART0.receiveChar(l_key);
00079       while (myUSART0.getNumberBytesReceived()==0)
00080       {
00081       }
00082       myUSART0.resetNumberBytesReceived();
00083
00084       /********** Play key notes ***********/
00085
00086       l_isNote = 0;
00087       // Loop through keys table
00088       for (uint8_t i = 0; i < sizeof(l_keys); i++) {
00089           // Found match in lookup table
00090           if (l_key == pgm_read_byte(&l_keys[i])) {
00091               Buzzer.buzz(pgm_read_word(&l_notes[i]), l_currentNoteLength);
00092               // Record that we've found a note
00093               l_isNote = 1;
00094               break;
00095           }
00096       }
00097
```

```
00098       // Handle non-note keys: tempo changes and rests
00099       if (!l_isNote) {
00100           if (l_key == '-') {
00101               // Code for short note: halve the length, but reject the
00102               // halving when the result would be zero, a zero length
00103               // would silence every following note
00104               if (l_currentNoteLength > 1)
00105               {
00106                   l_currentNoteLength = l_currentNoteLength / 2;
00107               }
00108           }
00109           else if (l_key == '+') {
00110               // Code for long note: double the length in 32 bits and
00111               // saturate at the 16-bit maximum instead of wrapping
00112               // around to a small or zero length
00113               uint32_t l_newNoteLength =
00114                   static_cast<uint32_t>(l_currentNoteLength) * 2UL;
00115               if (l_newNoteLength > 0xFFFFUL)
00116               {
00117                   l_newNoteLength = 0xFFFFUL;
00118               }
00119               l_currentNoteLength = static_cast<uint16_t>(l_newNoteLength);
00120           }
00121           else {
00122               // Unrecognized, just rest
00123               Buzzer.pause(l_currentNoteLength);
00124           }
00125       }
00126
00127   }
00128   return 0;
00129 }
```

9.15 applications/PID_MotorControl/main.cpp File Reference

Closed-loop DC motor velocity control using PID and Rotary Encoder.

```
#include "MCU.h"
#include "DCMotor.h"
#include "TimerCounter0.h"
#include "RotaryEncoder.h"
#include "PIDController.h"
```

Functions

- int main (void)

9.15.1 Detailed Description

Closed-loop DC motor velocity control using PID and Rotary Encoder.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file applications/PID_MotorControl/main.cpp.

9.15.2 Function Documentation

9.15.2.1 main()

```
int main (
            void )
```

Definition at line 15 of file applications/PID_MotorControl/main.cpp.

```
00015                {
00016     core::MCU::init();
00017
00018     core::TimerCounter0 &timer0 = core::TimerCounter0::getInstance();
00019     timer0.selectClockSource(core::clockSource::PS_64);
00020
00021     // Instantiate DC motor on Port D pin 6
00022     component::DCMotor motor(io::Pin(6, io::PortD));
00023     motor.connect(timer0);
00024
00025     // Instantiate Rotary Encoder on Port D pins 2 and 3
00026     component::RotaryEncoder encoder(io::Pin(2, io::PortD), io::Pin(3, io::PortD));
00027
00028     // Instantiate PID Controller (Kp=1.5, Ki=0.2, Kd=0.05)
00029     utils::PIDController pid(1.5f, 0.2f, 0.05f, 0.0f, 255.0f);
00030
00031     float targetVelocity = 100.0f;
00032     int32_t lastPos = 0;
00033
00034     while (1) {
00035         encoder.update();
00036         int32_t currentPos = encoder.getPosition();
00037         float currentVelocity = static_cast<float>(currentPos - lastPos);
00038         lastPos = currentPos;
00039
00040         float pwmOutput = pid.compute(targetVelocity, currentVelocity, 0.05f);
00041         motor.spin(timer0, static_cast<uint16_t>(pwmOutput));
00042
00043         _delay_ms(50);
00044     }
00045
00046     return 0;
00047 }
```

References utils::PIDController::compute(), component::DCMotor::connect(), core::TimerCounter0::getInstance(), component::RotaryEncoder::getPosition(), core::MCU::init(), io::PortD, core::PS_64, core::TimerCounter0::selectClockSource(), component::DCMotor::spin(), and component::RotaryEncoder::update().

9.16 applications/PID_MotorControl/main.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "MCU.h"
00010 #include "DCMotor.h"
00011 #include "TimerCounter0.h"
00012 #include "RotaryEncoder.h"
00013 #include "PIDController.h"
00014
00015 int main(void) {
00016     core::MCU::init();
00017
00018     core::TimerCounter0 &timer0 = core::TimerCounter0::getInstance();
00019     timer0.selectClockSource(core::clockSource::PS_64);
00020
00021     // Instantiate DC motor on Port D pin 6
00022     component::DCMotor motor(io::Pin(6, io::PortD));
00023     motor.connect(timer0);
00024
00025     // Instantiate Rotary Encoder on Port D pins 2 and 3
00026     component::RotaryEncoder encoder(io::Pin(2, io::PortD), io::Pin(3, io::PortD));
00027
00028     // Instantiate PID Controller (Kp=1.5, Ki=0.2, Kd=0.05)
00029     utils::PIDController pid(1.5f, 0.2f, 0.05f, 0.0f, 255.0f);
00030
```

```
00031     float targetVelocity = 100.0f;
00032     int32_t lastPos = 0;
00033
00034     while (1) {
00035         encoder.update();
00036         int32_t currentPos = encoder.getPosition();
00037         float currentVelocity = static_cast<float>(currentPos - lastPos);
00038         lastPos = currentPos;
00039
00040         float pwmOutput = pid.compute(targetVelocity, currentVelocity, 0.05f);
00041         motor.spin(timer0, static_cast<uint16_t>(pwmOutput));
00042
00043         _delay_ms(50);
00044     }
00045
00046     return 0;
00047 }
```

9.17 applications/TelemetryStreamer/main.cpp File Reference

Continuous hardware telemetry streaming over USART0.

```
#include "MCU.h"
#include "Telemetry.h"
#include "ADC.h"
#include "PushButton.h"
```

Functions

- int main (void)

9.17.1 Detailed Description

Continuous hardware telemetry streaming over USART0.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file applications/TelemetryStreamer/main.cpp.

9.17.2 Function Documentation

9.17.2.1 main()

```
int main (
            void )
```

Definition at line 14 of file applications/TelemetryStreamer/main.cpp.

```
00014                {
00015     core::MCU::init();
00016
00017     io::USART0 &usart = io::USART0::getInstance();
00018     io::Telemetry::init(usart);
00019
00020     core::ADConverter &adc = core::ADConverter::getInstance(
00021         core::referenceVoltage::AVCC,
00022         core::clockPrescaler::PS_128,
00023         core::autoTriggerSource::freeRunning,
00024         io::Pin(0, io::PortC)
00025     );
00026     adc.start();
00027
00028     component::PushButton button(io::Pin(2, io::PortD), 1, 1);
00029
00030     io::Telemetry::sendLog("Telemetry Streamer Started");
00031
00032     while (1) {
00033         uint16_t adcVal = adc.getConversionResult(core::resolution::res_10bit);
00034         uint8_t btnState = button.isPressed();
00035
00036         io::Telemetry::sendADC(0, adcVal);
00037         io::Telemetry::sendPin(2, 2, btnState);
00038
00039         _delay_ms(100);
00040     }
00041
00042     return 0;
00043 }
```

References core::AVCC, core::freeRunning, core::ADConverter::getConversionResult(), core::ADConverter::getInstance(), io::USART0::getInstance(), core::MCU::init(), io::Telemetry::init(), component::PushButton::isPressed(), io::PortC, io::PortD, core::PS_128, core::res_10bit, io::Telemetry::sendADC(), io::Telemetry::sendLog(), io::Telemetry::sendPin(), and core::ADConverter::start().

9.18 applications/TelemetryStreamer/main.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "MCU.h"
00010 #include "Telemetry.h"
00011 #include "ADC.h"
00012 #include "PushButton.h"
00013
00014 int main(void) {
00015     core::MCU::init();
00016
00017     io::USART0 &usart = io::USART0::getInstance();
00018     io::Telemetry::init(usart);
00019
00020     core::ADConverter &adc = core::ADConverter::getInstance(
00021         core::referenceVoltage::AVCC,
00022         core::clockPrescaler::PS_128,
00023         core::autoTriggerSource::freeRunning,
00024         io::Pin(0, io::PortC)
00025     );
00026     adc.start();
00027
00028     component::PushButton button(io::Pin(2, io::PortD), 1, 1);
00029
00030     io::Telemetry::sendLog("Telemetry Streamer Started");
00031
00032     while (1) {
```

```
00033         uint16_t adcVal = adc.getConversionResult(core::resolution::res_10bit);
00034         uint8_t btnState = button.isPressed();
00035
00036         io::Telemetry::sendADC(0, adcVal);
00037         io::Telemetry::sendPin(2, 2, btnState);
00038
00039         _delay_ms(100);
00040     }
00041
00042     return 0;
00043 }
```

9.19 applications/tests/WeakIsr/main.cpp File Reference

Link check: every library interrupt enabled with no app handlers.

```
#include "MCU.h"
#include "TimerCounter0.h"
#include "TimerCounter1.h"
#include "TimerCounter2.h"
#include "ExternInterrupt.h"
#include "WatchdogTimer.h"
```

Functions

- int main (void)

9.19.1 Detailed Description

Link check: every library interrupt enabled with no app handlers.

The library provides weak default handlers (empty bodies) for the timer 0/1/2, external interrupt and watchdog vectors. This app arms every single one of those interrupts without defining a handler, so its link proves the weak defaults cover all vectors; the resulting map can be checked with avr-objdump to confirm each vector points at a weak default instead of __bad_interrupt.

Definition in file applications/tests/WeakIsr/main.cpp.

9.19.2 Function Documentation

9.19.2.1 main()

```
int main (
            void )
```

Definition at line 20 of file applications/tests/WeakIsr/main.cpp.

```
00021 {
00022     core::TimerCounter0 &timer0 = core::TimerCounter0::getInstance();
00023     timer0.enableOutputCompareMatchInterrupt(core::channel::A, 1);
00024     timer0.enableOutputCompareMatchInterrupt(core::channel::B, 1);
00025     timer0.enableOverflowInterrupt(1);
00026
00027     core::TimerCounter1 &timer1 = core::TimerCounter1::getInstance();
00028     timer1.enableOutputCompareMatchInterrupt(core::channel::A, 1);
00029     timer1.enableOutputCompareMatchInterrupt(core::channel::B, 1);
00030     timer1.enableOverflowInterrupt(1);
```

```
00031     timer1.enableInputCaptureInterrupt(1);
00032
00033     core::TimerCounter2 &timer2 = core::TimerCounter2::getInstance();
00034     timer2.enableOutputCompareMatchInterrupt(core::channel::A, 1);
00035     timer2.enableOutputCompareMatchInterrupt(core::channel::B, 1);
00036     timer2.enableOverflowInterrupt(1);
00037
00038     core::ExternInterrupt &extint = core::ExternInterrupt::getInstance();
00039     extint.enableInt0(1);
00040     extint.enableInt1(1);
00041     extint.enablePinChange(core::pinChangePort::PCINTB, 1);
00042     extint.enablePinChange(core::pinChangePort::PCINTC, 1);
00043     extint.enablePinChange(core::pinChangePort::PCINTD, 1);
00044
00045     core::WatchdogTimer &watchdog = core::WatchdogTimer::getInstance();
00046     watchdog.start(core::watchdogOperationMode::interrupt, core::timeOut::to_2s);
00047
00048     core::MCU::enableInterrupts();
00049
00050     while (1)
00051     {
00052     }
00053
00054     return 0;
00055 }
```

References core::A, core::B, core::TimerCounter1::enableInputCaptureInterrupt(), core::ExternInterrupt::enableInt0(), core::ExternInterrupt::enableInt1(), core::MCU::enableInterrupts(), core::TimerCounter0::enableOutputCompareMatchInterrupt(), core::TimerCounter1::enableOutputCompareMatchInterrupt(), core::TimerCounter2::enableOutputCompareMatchInterrupt(), core::TimerCounter0::enableOverflowInterrupt(), core::TimerCounter1::enableOverflowInterrupt(), core::TimerCounter2::enableOverfl core::ExternInterrupt::enablePinChange(), core::ExternInterrupt::getInstance(), core::TimerCounter0::getInstance(), core::TimerCounter1::getInstance(), core::TimerCounter2::getInstance(), core::WatchdogTimer::getInstance(), core::interrupt, core::PCINTB, core::PCINTC, core::PCINTD, core::WatchdogTimer::start(), and core::to_2s.

9.20 applications/tests/WeakIsr/main.cpp

Go to the documentation of this file.

```
00001
00012
00013 #include "MCU.h"
00014 #include "TimerCounter0.h"
00015 #include "TimerCounter1.h"
00016 #include "TimerCounter2.h"
00017 #include "ExternInterrupt.h"
00018 #include "WatchdogTimer.h"
00019
00020 int main(void)
00021 {
00022     core::TimerCounter0 &timer0 = core::TimerCounter0::getInstance();
00023     timer0.enableOutputCompareMatchInterrupt(core::channel::A, 1);
00024     timer0.enableOutputCompareMatchInterrupt(core::channel::B, 1);
00025     timer0.enableOverflowInterrupt(1);
00026
00027     core::TimerCounter1 &timer1 = core::TimerCounter1::getInstance();
00028     timer1.enableOutputCompareMatchInterrupt(core::channel::A, 1);
00029     timer1.enableOutputCompareMatchInterrupt(core::channel::B, 1);
00030     timer1.enableOverflowInterrupt(1);
00031     timer1.enableInputCaptureInterrupt(1);
00032
00033     core::TimerCounter2 &timer2 = core::TimerCounter2::getInstance();
00034     timer2.enableOutputCompareMatchInterrupt(core::channel::A, 1);
00035     timer2.enableOutputCompareMatchInterrupt(core::channel::B, 1);
00036     timer2.enableOverflowInterrupt(1);
00037
00038     core::ExternInterrupt &extint = core::ExternInterrupt::getInstance();
00039     extint.enableInt0(1);
00040     extint.enableInt1(1);
00041     extint.enablePinChange(core::pinChangePort::PCINTB, 1);
00042     extint.enablePinChange(core::pinChangePort::PCINTC, 1);
00043     extint.enablePinChange(core::pinChangePort::PCINTD, 1);
00044
00045     core::WatchdogTimer &watchdog = core::WatchdogTimer::getInstance();
00046     watchdog.start(core::watchdogOperationMode::interrupt, core::timeOut::to_2s);
00047
00048     core::MCU::enableInterrupts();
00049
00050     while (1)
```

```
00051     {
00052     }
00053
00054     return 0;
00055 }
```

9.21 projects/AutonomousRobotController/main.cpp File Reference

Autonomous Robot Controller Project.

```
#include "MCU.h"
#include "DCMotor.h"
#include "TimerCounter0.h"
#include "TimerCounter2.h"
#include "RotaryEncoder.h"
#include "PIDController.h"
#include "Telemetry.h"
```

Functions

- int main (void)

9.21.1 Detailed Description

Autonomous Robot Controller Project.

Differential drive chassis controller using dual DC motors, quadrature encoders, closed-loop PID velocity control, and telemetry streaming.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file projects/AutonomousRobotController/main.cpp.

9.21.2 Function Documentation

9.21.2.1 main()

```
int main (
            void )
```

Definition at line 20 of file projects/AutonomousRobotController/main.cpp.

```
00020                {
00021     core::MCU::init();
00022
00023     io::USART0 &usart = io::USART0::getInstance();
00024     io::Telemetry::init(usart);
00025
00026     core::TimerCounter0 &timer0 = core::TimerCounter0::getInstance();
00027     timer0.selectClockSource(core::clockSource::PS_64);
00028
00029     core::TimerCounter2 &timer2 = core::TimerCounter2::getInstance();
00030     timer2.selectClockSource(core::clockSource::PS_64);
00031
00032     // Left Motor & Encoder
00033     component::DCMotor motorLeft(io::Pin(6, io::PortD));
00034     motorLeft.connect(timer0);
00035     component::RotaryEncoder encoderLeft(io::Pin(2, io::PortD), io::Pin(3, io::PortD));
00036     utils::PIDController pidLeft(1.5f, 0.2f, 0.05f, 0.0f, 255.0f);
00037
00038     // Right Motor & Encoder
00039     component::DCMotor motorRight(io::Pin(3, io::PortB));
00040     motorRight.connect(timer2);
00041     component::RotaryEncoder encoderRight(io::Pin(4, io::PortD), io::Pin(5, io::PortD));
00042     utils::PIDController pidRight(1.5f, 0.2f, 0.05f, 0.0f, 255.0f);
00043
00044     float targetSpeed = 80.0f;
00045     int32_t lastLeft = 0;
00046     int32_t lastRight = 0;
00047
00048     io::Telemetry::sendLog("Robot Drive Controller Active");
00049
00050     while (1) {
00051         encoderLeft.update();
00052         encoderRight.update();
00053
00054         int32_t posL = encoderLeft.getPosition();
00055         int32_t posR = encoderRight.getPosition();
00056
00057         float speedL = static_cast<float>(posL - lastLeft);
00058         float speedR = static_cast<float>(posR - lastRight);
00059
00060         lastLeft = posL;
00061         lastRight = posR;
00062
00063         float outL = pidLeft.compute(targetSpeed, speedL, 0.05f);
00064         float outR = pidRight.compute(targetSpeed, speedR, 0.05f);
00065
00066         motorLeft.spin(timer0, static_cast<uint16_t>(outL));
00067         motorRight.spin(timer2, static_cast<uint16_t>(outR));
00068
00069         io::Telemetry::sendMotor(static_cast<uint16_t>(posL), static_cast<uint16_t>(speedL));
00070
00071         _delay_ms(50);
00072     }
00073
00074     return 0;
00075 }
```

References utils::PIDController::compute(), component::DCMotor::connect(), core::TimerCounter0::getInstance(), core::TimerCounter2::getInstance(), io::USART0::getInstance(), component::RotaryEncoder::getPosition(), core::MCU::init(), io::Telemetry::init(), io::PortB, io::PortD, core::PS_64, core::TimerCounter0::selectClockSource(), core::TimerCounter2::selectClockSource(), io::Telemetry::sendLog(), io::Telemetry::sendMotor(), component::DCMotor::spin(), and component::RotaryEncoder::update().

9.22 projects/AutonomousRobotController/main.cpp

Go to the documentation of this file.

```
00001
00011
00012 #include "MCU.h"
00013 #include "DCMotor.h"
00014 #include "TimerCounter0.h"
00015 #include "TimerCounter2.h"
00016 #include "RotaryEncoder.h"
00017 #include "PIDController.h"
00018 #include "Telemetry.h"
00019
00020 int main(void) {
00021     core::MCU::init();
00022
00023     io::USART0 &usart = io::USART0::getInstance();
00024     io::Telemetry::init(usart);
00025
00026     core::TimerCounter0 &timer0 = core::TimerCounter0::getInstance();
00027     timer0.selectClockSource(core::clockSource::PS_64);
00028
00029     core::TimerCounter2 &timer2 = core::TimerCounter2::getInstance();
00030     timer2.selectClockSource(core::clockSource::PS_64);
00031
00032     // Left Motor & Encoder
00033     component::DCMotor motorLeft(io::Pin(6, io::PortD));
00034     motorLeft.connect(timer0);
00035     component::RotaryEncoder encoderLeft(io::Pin(2, io::PortD), io::Pin(3, io::PortD));
00036     utils::PIDController pidLeft(1.5f, 0.2f, 0.05f, 0.0f, 255.0f);
00037
00038     // Right Motor & Encoder
00039     component::DCMotor motorRight(io::Pin(3, io::PortB));
00040     motorRight.connect(timer2);
00041     component::RotaryEncoder encoderRight(io::Pin(4, io::PortD), io::Pin(5, io::PortD));
00042     utils::PIDController pidRight(1.5f, 0.2f, 0.05f, 0.0f, 255.0f);
00043
00044     float targetSpeed = 80.0f;
00045     int32_t lastLeft = 0;
00046     int32_t lastRight = 0;
00047
00048     io::Telemetry::sendLog("Robot Drive Controller Active");
00049
00050     while (1) {
00051         encoderLeft.update();
00052         encoderRight.update();
00053
00054         int32_t posL = encoderLeft.getPosition();
00055         int32_t posR = encoderRight.getPosition();
00056
00057         float speedL = static_cast<float>(posL - lastLeft);
00058         float speedR = static_cast<float>(posR - lastRight);
00059
00060         lastLeft = posL;
00061         lastRight = posR;
00062
00063         float outL = pidLeft.compute(targetSpeed, speedL, 0.05f);
00064         float outR = pidRight.compute(targetSpeed, speedR, 0.05f);
00065
00066         motorLeft.spin(timer0, static_cast<uint16_t>(outL));
00067         motorRight.spin(timer2, static_cast<uint16_t>(outR));
00068
00069         io::Telemetry::sendMotor(static_cast<uint16_t>(posL), static_cast<uint16_t>(speedL));
00070
00071         _delay_ms(50);
00072     }
00073
00074     return 0;
00075 }
```

9.23 projects/ESP32_SmartEnvironmentalNode/main.cpp File Reference

ESP32 Smart Environmental Sensing Node Project.

```
#include "MCU.h"
#include "Led.h"
#include "LDR.h"
#include "ADC.h"
#include "SSD1306_OLED.h"
#include "RotaryEncoder.h"
```

```
#include "PIDController.h"
#include "Telemetry.h"
```

Functions

- int main (void)

9.23.1 Detailed Description

ESP32 Smart Environmental Sensing Node Project.

Integrated hardware project for ESP32 featuring an SSD1306 OLED display, LDR light sensor, Quadrature Rotary Encoder, PID fan velocity controller, and live telemetry streaming ($ADC, $PIN, $MOTOR, $LOG).

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file projects/ESP32_SmartEnvironmentalNode/main.cpp.

9.23.2 Function Documentation

9.23.2.1 main()

```
int main (
            void )
```

Definition at line 22 of file projects/ESP32_SmartEnvironmentalNode/main.cpp.

```
00023 {
00024     // Initialize MCU hardware
00025     core::MCU::init();
00026
00027     // Status Indicator LED on GPIO 2
00028     component::Led statusLed(io::Pin(2, io::PortB));
00029     statusLed.on();
00030
00031     // Telemetry Streamer over USART0
00032     io::USART0 &usart = io::USART0::getInstance();
00033     io::Telemetry::init(usart);
00034     io::Telemetry::sendLog("ESP32 Smart Environmental Node Booted");
00035
00036     // Configure ADC Converter
00037     core::ADConverter &adc = core::ADConverter::getInstance(
00038         core::referenceVoltage::AVCC,
00039         core::clockPrescaler::PS_128,
00040         core::autoTriggerSource::freeRunning,
00041         io::Pin(0, io::PortC)
00042     );
00043     adc.start();
00044
00045     // LDR Light Sensor connected to Analog Pin 0
00046     component::LDR ldr(io::Pin(0, io::PortC), adc);
00047
00048     // Rotary Encoder connected to Pins PB0 and PB1
00049     component::RotaryEncoder encoder(io::Pin(0, io::PortB), io::Pin(1, io::PortB));
```

```
00050
00051     // SSD1306 I2C OLED Display (0x3C I2C Address)
00052     component::SSD1306_OLED oled(0x3C);
00053     oled.init();
00054     oled.clear();
00055     oled.setCursor(0, 0);
00056     oled.print("ESP32 ENV NODE");
00057     oled.setCursor(0, 2);
00058     oled.print("Status: ACTIVE");
00059
00060     // Fixed-Point PID Controller for Fan Velocity Loop
00061     utils::PIDController pid(2.5f, 0.4f, 0.1f, 0.0f, 255.0f);
00062
00063     float setpoint = 500.0f; // Target light setpoint
00064     uint32_t loopCounter = 0;
00065
00066     while (1) {
00067         // Step 1: Update Rotary Encoder input
00068         encoder.update();
00069         int16_t encoderPos = encoder.getPosition();
00070
00071         // Step 2: Sample LDR Ambient Light
00072         uint16_t rawAdc = adc.getConversionResult(core::resolution::res_10bit);
00073         uint16_t lux = ldr.getLux(5000U);
00074         (void)lux; // Explicitly marked used for sensor telemetry
00075
00076         // Step 3: Compute PID fan control output (dt = 0.1s)
00077         float pidOut = pid.compute(setpoint, (float)rawAdc, 0.1f);
00078
00079         // Step 4: Stream Telemetry to Web IDE Oscilloscope
00080         io::Telemetry::sendADC(0, rawAdc);
00081         io::Telemetry::sendPin(1, 2, 1);
00082         io::Telemetry::sendMotor((uint16_t)encoderPos, (uint16_t)pidOut);
00083
00084         // Step 5: Periodically Update SSD1306 Display
00085         if (loopCounter % 10 == 0) {
00086             oled.clear();
00087             oled.setCursor(0, 0);
00088             oled.print("ESP32 ENV NODE");
00089             oled.setCursor(0, 2);
00090             oled.print("Lux: ACTIVE");
00091             oled.setCursor(0, 3);
00092             oled.print("PID Out: ACTIVE");
00093         }
00094
00095         // Toggle Status Blinker
00096         statusLed.toggle();
00097         loopCounter++;
00098
00099         _delay_ms(100);
00100     }
00101
00102     return 0;
00103 }
```

References core::AVCC, component::SSD1306_OLED::clear(), utils::PIDController::compute(), core::freeRunning, core::ADConverter::getConversionResult(), core::ADConverter::getInstance(), io::USART0::getInstance(), component::LDR::getLux(), component::RotaryEncoder::getPosition(), component::SSD1306_OLED::init(), core::MCU::init(), io::Telemetry::init(), component::Led::on(), io::PortB, io::PortC, component::SSD1306_OLED::print(), core::PS_128, core::res_10bit, io::Telemetry::sendADC(), io::Telemetry::sendLog(), io::Telemetry::sendMotor(), io::Telemetry::sendPin(), component::SSD1306_OLE core::ADConverter::start(), component::Led::toggle(), and component::RotaryEncoder::update().

9.24 projects/ESP32_SmartEnvironmentalNode/main.cpp

Go to the documentation of this file.

```
00001
00012
00013 #include "MCU.h"
00014 #include "Led.h"
00015 #include "LDR.h"
00016 #include "ADC.h"
00017 #include "SSD1306_OLED.h"
00018 #include "RotaryEncoder.h"
00019 #include "PIDController.h"
00020 #include "Telemetry.h"
00021
00022 int main(void)
```

```
00023 {
00024     // Initialize MCU hardware
00025     core::MCU::init();
00026 
00027     // Status Indicator LED on GPIO 2
00028     component::Led statusLed(io::Pin(2, io::PortB));
00029     statusLed.on();
00030 
00031     // Telemetry Streamer over USART0
00032     io::USART0 &usart = io::USART0::getInstance();
00033     io::Telemetry::init(usart);
00034     io::Telemetry::sendLog("ESP32 Smart Environmental Node Booted");
00035 
00036     // Configure ADC Converter
00037     core::ADConverter &adc = core::ADConverter::getInstance(
00038         core::referenceVoltage::AVCC,
00039         core::clockPrescaler::PS_128,
00040         core::autoTriggerSource::freeRunning,
00041         io::Pin(0, io::PortC)
00042     );
00043     adc.start();
00044 
00045     // LDR Light Sensor connected to Analog Pin 0
00046     component::LDR ldr(io::Pin(0, io::PortC), adc);
00047 
00048     // Rotary Encoder connected to Pins PB0 and PB1
00049     component::RotaryEncoder encoder(io::Pin(0, io::PortB), io::Pin(1, io::PortB));
00050 
00051     // SSD1306 I2C OLED Display (0x3C I2C Address)
00052     component::SSD1306_OLED oled(0x3C);
00053     oled.init();
00054     oled.clear();
00055     oled.setCursor(0, 0);
00056     oled.print("ESP32 ENV NODE");
00057     oled.setCursor(0, 2);
00058     oled.print("Status: ACTIVE");
00059 
00060     // Fixed-Point PID Controller for Fan Velocity Loop
00061     utils::PIDController pid(2.5f, 0.4f, 0.1f, 0.0f, 255.0f);
00062 
00063     float setpoint = 500.0f; // Target light setpoint
00064     uint32_t loopCounter = 0;
00065 
00066     while (1) {
00067         // Step 1: Update Rotary Encoder input
00068         encoder.update();
00069         int16_t encoderPos = encoder.getPosition();
00070 
00071         // Step 2: Sample LDR Ambient Light
00072         uint16_t rawAdc = adc.getConversionResult(core::resolution::res_10bit);
00073         uint16_t lux = ldr.getLux(5000U);
00074         (void)lux; // Explicitly marked used for sensor telemetry
00075 
00076         // Step 3: Compute PID fan control output (dt = 0.1s)
00077         float pidOut = pid.compute(setpoint, (float)rawAdc, 0.1f);
00078 
00079         // Step 4: Stream Telemetry to Web IDE Oscilloscope
00080         io::Telemetry::sendADC(0, rawAdc);
00081         io::Telemetry::sendPin(1, 2, 1);
00082         io::Telemetry::sendMotor((uint16_t)encoderPos, (uint16_t)pidOut);
00083 
00084         // Step 5: Periodically Update SSD1306 Display
00085         if (loopCounter % 10 == 0) {
00086             oled.clear();
00087             oled.setCursor(0, 0);
00088             oled.print("ESP32 ENV NODE");
00089             oled.setCursor(0, 2);
00090             oled.print("Lux: ACTIVE");
00091             oled.setCursor(0, 3);
00092             oled.print("PID Out: ACTIVE");
00093         }
00094 
00095         // Toggle Status Blinker
00096         statusLed.toggle();
00097         loopCounter++;
00098 
00099         _delay_ms(100);
00100     }
00101 
00102     return 0;
00103 }
```

9.25 projects/SmartHomeGateway/main.cpp File Reference

Smart Home Peripheral Gateway Project.

```
#include "MCU.h"
#include "SSD1306_OLED.h"
#include "ServoMotor.h"
#include "TimerCounter1.h"
#include "PushButton.h"
#include "Buzzer.h"
#include "LDR.h"
#include "Telemetry.h"
```

Functions

- int main (void)

9.25.1 Detailed Description

Smart Home Peripheral Gateway Project.

Integrates OLED display, pushbutton door lock control, servo actuator, buzzer alert, LDR ambient light sensor, and serial telemetry.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file projects/SmartHomeGateway/main.cpp.

9.25.2 Function Documentation

9.25.2.1 main()

```
int main (
            void )
```

Definition at line 21 of file projects/SmartHomeGateway/main.cpp.

```
00021                {
00022     core::MCU::init();
00023
00024     io::USART0 &usart = io::USART0::getInstance();
00025     io::Telemetry::init(usart);
00026
00027     core::ADConverter &adc = core::ADConverter::getInstance(
00028         core::referenceVoltage::AVCC,
00029         core::clockPrescaler::PS_128,
00030         core::autoTriggerSource::freeRunning,
00031         io::Pin(0, io::PortC)
```

```
00032     );
00033     adc.start();
00034
00035     core::TimerCounter1 &timer1 = core::TimerCounter1::getInstance();
00036     timer1.selectClockSource(core::clockSource::PS_8);
00037
00038     component::SSD1306_OLED oled(0x3C, 128, 64);
00039     oled.init();
00040     oled.clear();
00041     oled.setCursor(0, 0);
00042     oled.print("SmartHome Gateway");
00043
00044     component::PushButton button(io::Pin(2, io::PortD), 1, 1);
00045     component::ServoMotor servo(io::Pin(1, io::PortB));
00046     servo.connect(timer1);
00047
00048     component::Buzzer buzzer(io::Pin(3, io::PortD));
00049     component::LDR ldr(io::Pin(0, io::PortC), adc);
00050
00051     uint8_t isLocked = 1;
00052     servo.rotate(timer1, 0);
00053
00054     io::Telemetry::sendLog("SmartHome Gateway Active");
00055
00056     while (1) {
00057         uint16_t lux = ldr.getLux(5);
00058         io::Telemetry::sendADC(0, lux);
00059
00060         if (button.isPressed()) {
00061             isLocked = !isLocked;
00062             if (isLocked) {
00063                 servo.rotate(timer1, 0);
00064                 buzzer.buzz(200, 100);
00065                 oled.setCursor(0, 2);
00066                 oled.print("Status: LOCKED  ");
00067                 io::Telemetry::sendLog("Door Locked");
00068             } else {
00069                 servo.rotate(timer1, 90);
00070                 buzzer.buzz(500, 200);
00071                 oled.setCursor(0, 2);
00072                 oled.print("Status: UNLOCKED");
00073                 io::Telemetry::sendLog("Door Unlocked");
00074             }
00075         }
00076
00077         _delay_ms(100);
00078     }
00079
00080     return 0;
00081 }
```

References core::AVCC, component::Buzzer::buzz(), component::SSD1306_OLED::clear(), component::ServoMotor::connect(), core::freeRunning, core::ADConverter::getInstance(), core::TimerCounter1::getInstance(), io::USART0::getInstance(), component::LDR::getLux(), component::SSD1306_OLED::init(), core::MCU::init(), io::Telemetry::init(), component::PushButton::isPres io::PortB, io::PortC, io::PortD, component::SSD1306_OLED::print(), core::PS_128, core::PS_8, component::ServoMotor::rotate(), core::TimerCounter1::selectClockSource(), io::Telemetry::sendADC(), io::Telemetry::sendLog(), component::SSD1306_OLED::setCu and core::ADConverter::start().

9.26 projects/SmartHomeGateway/main.cpp

Go to the documentation of this file.

```
00001
00011
00012 #include "MCU.h"
00013 #include "SSD1306_OLED.h"
00014 #include "ServoMotor.h"
00015 #include "TimerCounter1.h"
00016 #include "PushButton.h"
00017 #include "Buzzer.h"
00018 #include "LDR.h"
00019 #include "Telemetry.h"
00020
00021 int main(void) {
00022     core::MCU::init();
00023
00024     io::USART0 &usart = io::USART0::getInstance();
00025     io::Telemetry::init(usart);
```

```
00026
00027     core::ADConverter &adc = core::ADConverter::getInstance(
00028         core::referenceVoltage::AVCC,
00029         core::clockPrescaler::PS_128,
00030         core::autoTriggerSource::freeRunning,
00031         io::Pin(0, io::PortC)
00032     );
00033     adc.start();
00034
00035     core::TimerCounter1 &timer1 = core::TimerCounter1::getInstance();
00036     timer1.selectClockSource(core::clockSource::PS_8);
00037
00038     component::SSD1306_OLED oled(0x3C, 128, 64);
00039     oled.init();
00040     oled.clear();
00041     oled.setCursor(0, 0);
00042     oled.print("SmartHome Gateway");
00043
00044     component::PushButton button(io::Pin(2, io::PortD), 1, 1);
00045     component::ServoMotor servo(io::Pin(1, io::PortB));
00046     servo.connect(timer1);
00047
00048     component::Buzzer buzzer(io::Pin(3, io::PortD));
00049     component::LDR ldr(io::Pin(0, io::PortC), adc);
00050
00051     uint8_t isLocked = 1;
00052     servo.rotate(timer1, 0);
00053
00054     io::Telemetry::sendLog("SmartHome Gateway Active");
00055
00056     while (1) {
00057         uint16_t lux = ldr.getLux(5);
00058         io::Telemetry::sendADC(0, lux);
00059
00060         if (button.isPressed()) {
00061             isLocked = !isLocked;
00062             if (isLocked) {
00063                 servo.rotate(timer1, 0);
00064                 buzzer.buzz(200, 100);
00065                 oled.setCursor(0, 2);
00066                 oled.print("Status: LOCKED  ");
00067                 io::Telemetry::sendLog("Door Locked");
00068             } else {
00069                 servo.rotate(timer1, 90);
00070                 buzzer.buzz(500, 200);
00071                 oled.setCursor(0, 2);
00072                 oled.print("Status: UNLOCKED");
00073                 io::Telemetry::sendLog("Door Unlocked");
00074             }
00075         }
00076
00077         _delay_ms(100);
00078     }
00079
00080     return 0;
00081 }
```

9.27 README.md File Reference

9.28 README.md File Reference

9.29 README.md File Reference

9.30 README.md File Reference

9.31 README.md File Reference

9.32 README.md File Reference

9.33 README.md File Reference

9.34 README.md File Reference

9.35 README.md File Reference

9.36 README.md File Reference

9.37 README.md File Reference

9.38 README.md File Reference

9.39 README.md File Reference

9.40 README.md File Reference

9.41 Buzzer.cpp File Reference

```
#include "Buzzer.h"
```

9.42 Buzzer.cpp

Go to the documentation of this file.

```
00001
00009
00010 #include "Buzzer.h"
00011
00012
00013 component::Buzzer::Buzzer(const io::Pin &ar_pin)
00014                   : m_pin(ar_pin)
00015 {
00016     m_pin.toOutput();
00017
00018 }
00019
00020 component::Buzzer::~Buzzer()
00021 {
00022
00023 }
00024
00025 void component::Buzzer::buzz(const uint16_t &ar_period_us , const uint16_t &ar_duration_us)
00026 {
00027     // a zero period would step the pitch loop by 0 and spin forever
00028     if (ar_period_us == 0)
00029     {
00030         return;
00031     }
00032
00033     for (uint16_t i = 0; i < ar_duration_us; i += ar_period_us)
00034     {
00035         // for loop with variable delay selects the pitch
00036         for (uint16_t j = 0; j < ar_period_us; j++)
00037         {
00038             _delay_us(1);
00039         }
00040         m_pin.toggle();
00041     }
00042     m_pin.setLow();
00043
00044 }
00045
00046
00047 void component::Buzzer::pause(uint16_t a_duration_us)
00048 {
00049     // the decrement loop would wrap a zero around to 65535
00050     if (a_duration_us == 0)
00051     {
00052         return;
00053     }
00054     do {
00055       _delay_us(1);
00056     } while (--a_duration_us);
00057 }
```

9.43 Buzzer.h File Reference

```
#include "ha_base.h"
#include "Pin.h"
#include "TimerCounter.h"
```

Classes

- class component::Buzzer

  Class for handling a Buzzer component.

Namespaces

- namespace component

  AVR chip external components.

9.44 Buzzer.h

Go to the documentation of this file.

```
00001
00002 // #include "Buzzer.h"
00003 // #include "TimerCounter0.h"
00004 // #include "buzzer_pitches_8bit.h"
00005
00006
00007 // #define BUZZER 6
00008
00009
00010 // int main(void) {
00011
00012 //    // Init
00013 //    // instantiate the Buzzer object
00014 //    component::Buzzer Buzzer(io::Pin(BUZZER,io::PortD));
00015
00016
00017 //    // instantiate the TimerCounter0 object
00018 //    core::TimerCounter0 &myTimerCounter0 = core::TimerCounter0::getInstance();
00019
00020 //    // notes in the melody:
00021 //    const uint16_t notes[] = {C2, E2, G2, C3};
00022
00023
00024 //    for (uint8_t i = 0; i < sizeof (notes)/sizeof (uint16_t); i++)
00025 //    {
00026
00027 //        Buzzer.buzz(notes[i],200);
00028
00029 //    }
00030
00031 //    _delay_ms(1000);
00032 //    _delay_ms(1000);
00033 //    _delay_ms(1000);
00034
00035 //    for (uint8_t i = 0; i < sizeof (notes)/sizeof (uint16_t); i++)
00036 //    {
00037 //        Buzzer.buzz(myTimerCounter0,notes[i],200);
00038 //    }
00039
00040
00041 //    // ------ Event loop ------ //
00042 //    while (1) {
00043
00044
00045 //    }
00046 //    return 0;
00047 // }
00048
00049 #ifndef BUZZER_H
00050 #define BUZZER_H
00051 #include "ha_base.h"
00052 #include "Pin.h"
00053 #include "TimerCounter.h"
00054
00063 namespace component
00064 {
00065
00074 class Buzzer
00075 {
00076
00077 public:
00085     Buzzer(const io::Pin &ar_pin);
00089     ~Buzzer();
00090
00102     void buzz(const uint16_t &ar_period_us , const uint16_t &ar_duration_us);
00103
00112     void pause(uint16_t a_duration_us);
00113
00132     template<typename TC>
00133     void buzz(TC &ar_timerCounter,
00134               const uint16_t &ar_period_ticks,
00135               uint16_t a_duration_ms,
00136               const core::channel &ar_channel=core::channel::A,
00137               const core::clockSource &ar_clockSource=core::clockSource::PS_64
00138               )
00139     {
00140         (void)ar_clockSource;   // start() keeps the timer's current source
00141         ar_timerCounter.selectOperationMode(core::operationMode::CTC_OCR);
00142         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::toggle);
00143         ar_timerCounter.setCounter(0);
00144         ar_timerCounter.setOutputCompareRegister(ar_channel, ar_period_ticks);
00145         // start timer
```

```
00146         ar_timerCounter.start();
00147         // wait for the pitch duration
00148         while (a_duration_ms) {
00149           _delay_ms(1);
00150           a_duration_ms--;
00151         }
00152         // no buzz
00153         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::normal);
00154         ar_timerCounter.stop();
00155     }
00156
00157
00158 protected:
00159
00160 private:
00161
00162     io::Pin m_pin;
00163
00164
00165 };
00166 }
00167
00168
00169 #endif
```

9.45 DCMotor.cpp File Reference

```
#include "DCMotor.h"
```

9.46 DCMotor.cpp

Go to the documentation of this file.

```
00001 #include "DCMotor.h"
00002
00003 component::DCMotor::DCMotor(const io::Pin &ar_pin)
00004         : m_pin(ar_pin)
00005 {
00006     m_pin.toOutput();
00007
00008 }
00009
00010
00011 component::DCMotor::~DCMotor()
00012 {
00013
00014 }
00015
00016 void component::DCMotor::on()
00017 {
00018     m_pin.setHigh();
00019 }
00020
00021 void component::DCMotor::off()
00022 {
00023     m_pin.setLow();
00024 }
00025
00026 void component::DCMotor::toggle()
00027 {
00028     m_pin.toggle();
00029
00030 }
```

9.47 DCMotor.h File Reference

Header file of the DCMotor class.

```
#include "ha_base.h"
#include "Pin.h"
#include "TimerCounter.h"
```

Classes

- class component::DCMotor

Namespaces

- namespace component
  AVR chip external components.

9.47.1 Detailed Description

Header file of the DCMotor class.

Usage example (test): #include "MCU.h" #include "TimerCounter0.h" #include "DCMotor.h"

#define DCMOTOR_NUMBER 6

int main(void) {

Init initialize MCU core::MCU::init();

instantiate the TimerCounter0 object core::TimerCounter0 &myTimerCounter0 = core::TimerCounter0::getInstance(); myTimerCounter0.selectClockSource(core::clockSource::PS_1024);

instantiate a DCMotor object component::DCMotor myDCMotor(io::Pin(DCMOTOR_NUMBER,io::PortD));

myDCMotor.connect(myTimerCounter0);

myDCMotor.spin(myTimerCounter0,0); _delay_ms(5000);

myDCMotor.spin(myTimerCounter0,75); _delay_ms(5000);

myDCMotor.spin(myTimerCounter0,190); _delay_ms(5000);

myDCMotor.spin(myTimerCounter0,200); _delay_ms(5000);

myDCMotor.disconnect(myTimerCounter0);

Mainloop while (1) {

} return 0; } Usage example (H Bridge):

#include "MCU.h" #include "TimerCounter0.h" #include "DCMotor.h"

#define DCMOTOR_NUMBER 6 #define DCMOTOR_BACKWARD 0 #define DCMOTOR_FORWARD 1

int main(void) {

Init initialize MCU core::MCU::init();

instantiate the TimerCounter0 object core::TimerCounter0 &myTimerCounter0 = core::TimerCounter0::getInstance(); myTimerCounter0.selectClockSource(core::clockSource::PS_256);

instantiate a DCMotor object component::DCMotor myDCMotor(io::Pin(DCMOTOR_NUMBER,io::PortD));

instantiate direction pins objects

io::Pin myPinForward = io::Pin(DCMOTOR_FORWARD,io::PortB); myPinForward.toOutput();

io::Pin myPinBackward = io::Pin(DCMOTOR_BACKWARD,io::PortB); myPinBackward.toOutput();

myDCMotor.connect(myTimerCounter0);

myPinForward.setHigh(); myPinBackward.setLow(); myDCMotor.spin(myTimerCounter0,30); _delay_ms(5000);

myPinForward.setLow(); myPinBackward.setHigh(); myDCMotor.spin(myTimerCounter0,30); _delay_ms(5000);

myDCMotor.disconnect(myTimerCounter0);

Mainloop while (1) {

} return 0; }

Basic class for IO abstraction of Pin and Port

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file DCMotor.h.

9.48 DCMotor.h

Go to the documentation of this file.

```
00001
00116
00117 #ifndef DCMOTOR_H
00118 #define DCMOTOR_H
00119 #include "ha_base.h"
00120 #include "Pin.h"
00121 #include "TimerCounter.h"
00122
00123 namespace component
00124 {
00125
00126 class DCMotor
00127 {
00128 public:
00129     DCMotor(const io::Pin &ar_pin);
00130
00131     ~DCMotor();
00132
00135     void on();
00138     void off();
00141     void toggle();
00142
00143     template<typename TC>
00144     void spin(TC &ar_timerCounter,
00145               //const uint8_t &ar_dir,
00146               const uint16_t &ar_speed,
00147               const core::channel &ar_channel=core::channel::A)
00148     {
00149
00150         ar_timerCounter.setOutputCompareRegister(ar_channel, ar_speed);
00151         ar_timerCounter.start();
00152
00153
00154     }
00155     template<typename TC>
00156     void stop(TC &ar_timerCounter)
00157     {
00158         ar_timerCounter.stop();
00159
00160
00161     }
00162
00163     template<typename TC>
00164     void connect(TC &ar_timerCounter,
00165                  const core::channel &ar_channel=core::channel::A)
00166     {
00167         ar_timerCounter.selectOperationMode(core::operationMode::fast_PWM);
00168         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::clear);
00169         ar_timerCounter.setCounter(0);
00170
00171     }
00172
00173     template<typename TC>
00174     void disconnect(TC &ar_timerCounter,
00175                     const core::channel &ar_channel=core::channel::A)
00176     {
00177         ar_timerCounter.selectCompareOutputMode(ar_channel, core::compareOutputMode::normal);
00178         ar_timerCounter.stop();
00179     }
00180
00181
00182
00183 private:
```

```
00184
00185     io::Pin m_pin;
00186
00187 };
00188
00189 }
00190 #endif // DCMOTOR_H
```

9.49 LDR.cpp File Reference

```
#include "LDR.h"
#include <math.h>
```

9.50 LDR.cpp

Go to the documentation of this file.

```
00001 #include "LDR.h"
00002 #include <math.h>
00003
00004 component::LDR::LDR(const io::Pin &ar_pin, core::ADConverter &ar_ADC)
00005                 : m_pin(ar_pin), mr_ADC(ar_ADC)
00006
00007 {
00008     // selects the LDR pin as the ADC input and disables its digital
00009     // input buffer
00010     mr_ADC.selectAnalogInput(m_pin);
00011 }
00012
00013 component::LDR::~LDR()
00014 {
00015
00016 }
00017
00018 uint32_t component::LDR::getResistance(const uint16_t &ar_VDResistor)
00019 {
00020     uint16_t l_result = mr_ADC.getConversionResult(core::resolution::res_10bit);
00021     // an ADC result of 0 would divide by zero: report it as a sentinel
00022     if (l_result == 0)
00023     {
00024         return ADC_ZERO_SENTINEL;
00025     }
00026     // 10bit resolution -> 1023
00027     return (uint32_t)ar_VDResistor * ((1000UL * 1023UL / l_result) - 1000UL);
00028 }
00029
00030 uint16_t component::LDR::getLux(const uint16_t &ar_VDResistor)
00031 {
00032     uint32_t l_resistance = getResistance(ar_VDResistor);
00033     // ADC result 0 is the dark end of the scale: no illuminance
00034     if (l_resistance == ADC_ZERO_SENTINEL)
00035     {
00036         return 0;
00037     }
00038     // see:
      https://www.digikey.com/en/maker/projects/design-a-luxmeter-with-an-ldr-and-an-arduino/623aeee0f93e427bb57e02c4592567d1
00039     float l_lux = 12518931.0f * powf((float)l_resistance, -1.405f);
00040     // clamp to the uint16_t range: a brightly lit LDR gives a value far
00041     // above 65535 and the cast used to wrap around
00042     if (l_lux < 0.0f)
00043     {
00044         l_lux = 0.0f;
00045     }
00046     if (l_lux > 65535.0f)
00047     {
00048         l_lux = 65535.0f;
00049     }
00050     return (uint16_t)l_lux;
00051 }
```

9.51 LDR.h File Reference

```
#include "Pin.h"
#include "ADC.h"
```

Classes

- class component::LDR

  Class for handling an LDR component.

Namespaces

- namespace component

  AVR chip external components.

9.52 LDR.h

Go to the documentation of this file.

```
00001 #ifndef LDR_H
00002 #define LDR_H
00003 #include "Pin.h"
00004 #include "ADC.h"
00005
00014
00015 namespace component
00016 {
00017
00026
00027 class LDR
00028 {
00029
00030 public:
00036     static const uint32_t ADC_ZERO_SENTINEL = 0xFFFFFFFFUL;
00037
00048     LDR(const io::Pin &ar_pin,core::ADConverter &ar_ADC);
00052     ~LDR();
00062     uint32_t getResistance(const uint16_t &ar_VDResistor);
00063
00075     uint16_t getLux(const uint16_t &ar_VDResistor);
00076
00077 protected:
00078
00079 private:
00080
00081     io::Pin m_pin;
00082     core::ADConverter &mr_ADC;
00083
00084
00085
00086 };
00087
00088 }
00089
00090 #endif // LDR_H
```

9.53 Led.cpp File Reference

```
#include "Led.h"
```

9.54 Led.cpp

Go to the documentation of this file.

```
00001
00009
00010 #include "Led.h"
00011
00012 component::Led::Led(const io::Pin &ar_pin)
00013         : m_pin(ar_pin)
00014 {
00015     m_pin.toOutput();
00016 }
00017
00018
00019 component::Led::~Led()
00020 {
00021
00022 }
00023
00024
00025 void component::Led::on()
00026 {
00027     m_pin.setHigh();
00028
00029 }
00030
00031 void component::Led::off()
00032 {
00033     m_pin.setLow();
00034
00035 }
00036
00037
00038 void component::Led::toggle()
00039 {
00040     m_pin.toggle();
00041
00042 }
00043
00044 uint8_t component::Led::isOn()
00045 {
00046     return m_pin.isHigh();
00047
00048 }
00049
00050
00051 uint8_t component::Led::isOff()
00052 {
00053     return m_pin.isLow();
00054
00055 }
00056
```

9.55 Led.h File Reference

```
#include "Pin.h"
```

Classes

- class component::Led

  Class for handling a Led component.

Namespaces

- namespace component

  AVR chip external components.

9.56 Led.h

Go to the documentation of this file.

```
00001 #ifndef LED_H
00002 #define LED_H
00003 #include "Pin.h"
00004
00013
00014 namespace component
00015 {
00016
00025
00026 class Led
00027 {
00028
00029 public:
00037     Led(const io::Pin &ar_pin);
00041     ~Led();
00044     void on();
00047     void off();
00050     void toggle();
00056     uint8_t isOn();
00062     uint8_t isOff();
00063
00064 protected:
00065
00066 private:
00067
00068     io::Pin m_pin;
00069
00070
00071
00072 };
00073
00074 }
00075
00076
00077 #endif
```

9.57 PCF8574_LCD.cpp File Reference

Implementation of PCF8574 I2C LCD1602/LCD2004 character display driver.

```
#include "PCF8574_LCD.h"
```

Macros

- #define LCD_ENABLE_BIT 0x04
- #define LCD_RS_BIT 0x01
- #define LCD_BACKLIGHT 0x08

9.57.1 Detailed Description

Implementation of PCF8574 I2C LCD1602/LCD2004 character display driver.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file PCF8574_LCD.cpp.

9.57.2 Macro Definition Documentation

9.57.2.1 LCD_BACKLIGHT

```
#define LCD_BACKLIGHT 0x08
```

Definition at line 13 of file PCF8574_LCD.cpp.

Referenced by component::PCF8574_LCD::PCF8574_LCD(), and component::PCF8574_LCD::setBacklight().

9.57.2.2 LCD_ENABLE_BIT

```
#define LCD_ENABLE_BIT 0x04
```

Definition at line 11 of file PCF8574_LCD.cpp.

Referenced by component::PCF8574_LCD::sendNibble().

9.57.2.3 LCD_RS_BIT

```
#define LCD_RS_BIT 0x01
```

Definition at line 12 of file PCF8574_LCD.cpp.

Referenced by component::PCF8574_LCD::print().

9.58 PCF8574_LCD.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "PCF8574_LCD.h"
00010
00011 #define LCD_ENABLE_BIT  0x04
00012 #define LCD_RS_BIT      0x01
00013 #define LCD_BACKLIGHT   0x08
00014
00015 component::PCF8574_LCD::PCF8574_LCD(uint8_t a_i2cAddress, uint8_t a_cols, uint8_t a_rows)
00016     : m_i2cAddr(a_i2cAddress),
00017       m_cols(a_cols),
00018       m_rows(a_rows),
00019       m_backlightMask(LCD_BACKLIGHT),
00020       mr_twi(core::TWI::getInstance(100000UL))
00021 {
00022 }
00023
00024 void component::PCF8574_LCD::init()
00025 {
00026     _delay_ms(50);
00027     // Sequence to initialize HD44780 in 4-bit mode over PCF8574
00028     sendNibble(0x03, 0);
00029     _delay_ms(5);
00030     sendNibble(0x03, 0);
00031     _delay_us(150);
00032     sendNibble(0x03, 0);
00033     sendNibble(0x02, 0); // 4-bit mode
00034
00035     sendByte(0x28, 0); // 2 lines, 5x8 font
00036     sendByte(0x0C, 0); // Display ON, Cursor OFF
00037     clear();
00038     sendByte(0x06, 0); // Entry mode auto increment
00039 }
```

```
00040
00041 void component::PCF8574_LCD::clear()
00042 {
00043     sendByte(0x01, 0);
00044     _delay_ms(2);
00045 }
00046
00047 void component::PCF8574_LCD::setCursor(uint8_t a_col, uint8_t a_row)
00048 {
00049     static const uint8_t row_offsets[] = { 0x00, 0x40, 0x14, 0x54 };
00050     if (a_row >= m_rows) a_row = m_rows - 1;
00051     sendByte(0x80 | (a_col + row_offsets[a_row]), 0);
00052 }
00053
00054 void component::PCF8574_LCD::print(const char *ap_str)
00055 {
00056     if (!ap_str) return;
00057     while (*ap_str) {
00058         sendByte(static_cast<uint8_t>(*ap_str++), LCD_RS_BIT);
00059     }
00060 }
00061
00062 void component::PCF8574_LCD::setBacklight(uint8_t a_enable)
00063 {
00064     m_backlightMask = a_enable ? LCD_BACKLIGHT : 0x00;
00065     sendNibble(0x00, 0);
00066 }
00067
00068 void component::PCF8574_LCD::sendNibble(uint8_t a_nibble, uint8_t a_mode)
00069 {
00070     uint8_t payload = (a_nibble << 4) | a_mode | m_backlightMask;
00071     mr_twi.start();
00072     mr_twi.write(m_i2cAddr << 1);
00073     mr_twi.write(payload | LCD_ENABLE_BIT);
00074     _delay_us(1);
00075     mr_twi.write(payload & ~LCD_ENABLE_BIT);
00076     _delay_us(50);
00077     mr_twi.stop();
00078 }
00079
00080 void component::PCF8574_LCD::sendByte(uint8_t a_byte, uint8_t a_mode)
00081 {
00082     sendNibble(a_byte >> 4, a_mode);
00083     sendNibble(a_byte & 0x0F, a_mode);
00084 }
```

9.59 PCF8574_LCD.h File Reference

Header file for PCF8574 I2C LCD1602/LCD2004 character display driver.

```
#include "ha_base.h"
#include "TWI.h"
```

Classes

- class component::PCF8574_LCD

Namespaces

- namespace component

  AVR chip external components.

9.59.1 Detailed Description

Header file for PCF8574 I2C LCD1602/LCD2004 character display driver.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file PCF8574_LCD.h.

9.60 PCF8574_LCD.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef PCF8574_LCD_H
00010 #define PCF8574_LCD_H
00011
00012 #include "ha_base.h"
00013 #include "TWI.h"
00014
00015 namespace component
00016 {
00017
00018 class PCF8574_LCD
00019 {
00020 public:
00027     PCF8574_LCD(uint8_t a_i2cAddress = 0x27, uint8_t a_cols = 16, uint8_t a_rows = 2);
00028
00031     void init();
00032
00035     void clear();
00036
00042     void setCursor(uint8_t a_col, uint8_t a_row);
00043
00048     void print(const char *ap_str);
00049
00052     void setBacklight(uint8_t a_enable);
00053
00054 private:
00055     void sendNibble(uint8_t a_nibble, uint8_t a_mode);
00056     void sendByte(uint8_t a_byte, uint8_t a_mode);
00057
00058     uint8_t m_i2cAddr;
00059     uint8_t m_cols;
00060     uint8_t m_rows;
00061     uint8_t m_backlightMask;
00062     core::TWI &mr_twi;
00063 };
00064
00065 }
00066
00067 #endif
```

9.61 PushButton.cpp File Reference

```
#include "PushButton.h"
```

9.62 PushButton.cpp

Go to the documentation of this file.

```
00001
00009
00010 #include "PushButton.h"
00011
00012 component::PushButton::PushButton(const io::Pin& ar_pin, uint8_t ar_useInternalPullUp, uint8_t
      ar_isActiveLow)
00013                                   : m_pin(ar_pin),
00014                                     m_isActiveLow(ar_isActiveLow),
00015                                     m_useInternalPullUp(ar_useInternalPullUp),
00016                                     m_lastState(0),
00017                                     m_buttonPressed(0)
00018
00019 {
00020     m_pin.toInput(m_useInternalPullUp);
00021 }
00022
00023 component::PushButton::~PushButton()
00024 {
00025
00026 }
00027
00028
00029 uint8_t component::PushButton::isPressed()
00030 {
00031     uint8_t l_pressed = 0;
00032
00033     if (m_isActiveLow) {
00034         if (m_pin.isLow()) {
00035             _delay_us(PUSHBUTTON_DEBOUNCE_TIME_US);
00036             if (m_pin.isLow()) {
00037                 l_pressed = 1;
00038             }
00039         }
00040     } else {
00041         if (m_pin.isHigh()) {
00042             _delay_us(PUSHBUTTON_DEBOUNCE_TIME_US);
00043             if (m_pin.isHigh()) {
00044                 l_pressed = 1;
00045             }
00046         }
00047     }
00048
00049     // count one press per release->press transition
00050     if (l_pressed && !m_lastState) {
00051         ++m_buttonPressed;
00052     }
00053     m_lastState = l_pressed;
00054
00055     return l_pressed;
00056
00057 }
00058
00059 uint8_t component::PushButton::getPressedCount() const
00060 {
00061
00062     return m_buttonPressed;
00063
00064 }
00065
00066 void component::PushButton::resetPressedCount()
00067 {
00068     m_buttonPressed = 0;
00069
00070 }
```

9.63 PushButton.h File Reference

```
#include "Pin.h"
```

Classes

- class component::PushButton

  Class for handling a PushButton component.

Namespaces

- namespace component

  AVR chip external components.

9.64 PushButton.h

Go to the documentation of this file.

```
00001 #ifndef PUSHBUTTON_H
00002 #define PUSHBUTTON_H
00003 #include "Pin.h"
00004
00013
00014 namespace component
00015 {
00016
00025
00026 class PushButton
00027 {
00028 public:
00041     PushButton(const io::Pin &ar_pin, uint8_t ar_useInternalPullUp=1, uint8_t ar_isActiveLow=1);
00045     ~PushButton();
00059     uint8_t isPressed();
00065     uint8_t getPressedCount() const;
00070     void resetPressedCount();
00071
00072
00073 protected:
00074
00075 private:
00076     io::Pin m_pin;
00077     uint8_t m_isActiveLow;
00078     uint8_t m_useInternalPullUp;
00079     uint8_t m_lastState;
00080     uint8_t m_buttonPressed;
00081
00082
00083
00084 };
00085 }
00086
00087
00088
00089
00090 #endif
```

9.65 RotaryEncoder.cpp File Reference

Implementation of the Quadrature Rotary Encoder driver.

```
#include "RotaryEncoder.h"
```

9.65.1 Detailed Description

Implementation of the Quadrature Rotary Encoder driver.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file RotaryEncoder.cpp.

9.66 RotaryEncoder.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "RotaryEncoder.h"
00010
00011 component::RotaryEncoder::RotaryEncoder(const io::Pin &ar_pinA, const io::Pin &ar_pinB, uint8_t
      a_useInternalPullUp)
00012     : m_pinA(ar_pinA),
00013       m_pinB(ar_pinB),
00014       m_lastStateA(0),
00015       m_lastStateB(0),
00016       m_position(0),
00017       m_direction(0)
00018 {
00019     m_pinA.toInput(a_useInternalPullUp);
00020     m_pinB.toInput(a_useInternalPullUp);
00021     m_lastStateA = m_pinA.isHigh();
00022     m_lastStateB = m_pinB.isHigh();
00023 }
00024
00025 void component::RotaryEncoder::update()
00026 {
00027     uint8_t stateA = m_pinA.isHigh();
00028     uint8_t stateB = m_pinB.isHigh();
00029
00030     if (stateA != m_lastStateA) {
00031         if (stateA != stateB) {
00032             m_position++;
00033             m_direction = 1;
00034         } else {
00035             m_position--;
00036             m_direction = -1;
00037         }
00038         m_lastStateA = stateA;
00039         m_lastStateB = stateB;
00040     }
00041 }
00042
00043 int32_t component::RotaryEncoder::getPosition() const
00044 {
00045     return m_position;
00046 }
00047
00048 void component::RotaryEncoder::resetPosition()
00049 {
00050     m_position = 0;
00051     m_direction = 0;
00052 }
00053
00054 int8_t component::RotaryEncoder::getDirection() const
00055 {
00056     return m_direction;
00057 }
```

9.67 RotaryEncoder.h File Reference

Header file for the Quadrature Rotary Encoder driver.

```
#include "ha_base.h"
#include "Pin.h"
```

Classes

- class component::RotaryEncoder

Namespaces

- namespace component

  AVR chip external components.

9.67.1 Detailed Description

Header file for the Quadrature Rotary Encoder driver.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file RotaryEncoder.h.

9.68 RotaryEncoder.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef ROTARY_ENCODER_H
00010 #define ROTARY_ENCODER_H
00011
00012 #include "ha_base.h"
00013 #include "Pin.h"
00014
00015 namespace component
00016 {
00017
00018 class RotaryEncoder
00019 {
00020 public:
00027     RotaryEncoder(const io::Pin &ar_pinA, const io::Pin &ar_pinB, uint8_t a_useInternalPullUp = 1);
00028
00033     void update();
00034
00037     int32_t getPosition() const;
00038
00041     void resetPosition();
00042
00045     int8_t getDirection() const;
00046
00047 private:
00048     io::Pin m_pinA;
00049     io::Pin m_pinB;
00050     uint8_t m_lastStateA;
00051     uint8_t m_lastStateB;
00052     int32_t m_position;
00053     int8_t m_direction;
00054 };
00055
00056 }
00057
00058 #endif
```

9.69 ServoMotor.cpp File Reference

```
#include "ServoMotor.h"
```

9.70 ServoMotor.cpp

Go to the documentation of this file.

```
00001 #include "ServoMotor.h"
00002
00003
00004
00005 component::ServoMotor::ServoMotor(const io::Pin &ar_pin,
00006                                   const uint16_t &ar_pulseCycle,
00007                                   const uint16_t &ar_pulseWidthMin,
00008                                   const uint16_t &ar_pulseWidthMid,
00009                                   const uint16_t &ar_pulseWidthMax)
00010                       : m_pin(ar_pin),
00011                         m_pulseCycle(ar_pulseCycle),
00012                         m_pulseWidthMin(ar_pulseWidthMin),
00013                         m_pulseWidthMid(ar_pulseWidthMid),
00014                         m_pulseWidthMax(ar_pulseWidthMax)
00015 {
00016     m_pin.toOutput();
00017
00018 }
00019
00020 component::ServoMotor::~ServoMotor()
00021 {
00022
00023 }
00024
00025
00026
00027
00028
00029 uint16_t component::ServoMotor::clampTimerCount(uint32_t a_count)
00030 {
00031     if (a_count > UINT16_MAX)
00032     {
00033         a_count = UINT16_MAX;
00034     }
00035     return static_cast<uint16_t>(a_count);
00036 }
00037
00038 void component::ServoMotor::on()
00039 {
00040     m_pin.setHigh();
00041 }
00042
00043 void component::ServoMotor::off()
00044 {
00045     m_pin.setLow();
00046 }
00047
00048 void component::ServoMotor::toggle()
00049 {
00050     m_pin.toggle();
00051
00052 }
00053
00054 uint16_t component::ServoMotor::computePulseCycleCount(const uint16_t &ar_clockPrescaler)
00055 {
00056     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseCycle,ar_clockPrescaler));
00057 }
00058
00059 uint16_t component::ServoMotor::computePulseWidthMinCount(const uint16_t &ar_clockPrescaler)
00060 {
00061     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseWidthMin,ar_clockPrescaler));
00062 }
00063
00064 uint16_t component::ServoMotor::computePulseWidthMaxCount(const uint16_t &ar_clockPrescaler)
00065 {
00066     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseWidthMax,ar_clockPrescaler));
00067 }
00068
00069 uint16_t component::ServoMotor::computePulseWidthMidCount(const uint16_t &ar_clockPrescaler)
00070 {
00071     return clampTimerCount(SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(m_pulseWidthMid,ar_clockPrescaler));
00072 }
00073
00074 uint16_t component::ServoMotor::computeRotationAngleCount(const uint8_t &ar_angle_deg,const uint16_t
      &ar_clockPrescaler)
00075 {
00076     return
      clampTimerCount(SERVOMOTOR_TIMER_ANGLE_COUNT(ar_angle_deg,static_cast<long>(computePulseWidthMinCount(ar_clockPrescaler)
00077 }
00078
00079
00080
```

```
00081 void component::ServoMotor::rotate(core::TimerCounter1 &ar_timerCounter1,
00082                                     const uint8_t &ar_angle_deg,
00083                                     const core::channel &ar_channel)
00084 {
00085
00086     ar_timerCounter1.setOutputCompareRegister(ar_channel,
      computeRotationAngleCount(ar_angle_deg,ar_timerCounter1.getClockPrescaler()));
00087
00088     // start timer
00089     ar_timerCounter1.start();
00090
00091
00092 }
00093
00094
00095 void component::ServoMotor::connect(core::TimerCounter1 &ar_timerCounter1,
00096              const core::channel &ar_channel)
00097 {
00098
00099
00100
      ar_timerCounter1.setInputCaptureRegister(computePulseCycleCount(ar_timerCounter1.getClockPrescaler()));
00101     ar_timerCounter1.selectOperationMode(core::operationMode::fast_PWM_ICR);
00102     ar_timerCounter1.selectCompareOutputMode(ar_channel, core::compareOutputMode::clear);
00103     ar_timerCounter1.setCounter(0);
00104
00105 }
00106
00107 void component::ServoMotor::disconnect(core::TimerCounter1 &ar_timerCounter1,
00108                const core::channel &ar_channel)
00109 {
00110     ar_timerCounter1.selectCompareOutputMode(ar_channel, core::compareOutputMode::normal);
00111     // stop timer
00112     ar_timerCounter1.stop();
00113 }
```

9.71 ServoMotor.h File Reference

Header file of the ServoMotor class.

```
#include "ha_base.h"
#include "Pin.h"
#include "TimerCounter1.h"
```

Classes

- class component::ServoMotor

Namespaces

- namespace component

  AVR chip external components.

9.71.1 Detailed Description

Header file of the ServoMotor class.

Header file of the StepperMotor class.

Usage example (separate): #include "MCU.h" #include "ServoMotor.h" #include "TimerCounter1.h"

#define SERVOMOTOR_NUMBER 1 #define SERVOMOTOR_PULSE_CYCLE 30000 // pulse cycle [us] #define SERVOMOTOR_PULSE_WIDTH_MIN 500// pulse width min [us] #define SERVOMOTOR_PULSE_WIDTH_MID 1520 // pulse width mid [us] #define SERVOMOTOR_PULSE_WIDTH_MAX 3000 // pulse width max [us]

int main(void) {

Init initialize MCU core::MCU::init();

instantiate the TimerCounter0 object core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance(); myTimerCounter1.selectClockSource(core::clockSource::PS_8); myTimerCounter1.selectOperationMode(core↩ ::operationMode::fast_PWM_ICR); myTimerCounter1.selectCompareOutputMode(core::channel::A, core↩ ::compareOutputMode::clear);

instantiate the Buzzer object component::ServoMotor myServoMotor(io::Pin(SERVOMOTOR_NUMBER,io::PortB), SERVOMOTOR_PULSE_CYCLE, SERVOMOTOR_PULSE_WIDTH_MIN, SERVOMOTOR_PULSE_WIDTH_MID, SERVOMOTOR_PULSE_WIDTH_MAX);

myTimerCounter1.setInputCaptureRegister(myServoMotor.computePulseCycleCount(myTimerCounter1.get↩ ClockPrescaler())); myTimerCounter1.setOutputCompareRegister(core::channel::A, myServoMotor.compute↩ RotationAngleCount(0,myTimerCounter1.getClockPrescaler())); myTimerCounter1.start(); _delay_ms(2000); my↩ TimerCounter1.setOutputCompareRegister(core::channel::A, myServoMotor.computeRotationAngleCount(45,my↩ TimerCounter1.getClockPrescaler())); myTimerCounter1.start(); _delay_ms(2000); myTimerCounter1.set↩ OutputCompareRegister(core::channel::A, myServoMotor.computeRotationAngleCount(90,myTimerCounter1.↩ getClockPrescaler())); myTimerCounter1.start(); _delay_ms(2000); myTimerCounter1.setOutputCompare↩ Register(core::channel::A, myServoMotor.computeRotationAngleCount(135,myTimerCounter1.getClockPrescaler())); myTimerCounter1.start(); _delay_ms(2000); myTimerCounter1.setOutputCompareRegister(core::channel::A, my↩ ServoMotor.computeRotationAngleCount(180,myTimerCounter1.getClockPrescaler())); myTimerCounter1.start(); _delay_ms(2000); myTimerCounter1.setOutputCompareRegister(core::channel::A, myServoMotor.compute↩ RotationAngleCount(0,myTimerCounter1.getClockPrescaler())); myTimerCounter1.start(); _delay_ms(2000);

Mainloop while (1) {

} return 0; } Usage example (TimerCounter1): #include "MCU.h" #include "ServoMotor.h" #include "Timer↩ Counter1.h"

#define SERVOMOTOR_NUMBER 1 #define SERVOMOTOR_PULSE_CYCLE 30000 // pulse cycle [us] #define SERVOMOTOR_PULSE_WIDTH_MIN 500// pulse width min [us] #define SERVOMOTOR_PULSE_WIDTH_MID 1520 // pulse width mid [us] #define SERVOMOTOR_PULSE_WIDTH_MAX 3000 // pulse width max [us]

int main(void) {

Init initialize MCU core::MCU::init();

instantiate the TimerCounter1 object core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance(); myTimerCounter1.selectClockSource(core::clockSource::PS_8);

instantiate the Buzzer object component::ServoMotor myServoMotor(io::Pin(SERVOMOTOR_NUMBER,io::PortB), SERVOMOTOR_PULSE_CYCLE, SERVOMOTOR_PULSE_WIDTH_MIN, SERVOMOTOR_PULSE_WIDTH_MID, SERVOMOTOR_PULSE_WIDTH_MAX);

myServoMotor.connect(myTimerCounter1);

myServoMotor.rotate(myTimerCounter1,0); _delay_ms(2000); myServoMotor.rotate(myTimerCounter1,45); _↩ delay_ms(2000); myServoMotor.rotate(myTimerCounter1,90); _delay_ms(2000); myServoMotor.rotate(myTimer↩ Counter1,135); _delay_ms(2000); myServoMotor.rotate(myTimerCounter1,180); _delay_ms(2000); myServo↩ Motor.rotate(myTimerCounter1,0); _delay_ms(2000);

myServoMotor.disconnect(myTimerCounter1);

Mainloop while (1) {

```
}
return 0;
```

}

class to control a servo motor

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

The StepperMotor class is implemented and tested for the 28BYJ-48 Stepper Motor Usage example (basic):

#include "StepperMotor.h"

#define StepperMotor_Pin_1 0 #define StepperMotor_Pin_2 1 #define StepperMotor_Pin_3 2 #define Stepper↩ Motor_Pin_4 3

#define STEP_ANGLE_FULL_STEP 0.176 #define STEP_ANGLE_HALF_STEP 0.0879

int main(void) {

instantiate the StepperMotor object component::StepperMotor myStepperMotor(component::mode::halfStep, io::Pin(StepperMotor_Pin_1,io::PortB), io::Pin(StepperMotor_Pin_2,io::PortB), io::Pin(StepperMotor_Pin_3,io::PortB), io::Pin(StepperMotor_Pin_4,io::PortB));

int16_t l_angle = -180; int16_t l_step = static_cast<int16_t>(l_angle / STEP_ANGLE_HALF_STEP);

set speed [steps/s] full step: max speed: 500p/s half step: max speed: 1000p/s

set speed [°/s] full step: max speed: 40°/s half step: max speed: 80°/s

Mainloop while (1) {

```
while (!myStepperMotor.goalReached())
{
```

set motor steps myStepperMotor.step(l_step,1000); myStepperMotor.step(l_step,80,STEP_ANGLE_HALF_STEP);

```
    }
```

} return 0; }

class to control a servo motor

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file ServoMotor.h.

9.72 ServoMotor.h

Go to the documentation of this file.

```
00001
00126 #ifndef SERVOMOTOR_H
00127 #define SERVOMOTOR_H
00128 #include "ha_base.h"
00129 #include "Pin.h"
00130 #include "TimerCounter1.h"
00131
00132
00133
00134 namespace component
00135 {
00136
00137
00138 class ServoMotor
00139 {
00140 public:
00141
00142     ServoMotor(const io::Pin &ar_pin,
00143                const uint16_t &ar_pulseCycle=0,
00144                const uint16_t &ar_pulseWidthMin=0,
00145                const uint16_t &ar_pulseWidthMid=0,
00146                const uint16_t &ar_pulseWidthMax=0);
00147
00148     ~ServoMotor();
00149
00152     void on();
00155     void off();
00158     void toggle();
00159
00160     uint16_t computePulseCycleCount(const uint16_t &ar_clockPrescaler);
00161
00162     uint16_t computePulseWidthMinCount(const uint16_t &ar_clockPrescaler);
00163
00164     uint16_t computePulseWidthMidCount(const uint16_t &ar_clockPrescaler);
00165
00166     uint16_t computePulseWidthMaxCount(const uint16_t &ar_clockPrescaler);
00167
00168     uint16_t computeRotationAngleCount(const uint8_t &ar_angle_deg, const uint16_t
      &ar_clockPrescaler);
00169
00170
00171     void rotate(core::TimerCounter1 &ar_timerCounter1,
00172                 const uint8_t &ar_angle_deg,
00173                 const core::channel &ar_channel=core::channel::A);
00174
00175     void connect(core::TimerCounter1 &ar_timerCounter1,
00176                  const core::channel &ar_channel=core::channel::A);
00177
00178     void disconnect(core::TimerCounter1 &ar_timerCounter1,
00179                     const core::channel &ar_channel=core::channel::A);
00180
00181
00182
00183 protected:
00184
00185 private:
00186
00187
00193     static uint16_t clampTimerCount(uint32_t a_count);
00194
00195     io::Pin m_pin;
00196
00197     uint16_t m_pulseCycle;
00198
00199     uint16_t m_pulseWidthMin;
00200
00201     uint16_t m_pulseWidthMid;
00202
00203     uint16_t m_pulseWidthMax;
00204
00205
00206
00207
00208
00209
00210
00211
00212 };
00213
00214
00215 }
00216
```

```
00217
00218
00219 #endif
```

9.73 SSD1306_OLED.cpp File Reference

Implementation of SSD1306 128x64 / 128x32 I2C OLED display driver.

```
#include "SSD1306_OLED.h"
```

Variables

- static const uint8_t font5x7 [ ][5]

9.73.1 Detailed Description

Implementation of SSD1306 128x64 / 128x32 I2C OLED display driver.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file SSD1306_OLED.cpp.

9.73.2 Variable Documentation

9.73.2.1 font5x7

```
const uint8_t font5x7[ ][5]  [static]
```

Initial value:

```
= {
    { 0x00, 0x00, 0x00, 0x00, 0x00 },
    { 0x00, 0x00, 0x5F, 0x00, 0x00 },
    { 0x00, 0x07, 0x00, 0x07, 0x00 },
    { 0x14, 0x7F, 0x14, 0x7F, 0x14 },
    { 0x24, 0x2A, 0x7F, 0x2A, 0x12 },
    { 0x23, 0x13, 0x08, 0x64, 0x62 },
    { 0x36, 0x49, 0x55, 0x22, 0x50 },
    { 0x00, 0x05, 0x03, 0x00, 0x00 },
    { 0x00, 0x1C, 0x22, 0x41, 0x00 },
    { 0x00, 0x41, 0x22, 0x1C, 0x00 },
    { 0x14, 0x08, 0x3E, 0x08, 0x14 },
    { 0x08, 0x08, 0x3E, 0x08, 0x08 },
    { 0x00, 0x50, 0x30, 0x00, 0x00 },
    { 0x08, 0x08, 0x08, 0x08, 0x08 },
    { 0x00, 0x60, 0x60, 0x00, 0x00 },
    { 0x20, 0x10, 0x08, 0x04, 0x02 },
    { 0x3E, 0x51, 0x49, 0x45, 0x3E },
    { 0x00, 0x42, 0x7F, 0x40, 0x00 },
    { 0x42, 0x61, 0x51, 0x49, 0x46 },
```

```
    { 0x21, 0x41, 0x45, 0x4B, 0x31 },
    { 0x18, 0x14, 0x12, 0x7F, 0x10 },
    { 0x27, 0x45, 0x45, 0x45, 0x39 },
    { 0x3C, 0x4A, 0x49, 0x49, 0x30 },
    { 0x01, 0x71, 0x09, 0x05, 0x03 },
    { 0x36, 0x49, 0x49, 0x49, 0x36 },
    { 0x06, 0x49, 0x49, 0x29, 0x1E },
}
```

Definition at line 12 of file SSD1306_OLED.cpp.

```
00012                                       {
00013     { 0x00, 0x00, 0x00, 0x00, 0x00 }, // space
00014     { 0x00, 0x00, 0x5F, 0x00, 0x00 }, // !
00015     { 0x00, 0x07, 0x00, 0x07, 0x00 }, // "
00016     { 0x14, 0x7F, 0x14, 0x7F, 0x14 }, // #
00017     { 0x24, 0x2A, 0x7F, 0x2A, 0x12 }, // $
00018     { 0x23, 0x13, 0x08, 0x64, 0x62 }, // %
00019     { 0x36, 0x49, 0x55, 0x22, 0x50 }, // &
00020     { 0x00, 0x05, 0x03, 0x00, 0x00 }, // '
00021     { 0x00, 0x1C, 0x22, 0x41, 0x00 }, // (
00022     { 0x00, 0x41, 0x22, 0x1C, 0x00 }, // )
00023     { 0x14, 0x08, 0x3E, 0x08, 0x14 }, // *
00024     { 0x08, 0x08, 0x3E, 0x08, 0x08 }, // +
00025     { 0x00, 0x50, 0x30, 0x00, 0x00 }, // ,
00026     { 0x08, 0x08, 0x08, 0x08, 0x08 }, // -
00027     { 0x00, 0x60, 0x60, 0x00, 0x00 }, // .
00028     { 0x20, 0x10, 0x08, 0x04, 0x02 }, // /
00029     { 0x3E, 0x51, 0x49, 0x45, 0x3E }, // 0
00030     { 0x00, 0x42, 0x7F, 0x40, 0x00 }, // 1
00031     { 0x42, 0x61, 0x51, 0x49, 0x46 }, // 2
00032     { 0x21, 0x41, 0x45, 0x4B, 0x31 }, // 3
00033     { 0x18, 0x14, 0x12, 0x7F, 0x10 }, // 4
00034     { 0x27, 0x45, 0x45, 0x45, 0x39 }, // 5
00035     { 0x3C, 0x4A, 0x49, 0x49, 0x30 }, // 6
00036     { 0x01, 0x71, 0x09, 0x05, 0x03 }, // 7
00037     { 0x36, 0x49, 0x49, 0x49, 0x36 }, // 8
00038     { 0x06, 0x49, 0x49, 0x29, 0x1E }, // 9
00039 };
```

Referenced by component::SSD1306_OLED::print().

9.74 SSD1306_OLED.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "SSD1306_OLED.h"
00010
00011 // 5x7 ASCII Font Table subset
00012 static const uint8_t font5x7[][5] = {
00013     { 0x00, 0x00, 0x00, 0x00, 0x00 }, // space
00014     { 0x00, 0x00, 0x5F, 0x00, 0x00 }, // !
00015     { 0x00, 0x07, 0x00, 0x07, 0x00 }, // "
00016     { 0x14, 0x7F, 0x14, 0x7F, 0x14 }, // #
00017     { 0x24, 0x2A, 0x7F, 0x2A, 0x12 }, // $
00018     { 0x23, 0x13, 0x08, 0x64, 0x62 }, // %
00019     { 0x36, 0x49, 0x55, 0x22, 0x50 }, // &
00020     { 0x00, 0x05, 0x03, 0x00, 0x00 }, // '
00021     { 0x00, 0x1C, 0x22, 0x41, 0x00 }, // (
00022     { 0x00, 0x41, 0x22, 0x1C, 0x00 }, // )
00023     { 0x14, 0x08, 0x3E, 0x08, 0x14 }, // *
00024     { 0x08, 0x08, 0x3E, 0x08, 0x08 }, // +
00025     { 0x00, 0x50, 0x30, 0x00, 0x00 }, // ,
00026     { 0x08, 0x08, 0x08, 0x08, 0x08 }, // -
00027     { 0x00, 0x60, 0x60, 0x00, 0x00 }, // .
00028     { 0x20, 0x10, 0x08, 0x04, 0x02 }, // /
00029     { 0x3E, 0x51, 0x49, 0x45, 0x3E }, // 0
00030     { 0x00, 0x42, 0x7F, 0x40, 0x00 }, // 1
00031     { 0x42, 0x61, 0x51, 0x49, 0x46 }, // 2
00032     { 0x21, 0x41, 0x45, 0x4B, 0x31 }, // 3
00033     { 0x18, 0x14, 0x12, 0x7F, 0x10 }, // 4
00034     { 0x27, 0x45, 0x45, 0x45, 0x39 }, // 5
00035     { 0x3C, 0x4A, 0x49, 0x49, 0x30 }, // 6
00036     { 0x01, 0x71, 0x09, 0x05, 0x03 }, // 7
00037     { 0x36, 0x49, 0x49, 0x49, 0x36 }, // 8
00038     { 0x06, 0x49, 0x49, 0x29, 0x1E }, // 9
00039 };
00040
```

```
00041 component::SSD1306_OLED::SSD1306_OLED(uint8_t a_i2cAddress, uint8_t a_width, uint8_t a_height)
00042     : m_i2cAddr(a_i2cAddress),
00043       m_width(a_width),
00044       m_height(a_height),
00045       mr_twi(core::TWI::getInstance(400000UL))
00046 {
00047 }
00048
00049 void component::SSD1306_OLED::init()
00050 {
00051     _delay_ms(10);
00052     sendCommand(0xAE); // Display OFF
00053     sendCommand(0xD5); sendCommand(0x80); // Set Display Clock Divide Ratio
00054     sendCommand(0xA8); sendCommand(m_height - 1); // Set Multiplex Ratio
00055     sendCommand(0xD3); sendCommand(0x00); // Display Offset
00056     sendCommand(0x40); // Start Line 0
00057     sendCommand(0x8D); sendCommand(0x14); // Enable Charge Pump
00058     sendCommand(0x20); sendCommand(0x00); // Memory Addressing Mode: Horizontal
00059     sendCommand(0xA1); // Segment Re-map
00060     sendCommand(0xC8); // COM Output Scan Direction
00061     sendCommand(0xDA); sendCommand((m_height == 64) ? 0x12 : 0x02); // COM Pins Configuration
00062     sendCommand(0x81); sendCommand(0xCF); // Set Contrast Control
00063     sendCommand(0xD9); sendCommand(0xF1); // Pre-charge Period
00064     sendCommand(0xDB); sendCommand(0x40); // VCOMH Deselect Level
00065     sendCommand(0xA4); // Entire Display ON
00066     sendCommand(0xA6); // Normal Display
00067     sendCommand(0xAF); // Display ON
00068     clear();
00069 }
00070
00071 void component::SSD1306_OLED::clear()
00072 {
00073     for (uint8_t page = 0; page < (m_height / 8); page++) {
00074         setCursor(0, page);
00075         mr_twi.start();
00076         mr_twi.write(m_i2cAddr « 1);
00077         mr_twi.write(0x40); // Data mode
00078         for (uint8_t col = 0; col < m_width; col++) {
00079             mr_twi.write(0x00);
00080         }
00081         mr_twi.stop();
00082     }
00083     setCursor(0, 0);
00084 }
00085
00086 void component::SSD1306_OLED::setCursor(uint8_t a_col, uint8_t a_page)
00087 {
00088     sendCommand(0xB0 | (a_page & 0x07));
00089     sendCommand(0x00 | (a_col & 0x0F));
00090     sendCommand(0x10 | ((a_col » 4) & 0x0F));
00091 }
00092
00093 void component::SSD1306_OLED::print(const char *ap_str)
00094 {
00095     if (!ap_str) return;
00096     while (*ap_str) {
00097         char c = *ap_str++;
00098         uint8_t idx = (c >= ' ' && c <= '9') ? (c - ' ') : 0;
00099         mr_twi.start();
00100         mr_twi.write(m_i2cAddr « 1);
00101         mr_twi.write(0x40);
00102         for (uint8_t i = 0; i < 5; i++) {
00103             mr_twi.write(font5x7[idx][i]);
00104         }
00105         mr_twi.write(0x00); // char spacing
00106         mr_twi.stop();
00107     }
00108 }
00109
00110 void component::SSD1306_OLED::sendCommand(uint8_t a_cmd)
00111 {
00112     mr_twi.start();
00113     mr_twi.write(m_i2cAddr « 1);
00114     mr_twi.write(0x00); // Command mode
00115     mr_twi.write(a_cmd);
00116     mr_twi.stop();
00117 }
00118
00119 void component::SSD1306_OLED::sendData(uint8_t a_data)
00120 {
00121     mr_twi.start();
00122     mr_twi.write(m_i2cAddr « 1);
00123     mr_twi.write(0x40); // Data mode
00124     mr_twi.write(a_data);
00125     mr_twi.stop();
00126 }
```

9.75 SSD1306_OLED.h File Reference

Header file for SSD1306 128x64 / 128x32 I2C OLED display driver.

```
#include "ha_base.h"
#include "TWI.h"
```

Classes

- class component::SSD1306_OLED

Namespaces

- namespace component

  AVR chip external components.

9.75.1 Detailed Description

Header file for SSD1306 128x64 / 128x32 I2C OLED display driver.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file SSD1306_OLED.h.

9.76 SSD1306_OLED.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef SSD1306_OLED_H
00010 #define SSD1306_OLED_H
00011
00012 #include "ha_base.h"
00013 #include "TWI.h"
00014
00015 namespace component
00016 {
00017
00018 class SSD1306_OLED
00019 {
00020 public:
00027     SSD1306_OLED(uint8_t a_i2cAddress = 0x3C, uint8_t a_width = 128, uint8_t a_height = 64);
00028
00031     void init();
00032
00035     void clear();
00036
00042     void setCursor(uint8_t a_col, uint8_t a_page);
00043
```

```
00048     void print(const char *ap_str);
00049
00050 private:
00051     void sendCommand(uint8_t a_cmd);
00052     void sendData(uint8_t a_data);
00053
00054     uint8_t m_i2cAddr;
00055     uint8_t m_width;
00056     uint8_t m_height;
00057     core::TWI &mr_twi;
00058 };
00059
00060 }
00061
00062 #endif
```

9.77 StepperMotor.cpp File Reference

```
#include "StepperMotor.h"
#include <math.h>
```

9.78 StepperMotor.cpp

Go to the documentation of this file.

```
00001 #include "StepperMotor.h"
00002 #include <math.h>
00003
00004 // TODO: steps to angle
00005 // TODO: maximum speed
00006
00007 component::StepperMotor::StepperMotor(const mode &ar_mode,
00008                                       const io::Pin &ar_pinCoil1,
00009                                       const io::Pin &ar_pinCoil2,
00010                                       const io::Pin &ar_pinCoil3,
00011                                       const io::Pin &ar_pinCoil4)
00012     : m_pinCoil1(ar_pinCoil1),
00013       m_pinCoil2(ar_pinCoil2),
00014       m_pinCoil3(ar_pinCoil3),
00015       m_pinCoil4(ar_pinCoil4),
00016       stepMode(ar_mode),
00017       m_goalReached(0),
00018       m_stepNumber(0),
00019       m_targetStep(0),
00020       m_currentPos(0),
00021       m_profileTime(0),
00022       m_profileSpeed(0),
00023       m_speedTimeProduct(0),
00024       m_profilePosition(0)
00025 {
00026
00027     m_pinCoil1.toOutput();
00028     m_pinCoil2.toOutput();
00029     m_pinCoil3.toOutput();
00030     m_pinCoil4.toOutput();
00031
00032 }
00033
00034 component::StepperMotor::~StepperMotor()
00035 {
00036
00037 }
00038
00039 void component::StepperMotor::beginMove(const int16_t a_step)
00040 {
00041     m_stepNumber = 0;
00042     m_targetStep = a_step;
00043     m_profileTime = 0;
00044     m_profileSpeed = 0;
00045     m_speedTimeProduct = 0;
00046     m_profilePosition = 0;
00047     m_goalReached = (a_step == 0) ? 1 : 0;
00048 }
00049
```

```
00050 void component::StepperMotor::setCurrentPos(uint16_t a_currentPos)
00051 {
00052     m_currentPos = a_currentPos;
00053 }
00054
00055 uint16_t component::StepperMotor::currentPos()
00056 {
00057     return m_currentPos;
00058 }
00059
00060
00061 void component::StepperMotor::step(const int16_t a_step,
00062         const uint16_t a_speed,
00063         const uint16_t a_accel,
00064         const uint16_t a_decel)
00065 {
00066     uint16_t l_stepDelay_ms = 0;
00067     uint16_t l_step=0;
00068
00069     // a different target, or a call after the goal was reached, starts
00070     // a new move and resets the profile state of the previous move
00071     if (m_goalReached || (a_step != m_targetStep))
00072     {
00073         beginMove(a_step);
00074     }
00075
00076     if (a_step<=0)
00077     {
00078        l_step = -a_step;
00079     } else
00080     {
00081         l_step = a_step;
00082     }
00083
00084     l_stepDelay_ms = computeStepDelay(l_step,
00085                                       a_speed,
00086                                       a_accel,
00087                                       a_decel);
00088
00089     if (l_stepDelay_ms>0)
00090     {
00091
00092     if (a_step<=0)
00093     {
00094         m_stepNumber++;
00095         m_currentPos++;
00096         if (m_stepNumber == -a_step)
00097         {
00098             m_goalReached = 1;
00099         }
00100
00101     } else {
00102
00103         m_stepNumber--;
00104         m_currentPos--;
00105         if (m_stepNumber == -a_step)
00106         {
00107             m_goalReached = 1;
00108         }
00109     }
00110
00111
00112     switch (stepMode)
00113     {
00114         case mode::fullStep:
00115         {
00116             // equivalent to m_stepNumber % 4
00117             // 1 3 5 7 <->
00118             stepPulse(2*(m_stepNumber & 3)+1);
00119             break;
00120         }
00121
00122         case mode::halfStep:
00123         {
00124             // equivalent to m_stepNumber % 8
00125             // 0 1 2 3 4 5 6 7 <->
00126             stepPulse(m_stepNumber & 7);
00127             break;
00128         }
00129     }
00130
00131     stepDelay(l_stepDelay_ms);
00132
00133     }
00134     else
00135     {
00136         // no step fired this tick: keep the 1 ms polling cadence
```

```
00137         stepDelay(1);
00138     }
00139
00140 }
00141
00142 uint16_t component::StepperMotor::computeStepDelay(uint16_t a_step,
00143                         const uint16_t a_speed,
00144                         const uint16_t a_accel,
00145                         const uint16_t a_decel)
00146 {
00147
00148     // a profile cannot be driven by a zero speed, acceleration or
00149     // deceleration: fire no step and avoid the division by zero
00150     if ((a_speed == 0) || (a_accel == 0) || (a_decel == 0))
00151     {
00152         return 0;
00153     }
00154
00155     uint32_t l_accelTime= (1000UL*a_speed/a_accel);
00156     uint32_t l_decelTime = (1000UL*a_speed/a_decel);
00157     uint16_t l_speed = a_speed;   // rescaled when the ramp is shortened
00158     // signed and 32-bit wide: the constant-speed phase can be negative
00159     // before the ramp correction below, and 1000*steps/step/s overflows
00160     // int16 for any move longer than a few steps
00161     int32_t l_constSpeedTime = static_cast<int32_t>(1000UL*a_step/a_speed)
00162                                - static_cast<int32_t>(l_accelTime/2UL)
00163                                - static_cast<int32_t>(l_decelTime/2UL);
00164
00165
00166
00167
00168     if (l_constSpeedTime<0)
00169     {
00170         l_constSpeedTime=0;
00171         // sqrtf of the float ratio: the old integer division
00172         // a_step/a_accel truncated to 0 for any realistic ramp
00173         l_accelTime = static_cast<uint32_t>(1000UL*sqrtf(static_cast<float>(a_step)
00174                                                          / static_cast<float>(a_accel)));
00175         l_speed=static_cast<uint16_t>(l_accelTime*a_accel/1000UL);
00176         l_decelTime = 1000UL*l_speed/a_decel;
00177
00178     }
00179
00180
00181
00182
00183     if (m_profileTime<=l_accelTime)
00184     {
00185         m_profileSpeed=a_accel*m_profileTime;
00186
00187     }
00188     else if ((m_profileTime>l_accelTime) && (m_profileTime<=(l_accelTime+l_constSpeedTime)))
00189     {
00190          m_profileSpeed=1000UL*l_speed;
00191
00192     }
00193     else if
      ((m_profileTime>(l_accelTime+l_constSpeedTime))&&(m_profileTime<=l_accelTime+l_constSpeedTime+l_decelTime))
00194     {
00195         m_profileSpeed=
      (1000UL*l_speed)-a_decel*(m_profileTime-l_accelTime-static_cast<uint32_t>(l_constSpeedTime));
00196
00197
00198     }
00199
00200
00201     m_profileTime=m_profileTime+1;
00202
00203
00204     m_speedTimeProduct = m_speedTimeProduct+m_profileSpeed;
00205
00206     if (m_speedTimeProduct - m_profilePosition >= 1000000UL)
00207     {
00208         m_profilePosition=m_profilePosition+1000000UL;
00209         return 1;
00210     }
00211     else
00212     {
00213         return 0;
00214     }
00215
00216
00217
00218
00219
00220 }
00221
```

```
00222
00223 void component::StepperMotor::step(const int16_t a_step, const uint16_t a_speed)
00224 {
00225     // a different target, or a call after the goal was reached, starts a
00226     // new move; the step delay is recomputed on each call instead of
00227     // being cached once from the first call
00228     if (m_goalReached || (a_step != m_targetStep))
00229     {
00230         beginMove(a_step);
00231     }
00232     uint16_t l_stepDelay_ms = static_cast<uint16_t>(1000UL/a_speed);
00233
00234     if (a_step<=0)
00235     {
00236         m_stepNumber++;
00237         m_currentPos++;
00238         if (m_stepNumber == -a_step)
00239         {
00240             m_goalReached = 1;
00241         }
00242
00243     } else {
00244
00245         m_stepNumber--;
00246         m_currentPos--;
00247         if (m_stepNumber == -a_step)
00248         {
00249             m_goalReached = 1;
00250         }
00251     }
00252
00253     switch (stepMode)
00254     {
00255         case mode::fullStep:
00256         {
00257             // equivalent to m_stepNumber % 4
00258             // 1 3 5 7 <->
00259             stepPulse(2*(m_stepNumber & 3)+1);
00260             break;
00261         }
00262
00263         case mode::halfStep:
00264         {
00265             // equivalent to m_stepNumber % 8
00266             // 0 1 2 3 4 5 6 7 <->
00267             stepPulse(m_stepNumber & 7);
00268             break;
00269         }
00270     }
00271
00272     stepDelay(l_stepDelay_ms);
00273 }
00274
00275 void component::StepperMotor::step(const int16_t a_step, const uint16_t a_speed, const float
    a_stepAngle)
00276 {
00277     // a different target, or a call after the goal was reached, starts a
00278     // new move; the step delay is recomputed on each call instead of
00279     // being cached once from the first call
00280     if (m_goalReached || (a_step != m_targetStep))
00281     {
00282         beginMove(a_step);
00283     }
00284     uint16_t l_stepDelay_ms = static_cast<uint16_t>(1000UL*a_stepAngle/a_speed);
00285
00286     if (a_step<=0)
00287     {
00288         m_stepNumber++;
00289         m_currentPos++;
00290         if (m_stepNumber == -a_step)
00291         {
00292             m_goalReached = 1;
00293         }
00294
00295     } else {
00296
00297         m_stepNumber--;
00298         m_currentPos--;
00299         if (m_stepNumber == -a_step)
00300         {
00301             m_goalReached = 1;
00302         }
00303     }
00304
00305     switch (stepMode)
00306     {
00307         case mode::fullStep:
```

```
00308         {
00309             // equivalent to m_stepNumber % 4
00310             // 1 3 5 7 <->
00311             stepPulse(2*(m_stepNumber & 3)+1);
00312             break;
00313         }
00314 
00315         case mode::halfStep:
00316         {
00317             // equivalent to m_stepNumber % 8
00318             // 0 1 2 3 4 5 6 7 <->
00319             stepPulse(m_stepNumber & 7);
00320             break;
00321         }
00322     }
00323 
00324     stepDelay(l_stepDelay_ms);
00325 }
00326 
00327 
00328 void component::StepperMotor::stepPulse(const uint8_t a_stepPulse)
00329 {
00330 
00331     switch (a_stepPulse) {
00332       case 0:    // 1000
00333       {
00334         m_pinCoil1.setHigh();
00335         m_pinCoil2.setLow();
00336         m_pinCoil3.setLow();
00337         m_pinCoil4.setLow();
00338         break;
00339       }
00340       case 1:    // 1100
00341       {
00342         m_pinCoil1.setHigh();
00343         m_pinCoil2.setHigh();
00344         m_pinCoil3.setLow();
00345         m_pinCoil4.setLow();
00346         break;
00347       }
00348       case 2:    //0100
00349       {
00350         m_pinCoil1.setLow();
00351         m_pinCoil2.setHigh();
00352         m_pinCoil3.setLow();
00353         m_pinCoil4.setLow();
00354         break;
00355       }
00356       case 3:    //0110
00357       {
00358         m_pinCoil1.setLow();
00359         m_pinCoil2.setHigh();
00360         m_pinCoil3.setHigh();
00361         m_pinCoil4.setLow();
00362         break;
00363       }
00364       case 4:    //0010
00365       {
00366         m_pinCoil1.setLow();
00367         m_pinCoil2.setLow();
00368         m_pinCoil3.setHigh();
00369         m_pinCoil4.setLow();
00370         break;
00371       }
00372       case 5:    //0011
00373       {
00374         m_pinCoil1.setLow();
00375         m_pinCoil2.setLow();
00376         m_pinCoil3.setHigh();
00377         m_pinCoil4.setHigh();
00378         break;
00379       }
00380       case 6:    //0001
00381       {
00382         m_pinCoil1.setLow();
00383         m_pinCoil2.setLow();
00384         m_pinCoil3.setLow();
00385         m_pinCoil4.setHigh();
00386         break;
00387       }
00388       case 7:    //1001
00389       {
00390         m_pinCoil1.setHigh();
00391         m_pinCoil2.setLow();
00392         m_pinCoil3.setLow();
00393         m_pinCoil4.setHigh();
00394         break;
```

```
00395       }
00396     }
00397
00398
00399 }
00400
00401
00402
00403 void component::StepperMotor::stepDelay(uint16_t a_stepDelay)
00404 {
00405     while(a_stepDelay--)
00406     {
00407         _delay_ms(1);
00408     }
00409 }
00410
00411 uint8_t component::StepperMotor::goalReached()
00412 {
00413     return m_goalReached;
00414
00415 }
```

9.79 StepperMotor.h File Reference

```
#include "ha_base.h"
#include "Pin.h"
```

Classes

- class component::StepperMotor

Namespaces

- namespace component

  AVR chip external components.

Enumerations

- enum class component::mode : uint8_t { component::fullStep =0 , component::halfStep }

9.80 StepperMotor.h

Go to the documentation of this file.

```
00001
00060
00061
00062 #ifndef STEPPERMOTOR_H
00063 #define STEPPERMOTOR_H
00064 #include "ha_base.h"
00065 #include "Pin.h"
00066
00067 namespace component
00068 {
00069
00070 enum class mode : uint8_t {
00071     fullStep=0,
00072     halfStep,
00073 };
00074
00075 class StepperMotor
```

```
00076 {
00077 public:
00078
00079     StepperMotor(const mode &ar_mode,
00080                  const io::Pin &ar_pinCoil1,
00081                  const io::Pin &ar_pinCoil2,
00082                  const io::Pin &ar_pinCoil3,
00083                  const io::Pin &ar_pinCoil4);
00084
00085     ~StepperMotor();
00086
00089
00090     void step(const int16_t a_step,
00091               const uint16_t a_speed);
00092
00093     void step(const int16_t a_step,
00094               const uint16_t a_speed,
00095               const float a_stepAngle);
00096
00097     void step(const int16_t a_step,
00098               const uint16_t a_speed,
00099               const uint16_t a_accel,
00100              const uint16_t a_decel);
00101
00102     void stepPulse(const uint8_t a_stepPulse);
00103
00108     void stepDelay(uint16_t a_stepDelay);
00109
00110     uint8_t goalReached();
00111
00112     void setCurrentPos(uint16_t a_currentPos);
00113
00114     uint16_t currentPos();
00115
00116
00137     uint16_t computeStepDelay(uint16_t a_step,
00138                               const uint16_t a_speed,
00139                               const uint16_t a_accel,
00140                               const uint16_t a_decel);
00141
00142     uint16_t m_accelTime;
00143     uint16_t m_decelTime;
00144     uint16_t m_constSpeedTime;
00145
00146
00147
00148 private:
00149
00150 //    uint8_t computeStepDelay(int16_t a_step,
00151 //                             const uint16_t a_speed,
00152 //                             const uint16_t a_accel,
00153 //                             const uint16_t a_decel);
00154
00160     void beginMove(const int16_t a_step);
00161
00162     io::Pin m_pinCoil1;
00163     io::Pin m_pinCoil2;
00164     io::Pin m_pinCoil3;
00165     io::Pin m_pinCoil4;
00166
00167     mode stepMode;
00168     uint8_t m_goalReached;
00169     int16_t m_stepNumber;
00170     int16_t m_targetStep;
00171     uint16_t m_currentPos;
00172     // speed profile state of the current move, recomputed per call
00173     uint32_t m_profileTime;
00174     uint32_t m_profileSpeed;
00175     uint64_t m_speedTimeProduct;
00176     uint64_t m_profilePosition;
00177
00178
00179
00180
00181 };
00182
00183 }
00184 #endif // STEPPERMOTOR_H
```

9.81 ADC.cpp File Reference

```
#include "ADC.h"
#include <util/atomic.h>
#include "MCU.h"
```

9.82 ADC.cpp

Go to the documentation of this file.

```
00001 #include "ADC.h"
00002 #include <util/atomic.h>
00003 #include "MCU.h"
00004
00005 volatile uint16_t core::ADConverter::m_conversionResult = 0;
00006 uint8_t core::ADConverter::m_resolution = 10;
00007
00008
00009 core::ADConverter& core::ADConverter::getInstance(const referenceVoltage &ar_refVoltage,
00010                                                   const clockPrescaler& ar_clockPrescaler,
00011                                                   const autoTriggerSource& ar_autoTriggerSource,
00012                                                   const io::Pin &ar_pin)
00013 {
00014
00015     static ADConverter l_instance(ar_refVoltage,
00016                                   ar_clockPrescaler,
00017                                   ar_autoTriggerSource,
00018                                   ar_pin);
00019
00020     return l_instance;
00021 }
00022
00023 core::ADConverter::ADConverter(const referenceVoltage &ar_refVoltage,
00024                                const clockPrescaler& ar_clockPrescaler,
00025                                const autoTriggerSource& ar_autoTriggerSource,
00026                                const io::Pin &ar_pin)
00027 {
00028     core::MCU::enableADC(1);
00029     selectAnalogInput(ar_pin);
00030     selectReferenceVoltage(ar_refVoltage);
00031     selectClockPrescaler(ar_clockPrescaler);
00032     enableAutoTrigger(1);
00033     selectAutoTriggerSource(ar_autoTriggerSource);
00034     enableConversionCompleteInterrupt(1);
00035
00036
00037 }
00038
00039
00040 core::ADConverter::~ADConverter()
00041 {
00042
00043 }
00044
00045
00046 void core::ADConverter::selectReferenceVoltage(const referenceVoltage& ar_refVoltage)
00047 {
00048     ADC_SELECT_REF_VOLTAGE(static_cast<uint8_t>(ar_refVoltage));
00049
00050 }
00051
00052 void core::ADConverter::selectAnalogInput(io::Pin a_pin)
00053 {
00054     a_pin.toInput(0);
00055     ADC_SELECT_ANALOG_INPUT(a_pin.getPinNumber());
00056     // disables the digital input buffer of the selected pin only
00057     ADC_DISABLE_DIGITAL_INPUT_REGISTER(a_pin.getPinNumber());
00058
00059 }
00060
00061 void core::ADConverter::start()
00062 {
00063     ADC_ENABLE;
00064     ADC_START_CONVERSION;
00065 }
00066
00067 void core::ADConverter::stop()
```

```
00068 {
00069     // Clearing ADSC does not abort an ongoing conversion, it only stops the
00070     // auto-trigger sequence, so stopping relies on clearing ADEN instead.
00071     ADC_DISABLE;
00072 }
00073
00074 void core::ADConverter::enableAutoTrigger(const uint8_t a_enable)
00075 {
00076     if (a_enable) {
00077         ADC_ENABLE_AUTOTRIGGER;
00078
00079     } else {
00080         ADC_DISABLE_AUTOTRIGGER;
00081     }
00082
00083 }
00084
00085 void core::ADConverter::enableConversionCompleteInterrupt(const uint8_t a_enable)
00086 {
00087     if (a_enable) {
00088         ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT;
00089
00090     } else {
00091         ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT;
00092     }
00093
00094 }
00095
00096 void core::ADConverter::selectClockPrescaler(const clockPrescaler& ar_clockPrescaler)
00097 {
00098     ADC_SELECT_CLOCK_PRESCALER(static_cast<uint8_t>(ar_clockPrescaler));
00099
00100 }
00101
00102
00103 void core::ADConverter::selectAutoTriggerSource(const autoTriggerSource& ar_autoTriggerSource)
00104 {
00105     ADC_SELECT_AUTO_TRIGGER_SOURCE(static_cast<uint8_t>(ar_autoTriggerSource));
00106
00107
00108 }
00109
00110 void core::ADConverter::conversionCompleteServiceRoutine()
00111 {
00112
00113     static uint32_t l_resultData = 0;
00114     static uint16_t l_resultDataIndex = 0;
00115
00116
00117
00118     switch (m_resolution)
00119     {
00120         case 8:
00121         {
00122             m_conversionResult = ADC » 8;
00123             break;
00124         }
00125         case 9:
00126         {
00127             m_conversionResult = ADC » 7;
00128              break;
00129         }
00130         case 10:
00131         {
00132             m_conversionResult = ADC;
00133             break;
00134         }
00135         case 11:
00136         {
00137
00138             if (l_resultDataIndex < 4)
00139             {
00140                 l_resultData += ADC;
00141                 l_resultDataIndex++;
00142
00143             }
00144             else
00145             {
00146                 m_conversionResult = l_resultData » 1;
00147                 l_resultData = 0;
00148                 l_resultDataIndex = 0;
00149             }
00150
00151             break;
00152         }
00153         case 12:
00154         {
```

```
00155             if (l_resultDataIndex < 16)
00156             {
00157                 l_resultData += ADC;
00158                 l_resultDataIndex++;
00159 
00160             }
00161             else
00162             {
00163                 m_conversionResult = l_resultData » 2;
00164                 l_resultData = 0;
00165                 l_resultDataIndex = 0;
00166             }
00167             break;
00168         }
00169         case 13:
00170         {
00171             if (l_resultDataIndex < 64)
00172             {
00173                 l_resultData += ADC;
00174                 l_resultDataIndex++;
00175 
00176             }
00177             else
00178             {
00179                 m_conversionResult = l_resultData » 3;
00180                 l_resultData = 0;
00181                 l_resultDataIndex = 0;
00182             }
00183             break;
00184         }
00185         case 14:
00186         {
00187             if (l_resultDataIndex < 256)
00188             {
00189                 l_resultData += ADC;
00190                 l_resultDataIndex++;
00191 
00192             }
00193             else
00194             {
00195                 m_conversionResult = l_resultData » 4;
00196                 l_resultData = 0;
00197                 l_resultDataIndex = 0;
00198             }
00199             break;
00200         }
00201         case 15:
00202         {
00203             if (l_resultDataIndex < 1024)
00204             {
00205                 l_resultData += ADC;
00206                 l_resultDataIndex++;
00207 
00208             }
00209             else
00210             {
00211                 m_conversionResult = l_resultData » 5;
00212                 l_resultData = 0;
00213                 l_resultDataIndex = 0;
00214             }
00215             break;
00216         }
00217         case 16:
00218         {
00219             if (l_resultDataIndex < 4096)
00220             {
00221 
00222                 l_resultData += ADC;
00223                 l_resultDataIndex++;
00224 
00225             }
00226             else
00227             {
00228                 m_conversionResult = l_resultData » 6;
00229                 l_resultData = 0;
00230                 l_resultDataIndex = 0;
00231             }
00232             break;
00233         }
00234 
00235     }
00236 
00237 
00238 
00239 
00240 }
00241 
```

```
00242
00243
00244
00245 uint16_t core::ADConverter::getConversionResult( const resolution& ar_resolution)
00246 {
00247
00248
00249     switch (ar_resolution)
00250     {
00251         case core::resolution::res_8bit:
00252         {
00253             ADC_ADJUST_RESULT_LEFT;
00254             m_resolution = 8;
00255             break;
00256         }
00257         case core::resolution::res_9bit:
00258         {
00259             ADC_ADJUST_RESULT_LEFT;
00260             m_resolution = 9;
00261             break;
00262         }
00263         case core::resolution::res_10bit:
00264         {
00265             ADC_ADJUST_RESULT_RIGHT;
00266             m_resolution = 10;
00267             break;
00268         }
00269         case core::resolution::res_11bit:
00270         {
00271
00272             ADC_ADJUST_RESULT_RIGHT;
00273             m_resolution = 11;
00274             break;
00275         }
00276         case core::resolution::res_12bit:
00277         {
00278
00279             ADC_ADJUST_RESULT_RIGHT;
00280             m_resolution = 12;
00281             break;
00282         }
00283         case core::resolution::res_13bit:
00284         {
00285
00286             ADC_ADJUST_RESULT_RIGHT;
00287             m_resolution = 13;
00288             break;
00289         }
00290         case core::resolution::res_14bit:
00291         {
00292
00293             ADC_ADJUST_RESULT_RIGHT;
00294             m_resolution = 14;
00295             break;
00296         }
00297         case core::resolution::res_15bit:
00298         {
00299
00300             ADC_ADJUST_RESULT_RIGHT;
00301             m_resolution = 15;
00302             break;
00303         }
00304         case core::resolution::res_16bit:
00305         {
00306
00307             ADC_ADJUST_RESULT_RIGHT;
00308             m_resolution = 16;
00309             break;
00310         }
00311     }
00312
00313     uint16_t result;
00314     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00315     {
00316         result = m_conversionResult;
00317     }
00318     return result;
00319
00320
00321 }
```

9.83 ADC.h File Reference

```
#include "ha_base.h"
#include "Pin.h"
```

Classes

- class core::ADConverter

  Class for handling ADC.

Namespaces

- namespace core

  AVR chip internal core components.

Enumerations

- enum class core::resolution : uint8_t {
  core::res_8bit =0 , core::res_9bit , core::res_10bit , core::res_11bit ,
  core::res_12bit , core::res_13bit , core::res_14bit , core::res_15bit ,
  core::res_16bit }
- enum class core::referenceVoltage : uint8_t { core::AREF =0 , core::AVCC , core::internal =3 }
- enum class core::clockPrescaler : uint8_t {
  core::PS_2 = 1 , core::PS_4 , core::PS_8 , core::PS_16 ,
  core::PS_32 , core::PS_64 , core::PS_128 }
- enum class core::autoTriggerSource : uint8_t {
  core::freeRunning , core::analogComparator , core::extInterrupt , core::timer0CompareA ,
  core::timer0Overflow , core::timer1CompareB , core::timer1Overflow , core::timer1Capture }

9.84 ADC.h

Go to the documentation of this file.

```
00001 #ifndef ADC_H
00002 #define ADC_H
00003 #include "ha_base.h"
00004 #include "Pin.h"
00005
00014 namespace core
00015 {
00024 enum class resolution : uint8_t {
00025     res_8bit=0,
00026     res_9bit,
00027     res_10bit,
00028     res_11bit,
00029     res_12bit,
00030     res_13bit,
00031     res_14bit,
00032     res_15bit,
00033     res_16bit
00034 };
00035
00044 enum class referenceVoltage : uint8_t {
00045     AREF=0,
00046     AVCC,
00047     internal=3
00048 };
00049
00050 // The referenceVoltage values encode the ADMUX REFS field (bits 7:6) as a
```

```
00051 // 2-bit number: 0b00 selects AREF, 0b01 selects AVCC (REFS0) and 0b11
00052 // selects the internal 1.1V reference (REFS1 | REFS0).
00053 static_assert(static_cast<uint8_t>(referenceVoltage::AREF) == 0,
00054                "AREF must encode REFS = 0b00");
00055 static_assert(static_cast<uint8_t>(referenceVoltage::AVCC) == 1,
00056                "AVCC must encode REFS = 0b01");
00057 static_assert(static_cast<uint8_t>(referenceVoltage::internal) == 3,
00058                "internal must encode REFS = 0b11");
00059
00060
00069 enum class clockPrescaler : uint8_t {
00070      PS_2 = 1,
00071      PS_4,
00072      PS_8,
00073      PS_16,
00074      PS_32,
00075      PS_64,
00076      PS_128
00077 };
00086 enum class autoTriggerSource : uint8_t {
00087     freeRunning,
00088     analogComparator,
00089     extInterrupt,
00090     timer0CompareA,
00091     timer0Overflow,
00092     timer1CompareB,
00093     timer1Overflow,
00094     timer1Capture
00095 };
00104 class ADConverter
00105 {
00106 public:
00115     static ADConverter& getInstance(const referenceVoltage& ar_refVoltage = referenceVoltage::AVCC,
00116                                     const clockPrescaler& ar_clockPrescaler = clockPrescaler::PS_128,
00117                                     const autoTriggerSource& ar_autoTriggerSource =
      autoTriggerSource::freeRunning,
00118                                     const io::Pin &ar_pin = io::Pin(0,io::PortC));
00121     void start();
00124     void stop();
00129     void selectReferenceVoltage(const referenceVoltage& ar_refVoltage);
00134     void selectAnalogInput(io::Pin a_pin);
00139     void selectClockPrescaler(const clockPrescaler& ar_clockPrescaler);
00144     void enableConversionCompleteInterrupt(const uint8_t a_enable);
00149     void enableAutoTrigger(const uint8_t a_enable);
00154     void selectAutoTriggerSource(const autoTriggerSource& ar_autoTriggerSource);
00160     uint16_t getConversionResult(const resolution& ar_resolution = resolution::res_10bit);
00163     static void conversionCompleteServiceRoutine() __asm__(STR(ADC_CONVERSION_COMPLETE_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__));
00164
00165
00166 protected:
00167
00168
00169
00170 private:
00180     ADConverter(const referenceVoltage& ar_refVoltage,
00181                 const clockPrescaler& ar_clockPrescaler,
00182                 const autoTriggerSource &ar_autoTriggerSource,
00183                 const io::Pin &ar_pin);
00184
00187     ~ADConverter();
00188
00191     ADConverter(const ADConverter&);
00192
00195     const ADConverter& operator=(const ADConverter&);
00196
00197     static volatile uint16_t m_conversionResult;
00198
00199     static uint8_t m_resolution;
00200
00201 };
00202
00203 }
00204 #endif
```

9.85 ExternInterrupt.cpp File Reference

```
#include "ExternInterrupt.h"
```

9.86 ExternInterrupt.cpp

Go to the documentation of this file.

```
00001 #include "ExternInterrupt.h"
00002
00003 core::ExternInterrupt& core::ExternInterrupt::getInstance()
00004 {
00005     static ExternInterrupt l_instance;
00006     return l_instance;
00007
00008 }
00009
00010
00011 core::ExternInterrupt::ExternInterrupt()
00012 {
00013
00014 }
00015
00016 core::ExternInterrupt::~ExternInterrupt()
00017 {
00018
00019 }
00020
00021 void core::ExternInterrupt::setInt0SenseControl(const senseControl& ar_senseControl)
00022 {
00023     EXT_INT_SET_INT0_SENSE_CONTROL(static_cast<uint8_t>(ar_senseControl));
00024
00025 }
00026
00027 void core::ExternInterrupt::setInt1SenseControl(const senseControl& ar_senseControl)
00028 {
00029     EXT_INT_SET_INT1_SENSE_CONTROL(static_cast<uint8_t>(ar_senseControl));
00030
00031 }
00032
00033 void core::ExternInterrupt::enableInt0(const uint8_t a_enable)
00034 {
00035     if (a_enable) {
00036         EXT_INT_ENABLE_INT0;
00037
00038     } else {
00039         EXT_INT_DISABLE_INT0;
00040     }
00041
00042 }
00043
00044 void core::ExternInterrupt::enableInt1(const uint8_t a_enable)
00045 {
00046     if (a_enable) {
00047         EXT_INT_ENABLE_INT1;
00048
00049     } else {
00050         EXT_INT_DISABLE_INT1;
00051     }
00052 }
00053
00054
00055 void core::ExternInterrupt::enablePinChange(const pinChangePort& ar_pinChangePort, const uint8_t
    a_enable)
00056 {
00057     if (a_enable) {
00058         EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(static_cast<uint8_t>(ar_pinChangePort));
00059
00060     } else {
00061         EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(static_cast<uint8_t>(ar_pinChangePort));
00062     }
00063
00064 }
00065
00066 void core::ExternInterrupt::enablePinChangeMaskPortB(const uint8_t a_pinNumber, const uint8_t
    a_enable)
00067 {
00068     if (a_enable) {
00069         EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(a_pinNumber);
00070
00071     } else {
00072         EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(a_pinNumber);
00073     }
00074
00075 }
00076
00077
00078 void core::ExternInterrupt::enablePinChangeMaskPortC(const uint8_t a_pinNumber, const uint8_t
    a_enable)
00079 {
```

```
00080     if (a_enable) {
00081         EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(a_pinNumber);
00082 
00083     } else {
00084         EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(a_pinNumber);
00085     }
00086 
00087 }
00088 
00089 void core::ExternInterrupt::enablePinChangeMaskPortD(const uint8_t a_pinNumber, const uint8_t
      a_enable)
00090 {
00091     if (a_enable) {
00092         EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(a_pinNumber);
00093 
00094     } else {
00095         EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(a_pinNumber);
00096     }
00097 }
00098 
00099 // weak default interrupt handlers: empty bodies the application may
00100 // override by defining the same vector symbol (e.g. ISR(TIMER0_COMPA_vect))
00101 
00102 void core::ExternInterrupt::Int0ServiceRoutine() {}
00103 
00104 void core::ExternInterrupt::Int1ServiceRoutine() {}
00105 
00106 void core::ExternInterrupt::pinChangePortBServiceRoutine() {}
00107 
00108 void core::ExternInterrupt::pinChangePortCServiceRoutine() {}
00109 
00110 void core::ExternInterrupt::pinChangePortDServiceRoutine() {}
```

9.87 ExternInterrupt.h File Reference

Header file of the ExternInterrupt class.

```
#include "ha_base.h"
```

Classes

- class core::ExternInterrupt

Namespaces

- namespace core

  AVR chip internal core components.

Enumerations

- enum class core::senseControl : uint8_t { core::lowLevel =0 , core::logicalChange , core::fallingEdge , core::risingEdge }
- enum class core::pinChangePort : uint8_t { core::PCINTB =0 , core::PCINTC , core::PCINTD }

9.87.1 Detailed Description

Header file of the ExternInterrupt class.

class to handle externally triggered interrupts.

Usage example (external interrupt):

#include "PushButton.h" #include "Led.h" #include "ExternInterrupt.h"

#define PUSHBUTTON_NUMBER 2 #define MAIN_LED_NUMBER 0 #define INTERRUPT_LED_NUMBER 1 #define DELAYTIME 1000

instantiate a Led object declaration of global variables (shut up warning) extern component::Led MainLed; extern component::Led InterruptLed; component::Led MainLed(io::Pin(MAIN_LED_NUMBER,io::PortB)); component::Led InterruptLed(io::Pin(INTERRUPT_LED_NUMBER,io::PortB));

instantiate a Led object declaration of global variable (shut up warning) extern component::PushButton PushButton; component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));

int main(void) {

Init

instantiate the external interrupt manager core::ExternInterrupt &myExternInterrupt = core::ExternInterrupt::getInstance(); myExternInterrupt.enableInt0(1); myExternInterrupt.setInt0SenseControl(core::senseControl::logicalChange);

Mainloop while (1) {

MainLed.toggle(); _delay_ms(DELAYTIME);

} return 0; }

void core::ExternInterrupt::Int0ServiceRoutine() { if (PushButton.isPressed()) { InterruptLed.on(); } else { Interrupt↩ Led.off(); }

}

Usage example (pin change interrupt):

#include "PushButton.h" #include "Led.h" #include "ExternInterrupt.h"

#define PUSHBUTTON_1_NUMBER 2 #define PUSHBUTTON_2_NUMBER 3 #define MAIN_LED_NUMBER 0 #define INTERRUPT_LED_1_NUMBER 1 #define INTERRUPT_LED_2_NUMBER 2 #define DELAYTIME 1000

instantiate a Led objects declaration of global variables (shut up warning) extern component::Led MainLed; extern component::Led InterruptLed1; extern component::Led InterruptLed2; component::Led MainLed(io::Pin(↩ MAIN_LED_NUMBER,io::PortB)); component::Led InterruptLed1(io::Pin(INTERRUPT_LED_1_NUMBER,io::Port↩ B)); component::Led InterruptLed2(io::Pin(INTERRUPT_LED_2_NUMBER,io::PortB));

instantiate a PushButton objects declaration of global variable (shut up warning) extern component::PushButton PushButton1; extern component::PushButton PushButton2; component::PushButton PushButton1(io::Pin(↩ PUSHBUTTON_1_NUMBER,io::PortD)); component::PushButton PushButton2(io::Pin(PUSHBUTTON_2_↩ NUMBER,io::PortB));

int main(void) {

Init

instantiate the external interrupt manager core::ExternInterrupt &myExternInterrupt = core::ExternInterrupt::getInstance(); myExternInterrupt.enablePinChange(core::pinChangePort::PCINTD,1); myExternInterrupt.enablePinChange(core↩ ::pinChangePort::PCINTB,1); myExternInterrupt.enablePinChangeMaskPortD(PUSHBUTTON_1_NUMBER,1); myExternInterrupt.enablePinChangeMaskPortB(PUSHBUTTON_2_NUMBER,1);

Mainloop while (1) {

MainLed.toggle(); _delay_ms(DELAYTIME);

} return 0; }

void core::ExternInterrupt::pinChangePortDServiceRoutine() { if (PushButton1.isPressed()) { InterruptLed1.on();

} else { InterruptLed1.off();

} } void core::ExternInterrupt::pinChangePortBServiceRoutine() { if (PushButton2.isPressed()) { InterruptLed2.on(); } else { InterruptLed2.off(); }

}

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file ExternInterrupt.h.

9.88 ExternInterrupt.h

Go to the documentation of this file.

```
00001
00140 #ifndef EXTERN_INTERRUPTS_H
00141 #define EXTERN_INTERRUPTS_H
00142 #include "ha_base.h"
00143
00144
00145
00146 namespace core
00147 {
00148
00149 enum class senseControl : uint8_t {
00150     lowLevel=0,
00151     logicalChange,
00152     fallingEdge,
00153     risingEdge
00154 };
00155
00156 enum class pinChangePort : uint8_t {
00157     PCINTB=0,
00158     PCINTC,
00159     PCINTD,
00160 };
00161
00162 class ExternInterrupt
00163 {
00164
00165 public:
00166
00167     static ExternInterrupt& getInstance();
00168
00169     void setInt0SenseControl(const senseControl& ar_senseControl);
00170
```

```
00171     void setInt1SenseControl(const senseControl& ar_senseControl);
00172
00173     void enableInt0(const uint8_t a_enable);
00174
00175     void enableInt1(const uint8_t a_enable);
00176
00177     void enablePinChange(const pinChangePort& ar_pinChangePort, const uint8_t a_enable);
00178
00179     void enablePinChangeMaskPortB(const uint8_t a_pinNumber, const uint8_t a_enable);
00180
00181     void enablePinChangeMaskPortC(const uint8_t a_pinNumber, const uint8_t a_enable);
00182
00183     void enablePinChangeMaskPortD(const uint8_t a_pinNumber, const uint8_t a_enable);
00184
00185     static void Int0ServiceRoutine() __asm__(STR(EXT_INT_INT0_INTERRUPT)) __attribute__((__signal__,
    __used__, __externally_visible__, weak));
00186
00187     static void Int1ServiceRoutine() __asm__(STR(EXT_INT_INT1_INTERRUPT)) __attribute__((__signal__,
    __used__, __externally_visible__, weak));
00188
00189     static void pinChangePortBServiceRoutine() __asm__(STR(EXT_INT_PIN_CHANGE_PORTB_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00190
00191     static void pinChangePortCServiceRoutine() __asm__(STR(EXT_INT_PIN_CHANGE_PORTC_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00192
00193     static void pinChangePortDServiceRoutine() __asm__(STR(EXT_INT_PIN_CHANGE_PORTD_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00194
00195
00196 protected:
00197
00198 private:
00199
00200     ExternInterrupt();
00201
00204     ~ExternInterrupt();
00205
00208     ExternInterrupt(const ExternInterrupt&);
00209
00212     const ExternInterrupt& operator=(const ExternInterrupt&);
00213
00214 };
00215 }
00216
00217
00218
00219
00220 #endif
```

9.89 MCU.cpp File Reference

```
#include "MCU.h"
#include <avr/sleep.h>
#include <util/atomic.h>
```

9.90 MCU.cpp

Go to the documentation of this file.

```
00001 #include "MCU.h"
00002 #include <avr/sleep.h>
00003 #include <util/atomic.h>
00004
00005 void core::MCU::enableInterrupts()
00006 {
00007     sei();
00008
00009 }
00010
00011 void core::MCU::init()
00012 {
00013     enableUSART0(0);
```

```
00014     enableTimerCounter0(0);
00015     enableTimerCounter1(0);
00016     enableTimerCounter2(0);
00017     enableTWI(0);
00018     enableSPI(0);
00019     enableADC(0);
00020
00021 }
00022
00023 void core::MCU::selectSleepMode(const sleepMode &a_sleepMode)
00024 {
00025     MCU_SELECT_SLEEP_MODE(static_cast<uint8_t>(a_sleepMode));
00026
00027 }
00028
00029 void core::MCU::goToSleep(const BODMode &a_BODMode)
00030 {
00031     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00032     {
00033         switch (a_BODMode)
00034         {
00035             case core::BODMode::enabled:
00036             {
00037                 sleepEnable(1);
00038                 // the sleep instruction must run with interrupts enabled so
00039                 // that the chip can wake up again
00040                 sei();
00041                 sleep_cpu();
00042                 sleepEnable(0);
00043                 break;
00044             }
00045             case core::BODMode::disabled:
00046             {
00047                 sleepEnable(1);
00048                 // the sleep instruction must run with interrupts enabled so
00049                 // that the chip can wake up again
00050                 sei();
00051                 // the timed BOD disable sequence must immediately precede
00052                 // sleep_cpu(), an interrupt in between would break it
00053                 sleep_bod_disable();
00054                 sleep_cpu();
00055                 sleepEnable(0);
00056                 break;
00057             }
00058         }
00059     }
00060
00061 }
00062
00063 void core::MCU::sleepEnable(const uint8_t a_enable)
00064 {
00065     if (a_enable) {
00066         MCU_SLEEP_ENABLE;
00067     } else {
00068         MCU_SLEEP_DISABLE;
00069     }
00070
00071 }
00072
00073 void core::MCU::enableUSART0(const uint8_t a_enable)
00074 {
00075     if (a_enable) {
00076         MCU_USART0_ENABLE;
00077     } else {
00078         MCU_USART0_DISABLE;
00079     }
00080
00081 }
00082
00083 void core::MCU::enableTimerCounter0(const uint8_t a_enable)
00084 {
00085     if (a_enable) {
00086         MCU_TIMER0_ENABLE;
00087     } else {
00088         MCU_TIMER0_DISABLE;
00089     }
00090
00091 }
00092
00093 void core::MCU::enableTimerCounter1(const uint8_t a_enable)
00094 {
00095     if (a_enable) {
00096         MCU_TIMER1_ENABLE;
00097     } else {
00098         MCU_TIMER1_DISABLE;
00099     }
00100
```

```
00101 }
00102
00103 void core::MCU::enableTimerCounter2(const uint8_t a_enable)
00104 {
00105     if (a_enable) {
00106         MCU_TIMER2_ENABLE;
00107     } else {
00108         MCU_TIMER2_DISABLE;
00109     }
00110
00111 }
00112
00113 void core::MCU::enableTWI(const uint8_t a_enable)
00114 {
00115     if (a_enable) {
00116         MCU_TWI_ENABLE;
00117     } else {
00118         MCU_TWI_DISABLE;
00119     }
00120
00121 }
00122
00123 void core::MCU::enableSPI(const uint8_t a_enable)
00124 {
00125     if (a_enable) {
00126         MCU_SPI_ENABLE;
00127     } else {
00128         MCU_SPI_DISABLE;
00129     }
00130
00131 }
00132
00133 void core::MCU::enableADC(const uint8_t a_enable)
00134 {
00135     if (a_enable) {
00136         MCU_ADC_ENABLE;
00137     } else {
00138         MCU_ADC_DISABLE;
00139     }
00140 }
```

9.91 MCU.h File Reference

Header file of the MCU class.

```
#include "ha_base.h"
```

Classes

- class core::MCU

Namespaces

- namespace core

  AVR chip internal core components.

Enumerations

- enum class core::BODMode : uint8_t { core::enabled =0 , core::disabled }
- enum class core::sleepMode : uint8_t {
  core::Idle =0 , core::ADC_NoiseReduction , core::powerDown , core::powerSave ,
  core::standby =6 , core::extendedStandby }

9.91.1 Detailed Description

Header file of the MCU class.

Basic class abstraction of the MCU.

Usage example (test):

#include "PushButton.h" #include "Led.h" #include "ExternInterrupt.h" #include "MCU.h"

#define PUSHBUTTON_NUMBER 2 #define LED_NUMBER 0

int main(void) {

Init initialize MCU core::MCU::init();

instantiate a Led object component::Led Led(io::Pin(LED_NUMBER,io::PortB));

instantiate a Led object component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));

instantiate the external interrupt manager core::ExternInterrupt &myExternInterrupt = core::ExternInterrupt::getInstance(); myExternInterrupt.enableInt0(1); myExternInterrupt.setInt0SenseControl(core::senseControl::logicalChange);

set sleep mode core::MCU::selectSleepMode(core::sleepMode::powerDown);

Mainloop while (1) {

flash the LED for (uint8_t i=0;i<10;i++) { Led.on(); _delay_ms(100); Led.off(); _delay_ms(100); } _delay_ms(5000); Led.on(); _delay_ms(100); Led.off(); core::MCU::goToSleep(core::BODMode::enabled);

} return 0; }

void core::ExternInterrupt::Int0ServiceRoutine() {

}

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file MCU.h.

9.92 MCU.h

Go to the documentation of this file.

```
00001
00069 #ifndef MCU_H
00070 #define MCU_H
00071 #include "ha_base.h"
00072
00073
00074 namespace core
00075 {
00076
00077 enum class BODMode : uint8_t {
00078     enabled=0,
00079     disabled,
00080 };
00081
00082 enum class sleepMode : uint8_t {
00083     Idle=0,
00084     ADC_NoiseReduction,
00085     powerDown,
00086     powerSave,
00087     standby=6,
00088     extendedStandby,
00089 };
00090
00091 class MCU
00092 {
00093 public:
00094
00095     static void init();
00096
00104     static void enableInterrupts();
00105
00106     static void selectSleepMode(const sleepMode &a_sleepMode);
00107
00117     static void goToSleep(const BODMode &a_BODMode);
00118
00119     static void sleepEnable(const uint8_t ar_enable);
00120
00121     static void enableUSART0(const uint8_t a_enable);
00122
00123     static void enableTimerCounter0(const uint8_t a_enable);
00124
00125     static void enableTimerCounter1(const uint8_t a_enable);
00126
00127     static void enableTimerCounter2(const uint8_t a_enable);
00128
00129     static void enableTWI(const uint8_t a_enable);
00130
00131     static void enableSPI(const uint8_t a_enable);
00132
00133     static void enableADC(const uint8_t a_enable);
00134
00135
00136
00137
00138
00139 protected:
00140
00141
00142
00143 private:
00144
00145
00146
00147
00148
00149
00150
00151 };
00152
00153
00154
00155
00156
00157 }
00158 #endif
```

9.93 TimerCounter.h File Reference

Header file of the TimerCounter class.

```
#include "ha_base.h"
```

Classes

- class core::TimerCounter

Namespaces

- namespace core

  AVR chip internal core components.

Enumerations

- enum class core::channel : uint8_t { core::A =0 , core::B }
- enum class core::compareOutputMode : uint8_t { core::normal =0 , core::toggle , core::clear , core::set }
- enum class core::operationMode : uint8_t {
  core::normal =0 , core::PWM_PC , core::PWM_PC_8bit , core::PWM_PC_9bit ,
  core::PWM_PC_10bit , core::PWM_PFC_ICR , core::PWM_PFC_OCR , core::PWM_PC_ICR ,
  core::PWM_PC_OCR , core::fast_PWM , core::fast_PWM_8bit , core::fast_PWM_9bit ,
  core::fast_PWM_10bit , core::fast_PWM_ICR , core::fast_PWM_OCR , core::CTC_OCR ,
  core::CTC_ICR }
- enum class core::clockSource : uint16_t {
  core::noClock =0 , core::PS_1 , core::PS_8 , core::PS_32 ,
  core::PS_64 , core::PS_128 , core::PS_256 , core::PS_1024 ,
  core::extern_Clock_T0_Falling_Edge , core::extern_Clock_T0_Rising_Edge }

9.93.1 Detailed Description

Header file of the TimerCounter class.

Basic class for abstraction of the TimerCounter peripherals.

Usage example (Reaction Timer): #include "TimerCounter.h" #include "USART0.h" #include "PushButton.h" #include "Led.h"

void randomDelay(void); void printWord(uint16_t word);

instantiate the Counter object extern core::TimerCounter &myCounter; core::TimerCounter &myCounter = core::↩ TimerCounter::getInstanceTimerCounter0();

instantiate the USART0 object extern io::USART0 &myUSART0; io::USART0 &myUSART0 = io::USART0::getInstance();

#define BUFFER_SIZE 1 #define LED_NUMBER 0 #define PUSHBUTTON_NUMBER 2

int main(void) {

Init uint16_t timerValue;

char l_receiverBuffer[BUFFER_SIZE];

```
myCounter.selectClockSource(core::timerCounter::timerCounter1, core::clockSource::PS_1024);
```

/ instantiate a Led object component::Led Led(io::Pin(LED_NUMBER,io::PortB));

instantiate a Led object component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));

ready to send flag uint8_t l_ready2Send = 1;

if (myUSART0.ready2Send()) { myUSART0.sendString("Reaction Timer:\r\n"); }

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("----------\r\n"); }

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("Press any key to start.\r\n"); }

Mainloop while (1) {

myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);

wait for a key press while (!myUSART0.getNumberBytesReceived()) {

}

reset number of bytes after extracting the received data myUSART0.resetNumberBytesReceived();

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("\r\nGet ready..."); }

randomDelay();

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("\r\nGo!\r\n"); }

Led.on();

myCounter.setCounter(core::timerCounter::timerCounter1,0);

if (PushButton.isPressed()) { wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("You're only cheating yourself.\r\n"); }

} else {

wait for a button press while (!PushButton.isPressed()) {

} get counter value myCounter.getCounter(core::timerCounter::timerCounter1, &timerValue); bit shift divide by 16 = 2^4 to convert from micro to millseconds timerValue = timerValue >> 4; print response time printWord(timerValue); }

Led.off();

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("\r\nPress any key to try again.\r\n"); }

} return 0; }

void randomDelay(void) {

Waits for a "random" delay from 1 - 3.5 sec Requires timer 1 initialized and running It's not really random, but very hard to control –like coin-flipping.

```
uint16_t counter;
_delay_ms(1000);

myCounter.getCounter(core::timerCounter::timerCounter1, &counter);
uint8_t randomTime = static_cast<uint8_t>(counter);
```

type-casting the 16-bit TCNT1 as an 8-bit number keeps only the 8 least-significant (fastest-changing) bits

```
while (--randomTime) {
_delay_ms(10);
}
}

void printWord(uint16_t word) {

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + (word / 10000));

}

while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + ((word / 1000) % 10));
}

while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + ((word / 100) % 10));
}


while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + ((word / 10) % 10));
}



while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + (word % 10));
}

}
void printWord(uint16_t word) {

char timerValue = '0' + (word / 10000);

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

timerValue = '0' + ((word / 1000) % 10);
```

```
while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

timerValue = '0' + ((word / 100) % 10);

while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}
timerValue = '0' + ((word / 10) % 10);

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

timerValue = '0' + (word % 10);

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

}
```

Usage example (PWM):

#include "Led.h" #include "USART0.h" #include "TimerCounter1.h" #include "TimerCounter2.h"

uint8_t getNumber(const char *ap_receive);

#define LED_1 1 #define LED_2 2 #define LED_3 3 #define BUFFER_SIZE 4

int main(void) {

Init receiver buffer char l_receiverBuffer[BUFFER_SIZE];

instantiate Led objects component::Led Led1(io::Pin(LED_1,io::PortB)); component::Led Led2(io::Pin(LED_2,io::↩ PortB)); component::Led Led3(io::Pin(LED_3,io::PortB));

instantiate USART0 object io::USART0 &myUSART0 = io::USART0::getInstance();

instantiate Timer1 object core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance(); my↩ TimerCounter1.selectOperationMode(core::operationMode::Fast_PWM_8bit); myTimerCounter1.selectCompare↩ OutputMode(core::channel::A,core::compareOutputMode::Clear); myTimerCounter1.selectCompareOutput↩ Mode(core::channel::B,core::compareOutputMode::Clear); instantiate Timer2 object core::TimerCounter2 &my↩ TimerCounter2 = core::TimerCounter2::getInstance(); myTimerCounter2.selectOperationMode(core::operation↩ Mode::Fast_PWM); myTimerCounter2.selectCompareOutputMode(core::channel::A,core::compareOutputMode↩ ::Clear);

if (myUSART0.ready2Send()) { myUSART0.sendString("-- LED PWM Demo --\r\n"); } wait to send next string while (!myUSART0.ready2Send()){}

myTimerCounter1.start(core::clockSource::PS_64); myTimerCounter2.start(core::clockSource::PS_64);

---— Event loop ---— // while (1) {

```
if (myUSART0.ready2Send())
{
    myUSART0.sendString("\r\nEnter (0-255) for PWM duty cycle: ");
}

myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
```

wait for a key press while (myUSART0.getNumberBytesReceived()<4){}

reset number of bytes after extracting the received data myUSART0.resetNumberBytesReceived();

send back the received character if (myUSART0.ready2Send()) { myUSART0.sendFrame(reinterpret_cast<uint8↩
_t*>(l_receiverBuffer),BUFFER_SIZE); }

while (!myUSART0.ready2Send()){} myTimerCounter2.setOutputCompareRegister(core::channel::A,myTimer↩
Counter1.getOutputCompareRegister(core::channel::B)); myTimerCounter1.setOutputCompareRegister(core::channel::B,my↩
TimerCounter1.getOutputCompareRegister(core::channel::A)); myTimerCounter1.setOutputCompareRegister(core↩
::channel::A,getNumber(l_receiverBuffer));

```
}
return 0;
```

}

uint8_t getNumber(const char *ap_receive) { Gets a numerical 0-255 from the serial port. Converts from string to number. char hundreds = '0'; char tens = '0'; char ones = '0'; char thisChar = '0'; do { hundreds = tens; tens = ones; ones = thisChar; thisChar = *ap_receive; ap_receive++;

} while (*ap_receive != '\r'); return (100 * (hundreds - '0') + 10 * (tens - '0') + ones - '0'); }

Usage example (PWM on Any Pin):

#include "TimerCounter0.h" #include "Led.h"

#define LED_0 0 #define LED_1 1 #define LED_2 2 #define LED_3 3

instantiate Timer0 object extern core::TimerCounter0 &myTimerCounter0; core::TimerCounter0 &myTimerCounter0 = core::TimerCounter0::getInstance();

instantiate Led objects extern component::Led Led0; component::Led Led0(io::Pin(LED_0,io::PortB)); extern component::Led Led1; component::Led Led1(io::Pin(LED_1,io::PortB)); extern component::Led Led2; component::Led Led2(io::Pin(LED_2,io::PortB)); extern component::Led Led3; component::Led Led3(io::Pin(↩
LED_3,io::PortB));

#define DELAYTIME 3

volatile uint8_t l_brightnessA; volatile uint8_t l_brightnessB;

int main(void) {

Init uint8_t i;

sei();

myTimerCounter0.enableOutputCompareMatchInterrupt(core::channel::A,1); myTimerCounter0.enableOutput↩
CompareMatchInterrupt(core::channel::B,1); myTimerCounter0.enableOverflowInterrupt(1); myTimerCounter0.↩
start(core::clockSource::PS_1024);

---— Event loop ---— // while (1) {

```
for (i = 0; i < 255; i++) {
    _delay_ms(DELAYTIME);
    l_brightnessA = i;
    l_brightnessB = 255 - i;
}
for (i = 254; i > 0; i--) {
    _delay_ms(DELAYTIME);
    l_brightnessA = i;
    l_brightnessB = 255 - i;
}
```

} return 0; }

```
void core::TimerCounter0::overflowServiceRoutine()
{
    Led0.on();
    Led1.on();
    Led2.on();
    Led3.on();
    myTimerCounter0.setOutputCompareRegister(core::channel::A,l_brightnessA);
    myTimerCounter0.setOutputCompareRegister(core::channel::B,l_brightnessB);
}

void core::TimerCounter0::outputCompareMatchAServiceRoutine()
{
    Led0.off();
    Led1.off();
    Led2.on();
    Led3.on();
}
void core::TimerCounter0::outputCompareMatchBServiceRoutine()
{
    Led0.on();
    Led1.on();
    Led2.off();
    Led3.off();
}
```

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Basic class for abstraction of the TimerCounter peripherals.

Usage example (Reaction Timer): #include "TimerCounter.h" #include "USART0.h" #include "PushButton.h" #include "Led.h"

void randomDelay(void); void printWord(uint16_t word);

instantiate the Counter object extern core::TimerCounter &myCounter; core::TimerCounter &myCounter = core::↩ TimerCounter::getInstanceTimerCounter0();

instantiate the USART0 object extern io::USART0 &myUSART0; io::USART0 &myUSART0 = io::USART0::getInstance();

#define BUFFER_SIZE 1 #define LED_NUMBER 0 #define PUSHBUTTON_NUMBER 2

int main(void) {

Init uint16_t timerValue;

char l_receiverBuffer[BUFFER_SIZE];

```
myCounter.selectClockSource(core::timerCounter::timerCounter1, core::clockSource::PS_1024);
```

/ instantiate a Led object component::Led Led(io::Pin(LED_NUMBER,io::PortB));

instantiate a Led object component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));

ready to send flag uint8_t l_ready2Send = 1;

if (myUSART0.ready2Send()) { myUSART0.sendString("Reaction Timer:\r\n"); }

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("----------\r\n"); }

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("Press any key to start.\r\n"); }

Mainloop while (1) {

myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);

wait for a key press while (!myUSART0.getNumberBytesReceived()) {

}

reset number of bytes after extracting the received data myUSART0.resetNumberBytesReceived();

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("\r\nGet ready..."); }

randomDelay();

wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("\r\nGo!\r\n"); }

Led.on();

myCounter.setCounter(core::timerCounter::timerCounter1,0);

if (PushButton.isPressed()) { wait to send next string while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("You're only cheating yourself.\r\n"); }

} else {

wait for a button press while (!PushButton.isPressed()) {

} get counter value myCounter.getCounter(core::timerCounter::timerCounter1, &timerValue); bit shift divide by 16 = 2^4 to convert from micro to millseconds timerValue = timerValue >> 4; print response time printWord(timerValue); }

Led.off();

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send()) { myUSART0.sendString("\r\nPress any key to try again.\r\n"); }

} return 0; }

void randomDelay(void) {

Waits for a "random" delay from 1 - 3.5 sec Requires timer 1 initialized and running It's not really random, but very hard to control –like coin-flipping.

```
uint16_t counter;
_delay_ms(1000);

myCounter.getCounter(core::timerCounter::timerCounter1, &counter);
uint8_t randomTime = static_cast<uint8_t>(counter);
```

type-casting the 16-bit TCNT1 as an 8-bit number keeps only the 8 least-significant (fastest-changing) bits

```
while (--randomTime) {
_delay_ms(10);
}
}

void printWord(uint16_t word) {

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + (word / 10000));

}

while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + ((word / 1000) % 10));
}

while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + ((word / 100) % 10));
}


while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + ((word / 10) % 10));
}



while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
    myUSART0.sendByte('0' + (word % 10));
}

}
void printWord(uint16_t word) {

char timerValue = '0' + (word / 10000);

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

timerValue = '0' + ((word / 1000) % 10);
```

```
while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

timerValue = '0' + ((word / 100) % 10);

while (!myUSART0.ready2Send()){};


if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}
timerValue = '0' + ((word / 10) % 10);

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

timerValue = '0' + (word % 10);

while (!myUSART0.ready2Send()){};

if (myUSART0.ready2Send())
{
   myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);

}

}
```

Usage example (Reaction Timer):

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file TimerCounter.h.

9.94 TimerCounter.h

Go to the documentation of this file.

```
00001
00039     component::Led Led(io::Pin(LED_NUMBER,io::PortB));
00040
00041     // instantiate a Led object
00042     component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));
00043
00044
00045
```

```
00046     // ready to send flag
00047     uint8_t l_ready2Send = 1;
00048
00049     if (myUSART0.ready2Send())
00050     {
00051     myUSART0.sendString("Reaction Timer:\r\n");
00052     }
00053
00054     // wait to send next string
00055     while (!myUSART0.ready2Send()){};
00056
00057     if (myUSART0.ready2Send())
00058     {
00059     myUSART0.sendString("---------------\r\n");
00060     }
00061
00062     // wait to send next string
00063     while (!myUSART0.ready2Send()){};
00064
00065     if (myUSART0.ready2Send())
00066     {
00067     myUSART0.sendString("Press any key to start.\r\n");
00068     }
00069
00070     // Mainloop
00071     while (1) {
00072
00073       myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
00074
00075       // wait for a key press
00076       while (!myUSART0.getNumberBytesReceived())
00077       {
00078
00079       }
00080
00081       // reset number of bytes after extracting the received data
00082       myUSART0.resetNumberBytesReceived();
00083
00084       // wait to send next string
00085       while (!myUSART0.ready2Send()){};
00086
00087       if (myUSART0.ready2Send())
00088       {
00089           myUSART0.sendString("\r\nGet ready...");
00090       }
00091
00092       randomDelay();
00093
00094       // wait to send next string
00095       while (!myUSART0.ready2Send()){};
00096
00097       if (myUSART0.ready2Send())
00098       {
00099           myUSART0.sendString("\r\nGo!\r\n");
00100       }
00101
00102       Led.on();
00103
00104       myCounter.setCounter(core::timerCounter::timerCounter1,0);
00105
00106       if (PushButton.isPressed())
00107       {
00108           // wait to send next string
00109           while (!myUSART0.ready2Send()){};
00110
00111           if (myUSART0.ready2Send())
00112           {
00113               myUSART0.sendString("You're only cheating yourself.\r\n");
00114           }
00115
00116       } else {
00117
00118           // wait for a button press
00119           while (!PushButton.isPressed())
00120           {
00121
00122           }
00123           // get counter value
00124           myCounter.getCounter(core::timerCounter::timerCounter1, &timerValue);
00125           // bit shift divide by 16 = 2^4 to convert from micro to millseconds
00126           timerValue = timerValue » 4;
00127           // print response time
00128           printWord(timerValue);
00129       }
00130
00131       Led.off();
00132
```

```
00133       while (!myUSART0.ready2Send()){};
00134
00135       if (myUSART0.ready2Send())
00136       {
00137           myUSART0.sendString("\r\nPress any key to try again.\r\n");
00138       }
00139
00140     }
00141     return 0;
00142     }
00143
00144     void randomDelay(void) {
00145
00146     // Waits for a "random" delay from 1 - 3.5 sec
00147     // Requires timer 1 initialized and running
00148     // It's not really random, but very hard to control --like coin-flipping.
00149
00150     uint16_t counter;
00151     _delay_ms(1000);
00152
00153     myCounter.getCounter(core::timerCounter::timerCounter1, &counter);
00154     uint8_t randomTime = static_cast<uint8_t>(counter);
00155     // type-casting the 16-bit TCNT1 as an 8-bit number keeps
00156     //     only the 8 least-significant (fastest-changing) bits
00157
00158     while (--randomTime) {
00159     _delay_ms(10);
00160     }
00161     }
00162
00163     void printWord(uint16_t word) {
00164
00165     while (!myUSART0.ready2Send()){};
00166
00167     if (myUSART0.ready2Send())
00168     {
00169         myUSART0.sendByte('0' + (word / 10000));
00170
00171     }
00172
00173     while (!myUSART0.ready2Send()){};
00174
00175
00176     if (myUSART0.ready2Send())
00177     {
00178         myUSART0.sendByte('0' + ((word / 1000) % 10));
00179     }
00180
00181     while (!myUSART0.ready2Send()){};
00182
00183
00184     if (myUSART0.ready2Send())
00185     {
00186         myUSART0.sendByte('0' + ((word / 100) % 10));
00187     }
00188
00189
00190     while (!myUSART0.ready2Send()){};
00191
00192     if (myUSART0.ready2Send())
00193     {
00194         myUSART0.sendByte('0' + ((word / 10) % 10));
00195     }
00196
00197
00198
00199     while (!myUSART0.ready2Send()){};
00200
00201     if (myUSART0.ready2Send())
00202     {
00203         myUSART0.sendByte('0' + (word % 10));
00204     }
00205
00206     }
00207     void printWord(uint16_t word) {
00208
00209     char timerValue = '0' + (word / 10000);
00210
00211     while (!myUSART0.ready2Send()){};
00212
00213     if (myUSART0.ready2Send())
00214     {
00215        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00216
00217     }
00218
00219     timerValue = '0' + ((word / 1000) % 10);
```

```
00220
00221     while (!myUSART0.ready2Send()){};
00222
00223
00224     if (myUSART0.ready2Send())
00225     {
00226        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00227
00228     }
00229
00230     timerValue = '0' + ((word / 100) % 10);
00231
00232     while (!myUSART0.ready2Send()){};
00233
00234
00235     if (myUSART0.ready2Send())
00236     {
00237        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00238
00239     }
00240     timerValue = '0' + ((word / 10) % 10);
00241
00242     while (!myUSART0.ready2Send()){};
00243
00244     if (myUSART0.ready2Send())
00245     {
00246        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00247
00248     }
00249
00250     timerValue = '0' + (word % 10);
00251
00252     while (!myUSART0.ready2Send()){};
00253
00254     if (myUSART0.ready2Send())
00255     {
00256        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00257
00258     }
00259
00260     }
00261  *
00262  * Usage example (PWM):
00263  *
00264  #include "Led.h"
00265  #include "USART0.h"
00266  #include "TimerCounter1.h"
00267  #include "TimerCounter2.h"
00268
00269  uint8_t getNumber(const char *ap_receive);
00270
00271  #define LED_1 1
00272  #define LED_2 2
00273  #define LED_3 3
00274  #define BUFFER_SIZE 4
00275
00276  int main(void) {
00277
00278     // Init
00279     // receiver buffer
00280     char l_receiverBuffer[BUFFER_SIZE];
00281
00282     // instantiate Led objects
00283     component::Led Led1(io::Pin(LED_1,io::PortB));
00284     component::Led Led2(io::Pin(LED_2,io::PortB));
00285     component::Led Led3(io::Pin(LED_3,io::PortB));
00286
00287     // instantiate USART0 object
00288     io::USART0 &myUSART0 = io::USART0::getInstance();
00289
00290     // instantiate Timer1 object
00291     core::TimerCounter1 &myTimerCounter1 = core::TimerCounter1::getInstance();
00292     myTimerCounter1.selectOperationMode(core::operationMode::Fast_PWM_8bit);
00293     myTimerCounter1.selectCompareOutputMode(core::channel::A,core::compareOutputMode::Clear);
00294     myTimerCounter1.selectCompareOutputMode(core::channel::B,core::compareOutputMode::Clear);
00295     // instantiate Timer2 object
00296     core::TimerCounter2 &myTimerCounter2 = core::TimerCounter2::getInstance();
00297     myTimerCounter2.selectOperationMode(core::operationMode::Fast_PWM);
00298     myTimerCounter2.selectCompareOutputMode(core::channel::A,core::compareOutputMode::Clear);
00299
00300     if (myUSART0.ready2Send())
00301     {
00302         myUSART0.sendString("-- LED PWM Demo --\r\n");
00303     }
00304     // wait to send next string
00305     while (!myUSART0.ready2Send()){}
00306
```

```
00307     myTimerCounter1.start(core::clockSource::PS_64);
00308     myTimerCounter2.start(core::clockSource::PS_64);
00309
00310
00311
00312     // ------ Event loop ------ //
00313     while (1) {
00314
00315         if (myUSART0.ready2Send())
00316         {
00317             myUSART0.sendString("\r\nEnter (0-255) for PWM duty cycle: ");
00318         }
00319
00320         myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
00321
00322
00323
00324         // wait for a key press
00325         while (myUSART0.getNumberBytesReceived()<4){}
00326
00327
00328         // reset number of bytes after extracting the received data
00329         myUSART0.resetNumberBytesReceived();
00330
00331         // send back the received character
00332         if (myUSART0.ready2Send())
00333         {
00334             myUSART0.sendFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
00335         }
00336
00337         while (!myUSART0.ready2Send()){}
00338
      myTimerCounter2.setOutputCompareRegister(core::channel::A,myTimerCounter1.getOutputCompareRegister(core::channel::B));
00339
      myTimerCounter1.setOutputCompareRegister(core::channel::B,myTimerCounter1.getOutputCompareRegister(core::channel::A));
00340         myTimerCounter1.setOutputCompareRegister(core::channel::A,getNumber(l_receiverBuffer));
00341
00342
00343     }
00344     return 0;
00345  }
00346
00347  uint8_t getNumber(const char *ap_receive) {
00348   // Gets a numerical 0-255 from the serial port.
00349   // Converts from string to number.
00350   char hundreds = '0';
00351   char tens = '0';
00352   char ones = '0';
00353   char thisChar = '0';
00354   do {
00355     hundreds = tens;
00356     tens = ones;
00357     ones = thisChar;
00358     thisChar = *ap_receive;
00359     ap_receive++;
00360
00361   } while (*ap_receive != '\r');
00362   return (100 * (hundreds - '0') + 10 * (tens - '0') + ones - '0');
00363  }
00364  *
00365  * Usage example (PWM on Any Pin):
00366  *
00367  * #include "TimerCounter0.h"
00368    #include "Led.h"
00369
00370    #define LED_0 0
00371    #define LED_1 1
00372    #define LED_2 2
00373    #define LED_3 3
00374
00375    // instantiate Timer0 object
00376    extern core::TimerCounter0 &myTimerCounter0;
00377    core::TimerCounter0 &myTimerCounter0 = core::TimerCounter0::getInstance();
00378
00379    // instantiate Led objects
00380    extern component::Led Led0;
00381    component::Led Led0(io::Pin(LED_0,io::PortB));
00382    extern component::Led Led1;
00383    component::Led Led1(io::Pin(LED_1,io::PortB));
00384    extern component::Led Led2;
00385    component::Led Led2(io::Pin(LED_2,io::PortB));
00386    extern component::Led Led3;
00387    component::Led Led3(io::Pin(LED_3,io::PortB));
00388
00389    #define DELAYTIME 3
00390
00391    volatile uint8_t l_brightnessA;
```

```
00392    volatile uint8_t l_brightnessB;
00393
00394
00395
00396    int main(void) {
00397
00398     // Init
00399     uint8_t i;
00400
00401     sei();
00402
00403     myTimerCounter0.enableOutputCompareMatchInterrupt(core::channel::A,1);
00404     myTimerCounter0.enableOutputCompareMatchInterrupt(core::channel::B,1);
00405     myTimerCounter0.enableOverflowInterrupt(1);
00406     myTimerCounter0.start(core::clockSource::PS_1024);
00407
00408
00409
00410
00411     // ------ Event loop ------ //
00412     while (1) {
00413
00414         for (i = 0; i < 255; i++) {
00415             _delay_ms(DELAYTIME);
00416             l_brightnessA = i;
00417             l_brightnessB = 255 - i;
00418         }
00419         for (i = 254; i > 0; i--) {
00420             _delay_ms(DELAYTIME);
00421             l_brightnessA = i;
00422             l_brightnessB = 255 - i;
00423         }
00424
00425
00426
00427     }
00428     return 0;
00429     }
00430
00431
00432     void core::TimerCounter0::overflowServiceRoutine()
00433     {
00434         Led0.on();
00435         Led1.on();
00436         Led2.on();
00437         Led3.on();
00438         myTimerCounter0.setOutputCompareRegister(core::channel::A,l_brightnessA);
00439         myTimerCounter0.setOutputCompareRegister(core::channel::B,l_brightnessB);
00440     }
00441
00442     void core::TimerCounter0::outputCompareMatchAServiceRoutine()
00443     {
00444         Led0.off();
00445         Led1.off();
00446         Led2.on();
00447         Led3.on();
00448     }
00449     void core::TimerCounter0::outputCompareMatchBServiceRoutine()
00450     {
00451         Led0.on();
00452         Led1.on();
00453         Led2.off();
00454         Led3.off();
00455     }
00456  *
00457  * @author Farid Oubbati (https://github.com/faroub)
00458  * @date March 2020
00459 */
00460
00461 #ifndef TIMER_COUNTER_H
00462 #define TIMER_COUNTER_H
00463 #include "ha_base.h"
00464
00465
00466
00467 namespace core
00468 {
00469
00470 enum class channel : uint8_t {
00471     A=0,
00472     B,
00473 };
00474
00475
00476 enum class compareOutputMode : uint8_t {
00477     normal=0,
00478     toggle,
```

```
00479     clear,
00480     set,
00481 };
00482
00483
00484
00485 enum class operationMode : uint8_t {
00486     normal=0,
00487     PWM_PC,
00488     PWM_PC_8bit,
00489     PWM_PC_9bit,
00490     PWM_PC_10bit,
00491     PWM_PFC_ICR,
00492     PWM_PFC_OCR,
00493     PWM_PC_ICR,
00494     PWM_PC_OCR,
00495     fast_PWM,
00496     fast_PWM_8bit,
00497     fast_PWM_9bit,
00498     fast_PWM_10bit,
00499     fast_PWM_ICR,
00500     fast_PWM_OCR,
00501     CTC_OCR,
00502     CTC_ICR,
00503 };
00504 enum class clockSource : uint16_t {
00505      noClock=0,
00506      PS_1,
00507      PS_8,
00508      PS_32,
00509      PS_64,
00510      PS_128,
00511      PS_256,
00512      PS_1024,
00513      extern_Clock_T0_Falling_Edge,
00514      extern_Clock_T0_Rising_Edge,
00515 };
00516
00517 class TimerCounter
00518 {
00519
00520 public:
00521
00526     virtual ~TimerCounter() {}
00527
00535     static void operator delete(void *) {}
00536
00545     virtual void selectOperationMode(const operationMode &ar_operationMode) = 0;
00546
00555     virtual void selectClockSource(const clockSource &ar_clockSource) = 0;
00556
00557     virtual void start() = 0;
00558
00559     virtual void stop() = 0;
00560
00561     virtual void selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
    &ar_compareOutputMode) = 0;
00562
00563     virtual void setCounter(const uint16_t &ar_dataBuffer) = 0;
00564
00565     virtual uint16_t getCounter() const = 0;
00566
00567     virtual void setOutputCompareRegister(const channel &ar_channel, const uint16_t &ar_dataBuffer) =
    0;
00568
00569     virtual uint16_t getOutputCompareRegister(const channel &ar_channel) const = 0;
00570
00571     virtual void enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t a_enable)
    = 0;
00572
00573     virtual void enableOverflowInterrupt(const uint8_t a_enable) = 0;
00574
00575     virtual uint16_t getClockPrescaler() = 0;
00576
00577 protected:
00578
00579 private:
00580
00581
00582
00583
00584 };
00585
00586 }
00587
00588
00589 #endif
```

9.95 TimerCounter0.cpp File Reference

```
#include "TimerCounter0.h"
#include "MCU.h"
```

9.96 TimerCounter0.cpp

Go to the documentation of this file.

```
00001 #include "TimerCounter0.h"
00002 #include "MCU.h"
00003
00004
00005 core::TimerCounter0& core::TimerCounter0::getInstance(const channel &ar_channel,
00006                                                       const operationMode &ar_operationMode,
00007                                                       const clockSource &ar_clockSource,
00008                                                       const compareOutputMode &ar_compareOutputMode)
00009 {
00010     static TimerCounter0 l_instance(ar_channel,
00011                                     ar_operationMode,
00012                                     ar_clockSource,
00013                                     ar_compareOutputMode);
00014
00015     return l_instance;
00016
00017
00018 }
00019
00020 core::TimerCounter0::TimerCounter0(const channel &ar_channel,
00021                                    const operationMode &ar_operationMode,
00022                                    const clockSource &ar_clockSource,
00023                                    const compareOutputMode& ar_compareOutputMode)
00024 {
00025     core::MCU::enableTimerCounter0(1);
00026     stop();
00027     selectOperationMode(ar_operationMode);
00028     selectClockSource(ar_clockSource);
00029     selectCompareOutputMode(ar_channel,ar_compareOutputMode);
00030
00031
00032
00033 }
00034 core::TimerCounter0::~TimerCounter0()
00035 {
00036
00037 }
00038
00039 void core::TimerCounter0::selectClockSource(const clockSource &ar_clockSource)
00040 {
00041     switch (ar_clockSource)
00042     {
00043         case core::clockSource::noClock:
00044         {
00045             m_clockPrescaler=0;
00046             m_clockSource=0;
00047             break;
00048         }
00049         case core::clockSource::PS_1:
00050         {
00051             m_clockPrescaler=1;
00052             m_clockSource=1;
00053             break;
00054         }
00055         case core::clockSource::PS_8:
00056         {
00057             m_clockPrescaler=8;
00058             m_clockSource=2;
00059             break;
00060         }
00061         case core::clockSource::PS_64:
00062         {
00063             m_clockPrescaler=64;
00064             m_clockSource=3;
00065             break;
00066         }
00067         case core::clockSource::PS_256:
00068         {
```

```
00069             m_clockPrescaler=256;
00070             m_clockSource=4;
00071             break;
00072         }
00073         case core::clockSource::PS_1024:
00074         {
00075             m_clockPrescaler=1024;
00076             m_clockSource=5;
00077             break;
00078         }
00079         case core::clockSource::extern_Clock_T0_Falling_Edge:
00080         {
00081             m_clockPrescaler=0;
00082             m_clockSource=6;
00083             break;
00084         }
00085         case core::clockSource::extern_Clock_T0_Rising_Edge:
00086         {
00087             m_clockPrescaler=0;
00088             m_clockSource=7;
00089             break;
00090         }
00091         default:
00092         {
00093             // unsupported on this timer (e.g. PS_32, PS_128): stop the
00094             // clock instead of keeping the previously selected source
00095             m_clockPrescaler=0;
00096             m_clockSource=0;
00097             break;
00098         }
00099     }
00100
00101
00102 }
00103
00104 void core::TimerCounter0::selectOperationMode(const operationMode &ar_operationMode)
00105 {
00106     switch (ar_operationMode)
00107     {
00108         case core::operationMode::normal:
00109         {
00110             TIMER0_SELECT_OPERATION_MODE(0);
00111             break;
00112         }
00113         case core::operationMode::PWM_PC:
00114         {
00115             TIMER0_SELECT_OPERATION_MODE(1);
00116             break;
00117         }
00118         case core::operationMode::CTC_OCR:
00119         {
00120             TIMER0_SELECT_OPERATION_MODE(2);
00121             break;
00122         }
00123         case core::operationMode::fast_PWM:
00124         {
00125             TIMER0_SELECT_OPERATION_MODE(3);
00126             break;
00127         }
00128         case core::operationMode::PWM_PC_OCR:
00129         {
00130             TIMER0_SELECT_OPERATION_MODE(5);
00131             break;
00132         }
00133         case core::operationMode::fast_PWM_OCR:
00134         {
00135             TIMER0_SELECT_OPERATION_MODE(7);
00136             break;
00137         }
00138         default:
00139         {
00140             // unsupported on this timer: fall back to normal mode instead
00141             // of keeping the previously programmed mode
00142             TIMER0_SELECT_OPERATION_MODE(0);
00143             break;
00144         }
00145
00146     }
00147
00148 }
00149
00150
00151 void core::TimerCounter0::start()
00152 {
00153     TIMER0_SELECT_CLOCK_SOURCE(m_clockSource);
00154 }
00155
```

```
00156 uint16_t core::TimerCounter0::getClockPrescaler()
00157 {
00158     return m_clockPrescaler;
00159 }
00160
00161 void core::TimerCounter0::stop()
00162 {
00163     TIMER0_STOP;
00164 }
00165
00166
00167 void core::TimerCounter0::selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
      &ar_compareOutputMode)
00168 {
00169     switch (ar_channel)
00170     {
00171         case core::channel::A:
00172         {
00173             TIMER0_SELECT_COM_CHANNEL_A(static_cast<uint8_t>(ar_compareOutputMode));
00174             break;
00175         }
00176         case core::channel::B:
00177         {
00178             TIMER0_SELECT_COM_CHANNEL_B(static_cast<uint8_t>(ar_compareOutputMode));
00179             break;
00180         }
00181     }
00182 }
00183
00184
00185
00186 void core::TimerCounter0::setCounter(const uint16_t &ar_dataBuffer)
00187 {
00188     TCNT0 = static_cast<uint8_t>(ar_dataBuffer);
00189 }
00190
00191 uint16_t core::TimerCounter0::getCounter() const
00192 {
00193     return TCNT0;
00194 }
00195
00196 void core::TimerCounter0::setOutputCompareRegister(const channel &ar_channel, const uint16_t
      &ar_dataBuffer)
00197 {
00198     switch (ar_channel)
00199     {
00200         case core::channel::A:
00201         {
00202             OCR0A = static_cast<uint8_t>(ar_dataBuffer);
00203             break;
00204         }
00205         case core::channel::B:
00206         {
00207             OCR0B = static_cast<uint8_t>(ar_dataBuffer);
00208             break;
00209         }
00210
00211     }
00212 }
00213
00214
00215
00216 uint16_t core::TimerCounter0::getOutputCompareRegister(const channel &ar_channel) const
00217 {
00218     uint16_t compareRegister = 0;
00219     switch (ar_channel)
00220     {
00221         case core::channel::A:
00222         {
00223             compareRegister = OCR0A;
00224             break;
00225         }
00226         case core::channel::B:
00227         {
00228             compareRegister = OCR0B;
00229             break;
00230         }
00231     }
00232     return compareRegister;
00233 }
00234
00235 void core::TimerCounter0::enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t
      a_enable)
00236 {
00237     switch (ar_channel)
00238     {
00239         case core::channel::A:
```

```
00240         {
00241             if (a_enable) {
00242 
00243                 TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT;
00244 
00245             } else {
00246 
00247                 TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT;
00248             }
00249             break;
00250         }
00251         case core::channel::B:
00252         {
00253             if (a_enable) {
00254 
00255                 TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT;
00256 
00257             } else {
00258 
00259                 TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT;
00260 
00261             }
00262             break;
00263         }
00264     }
00265 
00266 }
00267 
00268 
00269 
00270 void core::TimerCounter0::enableOverflowInterrupt(const uint8_t a_enable)
00271 {
00272 
00273     if (a_enable) {
00274 
00275         TIMER0_ENABLE_OVERFLOW_INTERRUPT;
00276 
00277     } else {
00278 
00279         TIMER0_DISABLE_OVERFLOW_INTERRUPT;
00280     }
00281 
00282 }
00283 
00284 // weak default interrupt handlers: empty bodies the application may
00285 // override by defining the same vector symbol (e.g. ISR(TIMER0_COMPA_vect))
00286 
00287 void core::TimerCounter0::outputCompareMatchAServiceRoutine() {}
00288 
00289 void core::TimerCounter0::outputCompareMatchBServiceRoutine() {}
00290 
00291 void core::TimerCounter0::overflowServiceRoutine() {}
```

9.97 TimerCounter0.h File Reference

```
#include "TimerCounter.h"
#include "ha_base.h"
```

Classes

- class core::TimerCounter0

Namespaces

- namespace core

  AVR chip internal core components.

9.98 TimerCounter0.h

Go to the documentation of this file.

```
00001
00039     component::Led Led(io::Pin(LED_NUMBER,io::PortB));
00040
00041     // instantiate a Led object
00042     component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));
00043
00044
00045
00046     // ready to send flag
00047     uint8_t l_ready2Send = 1;
00048
00049     if (myUSART0.ready2Send())
00050     {
00051     myUSART0.sendString("Reaction Timer:\r\n");
00052     }
00053
00054     // wait to send next string
00055     while (!myUSART0.ready2Send()){};
00056
00057     if (myUSART0.ready2Send())
00058     {
00059     myUSART0.sendString("---------------\r\n");
00060     }
00061
00062     // wait to send next string
00063     while (!myUSART0.ready2Send()){};
00064
00065     if (myUSART0.ready2Send())
00066     {
00067     myUSART0.sendString("Press any key to start.\r\n");
00068     }
00069
00070     // Mainloop
00071     while (1) {
00072
00073       myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
00074
00075       // wait for a key press
00076       while (!myUSART0.getNumberBytesReceived())
00077       {
00078
00079       }
00080
00081       // reset number of bytes after extracting the received data
00082       myUSART0.resetNumberBytesReceived();
00083
00084       // wait to send next string
00085       while (!myUSART0.ready2Send()){};
00086
00087       if (myUSART0.ready2Send())
00088       {
00089           myUSART0.sendString("\r\nGet ready...");
00090       }
00091
00092       randomDelay();
00093
00094       // wait to send next string
00095       while (!myUSART0.ready2Send()){};
00096
00097       if (myUSART0.ready2Send())
00098       {
00099           myUSART0.sendString("\r\nGo!\r\n");
00100       }
00101
00102       Led.on();
00103
00104       myCounter.setCounter(core::timerCounter::timerCounter1,0);
00105
00106       if (PushButton.isPressed())
00107       {
00108           // wait to send next string
00109           while (!myUSART0.ready2Send()){};
00110
00111           if (myUSART0.ready2Send())
00112           {
00113               myUSART0.sendString("You're only cheating yourself.\r\n");
00114           }
00115
00116       } else {
00117
00118           // wait for a button press
00119           while (!PushButton.isPressed())
```

```
00120           {
00121
00122           }
00123           // get counter value
00124           myCounter.getCounter(core::timerCounter::timerCounter1, &timerValue);
00125           // bit shift divide by 16 = 2^4 to convert from micro to millseconds
00126           timerValue = timerValue » 4;
00127           // print response time
00128           printWord(timerValue);
00129       }
00130
00131       Led.off();
00132
00133       while (!myUSART0.ready2Send()){};
00134
00135       if (myUSART0.ready2Send())
00136       {
00137           myUSART0.sendString("\r\nPress any key to try again.\r\n");
00138       }
00139
00140     }
00141     return 0;
00142     }
00143
00144     void randomDelay(void) {
00145
00146     // Waits for a "random" delay from 1 - 3.5 sec
00147     // Requires timer 1 initialized and running
00148     // It's not really random, but very hard to control --like coin-flipping.
00149
00150     uint16_t counter;
00151     _delay_ms(1000);
00152
00153     myCounter.getCounter(core::timerCounter::timerCounter1, &counter);
00154     uint8_t randomTime = static_cast<uint8_t>(counter);
00155     // type-casting the 16-bit TCNT1 as an 8-bit number keeps
00156     //     only the 8 least-significant (fastest-changing) bits
00157
00158     while (--randomTime) {
00159     _delay_ms(10);
00160     }
00161     }
00162
00163     void printWord(uint16_t word) {
00164
00165     while (!myUSART0.ready2Send()){};
00166
00167     if (myUSART0.ready2Send())
00168     {
00169         myUSART0.sendByte('0' + (word / 10000));
00170
00171     }
00172
00173     while (!myUSART0.ready2Send()){};
00174
00175
00176     if (myUSART0.ready2Send())
00177     {
00178         myUSART0.sendByte('0' + ((word / 1000) % 10));
00179     }
00180
00181     while (!myUSART0.ready2Send()){};
00182
00183
00184     if (myUSART0.ready2Send())
00185     {
00186         myUSART0.sendByte('0' + ((word / 100) % 10));
00187     }
00188
00189
00190     while (!myUSART0.ready2Send()){};
00191
00192     if (myUSART0.ready2Send())
00193     {
00194         myUSART0.sendByte('0' + ((word / 10) % 10));
00195     }
00196
00197
00198
00199     while (!myUSART0.ready2Send()){};
00200
00201     if (myUSART0.ready2Send())
00202     {
00203         myUSART0.sendByte('0' + (word % 10));
00204     }
00205
00206     }
```

```
00207     void printWord(uint16_t word) {
00208
00209     char timerValue = '0' + (word / 10000);
00210
00211     while (!myUSART0.ready2Send()){};
00212
00213     if (myUSART0.ready2Send())
00214     {
00215        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00216
00217     }
00218
00219     timerValue = '0' + ((word / 1000) % 10);
00220
00221     while (!myUSART0.ready2Send()){};
00222
00223
00224     if (myUSART0.ready2Send())
00225     {
00226        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00227
00228     }
00229
00230     timerValue = '0' + ((word / 100) % 10);
00231
00232     while (!myUSART0.ready2Send()){};
00233
00234
00235     if (myUSART0.ready2Send())
00236     {
00237        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00238
00239     }
00240     timerValue = '0' + ((word / 10) % 10);
00241
00242     while (!myUSART0.ready2Send()){};
00243
00244     if (myUSART0.ready2Send())
00245     {
00246        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00247
00248     }
00249
00250     timerValue = '0' + (word % 10);
00251
00252     while (!myUSART0.ready2Send()){};
00253
00254     if (myUSART0.ready2Send())
00255     {
00256        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00257
00258     }
00259
00260     }
00261  *
00262  * Usage example (Reaction Timer):
00263  *
00264  * @author Farid Oubbati (https://github.com/faroub)
00265  * @date March 2020
00266 */
00267
00268 #ifndef TIMER_COUNTER0_H
00269 #define TIMER_COUNTER0_H
00270 #include "TimerCounter.h"
00271 #include "ha_base.h"
00272
00273
00274
00275 namespace core
00276 {
00277
00278
00279 class TimerCounter0 : public TimerCounter
00280 {
00281
00282 public:
00283
00284     static TimerCounter0& getInstance(const channel &ar_channel = channel::A,
00285                                       const operationMode &ar_operationMode = operationMode::normal,
00286                                       const clockSource &ar_clockSource= clockSource::noClock,
00287                                       const compareOutputMode& ar_compareOutputMode =
    compareOutputMode::normal);
00288
00289     void selectOperationMode(const operationMode &ar_operationMode) override;
00290
00291     void start() override;
00292
```

```
00293     void stop() override;
00294
00295     void selectClockSource(const clockSource &ar_clockSource) override;
00296
00297     void selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
      &ar_compareOutputMode) override;
00298
00299     void setCounter(const uint16_t &ar_dataBuffer) override;
00300
00301     uint16_t getCounter() const override;
00302
00303     void setOutputCompareRegister(const channel &ar_channel, const uint16_t &ar_dataBuffer) override;
00304
00305     uint16_t getOutputCompareRegister(const channel &ar_channel) const override;
00306
00307     void enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t a_enable)
      override;
00308
00309     void enableOverflowInterrupt(const uint8_t a_enable) override;
00310
00311     uint16_t getClockPrescaler() override;
00312
00313     static void outputCompareMatchAServiceRoutine() __asm__(STR(TIMER0_COM_CHANNEL_A_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00314
00315     static void outputCompareMatchBServiceRoutine() __asm__(STR(TIMER0_COM_CHANNEL_B_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__, weak));
00316
00317     static void overflowServiceRoutine() __asm__(STR(TIMER0_OVERFLOW_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00318
00319
00320
00321 protected:
00322
00323 private:
00324
00325     TimerCounter0(const channel &ar_channel,
00326                   const operationMode &ar_operationMode,
00327                   const clockSource &ar_clockSource,
00328                   const compareOutputMode& ar_compareOutputMode);
00329
00330     ~TimerCounter0();
00331
00334     TimerCounter0(const TimerCounter0&);
00335
00338     const TimerCounter0& operator=(const TimerCounter0&);
00339
00340     uint16_t m_clockPrescaler;
00341
00342     uint8_t m_clockSource;
00343
00344
00345
00346 };
00347
00348 }
00349
00350
00351 #endif
```

9.99 TimerCounter1.cpp File Reference

```
#include "TimerCounter1.h"
#include <util/atomic.h>
#include "MCU.h"
```

9.100 TimerCounter1.cpp

Go to the documentation of this file.

```
00001 #include "TimerCounter1.h"
00002 #include <util/atomic.h>
```

```
00003 #include "MCU.h"
00004
00005
00006 core::TimerCounter1& core::TimerCounter1::getInstance(const channel &ar_channel,
00007                                                        const operationMode &ar_operationMode,
00008                                                        const clockSource &ar_clockSource,
00009                                                        const compareOutputMode& ar_compareOutputMode)
00010 {
00011     static TimerCounter1 l_instance(ar_channel,
00012                                     ar_operationMode,
00013                                     ar_clockSource,
00014                                     ar_compareOutputMode);
00015
00016     return l_instance;
00017
00018
00019 }
00020
00021 core::TimerCounter1::TimerCounter1(const channel &ar_channel,
00022                                    const operationMode &ar_operationMode,
00023                                    const clockSource &ar_clockSource,
00024                                    const compareOutputMode& ar_compareOutputMode)
00025 {
00026     core::MCU::enableTimerCounter1(1);
00027     stop();
00028     selectOperationMode(ar_operationMode);
00029     selectClockSource(ar_clockSource);
00030     selectCompareOutputMode(ar_channel,ar_compareOutputMode);
00031
00032
00033 }
00034 core::TimerCounter1::~TimerCounter1()
00035 {
00036
00037 }
00038
00039 void core::TimerCounter1::selectClockSource(const clockSource &ar_clockSource)
00040 {
00041     switch (ar_clockSource)
00042     {
00043         case core::clockSource::noClock:
00044         {
00045             m_clockPrescaler=0;
00046             m_clockSource=0;
00047             break;
00048         }
00049         case core::clockSource::PS_1:
00050         {
00051             m_clockPrescaler=1;
00052             m_clockSource=1;
00053             break;
00054         }
00055         case core::clockSource::PS_8:
00056         {
00057             m_clockPrescaler=8;
00058             m_clockSource=2;
00059             break;
00060         }
00061         case core::clockSource::PS_64:
00062         {
00063             m_clockPrescaler=64;
00064             m_clockSource=3;
00065             break;
00066         }
00067         case core::clockSource::PS_256:
00068         {
00069             m_clockPrescaler=256;
00070             m_clockSource=4;
00071             break;
00072         }
00073         case core::clockSource::PS_1024:
00074         {
00075             m_clockPrescaler=1024;
00076             m_clockSource=5;
00077             break;
00078         }
00079         case core::clockSource::extern_Clock_T0_Falling_Edge:
00080         {
00081             m_clockPrescaler=0;
00082             m_clockSource=6;
00083             break;
00084         }
00085         case core::clockSource::extern_Clock_T0_Rising_Edge:
00086         {
00087             m_clockPrescaler=0;
00088             m_clockSource=7;
00089             break;
```

```
00090         }
00091         default:
00092         {
00093             // unsupported on this timer (e.g. PS_32, PS_128): stop the
00094             // clock instead of keeping the previously selected source
00095             m_clockPrescaler=0;
00096             m_clockSource=0;
00097             break;
00098         }
00099     }
00100
00101
00102 }
00103
00104 void core::TimerCounter1::selectOperationMode(const operationMode &ar_operationMode)
00105 {
00106     switch (ar_operationMode)
00107     {
00108         case core::operationMode::normal:
00109         {
00110             TIMER1_SELECT_OPERATION_MODE(0);
00111             break;
00112         }
00113         case core::operationMode::PWM_PC_8bit:
00114         {
00115             TIMER1_SELECT_OPERATION_MODE(1);
00116             break;
00117         }
00118         case core::operationMode::PWM_PC_9bit:
00119         {
00120             TIMER1_SELECT_OPERATION_MODE(2);
00121             break;
00122         }
00123         case core::operationMode::PWM_PC_10bit:
00124         {
00125             TIMER1_SELECT_OPERATION_MODE(3);
00126             break;
00127         }
00128         case core::operationMode::CTC_OCR:
00129         {
00130             TIMER1_SELECT_OPERATION_MODE(4);
00131             break;
00132         }
00133         case core::operationMode::fast_PWM_8bit:
00134         {
00135             TIMER1_SELECT_OPERATION_MODE(5);
00136             break;
00137         }
00138         case core::operationMode::fast_PWM_9bit:
00139         {
00140             TIMER1_SELECT_OPERATION_MODE(6);
00141             break;
00142         }
00143         case core::operationMode::fast_PWM_10bit:
00144         {
00145             TIMER1_SELECT_OPERATION_MODE(7);
00146             break;
00147         }
00148         case core::operationMode::PWM_PFC_ICR:
00149         {
00150             TIMER1_SELECT_OPERATION_MODE(8);
00151             break;
00152         }
00153         case core::operationMode::PWM_PFC_OCR:
00154         {
00155             TIMER1_SELECT_OPERATION_MODE(9);
00156             break;
00157         }
00158         case core::operationMode::PWM_PC_ICR:
00159         {
00160             TIMER1_SELECT_OPERATION_MODE(10);
00161             break;
00162         }
00163         case core::operationMode::PWM_PC_OCR:
00164         {
00165             TIMER1_SELECT_OPERATION_MODE(11);
00166             break;
00167         }
00168         case core::operationMode::CTC_ICR:
00169         {
00170             TIMER1_SELECT_OPERATION_MODE(12);
00171             break;
00172         }
00173         case core::operationMode::fast_PWM_ICR:
00174         {
00175             TIMER1_SELECT_OPERATION_MODE(14);
00176             break;
```

```
00177         }
00178         case core::operationMode::fast_PWM_OCR:
00179         {
00180             TIMER1_SELECT_OPERATION_MODE(15);
00181             break;
00182         }
00183         default:
00184         {
00185             // unsupported on this timer: fall back to normal mode instead
00186             // of keeping the previously programmed mode
00187             TIMER1_SELECT_OPERATION_MODE(0);
00188             break;
00189         }
00190 
00191     }
00192 
00193 
00194 }
00195 
00196 
00197 void core::TimerCounter1::start()
00198 {
00199     TIMER1_SELECT_CLOCK_SOURCE(m_clockSource);
00200 }
00201 
00202 uint16_t core::TimerCounter1::getClockPrescaler()
00203 {
00204     return m_clockPrescaler;
00205 }
00206 
00207 void core::TimerCounter1::stop()
00208 {
00209     TIMER1_STOP;
00210 }
00211 
00212 
00213 void core::TimerCounter1::selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
    &ar_compareOutputMode)
00214 {
00215     switch (ar_channel)
00216     {
00217         case core::channel::A:
00218         {
00219             TIMER1_SELECT_COM_CHANNEL_A(static_cast<uint8_t>(ar_compareOutputMode));
00220             break;
00221         }
00222         case core::channel::B:
00223         {
00224             TIMER1_SELECT_COM_CHANNEL_B(static_cast<uint8_t>(ar_compareOutputMode));
00225             break;
00226         }
00227     }
00228 
00229 }
00230 
00231 
00232 void core::TimerCounter1::setCounter(const uint16_t &ar_dataBuffer)
00233 {
00234     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00235     {
00236         TCNT1 = ar_dataBuffer;
00237     }
00238 }
00239 
00240 uint16_t core::TimerCounter1::getCounter() const
00241 {
00242     uint16_t counter;
00243     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00244     {
00245         counter = TCNT1;
00246     }
00247     return counter;
00248 }
00249 
00250 void core::TimerCounter1::setOutputCompareRegister(const channel &ar_channel, const uint16_t
    &ar_dataBuffer)
00251 {
00252     switch (ar_channel)
00253     {
00254         case core::channel::A:
00255         {
00256             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00257             {
00258                 OCR1A = ar_dataBuffer;
00259             }
00260             break;
00261         }
```

```
00262         case core::channel::B:
00263         {
00264             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00265             {
00266                 OCR1B = ar_dataBuffer;
00267             }
00268             break;
00269         }
00270     }
00271 }
00272
00273
00274
00275 uint16_t core::TimerCounter1::getOutputCompareRegister(const channel &ar_channel) const
00276 {
00277     uint16_t compareRegister = 0;
00278     switch (ar_channel)
00279     {
00280         case core::channel::A:
00281         {
00282             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00283             {
00284                 compareRegister = OCR1A;
00285             }
00286             break;
00287         }
00288         case core::channel::B:
00289         {
00290             ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00291             {
00292                 compareRegister = OCR1B;
00293             }
00294             break;
00295         }
00296     }
00297     return compareRegister;
00298 }
00299
00300 void core::TimerCounter1::enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t
      a_enable)
00301 {
00302     switch (ar_channel)
00303     {
00304         case core::channel::A:
00305         {
00306             if (a_enable) {
00307
00308                 TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT;
00309
00310             } else {
00311
00312                 TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT;
00313             }
00314             break;
00315         }
00316         case core::channel::B:
00317         {
00318             if (a_enable) {
00319
00320                 TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT;
00321
00322             } else {
00323
00324                 TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT;
00325
00326             }
00327             break;
00328         }
00329     }
00330
00331 }
00332
00333
00334
00335 void core::TimerCounter1::enableOverflowInterrupt(const uint8_t a_enable)
00336 {
00337
00338     if (a_enable) {
00339
00340         TIMER1_ENABLE_OVERFLOW_INTERRUPT;
00341
00342     } else {
00343
00344         TIMER1_DISABLE_OVERFLOW_INTERRUPT;
00345     }
00346
00347 }
```

```
00348
00349 void core::TimerCounter1::enableInputCaptureInterrupt(const uint8_t a_enable)
00350 {
00351     if (a_enable) {
00352
00353         TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT;
00354
00355     } else {
00356
00357         TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT;
00358     }
00359 }
00360
00361 void core::TimerCounter1::setInputCaptureRegister(const uint16_t &ar_dataBuffer)
00362 {
00363     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00364     {
00365         ICR1 = ar_dataBuffer;
00366     }
00367 }
00368
00369
00370 uint16_t core::TimerCounter1::getInputCaptureRegister() const
00371 {
00372     uint16_t captureRegister;
00373     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00374     {
00375         captureRegister = ICR1;
00376     }
00377     return captureRegister;
00378 }
00379
00380 // weak default interrupt handlers: empty bodies the application may
00381 // override by defining the same vector symbol (e.g. ISR(TIMER0_COMPA_vect))
00382
00383 void core::TimerCounter1::outputCompareMatchAServiceRoutine() {}
00384
00385 void core::TimerCounter1::outputCompareMatchBServiceRoutine() {}
00386
00387 void core::TimerCounter1::overflowServiceRoutine() {}
00388
00389 void core::TimerCounter1::inputCaptureServiceRoutine() {}
```

9.101 TimerCounter1.h File Reference

```
#include "TimerCounter.h"
#include "ha_base.h"
```

Classes

- class core::TimerCounter1

Namespaces

- namespace core

  AVR chip internal core components.

9.102 TimerCounter1.h

Go to the documentation of this file.

```
00001
00039     component::Led Led(io::Pin(LED_NUMBER,io::PortB));
00040
00041     // instantiate a Led object
```

```
00042     component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));
00043
00044
00045
00046     // ready to send flag
00047     uint8_t l_ready2Send = 1;
00048
00049     if (myUSART0.ready2Send())
00050     {
00051     myUSART0.sendString("Reaction Timer:\r\n");
00052     }
00053
00054     // wait to send next string
00055     while (!myUSART0.ready2Send()){};
00056
00057     if (myUSART0.ready2Send())
00058     {
00059     myUSART0.sendString("---------------\r\n");
00060     }
00061
00062     // wait to send next string
00063     while (!myUSART0.ready2Send()){};
00064
00065     if (myUSART0.ready2Send())
00066     {
00067     myUSART0.sendString("Press any key to start.\r\n");
00068     }
00069
00070     // Mainloop
00071     while (1) {
00072
00073       myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
00074
00075       // wait for a key press
00076       while (!myUSART0.getNumberBytesReceived())
00077       {
00078
00079       }
00080
00081       // reset number of bytes after extracting the received data
00082       myUSART0.resetNumberBytesReceived();
00083
00084       // wait to send next string
00085       while (!myUSART0.ready2Send()){};
00086
00087       if (myUSART0.ready2Send())
00088       {
00089           myUSART0.sendString("\r\nGet ready...");
00090       }
00091
00092       randomDelay();
00093
00094       // wait to send next string
00095       while (!myUSART0.ready2Send()){};
00096
00097       if (myUSART0.ready2Send())
00098       {
00099           myUSART0.sendString("\r\nGo!\r\n");
00100       }
00101
00102       Led.on();
00103
00104       myCounter.setCounter(core::timerCounter::timerCounter1,0);
00105
00106       if (PushButton.isPressed())
00107       {
00108           // wait to send next string
00109           while (!myUSART0.ready2Send()){};
00110
00111           if (myUSART0.ready2Send())
00112           {
00113               myUSART0.sendString("You're only cheating yourself.\r\n");
00114           }
00115
00116       } else {
00117
00118           // wait for a button press
00119           while (!PushButton.isPressed())
00120           {
00121
00122           }
00123           // get counter value
00124           myCounter.getCounter(core::timerCounter::timerCounter1, &timerValue);
00125           // bit shift divide by 16 = 2^4 to convert from micro to millseconds
00126           timerValue = timerValue » 4;
00127           // print response time
00128           printWord(timerValue);
```

```
00129       }
00130
00131       Led.off();
00132
00133       while (!myUSART0.ready2Send()){};
00134
00135       if (myUSART0.ready2Send())
00136       {
00137           myUSART0.sendString("\r\nPress any key to try again.\r\n");
00138       }
00139
00140     }
00141     return 0;
00142     }
00143
00144     void randomDelay(void) {
00145
00146     // Waits for a "random" delay from 1 - 3.5 sec
00147     // Requires timer 1 initialized and running
00148     // It's not really random, but very hard to control --like coin-flipping.
00149
00150     uint16_t counter;
00151     _delay_ms(1000);
00152
00153     myCounter.getCounter(core::timerCounter::timerCounter1, &counter);
00154     uint8_t randomTime = static_cast<uint8_t>(counter);
00155     // type-casting the 16-bit TCNT1 as an 8-bit number keeps
00156     //     only the 8 least-significant (fastest-changing) bits
00157
00158     while (--randomTime) {
00159     _delay_ms(10);
00160     }
00161     }
00162
00163     void printWord(uint16_t word) {
00164
00165     while (!myUSART0.ready2Send()){};
00166
00167     if (myUSART0.ready2Send())
00168     {
00169         myUSART0.sendByte('0' + (word / 10000));
00170
00171     }
00172
00173     while (!myUSART0.ready2Send()){};
00174
00175
00176     if (myUSART0.ready2Send())
00177     {
00178         myUSART0.sendByte('0' + ((word / 1000) % 10));
00179     }
00180
00181     while (!myUSART0.ready2Send()){};
00182
00183
00184     if (myUSART0.ready2Send())
00185     {
00186         myUSART0.sendByte('0' + ((word / 100) % 10));
00187     }
00188
00189
00190     while (!myUSART0.ready2Send()){};
00191
00192     if (myUSART0.ready2Send())
00193     {
00194         myUSART0.sendByte('0' + ((word / 10) % 10));
00195     }
00196
00197
00198
00199     while (!myUSART0.ready2Send()){};
00200
00201     if (myUSART0.ready2Send())
00202     {
00203         myUSART0.sendByte('0' + (word % 10));
00204     }
00205
00206     }
00207     void printWord(uint16_t word) {
00208
00209     char timerValue = '0' + (word / 10000);
00210
00211     while (!myUSART0.ready2Send()){};
00212
00213     if (myUSART0.ready2Send())
00214     {
00215        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
```

```
00216
00217     }
00218
00219     timerValue = '0' + ((word / 1000) % 10);
00220
00221     while (!myUSART0.ready2Send()){};
00222
00223
00224     if (myUSART0.ready2Send())
00225     {
00226        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00227
00228     }
00229
00230     timerValue = '0' + ((word / 100) % 10);
00231
00232     while (!myUSART0.ready2Send()){};
00233
00234
00235     if (myUSART0.ready2Send())
00236     {
00237        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00238
00239     }
00240     timerValue = '0' + ((word / 10) % 10);
00241
00242     while (!myUSART0.ready2Send()){};
00243
00244     if (myUSART0.ready2Send())
00245     {
00246        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00247
00248     }
00249
00250     timerValue = '0' + (word % 10);
00251
00252     while (!myUSART0.ready2Send()){};
00253
00254     if (myUSART0.ready2Send())
00255     {
00256        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00257
00258     }
00259
00260     }
00261  *
00262  * Usage example (Reaction Timer):
00263  *
00264  * @author Farid Oubbati (https://github.com/faroub)
00265  * @date March 2020
00266 */
00267
00268 #ifndef TIMER_COUNTER1_H
00269 #define TIMER_COUNTER1_H
00270 #include "TimerCounter.h"
00271 #include "ha_base.h"
00272
00273
00274
00275 namespace core
00276 {
00277
00278
00279 class TimerCounter1 : public TimerCounter
00280 {
00281
00282 public:
00283
00284     static TimerCounter1& getInstance(const channel &ar_channel = channel::A,
00285                                       const operationMode &ar_operationMode = operationMode::normal,
00286                                       const clockSource &ar_clockSource= clockSource::noClock,
00287                                       const compareOutputMode& ar_compareOutputMode =
      compareOutputMode::normal);
00288
00289     void selectOperationMode(const operationMode &ar_operationMode) override;
00290
00291     void start() override;
00292
00293     void stop() override;
00294
00295     void selectClockSource(const clockSource &ar_clockSource) override;
00296
00297
00298     void selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
      &ar_compareOutputMode) override;
00299
00300     void setCounter(const uint16_t &ar_dataBuffer) override;
```

```
00301
00302     uint16_t getCounter() const override;
00303
00304     void setOutputCompareRegister(const channel &ar_channel, const uint16_t &ar_dataBuffer) override;
00305
00306     uint16_t getOutputCompareRegister(const channel &ar_channel) const override;
00307
00308     void setInputCaptureRegister(const uint16_t &ar_dataBuffer);
00309
00310     uint16_t getInputCaptureRegister() const;
00311
00312     void enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t a_enable)
      override;
00313
00314     void enableOverflowInterrupt(const uint8_t a_enable) override;
00315
00316     void enableInputCaptureInterrupt(const uint8_t a_enable);
00317
00318     uint16_t getClockPrescaler() override;
00319
00320     static void outputCompareMatchAServiceRoutine() __asm__(STR(TIMER1_COM_CHANNEL_A_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00321
00322     static void outputCompareMatchBServiceRoutine() __asm__(STR(TIMER1_COM_CHANNEL_B_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__, weak));
00323
00324     static void overflowServiceRoutine() __asm__(STR(TIMER1_OVERFLOW_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00325
00326     static void inputCaptureServiceRoutine() __asm__(STR(TIMER1_INPUT_CAPTURE_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__, weak));
00327
00328
00329
00330 protected:
00331
00332 private:
00333
00334     TimerCounter1(const channel &ar_channel,
00335                   const operationMode &ar_operationMode,
00336                   const clockSource &ar_clockSource,
00337                   const compareOutputMode& ar_compareOutputMode);
00338
00339     ~TimerCounter1();
00340
00343     TimerCounter1(const TimerCounter1&);
00344
00347     const TimerCounter1& operator=(const TimerCounter1&);
00348
00349     uint16_t m_clockPrescaler;
00350
00351     uint8_t m_clockSource;
00352
00353 };
00354
00355 }
00356
00357
00358 #endif
```

9.103 TimerCounter2.cpp File Reference

```
#include "TimerCounter2.h"
#include "MCU.h"
```

9.104 TimerCounter2.cpp

Go to the documentation of this file.

```
00001 #include "TimerCounter2.h"
00002 #include "MCU.h"
00003
00004
00005 core::TimerCounter2& core::TimerCounter2::getInstance(const channel &ar_channel,
```

```
00006                                                       const operationMode &ar_operationMode,
00007                                                       const clockSource &ar_clockSource,
00008                                                       const compareOutputMode& ar_compareOutputMode)
00009 {
00010     static TimerCounter2 l_instance(ar_channel,
00011                                     ar_operationMode,
00012                                     ar_clockSource,
00013                                     ar_compareOutputMode);
00014
00015     return l_instance;
00016
00017
00018 }
00019
00020 core::TimerCounter2::TimerCounter2(const channel &ar_channel,
00021                                    const operationMode &ar_operationMode,
00022                                    const clockSource &ar_clockSource,
00023                                    const compareOutputMode& ar_compareOutputMode)
00024 {
00025     core::MCU::enableTimerCounter2(1);
00026     stop();
00027     selectOperationMode(ar_operationMode);
00028     selectClockSource(ar_clockSource);
00029     selectCompareOutputMode(ar_channel, ar_compareOutputMode);
00030
00031 }
00032 core::TimerCounter2::~TimerCounter2()
00033 {
00034
00035 }
00036
00037 void core::TimerCounter2::selectClockSource(const clockSource &ar_clockSource)
00038 {
00039     switch (ar_clockSource)
00040     {
00041         case core::clockSource::noClock:
00042         {
00043             m_clockPrescaler=0;
00044             m_clockSource=0;
00045             break;
00046         }
00047         case core::clockSource::PS_1:
00048         {
00049             m_clockPrescaler=1;
00050             m_clockSource=1;
00051             break;
00052         }
00053         case core::clockSource::PS_8:
00054         {
00055             m_clockPrescaler=8;
00056             m_clockSource=2;
00057             break;
00058         }
00059         case core::clockSource::PS_32:
00060         {
00061             m_clockPrescaler=32;
00062             m_clockSource=3;
00063             break;
00064         }
00065         case core::clockSource::PS_64:
00066         {
00067             m_clockPrescaler=64;
00068             m_clockSource=4;
00069             break;
00070         }
00071         case core::clockSource::PS_128:
00072         {
00073             m_clockPrescaler=128;
00074             m_clockSource=5;
00075             break;
00076         }
00077         case core::clockSource::PS_256:
00078         {
00079             m_clockPrescaler=256;
00080             m_clockSource=6;
00081             break;
00082         }
00083         case core::clockSource::PS_1024:
00084         {
00085             m_clockPrescaler=1024;
00086             m_clockSource=7;
00087             break;
00088         }
00089         default:
00090         {
00091             // unsupported on this timer (external clock options): stop the
00092             // clock instead of keeping the previously selected source
```

```
00093             m_clockPrescaler=0;
00094             m_clockSource=0;
00095             break;
00096         }
00097     }
00098
00099 }
00100
00101 void core::TimerCounter2::selectOperationMode(const operationMode &ar_operationMode)
00102 {
00103     switch (ar_operationMode)
00104     {
00105         case core::operationMode::normal:
00106         {
00107             TIMER2_SELECT_OPERATION_MODE(0);
00108             break;
00109         }
00110         case core::operationMode::PWM_PC:
00111         {
00112             TIMER2_SELECT_OPERATION_MODE(1);
00113             break;
00114         }
00115         case core::operationMode::CTC_OCR:
00116         {
00117             TIMER2_SELECT_OPERATION_MODE(2);
00118             break;
00119         }
00120         case core::operationMode::fast_PWM:
00121         {
00122             TIMER2_SELECT_OPERATION_MODE(3);
00123             break;
00124         }
00125         case core::operationMode::PWM_PC_OCR:
00126         {
00127             TIMER2_SELECT_OPERATION_MODE(5);
00128             break;
00129         }
00130         case core::operationMode::fast_PWM_OCR:
00131         {
00132             TIMER2_SELECT_OPERATION_MODE(7);
00133             break;
00134         }
00135         default:
00136         {
00137             // unsupported on this timer: fall back to normal mode instead
00138             // of keeping the previously programmed mode
00139             TIMER2_SELECT_OPERATION_MODE(0);
00140             break;
00141         }
00142
00143     }
00144
00145 }
00146
00147
00148 void core::TimerCounter2::start()
00149 {
00150     TIMER2_SELECT_CLOCK_SOURCE(m_clockSource);
00151 }
00152
00153 uint16_t core::TimerCounter2::getClockPrescaler()
00154 {
00155     return m_clockPrescaler;
00156 }
00157
00158
00159 void core::TimerCounter2::stop()
00160 {
00161     TIMER2_STOP;
00162 }
00163
00164
00165 void core::TimerCounter2::selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
      &ar_compareOutputMode)
00166 {
00167     switch (ar_channel)
00168     {
00169         case core::channel::A:
00170         {
00171             TIMER2_SELECT_COM_CHANNEL_A(static_cast<uint8_t>(ar_compareOutputMode));
00172             break;
00173         }
00174         case core::channel::B:
00175         {
00176             TIMER2_SELECT_COM_CHANNEL_B(static_cast<uint8_t>(ar_compareOutputMode));
00177             break;
00178         }
```

```
00179     }
00180
00181
00182 }
00183
00184
00185
00186 void core::TimerCounter2::setCounter(const uint16_t &ar_dataBuffer)
00187 {
00188     TCNT2 = static_cast<uint8_t>(ar_dataBuffer);
00189 }
00190
00191 uint16_t core::TimerCounter2::getCounter() const
00192 {
00193     return TCNT2;
00194 }
00195
00196 void core::TimerCounter2::setOutputCompareRegister(const channel &ar_channel, const uint16_t
      &ar_dataBuffer)
00197 {
00198     switch (ar_channel)
00199     {
00200         case core::channel::A:
00201         {
00202             OCR2A = static_cast<uint8_t>(ar_dataBuffer);
00203             break;
00204         }
00205         case core::channel::B:
00206         {
00207             OCR2B = static_cast<uint8_t>(ar_dataBuffer);
00208             break;
00209         }
00210     }
00211 }
00212
00213
00214 uint16_t core::TimerCounter2::getOutputCompareRegister(const channel &ar_channel) const
00215 {
00216     uint16_t compareRegister = 0;
00217     switch (ar_channel)
00218     {
00219         case core::channel::A:
00220         {
00221             compareRegister = OCR2A;
00222             break;
00223         }
00224         case core::channel::B:
00225         {
00226             compareRegister = OCR2B;
00227             break;
00228         }
00229     }
00230     return compareRegister;
00231
00232 }
00233
00234 void core::TimerCounter2::enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t
      a_enable)
00235 {
00236     switch (ar_channel)
00237     {
00238         case core::channel::A:
00239         {
00240             if (a_enable) {
00241
00242                 TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT;
00243
00244             } else {
00245
00246                 TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT;
00247             }
00248             break;
00249         }
00250         case core::channel::B:
00251         {
00252             if (a_enable) {
00253
00254                 TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT;
00255
00256             } else {
00257
00258                 TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT;
00259
00260             }
00261             break;
00262         }
00263     }
```

```
00264
00265 }
00266
00267
00268
00269 void core::TimerCounter2::enableOverflowInterrupt(const uint8_t a_enable)
00270 {
00271
00272     if (a_enable) {
00273
00274         TIMER2_ENABLE_OVERFLOW_INTERRUPT;
00275
00276     } else {
00277
00278         TIMER2_DISABLE_OVERFLOW_INTERRUPT;
00279     }
00280
00281 }
00282
00283 // weak default interrupt handlers: empty bodies the application may
00284 // override by defining the same vector symbol (e.g. ISR(TIMER0_COMPA_vect))
00285
00286 void core::TimerCounter2::outputCompareMatchAServiceRoutine() {}
00287
00288 void core::TimerCounter2::outputCompareMatchBServiceRoutine() {}
00289
00290 void core::TimerCounter2::overflowServiceRoutine() {}
```

9.105 TimerCounter2.h File Reference

```
#include "TimerCounter.h"
#include "ha_base.h"
```

Classes

- class core::TimerCounter2

Namespaces

- namespace core

  AVR chip internal core components.

9.106 TimerCounter2.h

Go to the documentation of this file.

```
00001
00039     component::Led Led(io::Pin(LED_NUMBER,io::PortB));
00040
00041     // instantiate a Led object
00042     component::PushButton PushButton(io::Pin(PUSHBUTTON_NUMBER,io::PortD));
00043
00044
00045
00046     // ready to send flag
00047     uint8_t l_ready2Send = 1;
00048
00049     if (myUSART0.ready2Send())
00050     {
00051     myUSART0.sendString("Reaction Timer:\r\n");
00052     }
00053
00054     // wait to send next string
00055     while (!myUSART0.ready2Send()){};
00056
```

```
00057     if (myUSART0.ready2Send())
00058     {
00059     myUSART0.sendString("---------------\r\n");
00060     }
00061
00062     // wait to send next string
00063     while (!myUSART0.ready2Send()){};
00064
00065     if (myUSART0.ready2Send())
00066     {
00067     myUSART0.sendString("Press any key to start.\r\n");
00068     }
00069
00070     // Mainloop
00071     while (1) {
00072
00073       myUSART0.receiveFrame(reinterpret_cast<uint8_t*>(l_receiverBuffer),BUFFER_SIZE);
00074
00075       // wait for a key press
00076       while (!myUSART0.getNumberBytesReceived())
00077       {
00078
00079       }
00080
00081       // reset number of bytes after extracting the received data
00082       myUSART0.resetNumberBytesReceived();
00083
00084       // wait to send next string
00085       while (!myUSART0.ready2Send()){};
00086
00087       if (myUSART0.ready2Send())
00088       {
00089           myUSART0.sendString("\r\nGet ready...");
00090       }
00091
00092       randomDelay();
00093
00094       // wait to send next string
00095       while (!myUSART0.ready2Send()){};
00096
00097       if (myUSART0.ready2Send())
00098       {
00099           myUSART0.sendString("\r\nGo!\r\n");
00100       }
00101
00102       Led.on();
00103
00104       myCounter.setCounter(core::timerCounter::timerCounter1,0);
00105
00106       if (PushButton.isPressed())
00107       {
00108           // wait to send next string
00109           while (!myUSART0.ready2Send()){};
00110
00111           if (myUSART0.ready2Send())
00112           {
00113               myUSART0.sendString("You're only cheating yourself.\r\n");
00114           }
00115
00116       } else {
00117
00118           // wait for a button press
00119           while (!PushButton.isPressed())
00120           {
00121
00122           }
00123           // get counter value
00124           myCounter.getCounter(core::timerCounter::timerCounter1, &timerValue);
00125           // bit shift divide by 16 = 2^4 to convert from micro to millseconds
00126           timerValue = timerValue » 4;
00127           // print response time
00128           printWord(timerValue);
00129       }
00130
00131       Led.off();
00132
00133       while (!myUSART0.ready2Send()){};
00134
00135       if (myUSART0.ready2Send())
00136       {
00137           myUSART0.sendString("\r\nPress any key to try again.\r\n");
00138       }
00139
00140     }
00141     return 0;
00142     }
00143
```

```
00144     void randomDelay(void) {
00145
00146     // Waits for a "random" delay from 1 - 3.5 sec
00147     // Requires timer 1 initialized and running
00148     // It's not really random, but very hard to control --like coin-flipping.
00149
00150     uint16_t counter;
00151     _delay_ms(1000);
00152
00153     myCounter.getCounter(core::timerCounter::timerCounter1, &counter);
00154     uint8_t randomTime = static_cast<uint8_t>(counter);
00155     // type-casting the 16-bit TCNT1 as an 8-bit number keeps
00156     //     only the 8 least-significant (fastest-changing) bits
00157
00158     while (--randomTime) {
00159     _delay_ms(10);
00160     }
00161     }
00162
00163     void printWord(uint16_t word) {
00164
00165     while (!myUSART0.ready2Send()){};
00166
00167     if (myUSART0.ready2Send())
00168     {
00169         myUSART0.sendByte('0' + (word / 10000));
00170
00171     }
00172
00173     while (!myUSART0.ready2Send()){};
00174
00175
00176     if (myUSART0.ready2Send())
00177     {
00178         myUSART0.sendByte('0' + ((word / 1000) % 10));
00179     }
00180
00181     while (!myUSART0.ready2Send()){};
00182
00183
00184     if (myUSART0.ready2Send())
00185     {
00186         myUSART0.sendByte('0' + ((word / 100) % 10));
00187     }
00188
00189
00190     while (!myUSART0.ready2Send()){};
00191
00192     if (myUSART0.ready2Send())
00193     {
00194         myUSART0.sendByte('0' + ((word / 10) % 10));
00195     }
00196
00197
00198
00199     while (!myUSART0.ready2Send()){};
00200
00201     if (myUSART0.ready2Send())
00202     {
00203         myUSART0.sendByte('0' + (word % 10));
00204     }
00205
00206     }
00207     void printWord(uint16_t word) {
00208
00209     char timerValue = '0' + (word / 10000);
00210
00211     while (!myUSART0.ready2Send()){};
00212
00213     if (myUSART0.ready2Send())
00214     {
00215        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00216
00217     }
00218
00219     timerValue = '0' + ((word / 1000) % 10);
00220
00221     while (!myUSART0.ready2Send()){};
00222
00223
00224     if (myUSART0.ready2Send())
00225     {
00226        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00227
00228     }
00229
00230     timerValue = '0' + ((word / 100) % 10);
```

```
00231
00232     while (!myUSART0.ready2Send()){};
00233
00234
00235     if (myUSART0.ready2Send())
00236     {
00237        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00238
00239     }
00240     timerValue = '0' + ((word / 10) % 10);
00241
00242     while (!myUSART0.ready2Send()){};
00243
00244     if (myUSART0.ready2Send())
00245     {
00246        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00247
00248     }
00249
00250     timerValue = '0' + (word % 10);
00251
00252     while (!myUSART0.ready2Send()){};
00253
00254     if (myUSART0.ready2Send())
00255     {
00256        myUSART0.sendFrame(reinterpret_cast<uint8_t*>(&timerValue),BUFFER_SIZE);
00257
00258     }
00259
00260     }
00261  *
00262  * Usage example (Reaction Timer):
00263  *
00264  * @author Farid Oubbati (https://github.com/faroub)
00265  * @date March 2020
00266 */
00267
00268 #ifndef TIMER_COUNTER2_H
00269 #define TIMER_COUNTER2_H
00270 #include "TimerCounter.h"
00271 #include "ha_base.h"
00272
00273
00274
00275 namespace core
00276 {
00277
00278
00279 class TimerCounter2 : public TimerCounter
00280 {
00281
00282 public:
00283
00284     static TimerCounter2& getInstance(const channel &ar_channel = channel::A,
00285                                       const operationMode &ar_operationMode = operationMode::normal,
00286                                       const clockSource &ar_clockSource= clockSource::noClock,
00287                                       const compareOutputMode& ar_compareOutputMode =
      compareOutputMode::normal);
00288
00289     void selectOperationMode(const operationMode &ar_operationMode) override;
00290
00291     void start() override;
00292
00293     void stop() override;
00294
00295     void selectClockSource(const clockSource &ar_clockSource) override;
00296
00297     void selectCompareOutputMode(const channel &ar_channel, const compareOutputMode
      &ar_compareOutputMode) override;
00298
00299     void setCounter(const uint16_t &ar_dataBuffer) override;
00300
00301     uint16_t getCounter() const override;
00302
00303     void setOutputCompareRegister(const channel &ar_channel, const uint16_t &ar_dataBuffer) override;
00304
00305     uint16_t getOutputCompareRegister(const channel &ar_channel) const override;
00306
00307     void enableOutputCompareMatchInterrupt(const channel &ar_channel, const uint8_t a_enable)
      override;
00308
00309     void enableOverflowInterrupt(const uint8_t a_enable) override;
00310
00311     uint16_t getClockPrescaler() override;
00312
00313     static void outputCompareMatchAServiceRoutine() __asm__(STR(TIMER2_COM_CHANNEL_A_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__, weak));
```

```
00314
00315     static void outputCompareMatchBServiceRoutine() __asm__(STR(TIMER2_COM_CHANNEL_B_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__, weak));
00316
00317     static void overflowServiceRoutine() __asm__(STR(TIMER2_OVERFLOW_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__, weak));
00318
00319 protected:
00320
00321 private:
00322
00323     TimerCounter2(const channel &ar_channel,
00324                   const operationMode &ar_operationMode,
00325                   const clockSource &ar_clockSource,
00326                   const compareOutputMode& ar_compareOutputMode);
00327
00328     ~TimerCounter2();
00329
00332     TimerCounter2(const TimerCounter2&);
00333
00336     const TimerCounter2& operator=(const TimerCounter2&);
00337
00338     uint16_t m_clockPrescaler;
00339
00340     uint8_t m_clockSource;
00341
00342 };
00343
00344 }
00345
00346
00347 #endif
```

9.107 TWI.cpp File Reference

Implementation of the Two-Wire Interface (I2C) class.

```
#include "TWI.h"
#include "MCU.h"
```

Macros

- #define TWI_TIMEOUT_CYCLES 10000U

9.107.1 Detailed Description

Implementation of the Two-Wire Interface (I2C) class.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file TWI.cpp.

9.107.2 Macro Definition Documentation

9.107.2.1 TWI_TIMEOUT_CYCLES

```
#define TWI_TIMEOUT_CYCLES 10000U
```

Definition at line 12 of file TWI.cpp.

Referenced by core::TWI::readAck(), core::TWI::readNack(), core::TWI::start(), and core::TWI::write().

9.108 TWI.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "TWI.h"
00010 #include "MCU.h"
00011
00012 #define TWI_TIMEOUT_CYCLES 10000U
00013
00014 core::TWI& core::TWI::getInstance(uint32_t a_clockFreq_hz)
00015 {
00016     static TWI l_instance(a_clockFreq_hz);
00017     return l_instance;
00018 }
00019
00020 core::TWI::TWI(uint32_t a_clockFreq_hz)
00021 {
00022     core::MCU::enableTWI(1);
00023     setClock(a_clockFreq_hz);
00024 }
00025
00026 core::TWI::~TWI()
00027 {
00028 }
00029
00030 void core::TWI::setClock(uint32_t a_clockFreq_hz)
00031 {
00032     // Clamp clock frequency to valid range [10 kHz, 400 kHz]
00033     if (a_clockFreq_hz < 10000UL) a_clockFreq_hz = 10000UL;
00034     if (a_clockFreq_hz > 400000UL) a_clockFreq_hz = 400000UL;
00035
00036     if (F_CPU < 16UL * a_clockFreq_hz) {
00037         TWBR = 0;
00038         TWSR = 0;
00039         return;
00040     }
00041
00042     uint32_t twbr_val = ((F_CPU / a_clockFreq_hz) - 16UL) / 2UL;
00043     if (twbr_val > 255UL) twbr_val = 255UL;
00044
00045     TWSR = 0; // Prescaler = 1
00046     TWBR = static_cast<uint8_t>(twbr_val);
00047 }
00048
00049 uint8_t core::TWI::start()
00050 {
00051     TWCR = (1 « TWINT) | (1 « TWSTA) | (1 « TWEN);
00052     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00053     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00054     if (timeout == 0) return 0; // Bus timeout
00055
00056     uint8_t status = TWSR & 0xF8;
00057     return (status == 0x08 || status == 0x10) ? 1 : 0;
00058 }
00059
00060 void core::TWI::stop()
00061 {
00062     TWCR = (1 « TWINT) | (1 « TWSTO) | (1 « TWEN);
00063 }
00064
00065 uint8_t core::TWI::write(uint8_t a_data)
00066 {
00067     TWDR = a_data;
00068     TWCR = (1 « TWINT) | (1 « TWEN);
```

```
00069     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00070     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00071     if (timeout == 0) return 0; // Bus timeout
00072
00073     uint8_t status = TWSR & 0xF8;
00074     return (status == 0x18 || status == 0x28 || status == 0x40) ? 1 : 0;
00075 }
00076
00077 uint8_t core::TWI::readAck()
00078 {
00079     TWCR = (1 « TWINT) | (1 « TWEN) | (1 « TWEA);
00080     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00081     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00082     if (timeout == 0) return 0xFF;
00083     return TWDR;
00084 }
00085
00086 uint8_t core::TWI::readNack()
00087 {
00088     TWCR = (1 « TWINT) | (1 « TWEN);
00089     uint16_t timeout = TWI_TIMEOUT_CYCLES;
00090     while (!(TWCR & (1 « TWINT)) && --timeout) {}
00091     if (timeout == 0) return 0xFF;
00092     return TWDR;
00093 }
```

9.109 TWI.h File Reference

Header file for the Two-Wire Interface (I2C) class.

```
#include "ha_base.h"
```

Classes

- class core::TWI

Namespaces

- namespace core

  AVR chip internal core components.

9.109.1 Detailed Description

Header file for the Two-Wire Interface (I2C) class.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file TWI.h.

9.110 TWI.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef TWI_H
00010 #define TWI_H
00011
00012 #include "ha_base.h"
00013
00014 namespace core
00015 {
00016
00017 class TWI
00018 {
00019 public:
00024     static TWI& getInstance(uint32_t a_clockFreq_hz = 100000UL);
00025
00030     void setClock(uint32_t a_clockFreq_hz);
00031
00036     uint8_t start();
00037
00040     void stop();
00041
00047     uint8_t write(uint8_t a_data);
00048
00053     uint8_t readAck();
00054
00059     uint8_t readNack();
00060
00061 private:
00062     TWI(uint32_t a_clockFreq_hz);
00063     ~TWI();
00064     TWI(const TWI&);
00065     const TWI& operator=(const TWI&);
00066 };
00067
00068 }
00069
00070 #endif
```

9.111 WatchdogTimer.cpp File Reference

```
#include "WatchdogTimer.h"
#include <util/atomic.h>
```

9.112 WatchdogTimer.cpp

Go to the documentation of this file.

```
00001 #include "WatchdogTimer.h"
00002 #include <util/atomic.h>
00003
00004
00005 core::WatchdogTimer& core::WatchdogTimer::getInstance()
00006 {
00007     static WatchdogTimer l_instance;
00008     return l_instance;
00009
00010 }
00011
00012 core::WatchdogTimer::WatchdogTimer()
00013 {
00014     stop();
00015 }
00016
00017 core::WatchdogTimer::~WatchdogTimer()
00018 {
00019
00020 }
00021
```

```
00022 void core::WatchdogTimer::selectTimeOut(const timeOut &ar_timeOut)
00023 {
00024     m_timeOut = static_cast<uint8_t>(ar_timeOut);
00025     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00026     {
00027         wdt_reset();
00028         if (m_isRunning)
00029         {
00030             // watchdog is running: follow the timed change sequence and keep the
00031             // programmed mode bits (WDIE/WDE) set, so the watchdog keeps running
00032             // with the new timeout
00033             WDTCSR |= (1 << WDCE) | (1 << WDE);
00034             WDTCSR = timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)) |
00035                      operationModeToWDTCSR(static_cast<watchdogOperationMode>(m_operationMode));
00036         }
00037         else
00038         {
00039             // watchdog not running: stage the timeout bits only
00040             WATCHDOG_SELECT_TIMEOUT(timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)));
00041         }
00042     }
00043
00044 }
00045
00046 void core::WatchdogTimer::reset()
00047 {
00048     wdt_reset();
00049
00050 }
00051
00052 void core::WatchdogTimer::start(const watchdogOperationMode &ar_operationMode)
00053 {
00054     m_operationMode = static_cast<uint8_t>(ar_operationMode);
00055     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00056     {
00057         wdt_reset();
00058         WATCHDOG_START(operationModeToWDTCSR(static_cast<watchdogOperationMode>(m_operationMode)),
00059                        timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)));
00060         m_isRunning = 1;
00061     }
00062
00063 }
00064
00065 void core::WatchdogTimer::start(const watchdogOperationMode &ar_operationMode, const timeOut
      &ar_timeOut)
00066 {
00067     m_timeOut = static_cast<uint8_t>(ar_timeOut);
00068     m_operationMode = static_cast<uint8_t>(ar_operationMode);
00069     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00070     {
00071         wdt_reset();
00072         WATCHDOG_START(operationModeToWDTCSR(static_cast<watchdogOperationMode>(m_operationMode)),
00073                        timeOutToWDTCSR(static_cast<timeOut>(m_timeOut)));
00074         m_isRunning = 1;
00075     }
00076
00077 }
00078
00079 void core::WatchdogTimer::stop()
00080 {
00081     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00082     {
00083         wdt_reset();
00084         WATCHDOG_STOP;
00085         m_isRunning = 0;
00086     }
00087
00088 }
00089
00090 // weak default interrupt handlers: empty bodies the application may
00091 // override by defining the same vector symbol (e.g. ISR(TIMER0_COMPA_vect))
00092
00093 void core::WatchdogTimer::timeOutServiceRoutine() {}
```

9.113 WatchdogTimer.h File Reference

Header file of the WatchdogTimer class.

```
#include "ha_base.h"
```

Classes

- class core::WatchdogTimer

Namespaces

- namespace core

  AVR chip internal core components.

Enumerations

- enum class core::timeOut : uint8_t { core::to_16ms =0 , core::to_32ms , core::to_64ms , core::to_125ms , core::to_250ms , core::to_500ms , core::to_1s , core::to_2s , core::to_4s , core::to_8s }
- enum class core::watchdogOperationMode : uint8_t { core::interrupt =1 , core::reset , core::interrupt_reset }

Functions

- constexpr uint8_t core::timeOutToWDTCSR (const timeOut ar_timeOut)
- constexpr uint8_t core::operationModeToWDTCSR (const watchdogOperationMode ar_operationMode)

9.113.1 Detailed Description

Header file of the WatchdogTimer class.

Basic class abstraction of the WatchdogTimer peripheral

Usage example (test):

#include "WatchdogTimer.h" #include "Led.h" instantiate a Led object extern component::Led Led; component::Led Led(io::Pin(1,io::PortB)); extern component::Led LedStart; component::Led LedStart(io::Pin(2,io::PortB));

int main(void) {

Init instantiate a Watchdog object core::WatchdogTimer &myWatchdog = core::WatchdogTimer::getInstance(); myWatchdog.selectTimeOut(core::timeOut::to_8s); LedStart.on(); _delay_ms(5000); LedStart.off(); myWatchdog.start(core::watchdogOperationMode::reset); ----- Event loop ----- // while (1) {

```
Led.on();
_delay_ms(1000);
Led.off();
_delay_ms(1000);
```

} return 0; }

void core::WatchdogTimer::timeOutServiceRoutine() { for (uint8_t i=0;i<10;i++) { Led.on(); _delay_ms(100); Led.off(); _delay_ms(100);

}

}

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file WatchdogTimer.h.

9.114 WatchdogTimer.h

Go to the documentation of this file.

```
00001
00056
00057 #ifndef WATCHDOG_TIMER_H
00058 #define WATCHDOG_TIMER_H
00059 #include "ha_base.h"
00060
00061
00062
00063 namespace core
00064 {
00065 enum class timeOut : uint8_t {
00066     to_16ms=0,
00067     to_32ms,
00068     to_64ms,
00069     to_125ms,
00070     to_250ms,
00071     to_500ms,
00072     to_1s,
00073     to_2s,
00074     to_4s,
00075     to_8s,
00076 };
00077
00078 enum class watchdogOperationMode : uint8_t {
00079     interrupt=1,
00080     reset,
00081     interrupt_reset,
00082 };
00083
00092 constexpr uint8_t timeOutToWDTCSR(const timeOut ar_timeOut)
00093 {
00094     return static_cast<uint8_t>((static_cast<uint8_t>(ar_timeOut) & 7u) |
00095                                 ((static_cast<uint8_t>(ar_timeOut) & 8u) << 2u));
00096 }
00097
00103 constexpr uint8_t operationModeToWDTCSR(const watchdogOperationMode ar_operationMode)
00104 {
00105     // mode bit 0 (value 1) maps to WDIE (bit 6); mode bit 1 (value 2) maps to
00106     // WDE (bit 3), so it must be shifted left by (WDE - 1): shifting by WDE
00107     // would land on WDCE (bit 4) instead
00108     return static_cast<uint8_t>(((static_cast<uint8_t>(ar_operationMode) & 1u) << WDIE) |
00109                                 ((static_cast<uint8_t>(ar_operationMode) & 2u) << (WDE - 1)));
00110 }
00111
00112 static_assert((operationModeToWDTCSR(watchdogOperationMode::reset) & (1u << WDE)) != 0,
00113               "watchdog reset mode must set WDE");
00114 static_assert((operationModeToWDTCSR(watchdogOperationMode::interrupt_reset) & (1u << WDE)) != 0,
00115               "watchdog interrupt+reset mode must set WDE");
00116 static_assert((operationModeToWDTCSR(watchdogOperationMode::interrupt) & (1u << WDE)) == 0,
00117               "watchdog interrupt-only mode must not set WDE");
00118
00119 class WatchdogTimer
00120 {
00121
00122 public:
00123
00124     static WatchdogTimer& getInstance();
00125
00136     void selectTimeOut(const timeOut &ar_timeOut);
00137
00138     void reset();
00139
00140     void start(const watchdogOperationMode &ar_operationMode);
00141
00142     void start(const watchdogOperationMode &ar_operationMode, const timeOut &ar_timeOut);
00143
00144
00145     void stop();
00146
00147     static void timeOutServiceRoutine() __asm__(STR(WATCHDOG_TIMEOUT_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__, weak));
00148
00149
00150 protected:
00151
00152 private:
00153
00154     WatchdogTimer();
00155
00158     ~WatchdogTimer();
00159
00162     WatchdogTimer(const WatchdogTimer&);
```

```
00163
00166     const WatchdogTimer& operator=(const WatchdogTimer&);
00167
00168     uint8_t m_timeOut;
00169
00170     uint8_t m_operationMode;
00171
00172     uint8_t m_isRunning;
00173
00174 };
00175
00176 }
00177
00178
00179 #endif
```

9.115 Pin.cpp File Reference

```
#include "Pin.h"
```

9.116 Pin.cpp

Go to the documentation of this file.

```
00001
00009 #include "Pin.h"
00010
00011 extern const io::Port io::PortB = { &DDRB, &PORTB, &PINB };
00012 extern const io::Port io::PortC = { &DDRC, &PORTC, &PINC };
00013 extern const io::Port io::PortD = { &DDRD, &PORTD, &PIND };
00014
00015 io::Pin::Pin(const uint8_t a_pinNumber, const Port &ar_portName)
00016          : mr_portName(ar_portName), m_pinNumber(a_pinNumber)
00017 {
00018
00019 }
00020
00021 io::Pin::~Pin()
00022
00023 {
00024
00025 }
00026
00027 void io::Pin::toOutput()
00028 {
00029     *mr_portName.mp_ddrReg  |=  (1 << m_pinNumber);
00030 }
00031
00032 void io::Pin::toInput(const uint8_t &ar_useInternalPullUp)
00033 {
00034     if (ar_useInternalPullUp)
00035     {
00036         *mr_portName.mp_portReg |= (1 << m_pinNumber);
00037         *mr_portName.mp_ddrReg  &= ~(1 << m_pinNumber);
00038
00039
00040     }
00041     else
00042     {
00043         *mr_portName.mp_portReg &= ~(1 << m_pinNumber);
00044         *mr_portName.mp_ddrReg  &= ~(1 << m_pinNumber);
00045     }
00046
00047 }
00048
00049 void io::Pin::setLow()
00050 {
00051     if (*mr_portName.mp_ddrReg & (1 << m_pinNumber))
00052     {
00053         *mr_portName.mp_portReg &= ~(1 << m_pinNumber);
00054     }
00055 }
00056
00057 void io::Pin::setHigh()
```

```
00058 {
00059     if (*mr_portName.mp_ddrReg & (1 « m_pinNumber))
00060     {
00061         *mr_portName.mp_portReg |=  (1 « m_pinNumber);
00062     }
00063 }
00064
00065 void io::Pin::toggle()
00066 {
00067     if (*mr_portName.mp_ddrReg & (1 « m_pinNumber))
00068     {
00069         *mr_portName.mp_portReg ^=   1 « m_pinNumber;
00070     }
00071 }
00072
00073 uint8_t io::Pin::isHigh()
00074 {
00075     return (*mr_portName.mp_pinReg & (1 « m_pinNumber)) ? 1 : 0;
00076 }
00077
00078 uint8_t io::Pin::isLow()
00079 {
00080     return (*mr_portName.mp_pinReg & (1 « m_pinNumber)) ? 0 : 1;
00081 }
00082
00083 uint8_t io::Pin::getPinNumber()
00084 {
00085     return m_pinNumber;
00086 }
```

9.117 Pin.h File Reference

```
#include "ha_base.h"
```

Classes

- struct io::Port

  Contains defintions of pointers to the AVR port's registers.
- class io::Pin

  Class for handling a Pin.

Namespaces

- namespace io

  AVR chip internal i/o components.

Variables

- const io::Port io::PortB = { &DDRB, &PORTB, &PINB }
- const io::Port io::PortC = { &DDRC, &PORTC, &PINC }
- const io::Port io::PortD = { &DDRD, &PORTD, &PIND }

9.118 Pin.h

Go to the documentation of this file.

```
00001 #ifndef PIN_H
00002 #define PIN_H
00003 #include "ha_base.h"
00012 namespace io
00013 {
00014
00019 struct Port
00020 {
00021   volatile uint8_t* mp_ddrReg;
00022
00023   volatile uint8_t* mp_portReg;
00024
00025   volatile uint8_t* mp_pinReg;
00026 };
00035 class Pin
00036 {
00037
00038 public:
00046     Pin(const uint8_t a_pinNumber, const Port &ar_portName);
00049     ~Pin();
00052     void toOutput();
00057     void toInput(const uint8_t &ar_useInternalPullUp);
00064     void setLow();
00071     void setHigh();
00072
00075     void toggle();
00081     uint8_t isHigh();
00087     uint8_t isLow();
00093     uint8_t getPinNumber();
00094
00095 protected:
00096
00097
00098 private:
00099
00100      const Port &mr_portName;
00101      const uint8_t m_pinNumber;
00102
00103
00104
00105
00106 };
00107
00108 // the port objects are defined once in Pin.cpp
00109 extern const io::Port PortB;
00110 extern const io::Port PortC;
00111 extern const io::Port PortD;
00112
00113 }
00114
00115
00116 #endif
```

9.119 SPI.cpp File Reference

```
#include "SPI.h"
#include "MCU.h"
```

9.120 SPI.cpp

Go to the documentation of this file.

```
00001 #include "SPI.h"
00002 #include "MCU.h"
00003
00004 volatile uint8_t io::SPI::m_data = 0;
00005 volatile uint8_t io::SPI::m_transferComplete = 0;
00006
```

```
00007
00008
00009 io::SPI& io::SPI::getInstance(const Pin &ar_pinSCK,
00010                               const Pin &ar_pinMISO,
00011                               const Pin &ar_pinMOSI,
00012                               const Pin &ar_pinSS)
00013 {
00014     static SPI l_instance(ar_pinSCK,
00015                           ar_pinMISO,
00016                           ar_pinMOSI,
00017                           ar_pinSS);
00018
00019     return l_instance;
00020
00021 }
00022
00023 io::SPI::SPI(const Pin &ar_pinSCK,
00024              const Pin &ar_pinMISO,
00025              const Pin &ar_pinMOSI,
00026              const Pin &ar_pinSS)
00027         : m_pinSCK(ar_pinSCK),
00028           m_pinMISO(ar_pinMISO),
00029           m_pinMOSI(ar_pinMOSI),
00030           m_pinSS(ar_pinSS)
00031 {
00032     core::MCU::enableSPI(1);
00033     enableTransferCompleteInterrupt(1);
00034
00035 }
00036
00037 io::SPI::~SPI()
00038 {
00039
00040 }
00041
00042 uint8_t io::SPI::writeCollision()
00043 {
00044     return (SPI_STATUS_REGISTER & (1 << SPI_WRITE_COLLISION));
00045 }
00046
00047
00048 uint8_t io::SPI::transferComplete()
00049 {
00050     return (SPI_STATUS_REGISTER & (1 << SPI_TRANSFER_COMPLETE));
00051 }
00052
00053 void io::SPI::selectClockPrescaler(const clockPrescaler& ar_clockPrescaler)
00054 {
00055     SPI_SELECT_CLOCK_PRESCALER(static_cast<uint8_t>(ar_clockPrescaler));
00056
00057 }
00058
00059 void io::SPI::selectOperationMode(const operationMode& ar_operationMode)
00060 {
00061     switch (ar_operationMode)
00062     {
00063         case operationMode::master:
00064         {
00065             m_pinMOSI.toOutput();
00066             m_pinSCK.toOutput();
00067             m_pinMISO.toInput(1);
00068             m_pinSS.toOutput();
00069             m_pinSS.setHigh();
00070             SPI_SELECT_MASTER_MODE;
00071             SPI_ENABLE;
00072             break;
00073         }
00074         case operationMode::slave:
00075         {
00076             m_pinMISO.toOutput();
00077             SPI_SELECT_SLAVE_MODE;
00078             SPI_ENABLE;
00079             break;
00080         }
00081         case operationMode::submaster:
00082         {
00083             m_pinMOSI.toOutput();
00084             m_pinSCK.toOutput();
00085             m_pinMISO.toInput(1);
00086             m_pinSS.toInput(1);
00087             SPI_SELECT_MASTER_MODE;
00088             SPI_ENABLE;
00089             break;
00090         }
00091         case operationMode::disable:
00092         {
00093             SPI_DISABLE;
```

```
00094             break;
00095         }
00096     }
00097 }
00098
00099 void io::SPI::selectDataMode(const dataMode& ar_dataMode)
00100 {
00101     SPI_SELECT_DATA_MODE(static_cast<uint8_t>(ar_dataMode));
00102 }
00103
00104 void io::SPI::selectDataOrder(const dataOrder& ar_dataOrder)
00105 {
00106     SPI_SELECT_DATA_ORDER(static_cast<uint8_t>(ar_dataOrder));
00107
00108 }
00109
00110 void io::SPI::enableTransferCompleteInterrupt(const uint8_t a_enable)
00111 {
00112     if (a_enable) {
00113         SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT;
00114     } else {
00115         SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT;
00116     }
00117 }
00118
00119 void io::SPI::selectSlave(const uint8_t a_select)
00120 {
00121     if (a_select) {
00122         m_pinSS.setLow();
00123
00124     } else {
00125         m_pinSS.setHigh();
00126     }
00127
00128 }
00129 void io::SPI::transferCompleteServiceRoutine()
00130 {
00131
00132         // reading SPDR also clears SPIF in the status register
00133         m_data = SPI_DATA_REGISTER;
00134         m_transferComplete = 1;
00135 }
00136
00137 void io::SPI::masterSendByte(const uint8_t &ar_byte)
00138 {
00139     // assert SS before writing SPDR, so the slave is selected when the
00140     // transfer starts
00141     selectSlave(1);
00142     m_transferComplete = 0;
00143     SPI_DATA_REGISTER = ar_byte;
00144     // busy-wait on the flag set by the transfer-complete ISR: the wait
00145     // runs with interrupts enabled, so the SPI interrupt (armed in the
00146     // constructor) can fire and release it
00147     while (m_transferComplete == 0) {}
00148     selectSlave(0);
00149
00150
00151
00152 }
00153
00154 void io::SPI::masterReceiveByte(uint8_t &ar_byte)
00155 {
00156     // a dummy write clocks 8 SCK pulses; the ISR reads the byte the slave
00157     // shifted into SPDR during the same transfer
00158     masterSendByte(0);
00159     ar_byte = m_data;
00160
00161 }
00162
00163 void io::SPI::slaveReceiveByte(uint8_t &ar_byte)
00164 {
00165     ar_byte = m_data;
00166
00167 }
```

9.121 SPI.h File Reference

Header file of the SPI class.

```
#include "ha_base.h"
#include "Pin.h"
```

Classes

- class io::SPI

Namespaces

- namespace io

  AVR chip internal i/o components.

Enumerations

- enum class io::operationMode : uint8_t { io::master =0 , io::slave , io::submaster , io::disable }
- enum class io::clockPrescaler : uint8_t {
  io::PS_4 = 0 , io::PS_16 , io::PS_64 , io::PS_128 ,
  io::PS_2 , io::PS_8 , io::PS_32 }
- enum class io::dataMode : uint8_t { io::mode_0 = 0 , io::mode_1 , io::mode_2 , io::mode_3 }
- enum class io::dataOrder : uint8_t { io::first_MSB = 0 , io::first_LSB }

9.121.1 Detailed Description

Header file of the SPI class.

Usage example (test): #include "MCU.h" #include "SPI.h"

#define SPI_SCK 5 #define SPI_MISO 4 #define SPI_MOSI 3 #define SPI_SS 2

int main(void) {

Init initialize MCU core::MCU::init(); instantiate a SPI object io::SPI &myISP = io::SPI::getInstance(io::Pin(SPI_SCK,io::PortB), io::Pin(SPI_MISO,io::PortB), io::Pin(SPI_MOSI,io::PortB), io::Pin(SPI_SS,io::PortB));

myISP.selectClockPrescaler(io::clockPrescaler::PS_128); myISP.selectDataMode(io::dataMode::mode_0); myISP.selectDataOrder(io::dataOrder::first_LSB); myISP.selectOperationMode(io::operationMode::master);

myISP.masterSendByte(0x03);

Mainloop while (1) {

} return 0; }

Basic class for IO abstraction of Pin and Port

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file SPI.h.

9.122 SPI.h

Go to the documentation of this file.

```
00001
00047
00048 #ifndef SPI_H
00049 #define SPI_H
00050 #include "ha_base.h"
00051 #include "Pin.h"
00052
00053 namespace io
00054 {
00055
00056 enum class operationMode : uint8_t {
00057     master=0,
00058     slave,
00059     submaster,
00060     disable,
00061 };
00062
00063 enum class clockPrescaler : uint8_t  {
00064      PS_4 = 0,
00065      PS_16,
00066      PS_64,
00067      PS_128,
00068      PS_2,
00069      PS_8,
00070      PS_32
00071 };
00072
00073 enum class dataMode : uint8_t  {
00074      mode_0 = 0,
00075      mode_1,
00076      mode_2,
00077      mode_3,
00078 };
00079
00080 enum class dataOrder : uint8_t  {
00081      first_MSB = 0,
00082      first_LSB
00083 };
00084
00085
00086 class SPI
00087 {
00088
00089 public:
00090     static SPI& getInstance(const io::Pin &ar_pinSCK,
00091                             const io::Pin &ar_pinMISO,
00092                             const io::Pin &ar_pinMOSI,
00093                             const io::Pin &ar_pinSS);
00094
00095     void selectDataMode(const dataMode& ar_dataMode);
00096
00097     void selectDataOrder(const dataOrder& ar_dataOrder);
00098
00099     void selectOperationMode(const operationMode& ar_operationMode);
00100
00101     void selectClockPrescaler(const clockPrescaler& ar_clockPrescaler);
00102
00103     void selectSlave(const uint8_t a_select);
00104
00107     uint8_t writeCollision();
00108
00111     uint8_t transferComplete();
00112
00113
00114     static void enableTransferCompleteInterrupt(const uint8_t a_enable);
00115
00129     void masterSendByte(const uint8_t &ar_byte);
00130
00141     void masterReceiveByte(uint8_t &ar_byte);
00142
00143     void slaveReceiveByte(uint8_t &ar_byte);
00144
00147     static void transferCompleteServiceRoutine() __asm__(STR(SPI_TRANSFER_COMPLETE_INTERRUPT))
      __attribute__((__signal__, __used__, __externally_visible__));
00148
00149
00150
00151 private:
00160     SPI(const io::Pin &ar_pinSCK,
00161         const io::Pin &ar_pinMISO,
00162         const io::Pin &ar_pinMOSI,
00163         const io::Pin &ar_pinSS);
```

```
00164
00167     ~SPI();
00168
00171     SPI(const SPI&);
00172
00175     const SPI& operator=(const SPI&);
00176
00177     static volatile uint8_t m_data;
00178
00179     static volatile uint8_t m_transferComplete;
00180
00181     io::Pin m_pinSCK;
00182     io::Pin m_pinMISO;
00183     io::Pin m_pinMOSI;
00184     io::Pin m_pinSS;
00185
00186
00187
00188
00189
00190
00191
00192 };
00193
00194 }
00195 #endif // SPI_H
```

9.123 Telemetry.cpp File Reference

Implementation of the Telemetry class.

```
#include "Telemetry.h"
```

9.123.1 Detailed Description

Implementation of the Telemetry class.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file Telemetry.cpp.

9.124 Telemetry.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "Telemetry.h"
00010
00011 io::USART0* io::Telemetry::mp_usart = nullptr;
00012
00013 void io::Telemetry::init(io::USART0 &ar_usart)
00014 {
00015     mp_usart = &ar_usart;
00016 }
```

```
00017
00018 void io::Telemetry::sendADC(uint8_t a_channel, uint16_t a_value)
00019 {
00020     if (!mp_usart) {
00021         mp_usart = &io::USART0::getInstance();
00022     }
00023     a_channel &= 0x0F; // Clamp channel 0..15
00024
00025     while (!mp_usart->ready2Send()) {}
00026     mp_usart->sendString("$ADC,");
00027     mp_usart->sendByte(a_channel);
00028     while (!mp_usart->ready2Send()) {}
00029     mp_usart->sendString(",");
00030     mp_usart->sendWord(a_value);
00031     while (!mp_usart->ready2Send()) {}
00032     mp_usart->sendString("\r\n");
00033 }
00034
00035 void io::Telemetry::sendPin(uint8_t a_port, uint8_t a_pin, uint8_t a_state)
00036 {
00037     if (!mp_usart) {
00038         mp_usart = &io::USART0::getInstance();
00039     }
00040     a_port %= 3;      // Clamp port 0..2
00041     a_pin &= 0x07;    // Clamp pin 0..7
00042     a_state = a_state ? 1 : 0;
00043
00044     while (!mp_usart->ready2Send()) {}
00045     mp_usart->sendString("$PIN,");
00046     mp_usart->sendByte(a_port);
00047     while (!mp_usart->ready2Send()) {}
00048     mp_usart->sendString(",");
00049     mp_usart->sendByte(a_pin);
00050     while (!mp_usart->ready2Send()) {}
00051     mp_usart->sendString(",");
00052     mp_usart->sendByte(a_state);
00053     while (!mp_usart->ready2Send()) {}
00054     mp_usart->sendString("\r\n");
00055 }
00056
00057 void io::Telemetry::sendMotor(uint16_t a_pos, uint16_t a_speed)
00058 {
00059     if (!mp_usart) {
00060         mp_usart = &io::USART0::getInstance();
00061     }
00062     while (!mp_usart->ready2Send()) {}
00063     mp_usart->sendString("$MOTOR,");
00064     mp_usart->sendWord(a_pos);
00065     while (!mp_usart->ready2Send()) {}
00066     mp_usart->sendString(",");
00067     mp_usart->sendWord(a_speed);
00068     while (!mp_usart->ready2Send()) {}
00069     mp_usart->sendString("\r\n");
00070 }
00071
00072 void io::Telemetry::sendLog(const char *ap_msg)
00073 {
00074     if (!mp_usart) {
00075         mp_usart = &io::USART0::getInstance();
00076     }
00077     if (!ap_msg) {
00078         ap_msg = "NULL";
00079     }
00080     while (!mp_usart->ready2Send()) {}
00081     mp_usart->sendString("$LOG,");
00082     while (!mp_usart->ready2Send()) {}
00083     mp_usart->sendString(ap_msg);
00084     while (!mp_usart->ready2Send()) {}
00085     mp_usart->sendString("\r\n");
00086 }
```

9.125 Telemetry.h File Reference

Header file for the Telemetry class.

```
#include "ha_base.h"
#include "USART0.h"
```

Classes

- class io::Telemetry

Namespaces

- namespace io

  AVR chip internal i/o components.

Enumerations

- enum class io::TelemetryType : uint8_t { io::ADCSample = 0 , io::PinState , io::MotorState , io::CustomLog }

9.125.1 Detailed Description

Header file for the Telemetry class.

Provides real-time hardware telemetry streaming over USART0.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file Telemetry.h.

9.126 Telemetry.h

Go to the documentation of this file.

```
00001
00010
00011 #ifndef TELEMETRY_H
00012 #define TELEMETRY_H
00013
00014 #include "ha_base.h"
00015 #include "USART0.h"
00016
00017 namespace io
00018 {
00019
00020 enum class TelemetryType : uint8_t {
00021     ADCSample = 0,
00022     PinState,
00023     MotorState,
00024     CustomLog
00025 };
00026
00027 class Telemetry
00028 {
00029 public:
00034     static void init(io::USART0 &ar_usart = io::USART0::getInstance());
00035
00041     static void sendADC(uint8_t a_channel, uint16_t a_value);
00042
00049     static void sendPin(uint8_t a_port, uint8_t a_pin, uint8_t a_state);
00050
00056     static void sendMotor(uint16_t a_pos, uint16_t a_speed);
00057
00062     static void sendLog(const char *ap_msg);
00063
00064 private:
00065     static io::USART0 *mp_usart;
00066 };
00067
00068 }
00069
00070 #endif
```

9.127 USART0.cpp File Reference

```
#include "USART0.h"
#include "MCU.h"
#include <util/atomic.h>
#include <string.h>
```

9.128 USART0.cpp

Go to the documentation of this file.

```
00001
00009
00010 #include "USART0.h"
00011 #include "MCU.h"
00012 #include <util/atomic.h>
00013 #include <string.h>
00014
00015 volatile uint8_t io::USART0::m_status = 0;
00016 const uint8_t* io::USART0::mp_data2Send = nullptr;
00017 uint8_t* io::USART0::mp_data2Receive = nullptr;
00018 uint16_t io::USART0::m_sizeData2Send = 0;
00019 volatile uint16_t io::USART0::m_sizeData2Receive = 0;
00020 volatile uint8_t io::USART0::m_stringReceive = 0;
00021 volatile uint16_t io::USART0::m_numberBytesReceived = 0;
00022 volatile uint16_t io::USART0::m_numberBytesSent = 0;
00023 volatile uint8_t io::USART0::m_ready2Send = 1;
00024
00025
00026
00027
00028 io::USART0& io::USART0::getInstance(const transmissionMode& ar_transMode,
00029                                     const communicationMode& ar_comMode,
00030                                     const frameSize& ar_frameSize,
00031                                     const stopBit& ar_stopBit,
00032                                     const parityMode& ar_parityMode)
00033 {
00034     static USART0 l_instance(ar_transMode,
00035                              ar_comMode,
00036                              ar_frameSize,
00037                              ar_stopBit,
00038                              ar_parityMode);
00039
00040     return l_instance;
00041 }
00042
00043
00044
00045 io::USART0::USART0(const transmissionMode& ar_transMode,
00046                    const communicationMode& ar_comMode,
00047                    const frameSize& ar_frameSize,
00048                    const stopBit& ar_stopBit,
00049                    const parityMode& ar_parityMode)
00050
00051 {
00052     core::MCU::enableUSART0(1);
00053     setBaudRate();
00054     setTransmissionMode(ar_transMode);
00055     setCommunicationMode(ar_comMode);
00056     setParityMode(ar_parityMode);
00057     setFrameSize(ar_frameSize);
00058     setStopBit(ar_stopBit);
00059     enableReceiveCompleteInterrupt(1);
00060 }
00061
00062 io::USART0::~USART0()
00063 {
00064
00065 }
00066
00067 void io::USART0::setBaudRate()
00068 {
00069     USART0_SET_BAUDRATE_HIGH_REGISTER;
00070     USART0_SET_BAUDRATE_LOW_REGISTER;
00071
00072 }
```

```
00073
00074 void io::USART0::setTransmissionMode(const transmissionMode& ar_transMode)
00075 {
00076     switch (ar_transMode)
00077     {
00078         case transmissionMode::async:
00079         {
00080             USART0_ENABLE_ASYNC_TRANSMISSION_MODE;
00081             break;
00082         }
00083         case transmissionMode::sync:
00084         {
00085             USART0_DISABLE_DOUBLE_SPEED_MODE;
00086             USART0_ENABLE_SYNC_TRANSMISSION_MODE;
00087             break;
00088         }
00089     case transmissionMode::masterSPI:
00090         {
00091             USART0_ENABLE_MASTER_SPI_MODE;
00092             break;
00093     }
00094     }
00095
00096
00097 }
00098
00099 void io::USART0::setCommunicationMode(const communicationMode &ar_comMode)
00100 {
00101     switch (ar_comMode)
00102     {
00103         case communicationMode::duplex:
00104         {
00105             USART0_ENABLE_TRANSMITTER;
00106             USART0_ENABLE_RECEIVER;
00107             break;
00108         }
00109         case communicationMode::receive:
00110         {
00111             USART0_ENABLE_RECEIVER;
00112             USART0_DISABLE_TRANSMITTER;
00113             break;
00114         }
00115         case communicationMode::transmit:
00116         {
00117             USART0_ENABLE_TRANSMITTER;
00118             USART0_DISABLE_RECEIVER;
00119             break;
00120         }
00121     }
00122
00123 }
00124
00125 void io::USART0::setParityMode(const parityMode& ar_parityMode)
00126 {
00127     switch (ar_parityMode)
00128     {
00129         case parityMode::noParity:
00130         {
00131             USART0_DISABLE_PARITY_MODE;
00132             break;
00133         }
00134         case parityMode::evenParity:
00135         {
00136             USART0_ENABLE_EVEN_PARITY_MODE;
00137             break;
00138         }
00139         case parityMode::oddParity:
00140         {
00141             USART0_ENABLE_ODD_PARITY_MODE;
00142             break;
00143         }
00144     }
00145
00146 }
00147
00148 void io::USART0::setFrameSize(const frameSize& ar_frameSize)
00149 {
00150     switch (ar_frameSize)
00151     {
00152         case frameSize::eightBits:
00153         {
00154             USART0_SET_8BIT_FRAME_SIZE;
00155             break;
00156         }
00157         case frameSize::sevenBits:
00158         {
00159             USART0_SET_7BIT_FRAME_SIZE;
```

```
00160             break;
00161         }
00162         case frameSize::sixBits:
00163         {
00164             USART0_SET_6BIT_FRAME_SIZE;
00165             break;
00166         }
00167         case frameSize::fiveBits:
00168         {
00169             USART0_SET_5BIT_FRAME_SIZE;
00170             break;
00171         }
00172         case frameSize::neineBits:
00173         {
00174             USART0_SET_9BIT_FRAME_SIZE;
00175             break;
00176         }
00177     }
00178
00179 }
00180 void io::USART0::setStopBit(const stopBit& ar_stopBit)
00181 {
00182     switch (ar_stopBit)
00183     {
00184         case stopBit::oneStopBit:
00185         {
00186             USART0_SET_ONE_STOP_BIT;
00187             break;
00188         }
00189         case stopBit::twoStopBits:
00190         {
00191             USART0_SET_TWO_STOP_BITS;
00192             break;
00193         }
00194     }
00195
00196 }
00197
00198
00199 uint8_t io::USART0::frameError()
00200 {
00201     return (m_status & (1 « USART0_FRAME_ERROR));
00202
00203 }
00204
00205 uint8_t io::USART0::dataOverrun()
00206 {
00207     return (m_status & (1 « USART0_DATA_OVERRUN));
00208
00209 }
00210
00211 uint8_t io::USART0::parityError()
00212 {
00213     return (m_status & (1 « USART0_PARITY_ERROR));
00214
00215 }
00216
00217 void io::USART0::sendFrame(const uint8_t* ap_dataBuffer, const uint8_t a_size)
00218 {
00219     while (!ready2Send()){};
00220     // claimed here, before the interrupt is enabled, so a second send cannot
00221     // observe a ready transmitter while this one is still running
00222     m_numberBytesSent = 0;
00223     m_sizeData2Send = a_size;
00224     mp_data2Send = ap_dataBuffer;
00225     m_ready2Send = 0;
00226     enableDataRegisterEmptyInterrupt(1);
00227 }
00228
00229
00230 void io::USART0::sendString(const char* ap_string)
00231 {
00232     while (!ready2Send()){};
00233     m_numberBytesSent = 0;
00234     m_sizeData2Send = strlen(ap_string);
00235     mp_data2Send = reinterpret_cast<const uint8_t*>(ap_string);
00236     m_ready2Send = 0;
00237     enableDataRegisterEmptyInterrupt(1);
00238
00239 }
00240
00241
00242 void io::USART0::sendByte(const uint8_t &ar_byte)
00243 {
00244     static uint8_t l_byte2Send[3];
00245     l_byte2Send[0] = '0' + (ar_byte / 100);
00246     l_byte2Send[1] = '0' + ((ar_byte / 10) % 10);
```

```
00247     l_byte2Send[2] = '0' + (ar_byte % 10);
00248     sendFrame(l_byte2Send, 3);
00249
00250 }
00251 void io::USART0::sendLong(const uint32_t &ar_long)
00252 {
00253     static uint8_t l_word2Send[10];
00254     l_word2Send[0] = '0' + (ar_long / 1000000000);
00255     l_word2Send[1] = '0' + ((ar_long / 100000000) % 10);
00256     l_word2Send[2] = '0' + ((ar_long /  10000000) % 10);
00257     l_word2Send[3] = '0' + ((ar_long /   1000000) % 10);
00258     l_word2Send[4] = '0' + ((ar_long /    100000) % 10);
00259     l_word2Send[5] = '0' + ((ar_long /     10000) % 10);
00260     l_word2Send[6] = '0' + ((ar_long /      1000) % 10);
00261     l_word2Send[7] = '0' + ((ar_long /       100) % 10);
00262     l_word2Send[8] = '0' + ((ar_long /        10) % 10);
00263     l_word2Send[9] = '0' + (ar_long % 10);
00264     sendFrame(l_word2Send, 10);
00265
00266 }
00267 void io::USART0::sendWord(const uint16_t &ar_word)
00268 {
00269     static uint8_t l_word2Send[5];
00270     l_word2Send[0] = '0' + (ar_word / 10000);
00271     l_word2Send[1] = '0' + ((ar_word / 1000) % 10);
00272     l_word2Send[2] = '0' + ((ar_word /  100) % 10);
00273     l_word2Send[3] = '0' + ((ar_word /   10) % 10);
00274     l_word2Send[4] = '0' + (ar_word %    10);
00275     sendFrame(l_word2Send, 5);
00276
00277 }
00278
00279 void io::USART0::sendChar(const uint8_t &ar_char)
00280 {
00281     // copied into internal storage: ar_char does not have to outlive the call
00282     static uint8_t l_char2Send = 0;
00283     l_char2Send = ar_char;
00284     sendFrame(&l_char2Send, 1);
00285
00286 }
00287
00288 void io::USART0::receiveChar(uint8_t &ar_char)
00289 {
00290     receiveFrame(&ar_char, 1);
00291 }
00292
00293
00294 void io::USART0::receiveFrame(uint8_t *ap_dataBuffer, const uint8_t a_size)
00295 {
00296     // arming is shared with the receive ISR, which runs at any priority
00297     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00298     {
00299         mp_data2Receive = ap_dataBuffer;
00300         m_sizeData2Receive = a_size;
00301         m_stringReceive = 0;
00302     }
00303 }
00304
00305 void io::USART0::receiveString(const char *ap_string)
00306 {
00307     // the const signature is kept for compatibility, but the receive ISR
00308     // stores into the buffer, so ap_string must be modifiable storage
00309     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00310     {
00311         mp_data2Receive = const_cast<uint8_t*>(reinterpret_cast<const uint8_t*>(ap_string));
00312         // no length parameter: bound the write by the receive counter and
00313         // stop after a NUL byte
00314         m_sizeData2Receive = 255;
00315         m_stringReceive = 1;
00316     }
00317 }
00318
00319
00320 void io::USART0::receiveCompleteServiceRoutine()
00321 {
00322     m_status = USART0_CONTROL_STATUS_REGISTER;
00323
00324     // always read UDR0 first, even with no buffer armed, so the hardware
00325     // receive register is drained and RXC is cleared
00326     const uint8_t l_data = USART0_DATA_REGISTER;
00327
00328     if ((mp_data2Receive != nullptr) && (m_sizeData2Receive != 0))
00329     {
00330         *mp_data2Receive++ = l_data;
00331         m_sizeData2Receive--;
00332         m_numberBytesReceived++;
00333
```

```
00334         if ((m_stringReceive != 0) && (l_data == 0))
00335         {
00336             // NUL terminator stored: string reception complete
00337             m_stringReceive = 0;
00338             m_sizeData2Receive = 0;
00339         }
00340     }
00341
00342 }
00343
00344 void io::USART0::dataRegisterEmptyServiceRoutine()
00345 {
00346
00347     if (m_sizeData2Send)
00348     {
00349         USART0_DATA_REGISTER = *mp_data2Send++;
00350         m_sizeData2Send--;
00351         m_numberBytesSent++;
00352
00353     }
00354     else
00355     {
00356         enableDataRegisterEmptyInterrupt(0);
00357         m_ready2Send = 1;
00358
00359     }
00360
00361 }
00362
00363
00364 void io::USART0::enableTransmitCompleteInterrupt(const uint8_t a_enable)
00365 {
00366     if (a_enable) {
00367         USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT;
00368     } else {
00369         USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT;
00370     }
00371
00372 }
00373
00374 void io::USART0::enableReceiveCompleteInterrupt(const uint8_t a_enable)
00375 {
00376     if (a_enable) {
00377         USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT;
00378
00379     } else {
00380         USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT;
00381     }
00382
00383 }
00384
00385 void io::USART0::enableDataRegisterEmptyInterrupt(const uint8_t a_enable)
00386 {
00387     if (a_enable) {
00388         USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT;
00389
00390     } else {
00391         USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT;
00392     }
00393
00394 }
00395
00396 void io::USART0::transmitCompleteServiceRoutine()
00397 {
00398
00399 }
00400
00401 uint16_t io::USART0::getNumberBytesSent()
00402 {
00403     return m_numberBytesSent;
00404 }
00405
00406 uint16_t io::USART0::getNumberBytesReceived()
00407 {
00408     return m_numberBytesReceived;
00409 }
00410
00411 void io::USART0::resetNumberBytesReceived()
00412 {
00413     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
00414     {
00415         m_numberBytesReceived = 0;
00416     }
00417 }
00418
00419 uint8_t io::USART0::ready2Send()
00420 {
```

```
00421     return m_ready2Send;
00422
00423 }
00424
```

9.129 USART0.h File Reference

```
#include "ha_base.h"
```

Classes

- class io::USART0

  Class for handling USART0.

Namespaces

- namespace io

  AVR chip internal i/o components.

Enumerations

- enum class io::transmissionMode : uint8_t { io::async =0 , io::sync , io::masterSPI }
- enum class io::communicationMode : uint8_t { io::duplex =0 , io::transmit , io::receive }
- enum class io::parityMode : uint8_t { io::noParity =0 , io::evenParity , io::oddParity }
- enum class io::frameSize : uint8_t {
  io::eightBits =0 , io::fiveBits , io::sixBits , io::sevenBits ,
  io::neineBits }
- enum class io::stopBit : uint8_t { io::oneStopBit =0 , io::twoStopBits }

9.130 USART0.h

Go to the documentation of this file.

```
00001 #ifndef USART_H
00002 #define USART_H
00003 #include "ha_base.h"
00004
00013 namespace io
00014 {
00015
00016
00025 enum class transmissionMode : uint8_t {
00026     async=0,
00027     sync,
00028     masterSPI
00029 };
00030
00039 enum class communicationMode : uint8_t {
00040     duplex=0,
00041     transmit,
00042     receive,
00043
00044 };
00045
00054 enum class parityMode : uint8_t {
00055     noParity=0,
00056     evenParity,
00057     oddParity
```

```
00058 };
00059
00068 enum class frameSize : uint8_t {
00069     eightBits=0,
00070     fiveBits,
00071     sixBits,
00072     sevenBits,
00073     neineBits
00074 };
00075
00084 enum class stopBit : uint8_t{
00085     oneStopBit=0,
00086     twoStopBits
00087 };
00088
00097
00098 class USART0
00099 {
00100
00101 public:
00102
00103
00116     static USART0& getInstance(const transmissionMode& ar_transMode = transmissionMode::async,
00117                                const communicationMode& ar_comMode = communicationMode::duplex,
00118                                const frameSize& ar_frameSize = frameSize::eightBits,
00119                                const stopBit& ar_stopBit = stopBit::oneStopBit,
00120                                const parityMode& ar_parityMode = parityMode::noParity);
00125     void setBaudRate();
00131     void setTransmissionMode(const transmissionMode& ar_transMode);
00137     void setCommunicationMode(const communicationMode& ar_comMode);
00143     void setParityMode(const parityMode& ar_parityMode);
00149     void setFrameSize(const frameSize& ar_frameSize);
00155     void setStopBit(const stopBit& ar_stopBit);
00170     void sendFrame(const uint8_t *ap_dataBuffer, const uint8_t a_size);
00171
00180     void sendString(const char *ap_string);
00181
00195     void receiveString(const char *ap_string);
00196
00205     void sendChar(const uint8_t &ar_char);
00206
00215     void sendByte(const uint8_t &ar_byte);
00216
00225     void sendWord(const uint16_t &ar_word);
00226
00235     void sendLong(const uint32_t &ar_long);
00236
00249     void receiveChar(uint8_t &ar_char);
00250
00264     void receiveFrame(uint8_t *ap_dataBuffer, const uint8_t a_size);
00265
00271     void enableTransmitCompleteInterrupt(const uint8_t a_enable);
00272
00278     void enableReceiveCompleteInterrupt(const uint8_t a_enable);
00279
00285     static void enableDataRegisterEmptyInterrupt(const uint8_t a_enable);
00286
00292     uint8_t frameError();
00293
00299     uint8_t dataOverrun();
00300
00306     uint8_t parityError();
00316     uint16_t getNumberBytesReceived();
00325     uint16_t getNumberBytesSent();
00331     uint8_t ready2Send();
00334     void resetNumberBytesReceived();
00337     static void receiveCompleteServiceRoutine() __asm__(STR(USART0_RECEIVE_COMPLETE_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__));
00340     static void dataRegisterEmptyServiceRoutine() __asm__(STR(USART0_DATA_REGISTER_EMPTY_INTERRUPT))
    __attribute__((__signal__,__used__, __externally_visible__));
00343     static void transmitCompleteServiceRoutine() __asm__(STR(USART0_TRANSMIT_COMPLETE_INTERRUPT))
    __attribute__((__signal__, __used__, __externally_visible__));
00344
00345
00346 protected:
00347
00348
00349
00350
00351 private:
00362     USART0(const transmissionMode& ar_transMode,
00363            const communicationMode& ar_comMode,
00364            const frameSize& ar_frameSize,
00365            const stopBit& ar_stopBit,
00366            const parityMode& ar_parityMode);
00367
00370     ~USART0();
```

```
00371
00375     USART0(const USART0&);
00376
00380     const USART0& operator=(const USART0&);
00381
00382     static volatile uint8_t m_status;
00383
00384     static const uint8_t *mp_data2Send;
00385
00386     static uint8_t *mp_data2Receive;
00387
00388     static uint16_t m_sizeData2Send;
00389
00390     static volatile uint16_t m_sizeData2Receive;
00391
00392     static volatile uint8_t m_stringReceive;
00393
00394     static volatile uint16_t m_numberBytesReceived;
00395
00396     static volatile uint16_t m_numberBytesSent;
00397
00398     static volatile uint8_t m_ready2Send;
00399
00400
00401 };
00402
00403
00404
00405 }
00406
00407
00408
00409
00410
00411
00412
00413
00414
00415
00416
00417 #endif
```

9.131 buzzer_pitches_16bit.h File Reference

Macros

- #define C0 25000
- #define Cx0 23597
- #define D0 22272
- #define Dx0 21022
- #define E0 19843
- #define F0 18729
- #define Fx0 17678
- #define G0 16685
- #define Gx0 15749
- #define A0 14865
- #define Ax0 14031
- #define B0 13243
- #define C1 12500
- #define Cx1 11798
- #define D1 11136
- #define Dx1 10511
- #define E1 9921
- #define F1 9364
- #define Fx1 8839
- #define G1 8343
- #define Gx1 7875

- #define A1 7433
- #define Ax1 7015
- #define B1 6622
- #define C2 6250
- #define Cx2 5899
- #define D2 5568
- #define Dx2 5256
- #define E2 4961
- #define F2 4682
- #define Fx2 4419
- #define G2 4171
- #define Gx2 3937
- #define A2 3716
- #define Ax2 3508
- #define B2 3311
- #define C3 3125
- #define Cx3 2950
- #define D3 2784
- #define Dx3 2628
- #define E3 2480
- #define F3 2341
- #define Fx3 2210
- #define G3 2086
- #define Gx3 1969
- #define A3 1858
- #define Ax3 1754
- #define B3 1655
- #define C4 1562
- #define Cx4 1474
- #define D4 1392
- #define Dx4 1313
- #define E4 1240
- #define F4 1170
- #define Fx4 1105
- #define G4 1043
- #define Gx4 984
- #define A4 929
- #define Ax4 877
- #define B4 827
- #define C5 781
- #define Cx5 737
- #define D5 696
- #define Dx5 657
- #define E5 620
- #define F5 585
- #define Fx5 552
- #define G5 521
- #define Gx5 492
- #define A5 464
- #define Ax5 438
- #define B5 414
- #define C6 390
- #define Cx6 368
- #define D6 347
- #define Dx6 328

- #define E6 310
- #define F6 292
- #define Fx6 276
- #define G6 260
- #define Gx6 246
- #define A6 232
- #define Ax6 219
- #define B6 207
- #define C7 195
- #define Cx7 184
- #define D7 174
- #define Dx7 164
- #define E7 155
- #define F7 146
- #define Fx7 138
- #define G7 130
- #define Gx7 123
- #define A7 116
- #define Ax7 109
- #define B7 103

9.131.1 Macro Definition Documentation

9.131.1.1 A0

```
#define A0 14865
```

Definition at line 21 of file buzzer_pitches_16bit.h.

9.131.1.2 A1

```
#define A1 7433
```

Definition at line 33 of file buzzer_pitches_16bit.h.

9.131.1.3 A2

```
#define A2 3716
```

Definition at line 45 of file buzzer_pitches_16bit.h.

9.131.1.4 A3

```
#define A3 1858
```

Definition at line 57 of file buzzer_pitches_16bit.h.

9.131.1.5 A4

```
#define A4 929
```

Definition at line 69 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.6 A5

```
#define A5 464
```

Definition at line 81 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.7 A6

```
#define A6 232
```

Definition at line 93 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.8 A7

```
#define A7 116
```

Definition at line 105 of file buzzer_pitches_16bit.h.

9.131.1.9 Ax0

```
#define Ax0 14031
```

Definition at line 22 of file buzzer_pitches_16bit.h.

9.131.1.10 Ax1

```
#define Ax1 7015
```

Definition at line 34 of file buzzer_pitches_16bit.h.

9.131.1.11 Ax2

```
#define Ax2 3508
```

Definition at line 46 of file buzzer_pitches_16bit.h.

9.131.1.12 Ax3

```
#define Ax3 1754
```

Definition at line 58 of file buzzer_pitches_16bit.h.

9.131.1.13 Ax4

```
#define Ax4 877
```

Definition at line 70 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.14 Ax5

```
#define Ax5 438
```

Definition at line 82 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.15 Ax6

```
#define Ax6 219
```

Definition at line 94 of file buzzer_pitches_16bit.h.

9.131.1.16 Ax7

```
#define Ax7 109
```

Definition at line 106 of file buzzer_pitches_16bit.h.

9.131.1.17 B0

```
#define B0 13243
```

Definition at line 23 of file buzzer_pitches_16bit.h.

9.131.1.18 B1

```
#define B1 6622
```

Definition at line 35 of file buzzer_pitches_16bit.h.

9.131.1.19 B2

```
#define B2 3311
```

Definition at line 47 of file buzzer_pitches_16bit.h.

9.131.1.20 B3

```
#define B3 1655
```

Definition at line 59 of file buzzer_pitches_16bit.h.

9.131.1.21 B4

```
#define B4 827
```

Definition at line 71 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.22 B5

```
#define B5 414
```

Definition at line 83 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.23 B6

```
#define B6 207
```

Definition at line 95 of file buzzer_pitches_16bit.h.

9.131.1.24 B7

```
#define B7 103
```

Definition at line 107 of file buzzer_pitches_16bit.h.

9.131.1.25 C0

```
#define C0 25000
```

Definition at line 12 of file buzzer_pitches_16bit.h.

9.131.1.26 C1

```
#define C1 12500
```

Definition at line 24 of file buzzer_pitches_16bit.h.

9.131.1.27 C2

```
#define C2 6250
```

Definition at line 36 of file buzzer_pitches_16bit.h.

9.131.1.28 C3

```
#define C3 3125
```

Definition at line 48 of file buzzer_pitches_16bit.h.

9.131.1.29 C4

```
#define C4 1562
```

Definition at line 60 of file buzzer_pitches_16bit.h.

9.131.1.30 C5

```
#define C5 781
```

Definition at line 72 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.31 C6

```
#define C6 390
```

Definition at line 84 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.32 C7

```
#define C7 195
```

Definition at line 96 of file buzzer_pitches_16bit.h.

9.131.1.33 Cx0

```
#define Cx0 23597
```

Definition at line 13 of file buzzer_pitches_16bit.h.

9.131.1.34 Cx1

```
#define Cx1 11798
```

Definition at line 25 of file buzzer_pitches_16bit.h.

9.131.1.35 Cx2

```
#define Cx2 5899
```

Definition at line 37 of file buzzer_pitches_16bit.h.

9.131.1.36 Cx3

```
#define Cx3 2950
```

Definition at line 49 of file buzzer_pitches_16bit.h.

9.131.1.37 Cx4

```
#define Cx4 1474
```

Definition at line 61 of file buzzer_pitches_16bit.h.

9.131.1.38 Cx5

```
#define Cx5 737
```

Definition at line 73 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.39 Cx6

```
#define Cx6 368
```

Definition at line 85 of file buzzer_pitches_16bit.h.

9.131.1.40 Cx7

```
#define Cx7 184
```

Definition at line 97 of file buzzer_pitches_16bit.h.

9.131.1.41 D0

```
#define D0 22272
```

Definition at line 14 of file buzzer_pitches_16bit.h.

9.131.1.42 D1

```
#define D1 11136
```

Definition at line 26 of file buzzer_pitches_16bit.h.

9.131.1.43 D2

```
#define D2 5568
```

Definition at line 38 of file buzzer_pitches_16bit.h.

9.131.1.44 D3

```
#define D3 2784
```

Definition at line 50 of file buzzer_pitches_16bit.h.

9.131.1.45 D4

```
#define D4 1392
```

Definition at line 62 of file buzzer_pitches_16bit.h.

9.131.1.46 D5

```
#define D5 696
```

Definition at line 74 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.47 D6

```
#define D6 347
```

Definition at line 86 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.48 D7

```
#define D7 174
```

Definition at line 98 of file buzzer_pitches_16bit.h.

9.131.1.49 Dx0

```
#define Dx0 21022
```

Definition at line 15 of file buzzer_pitches_16bit.h.

9.131.1.50 Dx1

```
#define Dx1 10511
```

Definition at line 27 of file buzzer_pitches_16bit.h.

9.131.1.51 Dx2

```
#define Dx2 5256
```

Definition at line 39 of file buzzer_pitches_16bit.h.

9.131.1.52 Dx3

```
#define Dx3 2628
```

Definition at line 51 of file buzzer_pitches_16bit.h.

9.131.1.53 Dx4

```
#define Dx4 1313
```

Definition at line 63 of file buzzer_pitches_16bit.h.

9.131.1.54 Dx5

```
#define Dx5 657
```

Definition at line 75 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.55 Dx6

```
#define Dx6 328
```

Definition at line 87 of file buzzer_pitches_16bit.h.

9.131.1.56 Dx7

```
#define Dx7 164
```

Definition at line 99 of file buzzer_pitches_16bit.h.

9.131.1.57 E0

```
#define E0 19843
```

Definition at line 16 of file buzzer_pitches_16bit.h.

9.131.1.58 E1

```
#define E1 9921
```

Definition at line 28 of file buzzer_pitches_16bit.h.

9.131.1.59 E2

```
#define E2 4961
```

Definition at line 40 of file buzzer_pitches_16bit.h.

9.131.1.60 E3

```
#define E3 2480
```

Definition at line 52 of file buzzer_pitches_16bit.h.

9.131.1.61 E4

```
#define E4 1240
```

Definition at line 64 of file buzzer_pitches_16bit.h.

9.131.1.62 E5

```
#define E5 620
```

Definition at line 76 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.63 E6

```
#define E6 310
```

Definition at line 88 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.64 E7

```
#define E7 155
```

Definition at line 100 of file buzzer_pitches_16bit.h.

9.131.1.65 F0

```
#define F0 18729
```

Definition at line 17 of file buzzer_pitches_16bit.h.

9.131.1.66 F1

```
#define F1 9364
```

Definition at line 29 of file buzzer_pitches_16bit.h.

9.131.1.67 F2

```
#define F2 4682
```

Definition at line 41 of file buzzer_pitches_16bit.h.

9.131.1.68 F3

```
#define F3 2341
```

Definition at line 53 of file buzzer_pitches_16bit.h.

9.131.1.69 F4

```
#define F4 1170
```

Definition at line 65 of file buzzer_pitches_16bit.h.

9.131.1.70 F5

```
#define F5 585
```

Definition at line 77 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.71 F6

```
#define F6 292
```

Definition at line 89 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.72 F7

```
#define F7 146
```

Definition at line 101 of file buzzer_pitches_16bit.h.

9.131.1.73 Fx0

```
#define Fx0 17678
```

Definition at line 18 of file buzzer_pitches_16bit.h.

9.131.1.74 Fx1

```
#define Fx1 8839
```

Definition at line 30 of file buzzer_pitches_16bit.h.

9.131.1.75 Fx2

```
#define Fx2 4419
```

Definition at line 42 of file buzzer_pitches_16bit.h.

9.131.1.76 Fx3

```
#define Fx3 2210
```

Definition at line 54 of file buzzer_pitches_16bit.h.

9.131.1.77 Fx4

```
#define Fx4 1105
```

Definition at line 66 of file buzzer_pitches_16bit.h.

9.131.1.78 Fx5

```
#define Fx5 552
```

Definition at line 78 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.79 Fx6

```
#define Fx6 276
```

Definition at line 90 of file buzzer_pitches_16bit.h.

9.131.1.80 Fx7

```
#define Fx7 138
```

Definition at line 102 of file buzzer_pitches_16bit.h.

9.131.1.81 G0

```
#define G0 16685
```

Definition at line 19 of file buzzer_pitches_16bit.h.

9.131.1.82 G1

```
#define G1 8343
```

Definition at line 31 of file buzzer_pitches_16bit.h.

9.131.1.83 G2

```
#define G2 4171
```

Definition at line 43 of file buzzer_pitches_16bit.h.

9.131.1.84 G3

```
#define G3 2086
```

Definition at line 55 of file buzzer_pitches_16bit.h.

9.131.1.85 G4

```
#define G4 1043
```

Definition at line 67 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.86 G5

```
#define G5 521
```

Definition at line 79 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.87 G6

```
#define G6 260
```

Definition at line 91 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.88 G7

```
#define G7 130
```

Definition at line 103 of file buzzer_pitches_16bit.h.

9.131.1.89 Gx0

```
#define Gx0 15749
```

Definition at line 20 of file buzzer_pitches_16bit.h.

9.131.1.90 Gx1

```
#define Gx1 7875
```

Definition at line 32 of file buzzer_pitches_16bit.h.

9.131.1.91 Gx2

```
#define Gx2 3937
```

Definition at line 44 of file buzzer_pitches_16bit.h.

9.131.1.92 Gx3

```
#define Gx3 1969
```

Definition at line 56 of file buzzer_pitches_16bit.h.

9.131.1.93 Gx4

```
#define Gx4 984
```

Definition at line 68 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.94 Gx5

```
#define Gx5 492
```

Definition at line 80 of file buzzer_pitches_16bit.h.

Referenced by main().

9.131.1.95 Gx6

```
#define Gx6 246
```

Definition at line 92 of file buzzer_pitches_16bit.h.

9.131.1.96 Gx7

```
#define Gx7 123
```

Definition at line 104 of file buzzer_pitches_16bit.h.

9.132 buzzer_pitches_16bit.h

Go to the documentation of this file.

```
00001 /* Scale in the key of 1/25000 */
00002 /*
00003    These are 1/2 periods (us) -- if you delay this long,
00004    then toggle the speaker pin, you'll get approximate
00005    pitches.
00006
00007    This is the 16-bit version.  The pitches get less accurate
00008    as they get higher.
00009
00010 */
00011
00012 #define  C0    25000 // freq = 1000000/ (2*25000) = 20hz
00013 #define  Cx0   23597
00014 #define  D0    22272
00015 #define  Dx0   21022
00016 #define  E0    19843
00017 #define  F0    18729
00018 #define  Fx0   17678
00019 #define  G0    16685
00020 #define  Gx0   15749
00021 #define  A0    14865
00022 #define  Ax0   14031
00023 #define  B0    13243
00024 #define  C1    12500
00025 #define  Cx1   11798
00026 #define  D1    11136
00027 #define  Dx1   10511
00028 #define  E1     9921
00029 #define  F1     9364
00030 #define  Fx1    8839
00031 #define  G1     8343
00032 #define  Gx1    7875
00033 #define  A1     7433
00034 #define  Ax1    7015
00035 #define  B1     6622
00036 #define  C2     6250
00037 #define  Cx2    5899
00038 #define  D2     5568
00039 #define  Dx2    5256
00040 #define  E2     4961
00041 #define  F2     4682
00042 #define  Fx2    4419
00043 #define  G2     4171
00044 #define  Gx2    3937
00045 #define  A2     3716
00046 #define  Ax2    3508
00047 #define  B2     3311
00048 #define  C3     3125
00049 #define  Cx3    2950
00050 #define  D3     2784
00051 #define  Dx3    2628
00052 #define  E3     2480
00053 #define  F3     2341
00054 #define  Fx3    2210
00055 #define  G3     2086
00056 #define  Gx3    1969
00057 #define  A3     1858
00058 #define  Ax3    1754
00059 #define  B3     1655
00060 #define  C4     1562
00061 #define  Cx4    1474
00062 #define  D4     1392
00063 #define  Dx4    1313
00064 #define  E4     1240
00065 #define  F4     1170
00066 #define  Fx4    1105
00067 #define  G4     1043
00068 #define  Gx4     984
00069 #define  A4      929
```

```
00070 #define  Ax4     877
00071 #define  B4      827
00072 #define  C5      781
00073 #define  Cx5     737
00074 #define  D5      696
00075 #define  Dx5     657
00076 #define  E5      620
00077 #define  F5      585
00078 #define  Fx5     552
00079 #define  G5      521
00080 #define  Gx5     492
00081 #define  A5      464
00082 #define  Ax5     438
00083 #define  B5      414
00084 #define  C6      390
00085 #define  Cx6     368
00086 #define  D6      347
00087 #define  Dx6     328
00088 #define  E6      310
00089 #define  F6      292
00090 #define  Fx6     276
00091 #define  G6      260
00092 #define  Gx6     246
00093 #define  A6      232
00094 #define  Ax6     219
00095 #define  B6      207
00096 #define  C7      195
00097 #define  Cx7     184
00098 #define  D7      174
00099 #define  Dx7     164
00100 #define  E7      155
00101 #define  F7      146
00102 #define  Fx7     138
00103 #define  G7      130
00104 #define  Gx7     123
00105 #define  A7      116
00106 #define  Ax7     109
00107 #define  B7      103 // freq = 1000000/ (2*103) = 4.85 khz
```

9.133 buzzer_pitches_8bit.h File Reference

Macros

- #define Gx0 252
- #define A0 238
- #define Ax0 224
- #define B0 212
- #define C1 200
- #define Cx0 189
- #define D1 178
- #define Dx0 168
- #define E1 159
- #define F1 150
- #define Fx1 141
- #define G1 133
- #define Gx1 126
- #define A1 119
- #define Ax1 112
- #define B1 106
- #define C2 100
- #define Cx2 94
- #define D2 89
- #define Dx2 84
- #define E2 79
- #define F2 75
- #define Fx2 71

- #define G2 67
- #define Gx2 63
- #define A2 59
- #define Ax2 56
- #define B2 53
- #define C3 50
- #define Cx3 47
- #define D3 44
- #define Dx3 42
- #define E3 40
- #define F3 37
- #define Fx3 35
- #define G3 33

9.133.1 Macro Definition Documentation

9.133.1.1 A0

```
#define A0 238
```

Definition at line 14 of file buzzer_pitches_8bit.h.

9.133.1.2 A1

```
#define A1 119
```

Definition at line 26 of file buzzer_pitches_8bit.h.

9.133.1.3 A2

```
#define A2 59
```

Definition at line 38 of file buzzer_pitches_8bit.h.

9.133.1.4 Ax0

```
#define Ax0 224
```

Definition at line 15 of file buzzer_pitches_8bit.h.

9.133.1.5 Ax1

```
#define Ax1 112
```

Definition at line 27 of file buzzer_pitches_8bit.h.

9.133.1.6 Ax2

```
#define Ax2 56
```

Definition at line 39 of file buzzer_pitches_8bit.h.

9.133.1.7 B0

```
#define B0 212
```

Definition at line 16 of file buzzer_pitches_8bit.h.

9.133.1.8 B1

```
#define B1 106
```

Definition at line 28 of file buzzer_pitches_8bit.h.

9.133.1.9 B2

```
#define B2 53
```

Definition at line 40 of file buzzer_pitches_8bit.h.

9.133.1.10 C1

```
#define C1 200
```

Definition at line 17 of file buzzer_pitches_8bit.h.

9.133.1.11 C2

```
#define C2 100
```

Definition at line 29 of file buzzer_pitches_8bit.h.

9.133.1.12 C3

```
#define C3 50
```

Definition at line 41 of file buzzer_pitches_8bit.h.

9.133.1.13 Cx0

```
#define Cx0 189
```

Definition at line 18 of file buzzer_pitches_8bit.h.

9.133.1.14 Cx2

```
#define Cx2 94
```

Definition at line 30 of file buzzer_pitches_8bit.h.

9.133.1.15 Cx3

```
#define Cx3 47
```

Definition at line 42 of file buzzer_pitches_8bit.h.

9.133.1.16 D1

```
#define D1 178
```

Definition at line 19 of file buzzer_pitches_8bit.h.

9.133.1.17 D2

```
#define D2 89
```

Definition at line 31 of file buzzer_pitches_8bit.h.

9.133.1.18 D3

```
#define D3 44
```

Definition at line 43 of file buzzer_pitches_8bit.h.

9.133.1.19 Dx0

```
#define Dx0 168
```

Definition at line 20 of file buzzer_pitches_8bit.h.

9.133.1.20 Dx2

```
#define Dx2 84
```

Definition at line 32 of file buzzer_pitches_8bit.h.

9.133.1.21 Dx3

```
#define Dx3 42
```

Definition at line 44 of file buzzer_pitches_8bit.h.

9.133.1.22 E1

```
#define E1 159
```

Definition at line 21 of file buzzer_pitches_8bit.h.

9.133.1.23 E2

```
#define E2 79
```

Definition at line 33 of file buzzer_pitches_8bit.h.

9.133.1.24 E3

```
#define E3 40
```

Definition at line 45 of file buzzer_pitches_8bit.h.

9.133.1.25 F1

```
#define F1 150
```

Definition at line 22 of file buzzer_pitches_8bit.h.

9.133.1.26 F2

```
#define F2 75
```

Definition at line 34 of file buzzer_pitches_8bit.h.

9.133.1.27 F3

```
#define F3 37
```

Definition at line 46 of file buzzer_pitches_8bit.h.

9.133.1.28 Fx1

```
#define Fx1 141
```

Definition at line 23 of file buzzer_pitches_8bit.h.

9.133.1.29 Fx2

```
#define Fx2 71
```

Definition at line 35 of file buzzer_pitches_8bit.h.

9.133.1.30 Fx3

```
#define Fx3 35
```

Definition at line 47 of file buzzer_pitches_8bit.h.

9.133.1.31 G1

```
#define G1 133
```

Definition at line 24 of file buzzer_pitches_8bit.h.

9.133.1.32 G2

```
#define G2 67
```

Definition at line 36 of file buzzer_pitches_8bit.h.

9.133.1.33 G3

```
#define G3 33
```

Definition at line 48 of file buzzer_pitches_8bit.h.

9.133.1.34 Gx0

```
#define Gx0 252
```

Definition at line 13 of file buzzer_pitches_8bit.h.

9.133.1.35 Gx1

```
#define Gx1 126
```

Definition at line 25 of file buzzer_pitches_8bit.h.

9.133.1.36 Gx2

```
#define Gx2 63
```

Definition at line 37 of file buzzer_pitches_8bit.h.

9.134 buzzer_pitches_8bit.h

Go to the documentation of this file.

```
00001 /* Scale in the key of 1/200 */
00002
00003 /*
00004    These are 1/2 periods (us) -- if you delay this long,
00005    then toggle the speaker pin, you'll get approximate
00006    pitches.
00007
00008    This is the 8-bit version.  The pitches get less accurate
00009    as they get higher.
00010
00011 */
00012
00013 #define Gx0 252 // freq = 1000000/ (2*252) = 1.98 khz
00014 #define A0  238
00015 #define Ax0 224
00016 #define B0  212
00017 #define C1  200
00018 #define Cx0 189
00019 #define D1  178
00020 #define Dx0 168
00021 #define E1  159
00022 #define F1  150
00023 #define Fx1 141
00024 #define G1  133
00025 #define Gx1 126
00026 #define A1  119
00027 #define Ax1 112
00028 #define B1  106
00029 #define C2  100
00030 #define Cx2 94
00031 #define D2  89
00032 #define Dx2 84
00033 #define E2  79
00034 #define F2  75
00035 #define Fx2 71
00036 #define G2  67
00037 #define Gx2 63
00038 #define A2  59
00039 #define Ax2 56
00040 #define B2  53
00041 #define C3  50
00042 #define Cx3 47
00043 #define D3  44
00044 #define Dx3 42
00045 #define E3  40
00046 #define F3  37
00047 #define Fx3 35
00048 #define G3  33 // freq = 1000000/ (2*33) = 15.15 khz
```

9.135 ha_attiny85.h File Reference

Header file for the hardware abstraction macros of ATtiny85.

Macros

- #define PUSHBUTTON_DEBOUNCE_TIME_US 1000
- #define ADC_SELECT_REF_VOLTAGE(refVoltage)
- #define ADC_ADJUST_RESULT_LEFT ADMUX |= 1 << ADLAR
- #define ADC_ADJUST_RESULT_RIGHT ADMUX &= ~(1 << ADLAR)
- #define ADC_SELECT_ANALOG_INPUT(pinNumber)
- #define ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber)
- #define ADC_ENABLE ADCSRA |= 1 << ADEN
- #define ADC_DISABLE ADCSRA &= ~(1 << ADEN)
- #define ADC_START_CONVERSION ADCSRA |= 1 << ADSC
- #define ADC_STOP_CONVERSION ADCSRA &= ~(1 << ADSC)
- #define ADC_ENABLE_AUTOTRIGGER ADCSRA |= 1 << ADATE

- #define ADC_DISABLE_AUTOTRIGGER ADCSRA &= ~(1 << ADATE)
- #define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA |= 1 << ADIE
- #define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA &= ~(1 << ADIE)
- #define ADC_SELECT_CLOCK_PRESCALER(clockPrescaler)
- #define ADC_CONVERSION_COMPLETE_INTERRUPT ADC_vect
- #define WATCHDOG_SELECT_TIMEOUT(timeOut)
- #define WATCHDOG_START(operationMode, timeOut)
- #define WATCHDOG_STOP
- #define WATCHDOG_TIMEOUT_INTERRUPT WDT_vect
- #define MCU_SELECT_SLEEP_MODE(sleepMode)
- #define MCU_SLEEP_ENABLE MCUCR |= 1 << SE
- #define MCU_SLEEP_DISABLE MCUCR &= ~(1 << SE)

9.135.1 Detailed Description

Header file for the hardware abstraction macros of ATtiny85.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file ha_attiny85.h.

9.135.2 Macro Definition Documentation

9.135.2.1 ADC_ADJUST_RESULT_LEFT

```
#define ADC_ADJUST_RESULT_LEFT ADMUX |= 1 << ADLAR
```

Definition at line 25 of file ha_attiny85.h.

Referenced by core::ADConverter::getConversionResult().

9.135.2.2 ADC_ADJUST_RESULT_RIGHT

```
#define ADC_ADJUST_RESULT_RIGHT ADMUX &= ~(1 << ADLAR)
```

Definition at line 26 of file ha_attiny85.h.

Referenced by core::ADConverter::getConversionResult().

9.135.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_CONVERSION_COMPLETE_INTERRUPT ADC_vect
```

Definition at line 57 of file ha_attiny85.h.

9.135.2.4 ADC_DISABLE

```
#define ADC_DISABLE ADCSRA &= ~(1 << ADEN)
```

Definition at line 40 of file ha_attiny85.h.

Referenced by core::ADConverter::stop().

9.135.2.5 ADC_DISABLE_AUTOTRIGGER

```
#define ADC_DISABLE_AUTOTRIGGER ADCSRA &= ~(1 << ADATE)
```

Definition at line 46 of file ha_attiny85.h.

Referenced by core::ADConverter::enableAutoTrigger().

9.135.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA &= ~(1 << ADIE)
```

Definition at line 49 of file ha_attiny85.h.

Referenced by core::ADConverter::enableConversionCompleteInterrupt().

9.135.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER

```
#define ADC_DISABLE_DIGITAL_INPUT_REGISTER(
              pinNumber)
```

Value:

```
    do { \
        DIDR0 |= (1 « (pinNumber)); \
    } while (0)
```

Definition at line 34 of file ha_attiny85.h.

```
00034 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) \
00035     do { \
00036         DIDR0 |= (1 « (pinNumber)); \
00037     } while (0)
```

Referenced by core::ADConverter::selectAnalogInput().

9.135.2.8 ADC_ENABLE

```
#define ADC_ENABLE ADCSRA |= 1 << ADEN
```

Definition at line 39 of file ha_attiny85.h.

Referenced by core::ADConverter::start().

9.135.2.9 ADC_ENABLE_AUTOTRIGGER

```
#define ADC_ENABLE_AUTOTRIGGER ADCSRA |= 1 << ADATE
```

Definition at line 45 of file ha_attiny85.h.

Referenced by core::ADConverter::enableAutoTrigger().

9.135.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA |= 1 << ADIE
```

Definition at line 48 of file ha_attiny85.h.

Referenced by core::ADConverter::enableConversionCompleteInterrupt().

9.135.2.11 ADC_SELECT_ANALOG_INPUT

```
#define ADC_SELECT_ANALOG_INPUT(
              pinNumber)
```

Value:

```
    do { \
        ADMUX &= 0xF0; \
        ADMUX |= ((pinNumber) & 0x0F); \
    } while (0)
```

Definition at line 28 of file ha_attiny85.h.

```
00028 #define   ADC_SELECT_ANALOG_INPUT(pinNumber) \
00029     do { \
00030         ADMUX &= 0xF0; \
00031         ADMUX |= ((pinNumber) & 0x0F); \
00032     } while (0)
```

Referenced by core::ADConverter::selectAnalogInput().

9.135.2.12 ADC_SELECT_CLOCK_PRESCALER

```
#define ADC_SELECT_CLOCK_PRESCALER(
              clockPrescaler)
```

Value:

```
    do { \
        ADCSRA &= 0xF8; \
        ADCSRA |= (clockPrescaler); \
    } while (0)
```

Definition at line 51 of file ha_attiny85.h.

```
00051 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00052     do { \
00053         ADCSRA &= 0xF8; \
00054         ADCSRA |= (clockPrescaler); \
00055     } while (0)
```

Referenced by core::ADConverter::selectClockPrescaler().

9.135.2.13 ADC_SELECT_REF_VOLTAGE

```
#define ADC_SELECT_REF_VOLTAGE(
                refVoltage)
```

Value:

```
    do { \
        ADMUX &= 0x3F; \
        ADMUX |= (refVoltage) « 6; \
    } while (0)
```

Definition at line 19 of file ha_attiny85.h.

```
00019 #define   ADC_SELECT_REF_VOLTAGE(refVoltage) \
00020     do { \
00021         ADMUX &= 0x3F; \
00022         ADMUX |= (refVoltage) « 6; \
00023     } while (0)
```

Referenced by core::ADConverter::selectReferenceVoltage().

9.135.2.14 ADC_START_CONVERSION

```
#define ADC_START_CONVERSION ADCSRA |= 1 << ADSC
```

Definition at line 42 of file ha_attiny85.h.

Referenced by core::ADConverter::start().

9.135.2.15 ADC_STOP_CONVERSION

```
#define ADC_STOP_CONVERSION ADCSRA &= ~(1 << ADSC)
```

Definition at line 43 of file ha_attiny85.h.

9.135.2.16 MCU_SELECT_SLEEP_MODE

```
#define MCU_SELECT_SLEEP_MODE(
                sleepMode)
```

Value:

```
    do { \
        MCUCR &= 0xCF; \
        MCUCR |= (sleepMode) « 4; \
    } while (0)
```

Definition at line 78 of file ha_attiny85.h.

```
00078 #define MCU_SELECT_SLEEP_MODE(sleepMode) \
00079     do { \
00080         MCUCR &= 0xCF; \
00081         MCUCR |= (sleepMode) « 4; \
00082     } while (0)
```

Referenced by core::MCU::selectSleepMode().

9.135.2.17 MCU_SLEEP_DISABLE

```
#define MCU_SLEEP_DISABLE MCUCR &= ~(1 << SE)
```

Definition at line 85 of file ha_attiny85.h.

Referenced by core::MCU::sleepEnable().

9.135.2.18 MCU_SLEEP_ENABLE

```
#define MCU_SLEEP_ENABLE MCUCR |= 1 << SE
```

Definition at line 84 of file ha_attiny85.h.

Referenced by core::MCU::sleepEnable().

9.135.2.19 PUSHBUTTON_DEBOUNCE_TIME_US

```
#define PUSHBUTTON_DEBOUNCE_TIME_US 1000
```

Definition at line 16 of file ha_attiny85.h.

Referenced by component::PushButton::isPressed().

9.135.2.20 WATCHDOG_SELECT_TIMEOUT

```
#define WATCHDOG_SELECT_TIMEOUT(
                timeOut)
```

Value:

```
    do { \
        WDTCR |= (1<<WDCE) | (1<<WDE); \
        WDTCR = (0<<WDIE) | (0<<WDE) | (timeOut); \
    } while (0)
```

Definition at line 60 of file ha_attiny85.h.

```
00060 #define WATCHDOG_SELECT_TIMEOUT(timeOut) \
00061     do { \
00062         WDTCR |= (1<<WDCE) | (1<<WDE); \
00063         WDTCR = (0<<WDIE) | (0<<WDE) | (timeOut); \
00064     } while (0)
```

Referenced by core::WatchdogTimer::selectTimeOut().

9.135.2.21 WATCHDOG_START

```
#define WATCHDOG_START(
                operationMode,
                timeOut)
```

Value:

```
    do { \
        WDTCR |= (1<<WDCE) | (1<<WDE); \
        WDTCR = (operationMode) | (timeOut); \
    } while (0)
```

Definition at line 65 of file ha_attiny85.h.

```
00065 #define WATCHDOG_START(operationMode,timeOut) \
00066     do { \
00067         WDTCR |= (1<<WDCE) | (1<<WDE); \
00068         WDTCR = (operationMode) | (timeOut); \
00069     } while (0)
```

Referenced by core::WatchdogTimer::start(), and core::WatchdogTimer::start().

9.135.2.22 WATCHDOG_STOP

```
#define WATCHDOG_STOP
```

Value:

```
    do { \
        WDTCR |= (1«WDCE) | (1«WDE); \
        WDTCR = 0x00; \
    } while (0)
```

Definition at line 70 of file ha_attiny85.h.

```
00070 #define WATCHDOG_STOP \
00071     do { \
00072         WDTCR |= (1«WDCE) | (1«WDE); \
00073         WDTCR = 0x00; \
00074     } while (0)
```

Referenced by core::WatchdogTimer::stop().

9.135.2.23 WATCHDOG_TIMEOUT_INTERRUPT

```
#define WATCHDOG_TIMEOUT_INTERRUPT WDT_vect
```

Definition at line 75 of file ha_attiny85.h.

9.136 ha_attiny85.h

Go to the documentation of this file.

```
00001
00008 #ifndef HA_ATTINY85_H
00009 #define HA_ATTINY85_H
00010
00011 #ifndef HABASE_H
00012     #error "don't use this file directly! Please include only ha_base.h!"
00013 #endif
00014
00015 // PushButton defines
00016 #define PUSHBUTTON_DEBOUNCE_TIME_US 1000
00017
00018 // ADC defines
00019 #define   ADC_SELECT_REF_VOLTAGE(refVoltage) \
00020     do { \
00021         ADMUX &= 0x3F; \
00022         ADMUX |= (refVoltage) « 6; \
00023     } while (0)
00024
00025 #define   ADC_ADJUST_RESULT_LEFT            ADMUX |=  1 « ADLAR
00026 #define   ADC_ADJUST_RESULT_RIGHT           ADMUX &= ~(1 « ADLAR)
00027
00028 #define   ADC_SELECT_ANALOG_INPUT(pinNumber) \
00029     do { \
00030         ADMUX &= 0xF0; \
00031         ADMUX |= ((pinNumber) & 0x0F); \
00032     } while (0)
00033
00034 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) \
00035     do { \
00036         DIDR0 |= (1 « (pinNumber)); \
00037     } while (0)
00038
00039 #define   ADC_ENABLE            ADCSRA |=  1 « ADEN
00040 #define   ADC_DISABLE           ADCSRA &= ~(1 « ADEN)
00041
00042 #define   ADC_START_CONVERSION            ADCSRA |=  1 « ADSC
00043 #define   ADC_STOP_CONVERSION           ADCSRA &= ~(1 « ADSC)
00044
00045 #define   ADC_ENABLE_AUTOTRIGGER          ADCSRA |=   1 « ADATE
00046 #define   ADC_DISABLE_AUTOTRIGGER       ADCSRA &= ~(1 « ADATE)
00047
```

```
00048 #define   ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT   ADCSRA |=   1 « ADIE
00049 #define   ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT  ADCSRA &= ~(1 « ADIE)
00050
00051 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00052     do { \
00053         ADCSRA &= 0xF8; \
00054         ADCSRA |= (clockPrescaler); \
00055     } while (0)
00056
00057 #define   ADC_CONVERSION_COMPLETE_INTERRUPT  ADC_vect
00058
00059 // Watchdog Timer defines
00060 #define WATCHDOG_SELECT_TIMEOUT(timeOut) \
00061     do { \
00062         WDTCR |= (1«WDCE) | (1«WDE); \
00063         WDTCR = (0«WDIE) | (0«WDE) | (timeOut); \
00064     } while (0)
00065 #define WATCHDOG_START(operationMode,timeOut) \
00066     do { \
00067         WDTCR |= (1«WDCE) | (1«WDE); \
00068         WDTCR = (operationMode) | (timeOut); \
00069     } while (0)
00070 #define WATCHDOG_STOP \
00071     do { \
00072         WDTCR |= (1«WDCE) | (1«WDE); \
00073         WDTCR = 0x00; \
00074     } while (0)
00075 #define WATCHDOG_TIMEOUT_INTERRUPT  WDT_vect
00076
00077 // MCU defines
00078 #define MCU_SELECT_SLEEP_MODE(sleepMode) \
00079     do { \
00080         MCUCR &= 0xCF; \
00081         MCUCR |= (sleepMode) « 4; \
00082     } while (0)
00083
00084 #define MCU_SLEEP_ENABLE MCUCR |=  1 « SE
00085 #define MCU_SLEEP_DISABLE MCUCR &= ~(1 « SE)
00086
00087 #endif
```

9.137 ha_base.h File Reference

Base header file for the basic hardware abstraction macros.

```
#include <stdint.h>
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <avr/sleep.h>
#include <avr/power.h>
#include <avr/wdt.h>
```

Macros

- #define STRx(s)
- #define STR(s)

9.137.1 Detailed Description

Base header file for the basic hardware abstraction macros.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file ha_base.h.

9.137.2 Macro Definition Documentation

9.137.2.1 STR

```
#define STR(
              s)
```

Value:

```
STRx(s)
```

Definition at line 16 of file ha_base.h.

9.137.2.2 STRx

```
#define STRx(
              s)
```

Value:

```
#s
```

Definition at line 15 of file ha_base.h.

9.138 ha_base.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef HABASE_H
00010 #define HABASE_H
00011
00012 /*
00013  *  Helper construct to get interrupt numbers from names
00014  */
00015 #define STRx(s)  #s
00016 #define STR(s)   STRx(s)
00017
00018 /*
00019  *  Any class needs these includes from avr-libc
00020  */
00021 #include <stdint.h>
00022 #include <avr/io.h>
00023 #include <avr/interrupt.h>
00024 #include <util/delay.h>
00025 #include <avr/sleep.h>
00026 #include <avr/power.h>
00027 #include <avr/wdt.h>
00028
00029
00030 /*
00031  * Include hardware abstraction for supported MCUs
00032  */
00033 #if defined(__AVR_ATmega328P__)
00034
00035     #include "ha_m328p.h"
00036     #include "utils_m328p.h"
00037
00038 #elif defined(__AVR_ATmega2560__)
00039
00040     #include "ha_m2560.h"
00041     #include "utils_m328p.h"
00042
00043 #elif defined(__AVR_ATmega168P__) || defined(__AVR_ATmega88P__) || defined(__AVR_ATmega48P__)
00044
00045     #include "ha_m168p.h"
00046     #include "utils_m328p.h"
00047
00048 #elif defined(__AVR_ATtiny85__)
```

```
00049
00050     #include "ha_attiny85.h"
00051     #include "utils_m328p.h"
00052
00053 #elif defined(ESP32) || defined(ESP_PLATFORM)
00054
00055     #include "ha_esp32.h"
00056     #include "utils_m328p.h"
00057
00058 #else
00059
00060     #error "TahakomAVRLib: unsupported MCU"
00061
00062 #endif
00063
00064
00065 #endif
```

9.139 ha_esp32.h File Reference

Header file for the hardware abstraction macros of ESP32 / ESP32-S3.

Macros

- #define PUSHBUTTON_DEBOUNCE_TIME_US 1000
- #define USART0_SET_BAUDRATE_HIGH_REGISTER
- #define USART0_SET_BAUDRATE_LOW_REGISTER
- #define USART0_ENABLE_ASYNC_TRANSMISSION_MODE
- #define USART0_ENABLE_SYNC_TRANSMISSION_MODE
- #define USART0_ENABLE_MASTER_SPI_MODE
- #define USART0_ENABLE_DOUBLE_SPEED_MODE
- #define USART0_DISABLE_DOUBLE_SPEED_MODE
- #define USART0_ENABLE_EVEN_PARITY_MODE
- #define USART0_ENABLE_ODD_PARITY_MODE
- #define USART0_DISABLE_PARITY_MODE
- #define USART0_SET_ONE_STOP_BIT
- #define USART0_SET_TWO_STOP_BITS
- #define USART0_SET_9BIT_FRAME_SIZE do {} while (0)
- #define USART0_SET_8BIT_FRAME_SIZE do {} while (0)
- #define USART0_SET_7BIT_FRAME_SIZE do {} while (0)
- #define USART0_SET_6BIT_FRAME_SIZE do {} while (0)
- #define USART0_SET_5BIT_FRAME_SIZE do {} while (0)
- #define USART0_ENABLE_TRANSMITTER do {} while (0)
- #define USART0_DISABLE_TRANSMITTER do {} while (0)
- #define USART0_ENABLE_RECEIVER do {} while (0)
- #define USART0_DISABLE_RECEIVER do {} while (0)
- #define USART0_CONTROL_STATUS_REGISTER 0
- #define USART0_DATA_REGISTER 0
- #define USART0_FRAME_ERROR 0
- #define USART0_DATA_OVERRUN 0
- #define USART0_PARITY_ERROR 0
- #define USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT do {} while (0)
- #define USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT do {} while (0)
- #define USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT do {} while (0)
- #define USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT do {} while (0)
- #define USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT do {} while (0)
- #define USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT do {} while (0)

- #define USART0_RECEIVE_COMPLETE_INTERRUPT 0
- #define USART0_TRANSMIT_COMPLETE_INTERRUPT 0
- #define USART0_DATA_REGISTER_EMPTY_INTERRUPT 0
- #define ADC_SELECT_REF_VOLTAGE(refVoltage)
- #define ADC_ADJUST_RESULT_LEFT do {} while (0)
- #define ADC_ADJUST_RESULT_RIGHT do {} while (0)
- #define ADC_SELECT_ANALOG_INPUT(pinNumber)
- #define ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber)
- #define ADC_ENABLE do {} while (0)
- #define ADC_DISABLE do {} while (0)
- #define ADC_START_CONVERSION do {} while (0)
- #define ADC_STOP_CONVERSION do {} while (0)
- #define ADC_ENABLE_AUTOTRIGGER do {} while (0)
- #define ADC_DISABLE_AUTOTRIGGER do {} while (0)
- #define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT do {} while (0)
- #define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT do {} while (0)
- #define ADC_SELECT_CLOCK_PRESCALER(clockPrescaler)
- #define ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource)
- #define ADC_CONVERSION_COMPLETE_INTERRUPT 0
- #define WATCHDOG_SELECT_TIMEOUT(timeOut)
- #define WATCHDOG_START(operationMode, timeOut)
- #define WATCHDOG_STOP do {} while (0)
- #define WATCHDOG_TIMEOUT_INTERRUPT 0
- #define MCU_SELECT_SLEEP_MODE(sleepMode)
- #define MCU_SLEEP_ENABLE do {} while (0)
- #define MCU_SLEEP_DISABLE do {} while (0)
- #define MCU_TWI_ENABLE do {} while (0)
- #define MCU_TWI_DISABLE do {} while (0)
- #define MCU_TIMER2_ENABLE do {} while (0)
- #define MCU_TIMER2_DISABLE do {} while (0)
- #define MCU_TIMER1_ENABLE do {} while (0)
- #define MCU_TIMER1_DISABLE do {} while (0)
- #define MCU_TIMER0_ENABLE do {} while (0)
- #define MCU_TIMER0_DISABLE do {} while (0)
- #define MCU_SPI_ENABLE do {} while (0)
- #define MCU_SPI_DISABLE do {} while (0)
- #define MCU_USART0_ENABLE do {} while (0)
- #define MCU_USART0_DISABLE do {} while (0)
- #define MCU_ADC_ENABLE do {} while (0)
- #define MCU_ADC_DISABLE do {} while (0)

9.139.1 Detailed Description

Header file for the hardware abstraction macros of ESP32 / ESP32-S3.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file ha_esp32.h.

9.139.2 Macro Definition Documentation

9.139.2.1 ADC_ADJUST_RESULT_LEFT

```
#define ADC_ADJUST_RESULT_LEFT do {} while (0)
```

Definition at line 70 of file ha_esp32.h.

9.139.2.2 ADC_ADJUST_RESULT_RIGHT

```
#define ADC_ADJUST_RESULT_RIGHT do {} while (0)
```

Definition at line 71 of file ha_esp32.h.

9.139.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_CONVERSION_COMPLETE_INTERRUPT 0
```

Definition at line 90 of file ha_esp32.h.

9.139.2.4 ADC_DISABLE

```
#define ADC_DISABLE do {} while (0)
```

Definition at line 76 of file ha_esp32.h.

9.139.2.5 ADC_DISABLE_AUTOTRIGGER

```
#define ADC_DISABLE_AUTOTRIGGER do {} while (0)
```

Definition at line 82 of file ha_esp32.h.

9.139.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT do {} while (0)
```

Definition at line 85 of file ha_esp32.h.

9.139.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER

```
#define ADC_DISABLE_DIGITAL_INPUT_REGISTER(
              pinNumber)
```

Value:

```
do {} while (0)
```

Definition at line 73 of file ha_esp32.h.

9.139.2.8 ADC_ENABLE

```
#define ADC_ENABLE do {} while (0)
```

Definition at line 75 of file ha_esp32.h.

9.139.2.9 ADC_ENABLE_AUTOTRIGGER

```
#define ADC_ENABLE_AUTOTRIGGER do {} while (0)
```

Definition at line 81 of file ha_esp32.h.

9.139.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT do {} while (0)
```

Definition at line 84 of file ha_esp32.h.

9.139.2.11 ADC_SELECT_ANALOG_INPUT

```
#define ADC_SELECT_ANALOG_INPUT(
              pinNumber)
```

Value:

```
do {} while (0)
```

Definition at line 72 of file ha_esp32.h.

9.139.2.12 ADC_SELECT_AUTO_TRIGGER_SOURCE

```
#define ADC_SELECT_AUTO_TRIGGER_SOURCE(
              triggerSource)
```

Value:

```
do {} while (0)
```

Definition at line 88 of file ha_esp32.h.

Referenced by core::ADConverter::selectAutoTriggerSource().

9.139.2.13 ADC_SELECT_CLOCK_PRESCALER

```
#define ADC_SELECT_CLOCK_PRESCALER(
              clockPrescaler)
```

Value:

```
do {} while (0)
```

Definition at line 87 of file ha_esp32.h.

9.139.2.14 ADC_SELECT_REF_VOLTAGE

```
#define ADC_SELECT_REF_VOLTAGE(
              refVoltage)
```

Value:

```
do {} while (0)
```

Definition at line 69 of file ha_esp32.h.

9.139.2.15 ADC_START_CONVERSION

```
#define ADC_START_CONVERSION do {} while (0)
```

Definition at line 78 of file ha_esp32.h.

9.139.2.16 ADC_STOP_CONVERSION

```
#define ADC_STOP_CONVERSION do {} while (0)
```

Definition at line 79 of file ha_esp32.h.

9.139.2.17 MCU_ADC_DISABLE

```
#define MCU_ADC_DISABLE do {} while (0)
```

Definition at line 123 of file ha_esp32.h.

Referenced by core::MCU::enableADC().

9.139.2.18 MCU_ADC_ENABLE

```
#define MCU_ADC_ENABLE do {} while (0)
```

Definition at line 122 of file ha_esp32.h.

Referenced by core::MCU::enableADC().

9.139.2.19 MCU_SELECT_SLEEP_MODE

```
#define MCU_SELECT_SLEEP_MODE(
              sleepMode)
```

Value:

```
do {} while (0)
```

Definition at line 99 of file ha_esp32.h.

9.139.2.20 MCU_SLEEP_DISABLE

```
#define MCU_SLEEP_DISABLE do {} while (0)
```

Definition at line 102 of file ha_esp32.h.

9.139.2.21 MCU_SLEEP_ENABLE

```
#define MCU_SLEEP_ENABLE do {} while (0)
```

Definition at line 101 of file ha_esp32.h.

9.139.2.22 MCU_SPI_DISABLE

```
#define MCU_SPI_DISABLE do {} while (0)
```

Definition at line 117 of file ha_esp32.h.

Referenced by core::MCU::enableSPI().

9.139.2.23 MCU_SPI_ENABLE

```
#define MCU_SPI_ENABLE do {} while (0)
```

Definition at line 116 of file ha_esp32.h.

Referenced by core::MCU::enableSPI().

9.139.2.24 MCU_TIMER0_DISABLE

```
#define MCU_TIMER0_DISABLE do {} while (0)
```

Definition at line 114 of file ha_esp32.h.

Referenced by core::MCU::enableTimerCounter0().

9.139.2.25 MCU_TIMER0_ENABLE

```
#define MCU_TIMER0_ENABLE do {} while (0)
```

Definition at line 113 of file ha_esp32.h.

Referenced by core::MCU::enableTimerCounter0().

9.139.2.26 MCU_TIMER1_DISABLE

```
#define MCU_TIMER1_DISABLE do {} while (0)
```

Definition at line 111 of file ha_esp32.h.

Referenced by core::MCU::enableTimerCounter1().

9.139.2.27 MCU_TIMER1_ENABLE

```
#define MCU_TIMER1_ENABLE do {} while (0)
```

Definition at line 110 of file ha_esp32.h.

Referenced by core::MCU::enableTimerCounter1().

9.139.2.28 MCU_TIMER2_DISABLE

```
#define MCU_TIMER2_DISABLE do {} while (0)
```

Definition at line 108 of file ha_esp32.h.

Referenced by core::MCU::enableTimerCounter2().

9.139.2.29 MCU_TIMER2_ENABLE

```
#define MCU_TIMER2_ENABLE do {} while (0)
```

Definition at line 107 of file ha_esp32.h.

Referenced by core::MCU::enableTimerCounter2().

9.139.2.30 MCU_TWI_DISABLE

```
#define MCU_TWI_DISABLE do {} while (0)
```

Definition at line 105 of file ha_esp32.h.

Referenced by core::MCU::enableTWI().

9.139.2.31 MCU_TWI_ENABLE

```
#define MCU_TWI_ENABLE do {} while (0)
```

Definition at line 104 of file ha_esp32.h.

Referenced by core::MCU::enableTWI().

9.139.2.32 MCU_USART0_DISABLE

```
#define MCU_USART0_DISABLE do {} while (0)
```

Definition at line 120 of file ha_esp32.h.

Referenced by core::MCU::enableUSART0().

9.139.2.33 MCU_USART0_ENABLE

```
#define MCU_USART0_ENABLE do {} while (0)
```

Definition at line 119 of file ha_esp32.h.

Referenced by core::MCU::enableUSART0().

9.139.2.34 PUSHBUTTON_DEBOUNCE_TIME_US

```
#define PUSHBUTTON_DEBOUNCE_TIME_US 1000
```

Definition at line 16 of file ha_esp32.h.

9.139.2.35 USART0_CONTROL_STATUS_REGISTER

```
#define USART0_CONTROL_STATUS_REGISTER 0
```

Definition at line 48 of file ha_esp32.h.

Referenced by io::USART0::receiveCompleteServiceRoutine().

9.139.2.36 USART0_DATA_OVERRUN

```
#define USART0_DATA_OVERRUN 0
```

Definition at line 52 of file ha_esp32.h.

Referenced by io::USART0::dataOverrun().

9.139.2.37 USART0_DATA_REGISTER

```
#define USART0_DATA_REGISTER 0
```

Definition at line 49 of file ha_esp32.h.

Referenced by io::USART0::dataRegisterEmptyServiceRoutine(), and io::USART0::receiveCompleteServiceRoutine().

9.139.2.38 USART0_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_DATA_REGISTER_EMPTY_INTERRUPT 0
```

Definition at line 66 of file ha_esp32.h.

9.139.2.39 USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT do {} while (0)
```

Definition at line 56 of file ha_esp32.h.

Referenced by io::USART0::enableDataRegisterEmptyInterrupt().

9.139.2.40 USART0_DISABLE_DOUBLE_SPEED_MODE

```
#define USART0_DISABLE_DOUBLE_SPEED_MODE
```

Definition at line 27 of file ha_esp32.h.

Referenced by io::USART0::setTransmissionMode().

9.139.2.41 USART0_DISABLE_PARITY_MODE

```
#define USART0_DISABLE_PARITY_MODE
```

Definition at line 31 of file ha_esp32.h.

Referenced by io::USART0::setParityMode().

9.139.2.42 USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT do {} while (0)
```

Definition at line 59 of file ha_esp32.h.

Referenced by io::USART0::enableReceiveCompleteInterrupt().

9.139.2.43 USART0_DISABLE_RECEIVER

```
#define USART0_DISABLE_RECEIVER do {} while (0)
```

Definition at line 46 of file ha_esp32.h.

Referenced by io::USART0::setCommunicationMode().

9.139.2.44 USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT do {} while (0)
```

Definition at line 62 of file ha_esp32.h.

Referenced by io::USART0::enableTransmitCompleteInterrupt().

9.139.2.45 USART0_DISABLE_TRANSMITTER

```
#define USART0_DISABLE_TRANSMITTER do {} while (0)
```

Definition at line 43 of file ha_esp32.h.

Referenced by io::USART0::setCommunicationMode().

9.139.2.46 USART0_ENABLE_ASYNC_TRANSMISSION_MODE

```
#define USART0_ENABLE_ASYNC_TRANSMISSION_MODE
```

Definition at line 22 of file ha_esp32.h.

Referenced by io::USART0::setTransmissionMode().

9.139.2.47 USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT do {} while (0)
```

Definition at line 55 of file ha_esp32.h.

Referenced by io::USART0::enableDataRegisterEmptyInterrupt().

9.139.2.48 USART0_ENABLE_DOUBLE_SPEED_MODE

```
#define USART0_ENABLE_DOUBLE_SPEED_MODE
```

Definition at line 26 of file ha_esp32.h.

9.139.2.49 USART0_ENABLE_EVEN_PARITY_MODE

```
#define USART0_ENABLE_EVEN_PARITY_MODE
```

Definition at line 29 of file ha_esp32.h.

Referenced by io::USART0::setParityMode().

9.139.2.50 USART0_ENABLE_MASTER_SPI_MODE

```
#define USART0_ENABLE_MASTER_SPI_MODE
```

Definition at line 24 of file ha_esp32.h.

Referenced by io::USART0::setTransmissionMode().

9.139.2.51 USART0_ENABLE_ODD_PARITY_MODE

```
#define USART0_ENABLE_ODD_PARITY_MODE
```

Definition at line 30 of file ha_esp32.h.

Referenced by io::USART0::setParityMode().

9.139.2.52 USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT do {} while (0)
```

Definition at line 58 of file ha_esp32.h.

Referenced by io::USART0::enableReceiveCompleteInterrupt().

9.139.2.53 USART0_ENABLE_RECEIVER

```
#define USART0_ENABLE_RECEIVER do {} while (0)
```

Definition at line 45 of file ha_esp32.h.

Referenced by io::USART0::setCommunicationMode().

9.139.2.54 USART0_ENABLE_SYNC_TRANSMISSION_MODE

```
#define USART0_ENABLE_SYNC_TRANSMISSION_MODE
```

Definition at line 23 of file ha_esp32.h.

Referenced by io::USART0::setTransmissionMode().

9.139.2.55 USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT do {} while (0)
```

Definition at line 61 of file ha_esp32.h.

Referenced by io::USART0::enableTransmitCompleteInterrupt().

9.139.2.56 USART0_ENABLE_TRANSMITTER

```
#define USART0_ENABLE_TRANSMITTER do {} while (0)
```

Definition at line 42 of file ha_esp32.h.

Referenced by io::USART0::setCommunicationMode().

9.139.2.57 USART0_FRAME_ERROR

```
#define USART0_FRAME_ERROR 0
```

Definition at line 51 of file ha_esp32.h.

Referenced by io::USART0::frameError().

9.139.2.58 USART0_PARITY_ERROR

```
#define USART0_PARITY_ERROR 0
```

Definition at line 53 of file ha_esp32.h.

Referenced by io::USART0::parityError().

9.139.2.59 USART0_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_RECEIVE_COMPLETE_INTERRUPT 0
```

Definition at line 64 of file ha_esp32.h.

9.139.2.60 USART0_SET_5BIT_FRAME_SIZE

```
#define USART0_SET_5BIT_FRAME_SIZE do {} while (0)
```

Definition at line 40 of file ha_esp32.h.

Referenced by io::USART0::setFrameSize().

9.139.2.61 USART0_SET_6BIT_FRAME_SIZE

```
#define USART0_SET_6BIT_FRAME_SIZE do {} while (0)
```

Definition at line 39 of file ha_esp32.h.

Referenced by io::USART0::setFrameSize().

9.139.2.62 USART0_SET_7BIT_FRAME_SIZE

```
#define USART0_SET_7BIT_FRAME_SIZE do {} while (0)
```

Definition at line 38 of file ha_esp32.h.

Referenced by io::USART0::setFrameSize().

9.139.2.63 USART0_SET_8BIT_FRAME_SIZE

```
#define USART0_SET_8BIT_FRAME_SIZE do {} while (0)
```

Definition at line 37 of file ha_esp32.h.

Referenced by io::USART0::setFrameSize().

9.139.2.64 USART0_SET_9BIT_FRAME_SIZE

```
#define USART0_SET_9BIT_FRAME_SIZE do {} while (0)
```

Definition at line 36 of file ha_esp32.h.

Referenced by io::USART0::setFrameSize().

9.139.2.65 USART0_SET_BAUDRATE_HIGH_REGISTER

```
#define USART0_SET_BAUDRATE_HIGH_REGISTER
```

Definition at line 19 of file ha_esp32.h.

Referenced by io::USART0::setBaudRate().

9.139.2.66 USART0_SET_BAUDRATE_LOW_REGISTER

```
#define USART0_SET_BAUDRATE_LOW_REGISTER
```

Definition at line 20 of file ha_esp32.h.

Referenced by io::USART0::setBaudRate().

9.139.2.67 USART0_SET_ONE_STOP_BIT

```
#define USART0_SET_ONE_STOP_BIT
```

Definition at line 33 of file ha_esp32.h.

Referenced by io::USART0::setStopBit().

9.139.2.68 USART0_SET_TWO_STOP_BITS

```
#define USART0_SET_TWO_STOP_BITS
```

Definition at line 34 of file ha_esp32.h.

Referenced by io::USART0::setStopBit().

9.139.2.69 USART0_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_TRANSMIT_COMPLETE_INTERRUPT 0
```

Definition at line 65 of file ha_esp32.h.

9.139.2.70 WATCHDOG_SELECT_TIMEOUT

```
#define WATCHDOG_SELECT_TIMEOUT(
              timeOut)
```

Value:

```
do {} while (0)
```

Definition at line 93 of file ha_esp32.h.

9.139.2.71 WATCHDOG_START

```
#define WATCHDOG_START(
              operationMode,
              timeOut)
```

Value:

```
do {} while (0)
```

Definition at line 94 of file ha_esp32.h.

9.139.2.72 WATCHDOG_STOP

```
#define WATCHDOG_STOP do {} while (0)
```

Definition at line 95 of file ha_esp32.h.

9.139.2.73 WATCHDOG_TIMEOUT_INTERRUPT

```
#define WATCHDOG_TIMEOUT_INTERRUPT 0
```

Definition at line 96 of file ha_esp32.h.

9.140 ha_esp32.h

Go to the documentation of this file.

```
00001
00008 #ifndef HA_ESP32_H
00009 #define HA_ESP32_H
00010
00011 #ifndef HABASE_H
00012     #error "don't use this file directly! Please include only ha_base.h!"
00013 #endif
00014
00015 // PushButton defines
00016 #define PUSHBUTTON_DEBOUNCE_TIME_US 1000
00017
00018 // USART / Serial defines for ESP32
00019 #define   USART0_SET_BAUDRATE_HIGH_REGISTER
00020 #define   USART0_SET_BAUDRATE_LOW_REGISTER
00021
00022 #define   USART0_ENABLE_ASYNC_TRANSMISSION_MODE
00023 #define   USART0_ENABLE_SYNC_TRANSMISSION_MODE
00024 #define   USART0_ENABLE_MASTER_SPI_MODE
00025
00026 #define   USART0_ENABLE_DOUBLE_SPEED_MODE
00027 #define   USART0_DISABLE_DOUBLE_SPEED_MODE
00028
00029 #define   USART0_ENABLE_EVEN_PARITY_MODE
00030 #define   USART0_ENABLE_ODD_PARITY_MODE
00031 #define   USART0_DISABLE_PARITY_MODE
00032
00033 #define   USART0_SET_ONE_STOP_BIT
00034 #define   USART0_SET_TWO_STOP_BITS
00035
00036 #define   USART0_SET_9BIT_FRAME_SIZE   do {} while (0)
00037 #define   USART0_SET_8BIT_FRAME_SIZE   do {} while (0)
00038 #define   USART0_SET_7BIT_FRAME_SIZE  do {} while (0)
00039 #define   USART0_SET_6BIT_FRAME_SIZE do {} while (0)
00040 #define   USART0_SET_5BIT_FRAME_SIZE do {} while (0)
00041
00042 #define   USART0_ENABLE_TRANSMITTER      do {} while (0)
00043 #define   USART0_DISABLE_TRANSMITTER     do {} while (0)
00044
00045 #define   USART0_ENABLE_RECEIVER         do {} while (0)
00046 #define   USART0_DISABLE_RECEIVER        do {} while (0)
00047
00048 #define   USART0_CONTROL_STATUS_REGISTER 0
00049 #define   USART0_DATA_REGISTER           0
00050
00051 #define   USART0_FRAME_ERROR             0
00052 #define   USART0_DATA_OVERRUN            0
00053 #define   USART0_PARITY_ERROR            0
00054
00055 #define   USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT   do {} while (0)
00056 #define   USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT  do {} while (0)
00057
00058 #define   USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT   do {} while (0)
00059 #define   USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT  do {} while (0)
00060
00061 #define   USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT   do {} while (0)
00062 #define   USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT  do {} while (0)
00063
00064 #define   USART0_RECEIVE_COMPLETE_INTERRUPT     0
00065 #define   USART0_TRANSMIT_COMPLETE_INTERRUPT    0
00066 #define   USART0_DATA_REGISTER_EMPTY_INTERRUPT  0
00067
00068 // ADC defines
00069 #define   ADC_SELECT_REF_VOLTAGE(refVoltage)            do {} while (0)
00070 #define   ADC_ADJUST_RESULT_LEFT                        do {} while (0)
00071 #define   ADC_ADJUST_RESULT_RIGHT                       do {} while (0)
00072 #define   ADC_SELECT_ANALOG_INPUT(pinNumber)            do {} while (0)
00073 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) do {} while (0)
00074
00075 #define   ADC_ENABLE                                    do {} while (0)
00076 #define   ADC_DISABLE                                   do {} while (0)
00077
00078 #define   ADC_START_CONVERSION                          do {} while (0)
00079 #define   ADC_STOP_CONVERSION                           do {} while (0)
00080
00081 #define   ADC_ENABLE_AUTOTRIGGER                        do {} while (0)
00082 #define   ADC_DISABLE_AUTOTRIGGER                      do {} while (0)
00083
00084 #define   ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT     do {} while (0)
00085 #define   ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT    do {} while (0)
00086
00087 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler)   do {} while (0)
00088 #define   ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource) do {} while (0)
```

```
00089
00090 #define   ADC_CONVERSION_COMPLETE_INTERRUPT            0
00091
00092 // Watchdog Timer defines
00093 #define WATCHDOG_SELECT_TIMEOUT(timeOut)               do {} while (0)
00094 #define WATCHDOG_START(operationMode,timeOut)          do {} while (0)
00095 #define WATCHDOG_STOP                                  do {} while (0)
00096 #define WATCHDOG_TIMEOUT_INTERRUPT                    0
00097
00098 // MCU defines
00099 #define MCU_SELECT_SLEEP_MODE(sleepMode)               do {} while (0)
00100
00101 #define MCU_SLEEP_ENABLE                               do {} while (0)
00102 #define MCU_SLEEP_DISABLE                              do {} while (0)
00103
00104 #define MCU_TWI_ENABLE                                 do {} while (0)
00105 #define MCU_TWI_DISABLE                                do {} while (0)
00106
00107 #define MCU_TIMER2_ENABLE                              do {} while (0)
00108 #define MCU_TIMER2_DISABLE                             do {} while (0)
00109
00110 #define MCU_TIMER1_ENABLE                              do {} while (0)
00111 #define MCU_TIMER1_DISABLE                             do {} while (0)
00112
00113 #define MCU_TIMER0_ENABLE                              do {} while (0)
00114 #define MCU_TIMER0_DISABLE                             do {} while (0)
00115
00116 #define MCU_SPI_ENABLE                                 do {} while (0)
00117 #define MCU_SPI_DISABLE                                do {} while (0)
00118
00119 #define MCU_USART0_ENABLE                              do {} while (0)
00120 #define MCU_USART0_DISABLE                             do {} while (0)
00121
00122 #define MCU_ADC_ENABLE                                 do {} while (0)
00123 #define MCU_ADC_DISABLE                                do {} while (0)
00124
00125 #endif
```

9.141 ha_m168p.h File Reference

Header file for the hardware abstraction macros of ATmega168P / ATmega88P / ATmega48P.

```
#include "ha_m328p.h"
```

9.141.1 Detailed Description

Header file for the hardware abstraction macros of ATmega168P / ATmega88P / ATmega48P.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file ha_m168p.h.

9.142 ha_m168p.h

Go to the documentation of this file.

```
00001
00008 #ifndef HA_M168P_H
00009 #define HA_M168P_H
00010
00011 #ifndef HABASE_H
00012     #error "don't use this file directly! Please include only ha_base.h!"
00013 #endif
00014
00015 // ATmega168P / 88P / 48P share register layouts with ATmega328P
00016 #include "ha_m328p.h"
00017
00018 #endif
```

9.143 ha_m2560.h File Reference

Header file for the hardware abstraction macros of the ATmega2560.

```
#include <util/setbaud.h>
```

Macros

- #define PUSHBUTTON_DEBOUNCE_TIME_US 1000
- #define USART0_SET_BAUDRATE_HIGH_REGISTER UBRR0H = UBRRH_VALUE
- #define USART0_SET_BAUDRATE_LOW_REGISTER UBRR0L = UBRRL_VALUE
- #define USART0_ENABLE_ASYNC_TRANSMISSION_MODE UCSR0C &= ~((1 << UMSEL01) | (1 << UMSEL00))
- #define USART0_ENABLE_SYNC_TRANSMISSION_MODE UCSR0C = (UCSR0C & ~(1 << UMSEL01)) | (1 << UMSEL00)
- #define USART0_ENABLE_MASTER_SPI_MODE UCSR0C |= (1 << UMSEL01) | (1 << UMSEL00)
- #define USART0_ENABLE_DOUBLE_SPEED_MODE
- #define USART0_DISABLE_DOUBLE_SPEED_MODE
- #define USART0_DISABLE_DOUBLE_SPEED_MODE UCSR0A &= ~(1 << U2X0)
- #define USART0_ENABLE_EVEN_PARITY_MODE UCSR0C = (1 << UPM01) | (UCSR0C & ~(1 << UPM00))
- #define USART0_ENABLE_ODD_PARITY_MODE UCSR0C |= ((1 << UPM01) | (1 << UPM00))
- #define USART0_DISABLE_PARITY_MODE UCSR0C &= ~((1 << UPM01) | (1 << UPM00))
- #define USART0_SET_ONE_STOP_BIT UCSR0C &= ~(1 << USBS0)
- #define USART0_SET_TWO_STOP_BITS UCSR0C |= (1 << USBS0)
- #define USART0_SET_9BIT_FRAME_SIZE
- #define USART0_SET_8BIT_FRAME_SIZE
- #define USART0_SET_7BIT_FRAME_SIZE
- #define USART0_SET_6BIT_FRAME_SIZE
- #define USART0_SET_5BIT_FRAME_SIZE
- #define USART0_ENABLE_TRANSMITTER UCSR0B |= 1 << TXEN0
- #define USART0_DISABLE_TRANSMITTER UCSR0B &= ~(1 << TXEN0)
- #define USART0_ENABLE_RECEIVER UCSR0B |= 1 << RXEN0
- #define USART0_DISABLE_RECEIVER UCSR0B &= ~(1 << RXEN0)
- #define USART0_CONTROL_STATUS_REGISTER UCSR0A
- #define USART0_DATA_REGISTER UDR0
- #define USART0_FRAME_ERROR FE0
- #define USART0_DATA_OVERRUN DOR0

- #define USART0_PARITY_ERROR UPE0
- #define USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B |= 1 << UDRIE0
- #define USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B &= ~(1 << UDRIE0)
- #define USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B |= 1 << RXCIE0
- #define USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B &= ~(1 << RXCIE0)
- #define USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B |= 1 << TXCIE0
- #define USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B &= ~(1 << TXCIE0)
- #define USART0_RECEIVE_COMPLETE_INTERRUPT USART0_RX_vect
- #define USART0_TRANSMIT_COMPLETE_INTERRUPT USART0_TX_vect
- #define USART0_DATA_REGISTER_EMPTY_INTERRUPT USART0_UDRE_vect
- #define ADC_SELECT_REF_VOLTAGE(refVoltage)
- #define ADC_ADJUST_RESULT_LEFT ADMUX |= 1 << ADLAR
- #define ADC_ADJUST_RESULT_RIGHT ADMUX &= ~(1 << ADLAR)
- #define ADC_SELECT_ANALOG_INPUT(pinNumber)
- #define ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber)
- #define ADC_ENABLE ADCSRA |= 1 << ADEN
- #define ADC_DISABLE ADCSRA &= ~(1 << ADEN)
- #define ADC_START_CONVERSION ADCSRA |= 1 << ADSC
- #define ADC_STOP_CONVERSION ADCSRA &= ~(1 << ADSC)
- #define ADC_ENABLE_AUTOTRIGGER ADCSRA |= 1 << ADATE
- #define ADC_DISABLE_AUTOTRIGGER ADCSRA &= ~(1 << ADATE)
- #define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA |= 1 << ADIE
- #define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA &= ~(1 << ADIE)
- #define ADC_SELECT_CLOCK_PRESCALER(clockPrescaler)
- #define ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource)
- #define ADC_CONVERSION_COMPLETE_INTERRUPT ADC_vect
- #define EXT_INT_SET_INT0_SENSE_CONTROL(senseControl)
- #define EXT_INT_SET_INT1_SENSE_CONTROL(senseControl)
- #define EXT_INT_ENABLE_INT0 EIMSK |= 1 << INT0
- #define EXT_INT_DISABLE_INT0 EIMSK &= ~(1 << INT0)
- #define EXT_INT_ENABLE_INT1 EIMSK |= 1 << INT1
- #define EXT_INT_DISABLE_INT1 EIMSK &= ~(1 << INT1)
- #define EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(pinChangePort)
- #define EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(pinChangePort)
- #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)
- #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)
- #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)
- #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)
- #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)
- #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)
- #define EXT_INT_INT0_INTERRUPT INT0_vect
- #define EXT_INT_INT1_INTERRUPT INT1_vect
- #define EXT_INT_PIN_CHANGE_PORTB_INTERRUPT PCINT0_vect
- #define EXT_INT_PIN_CHANGE_PORTC_INTERRUPT PCINT1_vect
- #define EXT_INT_PIN_CHANGE_PORTD_INTERRUPT PCINT2_vect
- #define TIMER0_STOP
- #define TIMER1_STOP
- #define TIMER2_STOP
- #define TIMER0_SELECT_CLOCK_SOURCE(clockSource)
- #define TIMER1_SELECT_CLOCK_SOURCE(clockSource)
- #define TIMER2_SELECT_CLOCK_SOURCE(clockSource)
- #define TIMER0_SELECT_OPERATION_MODE(operationMode)
- #define TIMER1_SELECT_OPERATION_MODE(operationMode)
- #define TIMER2_SELECT_OPERATION_MODE(operationMode)
- #define TIMER0_SELECT_COM_CHANNEL_A(compareOutputMode)

- #define TIMER0_SELECT_COM_CHANNEL_B(compareOutputMode)
- #define TIMER1_SELECT_COM_CHANNEL_A(compareOutputMode)
- #define TIMER1_SELECT_COM_CHANNEL_B(compareOutputMode)
- #define TIMER2_SELECT_COM_CHANNEL_A(compareOutputMode)
- #define TIMER2_SELECT_COM_CHANNEL_B(compareOutputMode)
- #define TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 |= 1 << OCIE0A
- #define TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 &= ~(1 << OCIE0A)
- #define TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 |= 1 << OCIE0B
- #define TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 &= ~(1 << OCIE0B)
- #define TIMER0_ENABLE_OVERFLOW_INTERRUPT TIMSK0 |= 1 << TOIE0
- #define TIMER0_DISABLE_OVERFLOW_INTERRUPT TIMSK0 &= ~(1 << TOIE0)
- #define TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 |= 1 << OCIE1A
- #define TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 &= ~(1 << OCIE1A)
- #define TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 |= 1 << OCIE1B
- #define TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 &= ~(1 << OCIE1B)
- #define TIMER1_ENABLE_OVERFLOW_INTERRUPT TIMSK1 |= 1 << TOIE1
- #define TIMER1_DISABLE_OVERFLOW_INTERRUPT TIMSK1 &= ~(1 << TOIE1)
- #define TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 |= 1 << ICIE1
- #define TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 &= ~(1 << ICIE1)
- #define TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 |= 1 << OCIE2A
- #define TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 &= ~(1 << OCIE2A)
- #define TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 |= 1 << OCIE2B
- #define TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 &= ~(1 << OCIE2B)
- #define TIMER2_ENABLE_OVERFLOW_INTERRUPT TIMSK2 |= 1 << TOIE2
- #define TIMER2_DISABLE_OVERFLOW_INTERRUPT TIMSK2 &= ~(1 << TOIE2)
- #define TIMER0_COM_CHANNEL_A_INTERRUPT TIMER0_COMPA_vect
- #define TIMER0_COM_CHANNEL_B_INTERRUPT TIMER0_COMPB_vect
- #define TIMER0_OVERFLOW_INTERRUPT TIMER0_OVF_vect
- #define TIMER1_COM_CHANNEL_A_INTERRUPT TIMER1_COMPA_vect
- #define TIMER1_COM_CHANNEL_B_INTERRUPT TIMER1_COMPB_vect
- #define TIMER1_OVERFLOW_INTERRUPT TIMER1_OVF_vect
- #define TIMER1_INPUT_CAPTURE_INTERRUPT TIMER1_CAPT_vect
- #define TIMER2_COM_CHANNEL_A_INTERRUPT TIMER2_COMPA_vect
- #define TIMER2_COM_CHANNEL_B_INTERRUPT TIMER2_COMPB_vect
- #define TIMER2_OVERFLOW_INTERRUPT TIMER2_OVF_vect
- #define WATCHDOG_SELECT_TIMEOUT(timeOut)
- #define WATCHDOG_START(operationMode, timeOut)
- #define WATCHDOG_STOP
- #define WATCHDOG_TIMEOUT_INTERRUPT WDT_vect
- #define MCU_SELECT_SLEEP_MODE(sleepMode)
- #define MCU_SLEEP_ENABLE SMCR |= 1 << SE
- #define MCU_SLEEP_DISABLE SMCR &= ~(1 << SE)
- #define MCU_TWI_ENABLE PRR0 &= ~(1 << PRTWI)
- #define MCU_TWI_DISABLE PRR0 |= 1 << PRTWI
- #define MCU_TIMER2_ENABLE PRR0 &= ~(1 << PRTIM2)
- #define MCU_TIMER2_DISABLE PRR0 |= 1 << PRTIM2
- #define MCU_TIMER1_ENABLE PRR0 &= ~(1 << PRTIM1)
- #define MCU_TIMER1_DISABLE PRR0 |= 1 << PRTIM1
- #define MCU_TIMER0_ENABLE PRR0 &= ~(1 << PRTIM0)
- #define MCU_TIMER0_DISABLE PRR0 |= 1 << PRTIM0
- #define MCU_SPI_ENABLE PRR0 &= ~(1 << PRSPI)
- #define MCU_SPI_DISABLE PRR0 |= 1 << PRSPI
- #define MCU_USART0_ENABLE PRR0 &= ~(1 << PRUSART0)
- #define MCU_USART0_DISABLE PRR0 |= 1 << PRUSART0
- #define MCU_ADC_ENABLE PRR0 &= ~(1 << PRADC)

- #define MCU_ADC_DISABLE PRR0 |= 1 << PRADC
- #define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(pulseWidth, clockPrescaler)
- #define SERVOMOTOR_TIMER_ANGLE_COUNT(angle, out_min, out_mid, out_max)
- #define SPI_ENABLE SPCR |= 1 << SPE
- #define SPI_DISABLE SPCR &= ~(1 << SPE)
- #define SPI_SELECT_MASTER_MODE SPCR |= 1 << MSTR
- #define SPI_SELECT_SLAVE_MODE SPCR &= ~(1 << MSTR)
- #define SPI_SELECT_DATA_MODE(dataMode)
- #define SPI_SELECT_CLOCK_PRESCALER(clockPrescaler)
- #define SPI_SELECT_DATA_ORDER(dataOrder)
- #define SPI_WRITE_COLLISION WCOL
- #define SPI_TRANSFER_COMPLETE SPIF
- #define SPI_MASTER_MODE MSTR
- #define SPI_CONTROL_REGISTER SPCR
- #define SPI_DATA_REGISTER SPDR
- #define SPI_STATUS_REGISTER SPSR
- #define SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT SPCR |= 1 << SPIE
- #define SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT SPCR &= ~(1 << SPIE)
- #define SPI_TRANSFER_COMPLETE_INTERRUPT SPI_STC_vect

9.143.1 Detailed Description

Header file for the hardware abstraction macros of the ATmega2560.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file ha_m2560.h.

9.143.2 Macro Definition Documentation

9.143.2.1 ADC_ADJUST_RESULT_LEFT

```
#define ADC_ADJUST_RESULT_LEFT ADMUX |= 1 << ADLAR
```

Definition at line 107 of file ha_m2560.h.

9.143.2.2 ADC_ADJUST_RESULT_RIGHT

```
#define ADC_ADJUST_RESULT_RIGHT ADMUX &= ~(1 << ADLAR)
```

Definition at line 108 of file ha_m2560.h.

9.143.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_CONVERSION_COMPLETE_INTERRUPT ADC_vect
```

Definition at line 153 of file ha_m2560.h.

9.143.2.4 ADC_DISABLE

```
#define ADC_DISABLE ADCSRA &= ~(1 << ADEN)
```

Definition at line 130 of file ha_m2560.h.

9.143.2.5 ADC_DISABLE_AUTOTRIGGER

```
#define ADC_DISABLE_AUTOTRIGGER ADCSRA &= ~(1 << ADATE)
```

Definition at line 136 of file ha_m2560.h.

9.143.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA &= ~(1 << ADIE)
```

Definition at line 139 of file ha_m2560.h.

9.143.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER

```
#define ADC_DISABLE_DIGITAL_INPUT_REGISTER(
            pinNumber)
```

Value:

```
    do { \
        if ((pinNumber) < 8) { \
            DIDR0 |= (1 « (pinNumber)); \
        } else { \
            DIDR2 |= (1 « ((pinNumber) - 8)); \
        } \
    } while (0)
```

Definition at line 120 of file ha_m2560.h.

```
00120 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) \
00121     do { \
00122         if ((pinNumber) < 8) { \
00123             DIDR0 |= (1 « (pinNumber)); \
00124         } else { \
00125             DIDR2 |= (1 « ((pinNumber) - 8)); \
00126         } \
00127     } while (0)
```

9.143.2.8 ADC_ENABLE

```
#define ADC_ENABLE ADCSRA |= 1 << ADEN
```

Definition at line 129 of file ha_m2560.h.

9.143.2.9 ADC_ENABLE_AUTOTRIGGER

```
#define ADC_ENABLE_AUTOTRIGGER ADCSRA |= 1 << ADATE
```

Definition at line 135 of file ha_m2560.h.

9.143.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA |= 1 << ADIE
```

Definition at line 138 of file ha_m2560.h.

9.143.2.11 ADC_SELECT_ANALOG_INPUT

```
#define ADC_SELECT_ANALOG_INPUT(
              pinNumber)
```

Value:

```
    do { \
        ADMUX &= 0xE0; \
        ADCSRB &= ~(1 « MUX5); \
        if ((pinNumber) & 8) { \
            ADCSRB |= (1 « MUX5); \
        } \
        ADMUX |= ((pinNumber) & 7); \
    } while (0)
```

Definition at line 110 of file ha_m2560.h.

```
00110 #define   ADC_SELECT_ANALOG_INPUT(pinNumber) \
00111     do { \
00112         ADMUX &= 0xE0; \
00113         ADCSRB &= ~(1 « MUX5); \
00114         if ((pinNumber) & 8) { \
00115             ADCSRB |= (1 « MUX5); \
00116         } \
00117         ADMUX |= ((pinNumber) & 7); \
00118     } while (0)
```

9.143.2.12 ADC_SELECT_AUTO_TRIGGER_SOURCE

```
#define ADC_SELECT_AUTO_TRIGGER_SOURCE(
              triggerSource)
```

Value:

```
    do { \
        ADCSRB &= 0xF8; \
        ADCSRB |= (triggerSource); \
    } while (0)
```

Definition at line 147 of file ha_m2560.h.

```
00147 #define   ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource) \
00148     do { \
00149         ADCSRB &= 0xF8; \
00150         ADCSRB |= (triggerSource); \
00151     } while (0)
```

9.143.2.13 ADC_SELECT_CLOCK_PRESCALER

```
#define ADC_SELECT_CLOCK_PRESCALER(
              clockPrescaler)
```

Value:

```
    do { \
        ADCSRA &= 0xF8; \
        ADCSRA |= (clockPrescaler); \
    } while (0)
```

Definition at line 141 of file ha_m2560.h.

```
00141 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00142     do { \
00143         ADCSRA &= 0xF8; \
00144         ADCSRA |= (clockPrescaler); \
00145     } while (0)
```

9.143.2.14 ADC_SELECT_REF_VOLTAGE

```
#define ADC_SELECT_REF_VOLTAGE(
              refVoltage)
```

Value:

```
    do { \
        ADMUX &= 0x3F; \
        ADMUX |= (refVoltage) « 6; \
    } while (0)
```

Definition at line 101 of file ha_m2560.h.

```
00101 #define   ADC_SELECT_REF_VOLTAGE(refVoltage) \
00102     do { \
00103         ADMUX &= 0x3F; \
00104         ADMUX |= (refVoltage) « 6; \
00105     } while (0)
```

9.143.2.15 ADC_START_CONVERSION

```
#define ADC_START_CONVERSION ADCSRA |= 1 << ADSC
```

Definition at line 132 of file ha_m2560.h.

9.143.2.16 ADC_STOP_CONVERSION

```
#define ADC_STOP_CONVERSION ADCSRA &= ~(1 << ADSC)
```

Definition at line 133 of file ha_m2560.h.

9.143.2.17 EXT_INT_DISABLE_INT0

```
#define EXT_INT_DISABLE_INT0 EIMSK &= ~(1 << INT0)
```

Definition at line 168 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enableInt0().

9.143.2.18 EXT_INT_DISABLE_INT1

```
#define EXT_INT_DISABLE_INT1 EIMSK &= ~(1 << INT1)
```

Definition at line 171 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enableInt1().

9.143.2.19 EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT

```
#define EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(
              pinChangePort)
```

Value:

```
PCICR &= ~(1 « (pinChangePort))
```

Definition at line 174 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChange().

9.143.2.20 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB

```
#define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(
              pinChangePin)
```

Value:

```
PCMSK0 &= ~(1 « (pinChangePin))
```

Definition at line 177 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChangeMaskPortB().

9.143.2.21 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC

```
#define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(
              pinChangePin)
```

Value:

```
PCMSK1 &= ~(1 « (pinChangePin))
```

Definition at line 180 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChangeMaskPortC().

9.143.2.22 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD

```
#define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(
              pinChangePin)
```

Value:

```
PCMSK2 &= ~(1 « (pinChangePin))
```

Definition at line 183 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChangeMaskPortD().

9.143.2.23 EXT_INT_ENABLE_INT0

```
#define EXT_INT_ENABLE_INT0 EIMSK |= 1 << INT0
```

Definition at line 167 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enableInt0().

9.143.2.24 EXT_INT_ENABLE_INT1

```
#define EXT_INT_ENABLE_INT1 EIMSK |= 1 << INT1
```

Definition at line 170 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enableInt1().

9.143.2.25 EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT

```
#define EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(
              pinChangePort)
```

Value:

```
PCICR |= 1 « (pinChangePort)
```

Definition at line 173 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChange().

9.143.2.26 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB

```
#define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(
              pinChangePin)
```

Value:

```
PCMSK0 |= 1 « (pinChangePin)
```

Definition at line 176 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChangeMaskPortB().

9.143.2.27 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC

```
#define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(
              pinChangePin)
```

Value:

```
PCMSK1 |= 1 « (pinChangePin)
```

Definition at line 179 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChangeMaskPortC().

9.143.2.28 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD

```
#define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(
              pinChangePin)
```

Value:

```
PCMSK2 |= 1 « (pinChangePin)
```

Definition at line 182 of file ha_m2560.h.

Referenced by core::ExternInterrupt::enablePinChangeMaskPortD().

9.143.2.29 EXT_INT_INT0_INTERRUPT

```
#define EXT_INT_INT0_INTERRUPT INT0_vect
```

Definition at line 185 of file ha_m2560.h.

9.143.2.30 EXT_INT_INT1_INTERRUPT

```
#define EXT_INT_INT1_INTERRUPT INT1_vect
```

Definition at line 186 of file ha_m2560.h.

9.143.2.31 EXT_INT_PIN_CHANGE_PORTB_INTERRUPT

```
#define EXT_INT_PIN_CHANGE_PORTB_INTERRUPT PCINT0_vect
```

Definition at line 188 of file ha_m2560.h.

9.143.2.32 EXT_INT_PIN_CHANGE_PORTC_INTERRUPT

```
#define EXT_INT_PIN_CHANGE_PORTC_INTERRUPT PCINT1_vect
```

Definition at line 189 of file ha_m2560.h.

9.143.2.33 EXT_INT_PIN_CHANGE_PORTD_INTERRUPT

```
#define EXT_INT_PIN_CHANGE_PORTD_INTERRUPT PCINT2_vect
```

Definition at line 190 of file ha_m2560.h.

9.143.2.34 EXT_INT_SET_INT0_SENSE_CONTROL

```
#define EXT_INT_SET_INT0_SENSE_CONTROL(
                senseControl)
```

Value:

```
    do { \
        EICRA &= 0xFC; \
        EICRA |= (senseControl); \
    } while (0)
```

Definition at line 156 of file ha_m2560.h.

```
00156 #define EXT_INT_SET_INT0_SENSE_CONTROL(senseControl) \
00157     do { \
00158         EICRA &= 0xFC; \
00159         EICRA |= (senseControl); \
00160     } while (0)
```

Referenced by core::ExternInterrupt::setInt0SenseControl().

9.143.2.35 EXT_INT_SET_INT1_SENSE_CONTROL

```
#define EXT_INT_SET_INT1_SENSE_CONTROL(
                senseControl)
```

Value:

```
    do { \
        EICRA &= 0xF3; \
        EICRA |= (senseControl) « 2; \
    } while (0)
```

Definition at line 161 of file ha_m2560.h.

```
00161 #define EXT_INT_SET_INT1_SENSE_CONTROL(senseControl) \
00162     do { \
00163         EICRA &= 0xF3; \
00164         EICRA |= (senseControl) « 2; \
00165     } while (0)
```

Referenced by core::ExternInterrupt::setInt1SenseControl().

9.143.2.36 MCU_ADC_DISABLE

```
#define MCU_ADC_DISABLE PRR0 |= 1 << PRADC
```

Definition at line 369 of file ha_m2560.h.

9.143.2.37 MCU_ADC_ENABLE

```
#define MCU_ADC_ENABLE PRR0 &= ~(1 << PRADC)
```

Definition at line 368 of file ha_m2560.h.

9.143.2.38 MCU_SELECT_SLEEP_MODE

```
#define MCU_SELECT_SLEEP_MODE(
                sleepMode)
```

Value:

```
    do { \
         SMCR &= 0xF1; \
         SMCR |= (sleepMode) « 1; \
    } while (0)
```

Definition at line 341 of file ha_m2560.h.

```
00341 #define MCU_SELECT_SLEEP_MODE(sleepMode) \
00342     do { \
00343          SMCR &= 0xF1; \
00344          SMCR |= (sleepMode) « 1; \
00345     } while (0)
```

9.143.2.39 MCU_SLEEP_DISABLE

```
#define MCU_SLEEP_DISABLE SMCR &= ~(1 << SE)
```

Definition at line 348 of file ha_m2560.h.

9.143.2.40 MCU_SLEEP_ENABLE

```
#define MCU_SLEEP_ENABLE SMCR |= 1 << SE
```

Definition at line 347 of file ha_m2560.h.

9.143.2.41 MCU_SPI_DISABLE

```
#define MCU_SPI_DISABLE PRR0 |= 1 << PRSPI
```

Definition at line 363 of file ha_m2560.h.

9.143.2.42 MCU_SPI_ENABLE

```
#define MCU_SPI_ENABLE PRR0 &= ~(1 << PRSPI)
```

Definition at line 362 of file ha_m2560.h.

9.143.2.43 MCU_TIMER0_DISABLE

```
#define MCU_TIMER0_DISABLE PRR0 |= 1 << PRTIM0
```

Definition at line 360 of file ha_m2560.h.

9.143.2.44 MCU_TIMER0_ENABLE

```
#define MCU_TIMER0_ENABLE PRR0 &= ~(1 << PRTIM0)
```

Definition at line 359 of file ha_m2560.h.

9.143.2.45 MCU_TIMER1_DISABLE

```
#define MCU_TIMER1_DISABLE PRR0 |= 1 << PRTIM1
```

Definition at line 357 of file ha_m2560.h.

9.143.2.46 MCU_TIMER1_ENABLE

```
#define MCU_TIMER1_ENABLE PRR0 &= ~(1 << PRTIM1)
```

Definition at line 356 of file ha_m2560.h.

9.143.2.47 MCU_TIMER2_DISABLE

```
#define MCU_TIMER2_DISABLE PRR0 |= 1 << PRTIM2
```

Definition at line 354 of file ha_m2560.h.

9.143.2.48 MCU_TIMER2_ENABLE

```
#define MCU_TIMER2_ENABLE PRR0 &= ~(1 << PRTIM2)
```

Definition at line 353 of file ha_m2560.h.

9.143.2.49 MCU_TWI_DISABLE

```
#define MCU_TWI_DISABLE PRR0 |= 1 << PRTWI
```

Definition at line 351 of file ha_m2560.h.

9.143.2.50 MCU_TWI_ENABLE

```
#define MCU_TWI_ENABLE PRR0 &= ~(1 << PRTWI)
```

Definition at line 350 of file ha_m2560.h.

9.143.2.51 MCU_USART0_DISABLE

```
#define MCU_USART0_DISABLE PRR0 |= 1 << PRUSART0
```

Definition at line 366 of file ha_m2560.h.

9.143.2.52 MCU_USART0_ENABLE

```
#define MCU_USART0_ENABLE PRR0 &= ~(1 << PRUSART0)
```

Definition at line 365 of file ha_m2560.h.

9.143.2.53 PUSHBUTTON_DEBOUNCE_TIME_US

```
#define PUSHBUTTON_DEBOUNCE_TIME_US 1000
```

Definition at line 16 of file ha_m2560.h.

9.143.2.54 SERVOMOTOR_TIMER_ANGLE_COUNT

```
#define SERVOMOTOR_TIMER_ANGLE_COUNT(
              angle,
              out_min,
              out_mid,
              out_max)
```

Value:

```
(((((out_min)*((angle) - 90L)*((angle) - 180L))/16200L) + (((angle)*(out_max)*((angle) - 90L))/16200L) -
      (((angle)*(out_mid)*((angle) - 180L))/8100L)))
```

Definition at line 375 of file ha_m2560.h.

Referenced by component::ServoMotor::computeRotationAngleCount().

9.143.2.55 SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT

```
#define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(
              pulseWidth,
              clockPrescaler)
```

Value:

```
    ((((uint32_t)(pulseWidth)) * ((F_CPU) / 1000000UL)) / (clockPrescaler))
```

Definition at line 372 of file ha_m2560.h.

```
00372 #define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(pulseWidth,clockPrescaler) \
00373     ((((uint32_t)(pulseWidth)) * ((F_CPU) / 1000000UL)) / (clockPrescaler))
```

Referenced by component::ServoMotor::computePulseCycleCount(), component::ServoMotor::computePulseWidthMaxCount(), component::ServoMotor::computePulseWidthMidCount(), and component::ServoMotor::computePulseWidthMinCount().

9.143.2.56 SPI_CONTROL_REGISTER

```
#define SPI_CONTROL_REGISTER SPCR
```

Definition at line 405 of file ha_m2560.h.

9.143.2.57 SPI_DATA_REGISTER

```
#define SPI_DATA_REGISTER SPDR
```

Definition at line 406 of file ha_m2560.h.

Referenced by io::SPI::masterSendByte(), and io::SPI::transferCompleteServiceRoutine().

9.143.2.58 SPI_DISABLE

```
#define SPI_DISABLE SPCR &= ~(1 << SPE)
```

Definition at line 379 of file ha_m2560.h.

Referenced by io::SPI::selectOperationMode().

9.143.2.59 SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT

```
#define SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT SPCR &= ~(1 << SPIE)
```

Definition at line 410 of file ha_m2560.h.

Referenced by io::SPI::enableTransferCompleteInterrupt().

9.143.2.60 SPI_ENABLE

```
#define SPI_ENABLE SPCR |= 1 << SPE
```

Definition at line 378 of file ha_m2560.h.

Referenced by io::SPI::selectOperationMode().

9.143.2.61 SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT

```
#define SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT SPCR |= 1 << SPIE
```

Definition at line 409 of file ha_m2560.h.

Referenced by io::SPI::enableTransferCompleteInterrupt().

9.143.2.62 SPI_MASTER_MODE

```
#define SPI_MASTER_MODE MSTR
```

Definition at line 404 of file ha_m2560.h.

9.143.2.63 SPI_SELECT_CLOCK_PRESCALER

```
#define SPI_SELECT_CLOCK_PRESCALER(
                clockPrescaler)
```

Value:

```
    do { \
        SPCR &= 0xFC; \
        SPCR |= ((clockPrescaler) & 3); \
        SPSR &= 0xFE; \
        SPSR |= (((clockPrescaler) & 4) » 2); \
    } while (0)
```

Definition at line 388 of file ha_m2560.h.

```
00388 #define   SPI_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00389     do { \
00390         SPCR &= 0xFC; \
00391         SPCR |= ((clockPrescaler) & 3); \
00392         SPSR &= 0xFE; \
00393         SPSR |= (((clockPrescaler) & 4) » 2); \
00394     } while (0)
```

Referenced by io::SPI::selectClockPrescaler().

9.143.2.64 SPI_SELECT_DATA_MODE

```
#define SPI_SELECT_DATA_MODE(
                dataMode)
```

Value:

```
    do { \
        SPCR &= 0xF3; \
        SPCR |= (dataMode) « 2; \
    } while (0)
```

Definition at line 383 of file ha_m2560.h.

```
00383 #define   SPI_SELECT_DATA_MODE(dataMode) \
00384     do { \
00385         SPCR &= 0xF3; \
00386         SPCR |= (dataMode) « 2; \
00387     } while (0)
```

Referenced by io::SPI::selectDataMode().

9.143.2.65 SPI_SELECT_DATA_ORDER

```
#define SPI_SELECT_DATA_ORDER(
                dataOrder)
```

Value:

```
    do { \
        SPCR &= 0xDF; \
        SPCR |= (dataOrder) « 5; \
    } while (0)
```

Definition at line 395 of file ha_m2560.h.

```
00395 #define   SPI_SELECT_DATA_ORDER(dataOrder) \
00396     do { \
00397         SPCR &= 0xDF; \
00398         SPCR |= (dataOrder) « 5; \
00399     } while (0)
```

Referenced by io::SPI::selectDataOrder().

9.143.2.66 SPI_SELECT_MASTER_MODE

```
#define SPI_SELECT_MASTER_MODE SPCR |= 1 << MSTR
```

Definition at line 380 of file ha_m2560.h.

Referenced by io::SPI::selectOperationMode().

9.143.2.67 SPI_SELECT_SLAVE_MODE

```
#define SPI_SELECT_SLAVE_MODE SPCR &= ~(1 << MSTR)
```

Definition at line 381 of file ha_m2560.h.

Referenced by io::SPI::selectOperationMode().

9.143.2.68 SPI_STATUS_REGISTER

```
#define SPI_STATUS_REGISTER SPSR
```

Definition at line 407 of file ha_m2560.h.

Referenced by io::SPI::transferComplete(), and io::SPI::writeCollision().

9.143.2.69 SPI_TRANSFER_COMPLETE

```
#define SPI_TRANSFER_COMPLETE SPIF
```

Definition at line 402 of file ha_m2560.h.

Referenced by io::SPI::transferComplete().

9.143.2.70 SPI_TRANSFER_COMPLETE_INTERRUPT

```
#define SPI_TRANSFER_COMPLETE_INTERRUPT SPI_STC_vect
```

Definition at line 412 of file ha_m2560.h.

9.143.2.71 SPI_WRITE_COLLISION

```
#define SPI_WRITE_COLLISION WCOL
```

Definition at line 401 of file ha_m2560.h.

Referenced by io::SPI::writeCollision().

9.143.2.72 TIMER0_COM_CHANNEL_A_INTERRUPT

```
#define TIMER0_COM_CHANNEL_A_INTERRUPT TIMER0_COMPA_vect
```

Definition at line 307 of file ha_m2560.h.

9.143.2.73 TIMER0_COM_CHANNEL_B_INTERRUPT

```
#define TIMER0_COM_CHANNEL_B_INTERRUPT TIMER0_COMPB_vect
```

Definition at line 308 of file ha_m2560.h.

9.143.2.74 TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 &= ~(1 << OCIE0A)
```

Definition at line 278 of file ha_m2560.h.

Referenced by core::TimerCounter0::enableOutputCompareMatchInterrupt().

9.143.2.75 TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 &= ~(1 << OCIE0B)
```

Definition at line 281 of file ha_m2560.h.

Referenced by core::TimerCounter0::enableOutputCompareMatchInterrupt().

9.143.2.76 TIMER0_DISABLE_OVERFLOW_INTERRUPT

```
#define TIMER0_DISABLE_OVERFLOW_INTERRUPT TIMSK0 &= ~(1 << TOIE0)
```

Definition at line 284 of file ha_m2560.h.

Referenced by core::TimerCounter0::enableOverflowInterrupt().

9.143.2.77 TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 |= 1 << OCIE0A
```

Definition at line 277 of file ha_m2560.h.

Referenced by core::TimerCounter0::enableOutputCompareMatchInterrupt().

9.143.2.78 TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 |= 1 << OCIE0B
```

Definition at line 280 of file ha_m2560.h.

Referenced by core::TimerCounter0::enableOutputCompareMatchInterrupt().

9.143.2.79 TIMER0_ENABLE_OVERFLOW_INTERRUPT

```
#define TIMER0_ENABLE_OVERFLOW_INTERRUPT TIMSK0 |= 1 << TOIE0
```

Definition at line 283 of file ha_m2560.h.

Referenced by core::TimerCounter0::enableOverflowInterrupt().

9.143.2.80 TIMER0_OVERFLOW_INTERRUPT

```
#define TIMER0_OVERFLOW_INTERRUPT TIMER0_OVF_vect
```

Definition at line 309 of file ha_m2560.h.

9.143.2.81 TIMER0_SELECT_CLOCK_SOURCE

```
#define TIMER0_SELECT_CLOCK_SOURCE(
              clockSource)
```

Value:

```
    do { \
        TCCR0B &= 0xF8; \
        TCCR0B |= (clockSource); \
    } while (0)
```

Definition at line 206 of file ha_m2560.h.

```
00206 #define TIMER0_SELECT_CLOCK_SOURCE(clockSource) \
00207     do { \
00208         TCCR0B &= 0xF8; \
00209         TCCR0B |= (clockSource); \
00210     } while (0)
```

Referenced by core::TimerCounter0::start().

9.143.2.82 TIMER0_SELECT_COM_CHANNEL_A

```
#define TIMER0_SELECT_COM_CHANNEL_A(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR0A &= 0x3F; \
        TCCR0A |= (compareOutputMode) « 6; \
    } while (0)
```

Definition at line 244 of file ha_m2560.h.

```
00244 #define TIMER0_SELECT_COM_CHANNEL_A(compareOutputMode) \
00245     do { \
00246         TCCR0A &= 0x3F; \
00247         TCCR0A |= (compareOutputMode) « 6; \
00248     } while (0)
```

Referenced by core::TimerCounter0::selectCompareOutputMode().

9.143.2.83 TIMER0_SELECT_COM_CHANNEL_B

```
#define TIMER0_SELECT_COM_CHANNEL_B(
                compareOutputMode)
```

Value:

```
    do { \
        TCCR0A &= 0xCF; \
        TCCR0A |= (compareOutputMode) « 4; \
    } while (0)
```

Definition at line 249 of file ha_m2560.h.

```
00249 #define TIMER0_SELECT_COM_CHANNEL_B(compareOutputMode) \
00250     do { \
00251         TCCR0A &= 0xCF; \
00252         TCCR0A |= (compareOutputMode) « 4; \
00253     } while (0)
```

Referenced by core::TimerCounter0::selectCompareOutputMode().

9.143.2.84 TIMER0_SELECT_OPERATION_MODE

```
#define TIMER0_SELECT_OPERATION_MODE(
                operationMode)
```

Value:

```
    do { \
        TCCR0A &= 0xFC; \
        TCCR0A |= ((operationMode) & 3); \
        TCCR0B &= 0xF7; \
        TCCR0B |= (((operationMode) & 12) « 1); \
    } while (0)
```

Definition at line 222 of file ha_m2560.h.

```
00222 #define TIMER0_SELECT_OPERATION_MODE(operationMode) \
00223     do { \
00224         TCCR0A &= 0xFC; \
00225         TCCR0A |= ((operationMode) & 3); \
00226         TCCR0B &= 0xF7; \
00227         TCCR0B |= (((operationMode) & 12) « 1); \
00228     } while (0)
```

Referenced by core::TimerCounter0::selectOperationMode().

9.143.2.85 TIMER0_STOP

```
#define TIMER0_STOP
```

Value:

```
    do { \
        TCCR0B &= 0xF8; \
    } while (0)
```

Definition at line 193 of file ha_m2560.h.

```
00193 #define TIMER0_STOP \
00194     do { \
00195         TCCR0B &= 0xF8; \
00196     } while (0)
```

Referenced by core::TimerCounter0::stop().

9.143.2.86 TIMER1_COM_CHANNEL_A_INTERRUPT

```
#define TIMER1_COM_CHANNEL_A_INTERRUPT TIMER1_COMPA_vect
```

Definition at line 311 of file ha_m2560.h.

9.143.2.87 TIMER1_COM_CHANNEL_B_INTERRUPT

```
#define TIMER1_COM_CHANNEL_B_INTERRUPT TIMER1_COMPB_vect
```

Definition at line 312 of file ha_m2560.h.

9.143.2.88 TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 &= ~(1 << OCIE1A)
```

Definition at line 287 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableOutputCompareMatchInterrupt().

9.143.2.89 TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 &= ~(1 << OCIE1B)
```

Definition at line 290 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableOutputCompareMatchInterrupt().

9.143.2.90 TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT

```
#define TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 &= ~(1 << ICIE1)
```

Definition at line 296 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableInputCaptureInterrupt().

9.143.2.91 TIMER1_DISABLE_OVERFLOW_INTERRUPT

```
#define TIMER1_DISABLE_OVERFLOW_INTERRUPT TIMSK1 &= ~(1 << TOIE1)
```

Definition at line 293 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableOverflowInterrupt().

9.143.2.92 TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 |= 1 << OCIE1A
```

Definition at line 286 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableOutputCompareMatchInterrupt().

9.143.2.93 TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 |= 1 << OCIE1B
```

Definition at line 289 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableOutputCompareMatchInterrupt().

9.143.2.94 TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT

```
#define TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 |= 1 << ICIE1
```

Definition at line 295 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableInputCaptureInterrupt().

9.143.2.95 TIMER1_ENABLE_OVERFLOW_INTERRUPT

```
#define TIMER1_ENABLE_OVERFLOW_INTERRUPT TIMSK1 |= 1 << TOIE1
```

Definition at line 292 of file ha_m2560.h.

Referenced by core::TimerCounter1::enableOverflowInterrupt().

9.143.2.96 TIMER1_INPUT_CAPTURE_INTERRUPT

```
#define TIMER1_INPUT_CAPTURE_INTERRUPT TIMER1_CAPT_vect
```

Definition at line 314 of file ha_m2560.h.

9.143.2.97 TIMER1_OVERFLOW_INTERRUPT

```
#define TIMER1_OVERFLOW_INTERRUPT TIMER1_OVF_vect
```

Definition at line 313 of file ha_m2560.h.

9.143.2.98 TIMER1_SELECT_CLOCK_SOURCE

```
#define TIMER1_SELECT_CLOCK_SOURCE(
              clockSource)
```

Value:

```
    do { \
        TCCR1B &= 0xF8; \
        TCCR1B |= (clockSource); \
    } while (0)
```

Definition at line 211 of file ha_m2560.h.

```
00211 #define TIMER1_SELECT_CLOCK_SOURCE(clockSource) \
00212     do { \
00213         TCCR1B &= 0xF8; \
00214         TCCR1B |= (clockSource); \
00215     } while (0)
```

Referenced by core::TimerCounter1::start().

9.143.2.99 TIMER1_SELECT_COM_CHANNEL_A

```
#define TIMER1_SELECT_COM_CHANNEL_A(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR1A &= 0x3F; \
        TCCR1A |= (compareOutputMode) « 6; \
    } while (0)
```

Definition at line 255 of file ha_m2560.h.

```
00255 #define TIMER1_SELECT_COM_CHANNEL_A(compareOutputMode) \
00256     do { \
00257         TCCR1A &= 0x3F; \
00258         TCCR1A |= (compareOutputMode) « 6; \
00259     } while (0)
```

Referenced by core::TimerCounter1::selectCompareOutputMode().

9.143.2.100 TIMER1_SELECT_COM_CHANNEL_B

```
#define TIMER1_SELECT_COM_CHANNEL_B(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR1A &= 0xCF; \
        TCCR1A |= (compareOutputMode) « 4; \
    } while (0)
```

Definition at line 260 of file ha_m2560.h.

```
00260 #define TIMER1_SELECT_COM_CHANNEL_B(compareOutputMode) \
00261     do { \
00262         TCCR1A &= 0xCF; \
00263         TCCR1A |= (compareOutputMode) « 4; \
00264     } while (0)
```

Referenced by core::TimerCounter1::selectCompareOutputMode().

9.143.2.101 TIMER1_SELECT_OPERATION_MODE

```
#define TIMER1_SELECT_OPERATION_MODE(
              operationMode)
```

Value:

```
    do { \
        TCCR1A &= 0xFC; \
        TCCR1A |= ((operationMode) & 3); \
        TCCR1B &= 0xE7; \
        TCCR1B |= (((operationMode) & 12) << 1); \
    } while (0)
```

Definition at line 229 of file ha_m2560.h.

```
00229 #define TIMER1_SELECT_OPERATION_MODE(operationMode) \
00230     do { \
00231         TCCR1A &= 0xFC; \
00232         TCCR1A |= ((operationMode) & 3); \
00233         TCCR1B &= 0xE7; \
00234         TCCR1B |= (((operationMode) & 12) << 1); \
00235     } while (0)
```

Referenced by core::TimerCounter1::selectOperationMode().

9.143.2.102 TIMER1_STOP

```
#define TIMER1_STOP
```

Value:

```
    do { \
        TCCR1B &= 0xF8; \
    } while (0)
```

Definition at line 197 of file ha_m2560.h.

```
00197 #define TIMER1_STOP \
00198     do { \
00199         TCCR1B &= 0xF8; \
00200     } while (0)
```

Referenced by core::TimerCounter1::stop().

9.143.2.103 TIMER2_COM_CHANNEL_A_INTERRUPT

```
#define TIMER2_COM_CHANNEL_A_INTERRUPT TIMER2_COMPA_vect
```

Definition at line 316 of file ha_m2560.h.

9.143.2.104 TIMER2_COM_CHANNEL_B_INTERRUPT

```
#define TIMER2_COM_CHANNEL_B_INTERRUPT TIMER2_COMPB_vect
```

Definition at line 317 of file ha_m2560.h.

9.143.2.105 TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 &= ~(1 << OCIE2A)
```

Definition at line 299 of file ha_m2560.h.

Referenced by core::TimerCounter2::enableOutputCompareMatchInterrupt().

9.143.2.106 TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 &= ~(1 << OCIE2B)
```

Definition at line 302 of file ha_m2560.h.

Referenced by core::TimerCounter2::enableOutputCompareMatchInterrupt().

9.143.2.107 TIMER2_DISABLE_OVERFLOW_INTERRUPT

```
#define TIMER2_DISABLE_OVERFLOW_INTERRUPT TIMSK2 &= ~(1 << TOIE2)
```

Definition at line 305 of file ha_m2560.h.

Referenced by core::TimerCounter2::enableOverflowInterrupt().

9.143.2.108 TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 |= 1 << OCIE2A
```

Definition at line 298 of file ha_m2560.h.

Referenced by core::TimerCounter2::enableOutputCompareMatchInterrupt().

9.143.2.109 TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 |= 1 << OCIE2B
```

Definition at line 301 of file ha_m2560.h.

Referenced by core::TimerCounter2::enableOutputCompareMatchInterrupt().

9.143.2.110 TIMER2_ENABLE_OVERFLOW_INTERRUPT

```
#define TIMER2_ENABLE_OVERFLOW_INTERRUPT TIMSK2 |= 1 << TOIE2
```

Definition at line 304 of file ha_m2560.h.

Referenced by core::TimerCounter2::enableOverflowInterrupt().

9.143.2.111 TIMER2_OVERFLOW_INTERRUPT

```
#define TIMER2_OVERFLOW_INTERRUPT TIMER2_OVF_vect
```

Definition at line 318 of file ha_m2560.h.

9.143.2.112 TIMER2_SELECT_CLOCK_SOURCE

```
#define TIMER2_SELECT_CLOCK_SOURCE(
                clockSource)
```

Value:

```
    do { \
        TCCR2B &= 0xF8; \
        TCCR2B |= (clockSource); \
    } while (0)
```

Definition at line 216 of file ha_m2560.h.

```
00216 #define TIMER2_SELECT_CLOCK_SOURCE(clockSource) \
00217     do { \
00218         TCCR2B &= 0xF8; \
00219         TCCR2B |= (clockSource); \
00220     } while (0)
```

Referenced by core::TimerCounter2::start().

9.143.2.113 TIMER2_SELECT_COM_CHANNEL_A

```
#define TIMER2_SELECT_COM_CHANNEL_A(
                compareOutputMode)
```

Value:

```
    do { \
        TCCR2A &= 0x3F; \
        TCCR2A |= (compareOutputMode) « 6; \
    } while (0)
```

Definition at line 266 of file ha_m2560.h.

```
00266 #define TIMER2_SELECT_COM_CHANNEL_A(compareOutputMode) \
00267     do { \
00268         TCCR2A &= 0x3F; \
00269         TCCR2A |= (compareOutputMode) « 6; \
00270     } while (0)
```

Referenced by core::TimerCounter2::selectCompareOutputMode().

9.143.2.114 TIMER2_SELECT_COM_CHANNEL_B

```
#define TIMER2_SELECT_COM_CHANNEL_B(
                compareOutputMode)
```

Value:

```
    do { \
        TCCR2A &= 0xCF; \
        TCCR2A |= (compareOutputMode) « 4; \
    } while (0)
```

Definition at line 271 of file ha_m2560.h.

```
00271 #define TIMER2_SELECT_COM_CHANNEL_B(compareOutputMode) \
00272     do { \
00273         TCCR2A &= 0xCF; \
00274         TCCR2A |= (compareOutputMode) « 4; \
00275     } while (0)
```

Referenced by core::TimerCounter2::selectCompareOutputMode().

9.143.2.115 TIMER2_SELECT_OPERATION_MODE

```
#define TIMER2_SELECT_OPERATION_MODE(
              operationMode)
```

Value:

```
    do { \
        TCCR2A &= 0xFC; \
        TCCR2A |= ((operationMode) & 3); \
        TCCR2B &= 0xF7; \
        TCCR2B |= (((operationMode) & 12) « 1); \
    } while (0)
```

Definition at line 236 of file ha_m2560.h.

```
00236 #define TIMER2_SELECT_OPERATION_MODE(operationMode) \
00237     do { \
00238         TCCR2A &= 0xFC; \
00239         TCCR2A |= ((operationMode) & 3); \
00240         TCCR2B &= 0xF7; \
00241         TCCR2B |= (((operationMode) & 12) « 1); \
00242     } while (0)
```

Referenced by core::TimerCounter2::selectOperationMode().

9.143.2.116 TIMER2_STOP

```
#define TIMER2_STOP
```

Value:

```
    do { \
        TCCR2B &= 0xF8; \
    } while (0)
```

Definition at line 201 of file ha_m2560.h.

```
00201 #define TIMER2_STOP \
00202     do { \
00203         TCCR2B &= 0xF8; \
00204     } while (0)
```

Referenced by core::TimerCounter2::stop().

9.143.2.117 USART0_CONTROL_STATUS_REGISTER

```
#define USART0_CONTROL_STATUS_REGISTER UCSR0A
```

Definition at line 80 of file ha_m2560.h.

9.143.2.118 USART0_DATA_OVERRUN

```
#define USART0_DATA_OVERRUN DOR0
```

Definition at line 84 of file ha_m2560.h.

9.143.2.119 USART0_DATA_REGISTER

```
#define USART0_DATA_REGISTER UDR0
```

Definition at line 81 of file ha_m2560.h.

9.143.2.120 USART0_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_DATA_REGISTER_EMPTY_INTERRUPT USART0_UDRE_vect
```

Definition at line 98 of file ha_m2560.h.

9.143.2.121 USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B &= ~(1 << UDRIE0)
```

Definition at line 88 of file ha_m2560.h.

9.143.2.122 USART0_DISABLE_DOUBLE_SPEED_MODE [1/2]

```
#define USART0_DISABLE_DOUBLE_SPEED_MODE UCSR0A &= ~(1 << U2X0)
```

Definition at line 29 of file ha_m2560.h.

9.143.2.123 USART0_DISABLE_DOUBLE_SPEED_MODE [2/2]

```
#define USART0_DISABLE_DOUBLE_SPEED_MODE
```

Definition at line 29 of file ha_m2560.h.

9.143.2.124 USART0_DISABLE_PARITY_MODE

```
#define USART0_DISABLE_PARITY_MODE UCSR0C &= ~((1 << UPM01) | (1 << UPM00))
```

Definition at line 41 of file ha_m2560.h.

9.143.2.125 USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B &= ~(1 << RXCIE0)
```

Definition at line 91 of file ha_m2560.h.

9.143.2.126 USART0_DISABLE_RECEIVER

```
#define USART0_DISABLE_RECEIVER UCSR0B &= ~(1 << RXEN0)
```

Definition at line 78 of file ha_m2560.h.

9.143.2.127 USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B &= ~(1 << TXCIE0)
```

Definition at line 94 of file ha_m2560.h.

9.143.2.128 USART0_DISABLE_TRANSMITTER

```
#define USART0_DISABLE_TRANSMITTER UCSR0B &= ~(1 << TXEN0)
```

Definition at line 75 of file ha_m2560.h.

9.143.2.129 USART0_ENABLE_ASYNC_TRANSMISSION_MODE

```
#define USART0_ENABLE_ASYNC_TRANSMISSION_MODE UCSR0C &= ~((1 << UMSEL01) | (1 << UMSEL00))
```

Definition at line 24 of file ha_m2560.h.

9.143.2.130 USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B |= 1 << UDRIE0
```

Definition at line 87 of file ha_m2560.h.

9.143.2.131 USART0_ENABLE_DOUBLE_SPEED_MODE

```
#define USART0_ENABLE_DOUBLE_SPEED_MODE
```

Definition at line 28 of file ha_m2560.h.

9.143.2.132 USART0_ENABLE_EVEN_PARITY_MODE

```
#define USART0_ENABLE_EVEN_PARITY_MODE UCSR0C = (1 << UPM01) | (UCSR0C & ~(1 << UPM00))
```

Definition at line 39 of file ha_m2560.h.

9.143.2.133 USART0_ENABLE_MASTER_SPI_MODE

```
#define USART0_ENABLE_MASTER_SPI_MODE UCSR0C |= (1 << UMSEL01) | (1 << UMSEL00)
```

Definition at line 26 of file ha_m2560.h.

9.143.2.134 USART0_ENABLE_ODD_PARITY_MODE

```
#define USART0_ENABLE_ODD_PARITY_MODE UCSR0C |= ((1 << UPM01) | (1 << UPM00))
```

Definition at line 40 of file ha_m2560.h.

9.143.2.135 USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B |= 1 << RXCIE0
```

Definition at line 90 of file ha_m2560.h.

9.143.2.136 USART0_ENABLE_RECEIVER

```
#define USART0_ENABLE_RECEIVER UCSR0B |= 1 << RXEN0
```

Definition at line 77 of file ha_m2560.h.

9.143.2.137 USART0_ENABLE_SYNC_TRANSMISSION_MODE

```
#define USART0_ENABLE_SYNC_TRANSMISSION_MODE UCSR0C = (UCSR0C & ~(1 << UMSEL01)) | (1 <<
UMSEL00)
```

Definition at line 25 of file ha_m2560.h.

9.143.2.138 USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B |= 1 << TXCIE0
```

Definition at line 93 of file ha_m2560.h.

9.143.2.139 USART0_ENABLE_TRANSMITTER

```
#define USART0_ENABLE_TRANSMITTER UCSR0B |= 1 << TXEN0
```

Definition at line 74 of file ha_m2560.h.

9.143.2.140 USART0_FRAME_ERROR

```
#define USART0_FRAME_ERROR FE0
```

Definition at line 83 of file ha_m2560.h.

9.143.2.141 USART0_PARITY_ERROR

```
#define USART0_PARITY_ERROR UPE0
```

Definition at line 85 of file ha_m2560.h.

9.143.2.142 USART0_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_RECEIVE_COMPLETE_INTERRUPT USART0_RX_vect
```

Definition at line 96 of file ha_m2560.h.

9.143.2.143 USART0_SET_5BIT_FRAME_SIZE

```
#define USART0_SET_5BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C &= ~((1 « UCSZ01) | (1 « UCSZ00)); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 68 of file ha_m2560.h.

```
00068 #define   USART0_SET_5BIT_FRAME_SIZE \
00069     do { \
00070         UCSR0C &= ~((1 « UCSZ01) | (1 « UCSZ00)); \
00071         UCSR0B &= ~(1 « UCSZ02); \
00072     } while (0)
```

9.143.2.144 USART0_SET_6BIT_FRAME_SIZE

```
#define USART0_SET_6BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C = (UCSR0C & ~(1 « UCSZ01)) | (1 « UCSZ00); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 63 of file ha_m2560.h.

```
00063 #define   USART0_SET_6BIT_FRAME_SIZE \
00064     do { \
00065         UCSR0C = (UCSR0C & ~(1 « UCSZ01)) | (1 « UCSZ00); \
00066         UCSR0B &= ~(1 « UCSZ02); \
00067     } while (0)
```

9.143.2.145 USART0_SET_7BIT_FRAME_SIZE

```
#define USART0_SET_7BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C = (1 « UCSZ01) | (UCSR0C & ~(1 « UCSZ00)); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 58 of file ha_m2560.h.

```
00058 #define   USART0_SET_7BIT_FRAME_SIZE  \
00059     do { \
00060         UCSR0C = (1 « UCSZ01) | (UCSR0C & ~(1 « UCSZ00)); \
00061         UCSR0B &= ~(1 « UCSZ02); \
00062     } while (0)
```

9.143.2.146 USART0_SET_8BIT_FRAME_SIZE

```
#define USART0_SET_8BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 52 of file ha_m2560.h.

```
00052 #define   USART0_SET_8BIT_FRAME_SIZE   \
00053     do { \
00054         UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
00055         UCSR0B &= ~(1 « UCSZ02); \
00056     } while (0)
```

9.143.2.147 USART0_SET_9BIT_FRAME_SIZE

```
#define USART0_SET_9BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
        UCSR0B |= (1 « UCSZ02); \
    } while (0)
```

Definition at line 46 of file ha_m2560.h.

```
00046 #define   USART0_SET_9BIT_FRAME_SIZE   \
00047     do { \
00048         UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
00049         UCSR0B |= (1 « UCSZ02); \
00050     } while (0)
```

9.143.2.148 USART0_SET_BAUDRATE_HIGH_REGISTER

```
#define USART0_SET_BAUDRATE_HIGH_REGISTER UBRR0H = UBRRH_VALUE
```

Definition at line 21 of file ha_m2560.h.

9.143.2.149 USART0_SET_BAUDRATE_LOW_REGISTER

```
#define USART0_SET_BAUDRATE_LOW_REGISTER UBRR0L = UBRRL_VALUE
```

Definition at line 22 of file ha_m2560.h.

9.143.2.150 USART0_SET_ONE_STOP_BIT

```
#define USART0_SET_ONE_STOP_BIT UCSR0C &= ~(1 << USBS0)
```

Definition at line 43 of file ha_m2560.h.

9.143.2.151 USART0_SET_TWO_STOP_BITS

```
#define USART0_SET_TWO_STOP_BITS UCSR0C |= (1 << USBS0)
```

Definition at line 44 of file ha_m2560.h.

9.143.2.152 USART0_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_TRANSMIT_COMPLETE_INTERRUPT USART0_TX_vect
```

Definition at line 97 of file ha_m2560.h.

9.143.2.153 WATCHDOG_SELECT_TIMEOUT

```
#define WATCHDOG_SELECT_TIMEOUT(
                timeOut)
```

Value:

```
    do { \
        WDTCSR |= (1<<WDCE) | (1<<WDE); \
        WDTCSR = (0<<WDIE) | (0<<WDE) | (timeOut); \
    } while (0)
```

Definition at line 321 of file ha_m2560.h.

```
00321 #define WATCHDOG_SELECT_TIMEOUT(timeOut) \
00322     do { \
00323         WDTCSR |= (1<<WDCE) | (1<<WDE); \
00324         WDTCSR = (0<<WDIE) | (0<<WDE) | (timeOut); \
00325     } while (0)
```

9.143.2.154 WATCHDOG_START

```
#define WATCHDOG_START(
                operationMode,
                timeOut)
```

Value:

```
    do { \
        MCUSR &= ~(1<<WDRF); \
        WDTCSR |= (1<<WDCE) | (1<<WDE); \
        WDTCSR = (operationMode) | (timeOut); \
    } while (0)
```

Definition at line 326 of file ha_m2560.h.

```
00326 #define WATCHDOG_START(operationMode,timeOut) \
00327     do { \
00328         MCUSR &= ~(1<<WDRF); \
00329         WDTCSR |= (1<<WDCE) | (1<<WDE); \
00330         WDTCSR = (operationMode) | (timeOut); \
00331     } while (0)
```

9.143.2.155 WATCHDOG_STOP

```
#define WATCHDOG_STOP
```

Value:

```
    do { \
        MCUSR &= ~(1<<WDRF); \
        WDTCSR |= (1<<WDCE) | (1<<WDE); \
        WDTCSR = 0x00; \
    } while (0)
```

Definition at line 332 of file ha_m2560.h.

```
00332 #define WATCHDOG_STOP \
00333     do { \
00334         MCUSR &= ~(1<<WDRF); \
00335         WDTCSR |= (1<<WDCE) | (1<<WDE); \
00336         WDTCSR = 0x00; \
00337     } while (0)
```

9.143.2.156 WATCHDOG_TIMEOUT_INTERRUPT

```
#define WATCHDOG_TIMEOUT_INTERRUPT WDT_vect
```

Definition at line 338 of file ha_m2560.h.

9.144 ha_m2560.h

Go to the documentation of this file.

```
00001
00008 #ifndef HA_M2560_H
00009 #define HA_M2560_H
00010
00011 #ifndef HABASE_H
00012     #error "don't use this file directly! Please include only ha_base.h!"
00013 #endif
00014
00015 // PushButton defines
00016 #define PUSHBUTTON_DEBOUNCE_TIME_US 1000
00017
00018 // USART defines
00019 #include <util/setbaud.h>
00020
00021 #define   USART0_SET_BAUDRATE_HIGH_REGISTER     UBRR0H = UBRRH_VALUE
00022 #define   USART0_SET_BAUDRATE_LOW_REGISTER      UBRR0L = UBRRL_VALUE
00023
00024 #define   USART0_ENABLE_ASYNC_TRANSMISSION_MODE     UCSR0C &= ~((1 << UMSEL01) | (1 << UMSEL00))
00025 #define   USART0_ENABLE_SYNC_TRANSMISSION_MODE      UCSR0C = (UCSR0C & ~(1 << UMSEL01)) | (1 << UMSEL00)
00026 #define   USART0_ENABLE_MASTER_SPI_MODE             UCSR0C |= (1 << UMSEL01) | (1 << UMSEL00)
00027
00028 #define   USART0_ENABLE_DOUBLE_SPEED_MODE
00029 #define   USART0_DISABLE_DOUBLE_SPEED_MODE
00030
00031 #if USE_2X
00032     #undef  USART0_ENABLE_DOUBLE_SPEED_MODE
00033     #define USART0_ENABLE_DOUBLE_SPEED_MODE UCSR0A |= (1 << U2X0)
00034 #else
00035     #undef  USART0_DISABLE_DOUBLE_SPEED_MODE
00036     #define USART0_DISABLE_DOUBLE_SPEED_MODE UCSR0A &= ~(1 << U2X0)
00037 #endif
00038
00039 #define   USART0_ENABLE_EVEN_PARITY_MODE    UCSR0C = (1 << UPM01) | (UCSR0C & ~(1 << UPM00))
00040 #define   USART0_ENABLE_ODD_PARITY_MODE     UCSR0C |=   ((1 << UPM01) | (1 << UPM00))
00041 #define   USART0_DISABLE_PARITY_MODE        UCSR0C &= ~((1 << UPM01) | (1 << UPM00))
00042
00043 #define   USART0_SET_ONE_STOP_BIT   UCSR0C &= ~(1 << USBS0)
00044 #define   USART0_SET_TWO_STOP_BITS  UCSR0C |= (1 << USBS0)
00045
00046 #define   USART0_SET_9BIT_FRAME_SIZE   \
00047     do { \
00048         UCSR0C |= ((1 << UCSZ01) | (1 << UCSZ00)); \
00049         UCSR0B |= (1 << UCSZ02); \
00050     } while (0)
00051
00052 #define   USART0_SET_8BIT_FRAME_SIZE   \
00053     do { \
00054         UCSR0C |= ((1 << UCSZ01) | (1 << UCSZ00)); \
00055         UCSR0B &= ~(1 << UCSZ02); \
00056     } while (0)
00057
00058 #define   USART0_SET_7BIT_FRAME_SIZE  \
00059     do { \
00060         UCSR0C = (1 << UCSZ01) | (UCSR0C & ~(1 << UCSZ00)); \
00061         UCSR0B &= ~(1 << UCSZ02); \
00062     } while (0)
00063 #define   USART0_SET_6BIT_FRAME_SIZE \
00064     do { \
00065         UCSR0C = (UCSR0C & ~(1 << UCSZ01)) | (1 << UCSZ00); \
00066         UCSR0B &= ~(1 << UCSZ02); \
00067     } while (0)
00068 #define   USART0_SET_5BIT_FRAME_SIZE \
00069     do { \
00070         UCSR0C &= ~((1 << UCSZ01) | (1 << UCSZ00)); \
00071         UCSR0B &= ~(1 << UCSZ02); \
00072     } while (0)
00073
00074 #define   USART0_ENABLE_TRANSMITTER      UCSR0B |=   1 << TXEN0
00075 #define   USART0_DISABLE_TRANSMITTER     UCSR0B &= ~(1 << TXEN0)
00076
00077 #define   USART0_ENABLE_RECEIVER         UCSR0B |=   1 << RXEN0
00078 #define   USART0_DISABLE_RECEIVER        UCSR0B &= ~(1 << RXEN0)
00079
00080 #define   USART0_CONTROL_STATUS_REGISTER UCSR0A
00081 #define   USART0_DATA_REGISTER           UDR0
00082
00083 #define   USART0_FRAME_ERROR             FE0
00084 #define   USART0_DATA_OVERRUN            DOR0
00085 #define   USART0_PARITY_ERROR            UPE0
00086
00087 #define   USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT   UCSR0B |=   1 << UDRIE0
00088 #define   USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT  UCSR0B &= ~(1 << UDRIE0)
```

```
00089
00090 #define   USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT   UCSR0B |=   1 « RXCIE0
00091 #define   USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT  UCSR0B &= ~(1 « RXCIE0)
00092
00093 #define   USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT   UCSR0B |=   1 « TXCIE0
00094 #define   USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT  UCSR0B &= ~(1 « TXCIE0)
00095
00096 #define   USART0_RECEIVE_COMPLETE_INTERRUPT     USART0_RX_vect
00097 #define   USART0_TRANSMIT_COMPLETE_INTERRUPT    USART0_TX_vect
00098 #define   USART0_DATA_REGISTER_EMPTY_INTERRUPT  USART0_UDRE_vect
00099
00100 // ADC defines
00101 #define   ADC_SELECT_REF_VOLTAGE(refVoltage) \
00102     do { \
00103         ADMUX &= 0x3F; \
00104         ADMUX |= (refVoltage) « 6; \
00105     } while (0)
00106
00107 #define   ADC_ADJUST_RESULT_LEFT            ADMUX |=  1 « ADLAR
00108 #define   ADC_ADJUST_RESULT_RIGHT           ADMUX &= ~(1 « ADLAR)
00109
00110 #define   ADC_SELECT_ANALOG_INPUT(pinNumber) \
00111     do { \
00112         ADMUX &= 0xE0; \
00113         ADCSRB &= ~(1 « MUX5); \
00114         if ((pinNumber) & 8) { \
00115             ADCSRB |= (1 « MUX5); \
00116         } \
00117         ADMUX |= ((pinNumber) & 7); \
00118     } while (0)
00119
00120 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) \
00121     do { \
00122         if ((pinNumber) < 8) { \
00123             DIDR0 |= (1 « (pinNumber)); \
00124         } else { \
00125             DIDR2 |= (1 « ((pinNumber) - 8)); \
00126         } \
00127     } while (0)
00128
00129 #define   ADC_ENABLE            ADCSRA |=  1 « ADEN
00130 #define   ADC_DISABLE           ADCSRA &= ~(1 « ADEN)
00131
00132 #define   ADC_START_CONVERSION            ADCSRA |=  1 « ADSC
00133 #define   ADC_STOP_CONVERSION           ADCSRA &= ~(1 « ADSC)
00134
00135 #define   ADC_ENABLE_AUTOTRIGGER          ADCSRA |=   1 « ADATE
00136 #define   ADC_DISABLE_AUTOTRIGGER        ADCSRA &= ~(1 « ADATE)
00137
00138 #define   ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT   ADCSRA |=   1 « ADIE
00139 #define   ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT  ADCSRA &= ~(1 « ADIE)
00140
00141 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00142     do { \
00143         ADCSRA &= 0xF8; \
00144         ADCSRA |= (clockPrescaler); \
00145     } while (0)
00146
00147 #define   ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource) \
00148     do { \
00149         ADCSRB &= 0xF8; \
00150         ADCSRB |= (triggerSource); \
00151     } while (0)
00152
00153 #define   ADC_CONVERSION_COMPLETE_INTERRUPT  ADC_vect
00154
00155 // Externally Triggered Interrupts defines
00156 #define EXT_INT_SET_INT0_SENSE_CONTROL(senseControl) \
00157     do { \
00158         EICRA &= 0xFC; \
00159         EICRA |= (senseControl); \
00160     } while (0)
00161 #define EXT_INT_SET_INT1_SENSE_CONTROL(senseControl) \
00162     do { \
00163         EICRA &= 0xF3; \
00164         EICRA |= (senseControl) « 2; \
00165     } while (0)
00166
00167 #define EXT_INT_ENABLE_INT0   EIMSK |= 1 « INT0
00168 #define EXT_INT_DISABLE_INT0   EIMSK &= ~(1 « INT0)
00169
00170 #define EXT_INT_ENABLE_INT1   EIMSK |= 1 « INT1
00171 #define EXT_INT_DISABLE_INT1   EIMSK &= ~(1 « INT1)
00172
00173 #define EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(pinChangePort)    PCICR |= 1 « (pinChangePort)
00174 #define EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(pinChangePort)   PCICR &= ~(1 « (pinChangePort))
00175
```

```
00176 #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)    PCMSK0 |= 1 « (pinChangePin)
00177 #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)   PCMSK0 &= ~(1 « (pinChangePin))
00178
00179 #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)    PCMSK1 |= 1 « (pinChangePin)
00180 #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)   PCMSK1 &= ~(1 « (pinChangePin))
00181
00182 #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)    PCMSK2 |= 1 « (pinChangePin)
00183 #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)   PCMSK2 &= ~(1 « (pinChangePin))
00184
00185 #define  EXT_INT_INT0_INTERRUPT   INT0_vect
00186 #define  EXT_INT_INT1_INTERRUPT   INT1_vect
00187
00188 #define  EXT_INT_PIN_CHANGE_PORTB_INTERRUPT   PCINT0_vect
00189 #define  EXT_INT_PIN_CHANGE_PORTC_INTERRUPT   PCINT1_vect
00190 #define  EXT_INT_PIN_CHANGE_PORTD_INTERRUPT   PCINT2_vect
00191
00192 // Timers/Counters defines
00193 #define TIMER0_STOP \
00194     do { \
00195         TCCR0B &= 0xF8; \
00196     } while (0)
00197 #define TIMER1_STOP \
00198     do { \
00199         TCCR1B &= 0xF8; \
00200     } while (0)
00201 #define TIMER2_STOP \
00202     do { \
00203         TCCR2B &= 0xF8; \
00204     } while (0)
00205
00206 #define TIMER0_SELECT_CLOCK_SOURCE(clockSource) \
00207     do { \
00208         TCCR0B &= 0xF8; \
00209         TCCR0B |= (clockSource); \
00210     } while (0)
00211 #define TIMER1_SELECT_CLOCK_SOURCE(clockSource) \
00212     do { \
00213         TCCR1B &= 0xF8; \
00214         TCCR1B |= (clockSource); \
00215     } while (0)
00216 #define TIMER2_SELECT_CLOCK_SOURCE(clockSource) \
00217     do { \
00218         TCCR2B &= 0xF8; \
00219         TCCR2B |= (clockSource); \
00220     } while (0)
00221
00222 #define TIMER0_SELECT_OPERATION_MODE(operationMode) \
00223     do { \
00224         TCCR0A &= 0xFC; \
00225         TCCR0A |= ((operationMode) & 3); \
00226         TCCR0B &= 0xF7; \
00227         TCCR0B |= (((operationMode) & 12) « 1); \
00228     } while (0)
00229 #define TIMER1_SELECT_OPERATION_MODE(operationMode) \
00230     do { \
00231         TCCR1A &= 0xFC; \
00232         TCCR1A |= ((operationMode) & 3); \
00233         TCCR1B &= 0xE7; \
00234         TCCR1B |= (((operationMode) & 12) « 1); \
00235     } while (0)
00236 #define TIMER2_SELECT_OPERATION_MODE(operationMode) \
00237     do { \
00238         TCCR2A &= 0xFC; \
00239         TCCR2A |= ((operationMode) & 3); \
00240         TCCR2B &= 0xF7; \
00241         TCCR2B |= (((operationMode) & 12) « 1); \
00242     } while (0)
00243
00244 #define TIMER0_SELECT_COM_CHANNEL_A(compareOutputMode) \
00245     do { \
00246         TCCR0A &= 0x3F; \
00247         TCCR0A |= (compareOutputMode) « 6; \
00248     } while (0)
00249 #define TIMER0_SELECT_COM_CHANNEL_B(compareOutputMode) \
00250     do { \
00251         TCCR0A &= 0xCF; \
00252         TCCR0A |= (compareOutputMode) « 4; \
00253     } while (0)
00254
00255 #define TIMER1_SELECT_COM_CHANNEL_A(compareOutputMode) \
00256     do { \
00257         TCCR1A &= 0x3F; \
00258         TCCR1A |= (compareOutputMode) « 6; \
00259     } while (0)
00260 #define TIMER1_SELECT_COM_CHANNEL_B(compareOutputMode) \
00261     do { \
00262         TCCR1A &= 0xCF; \
```

```
00263         TCCR1A |= (compareOutputMode) « 4; \
00264     } while (0)
00265
00266 #define TIMER2_SELECT_COM_CHANNEL_A(compareOutputMode) \
00267     do { \
00268         TCCR2A &= 0x3F; \
00269         TCCR2A |= (compareOutputMode) « 6; \
00270     } while (0)
00271 #define TIMER2_SELECT_COM_CHANNEL_B(compareOutputMode) \
00272     do { \
00273         TCCR2A &= 0xCF; \
00274         TCCR2A |= (compareOutputMode) « 4; \
00275     } while (0)
00276
00277 #define TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 |=  1 « OCIE0A
00278 #define TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 &= ~(1 « OCIE0A)
00279
00280 #define TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 |=  1 « OCIE0B
00281 #define TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 &= ~(1 « OCIE0B)
00282
00283 #define TIMER0_ENABLE_OVERFLOW_INTERRUPT TIMSK0 |=  1 « TOIE0
00284 #define TIMER0_DISABLE_OVERFLOW_INTERRUPT TIMSK0 &= ~(1 « TOIE0)
00285
00286 #define TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 |=  1 « OCIE1A
00287 #define TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 &= ~(1 « OCIE1A)
00288
00289 #define TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 |=  1 « OCIE1B
00290 #define TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 &= ~(1 « OCIE1B)
00291
00292 #define TIMER1_ENABLE_OVERFLOW_INTERRUPT TIMSK1 |=  1 « TOIE1
00293 #define TIMER1_DISABLE_OVERFLOW_INTERRUPT TIMSK1 &= ~(1 « TOIE1)
00294
00295 #define TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 |=  1 « ICIE1
00296 #define TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 &= ~(1 « ICIE1)
00297
00298 #define TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 |=  1 « OCIE2A
00299 #define TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 &= ~(1 « OCIE2A)
00300
00301 #define TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 |=  1 « OCIE2B
00302 #define TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 &= ~(1 « OCIE2B)
00303
00304 #define TIMER2_ENABLE_OVERFLOW_INTERRUPT TIMSK2 |=  1 « TOIE2
00305 #define TIMER2_DISABLE_OVERFLOW_INTERRUPT TIMSK2 &= ~(1 « TOIE2)
00306
00307 #define   TIMER0_COM_CHANNEL_A_INTERRUPT  TIMER0_COMPA_vect
00308 #define   TIMER0_COM_CHANNEL_B_INTERRUPT  TIMER0_COMPB_vect
00309 #define   TIMER0_OVERFLOW_INTERRUPT  TIMER0_OVF_vect
00310
00311 #define   TIMER1_COM_CHANNEL_A_INTERRUPT  TIMER1_COMPA_vect
00312 #define   TIMER1_COM_CHANNEL_B_INTERRUPT  TIMER1_COMPB_vect
00313 #define   TIMER1_OVERFLOW_INTERRUPT  TIMER1_OVF_vect
00314 #define   TIMER1_INPUT_CAPTURE_INTERRUPT  TIMER1_CAPT_vect
00315
00316 #define   TIMER2_COM_CHANNEL_A_INTERRUPT  TIMER2_COMPA_vect
00317 #define   TIMER2_COM_CHANNEL_B_INTERRUPT  TIMER2_COMPB_vect
00318 #define   TIMER2_OVERFLOW_INTERRUPT  TIMER2_OVF_vect
00319
00320 // Watchdog Timer defines
00321 #define WATCHDOG_SELECT_TIMEOUT(timeOut) \
00322     do { \
00323         WDTCSR |= (1«WDCE) | (1«WDE); \
00324         WDTCSR = (0«WDIE) | (0«WDE) | (timeOut); \
00325     } while (0)
00326 #define WATCHDOG_START(operationMode,timeOut) \
00327     do { \
00328         MCUSR &= ~(1«WDRF); \
00329         WDTCSR |= (1«WDCE) | (1«WDE); \
00330         WDTCSR = (operationMode) | (timeOut); \
00331     } while (0)
00332 #define WATCHDOG_STOP \
00333     do { \
00334         MCUSR &= ~(1«WDRF); \
00335         WDTCSR |= (1«WDCE) | (1«WDE); \
00336         WDTCSR = 0x00; \
00337     } while (0)
00338 #define WATCHDOG_TIMEOUT_INTERRUPT  WDT_vect
00339
00340 // MCU defines
00341 #define MCU_SELECT_SLEEP_MODE(sleepMode) \
00342     do { \
00343         SMCR &= 0xF1; \
00344         SMCR |= (sleepMode) « 1; \
00345     } while (0)
00346
00347 #define MCU_SLEEP_ENABLE SMCR |=  1 « SE
00348 #define MCU_SLEEP_DISABLE SMCR &= ~(1 « SE)
00349
```

```
00350 #define MCU_TWI_ENABLE PRR0 &= ~(1 « PRTWI)
00351 #define MCU_TWI_DISABLE PRR0 |=  1 « PRTWI
00352
00353 #define MCU_TIMER2_ENABLE PRR0 &= ~(1 « PRTIM2)
00354 #define MCU_TIMER2_DISABLE PRR0 |=  1 « PRTIM2
00355
00356 #define MCU_TIMER1_ENABLE PRR0 &= ~(1 « PRTIM1)
00357 #define MCU_TIMER1_DISABLE PRR0 |=  1 « PRTIM1
00358
00359 #define MCU_TIMER0_ENABLE PRR0 &= ~(1 « PRTIM0)
00360 #define MCU_TIMER0_DISABLE PRR0 |=  1 « PRTIM0
00361
00362 #define MCU_SPI_ENABLE PRR0 &= ~(1 « PRSPI)
00363 #define MCU_SPI_DISABLE PRR0 |=  1 « PRSPI
00364
00365 #define MCU_USART0_ENABLE PRR0 &= ~(1 « PRUSART0)
00366 #define MCU_USART0_DISABLE PRR0 |=  1 « PRUSART0
00367
00368 #define MCU_ADC_ENABLE PRR0 &= ~(1 « PRADC)
00369 #define MCU_ADC_DISABLE PRR0 |=  1 « PRADC
00370
00371 // ServoMotor defines
00372 #define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(pulseWidth,clockPrescaler) \
00373     ((((uint32_t)(pulseWidth)) * ((F_CPU) / 1000000UL)) / (clockPrescaler))
00374
00375 #define SERVOMOTOR_TIMER_ANGLE_COUNT(angle,out_min,out_mid,out_max) (((((out_min)*((angle) -
      90L)*((angle) - 180L))/16200L) + (((angle)*(out_max)*((angle) - 90L))/16200L) -
      (((angle)*(out_mid)*((angle) - 180L))/8100L)))
00376
00377 // SPI defines
00378 #define   SPI_ENABLE            SPCR |=  1 « SPE
00379 #define   SPI_DISABLE           SPCR &= ~(1 « SPE)
00380 #define   SPI_SELECT_MASTER_MODE SPCR |=  1 « MSTR
00381 #define   SPI_SELECT_SLAVE_MODE SPCR &= ~(1 « MSTR)
00382
00383 #define   SPI_SELECT_DATA_MODE(dataMode) \
00384     do { \
00385         SPCR &= 0xF3; \
00386         SPCR |= (dataMode) « 2; \
00387     } while (0)
00388 #define   SPI_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00389     do { \
00390         SPCR &= 0xFC; \
00391         SPCR |= ((clockPrescaler) & 3); \
00392         SPSR &= 0xFE; \
00393         SPSR |= (((clockPrescaler) & 4) » 2); \
00394     } while (0)
00395 #define   SPI_SELECT_DATA_ORDER(dataOrder) \
00396     do { \
00397         SPCR &= 0xDF; \
00398         SPCR |= (dataOrder) « 5; \
00399     } while (0)
00400
00401 #define   SPI_WRITE_COLLISION WCOL
00402 #define   SPI_TRANSFER_COMPLETE SPIF
00403
00404 #define   SPI_MASTER_MODE MSTR
00405 #define   SPI_CONTROL_REGISTER SPCR
00406 #define   SPI_DATA_REGISTER SPDR
00407 #define   SPI_STATUS_REGISTER SPSR
00408
00409 #define   SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT SPCR |= 1 « SPIE
00410 #define   SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT SPCR &= ~(1 « SPIE)
00411
00412 #define   SPI_TRANSFER_COMPLETE_INTERRUPT SPI_STC_vect
00413
00414 #endif
```

9.145 ha_m328p.h File Reference

Header file for the hardware abstraction macros of the Atmega328p.

```
#include <util/setbaud.h>
```

Macros

- #define PUSHBUTTON_DEBOUNCE_TIME_US 1000

- #define USART0_SET_BAUDRATE_HIGH_REGISTER UBRR0H = UBRRH_VALUE
- #define USART0_SET_BAUDRATE_LOW_REGISTER UBRR0L = UBRRL_VALUE
- #define USART0_ENABLE_ASYNC_TRANSMISSION_MODE UCSR0C &= ~((1 << UMSEL01) | (1 << UMSEL00))
- #define USART0_ENABLE_SYNC_TRANSMISSION_MODE UCSR0C = (UCSR0C & ~(1 << UMSEL01)) | (1 << UMSEL00)
- #define USART0_ENABLE_MASTER_SPI_MODE UCSR0C |= (1 << UMSEL01) | (1 << UMSEL00)
- #define USART0_ENABLE_DOUBLE_SPEED_MODE
- #define USART0_DISABLE_DOUBLE_SPEED_MODE
- #define USART0_DISABLE_DOUBLE_SPEED_MODE UCSR0A &= ~(1 << U2X0)
- #define USART0_ENABLE_EVEN_PARITY_MODE UCSR0C = (1 << UPM01) | (UCSR0C & ~(1 << UPM00))
- #define USART0_ENABLE_ODD_PARITY_MODE UCSR0C |= ((1 << UPM01) | (1 << UPM00))
- #define USART0_DISABLE_PARITY_MODE UCSR0C &= ~((1 << UPM01) | (1 << UPM00))
- #define USART0_SET_ONE_STOP_BIT UCSR0C &= ~(1 << USBS0)
- #define USART0_SET_TWO_STOP_BITS UCSR0C |= (1 << USBS0)
- #define USART0_SET_9BIT_FRAME_SIZE
- #define USART0_SET_8BIT_FRAME_SIZE
- #define USART0_SET_7BIT_FRAME_SIZE
- #define USART0_SET_6BIT_FRAME_SIZE
- #define USART0_SET_5BIT_FRAME_SIZE
- #define USART0_ENABLE_TRANSMITTER UCSR0B |= 1 << TXEN0
- #define USART0_DISABLE_TRANSMITTER UCSR0B &= ~(1 << TXEN0)
- #define USART0_ENABLE_RECEIVER UCSR0B |= 1 << RXEN0
- #define USART0_DISABLE_RECEIVER UCSR0B &= ~(1 << RXEN0)
- #define USART0_CONTROL_STATUS_REGISTER UCSR0A
- #define USART0_DATA_REGISTER UDR0
- #define USART0_FRAME_ERROR FE0
- #define USART0_DATA_OVERRUN DOR0
- #define USART0_PARITY_ERROR UPE0
- #define USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B |= 1 << UDRIE0
- #define USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B &= ~(1 << UDRIE0)
- #define USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B |= 1 << RXCIE0
- #define USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B &= ~(1 << RXCIE0)
- #define USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B |= 1 << TXCIE0
- #define USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B &= ~(1 << TXCIE0)
- #define USART0_RECEIVE_COMPLETE_INTERRUPT USART_RX_vect
- #define USART0_TRANSMIT_COMPLETE_INTERRUPT USART_TX_vect
- #define USART0_DATA_REGISTER_EMPTY_INTERRUPT USART_UDRE_vect
- #define ADC_SELECT_REF_VOLTAGE(refVoltage)
- #define ADC_ADJUST_RESULT_LEFT ADMUX |= 1 << ADLAR
- #define ADC_ADJUST_RESULT_RIGHT ADMUX &= ~(1 << ADLAR)
- #define ADC_SELECT_ANALOG_INPUT(pinNumber)
- #define ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber)
- #define ADC_ENABLE ADCSRA |= 1 << ADEN
- #define ADC_DISABLE ADCSRA &= ~(1 << ADEN)
- #define ADC_START_CONVERSION ADCSRA |= 1 << ADSC
- #define ADC_STOP_CONVERSION ADCSRA &= ~(1 << ADSC)
- #define ADC_ENABLE_AUTOTRIGGER ADCSRA |= 1 << ADATE
- #define ADC_DISABLE_AUTOTRIGGER ADCSRA &= ~(1 << ADATE)
- #define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA |= 1 << ADIE
- #define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA &= ~(1 << ADIE)
- #define ADC_SELECT_CLOCK_PRESCALER(clockPrescaler)
- #define ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource)
- #define ADC_CONVERSION_COMPLETE_INTERRUPT ADC_vect

- #define EXT_INT_SET_INT0_SENSE_CONTROL(senseControl)
- #define EXT_INT_SET_INT1_SENSE_CONTROL(senseControl)
- #define EXT_INT_ENABLE_INT0 EIMSK |= 1 << INT0
- #define EXT_INT_DISABLE_INT0 EIMSK &= ~(1 << INT0)
- #define EXT_INT_ENABLE_INT1 EIMSK |= 1 << INT1
- #define EXT_INT_DISABLE_INT1 EIMSK &= ~(1 << INT1)
- #define EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(pinChangePort)
- #define EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(pinChangePort)
- #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)
- #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)
- #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)
- #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)
- #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)
- #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)
- #define EXT_INT_INT0_INTERRUPT INT0_vect
- #define EXT_INT_INT1_INTERRUPT INT1_vect
- #define EXT_INT_PIN_CHANGE_PORTB_INTERRUPT PCINT0_vect
- #define EXT_INT_PIN_CHANGE_PORTC_INTERRUPT PCINT1_vect
- #define EXT_INT_PIN_CHANGE_PORTD_INTERRUPT PCINT2_vect
- #define TIMER0_STOP
- #define TIMER1_STOP
- #define TIMER2_STOP
- #define TIMER0_SELECT_CLOCK_SOURCE(clockSource)
- #define TIMER1_SELECT_CLOCK_SOURCE(clockSource)
- #define TIMER2_SELECT_CLOCK_SOURCE(clockSource)
- #define TIMER0_SELECT_OPERATION_MODE(operationMode)
- #define TIMER1_SELECT_OPERATION_MODE(operationMode)
- #define TIMER2_SELECT_OPERATION_MODE(operationMode)
- #define TIMER0_SELECT_COM_CHANNEL_A(compareOutputMode)
- #define TIMER0_SELECT_COM_CHANNEL_B(compareOutputMode)
- #define TIMER1_SELECT_COM_CHANNEL_A(compareOutputMode)
- #define TIMER1_SELECT_COM_CHANNEL_B(compareOutputMode)
- #define TIMER2_SELECT_COM_CHANNEL_A(compareOutputMode)
- #define TIMER2_SELECT_COM_CHANNEL_B(compareOutputMode)
- #define TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 |= 1 << OCIE0A
- #define TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 &= ~(1 << OCIE0A)
- #define TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 |= 1 << OCIE0B
- #define TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 &= ~(1 << OCIE0B)
- #define TIMER0_ENABLE_OVERFLOW_INTERRUPT TIMSK0 |= 1 << TOIE0
- #define TIMER0_DISABLE_OVERFLOW_INTERRUPT TIMSK0 &= ~(1 << TOIE0)
- #define TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 |= 1 << OCIE1A
- #define TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 &= ~(1 << OCIE1A)
- #define TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 |= 1 << OCIE1B
- #define TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 &= ~(1 << OCIE1B)
- #define TIMER1_ENABLE_OVERFLOW_INTERRUPT TIMSK1 |= 1 << TOIE1
- #define TIMER1_DISABLE_OVERFLOW_INTERRUPT TIMSK1 &= ~(1 << TOIE1)
- #define TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 |= 1 << ICIE1
- #define TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 &= ~(1 << ICIE1)
- #define TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 |= 1 << OCIE2A
- #define TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 &= ~(1 << OCIE2A)
- #define TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 |= 1 << OCIE2B
- #define TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 &= ~(1 << OCIE2B)
- #define TIMER2_ENABLE_OVERFLOW_INTERRUPT TIMSK2 |= 1 << TOIE2
- #define TIMER2_DISABLE_OVERFLOW_INTERRUPT TIMSK2 &= ~(1 << TOIE2)
- #define TIMER0_COM_CHANNEL_A_INTERRUPT TIMER0_COMPA_vect

- #define TIMER0_COM_CHANNEL_B_INTERRUPT TIMER0_COMPB_vect
- #define TIMER0_OVERFLOW_INTERRUPT TIMER0_OVF_vect
- #define TIMER1_COM_CHANNEL_A_INTERRUPT TIMER1_COMPA_vect
- #define TIMER1_COM_CHANNEL_B_INTERRUPT TIMER1_COMPB_vect
- #define TIMER1_OVERFLOW_INTERRUPT TIMER1_OVF_vect
- #define TIMER1_INPUT_CAPTURE_INTERRUPT TIMER1_CAPT_vect
- #define TIMER2_COM_CHANNEL_A_INTERRUPT TIMER2_COMPA_vect
- #define TIMER2_COM_CHANNEL_B_INTERRUPT TIMER2_COMPB_vect
- #define TIMER2_OVERFLOW_INTERRUPT TIMER2_OVF_vect
- #define WATCHDOG_SELECT_TIMEOUT(timeOut)
- #define WATCHDOG_START(operationMode, timeOut)
- #define WATCHDOG_STOP
- #define WATCHDOG_TIMEOUT_INTERRUPT WDT_vect
- #define MCU_SELECT_SLEEP_MODE(sleepMode)
- #define MCU_SLEEP_ENABLE SMCR |= 1 << SE
- #define MCU_SLEEP_DISABLE SMCR &= ~(1 << SE)
- #define MCU_TWI_ENABLE PRR &= ~(1 << PRTWI)
- #define MCU_TWI_DISABLE PRR |= 1 << PRTWI
- #define MCU_TIMER2_ENABLE PRR &= ~(1 << PRTIM2)
- #define MCU_TIMER2_DISABLE PRR |= 1 << PRTIM2
- #define MCU_TIMER1_ENABLE PRR &= ~(1 << PRTIM1)
- #define MCU_TIMER1_DISABLE PRR |= 1 << PRTIM1
- #define MCU_TIMER0_ENABLE PRR &= ~(1 << PRTIM0)
- #define MCU_TIMER0_DISABLE PRR |= 1 << PRTIM0
- #define MCU_SPI_ENABLE PRR &= ~(1 << PRSPI)
- #define MCU_SPI_DISABLE PRR |= 1 << PRSPI
- #define MCU_USART0_ENABLE PRR &= ~(1 << PRUSART0)
- #define MCU_USART0_DISABLE PRR |= 1 << PRUSART0
- #define MCU_ADC_ENABLE PRR &= ~(1 << PRADC)
- #define MCU_ADC_DISABLE PRR |= 1 << PRADC
- #define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(pulseWidth, clockPrescaler)
- #define SERVOMOTOR_TIMER_ANGLE_COUNT(angle, out_min, out_mid, out_max)
- #define SPI_ENABLE SPCR |= 1 << SPE
- #define SPI_DISABLE SPCR &= ~(1 << SPE)
- #define SPI_SELECT_MASTER_MODE SPCR |= 1 << MSTR
- #define SPI_SELECT_SLAVE_MODE SPCR &= ~(1 << MSTR)
- #define SPI_SELECT_DATA_MODE(dataMode)
- #define SPI_SELECT_CLOCK_PRESCALER(clockPrescaler)
- #define SPI_SELECT_DATA_ORDER(dataOrder)
- #define SPI_WRITE_COLLISION WCOL
- #define SPI_TRANSFER_COMPLETE SPIF
- #define SPI_MASTER_MODE MSTR
- #define SPI_CONTROL_REGISTER SPCR
- #define SPI_DATA_REGISTER SPDR
- #define SPI_STATUS_REGISTER SPSR
- #define SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT SPCR |= 1 << SPIE
- #define SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT SPCR &= ~(1 << SPIE)
- #define SPI_TRANSFER_COMPLETE_INTERRUPT SPI_STC_vect

9.145.1 Detailed Description

Header file for the hardware abstraction macros of the Atmega328p.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2020

Definition in file ha_m328p.h.

9.145.2 Macro Definition Documentation

9.145.2.1 ADC_ADJUST_RESULT_LEFT

```
#define ADC_ADJUST_RESULT_LEFT ADMUX |= 1 << ADLAR
```

Definition at line 131 of file ha_m328p.h.

9.145.2.2 ADC_ADJUST_RESULT_RIGHT

```
#define ADC_ADJUST_RESULT_RIGHT ADMUX &= ~(1 << ADLAR)
```

Definition at line 132 of file ha_m328p.h.

9.145.2.3 ADC_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_CONVERSION_COMPLETE_INTERRUPT ADC_vect
```

Definition at line 178 of file ha_m328p.h.

9.145.2.4 ADC_DISABLE

```
#define ADC_DISABLE ADCSRA &= ~(1 << ADEN)
```

Definition at line 149 of file ha_m328p.h.

9.145.2.5 ADC_DISABLE_AUTOTRIGGER

```
#define ADC_DISABLE_AUTOTRIGGER ADCSRA &= ~(1 << ADATE)
```

Definition at line 159 of file ha_m328p.h.

9.145.2.6 ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA &= ~(1 << ADIE)
```

Definition at line 162 of file ha_m328p.h.

9.145.2.7 ADC_DISABLE_DIGITAL_INPUT_REGISTER

```
#define ADC_DISABLE_DIGITAL_INPUT_REGISTER(
              pinNumber)
```

Value:

```
    do { \
        DIDR0 |= (1 « (pinNumber)); \
    } while (0)
```

Definition at line 142 of file ha_m328p.h.

```
00142 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) \
00143     do { \
00144         DIDR0 |= (1 « (pinNumber)); \
00145     } while (0)
```

9.145.2.8 ADC_ENABLE

```
#define ADC_ENABLE ADCSRA |= 1 << ADEN
```

Definition at line 148 of file ha_m328p.h.

9.145.2.9 ADC_ENABLE_AUTOTRIGGER

```
#define ADC_ENABLE_AUTOTRIGGER ADCSRA |= 1 << ADATE
```

Definition at line 158 of file ha_m328p.h.

9.145.2.10 ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT

```
#define ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT ADCSRA |= 1 << ADIE
```

Definition at line 161 of file ha_m328p.h.

9.145.2.11 ADC_SELECT_ANALOG_INPUT

```
#define ADC_SELECT_ANALOG_INPUT(
              pinNumber)
```

Value:

```
    do { \
        ADMUX &= 0xF0; \
        ADMUX |= (pinNumber); \
    } while (0)
```

Definition at line 134 of file ha_m328p.h.

```
00134 #define   ADC_SELECT_ANALOG_INPUT(pinNumber) \
00135     do { \
00136         ADMUX &= 0xF0; \
00137         ADMUX |= (pinNumber); \
00138     } while (0)
```

9.145.2.12 ADC_SELECT_AUTO_TRIGGER_SOURCE

```
#define ADC_SELECT_AUTO_TRIGGER_SOURCE(
              triggerSource)
```

Value:

```
    do { \
        ADCSRB &= 0xF8; \
        ADCSRB |= (triggerSource); \
    } while (0)
```

Definition at line 171 of file ha_m328p.h.

```
00171 #define   ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource) \
00172     do { \
00173         ADCSRB &= 0xF8; \
00174         ADCSRB |= (triggerSource); \
00175     } while (0)
```

9.145.2.13 ADC_SELECT_CLOCK_PRESCALER

```
#define ADC_SELECT_CLOCK_PRESCALER(
              clockPrescaler)
```

Value:

```
    do { \
        ADCSRA &= 0xF8; \
        ADCSRA |= (clockPrescaler); \
    } while (0)
```

Definition at line 164 of file ha_m328p.h.

```
00164 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00165     do { \
00166         ADCSRA &= 0xF8; \
00167         ADCSRA |= (clockPrescaler); \
00168     } while (0)
```

9.145.2.14 ADC_SELECT_REF_VOLTAGE

```
#define ADC_SELECT_REF_VOLTAGE(
              refVoltage)
```

Value:

```
    do { \
        ADMUX &= 0x3F; \
        ADMUX |= (refVoltage) « 6; \
    } while (0)
```

Definition at line 125 of file ha_m328p.h.

```
00125 #define   ADC_SELECT_REF_VOLTAGE(refVoltage) \
00126     do { \
00127         ADMUX &= 0x3F; \
00128         ADMUX |= (refVoltage) « 6; \
00129     } while (0)
```

9.145.2.15 ADC_START_CONVERSION

```
#define ADC_START_CONVERSION ADCSRA |= 1 << ADSC
```

Definition at line 151 of file ha_m328p.h.

9.145.2.16 ADC_STOP_CONVERSION

```
#define ADC_STOP_CONVERSION ADCSRA &= ~(1 << ADSC)
```

Definition at line 156 of file ha_m328p.h.

9.145.2.17 EXT_INT_DISABLE_INT0

```
#define EXT_INT_DISABLE_INT0 EIMSK &= ~(1 << INT0)
```

Definition at line 194 of file ha_m328p.h.

9.145.2.18 EXT_INT_DISABLE_INT1

```
#define EXT_INT_DISABLE_INT1 EIMSK &= ~(1 << INT1)
```

Definition at line 197 of file ha_m328p.h.

9.145.2.19 EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT

```
#define EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(
              pinChangePort)
```

Value:

```
PCICR &= ~(1 « (pinChangePort))
```

Definition at line 201 of file ha_m328p.h.

9.145.2.20 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB

```
#define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(
              pinChangePin)
```

Value:

```
PCMSK0 &= ~(1 « (pinChangePin))
```

Definition at line 205 of file ha_m328p.h.

9.145.2.21 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC

```
#define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(
              pinChangePin)
```

Value:

```
PCMSK1 &= ~(1 « (pinChangePin))
```

Definition at line 208 of file ha_m328p.h.

9.145.2.22 EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD

```
#define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(
              pinChangePin)
```

Value:

```
PCMSK2 &= ~(1 « (pinChangePin))
```

Definition at line 211 of file ha_m328p.h.

9.145.2.23 EXT_INT_ENABLE_INT0

```
#define EXT_INT_ENABLE_INT0 EIMSK |= 1 << INT0
```

Definition at line 193 of file ha_m328p.h.

9.145.2.24 EXT_INT_ENABLE_INT1

```
#define EXT_INT_ENABLE_INT1 EIMSK |= 1 << INT1
```

Definition at line 196 of file ha_m328p.h.

9.145.2.25 EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT

```
#define EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(
              pinChangePort)
```

Value:

```
PCICR |= 1 « (pinChangePort)
```

Definition at line 200 of file ha_m328p.h.

9.145.2.26 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB

```
#define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(
              pinChangePin)
```

Value:

```
PCMSK0 |= 1 « (pinChangePin)
```

Definition at line 204 of file ha_m328p.h.

9.145.2.27 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC

```
#define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(
              pinChangePin)
```

Value:

```
PCMSK1 |= 1 « (pinChangePin)
```

Definition at line 207 of file ha_m328p.h.

9.145.2.28 EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD

```
#define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(
              pinChangePin)
```

Value:

```
PCMSK2 |= 1 « (pinChangePin)
```

Definition at line 210 of file ha_m328p.h.

9.145.2.29 EXT_INT_INT0_INTERRUPT

```
#define EXT_INT_INT0_INTERRUPT INT0_vect
```

Definition at line 214 of file ha_m328p.h.

9.145.2.30 EXT_INT_INT1_INTERRUPT

```
#define EXT_INT_INT1_INTERRUPT INT1_vect
```

Definition at line 215 of file ha_m328p.h.

9.145.2.31 EXT_INT_PIN_CHANGE_PORTB_INTERRUPT

```
#define EXT_INT_PIN_CHANGE_PORTB_INTERRUPT PCINT0_vect
```

Definition at line 217 of file ha_m328p.h.

9.145.2.32 EXT_INT_PIN_CHANGE_PORTC_INTERRUPT

```
#define EXT_INT_PIN_CHANGE_PORTC_INTERRUPT PCINT1_vect
```

Definition at line 218 of file ha_m328p.h.

9.145.2.33 EXT_INT_PIN_CHANGE_PORTD_INTERRUPT

```
#define EXT_INT_PIN_CHANGE_PORTD_INTERRUPT PCINT2_vect
```

Definition at line 219 of file ha_m328p.h.

9.145.2.34 EXT_INT_SET_INT0_SENSE_CONTROL

```
#define EXT_INT_SET_INT0_SENSE_CONTROL(
            senseControl)
```

Value:

```
    do { \
        EICRA &= 0xFC; \
        EICRA |= (senseControl); \
    } while (0)
```

Definition at line 182 of file ha_m328p.h.

```
00182 #define EXT_INT_SET_INT0_SENSE_CONTROL(senseControl) \
00183     do { \
00184         EICRA &= 0xFC; \
00185         EICRA |= (senseControl); \
00186     } while (0)
```

9.145.2.35 EXT_INT_SET_INT1_SENSE_CONTROL

```
#define EXT_INT_SET_INT1_SENSE_CONTROL(
            senseControl)
```

Value:

```
    do { \
        EICRA &= 0xF3; \
        EICRA |= (senseControl) « 2; \
    } while (0)
```

Definition at line 187 of file ha_m328p.h.

```
00187 #define EXT_INT_SET_INT1_SENSE_CONTROL(senseControl) \
00188     do { \
00189         EICRA &= 0xF3; \
00190         EICRA |= (senseControl) « 2; \
00191     } while (0)
```

9.145.2.36 MCU_ADC_DISABLE

```
#define MCU_ADC_DISABLE PRR |= 1 << PRADC
```

Definition at line 402 of file ha_m328p.h.

9.145.2.37 MCU_ADC_ENABLE

```
#define MCU_ADC_ENABLE PRR &= ~(1 << PRADC)
```

Definition at line 401 of file ha_m328p.h.

9.145.2.38 MCU_SELECT_SLEEP_MODE

```
#define MCU_SELECT_SLEEP_MODE(
            sleepMode)
```

Value:

```
    do { \
        SMCR &= 0xF1; \
        SMCR |= (sleepMode) « 1; \
    } while (0)
```

Definition at line 374 of file ha_m328p.h.

```
00374 #define MCU_SELECT_SLEEP_MODE(sleepMode) \
00375     do { \
00376         SMCR &= 0xF1; \
00377         SMCR |= (sleepMode) « 1; \
00378     } while (0)
```

9.145.2.39 MCU_SLEEP_DISABLE

```
#define MCU_SLEEP_DISABLE SMCR &= ~(1 << SE)
```

Definition at line 381 of file ha_m328p.h.

9.145.2.40 MCU_SLEEP_ENABLE

```
#define MCU_SLEEP_ENABLE SMCR |= 1 << SE
```

Definition at line 380 of file ha_m328p.h.

9.145.2.41 MCU_SPI_DISABLE

```
#define MCU_SPI_DISABLE PRR |= 1 << PRSPI
```

Definition at line 396 of file ha_m328p.h.

9.145.2.42 MCU_SPI_ENABLE

```
#define MCU_SPI_ENABLE PRR &= ~(1 << PRSPI)
```

Definition at line 395 of file ha_m328p.h.

9.145.2.43 MCU_TIMER0_DISABLE

```
#define MCU_TIMER0_DISABLE PRR |= 1 << PRTIM0
```

Definition at line 393 of file ha_m328p.h.

9.145.2.44 MCU_TIMER0_ENABLE

```
#define MCU_TIMER0_ENABLE PRR &= ~(1 << PRTIM0)
```

Definition at line 392 of file ha_m328p.h.

9.145.2.45 MCU_TIMER1_DISABLE

```
#define MCU_TIMER1_DISABLE PRR |= 1 << PRTIM1
```

Definition at line 390 of file ha_m328p.h.

9.145.2.46 MCU_TIMER1_ENABLE

```
#define MCU_TIMER1_ENABLE PRR &= ~(1 << PRTIM1)
```

Definition at line 389 of file ha_m328p.h.

9.145.2.47 MCU_TIMER2_DISABLE

```
#define MCU_TIMER2_DISABLE PRR |= 1 << PRTIM2
```

Definition at line 387 of file ha_m328p.h.

9.145.2.48 MCU_TIMER2_ENABLE

```
#define MCU_TIMER2_ENABLE PRR &= ~(1 << PRTIM2)
```

Definition at line 386 of file ha_m328p.h.

9.145.2.49 MCU_TWI_DISABLE

```
#define MCU_TWI_DISABLE PRR |= 1 << PRTWI
```

Definition at line 384 of file ha_m328p.h.

9.145.2.50 MCU_TWI_ENABLE

```
#define MCU_TWI_ENABLE PRR &= ~(1 << PRTWI)
```

Definition at line 383 of file ha_m328p.h.

9.145.2.51 MCU_USART0_DISABLE

```
#define MCU_USART0_DISABLE PRR |= 1 << PRUSART0
```

Definition at line 399 of file ha_m328p.h.

9.145.2.52 MCU_USART0_ENABLE

```
#define MCU_USART0_ENABLE PRR &= ~(1 << PRUSART0)
```

Definition at line 398 of file ha_m328p.h.

9.145.2.53 PUSHBUTTON_DEBOUNCE_TIME_US

```
#define PUSHBUTTON_DEBOUNCE_TIME_US 1000
```

Definition at line 18 of file ha_m328p.h.

9.145.2.54 SERVOMOTOR_TIMER_ANGLE_COUNT

```
#define SERVOMOTOR_TIMER_ANGLE_COUNT(
              angle,
              out_min,
              out_mid,
              out_max)
```

Value:

```
(((((out_min)*((angle) - 90L)*((angle) - 180L))/16200L) + (((angle)*(out_max)*((angle) - 90L))/16200L) -
      (((angle)*(out_mid)*((angle) - 180L))/8100L)))
```

Definition at line 414 of file ha_m328p.h.

9.145.2.55 SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT

```
#define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(
              pulseWidth,
              clockPrescaler)
```

Value:

```
    ((((uint32_t)(pulseWidth)) * ((F_CPU) / 1000000UL)) / (clockPrescaler))
```

Definition at line 411 of file ha_m328p.h.

```
00411 #define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(pulseWidth,clockPrescaler) \
00412     ((((uint32_t)(pulseWidth)) * ((F_CPU) / 1000000UL)) / (clockPrescaler))
```

9.145.2.56 SPI_CONTROL_REGISTER

```
#define SPI_CONTROL_REGISTER SPCR
```

Definition at line 446 of file ha_m328p.h.

9.145.2.57 SPI_DATA_REGISTER

```
#define SPI_DATA_REGISTER SPDR
```

Definition at line 447 of file ha_m328p.h.

9.145.2.58 SPI_DISABLE

```
#define SPI_DISABLE SPCR &= ~(1 << SPE)
```

Definition at line 419 of file ha_m328p.h.

9.145.2.59 SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT

```
#define SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT SPCR &= ~(1 << SPIE)
```

Definition at line 451 of file ha_m328p.h.

9.145.2.60 SPI_ENABLE

```
#define SPI_ENABLE SPCR |= 1 << SPE
```

Definition at line 418 of file ha_m328p.h.

9.145.2.61 SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT

```
#define SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT SPCR |= 1 << SPIE
```

Definition at line 450 of file ha_m328p.h.

9.145.2.62 SPI_MASTER_MODE

```
#define SPI_MASTER_MODE MSTR
```

Definition at line 445 of file ha_m328p.h.

9.145.2.63 SPI_SELECT_CLOCK_PRESCALER

```
#define SPI_SELECT_CLOCK_PRESCALER(
              clockPrescaler)
```

Value:

```
    do { \
        SPCR &= 0xFC; \
        SPCR |= ((clockPrescaler) & 3); \
        SPSR &= 0xFE; \
        SPSR |= (((clockPrescaler) & 4) » 2); \
    } while (0)
```

Definition at line 429 of file ha_m328p.h.

```
00429 #define   SPI_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00430     do { \
00431         SPCR &= 0xFC; \
00432         SPCR |= ((clockPrescaler) & 3); \
00433         SPSR &= 0xFE; \
00434         SPSR |= (((clockPrescaler) & 4) » 2); \
00435     } while (0)
```

9.145.2.64 SPI_SELECT_DATA_MODE

```
#define SPI_SELECT_DATA_MODE(
              dataMode)
```

Value:

```
    do { \
        SPCR &= 0xF3; \
        SPCR |= (dataMode) « 2; \
    } while (0)
```

Definition at line 424 of file ha_m328p.h.

```
00424 #define   SPI_SELECT_DATA_MODE(dataMode) \
00425     do { \
00426         SPCR &= 0xF3; \
00427         SPCR |= (dataMode) « 2; \
00428     } while (0)
```

9.145.2.65 SPI_SELECT_DATA_ORDER

```
#define SPI_SELECT_DATA_ORDER(
                dataOrder)
```

Value:

```
    do { \
        SPCR &= 0xDF; \
        SPCR |= (dataOrder) « 5; \
    } while (0)
```

Definition at line 436 of file ha_m328p.h.

```
00436 #define   SPI_SELECT_DATA_ORDER(dataOrder) \
00437     do { \
00438         SPCR &= 0xDF; \
00439         SPCR |= (dataOrder) « 5; \
00440     } while (0)
```

9.145.2.66 SPI_SELECT_MASTER_MODE

```
#define SPI_SELECT_MASTER_MODE SPCR |= 1 << MSTR
```

Definition at line 420 of file ha_m328p.h.

9.145.2.67 SPI_SELECT_SLAVE_MODE

```
#define SPI_SELECT_SLAVE_MODE SPCR &= ~(1 << MSTR)
```

Definition at line 421 of file ha_m328p.h.

9.145.2.68 SPI_STATUS_REGISTER

```
#define SPI_STATUS_REGISTER SPSR
```

Definition at line 448 of file ha_m328p.h.

9.145.2.69 SPI_TRANSFER_COMPLETE

```
#define SPI_TRANSFER_COMPLETE SPIF
```

Definition at line 443 of file ha_m328p.h.

9.145.2.70 SPI_TRANSFER_COMPLETE_INTERRUPT

```
#define SPI_TRANSFER_COMPLETE_INTERRUPT SPI_STC_vect
```

Definition at line 454 of file ha_m328p.h.

9.145.2.71 SPI_WRITE_COLLISION

```
#define SPI_WRITE_COLLISION WCOL
```

Definition at line 442 of file ha_m328p.h.

9.145.2.72 TIMER0_COM_CHANNEL_A_INTERRUPT

```
#define TIMER0_COM_CHANNEL_A_INTERRUPT TIMER0_COMPA_vect
```

Definition at line 337 of file ha_m328p.h.

9.145.2.73 TIMER0_COM_CHANNEL_B_INTERRUPT

```
#define TIMER0_COM_CHANNEL_B_INTERRUPT TIMER0_COMPB_vect
```

Definition at line 338 of file ha_m328p.h.

9.145.2.74 TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 &= ~(1 << OCIE0A)
```

Definition at line 308 of file ha_m328p.h.

9.145.2.75 TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 &= ~(1 << OCIE0B)
```

Definition at line 311 of file ha_m328p.h.

9.145.2.76 TIMER0_DISABLE_OVERFLOW_INTERRUPT

```
#define TIMER0_DISABLE_OVERFLOW_INTERRUPT TIMSK0 &= ~(1 << TOIE0)
```

Definition at line 314 of file ha_m328p.h.

9.145.2.77 TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 |= 1 << OCIE0A
```

Definition at line 307 of file ha_m328p.h.

9.145.2.78 TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 |= 1 << OCIE0B
```

Definition at line 310 of file ha_m328p.h.

9.145.2.79 TIMER0_ENABLE_OVERFLOW_INTERRUPT

```
#define TIMER0_ENABLE_OVERFLOW_INTERRUPT TIMSK0 |= 1 << TOIE0
```

Definition at line 313 of file ha_m328p.h.

9.145.2.80 TIMER0_OVERFLOW_INTERRUPT

```
#define TIMER0_OVERFLOW_INTERRUPT TIMER0_OVF_vect
```

Definition at line 339 of file ha_m328p.h.

9.145.2.81 TIMER0_SELECT_CLOCK_SOURCE

```
#define TIMER0_SELECT_CLOCK_SOURCE(
              clockSource)
```

Value:

```
    do { \
        TCCR0B &= 0xF8; \
        TCCR0B |= (clockSource); \
    } while (0)
```

Definition at line 236 of file ha_m328p.h.

```
00236 #define TIMER0_SELECT_CLOCK_SOURCE(clockSource) \
00237     do { \
00238         TCCR0B &= 0xF8; \
00239         TCCR0B |= (clockSource); \
00240     } while (0)
```

9.145.2.82 TIMER0_SELECT_COM_CHANNEL_A

```
#define TIMER0_SELECT_COM_CHANNEL_A(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR0A &= 0x3F; \
        TCCR0A |= (compareOutputMode) « 6; \
    } while (0)
```

Definition at line 274 of file ha_m328p.h.

```
00274 #define TIMER0_SELECT_COM_CHANNEL_A(compareOutputMode) \
00275     do { \
00276         TCCR0A &= 0x3F; \
00277         TCCR0A |= (compareOutputMode) « 6; \
00278     } while (0)
```

9.145.2.83 TIMER0_SELECT_COM_CHANNEL_B

```
#define TIMER0_SELECT_COM_CHANNEL_B(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR0A &= 0xCF; \
        TCCR0A |= (compareOutputMode) « 4; \
    } while (0)
```

Definition at line 279 of file ha_m328p.h.

```
00279 #define TIMER0_SELECT_COM_CHANNEL_B(compareOutputMode) \
00280     do { \
00281         TCCR0A &= 0xCF; \
00282         TCCR0A |= (compareOutputMode) « 4; \
00283     } while (0)
```

9.145.2.84 TIMER0_SELECT_OPERATION_MODE

```
#define TIMER0_SELECT_OPERATION_MODE(
              operationMode)
```

Value:

```
    do { \
        TCCR0A &= 0xFC; \
        TCCR0A |= ((operationMode) & 3); \
        TCCR0B &= 0xF7; \
        TCCR0B |= (((operationMode) & 12) « 1); \
    } while (0)
```

Definition at line 252 of file ha_m328p.h.

```
00252 #define TIMER0_SELECT_OPERATION_MODE(operationMode) \
00253     do { \
00254         TCCR0A &= 0xFC; \
00255         TCCR0A |= ((operationMode) & 3); \
00256         TCCR0B &= 0xF7; \
00257         TCCR0B |= (((operationMode) & 12) « 1); \
00258     } while (0)
```

9.145.2.85 TIMER0_STOP

```
#define TIMER0_STOP
```

Value:

```
    do { \
        TCCR0B &= 0xF8; \
    } while (0)
```

Definition at line 223 of file ha_m328p.h.

```
00223 #define TIMER0_STOP \
00224     do { \
00225         TCCR0B &= 0xF8; \
00226     } while (0)
```

9.145.2.86 TIMER1_COM_CHANNEL_A_INTERRUPT

```
#define TIMER1_COM_CHANNEL_A_INTERRUPT TIMER1_COMPA_vect
```

Definition at line 341 of file ha_m328p.h.

9.145.2.87 TIMER1_COM_CHANNEL_B_INTERRUPT

```
#define TIMER1_COM_CHANNEL_B_INTERRUPT TIMER1_COMPB_vect
```

Definition at line 342 of file ha_m328p.h.

9.145.2.88 TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 &= ~(1 << OCIE1A)
```

Definition at line 317 of file ha_m328p.h.

9.145.2.89 TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 &= ~(1 << OCIE1B)
```

Definition at line 320 of file ha_m328p.h.

9.145.2.90 TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT

```
#define TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 &= ~(1 << ICIE1)
```

Definition at line 326 of file ha_m328p.h.

9.145.2.91 TIMER1_DISABLE_OVERFLOW_INTERRUPT

```
#define TIMER1_DISABLE_OVERFLOW_INTERRUPT TIMSK1 &= ~(1 << TOIE1)
```

Definition at line 323 of file ha_m328p.h.

9.145.2.92 TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 |= 1 << OCIE1A
```

Definition at line 316 of file ha_m328p.h.

9.145.2.93 TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 |= 1 << OCIE1B
```

Definition at line 319 of file ha_m328p.h.

9.145.2.94 TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT

```
#define TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 |= 1 << ICIE1
```

Definition at line 325 of file ha_m328p.h.

9.145.2.95 TIMER1_ENABLE_OVERFLOW_INTERRUPT

```
#define TIMER1_ENABLE_OVERFLOW_INTERRUPT TIMSK1 |= 1 << TOIE1
```

Definition at line 322 of file ha_m328p.h.

9.145.2.96 TIMER1_INPUT_CAPTURE_INTERRUPT

```
#define TIMER1_INPUT_CAPTURE_INTERRUPT TIMER1_CAPT_vect
```

Definition at line 344 of file ha_m328p.h.

9.145.2.97 TIMER1_OVERFLOW_INTERRUPT

```
#define TIMER1_OVERFLOW_INTERRUPT TIMER1_OVF_vect
```

Definition at line 343 of file ha_m328p.h.

9.145.2.98 TIMER1_SELECT_CLOCK_SOURCE

```
#define TIMER1_SELECT_CLOCK_SOURCE(
              clockSource)
```

Value:

```
    do { \
        TCCR1B &= 0xF8; \
        TCCR1B |= (clockSource); \
    } while (0)
```

Definition at line 241 of file ha_m328p.h.

```
00241 #define TIMER1_SELECT_CLOCK_SOURCE(clockSource) \
00242     do { \
00243         TCCR1B &= 0xF8; \
00244         TCCR1B |= (clockSource); \
00245     } while (0)
```

9.145.2.99 TIMER1_SELECT_COM_CHANNEL_A

```
#define TIMER1_SELECT_COM_CHANNEL_A(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR1A &= 0x3F; \
        TCCR1A |= (compareOutputMode) « 6; \
    } while (0)
```

Definition at line 285 of file ha_m328p.h.

```
00285 #define TIMER1_SELECT_COM_CHANNEL_A(compareOutputMode) \
00286     do { \
00287         TCCR1A &= 0x3F; \
00288         TCCR1A |= (compareOutputMode) « 6; \
00289     } while (0)
```

9.145.2.100 TIMER1_SELECT_COM_CHANNEL_B

```
#define TIMER1_SELECT_COM_CHANNEL_B(
              compareOutputMode)
```

Value:

```
    do { \
        TCCR1A &= 0xCF; \
        TCCR1A |= (compareOutputMode) « 4; \
    } while (0)
```

Definition at line 290 of file ha_m328p.h.

```
00290 #define TIMER1_SELECT_COM_CHANNEL_B(compareOutputMode) \
00291     do { \
00292         TCCR1A &= 0xCF; \
00293         TCCR1A |= (compareOutputMode) « 4; \
00294     } while (0)
```

9.145.2.101 TIMER1_SELECT_OPERATION_MODE

```
#define TIMER1_SELECT_OPERATION_MODE(
              operationMode)
```

Value:

```
    do { \
        TCCR1A &= 0xFC; \
        TCCR1A |= ((operationMode) & 3); \
        TCCR1B &= 0xE7; \
        TCCR1B |= (((operationMode) & 12) << 1); \
    } while (0)
```

Definition at line 259 of file ha_m328p.h.

```
00259 #define TIMER1_SELECT_OPERATION_MODE(operationMode) \
00260     do { \
00261         TCCR1A &= 0xFC; \
00262         TCCR1A |= ((operationMode) & 3); \
00263         TCCR1B &= 0xE7; \
00264         TCCR1B |= (((operationMode) & 12) << 1); \
00265     } while (0)
```

9.145.2.102 TIMER1_STOP

```
#define TIMER1_STOP
```

Value:

```
    do { \
        TCCR1B &= 0xF8; \
    } while (0)
```

Definition at line 227 of file ha_m328p.h.

```
00227 #define TIMER1_STOP \
00228     do { \
00229         TCCR1B &= 0xF8; \
00230     } while (0)
```

9.145.2.103 TIMER2_COM_CHANNEL_A_INTERRUPT

```
#define TIMER2_COM_CHANNEL_A_INTERRUPT TIMER2_COMPA_vect
```

Definition at line 346 of file ha_m328p.h.

9.145.2.104 TIMER2_COM_CHANNEL_B_INTERRUPT

```
#define TIMER2_COM_CHANNEL_B_INTERRUPT TIMER2_COMPB_vect
```

Definition at line 347 of file ha_m328p.h.

9.145.2.105 TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 &= ~(1 << OCIE2A)
```

Definition at line 329 of file ha_m328p.h.

9.145.2.106 TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 &= ~(1 << OCIE2B)
```

Definition at line 332 of file ha_m328p.h.

9.145.2.107 TIMER2_DISABLE_OVERFLOW_INTERRUPT

```
#define TIMER2_DISABLE_OVERFLOW_INTERRUPT TIMSK2 &= ~(1 << TOIE2)
```

Definition at line 335 of file ha_m328p.h.

9.145.2.108 TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT

```
#define TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 |= 1 << OCIE2A
```

Definition at line 328 of file ha_m328p.h.

9.145.2.109 TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT

```
#define TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 |= 1 << OCIE2B
```

Definition at line 331 of file ha_m328p.h.

9.145.2.110 TIMER2_ENABLE_OVERFLOW_INTERRUPT

```
#define TIMER2_ENABLE_OVERFLOW_INTERRUPT TIMSK2 |= 1 << TOIE2
```

Definition at line 334 of file ha_m328p.h.

9.145.2.111 TIMER2_OVERFLOW_INTERRUPT

```
#define TIMER2_OVERFLOW_INTERRUPT TIMER2_OVF_vect
```

Definition at line 348 of file ha_m328p.h.

9.145.2.112 TIMER2_SELECT_CLOCK_SOURCE

```
#define TIMER2_SELECT_CLOCK_SOURCE(
              clockSource)
```

Value:

```
    do { \
        TCCR2B &= 0xF8; \
        TCCR2B |= (clockSource); \
    } while (0)
```

Definition at line 246 of file ha_m328p.h.

```
00246 #define TIMER2_SELECT_CLOCK_SOURCE(clockSource) \
00247     do { \
00248         TCCR2B &= 0xF8; \
00249         TCCR2B |= (clockSource); \
00250     } while (0)
```

9.145.2.113 TIMER2_SELECT_COM_CHANNEL_A

```
#define TIMER2_SELECT_COM_CHANNEL_A(
                compareOutputMode)
```

Value:

```
    do { \
        TCCR2A &= 0x3F; \
        TCCR2A |= (compareOutputMode) « 6; \
    } while (0)
```

Definition at line 296 of file ha_m328p.h.

```
00296 #define TIMER2_SELECT_COM_CHANNEL_A(compareOutputMode) \
00297     do { \
00298         TCCR2A &= 0x3F; \
00299         TCCR2A |= (compareOutputMode) « 6; \
00300     } while (0)
```

9.145.2.114 TIMER2_SELECT_COM_CHANNEL_B

```
#define TIMER2_SELECT_COM_CHANNEL_B(
                compareOutputMode)
```

Value:

```
    do { \
        TCCR2A &= 0xCF; \
        TCCR2A |= (compareOutputMode) « 4; \
    } while (0)
```

Definition at line 301 of file ha_m328p.h.

```
00301 #define TIMER2_SELECT_COM_CHANNEL_B(compareOutputMode) \
00302     do { \
00303         TCCR2A &= 0xCF; \
00304         TCCR2A |= (compareOutputMode) « 4; \
00305     } while (0)
```

9.145.2.115 TIMER2_SELECT_OPERATION_MODE

```
#define TIMER2_SELECT_OPERATION_MODE(
                operationMode)
```

Value:

```
    do { \
        TCCR2A &= 0xFC; \
        TCCR2A |= ((operationMode) & 3); \
        TCCR2B &= 0xF7; \
        TCCR2B |= (((operationMode) & 12) « 1); \
    } while (0)
```

Definition at line 266 of file ha_m328p.h.

```
00266 #define TIMER2_SELECT_OPERATION_MODE(operationMode) \
00267     do { \
00268         TCCR2A &= 0xFC; \
00269         TCCR2A |= ((operationMode) & 3); \
00270         TCCR2B &= 0xF7; \
00271         TCCR2B |= (((operationMode) & 12) « 1); \
00272     } while (0)
```

9.145.2.116 TIMER2_STOP

```
#define TIMER2_STOP
```

Value:

```
    do { \
        TCCR2B &= 0xF8; \
    } while (0)
```

Definition at line 231 of file ha_m328p.h.

```
00231 #define TIMER2_STOP \
00232     do { \
00233         TCCR2B &= 0xF8; \
00234     } while (0)
```

9.145.2.117 USART0_CONTROL_STATUS_REGISTER

```
#define USART0_CONTROL_STATUS_REGISTER UCSR0A
```

Definition at line 92 of file ha_m328p.h.

9.145.2.118 USART0_DATA_OVERRUN

```
#define USART0_DATA_OVERRUN DOR0
```

Definition at line 100 of file ha_m328p.h.

9.145.2.119 USART0_DATA_REGISTER

```
#define USART0_DATA_REGISTER UDR0
```

Definition at line 93 of file ha_m328p.h.

9.145.2.120 USART0_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_DATA_REGISTER_EMPTY_INTERRUPT USART_UDRE_vect
```

Definition at line 120 of file ha_m328p.h.

9.145.2.121 USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B &= ~(1 << UDRIE0)
```

Definition at line 106 of file ha_m328p.h.

9.145.2.122 USART0_DISABLE_DOUBLE_SPEED_MODE [1/2]

```
#define USART0_DISABLE_DOUBLE_SPEED_MODE UCSR0A &= ~(1 << U2X0)
```

Definition at line 33 of file ha_m328p.h.

9.145.2.123 USART0_DISABLE_DOUBLE_SPEED_MODE [2/2]

```
#define USART0_DISABLE_DOUBLE_SPEED_MODE
```

Definition at line 33 of file ha_m328p.h.

9.145.2.124 USART0_DISABLE_PARITY_MODE

```
#define USART0_DISABLE_PARITY_MODE UCSR0C &= ~((1 << UPM01) | (1 << UPM00))
```

Definition at line 46 of file ha_m328p.h.

9.145.2.125 USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B &= ~(1 << RXCIE0)
```

Definition at line 109 of file ha_m328p.h.

9.145.2.126 USART0_DISABLE_RECEIVER

```
#define USART0_DISABLE_RECEIVER UCSR0B &= ~(1 << RXEN0)
```

Definition at line 88 of file ha_m328p.h.

9.145.2.127 USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B &= ~(1 << TXCIE0)
```

Definition at line 112 of file ha_m328p.h.

9.145.2.128 USART0_DISABLE_TRANSMITTER

```
#define USART0_DISABLE_TRANSMITTER UCSR0B &= ~(1 << TXEN0)
```

Definition at line 83 of file ha_m328p.h.

9.145.2.129 USART0_ENABLE_ASYNC_TRANSMISSION_MODE

```
#define USART0_ENABLE_ASYNC_TRANSMISSION_MODE UCSR0C &= ~((1 << UMSEL01) | (1 << UMSEL00))
```

Definition at line 27 of file ha_m328p.h.

9.145.2.130 USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT

```
#define USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT UCSR0B |= 1 << UDRIE0
```

Definition at line 105 of file ha_m328p.h.

9.145.2.131 USART0_ENABLE_DOUBLE_SPEED_MODE

```
#define USART0_ENABLE_DOUBLE_SPEED_MODE
```

Definition at line 32 of file ha_m328p.h.

9.145.2.132 USART0_ENABLE_EVEN_PARITY_MODE

```
#define USART0_ENABLE_EVEN_PARITY_MODE UCSR0C = (1 << UPM01) | (UCSR0C & ~(1 << UPM00))
```

Definition at line 44 of file ha_m328p.h.

9.145.2.133 USART0_ENABLE_MASTER_SPI_MODE

```
#define USART0_ENABLE_MASTER_SPI_MODE UCSR0C |= (1 << UMSEL01) | (1 << UMSEL00)
```

Definition at line 29 of file ha_m328p.h.

9.145.2.134 USART0_ENABLE_ODD_PARITY_MODE

```
#define USART0_ENABLE_ODD_PARITY_MODE UCSR0C |= ((1 << UPM01) | (1 << UPM00))
```

Definition at line 45 of file ha_m328p.h.

9.145.2.135 USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT UCSR0B |= 1 << RXCIE0
```

Definition at line 108 of file ha_m328p.h.

9.145.2.136 USART0_ENABLE_RECEIVER

```
#define USART0_ENABLE_RECEIVER UCSR0B |= 1 << RXEN0
```

Definition at line 87 of file ha_m328p.h.

9.145.2.137 USART0_ENABLE_SYNC_TRANSMISSION_MODE

```
#define USART0_ENABLE_SYNC_TRANSMISSION_MODE UCSR0C = (UCSR0C & ~(1 << UMSEL01)) | (1 <<
UMSEL00)
```

Definition at line 28 of file ha_m328p.h.

9.145.2.138 USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT UCSR0B |= 1 << TXCIE0
```

Definition at line 111 of file ha_m328p.h.

9.145.2.139 USART0_ENABLE_TRANSMITTER

```
#define USART0_ENABLE_TRANSMITTER UCSR0B |= 1 << TXEN0
```

Definition at line 82 of file ha_m328p.h.

9.145.2.140 USART0_FRAME_ERROR

```
#define USART0_FRAME_ERROR FE0
```

Definition at line 98 of file ha_m328p.h.

9.145.2.141 USART0_PARITY_ERROR

```
#define USART0_PARITY_ERROR UPE0
```

Definition at line 102 of file ha_m328p.h.

9.145.2.142 USART0_RECEIVE_COMPLETE_INTERRUPT

```
#define USART0_RECEIVE_COMPLETE_INTERRUPT USART_RX_vect
```

Definition at line 116 of file ha_m328p.h.

9.145.2.143 USART0_SET_5BIT_FRAME_SIZE

```
#define USART0_SET_5BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C &= ~((1 « UCSZ01) | (1 « UCSZ00)); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 75 of file ha_m328p.h.

```
00075 #define   USART0_SET_5BIT_FRAME_SIZE \
00076     do { \
00077         UCSR0C &= ~((1 « UCSZ01) | (1 « UCSZ00)); \
00078         UCSR0B &= ~(1 « UCSZ02); \
00079     } while (0)
```

9.145.2.144 USART0_SET_6BIT_FRAME_SIZE

```
#define USART0_SET_6BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C = (UCSR0C & ~(1 « UCSZ01)) | (1 « UCSZ00); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 70 of file ha_m328p.h.

```
00070 #define   USART0_SET_6BIT_FRAME_SIZE \
00071     do { \
00072         UCSR0C = (UCSR0C & ~(1 « UCSZ01)) | (1 « UCSZ00); \
00073         UCSR0B &= ~(1 « UCSZ02); \
00074     } while (0)
```

9.145.2.145 USART0_SET_7BIT_FRAME_SIZE

```
#define USART0_SET_7BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C = (1 « UCSZ01) | (UCSR0C & ~(1 « UCSZ00)); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 65 of file ha_m328p.h.

```
00065 #define   USART0_SET_7BIT_FRAME_SIZE  \
00066     do { \
00067         UCSR0C = (1 « UCSZ01) | (UCSR0C & ~(1 « UCSZ00)); \
00068         UCSR0B &= ~(1 « UCSZ02); \
00069     } while (0)
```

9.145.2.146 USART0_SET_8BIT_FRAME_SIZE

```
#define USART0_SET_8BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
        UCSR0B &= ~(1 « UCSZ02); \
    } while (0)
```

Definition at line 59 of file ha_m328p.h.

```
00059 #define   USART0_SET_8BIT_FRAME_SIZE   \
00060     do { \
00061         UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
00062         UCSR0B &= ~(1 « UCSZ02); \
00063     } while (0)
```

9.145.2.147 USART0_SET_9BIT_FRAME_SIZE

```
#define USART0_SET_9BIT_FRAME_SIZE
```

Value:

```
    do { \
        UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
        UCSR0B |= (1 « UCSZ02); \
    } while (0)
```

Definition at line 53 of file ha_m328p.h.

```
00053 #define   USART0_SET_9BIT_FRAME_SIZE   \
00054     do { \
00055         UCSR0C |= ((1 « UCSZ01) | (1 « UCSZ00)); \
00056         UCSR0B |= (1 « UCSZ02); \
00057     } while (0)
```

9.145.2.148 USART0_SET_BAUDRATE_HIGH_REGISTER

```
#define USART0_SET_BAUDRATE_HIGH_REGISTER UBRR0H = UBRRH_VALUE
```

Definition at line 24 of file ha_m328p.h.

9.145.2.149 USART0_SET_BAUDRATE_LOW_REGISTER

```
#define USART0_SET_BAUDRATE_LOW_REGISTER UBRR0L = UBRRL_VALUE
```

Definition at line 25 of file ha_m328p.h.

9.145.2.150 USART0_SET_ONE_STOP_BIT

```
#define USART0_SET_ONE_STOP_BIT UCSR0C &= ~(1 << USBS0)
```

Definition at line 49 of file ha_m328p.h.

9.145.2.151 USART0_SET_TWO_STOP_BITS

```
#define USART0_SET_TWO_STOP_BITS UCSR0C |= (1 << USBS0)
```

Definition at line 50 of file ha_m328p.h.

9.145.2.152 USART0_TRANSMIT_COMPLETE_INTERRUPT

```
#define USART0_TRANSMIT_COMPLETE_INTERRUPT USART_TX_vect
```

Definition at line 118 of file ha_m328p.h.

9.145.2.153 WATCHDOG_SELECT_TIMEOUT

```
#define WATCHDOG_SELECT_TIMEOUT(
              timeOut)
```

Value:

```
    do { \
        WDTCSR |= (1«WDCE) | (1«WDE); \
        WDTCSR = (0«WDIE) | (0«WDE) | (timeOut); \
    } while (0)
```

Definition at line 352 of file ha_m328p.h.

```
00352 #define WATCHDOG_SELECT_TIMEOUT(timeOut) \
00353     do { \
00354         WDTCSR |= (1«WDCE) | (1«WDE); \
00355         WDTCSR = (0«WDIE) | (0«WDE) | (timeOut); \
00356     } while (0)
```

9.145.2.154 WATCHDOG_START

```
#define WATCHDOG_START(
              operationMode,
              timeOut)
```

Value:

```
    do { \
        MCUSR &= ~(1<<WDRF); \
        WDTCSR |= (1<<WDCE) | (1<<WDE); \
        WDTCSR = (operationMode) | (timeOut); \
    } while (0)
```

Definition at line 357 of file ha_m328p.h.

```
00357 #define WATCHDOG_START(operationMode,timeOut) \
00358     do { \
00359         MCUSR &= ~(1<<WDRF); \
00360         WDTCSR |= (1<<WDCE) | (1<<WDE); \
00361         WDTCSR = (operationMode) | (timeOut); \
00362     } while (0)
```

9.145.2.155 WATCHDOG_STOP

```
#define WATCHDOG_STOP
```

Value:

```
    do { \
        MCUSR &= ~(1<<WDRF); \
        WDTCSR |= (1<<WDCE) | (1<<WDE); \
        WDTCSR = 0x00; \
    } while (0)
```

Definition at line 363 of file ha_m328p.h.

```
00363 #define WATCHDOG_STOP \
00364     do { \
00365         MCUSR &= ~(1<<WDRF); \
00366         WDTCSR |= (1<<WDCE) | (1<<WDE); \
00367         WDTCSR = 0x00; \
00368     } while (0)
```

9.145.2.156 WATCHDOG_TIMEOUT_INTERRUPT

```
#define WATCHDOG_TIMEOUT_INTERRUPT WDT_vect
```

Definition at line 369 of file ha_m328p.h.

9.146 ha_m328p.h

Go to the documentation of this file.

```
00001
00008 #ifndef HA_M328P_H
00009 #define HA_M328P_H
00010
00011 #ifndef HABASE_H
00012     #error "don't use this file directly! Please include only ha_base.h!"
00013 #endif
00014
00015 // PushButton defines
00016 // Debounce time between the two pin samples of isPressed(): 1 ms, long
00017 // enough to skip the bounce of a typical tactile switch
00018 #define PUSHBUTTON_DEBOUNCE_TIME_US 1000 // microseconds to wait for next check
00019
```

```
00020
00021 // USART defines
00022 #include <util/setbaud.h>
00023
00024 #define   USART0_SET_BAUDRATE_HIGH_REGISTER     UBRR0H = UBRRH_VALUE
00025 #define   USART0_SET_BAUDRATE_LOW_REGISTER      UBRR0L = UBRRL_VALUE
00026
00027 #define   USART0_ENABLE_ASYNC_TRANSMISSION_MODE     UCSR0C &= ~((1 << UMSEL01) | (1 << UMSEL00))
00028 #define   USART0_ENABLE_SYNC_TRANSMISSION_MODE      UCSR0C = (UCSR0C & ~(1 << UMSEL01)) | (1 << UMSEL00)
00029 #define   USART0_ENABLE_MASTER_SPI_MODE             UCSR0C |= (1 << UMSEL01) | (1 << UMSEL00)
00030
00031
00032 #define   USART0_ENABLE_DOUBLE_SPEED_MODE
00033 #define   USART0_DISABLE_DOUBLE_SPEED_MODE
00034
00035 #if USE_2X
00036     #undef  USART0_ENABLE_DOUBLE_SPEED_MODE
00037     #define USART0_ENABLE_DOUBLE_SPEED_MODE UCSR0A |= (1 << U2X0)
00038     #warning "double speed operation activated"
00039 #else
00040     #undef  USART0_DISABLE_DOUBLE_SPEED_MODE
00041     #define USART0_DISABLE_DOUBLE_SPEED_MODE UCSR0A &= ~(1 << U2X0)
00042 #endif
00043
00044 #define   USART0_ENABLE_EVEN_PARITY_MODE    UCSR0C = (1 << UPM01) | (UCSR0C & ~(1 << UPM00))
00045 #define   USART0_ENABLE_ODD_PARITY_MODE     UCSR0C |=   ((1 << UPM01) | (1 << UPM00))
00046 #define   USART0_DISABLE_PARITY_MODE        UCSR0C &= ~((1 << UPM01) | (1 << UPM00))
00047
00048
00049 #define   USART0_SET_ONE_STOP_BIT   UCSR0C &= ~(1 << USBS0)
00050 #define   USART0_SET_TWO_STOP_BITS  UCSR0C |= (1 << USBS0)
00051
00052
00053 #define   USART0_SET_9BIT_FRAME_SIZE   \
00054     do { \
00055         UCSR0C |= ((1 << UCSZ01) | (1 << UCSZ00)); \
00056         UCSR0B |= (1 << UCSZ02); \
00057     } while (0)
00058
00059 #define   USART0_SET_8BIT_FRAME_SIZE   \
00060     do { \
00061         UCSR0C |= ((1 << UCSZ01) | (1 << UCSZ00)); \
00062         UCSR0B &= ~(1 << UCSZ02); \
00063     } while (0)
00064
00065 #define   USART0_SET_7BIT_FRAME_SIZE  \
00066     do { \
00067         UCSR0C = (1 << UCSZ01) | (UCSR0C & ~(1 << UCSZ00)); \
00068         UCSR0B &= ~(1 << UCSZ02); \
00069     } while (0)
00070 #define   USART0_SET_6BIT_FRAME_SIZE \
00071     do { \
00072         UCSR0C = (UCSR0C & ~(1 << UCSZ01)) | (1 << UCSZ00); \
00073         UCSR0B &= ~(1 << UCSZ02); \
00074     } while (0)
00075 #define   USART0_SET_5BIT_FRAME_SIZE \
00076     do { \
00077         UCSR0C &= ~((1 << UCSZ01) | (1 << UCSZ00)); \
00078         UCSR0B &= ~(1 << UCSZ02); \
00079     } while (0)
00080
00081
00082 #define   USART0_ENABLE_TRANSMITTER      UCSR0B |=   1 << TXEN0
00083 #define   USART0_DISABLE_TRANSMITTER     UCSR0B &= ~(1 << TXEN0)
00084
00085
00086
00087 #define   USART0_ENABLE_RECEIVER         UCSR0B |=   1 << RXEN0
00088 #define   USART0_DISABLE_RECEIVER        UCSR0B &= ~(1 << RXEN0)
00089
00090
00091
00092 #define   USART0_CONTROL_STATUS_REGISTER UCSR0A
00093 #define   USART0_DATA_REGISTER           UDR0
00094
00095
00096
00097
00098 #define   USART0_FRAME_ERROR             FE0
00099
00100 #define   USART0_DATA_OVERRUN            DOR0
00101
00102 #define   USART0_PARITY_ERROR            UPE0
00103
00104
00105 #define   USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT   UCSR0B |=   1 << UDRIE0
00106 #define   USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT  UCSR0B &= ~(1 << UDRIE0)
```

```
00107
00108 #define   USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT   UCSR0B |=   1 « RXCIE0
00109 #define   USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT  UCSR0B &= ~(1 « RXCIE0)
00110
00111 #define   USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT   UCSR0B |=   1 « TXCIE0
00112 #define   USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT  UCSR0B &= ~(1 « TXCIE0)
00113
00114
00115
00116 #define   USART0_RECEIVE_COMPLETE_INTERRUPT     USART_RX_vect
00117
00118 #define   USART0_TRANSMIT_COMPLETE_INTERRUPT    USART_TX_vect
00119
00120 #define   USART0_DATA_REGISTER_EMPTY_INTERRUPT  USART_UDRE_vect
00121
00122
00123 // ADC defines
00124
00125 #define   ADC_SELECT_REF_VOLTAGE(refVoltage) \
00126     do { \
00127         ADMUX &= 0x3F; \
00128         ADMUX |= (refVoltage) « 6; \
00129     } while (0)
00130
00131 #define   ADC_ADJUST_RESULT_LEFT            ADMUX |=  1 « ADLAR
00132 #define   ADC_ADJUST_RESULT_RIGHT           ADMUX &= ~(1 « ADLAR)
00133
00134 #define   ADC_SELECT_ANALOG_INPUT(pinNumber) \
00135     do { \
00136         ADMUX &= 0xF0; \
00137         ADMUX |= (pinNumber); \
00138     } while (0)
00139 // Disable the digital input buffer of a single ADC pin.  Only sets the
00140 // pin's ADCnD bit; it does not clear the other bits, so switching analog
00141 // inputs leaves previously disabled digital inputs disabled.
00142 #define   ADC_DISABLE_DIGITAL_INPUT_REGISTER(pinNumber) \
00143     do { \
00144         DIDR0 |= (1 « (pinNumber)); \
00145     } while (0)
00146
00147
00148 #define   ADC_ENABLE            ADCSRA |=  1 « ADEN
00149 #define   ADC_DISABLE           ADCSRA &= ~(1 « ADEN)
00150
00151 #define   ADC_START_CONVERSION            ADCSRA |=  1 « ADSC
00152 // NOTE: clearing ADSC does not abort an ongoing conversion, it only stops
00153 // the auto-trigger sequence from starting new conversions (see the ADC
00154 // conversion timing in the datasheet).  To stop the ADC itself, clear ADEN
00155 // with ADC_DISABLE.
00156 #define   ADC_STOP_CONVERSION           ADCSRA &= ~(1 « ADSC)
00157
00158 #define   ADC_ENABLE_AUTOTRIGGER          ADCSRA |=   1 « ADATE
00159 #define   ADC_DISABLE_AUTOTRIGGER        ADCSRA &= ~(1 « ADATE)
00160
00161 #define   ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT   ADCSRA |=   1 « ADIE
00162 #define   ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT  ADCSRA &= ~(1 « ADIE)
00163
00164 #define   ADC_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00165     do { \
00166         ADCSRA &= 0xF8; \
00167         ADCSRA |= (clockPrescaler); \
00168     } while (0)
00169
00170
00171 #define   ADC_SELECT_AUTO_TRIGGER_SOURCE(triggerSource) \
00172     do { \
00173         ADCSRB &= 0xF8; \
00174         ADCSRB |= (triggerSource); \
00175     } while (0)
00176
00177
00178 #define   ADC_CONVERSION_COMPLETE_INTERRUPT  ADC_vect
00179
00180 // Externally Triggered Interrupts defines
00181
00182 #define EXT_INT_SET_INT0_SENSE_CONTROL(senseControl) \
00183     do { \
00184         EICRA &= 0xFC; \
00185         EICRA |= (senseControl); \
00186     } while (0)
00187 #define EXT_INT_SET_INT1_SENSE_CONTROL(senseControl) \
00188     do { \
00189         EICRA &= 0xF3; \
00190         EICRA |= (senseControl) « 2; \
00191     } while (0)
00192
00193 #define EXT_INT_ENABLE_INT0   EIMSK |= 1 « INT0
```

```
00194 #define EXT_INT_DISABLE_INT0   EIMSK &= ~(1 « INT0)
00195
00196 #define EXT_INT_ENABLE_INT1   EIMSK |= 1 « INT1
00197 #define EXT_INT_DISABLE_INT1   EIMSK &= ~(1 « INT1)
00198
00199
00200 #define EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT(pinChangePort)    PCICR |= 1 « (pinChangePort)
00201 #define EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT(pinChangePort)   PCICR &= ~(1 « (pinChangePort))
00202
00203
00204 #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)    PCMSK0 |= 1 « (pinChangePin)
00205 #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB(pinChangePin)   PCMSK0 &= ~(1 « (pinChangePin))
00206
00207 #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)    PCMSK1 |= 1 « (pinChangePin)
00208 #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC(pinChangePin)   PCMSK1 &= ~(1 « (pinChangePin))
00209
00210 #define EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)    PCMSK2 |= 1 « (pinChangePin)
00211 #define EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD(pinChangePin)   PCMSK2 &= ~(1 « (pinChangePin))
00212
00213
00214 #define  EXT_INT_INT0_INTERRUPT   INT0_vect
00215 #define  EXT_INT_INT1_INTERRUPT   INT1_vect
00216
00217 #define  EXT_INT_PIN_CHANGE_PORTB_INTERRUPT   PCINT0_vect
00218 #define  EXT_INT_PIN_CHANGE_PORTC_INTERRUPT   PCINT1_vect
00219 #define  EXT_INT_PIN_CHANGE_PORTD_INTERRUPT   PCINT2_vect
00220
00221 // Timers/Counters defines
00222
00223 #define TIMER0_STOP \
00224     do { \
00225         TCCR0B &= 0xF8; \
00226     } while (0)
00227 #define TIMER1_STOP \
00228     do { \
00229         TCCR1B &= 0xF8; \
00230     } while (0)
00231 #define TIMER2_STOP \
00232     do { \
00233         TCCR2B &= 0xF8; \
00234     } while (0)
00235
00236 #define TIMER0_SELECT_CLOCK_SOURCE(clockSource) \
00237     do { \
00238         TCCR0B &= 0xF8; \
00239         TCCR0B |= (clockSource); \
00240     } while (0)
00241 #define TIMER1_SELECT_CLOCK_SOURCE(clockSource) \
00242     do { \
00243         TCCR1B &= 0xF8; \
00244         TCCR1B |= (clockSource); \
00245     } while (0)
00246 #define TIMER2_SELECT_CLOCK_SOURCE(clockSource) \
00247     do { \
00248         TCCR2B &= 0xF8; \
00249         TCCR2B |= (clockSource); \
00250     } while (0)
00251
00252 #define TIMER0_SELECT_OPERATION_MODE(operationMode) \
00253     do { \
00254         TCCR0A &= 0xFC; \
00255         TCCR0A |= ((operationMode) & 3); \
00256         TCCR0B &= 0xF7; \
00257         TCCR0B |= (((operationMode) & 12) « 1); \
00258     } while (0)
00259 #define TIMER1_SELECT_OPERATION_MODE(operationMode) \
00260     do { \
00261         TCCR1A &= 0xFC; \
00262         TCCR1A |= ((operationMode) & 3); \
00263         TCCR1B &= 0xE7; \
00264         TCCR1B |= (((operationMode) & 12) « 1); \
00265     } while (0)
00266 #define TIMER2_SELECT_OPERATION_MODE(operationMode) \
00267     do { \
00268         TCCR2A &= 0xFC; \
00269         TCCR2A |= ((operationMode) & 3); \
00270         TCCR2B &= 0xF7; \
00271         TCCR2B |= (((operationMode) & 12) « 1); \
00272     } while (0)
00273
00274 #define TIMER0_SELECT_COM_CHANNEL_A(compareOutputMode) \
00275     do { \
00276         TCCR0A &= 0x3F; \
00277         TCCR0A |= (compareOutputMode) « 6; \
00278     } while (0)
00279 #define TIMER0_SELECT_COM_CHANNEL_B(compareOutputMode) \
00280     do { \
```

```
00281         TCCR0A &= 0xCF; \
00282         TCCR0A |= (compareOutputMode) « 4; \
00283     } while (0)
00284
00285 #define TIMER1_SELECT_COM_CHANNEL_A(compareOutputMode) \
00286     do { \
00287         TCCR1A &= 0x3F; \
00288         TCCR1A |= (compareOutputMode) « 6; \
00289     } while (0)
00290 #define TIMER1_SELECT_COM_CHANNEL_B(compareOutputMode) \
00291     do { \
00292         TCCR1A &= 0xCF; \
00293         TCCR1A |= (compareOutputMode) « 4; \
00294     } while (0)
00295
00296 #define TIMER2_SELECT_COM_CHANNEL_A(compareOutputMode) \
00297     do { \
00298         TCCR2A &= 0x3F; \
00299         TCCR2A |= (compareOutputMode) « 6; \
00300     } while (0)
00301 #define TIMER2_SELECT_COM_CHANNEL_B(compareOutputMode) \
00302     do { \
00303         TCCR2A &= 0xCF; \
00304         TCCR2A |= (compareOutputMode) « 4; \
00305     } while (0)
00306
00307 #define TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 |=  1 « OCIE0A
00308 #define TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK0 &= ~(1 « OCIE0A)
00309
00310 #define TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 |=  1 « OCIE0B
00311 #define TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK0 &= ~(1 « OCIE0B)
00312
00313 #define TIMER0_ENABLE_OVERFLOW_INTERRUPT TIMSK0 |=  1 « TOIE0
00314 #define TIMER0_DISABLE_OVERFLOW_INTERRUPT TIMSK0 &= ~(1 « TOIE0)
00315
00316 #define TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 |=  1 « OCIE1A
00317 #define TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK1 &= ~(1 « OCIE1A)
00318
00319 #define TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 |=  1 « OCIE1B
00320 #define TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK1 &= ~(1 « OCIE1B)
00321
00322 #define TIMER1_ENABLE_OVERFLOW_INTERRUPT TIMSK1 |=  1 « TOIE1
00323 #define TIMER1_DISABLE_OVERFLOW_INTERRUPT TIMSK1 &= ~(1 « TOIE1)
00324
00325 #define TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 |=  1 « ICIE1
00326 #define TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT TIMSK1 &= ~(1 « ICIE1)
00327
00328 #define TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 |=  1 « OCIE2A
00329 #define TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT TIMSK2 &= ~(1 « OCIE2A)
00330
00331 #define TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 |=  1 « OCIE2B
00332 #define TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT TIMSK2 &= ~(1 « OCIE2B)
00333
00334 #define TIMER2_ENABLE_OVERFLOW_INTERRUPT TIMSK2 |=  1 « TOIE2
00335 #define TIMER2_DISABLE_OVERFLOW_INTERRUPT TIMSK2 &= ~(1 « TOIE2)
00336
00337 #define   TIMER0_COM_CHANNEL_A_INTERRUPT  TIMER0_COMPA_vect
00338 #define   TIMER0_COM_CHANNEL_B_INTERRUPT  TIMER0_COMPB_vect
00339 #define   TIMER0_OVERFLOW_INTERRUPT  TIMER0_OVF_vect
00340
00341 #define   TIMER1_COM_CHANNEL_A_INTERRUPT  TIMER1_COMPA_vect
00342 #define   TIMER1_COM_CHANNEL_B_INTERRUPT  TIMER1_COMPB_vect
00343 #define   TIMER1_OVERFLOW_INTERRUPT  TIMER1_OVF_vect
00344 #define   TIMER1_INPUT_CAPTURE_INTERRUPT  TIMER1_CAPT_vect
00345
00346 #define   TIMER2_COM_CHANNEL_A_INTERRUPT  TIMER2_COMPA_vect
00347 #define   TIMER2_COM_CHANNEL_B_INTERRUPT  TIMER2_COMPB_vect
00348 #define   TIMER2_OVERFLOW_INTERRUPT  TIMER2_OVF_vect
00349
00350 // Watchdog Timer defines
00351
00352 #define WATCHDOG_SELECT_TIMEOUT(timeOut) \
00353     do { \
00354         WDTCSR |= (1«WDCE) | (1«WDE); \
00355         WDTCSR = (0«WDIE) | (0«WDE) | (timeOut); \
00356     } while (0)
00357 #define WATCHDOG_START(operationMode,timeOut) \
00358     do { \
00359         MCUSR &= ~(1«WDRF); \
00360         WDTCSR |= (1«WDCE) | (1«WDE); \
00361         WDTCSR = (operationMode) | (timeOut); \
00362     } while (0)
00363 #define WATCHDOG_STOP \
00364     do { \
00365         MCUSR &= ~(1«WDRF); \
00366         WDTCSR |= (1«WDCE) | (1«WDE); \
00367         WDTCSR = 0x00; \
```

```
00368     } while (0)
00369 #define WATCHDOG_TIMEOUT_INTERRUPT  WDT_vect
00370
00371 // MCU defines
00372
00373
00374 #define MCU_SELECT_SLEEP_MODE(sleepMode) \
00375     do { \
00376         SMCR &= 0xF1; \
00377         SMCR |= (sleepMode) « 1; \
00378     } while (0)
00379
00380 #define MCU_SLEEP_ENABLE SMCR |=  1 « SE
00381 #define MCU_SLEEP_DISABLE SMCR &= ~(1 « SE)
00382
00383 #define MCU_TWI_ENABLE PRR &= ~(1 « PRTWI)
00384 #define MCU_TWI_DISABLE PRR |=  1 « PRTWI
00385
00386 #define MCU_TIMER2_ENABLE PRR &= ~(1 « PRTIM2)
00387 #define MCU_TIMER2_DISABLE PRR |=  1 « PRTIM2
00388
00389 #define MCU_TIMER1_ENABLE PRR &= ~(1 « PRTIM1)
00390 #define MCU_TIMER1_DISABLE PRR |=  1 « PRTIM1
00391
00392 #define MCU_TIMER0_ENABLE PRR &= ~(1 « PRTIM0)
00393 #define MCU_TIMER0_DISABLE PRR |=  1 « PRTIM0
00394
00395 #define MCU_SPI_ENABLE PRR &= ~(1 « PRSPI)
00396 #define MCU_SPI_DISABLE PRR |=  1 « PRSPI
00397
00398 #define MCU_USART0_ENABLE PRR &= ~(1 « PRUSART0)
00399 #define MCU_USART0_DISABLE PRR |=  1 « PRUSART0
00400
00401 #define MCU_ADC_ENABLE PRR &= ~(1 « PRADC)
00402 #define MCU_ADC_DISABLE PRR |=  1 « PRADC
00403
00404
00405 // ServoMotor defines
00406
00407 // Timer count for a servo pulse width given in microseconds: the pulse
00408 // width is multiplied by the timer tick rate (F_CPU / 1 MHz) first and only
00409 // then divided by the prescaler, so no precision is lost to integer
00410 // division (1500 us at prescaler 8 gives exactly 3000 ticks at F_CPU=16 MHz).
00411 #define SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT(pulseWidth,clockPrescaler) \
00412     ((((uint32_t)(pulseWidth)) * ((F_CPU) / 1000000UL)) / (clockPrescaler))
00413
00414 #define SERVOMOTOR_TIMER_ANGLE_COUNT(angle,out_min,out_mid,out_max) (((((out_min)*((angle) -
      90L)*((angle) - 180L))/16200L) + (((angle)*(out_max)*((angle) - 90L))/16200L) -
      (((angle)*(out_mid)*((angle) - 180L))/8100L)))
00415
00416 // SPI defines
00417
00418 #define   SPI_ENABLE            SPCR |=  1 « SPE
00419 #define   SPI_DISABLE           SPCR &= ~(1 « SPE)
00420 #define   SPI_SELECT_MASTER_MODE SPCR |=  1 « MSTR
00421 #define   SPI_SELECT_SLAVE_MODE SPCR &= ~(1 « MSTR)
00422
00423
00424 #define   SPI_SELECT_DATA_MODE(dataMode) \
00425     do { \
00426         SPCR &= 0xF3; \
00427         SPCR |= (dataMode) « 2; \
00428     } while (0)
00429 #define   SPI_SELECT_CLOCK_PRESCALER(clockPrescaler) \
00430     do { \
00431         SPCR &= 0xFC; \
00432         SPCR |= ((clockPrescaler) & 3); \
00433         SPSR &= 0xFE; \
00434         SPSR |= (((clockPrescaler) & 4) » 2); \
00435     } while (0)
00436 #define   SPI_SELECT_DATA_ORDER(dataOrder) \
00437     do { \
00438         SPCR &= 0xDF; \
00439         SPCR |= (dataOrder) « 5; \
00440     } while (0)
00441
00442 #define   SPI_WRITE_COLLISION WCOL
00443 #define   SPI_TRANSFER_COMPLETE SPIF
00444
00445 #define   SPI_MASTER_MODE MSTR
00446 #define   SPI_CONTROL_REGISTER SPCR
00447 #define   SPI_DATA_REGISTER SPDR
00448 #define   SPI_STATUS_REGISTER SPSR
00449
00450 #define   SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT SPCR |= 1 « SPIE
00451 #define   SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT SPCR &= ~(1 « SPIE)
00452
```

```
00453
00454 #define   SPI_TRANSFER_COMPLETE_INTERRUPT SPI_STC_vect
00455
00456 // StepperMotor defines
00457
00458
00459
00460
00461
00462
00463 #endif
00464
```

9.147 PIDController.cpp File Reference

Implementation of the PID Controller class.

```
#include "PIDController.h"
#include <math.h>
```

9.147.1 Detailed Description

Implementation of the PID Controller class.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file PIDController.cpp.

9.148 PIDController.cpp

Go to the documentation of this file.

```
00001
00008
00009 #include "PIDController.h"
00010 #include <math.h>
00011
00012 utils::PIDController::PIDController(float a_kp, float a_ki, float a_kd, float a_minOutput, float
      a_maxOutput)
00013     : m_integral(0.0f),
00014       m_prevInput(0.0f),
00015       m_firstRun(1)
00016 {
00017     setGains(a_kp, a_ki, a_kd);
00018     setOutputLimits(a_minOutput, a_maxOutput);
00019 }
00020
00021 void utils::PIDController::setGains(float a_kp, float a_ki, float a_kd)
00022 {
00023     m_kp = (isnan(a_kp) || isinf(a_kp)) ? 0.0f : a_kp;
00024     m_ki = (isnan(a_ki) || isinf(a_ki)) ? 0.0f : a_ki;
00025     m_kd = (isnan(a_kd) || isinf(a_kd)) ? 0.0f : a_kd;
00026 }
00027
00028 void utils::PIDController::setOutputLimits(float a_minOutput, float a_maxOutput)
```

```
00029 {
00030     if (isnan(a_minOutput) || isinf(a_minOutput)) a_minOutput = -255.0f;
00031     if (isnan(a_maxOutput) || isinf(a_maxOutput)) a_maxOutput = 255.0f;
00032 
00033     if (a_minOutput > a_maxOutput) {
00034         float tmp = a_minOutput;
00035         a_minOutput = a_maxOutput;
00036         a_maxOutput = tmp;
00037     }
00038 
00039     m_minOutput = a_minOutput;
00040     m_maxOutput = a_maxOutput;
00041 }
00042 
00043 void utils::PIDController::reset()
00044 {
00045     m_integral = 0.0f;
00046     m_prevInput = 0.0f;
00047     m_firstRun = 1;
00048 }
00049 
00050 float utils::PIDController::compute(float a_setpoint, float a_input, float a_dt_s)
00051 {
00052     if (isnan(a_setpoint) || isinf(a_setpoint) || isnan(a_input) || isinf(a_input) || isnan(a_dt_s) ||
      isinf(a_dt_s) || a_dt_s <= 0.0f) {
00053         return 0.0f;
00054     }
00055 
00056     if (m_firstRun) {
00057         m_prevInput = a_input;
00058         m_firstRun = 0;
00059     }
00060 
00061     float error = a_setpoint - a_input;
00062 
00063     // Proportional term
00064     float p_out = m_kp * error;
00065 
00066     // Integral term
00067     m_integral += error * a_dt_s;
00068     float i_out = m_ki * m_integral;
00069 
00070     // Derivative on measurement (prevents derivative kick on setpoint change)
00071     float d_input = (a_input - m_prevInput) / a_dt_s;
00072     float d_out = -m_kd * d_input;
00073 
00074     float total = p_out + i_out + d_out;
00075 
00076     // Anti-windup clamping
00077     if (total > m_maxOutput) {
00078         total = m_maxOutput;
00079         m_integral -= error * a_dt_s; // freeze integration
00080     } else if (total < m_minOutput) {
00081         total = m_minOutput;
00082         m_integral -= error * a_dt_s;
00083     }
00084 
00085     m_prevInput = a_input;
00086     return total;
00087 }
```

9.149 PIDController.h File Reference

Header file for the PID Controller class.

```
#include <stdint.h>
```

Classes

- class utils::PIDController

Namespaces

- namespace utils

9.149.1 Detailed Description

Header file for the PID Controller class.

Author

Farid Oubbati (https://github.com/faroub)

Date

March 2026

Definition in file PIDController.h.

9.150 PIDController.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef PID_CONTROLLER_H
00010 #define PID_CONTROLLER_H
00011
00012 #include <stdint.h>
00013
00014 namespace utils
00015 {
00016
00017 class PIDController
00018 {
00019 public:
00028     PIDController(float a_kp, float a_ki, float a_kd, float a_minOutput = -255.0f, float a_maxOutput =
      255.0f);
00029
00032     void setGains(float a_kp, float a_ki, float a_kd);
00033
00036     void setOutputLimits(float a_minOutput, float a_maxOutput);
00037
00040     void reset();
00041
00049     float compute(float a_setpoint, float a_input, float a_dt_s);
00050
00051 private:
00052     float m_kp;
00053     float m_ki;
00054     float m_kd;
00055     float m_minOutput;
00056     float m_maxOutput;
00057     float m_integral;
00058     float m_prevInput;
00059     uint8_t m_firstRun;
00060 };
00061
00062 }
00063
00064 #endif
```

9.151 utils_m328p.cpp File Reference

```
#include "utils_m328p.h"
```

9.152 utils_m328p.cpp

Go to the documentation of this file.

```
00001 #include "utils_m328p.h"
00002
00003 long utils::map(long x, long in_min, long in_max, long out_min, long out_max)
00004 {
00005   return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
00006 }
```

9.153 utils_m328p.h File Reference

Namespaces

- namespace utils

Functions

- long utils::map (long x, long in_min, long in_max, long out_min, long out_max)

9.154 utils_m328p.h

Go to the documentation of this file.

```
00001 #ifndef UTILS_M328P_H
00002 #define UTILS_M328P_H
00003 namespace utils
00004 {
00005
00006
00007 // delay functions
00008
00009
00010
00011 // multiplication / division operations bit-shift devide multiply
00012
00013
00014 // clock division (set prescaler)
00015
00016 // clock power saving through macros
00017
00018
00019 // mapping values
00020
00021 long map(long x, long in_min, long in_max, long out_min, long out_max);
00022
00023
00024 }
00025 #endif
```

Index

__externally_visible__
core::ADConverter, 60
core::ExternInterrupt, 73
core::TimerCounter0, 151
core::TimerCounter1, 162
core::TimerCounter2, 172
core::WatchdogTimer, 200
io::SPI, 119
io::USART0, 193
__used__
core::ADConverter, 60
core::ExternInterrupt, 73
core::TimerCounter0, 151
core::TimerCounter1, 162
core::TimerCounter2, 172
core::WatchdogTimer, 200
io::SPI, 119
io::USART0, 193
~ADConverter
core::ADConverter, 53
~Buzzer
component::Buzzer, 62
~DCMotor
component::DCMotor, 66
~ExternInterrupt
core::ExternInterrupt, 69
~LDR
component::LDR, 75
~Led
component::Led, 78
~Pin
io::Pin, 95
~PushButton
component::PushButton, 101
~SPI
io::SPI, 114
~ServoMotor
component::ServoMotor, 107
~StepperMotor
component::StepperMotor, 126
~TWI
core::TWI, 174
~TimerCounter
core::TimerCounter, 140
~TimerCounter0
core::TimerCounter0, 144
~TimerCounter1
core::TimerCounter1, 153
~TimerCounter2
core::TimerCounter2, 165
~USART0
io::USART0, 179
~WatchdogTimer
core::WatchdogTimer, 197

A
core, 36
A0
buzzer_pitches_16bit.h, 350
buzzer_pitches_8bit.h, 366
A1
buzzer_pitches_16bit.h, 350
buzzer_pitches_8bit.h, 366
A2
buzzer_pitches_16bit.h, 350
buzzer_pitches_8bit.h, 366
A3
buzzer_pitches_16bit.h, 350
A4
buzzer_pitches_16bit.h, 350
A5
buzzer_pitches_16bit.h, 351
A6
buzzer_pitches_16bit.h, 351
A7
buzzer_pitches_16bit.h, 351
ADC.cpp, 270
ADC.h, 274
ADC_ADJUST_RESULT_LEFT
ha_attiny85.h, 372
ha_esp32.h, 382
ha_m2560.h, 399
ha_m328p.h, 437
ADC_ADJUST_RESULT_RIGHT
ha_attiny85.h, 372
ha_esp32.h, 382
ha_m2560.h, 399
ha_m328p.h, 437
ADC_CONVERSION_COMPLETE_INTERRUPT
ha_attiny85.h, 372
ha_esp32.h, 382
ha_m2560.h, 399
ha_m328p.h, 437
ADC_DISABLE
ha_attiny85.h, 372
ha_esp32.h, 382
ha_m2560.h, 400
ha_m328p.h, 437
ADC_DISABLE_AUTOTRIGGER

ha_attiny85.h, 373
ha_esp32.h, 382
ha_m2560.h, 400
ha_m328p.h, 437
ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT
ha_attiny85.h, 373
ha_esp32.h, 382
ha_m2560.h, 400
ha_m328p.h, 437
ADC_DISABLE_DIGITAL_INPUT_REGISTER
ha_attiny85.h, 373
ha_esp32.h, 382
ha_m2560.h, 400
ha_m328p.h, 438
ADC_ENABLE
ha_attiny85.h, 373
ha_esp32.h, 382
ha_m2560.h, 400
ha_m328p.h, 438
ADC_ENABLE_AUTOTRIGGER
ha_attiny85.h, 373
ha_esp32.h, 383
ha_m2560.h, 400
ha_m328p.h, 438
ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT
ha_attiny85.h, 374
ha_esp32.h, 383
ha_m2560.h, 401
ha_m328p.h, 438
ADC_NoiseReduction
core, 41
ADC_SELECT_ANALOG_INPUT
ha_attiny85.h, 374
ha_esp32.h, 383
ha_m2560.h, 401
ha_m328p.h, 438
ADC_SELECT_AUTO_TRIGGER_SOURCE
ha_esp32.h, 383
ha_m2560.h, 401
ha_m328p.h, 438
ADC_SELECT_CLOCK_PRESCALER
ha_attiny85.h, 374
ha_esp32.h, 383
ha_m2560.h, 401
ha_m328p.h, 439
ADC_SELECT_REF_VOLTAGE
ha_attiny85.h, 374
ha_esp32.h, 383
ha_m2560.h, 402
ha_m328p.h, 439
ADC_START_CONVERSION
ha_attiny85.h, 375
ha_esp32.h, 384
ha_m2560.h, 402
ha_m328p.h, 439
ADC_STOP_CONVERSION
ha_attiny85.h, 375
ha_esp32.h, 384
ha_m2560.h, 402
ha_m328p.h, 439
ADC_ZERO_SENTINEL
component::LDR, 77
ADConverter
core::ADConverter, 52, 53
ADCSample
io, 47
analogComparator
core, 35
applications Directory Reference, 15
AREF
core, 39
async
io, 47
AutonomousRobotController Directory Reference, 15
autoTriggerSource
core, 35
AVCC
core, 39
Ax0
buzzer_pitches_16bit.h, 351
buzzer_pitches_8bit.h, 366
Ax1
buzzer_pitches_16bit.h, 351
buzzer_pitches_8bit.h, 366
Ax2
buzzer_pitches_16bit.h, 351
buzzer_pitches_8bit.h, 366
Ax3
buzzer_pitches_16bit.h, 351
Ax4
buzzer_pitches_16bit.h, 352
Ax5
buzzer_pitches_16bit.h, 352
Ax6
buzzer_pitches_16bit.h, 352
Ax7
buzzer_pitches_16bit.h, 352

B
core, 36
B0
buzzer_pitches_16bit.h, 352
buzzer_pitches_8bit.h, 367
B1
buzzer_pitches_16bit.h, 352
buzzer_pitches_8bit.h, 367
B2
buzzer_pitches_16bit.h, 352
buzzer_pitches_8bit.h, 367
B3
buzzer_pitches_16bit.h, 353
B4
buzzer_pitches_16bit.h, 353
B5
buzzer_pitches_16bit.h, 353
B6
buzzer_pitches_16bit.h, 353

B7
buzzer_pitches_16bit.h, 353
beginMove
component::StepperMotor, 126
BlinkLed Directory Reference, 16
BODMode
core, 36
buzz
component::Buzzer, 63
BUZZER
main.cpp, 215, 219
Buzzer
component::Buzzer, 62
Buzzer.cpp, 237, 238
Buzzer.h, 238, 239
buzzer_pitches_16bit.h, 348, 364
A0, 350
A1, 350
A2, 350
A3, 350
A4, 350
A5, 351
A6, 351
A7, 351
Ax0, 351
Ax1, 351
Ax2, 351
Ax3, 351
Ax4, 352
Ax5, 352
Ax6, 352
Ax7, 352
B0, 352
B1, 352
B2, 352
B3, 353
B4, 353
B5, 353
B6, 353
B7, 353
C0, 353
C1, 353
C2, 354
C3, 354
C4, 354
C5, 354
C6, 354
C7, 354
Cx0, 354
Cx1, 355
Cx2, 355
Cx3, 355
Cx4, 355
Cx5, 355
Cx6, 355
Cx7, 355
D0, 356
D1, 356
D2, 356
D3, 356
D4, 356
D5, 356
D6, 356
D7, 357
Dx0, 357
Dx1, 357
Dx2, 357
Dx3, 357
Dx4, 357
Dx5, 357
Dx6, 358
Dx7, 358
E0, 358
E1, 358
E2, 358
E3, 358
E4, 358
E5, 359
E6, 359
E7, 359
F0, 359
F1, 359
F2, 359
F3, 359
F4, 360
F5, 360
F6, 360
F7, 360
Fx0, 360
Fx1, 360
Fx2, 360
Fx3, 361
Fx4, 361
Fx5, 361
Fx6, 361
Fx7, 361
G0, 361
G1, 361
G2, 362
G3, 362
G4, 362
G5, 362
G6, 362
G7, 362
Gx0, 362
Gx1, 363
Gx2, 363
Gx3, 363
Gx4, 363
Gx5, 363
Gx6, 363
Gx7, 363
buzzer_pitches_8bit.h, 365, 371
A0, 366
A1, 366
A2, 366

Ax0, 366
Ax1, 366
Ax2, 366
B0, 367
B1, 367
B2, 367
C1, 367
C2, 367
C3, 367
Cx0, 367
Cx2, 367
Cx3, 368
D1, 368
D2, 368
D3, 368
Dx0, 368
Dx2, 368
Dx3, 368
E1, 368
E2, 369
E3, 369
F1, 369
F2, 369
F3, 369
Fx1, 369
Fx2, 369
Fx3, 369
G1, 370
G2, 370
G3, 370
Gx0, 370
Gx1, 370
Gx2, 370

C0
buzzer_pitches_16bit.h, 353
C1
buzzer_pitches_16bit.h, 353
buzzer_pitches_8bit.h, 367
C2
buzzer_pitches_16bit.h, 354
buzzer_pitches_8bit.h, 367
C3
buzzer_pitches_16bit.h, 354
buzzer_pitches_8bit.h, 367
C4
buzzer_pitches_16bit.h, 354
C5
buzzer_pitches_16bit.h, 354
C6
buzzer_pitches_16bit.h, 354
C7
buzzer_pitches_16bit.h, 354
channel
core, 36
clampTimerCount
component::ServoMotor, 108
clear
component::PCF8574_LCD, 86
component::SSD1306_OLED, 122
core, 38
clockPrescaler
core, 36
io, 44
clockSource
core, 37
communicationMode
io, 44
compareOutputMode
core, 37
component, 33
fullStep, 34
halfStep, 34
mode, 34
component Directory Reference, 17
component::Buzzer, 61
~Buzzer, 62
buzz, 63
Buzzer, 62
m_pin, 65
pause, 64
component::DCMotor, 65
~DCMotor, 66
connect, 66
DCMotor, 66
disconnect, 66
m_pin, 68
off, 67
on, 67
spin, 67
stop, 67
toggle, 68
component::LDR, 74
~LDR, 75
ADC_ZERO_SENTINEL, 77
getLux, 75
getResistance, 76
LDR, 75
m_pin, 77
mr_ADC, 77
component::Led, 77
~Led, 78
isOff, 79
isOn, 79
Led, 78
m_pin, 80
off, 79
on, 80
toggle, 80
component::PCF8574_LCD, 85
clear, 86
init, 86
m_backlightMask, 89
m_cols, 89
m_i2cAddr, 89
m_rows, 89
mr_twi, 89

PCF8574_LCD, 86
print, 87
sendByte, 87
sendNibble, 87
setBacklight, 88
setCursor, 88
component::PushButton, 99
~PushButton, 101
getPressedCount, 101
isPressed, 101
m_buttonPressed, 103
m_isActiveLow, 103
m_lastState, 103
m_pin, 103
m_useInternalPullUp, 103
PushButton, 100
resetPressedCount, 102
component::RotaryEncoder, 104
getDirection, 105
getPosition, 105
m_direction, 106
m_lastStateA, 106
m_lastStateB, 106
m_pinA, 106
m_pinB, 106
m_position, 106
resetPosition, 105
RotaryEncoder, 104
update, 105
component::ServoMotor, 107
~ServoMotor, 107
clampTimerCount, 108
computePulseCycleCount, 108
computePulseWidthMaxCount, 108
computePulseWidthMidCount, 109
computePulseWidthMinCount, 109
computeRotationAngleCount, 109
connect, 109
disconnect, 110
m_pin, 111
m_pulseCycle, 111
m_pulseWidthMax, 111
m_pulseWidthMid, 112
m_pulseWidthMin, 112
off, 110
on, 110
rotate, 110
ServoMotor, 107
toggle, 111
component::SSD1306_OLED, 120
clear, 122
init, 122
m_height, 124
m_i2cAddr, 124
m_width, 124
mr_twi, 124
print, 122
sendCommand, 123
sendData, 123
setCursor, 123
SSD1306_OLED, 121
component::StepperMotor, 125
~StepperMotor, 126
beginMove, 126
computeStepDelay, 127
currentPos, 128
goalReached, 128
m_accelTime, 133
m_constSpeedTime, 133
m_currentPos, 133
m_decelTime, 134
m_goalReached, 134
m_pinCoil1, 134
m_pinCoil2, 134
m_pinCoil3, 134
m_pinCoil4, 135
m_profilePosition, 135
m_profileSpeed, 135
m_profileTime, 135
m_speedTimeProduct, 135
m_stepNumber, 136
m_targetStep, 136
setCurrentPos, 129
step, 129, 130
stepDelay, 132
stepMode, 136
StepperMotor, 126
stepPulse, 132
compute
utils::PIDController, 91
computePulseCycleCount
component::ServoMotor, 108
computePulseWidthMaxCount
component::ServoMotor, 108
computePulseWidthMidCount
component::ServoMotor, 109
computePulseWidthMinCount
component::ServoMotor, 109
computeRotationAngleCount
component::ServoMotor, 109
computeStepDelay
component::StepperMotor, 127
connect
component::DCMotor, 66
component::ServoMotor, 109
conversionCompleteServiceRoutine
core::ADConverter, 53
core, 34
A, 36
ADC_NoiseReduction, 41
analogComparator, 35
AREF, 39
autoTriggerSource, 35
AVCC, 39
B, 36
BODMode, 36

channel, 36
clear, 38
clockPrescaler, 36
clockSource, 37
compareOutputMode, 37
CTC_ICR, 38
CTC_OCR, 38
disabled, 36
enabled, 36
extendedStandby, 41
extern_Clock_T0_Falling_Edge, 37
extern_Clock_T0_Rising_Edge, 37
extInterrupt, 35
fallingEdge, 40
fast_PWM, 38
fast_PWM_10bit, 38
fast_PWM_8bit, 38
fast_PWM_9bit, 38
fast_PWM_ICR, 38
fast_PWM_OCR, 38
freeRunning, 35
Idle, 40
internal, 39
interrupt, 42
interrupt_reset, 42
logicalChange, 40
lowLevel, 40
noClock, 37
normal, 38
operationMode, 38
operationModeToWDTCSR, 42
PCINTB, 39
PCINTC, 39
PCINTD, 39
pinChangePort, 39
powerDown, 41
powerSave, 41
PS_1, 37
PS_1024, 37
PS_128, 37
PS_16, 37
PS_2, 37
PS_256, 37
PS_32, 37
PS_4, 37
PS_64, 37
PS_8, 37
PWM_PC, 38
PWM_PC_10bit, 38
PWM_PC_8bit, 38
PWM_PC_9bit, 38
PWM_PC_ICR, 38
PWM_PC_OCR, 38
PWM_PFC_ICR, 38
PWM_PFC_OCR, 38
referenceVoltage, 39
res_10bit, 40
res_11bit, 40
res_12bit, 40
res_13bit, 40
res_14bit, 40
res_15bit, 40
res_16bit, 40
res_8bit, 40
res_9bit, 40
reset, 42
resolution, 39
risingEdge, 40
senseControl, 40
set, 38
sleepMode, 40
standby, 41
timeOut, 41
timeOutToWDTCSR, 42
timer0CompareA, 35
timer0Overflow, 35
timer1Capture, 36
timer1CompareB, 35
timer1Overflow, 35
to_125ms, 41
to_16ms, 41
to_1s, 41
to_250ms, 41
to_2s, 41
to_32ms, 41
to_4s, 41
to_500ms, 41
to_64ms, 41
to_8s, 41
toggle, 38
watchdogOperationMode, 41
core Directory Reference, 18
core::ADConverter, 51
__externally_visible__, 60
__used__, 60
~ADConverter, 53
ADConverter, 52, 53
conversionCompleteServiceRoutine, 53
enableAutoTrigger, 55
enableConversionCompleteInterrupt, 55
getConversionResult, 56
getInstance, 57
m_conversionResult, 61
m_resolution, 61
operator=, 58
selectAnalogInput, 58
selectAutoTriggerSource, 58
selectClockPrescaler, 59
selectReferenceVoltage, 59
start, 60
stop, 60
core::ExternInterrupt, 68
__externally_visible__, 73
__used__, 73
~ExternInterrupt, 69
enableInt0, 70

enableInt1, 70
enablePinChange, 70
enablePinChangeMaskPortB, 71
enablePinChangeMaskPortC, 71
enablePinChangeMaskPortD, 71
ExternInterrupt, 69
getInstance, 72
Int0ServiceRoutine, 72
Int1ServiceRoutine, 72
operator=, 72
pinChangePortBServiceRoutine, 72
pinChangePortCServiceRoutine, 73
pinChangePortDServiceRoutine, 73
setInt0SenseControl, 73
setInt1SenseControl, 73
weak, 74
core::MCU, 81
enableADC, 81
enableInterrupts, 81
enableSPI, 81
enableTimerCounter0, 82
enableTimerCounter1, 82
enableTimerCounter2, 82
enableTWI, 83
enableUSART0, 83
goToSleep, 83
init, 84
selectSleepMode, 84
sleepEnable, 85
core::TimerCounter, 139
~TimerCounter, 140
enableOutputCompareMatchInterrupt, 140
enableOverflowInterrupt, 140
getClockPrescaler, 141
getCounter, 141
getOutputCompareRegister, 141
operator delete, 141
selectClockSource, 141
selectCompareOutputMode, 141
selectOperationMode, 142
setCounter, 142
setOutputCompareRegister, 142
start, 142
stop, 142
core::TimerCounter0, 143
__externally_visible__, 151
__used__, 151
~TimerCounter0, 144
enableOutputCompareMatchInterrupt, 145
enableOverflowInterrupt, 145
getClockPrescaler, 146
getCounter, 146
getInstance, 146
getOutputCompareRegister, 146
m_clockPrescaler, 151
m_clockSource, 151
operator=, 147
outputCompareMatchAServiceRoutine, 147
outputCompareMatchBServiceRoutine, 147
overflowServiceRoutine, 147
selectClockSource, 147
selectCompareOutputMode, 148
selectOperationMode, 149
setCounter, 150
setOutputCompareRegister, 150
start, 150
stop, 151
TimerCounter0, 144
weak, 152
core::TimerCounter1, 152
__externally_visible__, 162
__used__, 162
~TimerCounter1, 153
enableInputCaptureInterrupt, 154
enableOutputCompareMatchInterrupt, 154
enableOverflowInterrupt, 155
getClockPrescaler, 155
getCounter, 155
getInputCaptureRegister, 156
getInstance, 156
getOutputCompareRegister, 156
inputCaptureServiceRoutine, 157
m_clockPrescaler, 163
m_clockSource, 163
operator=, 157
outputCompareMatchAServiceRoutine, 157
outputCompareMatchBServiceRoutine, 157
overflowServiceRoutine, 158
selectClockSource, 158
selectCompareOutputMode, 159
selectOperationMode, 159
setCounter, 161
setInputCaptureRegister, 161
setOutputCompareRegister, 161
start, 162
stop, 162
TimerCounter1, 153, 154
weak, 163
core::TimerCounter2, 163
__externally_visible__, 172
__used__, 172
~TimerCounter2, 165
enableOutputCompareMatchInterrupt, 166
enableOverflowInterrupt, 166
getClockPrescaler, 166
getCounter, 167
getInstance, 167
getOutputCompareRegister, 167
m_clockPrescaler, 172
m_clockSource, 172
operator=, 168
outputCompareMatchAServiceRoutine, 168
outputCompareMatchBServiceRoutine, 168
overflowServiceRoutine, 168
selectClockSource, 168
selectCompareOutputMode, 169

selectOperationMode, 170
setCounter, 171
setOutputCompareRegister, 171
start, 171
stop, 172
TimerCounter2, 165
weak, 173
core::TWI, 173
~TWI, 174
getInstance, 174
operator=, 174
readAck, 175
readNack, 175
setClock, 175
start, 176
stop, 176
TWI, 174
write, 176
core::WatchdogTimer, 196
__externally_visible__, 200
__used__, 200
~WatchdogTimer, 197
getInstance, 197
m_isRunning, 200
m_operationMode, 200
m_timeOut, 200
operator=, 197
reset, 198
selectTimeOut, 198
start, 198, 199
stop, 199
timeOutServiceRoutine, 199
WatchdogTimer, 197
weak, 200
CTC_ICR
core, 38
CTC_OCR
core, 38
currentPos
component::StepperMotor, 128
CustomLog
io, 47
Cx0
buzzer_pitches_16bit.h, 354
buzzer_pitches_8bit.h, 367
Cx1
buzzer_pitches_16bit.h, 355
Cx2
buzzer_pitches_16bit.h, 355
buzzer_pitches_8bit.h, 367
Cx3
buzzer_pitches_16bit.h, 355
buzzer_pitches_8bit.h, 368
Cx4
buzzer_pitches_16bit.h, 355
Cx5
buzzer_pitches_16bit.h, 355
Cx6
buzzer_pitches_16bit.h, 355
Cx7
buzzer_pitches_16bit.h, 355
D0
buzzer_pitches_16bit.h, 356
D1
buzzer_pitches_16bit.h, 356
buzzer_pitches_8bit.h, 368
D2
buzzer_pitches_16bit.h, 356
buzzer_pitches_8bit.h, 368
D3
buzzer_pitches_16bit.h, 356
buzzer_pitches_8bit.h, 368
D4
buzzer_pitches_16bit.h, 356
D5
buzzer_pitches_16bit.h, 356
D6
buzzer_pitches_16bit.h, 356
D7
buzzer_pitches_16bit.h, 357
dataMode
io, 45
dataOrder
io, 45
dataOverrun
io::USART0, 180
dataRegisterEmptyServiceRoutine
io::USART0, 180
DCMotor
component::DCMotor, 66
DCMotor.cpp, 240
DCMotor.h, 240, 242
disable
io, 46
disabled
core, 36
disconnect
component::DCMotor, 66
component::ServoMotor, 110
duplex
io, 45
Dx0
buzzer_pitches_16bit.h, 357
buzzer_pitches_8bit.h, 368
Dx1
buzzer_pitches_16bit.h, 357
Dx2
buzzer_pitches_16bit.h, 357
buzzer_pitches_8bit.h, 368
Dx3
buzzer_pitches_16bit.h, 357
buzzer_pitches_8bit.h, 368
Dx4
buzzer_pitches_16bit.h, 357
Dx5
buzzer_pitches_16bit.h, 357

Dx6
buzzer_pitches_16bit.h, 358
Dx7
buzzer_pitches_16bit.h, 358

E0
buzzer_pitches_16bit.h, 358
E1
buzzer_pitches_16bit.h, 358
buzzer_pitches_8bit.h, 368
E2
buzzer_pitches_16bit.h, 358
buzzer_pitches_8bit.h, 369
E3
buzzer_pitches_16bit.h, 358
buzzer_pitches_8bit.h, 369
E4
buzzer_pitches_16bit.h, 358
E5
buzzer_pitches_16bit.h, 359
E6
buzzer_pitches_16bit.h, 359
E7
buzzer_pitches_16bit.h, 359
eightBits
io, 46
enableADC
core::MCU, 81
enableAutoTrigger
core::ADConverter, 55
enableConversionCompleteInterrupt
core::ADConverter, 55
enabled
core, 36
enableDataRegisterEmptyInterrupt
io::USART0, 180
enableInputCaptureInterrupt
core::TimerCounter1, 154
enableInt0
core::ExternInterrupt, 70
enableInt1
core::ExternInterrupt, 70
enableInterrupts
core::MCU, 81
enableOutputCompareMatchInterrupt
core::TimerCounter, 140
core::TimerCounter0, 145
core::TimerCounter1, 154
core::TimerCounter2, 166
enableOverflowInterrupt
core::TimerCounter, 140
core::TimerCounter0, 145
core::TimerCounter1, 155
core::TimerCounter2, 166
enablePinChange
core::ExternInterrupt, 70
enablePinChangeMaskPortB
core::ExternInterrupt, 71
enablePinChangeMaskPortC
core::ExternInterrupt, 71
enablePinChangeMaskPortD
core::ExternInterrupt, 71
enableReceiveCompleteInterrupt
io::USART0, 181
enableSPI
core::MCU, 81
enableTimerCounter0
core::MCU, 82
enableTimerCounter1
core::MCU, 82
enableTimerCounter2
core::MCU, 82
enableTransferCompleteInterrupt
io::SPI, 115
enableTransmitCompleteInterrupt
io::USART0, 181
enableTWI
core::MCU, 83
enableUSART0
core::MCU, 83
ESP32_SmartEnvironmentalNode Directory Reference, 18
ESP32_WiFi_Telemetry Directory Reference, 19
evenParity
io, 46
EXT_INT_DISABLE_INT0
ha_m2560.h, 402
ha_m328p.h, 440
EXT_INT_DISABLE_INT1
ha_m2560.h, 402
ha_m328p.h, 440
EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT
ha_m2560.h, 403
ha_m328p.h, 440
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB
ha_m2560.h, 403
ha_m328p.h, 440
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC
ha_m2560.h, 403
ha_m328p.h, 440
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD
ha_m2560.h, 403
ha_m328p.h, 440
EXT_INT_ENABLE_INT0
ha_m2560.h, 403
ha_m328p.h, 441
EXT_INT_ENABLE_INT1
ha_m2560.h, 404
ha_m328p.h, 441
EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT
ha_m2560.h, 404
ha_m328p.h, 441
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB
ha_m2560.h, 404
ha_m328p.h, 441
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC
ha_m2560.h, 404

ha_m328p.h, 441
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD
ha_m2560.h, 404
ha_m328p.h, 441
EXT_INT_INT0_INTERRUPT
ha_m2560.h, 405
ha_m328p.h, 442
EXT_INT_INT1_INTERRUPT
ha_m2560.h, 405
ha_m328p.h, 442
EXT_INT_PIN_CHANGE_PORTB_INTERRUPT
ha_m2560.h, 405
ha_m328p.h, 442
EXT_INT_PIN_CHANGE_PORTC_INTERRUPT
ha_m2560.h, 405
ha_m328p.h, 442
EXT_INT_PIN_CHANGE_PORTD_INTERRUPT
ha_m2560.h, 405
ha_m328p.h, 442
EXT_INT_SET_INT0_SENSE_CONTROL
ha_m2560.h, 405
ha_m328p.h, 442
EXT_INT_SET_INT1_SENSE_CONTROL
ha_m2560.h, 406
ha_m328p.h, 443
extendedStandby
core, 41
extern_Clock_T0_Falling_Edge
core, 37
extern_Clock_T0_Rising_Edge
core, 37
ExternInterrupt
core::ExternInterrupt, 69
ExternInterrupt.cpp, 275, 276
ExternInterrupt.h, 277, 279
extInterrupt
core, 35

F0
buzzer_pitches_16bit.h, 359
F1
buzzer_pitches_16bit.h, 359
buzzer_pitches_8bit.h, 369
F2
buzzer_pitches_16bit.h, 359
buzzer_pitches_8bit.h, 369
F3
buzzer_pitches_16bit.h, 359
buzzer_pitches_8bit.h, 369
F4
buzzer_pitches_16bit.h, 360
F5
buzzer_pitches_16bit.h, 360
F6
buzzer_pitches_16bit.h, 360
F7
buzzer_pitches_16bit.h, 360
fallingEdge
core, 40
fast_PWM
core, 38
fast_PWM_10bit
core, 38
fast_PWM_8bit
core, 38
fast_PWM_9bit
core, 38
fast_PWM_ICR
core, 38
fast_PWM_OCR
core, 38
first_LSB
io, 45
first_MSB
io, 45
fiveBits
io, 46
font5x7
SSD1306_OLED.cpp, 259
frameError
io::USART0, 181
frameSize
io, 45
freeRunning
core, 35
fullStep
component, 34
Fx0
buzzer_pitches_16bit.h, 360
Fx1
buzzer_pitches_16bit.h, 360
buzzer_pitches_8bit.h, 369
Fx2
buzzer_pitches_16bit.h, 360
buzzer_pitches_8bit.h, 369
Fx3
buzzer_pitches_16bit.h, 361
buzzer_pitches_8bit.h, 369
Fx4
buzzer_pitches_16bit.h, 361
Fx5
buzzer_pitches_16bit.h, 361
Fx6
buzzer_pitches_16bit.h, 361
Fx7
buzzer_pitches_16bit.h, 361

G0
buzzer_pitches_16bit.h, 361
G1
buzzer_pitches_16bit.h, 361
buzzer_pitches_8bit.h, 370
G2
buzzer_pitches_16bit.h, 362
buzzer_pitches_8bit.h, 370
G3
buzzer_pitches_16bit.h, 362
buzzer_pitches_8bit.h, 370

G4
buzzer_pitches_16bit.h, 362
G5
buzzer_pitches_16bit.h, 362
G6
buzzer_pitches_16bit.h, 362
G7
buzzer_pitches_16bit.h, 362
getClockPrescaler
core::TimerCounter, 141
core::TimerCounter0, 146
core::TimerCounter1, 155
core::TimerCounter2, 166
getConversionResult
core::ADConverter, 56
getCounter
core::TimerCounter, 141
core::TimerCounter0, 146
core::TimerCounter1, 155
core::TimerCounter2, 167
getDirection
component::RotaryEncoder, 105
getInputCaptureRegister
core::TimerCounter1, 156
getInstance
core::ADConverter, 57
core::ExternInterrupt, 72
core::TimerCounter0, 146
core::TimerCounter1, 156
core::TimerCounter2, 167
core::TWI, 174
core::WatchdogTimer, 197
io::SPI, 115
io::USART0, 182
getLux
component::LDR, 75
getNumberBytesReceived
io::USART0, 182
getNumberBytesSent
io::USART0, 183
getOutputCompareRegister
core::TimerCounter, 141
core::TimerCounter0, 146
core::TimerCounter1, 156
core::TimerCounter2, 167
getPinNumber
io::Pin, 95
getPosition
component::RotaryEncoder, 105
getPressedCount
component::PushButton, 101
getResistance
component::LDR, 76
goalReached
component::StepperMotor, 128
goToSleep
core::MCU, 83
Gx0
buzzer_pitches_16bit.h, 362
buzzer_pitches_8bit.h, 370
Gx1
buzzer_pitches_16bit.h, 363
buzzer_pitches_8bit.h, 370
Gx2
buzzer_pitches_16bit.h, 363
buzzer_pitches_8bit.h, 370
Gx3
buzzer_pitches_16bit.h, 363
Gx4
buzzer_pitches_16bit.h, 363
Gx5
buzzer_pitches_16bit.h, 363
Gx6
buzzer_pitches_16bit.h, 363
Gx7
buzzer_pitches_16bit.h, 363

ha_attiny85.h, 371, 377
ADC_ADJUST_RESULT_LEFT, 372
ADC_ADJUST_RESULT_RIGHT, 372
ADC_CONVERSION_COMPLETE_INTERRUPT, 372
ADC_DISABLE, 372
ADC_DISABLE_AUTOTRIGGER, 373
ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT, 373
ADC_DISABLE_DIGITAL_INPUT_REGISTER, 373
ADC_ENABLE, 373
ADC_ENABLE_AUTOTRIGGER, 373
ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT, 374
ADC_SELECT_ANALOG_INPUT, 374
ADC_SELECT_CLOCK_PRESCALER, 374
ADC_SELECT_REF_VOLTAGE, 374
ADC_START_CONVERSION, 375
ADC_STOP_CONVERSION, 375
MCU_SELECT_SLEEP_MODE, 375
MCU_SLEEP_DISABLE, 375
MCU_SLEEP_ENABLE, 376
PUSHBUTTON_DEBOUNCE_TIME_US, 376
WATCHDOG_SELECT_TIMEOUT, 376
WATCHDOG_START, 376
WATCHDOG_STOP, 376
WATCHDOG_TIMEOUT_INTERRUPT, 377
ha_base.h, 378, 379
STR, 379
STRx, 379
ha_esp32.h, 380, 394
ADC_ADJUST_RESULT_LEFT, 382
ADC_ADJUST_RESULT_RIGHT, 382
ADC_CONVERSION_COMPLETE_INTERRUPT, 382
ADC_DISABLE, 382
ADC_DISABLE_AUTOTRIGGER, 382
ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT, 382

ADC_DISABLE_DIGITAL_INPUT_REGISTER, 382
ADC_ENABLE, 382
ADC_ENABLE_AUTOTRIGGER, 383
ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT, 383
ADC_SELECT_ANALOG_INPUT, 383
ADC_SELECT_AUTO_TRIGGER_SOURCE, 383
ADC_SELECT_CLOCK_PRESCALER, 383
ADC_SELECT_REF_VOLTAGE, 383
ADC_START_CONVERSION, 384
ADC_STOP_CONVERSION, 384
MCU_ADC_DISABLE, 384
MCU_ADC_ENABLE, 384
MCU_SELECT_SLEEP_MODE, 384
MCU_SLEEP_DISABLE, 384
MCU_SLEEP_ENABLE, 385
MCU_SPI_DISABLE, 385
MCU_SPI_ENABLE, 385
MCU_TIMER0_DISABLE, 385
MCU_TIMER0_ENABLE, 385
MCU_TIMER1_DISABLE, 385
MCU_TIMER1_ENABLE, 386
MCU_TIMER2_DISABLE, 386
MCU_TIMER2_ENABLE, 386
MCU_TWI_DISABLE, 386
MCU_TWI_ENABLE, 386
MCU_USART0_DISABLE, 386
MCU_USART0_ENABLE, 387
PUSHBUTTON_DEBOUNCE_TIME_US, 387
USART0_CONTROL_STATUS_REGISTER, 387
USART0_DATA_OVERRUN, 387
USART0_DATA_REGISTER, 387
USART0_DATA_REGISTER_EMPTY_INTERRUPT, 387
USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT, 388
USART0_DISABLE_DOUBLE_SPEED_MODE, 388
USART0_DISABLE_PARITY_MODE, 388
USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT, 388
USART0_DISABLE_RECEIVER, 388
USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT, 388
USART0_DISABLE_TRANSMITTER, 389
USART0_ENABLE_ASYNC_TRANSMISSION_MODE, 389
USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT, 389
USART0_ENABLE_DOUBLE_SPEED_MODE, 389
USART0_ENABLE_EVEN_PARITY_MODE, 389
USART0_ENABLE_MASTER_SPI_MODE, 389
USART0_ENABLE_ODD_PARITY_MODE, 390
USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT, 390
USART0_ENABLE_RECEIVER, 390
USART0_ENABLE_SYNC_TRANSMISSION_MODE, 390
USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT, 390
USART0_ENABLE_TRANSMITTER, 390
USART0_FRAME_ERROR, 391
USART0_PARITY_ERROR, 391
USART0_RECEIVE_COMPLETE_INTERRUPT, 391
USART0_SET_5BIT_FRAME_SIZE, 391
USART0_SET_6BIT_FRAME_SIZE, 391
USART0_SET_7BIT_FRAME_SIZE, 391
USART0_SET_8BIT_FRAME_SIZE, 392
USART0_SET_9BIT_FRAME_SIZE, 392
USART0_SET_BAUDRATE_HIGH_REGISTER, 392
USART0_SET_BAUDRATE_LOW_REGISTER, 392
USART0_SET_ONE_STOP_BIT, 392
USART0_SET_TWO_STOP_BITS, 392
USART0_TRANSMIT_COMPLETE_INTERRUPT, 393
WATCHDOG_SELECT_TIMEOUT, 393
WATCHDOG_START, 393
WATCHDOG_STOP, 393
WATCHDOG_TIMEOUT_INTERRUPT, 393
ha_m168p.h, 395, 396
ha_m2560.h, 396, 429
ADC_ADJUST_RESULT_LEFT, 399
ADC_ADJUST_RESULT_RIGHT, 399
ADC_CONVERSION_COMPLETE_INTERRUPT, 399
ADC_DISABLE, 400
ADC_DISABLE_AUTOTRIGGER, 400
ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT, 400
ADC_DISABLE_DIGITAL_INPUT_REGISTER, 400
ADC_ENABLE, 400
ADC_ENABLE_AUTOTRIGGER, 400
ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT, 401
ADC_SELECT_ANALOG_INPUT, 401
ADC_SELECT_AUTO_TRIGGER_SOURCE, 401
ADC_SELECT_CLOCK_PRESCALER, 401
ADC_SELECT_REF_VOLTAGE, 402
ADC_START_CONVERSION, 402
ADC_STOP_CONVERSION, 402
EXT_INT_DISABLE_INT0, 402
EXT_INT_DISABLE_INT1, 402
EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT, 403
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB, 403
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC, 403
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD, 403

EXT_INT_ENABLE_INT0, 403
EXT_INT_ENABLE_INT1, 404
EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT, 404
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB, 404
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC, 404
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD, 404
EXT_INT_INT0_INTERRUPT, 405
EXT_INT_INT1_INTERRUPT, 405
EXT_INT_PIN_CHANGE_PORTB_INTERRUPT, 405
EXT_INT_PIN_CHANGE_PORTC_INTERRUPT, 405
EXT_INT_PIN_CHANGE_PORTD_INTERRUPT, 405
EXT_INT_SET_INT0_SENSE_CONTROL, 405
EXT_INT_SET_INT1_SENSE_CONTROL, 406
MCU_ADC_DISABLE, 406
MCU_ADC_ENABLE, 406
MCU_SELECT_SLEEP_MODE, 406
MCU_SLEEP_DISABLE, 407
MCU_SLEEP_ENABLE, 407
MCU_SPI_DISABLE, 407
MCU_SPI_ENABLE, 407
MCU_TIMER0_DISABLE, 407
MCU_TIMER0_ENABLE, 407
MCU_TIMER1_DISABLE, 408
MCU_TIMER1_ENABLE, 408
MCU_TIMER2_DISABLE, 408
MCU_TIMER2_ENABLE, 408
MCU_TWI_DISABLE, 408
MCU_TWI_ENABLE, 408
MCU_USART0_DISABLE, 408
MCU_USART0_ENABLE, 408
PUSHBUTTON_DEBOUNCE_TIME_US, 409
SERVOMOTOR_TIMER_ANGLE_COUNT, 409
SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT, 409
SPI_CONTROL_REGISTER, 409
SPI_DATA_REGISTER, 409
SPI_DISABLE, 410
SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT, 410
SPI_ENABLE, 410
SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT, 410
SPI_MASTER_MODE, 410
SPI_SELECT_CLOCK_PRESCALER, 410
SPI_SELECT_DATA_MODE, 411
SPI_SELECT_DATA_ORDER, 411
SPI_SELECT_MASTER_MODE, 411
SPI_SELECT_SLAVE_MODE, 412
SPI_STATUS_REGISTER, 412
SPI_TRANSFER_COMPLETE, 412
SPI_TRANSFER_COMPLETE_INTERRUPT, 412
SPI_WRITE_COLLISION, 412
TIMER0_COM_CHANNEL_A_INTERRUPT, 412
TIMER0_COM_CHANNEL_B_INTERRUPT, 413
TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT, 413
TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT, 413
TIMER0_DISABLE_OVERFLOW_INTERRUPT, 413
TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT, 413
TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT, 413
TIMER0_ENABLE_OVERFLOW_INTERRUPT, 414
TIMER0_OVERFLOW_INTERRUPT, 414
TIMER0_SELECT_CLOCK_SOURCE, 414
TIMER0_SELECT_COM_CHANNEL_A, 414
TIMER0_SELECT_COM_CHANNEL_B, 414
TIMER0_SELECT_OPERATION_MODE, 415
TIMER0_STOP, 415
TIMER1_COM_CHANNEL_A_INTERRUPT, 415
TIMER1_COM_CHANNEL_B_INTERRUPT, 416
TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT, 416
TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT, 416
TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT, 416
TIMER1_DISABLE_OVERFLOW_INTERRUPT, 416
TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT, 416
TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT, 417
TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT, 417
TIMER1_ENABLE_OVERFLOW_INTERRUPT, 417
TIMER1_INPUT_CAPTURE_INTERRUPT, 417
TIMER1_OVERFLOW_INTERRUPT, 417
TIMER1_SELECT_CLOCK_SOURCE, 417
TIMER1_SELECT_COM_CHANNEL_A, 418
TIMER1_SELECT_COM_CHANNEL_B, 418
TIMER1_SELECT_OPERATION_MODE, 418
TIMER1_STOP, 419
TIMER2_COM_CHANNEL_A_INTERRUPT, 419
TIMER2_COM_CHANNEL_B_INTERRUPT, 419
TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT, 419
TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT, 419
TIMER2_DISABLE_OVERFLOW_INTERRUPT, 420
TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT, 420
TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT, 420

TIMER2_ENABLE_OVERFLOW_INTERRUPT, 420
TIMER2_OVERFLOW_INTERRUPT, 420
TIMER2_SELECT_CLOCK_SOURCE, 420
TIMER2_SELECT_COM_CHANNEL_A, 421
TIMER2_SELECT_COM_CHANNEL_B, 421
TIMER2_SELECT_OPERATION_MODE, 421
TIMER2_STOP, 422
USART0_CONTROL_STATUS_REGISTER, 422
USART0_DATA_OVERRUN, 422
USART0_DATA_REGISTER, 422
USART0_DATA_REGISTER_EMPTY_INTERRUPT, 422
USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT, 423
USART0_DISABLE_DOUBLE_SPEED_MODE, 423
USART0_DISABLE_PARITY_MODE, 423
USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT, 423
USART0_DISABLE_RECEIVER, 423
USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT, 423
USART0_DISABLE_TRANSMITTER, 423
USART0_ENABLE_ASYNC_TRANSMISSION_MODE, 424
USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT, 424
USART0_ENABLE_DOUBLE_SPEED_MODE, 424
USART0_ENABLE_EVEN_PARITY_MODE, 424
USART0_ENABLE_MASTER_SPI_MODE, 424
USART0_ENABLE_ODD_PARITY_MODE, 424
USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT, 424
USART0_ENABLE_RECEIVER, 424
USART0_ENABLE_SYNC_TRANSMISSION_MODE, 425
USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT, 425
USART0_ENABLE_TRANSMITTER, 425
USART0_FRAME_ERROR, 425
USART0_PARITY_ERROR, 425
USART0_RECEIVE_COMPLETE_INTERRUPT, 425
USART0_SET_5BIT_FRAME_SIZE, 425
USART0_SET_6BIT_FRAME_SIZE, 426
USART0_SET_7BIT_FRAME_SIZE, 426
USART0_SET_8BIT_FRAME_SIZE, 426
USART0_SET_9BIT_FRAME_SIZE, 426
USART0_SET_BAUDRATE_HIGH_REGISTER, 427
USART0_SET_BAUDRATE_LOW_REGISTER, 427
USART0_SET_ONE_STOP_BIT, 427
USART0_SET_TWO_STOP_BITS, 427
USART0_TRANSMIT_COMPLETE_INTERRUPT, 427
WATCHDOG_SELECT_TIMEOUT, 427
WATCHDOG_START, 428
WATCHDOG_STOP, 428
WATCHDOG_TIMEOUT_INTERRUPT, 428
ha_m328p.h, 433, 463
ADC_ADJUST_RESULT_LEFT, 437
ADC_ADJUST_RESULT_RIGHT, 437
ADC_CONVERSION_COMPLETE_INTERRUPT, 437
ADC_DISABLE, 437
ADC_DISABLE_AUTOTRIGGER, 437
ADC_DISABLE_CONVERSION_COMPLETE_INTERRUPT, 437
ADC_DISABLE_DIGITAL_INPUT_REGISTER, 438
ADC_ENABLE, 438
ADC_ENABLE_AUTOTRIGGER, 438
ADC_ENABLE_CONVERSION_COMPLETE_INTERRUPT, 438
ADC_SELECT_ANALOG_INPUT, 438
ADC_SELECT_AUTO_TRIGGER_SOURCE, 438
ADC_SELECT_CLOCK_PRESCALER, 439
ADC_SELECT_REF_VOLTAGE, 439
ADC_START_CONVERSION, 439
ADC_STOP_CONVERSION, 439
EXT_INT_DISABLE_INT0, 440
EXT_INT_DISABLE_INT1, 440
EXT_INT_DISABLE_PIN_CHANGE_INTERRUPT, 440
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTB, 440
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTC, 440
EXT_INT_DISABLE_PIN_CHANGE_MASK_PORTD, 440
EXT_INT_ENABLE_INT0, 441
EXT_INT_ENABLE_INT1, 441
EXT_INT_ENABLE_PIN_CHANGE_INTERRUPT, 441
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTB, 441
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTC, 441
EXT_INT_ENABLE_PIN_CHANGE_MASK_PORTD, 441
EXT_INT_INT0_INTERRUPT, 442
EXT_INT_INT1_INTERRUPT, 442
EXT_INT_PIN_CHANGE_PORTB_INTERRUPT, 442
EXT_INT_PIN_CHANGE_PORTC_INTERRUPT, 442
EXT_INT_PIN_CHANGE_PORTD_INTERRUPT, 442
EXT_INT_SET_INT0_SENSE_CONTROL, 442
EXT_INT_SET_INT1_SENSE_CONTROL, 443
MCU_ADC_DISABLE, 443
MCU_ADC_ENABLE, 443
MCU_SELECT_SLEEP_MODE, 443

MCU_SLEEP_DISABLE, 443
MCU_SLEEP_ENABLE, 444
MCU_SPI_DISABLE, 444
MCU_SPI_ENABLE, 444
MCU_TIMER0_DISABLE, 444
MCU_TIMER0_ENABLE, 444
MCU_TIMER1_DISABLE, 444
MCU_TIMER1_ENABLE, 444
MCU_TIMER2_DISABLE, 444
MCU_TIMER2_ENABLE, 445
MCU_TWI_DISABLE, 445
MCU_TWI_ENABLE, 445
MCU_USART0_DISABLE, 445
MCU_USART0_ENABLE, 445
PUSHBUTTON_DEBOUNCE_TIME_US, 445
SERVOMOTOR_TIMER_ANGLE_COUNT, 445
SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT, 446
SPI_CONTROL_REGISTER, 446
SPI_DATA_REGISTER, 446
SPI_DISABLE, 446
SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT, 446
SPI_ENABLE, 446
SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT, 447
SPI_MASTER_MODE, 447
SPI_SELECT_CLOCK_PRESCALER, 447
SPI_SELECT_DATA_MODE, 447
SPI_SELECT_DATA_ORDER, 447
SPI_SELECT_MASTER_MODE, 448
SPI_SELECT_SLAVE_MODE, 448
SPI_STATUS_REGISTER, 448
SPI_TRANSFER_COMPLETE, 448
SPI_TRANSFER_COMPLETE_INTERRUPT, 448
SPI_WRITE_COLLISION, 448
TIMER0_COM_CHANNEL_A_INTERRUPT, 449
TIMER0_COM_CHANNEL_B_INTERRUPT, 449
TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT, 449
TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT, 449
TIMER0_DISABLE_OVERFLOW_INTERRUPT, 449
TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT, 449
TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT, 449
TIMER0_ENABLE_OVERFLOW_INTERRUPT, 449
TIMER0_OVERFLOW_INTERRUPT, 450
TIMER0_SELECT_CLOCK_SOURCE, 450
TIMER0_SELECT_COM_CHANNEL_A, 450
TIMER0_SELECT_COM_CHANNEL_B, 450
TIMER0_SELECT_OPERATION_MODE, 450
TIMER0_STOP, 451
TIMER1_COM_CHANNEL_A_INTERRUPT, 451
TIMER1_COM_CHANNEL_B_INTERRUPT, 451
TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT, 451
TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT, 451
TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT, 452
TIMER1_DISABLE_OVERFLOW_INTERRUPT, 452
TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT, 452
TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT, 452
TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT, 452
TIMER1_ENABLE_OVERFLOW_INTERRUPT, 452
TIMER1_INPUT_CAPTURE_INTERRUPT, 452
TIMER1_OVERFLOW_INTERRUPT, 452
TIMER1_SELECT_CLOCK_SOURCE, 453
TIMER1_SELECT_COM_CHANNEL_A, 453
TIMER1_SELECT_COM_CHANNEL_B, 453
TIMER1_SELECT_OPERATION_MODE, 453
TIMER1_STOP, 454
TIMER2_COM_CHANNEL_A_INTERRUPT, 454
TIMER2_COM_CHANNEL_B_INTERRUPT, 454
TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT, 454
TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT, 454
TIMER2_DISABLE_OVERFLOW_INTERRUPT, 455
TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT, 455
TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT, 455
TIMER2_ENABLE_OVERFLOW_INTERRUPT, 455
TIMER2_OVERFLOW_INTERRUPT, 455
TIMER2_SELECT_CLOCK_SOURCE, 455
TIMER2_SELECT_COM_CHANNEL_A, 455
TIMER2_SELECT_COM_CHANNEL_B, 456
TIMER2_SELECT_OPERATION_MODE, 456
TIMER2_STOP, 456
USART0_CONTROL_STATUS_REGISTER, 457
USART0_DATA_OVERRUN, 457
USART0_DATA_REGISTER, 457
USART0_DATA_REGISTER_EMPTY_INTERRUPT, 457
USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT, 457
USART0_DISABLE_DOUBLE_SPEED_MODE, 457
USART0_DISABLE_PARITY_MODE, 458
USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT, 458
USART0_DISABLE_RECEIVER, 458
USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT, 458

USART0_DISABLE_TRANSMITTER, 458
USART0_ENABLE_ASYNC_TRANSMISSION_MODE, 458
USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT, 458
USART0_ENABLE_DOUBLE_SPEED_MODE, 458
USART0_ENABLE_EVEN_PARITY_MODE, 459
USART0_ENABLE_MASTER_SPI_MODE, 459
USART0_ENABLE_ODD_PARITY_MODE, 459
USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT, 459
USART0_ENABLE_RECEIVER, 459
USART0_ENABLE_SYNC_TRANSMISSION_MODE, 459
USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT, 459
USART0_ENABLE_TRANSMITTER, 460
USART0_FRAME_ERROR, 460
USART0_PARITY_ERROR, 460
USART0_RECEIVE_COMPLETE_INTERRUPT, 460
USART0_SET_5BIT_FRAME_SIZE, 460
USART0_SET_6BIT_FRAME_SIZE, 460
USART0_SET_7BIT_FRAME_SIZE, 461
USART0_SET_8BIT_FRAME_SIZE, 461
USART0_SET_9BIT_FRAME_SIZE, 461
USART0_SET_BAUDRATE_HIGH_REGISTER, 461
USART0_SET_BAUDRATE_LOW_REGISTER, 462
USART0_SET_ONE_STOP_BIT, 462
USART0_SET_TWO_STOP_BITS, 462
USART0_TRANSMIT_COMPLETE_INTERRUPT, 462
WATCHDOG_SELECT_TIMEOUT, 462
WATCHDOG_START, 462
WATCHDOG_STOP, 463
WATCHDOG_TIMEOUT_INTERRUPT, 463
halfStep
component, 34

I2C_OLED_Dashboard Directory Reference, 20
Idle
core, 40
init
component::PCF8574_LCD, 86
component::SSD1306_OLED, 122
core::MCU, 84
io::Telemetry, 137
inputCaptureServiceRoutine
core::TimerCounter1, 157
Int0ServiceRoutine
core::ExternInterrupt, 72
Int1ServiceRoutine
core::ExternInterrupt, 72
internal
core, 39
interrupt
core, 42
Interrupt_reset
core, 42
INTERRUPT, 43
ADCSample, 47
async, 47
clockPrescaler, 44
communicationMode, 44
CustomLog, 47
dataMode, 45
dataOrder, 45
disable, 46
duplex, 45
eightBits, 46
evenParity, 46
first_LSB, 45
first_MSB, 45
fiveBits, 46
frameSize, 45
master, 46
masterSPI, 47
mode_0, 45
mode_1, 45
mode_2, 45
mode_3, 45
MotorState, 47
neineBits, 46
noParity, 46
oddParity, 46
oneStopBit, 47
operationMode, 46
parityMode, 46
PinState, 47
PortB, 48
PortC, 48
PortD, 48
PS_128, 44
PS_16, 44
PS_2, 44
PS_32, 44
PS_4, 44
PS_64, 44
PS_8, 44
receive, 45
sevenBits, 46
sixBits, 46
slave, 46
stopBit, 46
submaster, 46
sync, 47
TelemetryType, 47
transmissionMode, 47
transmit, 45
twoStopBits, 47
io Directory Reference, 21
io::Pin, 94
~Pin, 95
getPinNumber, 95

isHigh, 95
isLow, 96
m_pinNumber, 98
mr_portName, 98
Pin, 95
setHigh, 96
setLow, 96
toggle, 97
toInput, 97
toOutput, 97
io::Port, 98
mp_ddrReg, 99
mp_pinReg, 99
mp_portReg, 99
io::SPI, 112
__externally_visible__, 119
__used__, 119
∼SPI, 114
enableTransferCompleteInterrupt, 115
getInstance, 115
m_data, 119
m_pinMISO, 119
m_pinMOSI, 119
m_pinSCK, 120
m_pinSS, 120
m_transferComplete, 120
masterReceiveByte, 115
masterSendByte, 115
operator=, 116
selectClockPrescaler, 116
selectDataMode, 116
selectDataOrder, 117
selectOperationMode, 117
selectSlave, 117
slaveReceiveByte, 118
SPI, 114
transferComplete, 118
transferCompleteServiceRoutine, 118
writeCollision, 118
io::Telemetry, 136
init, 137
mp_usart, 139
sendADC, 137
sendLog, 137
sendMotor, 138
sendPin, 138
io::USART0, 177
__externally_visible__, 193
__used__, 193
∼USART0, 179
dataOverrun, 180
dataRegisterEmptyServiceRoutine, 180
enableDataRegisterEmptyInterrupt, 180
enableReceiveCompleteInterrupt, 181
enableTransmitCompleteInterrupt, 181
frameError, 181
getInstance, 182
getNumberBytesReceived, 182
getNumberBytesSent, 183
m_numberBytesReceived, 194
m_numberBytesSent, 194
m_ready2Send, 194
m_sizeData2Receive, 194
m_sizeData2Send, 194
m_status, 195
m_stringReceive, 195
mp_data2Receive, 195
mp_data2Send, 195
operator=, 183
parityError, 183
ready2Send, 184
receiveChar, 184
receiveCompleteServiceRoutine, 184
receiveFrame, 185
receiveString, 185
resetNumberBytesReceived, 186
sendByte, 186
sendChar, 187
sendFrame, 187
sendLong, 188
sendString, 189
sendWord, 189
setBaudRate, 190
setCommunicationMode, 190
setFrameSize, 191
setParityMode, 191
setStopBit, 192
setTransmissionMode, 192
transmitCompleteServiceRoutine, 193
USART0, 179
isHigh
io::Pin, 95
isLow
io::Pin, 96
isOff
component::Led, 79
isOn
component::Led, 79
isPressed
component::PushButton, 101

LCD_BACKLIGHT
PCF8574_LCD.cpp, 247
LCD_ENABLE_BIT
PCF8574_LCD.cpp, 247
LCD_EnvironmentMonitor Directory Reference, 21
LCD_RS_BIT
PCF8574_LCD.cpp, 247
LDR
component::LDR, 75
LDR.cpp, 243
LDR.h, 244
LDRVD
main.cpp, 210
LED
main.cpp, 202
Led

component::Led, 78
Led.cpp, 244, 245
Led.h, 245, 246
LED0
main.cpp, 210
LED1
main.cpp, 210
LED2
main.cpp, 210
LED3
main.cpp, 211
LED4
main.cpp, 211
LED5
main.cpp, 211
LED6
main.cpp, 211
LED7
main.cpp, 211
lib Directory Reference, 21
LightMeter Directory Reference, 22
logicalChange
core, 40
lowLevel
core, 40
LUX_MAX
main.cpp, 212
LUX_MIN
main.cpp, 212

m_accelTime
component::StepperMotor, 133
m_backlightMask
component::PCF8574_LCD, 89
m_buttonPressed
component::PushButton, 103
m_clockPrescaler
core::TimerCounter0, 151
core::TimerCounter1, 163
core::TimerCounter2, 172
m_clockSource
core::TimerCounter0, 151
core::TimerCounter1, 163
core::TimerCounter2, 172
m_cols
component::PCF8574_LCD, 89
m_constSpeedTime
component::StepperMotor, 133
m_conversionResult
core::ADConverter, 61
m_currentPos
component::StepperMotor, 133
m_data
io::SPI, 119
m_decelTime
component::StepperMotor, 134
m_direction
component::RotaryEncoder, 106
m_firstRun
utils::PIDController, 93
m_goalReached
component::StepperMotor, 134
m_height
component::SSD1306_OLED, 124
m_i2cAddr
component::PCF8574_LCD, 89
component::SSD1306_OLED, 124
m_integral
utils::PIDController, 93
m_isActiveLow
component::PushButton, 103
m_isRunning
core::WatchdogTimer, 200
m_kd
utils::PIDController, 93
m_ki
utils::PIDController, 93
m_kp
utils::PIDController, 93
m_lastState
component::PushButton, 103
m_lastStateA
component::RotaryEncoder, 106
m_lastStateB
component::RotaryEncoder, 106
m_maxOutput
utils::PIDController, 93
m_minOutput
utils::PIDController, 93
m_numberBytesReceived
io::USART0, 194
m_numberBytesSent
io::USART0, 194
m_operationMode
core::WatchdogTimer, 200
m_pin
component::Buzzer, 65
component::DCMotor, 68
component::LDR, 77
component::Led, 80
component::PushButton, 103
component::ServoMotor, 111
m_pinA
component::RotaryEncoder, 106
m_pinB
component::RotaryEncoder, 106
m_pinCoil1
component::StepperMotor, 134
m_pinCoil2
component::StepperMotor, 134
m_pinCoil3
component::StepperMotor, 134
m_pinCoil4
component::StepperMotor, 135
m_pinMISO
io::SPI, 119
m_pinMOSI

io::SPI, 119
m_pinNumber
io::Pin, 98
m_pinSCK
io::SPI, 120
m_pinSS
io::SPI, 120
m_position
component::RotaryEncoder, 106
m_prevInput
utils::PIDController, 94
m_profilePosition
component::StepperMotor, 135
m_profileSpeed
component::StepperMotor, 135
m_profileTime
component::StepperMotor, 135
m_pulseCycle
component::ServoMotor, 111
m_pulseWidthMax
component::ServoMotor, 111
m_pulseWidthMid
component::ServoMotor, 112
m_pulseWidthMin
component::ServoMotor, 112
m_ready2Send
io::USART0, 194
m_resolution
core::ADConverter, 61
m_rows
component::PCF8574_LCD, 89
m_sizeData2Receive
io::USART0, 194
m_sizeData2Send
io::USART0, 194
m_speedTimeProduct
component::StepperMotor, 135
m_status
io::USART0, 195
m_stepNumber
component::StepperMotor, 136
m_stringReceive
io::USART0, 195
m_targetStep
component::StepperMotor, 136
m_timeOut
core::WatchdogTimer, 200
m_transferComplete
io::SPI, 120
m_useInternalPullUp
component::PushButton, 103
m_width
component::SSD1306_OLED, 124
main
main.cpp, 202, 204, 206, 208, 212, 216, 219, 223, 225, 226, 229, 231, 234
main.cpp, 201, 203–209, 213, 214, 217, 218, 221–230, 232, 234, 235
BUZZER, 215, 219
LDRVD, 210
LED, 202
LED0, 210
LED1, 210
LED2, 210
LED3, 211
LED4, 211
LED5, 211
LED6, 211
LED7, 211
LUX_MAX, 212
LUX_MIN, 212
main, 202, 204, 206, 208, 212, 216, 219, 223, 225, 226, 229, 231, 234
NOTE_DURATION, 219
NOTE_DURATION_MS, 215
PUSHBUTTON, 215
SONG_LENGTH, 215
TIMEDELAY, 202
map
utils, 49
master
io, 46
masterReceiveByte
io::SPI, 115
masterSendByte
io::SPI, 115
masterSPI
io, 47
MCU.cpp, 280
MCU.h, 282, 284
MCU_ADC_DISABLE
ha_esp32.h, 384
ha_m2560.h, 406
ha_m328p.h, 443
MCU_ADC_ENABLE
ha_esp32.h, 384
ha_m2560.h, 406
ha_m328p.h, 443
MCU_SELECT_SLEEP_MODE
ha_attiny85.h, 375
ha_esp32.h, 384
ha_m2560.h, 406
ha_m328p.h, 443
MCU_SLEEP_DISABLE
ha_attiny85.h, 375
ha_esp32.h, 384
ha_m2560.h, 407
ha_m328p.h, 443
MCU_SLEEP_ENABLE
ha_attiny85.h, 376
ha_esp32.h, 385
ha_m2560.h, 407
ha_m328p.h, 444
MCU_SPI_DISABLE
ha_esp32.h, 385
ha_m2560.h, 407

ha_m328p.h, 444
MCU_SPI_ENABLE
ha_esp32.h, 385
ha_m2560.h, 407
ha_m328p.h, 444
MCU_TIMER0_DISABLE
ha_esp32.h, 385
ha_m2560.h, 407
ha_m328p.h, 444
MCU_TIMER0_ENABLE
ha_esp32.h, 385
ha_m2560.h, 407
ha_m328p.h, 444
MCU_TIMER1_DISABLE
ha_esp32.h, 385
ha_m2560.h, 408
ha_m328p.h, 444
MCU_TIMER1_ENABLE
ha_esp32.h, 386
ha_m2560.h, 408
ha_m328p.h, 444
MCU_TIMER2_DISABLE
ha_esp32.h, 386
ha_m2560.h, 408
ha_m328p.h, 444
MCU_TIMER2_ENABLE
ha_esp32.h, 386
ha_m2560.h, 408
ha_m328p.h, 445
MCU_TWI_DISABLE
ha_esp32.h, 386
ha_m2560.h, 408
ha_m328p.h, 445
MCU_TWI_ENABLE
ha_esp32.h, 386
ha_m2560.h, 408
ha_m328p.h, 445
MCU_USART0_DISABLE
ha_esp32.h, 386
ha_m2560.h, 408
ha_m328p.h, 445
MCU_USART0_ENABLE
ha_esp32.h, 387
ha_m2560.h, 408
ha_m328p.h, 445
mode
component, 34
mode_0
io, 45
mode_1
io, 45
mode_2
io, 45
mode_3
io, 45
MotorState
io, 47
mp_data2Receive
io::USART0, 195
mp_data2Send
io::USART0, 195
mp_ddrReg
io::Port, 99
mp_pinReg
io::Port, 99
mp_portReg
io::Port, 99
mp_usart
io::Telemetry, 139
mr_ADC
component::LDR, 77
mr_portName
io::Pin, 98
mr_twi
component::PCF8574_LCD, 89
component::SSD1306_OLED, 124
MusicBox Directory Reference, 24

neineBits
io, 46
noClock
core, 37
noParity
io, 46
normal
core, 38
NOTE_DURATION
main.cpp, 219
NOTE_DURATION_MS
main.cpp, 215

oddParity
io, 46
off
component::DCMotor, 67
component::Led, 79
component::ServoMotor, 110
on
component::DCMotor, 67
component::Led, 80
component::ServoMotor, 110
oneStopBit
io, 47
operationMode
core, 38
io, 46
operationModeToWDTCSR
core, 42
operator delete
core::TimerCounter, 141
operator=
core::ADConverter, 58
core::ExternInterrupt, 72
core::TimerCounter0, 147
core::TimerCounter1, 157
core::TimerCounter2, 168
core::TWI, 174

core::WatchdogTimer, 197
io::SPI, 116
io::USART0, 183
Organ Directory Reference, 26
outputCompareMatchAServiceRoutine
core::TimerCounter0, 147
core::TimerCounter1, 157
core::TimerCounter2, 168
outputCompareMatchBServiceRoutine
core::TimerCounter0, 147
core::TimerCounter1, 157
core::TimerCounter2, 168
overflowServiceRoutine
core::TimerCounter0, 147
core::TimerCounter1, 158
core::TimerCounter2, 168

parityError
io::USART0, 183
parityMode
io, 46
pause
component::Buzzer, 64
PCF8574_LCD
component::PCF8574_LCD, 86
PCF8574_LCD.cpp, 246, 247
LCD_BACKLIGHT, 247
LCD_ENABLE_BIT, 247
LCD_RS_BIT, 247
PCF8574_LCD.h, 248, 249
PCINTB
core, 39
PCINTC
core, 39
PCINTD
core, 39
PID_MotorControl Directory Reference, 28
PIDController
utils::PIDController, 90
PIDController.cpp, 469
PIDController.h, 470, 471
Pin
io::Pin, 95
Pin.cpp, 331
Pin.h, 332, 333
pinChangePort
core, 39
pinChangePortBServiceRoutine
core::ExternInterrupt, 72
pinChangePortCServiceRoutine
core::ExternInterrupt, 73
pinChangePortDServiceRoutine
core::ExternInterrupt, 73
PinState
io, 47
PortB
io, 48
PortC
io, 48
PortD
io, 48
powerDown
core, 41
powerSave
core, 41
print
component::PCF8574_LCD, 87
component::SSD1306_OLED, 122
projects Directory Reference, 29
PS_1
core, 37
PS_1024
core, 37
PS_128
core, 37
io, 44
PS_16
core, 37
io, 44
PS_2
core, 37
io, 44
PS_256
core, 37
PS_32
core, 37
io, 44
PS_4
core, 37
io, 44
PS_64
core, 37
io, 44
PS_8
core, 37
io, 44
PUSHBUTTON
main.cpp, 215
PushButton
component::PushButton, 100
PushButton.cpp, 249, 250
PushButton.h, 250, 251
PUSHBUTTON_DEBOUNCE_TIME_US
ha_attiny85.h, 376
ha_esp32.h, 387
ha_m2560.h, 409
ha_m328p.h, 445
PWM_PC
core, 38
PWM_PC_10bit
core, 38
PWM_PC_8bit
core, 38
PWM_PC_9bit
core, 38
PWM_PC_ICR
core, 38

PWM_PC_OCR
core, 38
PWM_PFC_ICR
core, 38
PWM_PFC_OCR
core, 38

readAck
core::TWI, 175
README.md, 237
readNack
core::TWI, 175
ready2Send
io::USART0, 184
receive
io, 45
receiveChar
io::USART0, 184
receiveCompleteServiceRoutine
io::USART0, 184
receiveFrame
io::USART0, 185
receiveString
io::USART0, 185
referenceVoltage
core, 39
res_10bit
core, 40
res_11bit
core, 40
res_12bit
core, 40
res_13bit
core, 40
res_14bit
core, 40
res_15bit
core, 40
res_16bit
core, 40
res_8bit
core, 40
res_9bit
core, 40
reset
core, 42
core::WatchdogTimer, 198
utils::PIDController, 91
resetNumberBytesReceived
io::USART0, 186
resetPosition
component::RotaryEncoder, 105
resetPressedCount
component::PushButton, 102
resolution
core, 39
risingEdge
core, 40
RotaryEncoder
component::RotaryEncoder, 104
RotaryEncoder.cpp, 251, 252
RotaryEncoder.h, 252, 253
rotate
component::ServoMotor, 110

selectAnalogInput
core::ADConverter, 58
selectAutoTriggerSource
core::ADConverter, 58
selectClockPrescaler
core::ADConverter, 59
io::SPI, 116
selectClockSource
core::TimerCounter, 141
core::TimerCounter0, 147
core::TimerCounter1, 158
core::TimerCounter2, 168
selectCompareOutputMode
core::TimerCounter, 141
core::TimerCounter0, 148
core::TimerCounter1, 159
core::TimerCounter2, 169
selectDataMode
io::SPI, 116
selectDataOrder
io::SPI, 117
selectOperationMode
core::TimerCounter, 142
core::TimerCounter0, 149
core::TimerCounter1, 159
core::TimerCounter2, 170
io::SPI, 117
selectReferenceVoltage
core::ADConverter, 59
selectSlave
io::SPI, 117
selectSleepMode
core::MCU, 84
selectTimeOut
core::WatchdogTimer, 198
sendADC
io::Telemetry, 137
sendByte
component::PCF8574_LCD, 87
io::USART0, 186
sendChar
io::USART0, 187
sendCommand
component::SSD1306_OLED, 123
sendData
component::SSD1306_OLED, 123
sendFrame
io::USART0, 187
sendLog
io::Telemetry, 137
sendLong
io::USART0, 188
sendMotor

io::Telemetry, 138
sendNibble
component::PCF8574_LCD, 87
sendPin
io::Telemetry, 138
sendString
io::USART0, 189
sendWord
io::USART0, 189
senseControl
core, 40
ServoMotor
component::ServoMotor, 107
ServoMotor.cpp, 253, 254
ServoMotor.h, 255, 258
SERVOMOTOR_TIMER_ANGLE_COUNT
ha_m2560.h, 409
ha_m328p.h, 445
SERVOMOTOR_TIMER_PULSE_WIDTH_COUNT
ha_m2560.h, 409
ha_m328p.h, 446
set
core, 38
setBacklight
component::PCF8574_LCD, 88
setBaudRate
io::USART0, 190
setClock
core::TWI, 175
setCommunicationMode
io::USART0, 190
setCounter
core::TimerCounter, 142
core::TimerCounter0, 150
core::TimerCounter1, 161
core::TimerCounter2, 171
setCurrentPos
component::StepperMotor, 129
setCursor
component::PCF8574_LCD, 88
component::SSD1306_OLED, 123
setFrameSize
io::USART0, 191
setGains
utils::PIDController, 92
setHigh
io::Pin, 96
setInputCaptureRegister
core::TimerCounter1, 161
setInt0SenseControl
core::ExternInterrupt, 73
setInt1SenseControl
core::ExternInterrupt, 73
setLow
io::Pin, 96
setOutputCompareRegister
core::TimerCounter, 142
core::TimerCounter0, 150
core::TimerCounter1, 161
core::TimerCounter2, 171
setOutputLimits
utils::PIDController, 92
setParityMode
io::USART0, 191
setStopBit
io::USART0, 192
setTransmissionMode
io::USART0, 192
sevenBits
io, 46
sixBits
io, 46
slave
io, 46
slaveReceiveByte
io::SPI, 118
sleepEnable
core::MCU, 85
sleepMode
core, 40
SmartHomeGateway Directory Reference, 29
SONG_LENGTH
main.cpp, 215
SPI
io::SPI, 114
SPI.cpp, 333
SPI.h, 335, 337
SPI_CONTROL_REGISTER
ha_m2560.h, 409
ha_m328p.h, 446
SPI_DATA_REGISTER
ha_m2560.h, 409
ha_m328p.h, 446
SPI_DISABLE
ha_m2560.h, 410
ha_m328p.h, 446
SPI_DISABLE_TRANSFER_COMPLETE_INTERRUPT
ha_m2560.h, 410
ha_m328p.h, 446
SPI_ENABLE
ha_m2560.h, 410
ha_m328p.h, 446
SPI_ENABLE_TRANSFER_COMPLETE_INTERRUPT
ha_m2560.h, 410
ha_m328p.h, 447
SPI_MASTER_MODE
ha_m2560.h, 410
ha_m328p.h, 447
SPI_SELECT_CLOCK_PRESCALER
ha_m2560.h, 410
ha_m328p.h, 447
SPI_SELECT_DATA_MODE
ha_m2560.h, 411
ha_m328p.h, 447
SPI_SELECT_DATA_ORDER
ha_m2560.h, 411

ha_m328p.h, 447
SPI_SELECT_MASTER_MODE
ha_m2560.h, 411
ha_m328p.h, 448
SPI_SELECT_SLAVE_MODE
ha_m2560.h, 412
ha_m328p.h, 448
SPI_STATUS_REGISTER
ha_m2560.h, 412
ha_m328p.h, 448
SPI_TRANSFER_COMPLETE
ha_m2560.h, 412
ha_m328p.h, 448
SPI_TRANSFER_COMPLETE_INTERRUPT
ha_m2560.h, 412
ha_m328p.h, 448
SPI_WRITE_COLLISION
ha_m2560.h, 412
ha_m328p.h, 448
spin
component::DCMotor, 67
SSD1306_OLED
component::SSD1306_OLED, 121
SSD1306_OLED.cpp, 259, 260
font5x7, 259
SSD1306_OLED.h, 262
standby
core, 41
start
core::ADConverter, 60
core::TimerCounter, 142
core::TimerCounter0, 150
core::TimerCounter1, 162
core::TimerCounter2, 171
core::TWI, 176
core::WatchdogTimer, 198, 199
step
component::StepperMotor, 129, 130
stepDelay
component::StepperMotor, 132
stepMode
component::StepperMotor, 136
StepperMotor
component::StepperMotor, 126
StepperMotor.cpp, 263
StepperMotor.h, 268
stepPulse
component::StepperMotor, 132
stop
component::DCMotor, 67
core::ADConverter, 60
core::TimerCounter, 142
core::TimerCounter0, 151
core::TimerCounter1, 162
core::TimerCounter2, 172
core::TWI, 176
core::WatchdogTimer, 199
stopBit
io, 46
STR
ha_base.h, 379
STRx
ha_base.h, 379
submaster
io, 46
sync
io, 47

Telemetry.cpp, 338
Telemetry.h, 339, 340
TelemetryStreamer Directory Reference, 30
TelemetryType
io, 47
tests Directory Reference, 30
TIMEDELAY
main.cpp, 202
timeOut
core, 41
timeOutServiceRoutine
core::WatchdogTimer, 199
timeOutToWDTCSR
core, 42
TIMER0_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 412
ha_m328p.h, 449
TIMER0_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 413
ha_m328p.h, 449
TIMER0_DISABLE_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 413
ha_m328p.h, 449
TIMER0_DISABLE_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 413
ha_m328p.h, 449
TIMER0_DISABLE_OVERFLOW_INTERRUPT
ha_m2560.h, 413
ha_m328p.h, 449
TIMER0_ENABLE_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 413
ha_m328p.h, 449
TIMER0_ENABLE_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 413
ha_m328p.h, 449
TIMER0_ENABLE_OVERFLOW_INTERRUPT
ha_m2560.h, 414
ha_m328p.h, 449
TIMER0_OVERFLOW_INTERRUPT
ha_m2560.h, 414
ha_m328p.h, 450
TIMER0_SELECT_CLOCK_SOURCE
ha_m2560.h, 414
ha_m328p.h, 450
TIMER0_SELECT_COM_CHANNEL_A
ha_m2560.h, 414
ha_m328p.h, 450
TIMER0_SELECT_COM_CHANNEL_B
ha_m2560.h, 414

ha_m328p.h, 450
TIMER0_SELECT_OPERATION_MODE
ha_m2560.h, 415
ha_m328p.h, 450
TIMER0_STOP
ha_m2560.h, 415
ha_m328p.h, 451
timer0CompareA
core, 35
timer0Overflow
core, 35
TIMER1_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 415
ha_m328p.h, 451
TIMER1_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 416
ha_m328p.h, 451
TIMER1_DISABLE_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 416
ha_m328p.h, 451
TIMER1_DISABLE_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 416
ha_m328p.h, 451
TIMER1_DISABLE_INPUT_CAPTURE_INTERRUPT
ha_m2560.h, 416
ha_m328p.h, 452
TIMER1_DISABLE_OVERFLOW_INTERRUPT
ha_m2560.h, 416
ha_m328p.h, 452
TIMER1_ENABLE_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 416
ha_m328p.h, 452
TIMER1_ENABLE_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 417
ha_m328p.h, 452
TIMER1_ENABLE_INPUT_CAPTURE_INTERRUPT
ha_m2560.h, 417
ha_m328p.h, 452
TIMER1_ENABLE_OVERFLOW_INTERRUPT
ha_m2560.h, 417
ha_m328p.h, 452
TIMER1_INPUT_CAPTURE_INTERRUPT
ha_m2560.h, 417
ha_m328p.h, 452
TIMER1_OVERFLOW_INTERRUPT
ha_m2560.h, 417
ha_m328p.h, 452
TIMER1_SELECT_CLOCK_SOURCE
ha_m2560.h, 417
ha_m328p.h, 453
TIMER1_SELECT_COM_CHANNEL_A
ha_m2560.h, 418
ha_m328p.h, 453
TIMER1_SELECT_COM_CHANNEL_B
ha_m2560.h, 418
ha_m328p.h, 453
TIMER1_SELECT_OPERATION_MODE
ha_m2560.h, 418
ha_m328p.h, 453
TIMER1_STOP
ha_m2560.h, 419
ha_m328p.h, 454
timer1Capture
core, 36
timer1CompareB
core, 35
timer1Overflow
core, 35
TIMER2_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 419
ha_m328p.h, 454
TIMER2_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 419
ha_m328p.h, 454
TIMER2_DISABLE_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 419
ha_m328p.h, 454
TIMER2_DISABLE_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 419
ha_m328p.h, 454
TIMER2_DISABLE_OVERFLOW_INTERRUPT
ha_m2560.h, 420
ha_m328p.h, 455
TIMER2_ENABLE_COM_CHANNEL_A_INTERRUPT
ha_m2560.h, 420
ha_m328p.h, 455
TIMER2_ENABLE_COM_CHANNEL_B_INTERRUPT
ha_m2560.h, 420
ha_m328p.h, 455
TIMER2_ENABLE_OVERFLOW_INTERRUPT
ha_m2560.h, 420
ha_m328p.h, 455
TIMER2_OVERFLOW_INTERRUPT
ha_m2560.h, 420
ha_m328p.h, 455
TIMER2_SELECT_CLOCK_SOURCE
ha_m2560.h, 420
ha_m328p.h, 455
TIMER2_SELECT_COM_CHANNEL_A
ha_m2560.h, 421
ha_m328p.h, 455
TIMER2_SELECT_COM_CHANNEL_B
ha_m2560.h, 421
ha_m328p.h, 456
TIMER2_SELECT_OPERATION_MODE
ha_m2560.h, 421
ha_m328p.h, 456
TIMER2_STOP
ha_m2560.h, 422
ha_m328p.h, 456
TimerCounter.h, 285, 293
TimerCounter0
core::TimerCounter0, 144
TimerCounter0.cpp, 300
TimerCounter0.h, 303, 304
TimerCounter1

core::TimerCounter1, 153, 154
TimerCounter1.cpp, 307
TimerCounter1.h, 312
TimerCounter2
core::TimerCounter2, 165
TimerCounter2.cpp, 316
TimerCounter2.h, 320
to_125ms
core, 41
to_16ms
core, 41
to_1s
core, 41
to_250ms
core, 41
to_2s
core, 41
to_32ms
core, 41
to_4s
core, 41
to_500ms
core, 41
to_64ms
core, 41
to_8s
core, 41
toggle
component::DCMotor, 68
component::Led, 80
component::ServoMotor, 111
core, 38
io::Pin, 97
toInput
io::Pin, 97
toOutput
io::Pin, 97
transferComplete
io::SPI, 118
transferCompleteServiceRoutine
io::SPI, 118
transmissionMode
io, 47
transmit
io, 45
transmitCompleteServiceRoutine
io::USART0, 193
TWI
core::TWI, 174
TWI.cpp, 324, 325
TWI_TIMEOUT_CYCLES, 325
TWI.h, 326, 327
TWI_TIMEOUT_CYCLES
TWI.cpp, 325
twoStopBits
io, 47

update
component::RotaryEncoder, 105
USART0
io::USART0, 179
USART0.cpp, 341
USART0.h, 346
USART0_CONTROL_STATUS_REGISTER
ha_esp32.h, 387
ha_m2560.h, 422
ha_m328p.h, 457
USART0_DATA_OVERRUN
ha_esp32.h, 387
ha_m2560.h, 422
ha_m328p.h, 457
USART0_DATA_REGISTER
ha_esp32.h, 387
ha_m2560.h, 422
ha_m328p.h, 457
USART0_DATA_REGISTER_EMPTY_INTERRUPT
ha_esp32.h, 387
ha_m2560.h, 422
ha_m328p.h, 457
USART0_DISABLE_DATA_REGISTER_EMPTY_INTERRUPT
ha_esp32.h, 388
ha_m2560.h, 423
ha_m328p.h, 457
USART0_DISABLE_DOUBLE_SPEED_MODE
ha_esp32.h, 388
ha_m2560.h, 423
ha_m328p.h, 457
USART0_DISABLE_PARITY_MODE
ha_esp32.h, 388
ha_m2560.h, 423
ha_m328p.h, 458
USART0_DISABLE_RECEIVE_COMPLETE_INTERRUPT
ha_esp32.h, 388
ha_m2560.h, 423
ha_m328p.h, 458
USART0_DISABLE_RECEIVER
ha_esp32.h, 388
ha_m2560.h, 423
ha_m328p.h, 458
USART0_DISABLE_TRANSMIT_COMPLETE_INTERRUPT
ha_esp32.h, 388
ha_m2560.h, 423
ha_m328p.h, 458
USART0_DISABLE_TRANSMITTER
ha_esp32.h, 389
ha_m2560.h, 423
ha_m328p.h, 458
USART0_ENABLE_ASYNC_TRANSMISSION_MODE
ha_esp32.h, 389
ha_m2560.h, 424
ha_m328p.h, 458
USART0_ENABLE_DATA_REGISTER_EMPTY_INTERRUPT
ha_esp32.h, 389
ha_m2560.h, 424
ha_m328p.h, 458
USART0_ENABLE_DOUBLE_SPEED_MODE
ha_esp32.h, 389

ha_m2560.h, 424
ha_m328p.h, 458
USART0_ENABLE_EVEN_PARITY_MODE
ha_esp32.h, 389
ha_m2560.h, 424
ha_m328p.h, 459
USART0_ENABLE_MASTER_SPI_MODE
ha_esp32.h, 389
ha_m2560.h, 424
ha_m328p.h, 459
USART0_ENABLE_ODD_PARITY_MODE
ha_esp32.h, 390
ha_m2560.h, 424
ha_m328p.h, 459
USART0_ENABLE_RECEIVE_COMPLETE_INTERRUPT
ha_esp32.h, 390
ha_m2560.h, 424
ha_m328p.h, 459
USART0_ENABLE_RECEIVER
ha_esp32.h, 390
ha_m2560.h, 424
ha_m328p.h, 459
USART0_ENABLE_SYNC_TRANSMISSION_MODE
ha_esp32.h, 390
ha_m2560.h, 425
ha_m328p.h, 459
USART0_ENABLE_TRANSMIT_COMPLETE_INTERRUPT
ha_esp32.h, 390
ha_m2560.h, 425
ha_m328p.h, 459
USART0_ENABLE_TRANSMITTER
ha_esp32.h, 390
ha_m2560.h, 425
ha_m328p.h, 460
USART0_FRAME_ERROR
ha_esp32.h, 391
ha_m2560.h, 425
ha_m328p.h, 460
USART0_PARITY_ERROR
ha_esp32.h, 391
ha_m2560.h, 425
ha_m328p.h, 460
USART0_RECEIVE_COMPLETE_INTERRUPT
ha_esp32.h, 391
ha_m2560.h, 425
ha_m328p.h, 460
USART0_SET_5BIT_FRAME_SIZE
ha_esp32.h, 391
ha_m2560.h, 425
ha_m328p.h, 460
USART0_SET_6BIT_FRAME_SIZE
ha_esp32.h, 391
ha_m2560.h, 426
ha_m328p.h, 460
USART0_SET_7BIT_FRAME_SIZE
ha_esp32.h, 391
ha_m2560.h, 426
ha_m328p.h, 461
USART0_SET_8BIT_FRAME_SIZE
ha_esp32.h, 392
ha_m2560.h, 426
ha_m328p.h, 461
USART0_SET_9BIT_FRAME_SIZE
ha_esp32.h, 392
ha_m2560.h, 426
ha_m328p.h, 461
USART0_SET_BAUDRATE_HIGH_REGISTER
ha_esp32.h, 392
ha_m2560.h, 427
ha_m328p.h, 461
USART0_SET_BAUDRATE_LOW_REGISTER
ha_esp32.h, 392
ha_m2560.h, 427
ha_m328p.h, 462
USART0_SET_ONE_STOP_BIT
ha_esp32.h, 392
ha_m2560.h, 427
ha_m328p.h, 462
USART0_SET_TWO_STOP_BITS
ha_esp32.h, 392
ha_m2560.h, 427
ha_m328p.h, 462
USART0_TRANSMIT_COMPLETE_INTERRUPT
ha_esp32.h, 393
ha_m2560.h, 427
ha_m328p.h, 462
utils, 48
map, 49
utils Directory Reference, 30
utils::PIDController, 89
compute, 91
m_firstRun, 93
m_integral, 93
m_kd, 93
m_ki, 93
m_kp, 93
m_maxOutput, 93
m_minOutput, 93
m_prevInput, 94
PIDController, 90
reset, 91
setGains, 92
setOutputLimits, 92
utils_m328p.cpp, 471, 472
utils_m328p.h, 472

WATCHDOG_SELECT_TIMEOUT
ha_attiny85.h, 376
ha_esp32.h, 393
ha_m2560.h, 427
ha_m328p.h, 462
WATCHDOG_START
ha_attiny85.h, 376
ha_esp32.h, 393
ha_m2560.h, 428
ha_m328p.h, 462
WATCHDOG_STOP

ha_attiny85.h, 376
ha_esp32.h, 393
ha_m2560.h, 428
ha_m328p.h, 463
WATCHDOG_TIMEOUT_INTERRUPT
ha_attiny85.h, 377
ha_esp32.h, 393
ha_m2560.h, 428
ha_m328p.h, 463
watchdogOperationMode
core, 41
WatchdogTimer
core::WatchdogTimer, 197
WatchdogTimer.cpp, 327
WatchdogTimer.h, 328, 330
weak
core::ExternInterrupt, 74
core::TimerCounter0, 152
core::TimerCounter1, 163
core::TimerCounter2, 173
core::WatchdogTimer, 200
WeakIsr Directory Reference, 31
write
core::TWI, 176
writeCollision
io::SPI, 118